

# OpenShift Virtualization cookbook

Version 1, 2026-09-01

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

# cookbook OpenShift Virtualization

# OpenShift

(□□□□□:)

□ □ □ □

`cookbook`

- **목표:** OpenShift Virtualization Operator 배포/관리 OpenShift 4.18 이상
- **도구:** `oc`/`kubectl`/`virtctl` (필요시: `kustomize`/`helm`/`ansible`)
- **언어:** YAML 기반 CLI 인터페이스 (기본적으로 Kubernetes/OpenShift 클러스터에서 실행)

[illegible]

# cookbook □□□□□□

□□□□□□□□□□□□□□ 1 □□□□□□□□□□□□□□

- [Network](#) - ネットワーク設定
- [Storage](#) - ストレージ設定
- [Security](#) - セキュリティ設定 (IP 設定)
- [VM](#) - VM の DNS 設定
- [VM](#) - CPU NUMA Hugepages VM
- [VM](#) - VM のメモリ設定
- [Network](#) - ネットワーク設定
- [API](#) - API の設定

|  |  |  |  |  |  |
|--|--|--|--|--|--|
|  |  |  |  |  |  |
|--|--|--|--|--|--|

- OpenShift Virtualization
  - [Overview](#)
  - [Architecture](#)
  - [Getting started](#) (pdf)
  - [Getting started](#)

- □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □

[Open Source](#) | Issue | [ocp-virt-cookbook@redhat.com](mailto:ocp-virt-cookbook@redhat.com)

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|--|--|--|--|--|

📖 [cookbook](#) 📖 [OpenShift Virtualization Onboarding Kit](#) 📖 [Alexandre Menezes](#) [Andrea Perotti](#) [Daphne Hoover](#) [Josh Manning](#) [Melvin Hillsman](#) [Roshan Warriier](#) [Yash Devan Oza](#)

[illegible]

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|--|--|--|--|--|

MIT

[illegible]

# 목차

## 01

OpenShift Virtualization 소개  
OpenShift Virtualization 소개

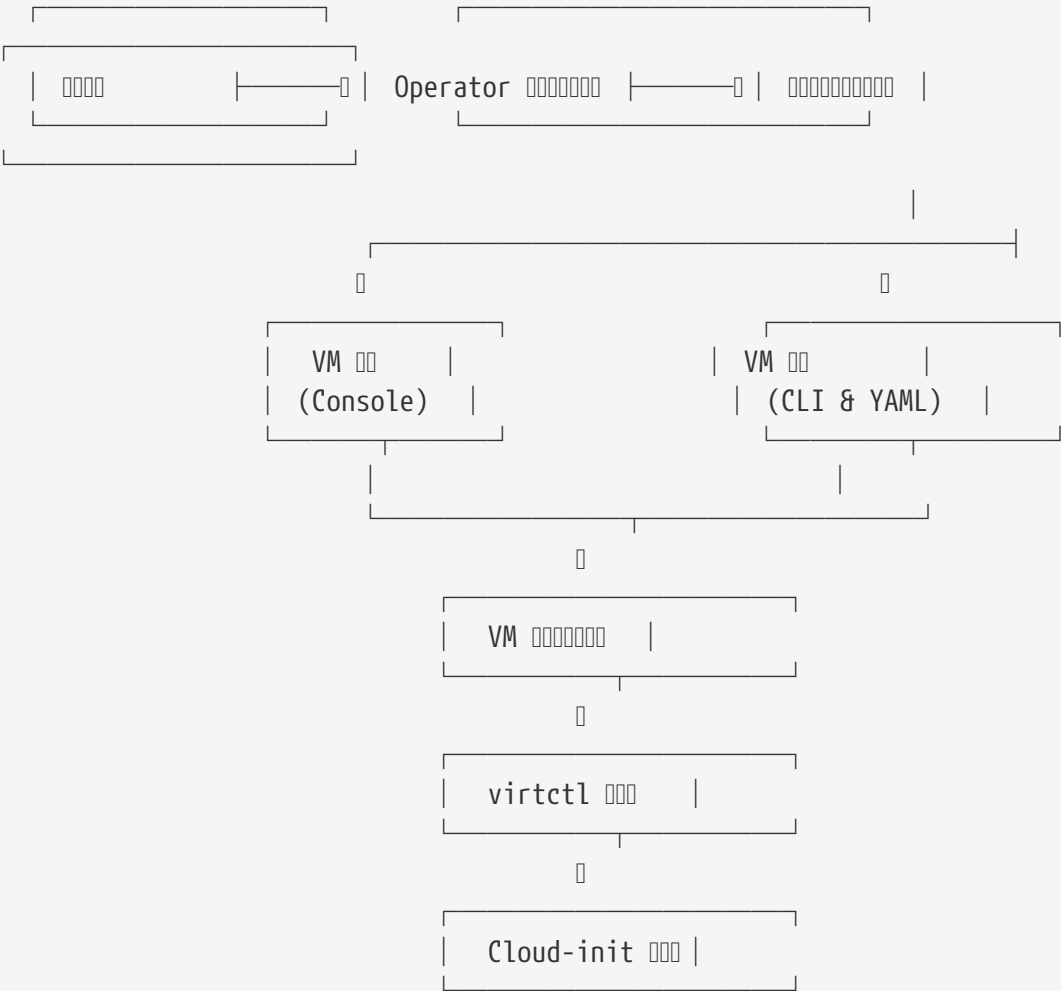
## 02

OpenShift Virtualization VM 소개

### FAST TRACK

OpenShift Virtualization – 5 VM 소개

### FULL LEARNING PATH



Operator 소개



# 

## VMware Hyper-V VM

OpenShift Virtualization
 Web
 VM
 Web
 VM
 OpenShift
 VM

## VM

VM
 Virtctl
 VM

## 

OpenShift
 Virtualization
 VM
 Networking
 Storage

## 

OpenShift Virtualization
 Operator
 Web
 VM
 VM

# 

- OpenShift Virtualization
- virtctl
- 
- cloud-init VM

# 

- OpenShift Virtualization Operator OpenShift 4.18+
- RBAC
- CLI: oc, kubectl, virtctl
- Kubernetes/OpenShift

# 

OpenShift Virtualization

## OpenShift Virtualization

OpenShift Virtualization Operator

## Virtctl 简介

virtctl 是 OpenShift 4.18+ 版本中引入的虚拟机管理工具。

## 部署 virtctl

virtctl 部署需要 Storage Class 支持。

## Web 控制台部署 VM 简介

OpenShift Web 控制台部署 VM 简介。

## VM 部署简介

部署 VM 需要 VM 镜像和存储。

## cloud-init 简介

cloud-init 是 OpenShift 4.18+ 版本中引入的虚拟机初始化工具。

## 部署 virtctl

部署 virtctl 需要 Storage Class 支持。

- [virtctl 部署](#) - virtctl 部署指南
- [virtctl 部署](#) - VM 部署指南
- [cloud-init 部署](#) - cloud-init 部署指南

## 部署 virtctl

部署

OpenShift Virtualization 部署指南。5 部署指南。

部署 virtctl 需要 Storage Class 支持。

部署

- 3 部署指南
- 部署 YAML 部署指南 Fedora VM 部署
- VM 部署指南
- 部署指南

## 部署

OpenShift Virtualization 部署指南。OpenShift 4.18+ 版本 VM 部署指南。virtctl CLI 部署指南 2 部署指南。

部署指南:

```
oc get nodes
```

OpenShift Virtualization Operator 確認

```
oc get csv -n openshift-cnv | grep kubevirt-hyperconverged
```

確認 確認

確認 **VM** 確認

確認 Fedora VM 確認

**fedora-quickstart.yaml** 確認

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-quickstart
  namespace: default
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/vm: fedora-quickstart
    spec:
      domain:
        cpu:
          cores: 1
        memory:
          guest: 2Gi
        devices:
          disks:
            - name: containerdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          volumes:
            - name: containerdisk
              containerDisk:
                image: quay.io/containerdisks/fedora:latest
            - name: cloudinitdisk
              cloudInitNoCloud:
                userData: |
                  #cloud-config
```

```
password: fedora
chpasswd: { expire: False }
ssh_pwauth: True

EOF
```

## VM 確認

VM 確認コマンド

```
oc get vm fedora-quickstart
```

確認結果 **Running** 状態

| NAME              | AGE | STATUS  | READY |
|-------------------|-----|---------|-------|
| fedora-quickstart | 30s | Running | True  |

VirtualMachineInstance 確認コマンド

```
oc get vmi fedora-quickstart
```

確認結果 **Running** 状態

| NAME              | AGE | PHASE   | IP         | NODENAME                    | READY |
|-------------------|-----|---------|------------|-----------------------------|-------|
| fedora-quickstart | 45s | Running | 10.244.2.5 | ip-10-0-22-218.ec2.internal | True  |

## VM 接続

VM 接続コマンド (virtctl 使用)

```
virtctl console fedora-quickstart
```

接続結果 **fedora** プロンプト **exit** で終了



virtctl 接続コマンド: [Virtctl](#) 接続

確認コマンド

VM 確認:

- Pod 確認: `oc describe vm fedora-quickstart`
- ディスク: `oc get pvc` (containerDisk 型 PVC 確認)
- ログ: `oc logs -l kubevirt.io/vm=fedora-quickstart`

確認:

- `StorageClass` 名: `storageclass` 確認
- `oc describe nodes` 確認
- `oc get pods -o wide` Pod 確認

確認

VM 削除:

```
oc delete vm fedora-quickstart
```

確認:

```
oc get vm fedora-quickstart
```

結果:

```
No resources found in default namespace.
```

完了

確認:

- OpenShift Virtualization 確認
- `oc apply` Fedora VM 確認
- VM 確認
- 確認

完了

確認

VM 確認

- `CLI` VM 確認 - `CLI` VM 確認
- `cloud-init` 確認 - VM 確認
- `VM` 確認 - VM 確認
- 確認 - 確認
- 確認 - VM 確認

□ □ □ □

- **VMware** - 仮想マシンソフトウェア
- **Virtctl** **Pod** - **Pod** **virtctl** **Pod**
- **OpenStack** - 仮想マシンソフトウェア
- **OpenShift Virtualization** **Pod**

□ □ □ □

11

# cookbook OpenShift Virtualization

□ □ □ □ □ □ □

# OpenShift

- [illegible]

□□□□□□□□□□

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- **Cluster Admin:** Operator 管理整个 Kubernetes 集群
- **Namespace Admin:** 管理 namespace 内 VM 资源
- **Developer:** 管理 RBAC 权限，管理 VM 资源

□□□ **CLI** □□□

## OpenShift CLI (oc)

OpenShift

## ○○○○○○○: OpenShift CLI ○○○○○○

□□□□□□□□:

oc version

## Kubernetes CLI (kubectl)

oc `oc` `kubectl` `kubectl`

## □□□□□□: Kubernetes □□□□□□

□□□□□□□□:

```
kubectl version
```

## Virtctl

**virtctl** は KubeVirt/OpenShift Virtualization を操作するための CLI ツールです。

インストール: [OpenShift Virtualization](#) を参照してください。

ヘルプ:

```
virtctl version
```

インストール

インストール方法

- **kustomize**: Kubernetes をインストール
- **helm**: Helm をインストール
- **ansible**: Ansible をインストール

インストール

インストール

インストール方法

- デフォルト 1 つの StorageClass をインストール
- VM をインストール
- VM をインストール

StorageClass をインストール:

```
oc get storageclass
```

インストール

インストール方法

- VM をインストール
- Pod をインストール
- VM をインストール

[illegible]

□ □ □ □ □ □

□□□□□□□□□□□□□□□□

- 11

- OpenShift Virtualization
- KubeVirt



11

|                       |                |                              |
|-----------------------|----------------|------------------------------|
| OpenShift             | Virtualization | OpenShift                    |
| Operator              | KubeVirt       | (Custom Resource Definition) |
| PersistentVolumeClaim | Pod            | Kubernetes                   |
| Operator              | Kubernetes     | VM                           |
| Virtualization        |                | OpenShift                    |

□□□□

- VM 管理
- HyperConverged CR Operator 与 Operator 包
- VirtualMachine CR 与 QEMU/KVM 管理
- 虚拟机生命周期管理 VM 生命周期
- VM 与 Pod 管理
- OpenShift Virtualization 与 VMware vSphere、Microsoft Hyper-V 集成

0000:





## Operator

OpenShift Virtualization namespace **openshift-cnv** namespace **HyperConverged**  
CR 4 Operator

**KubeVirt Operator** VM API (**virt-api**) VM (**virt-controller**) VM (**virt-handler**)

**CDI Operator** (Containerized Data Importer) HTTP S3 PersistentVolumeClaim (PVC) VM

**SSP Operator** (SchedulingScalePerformance) VM

**Network Addons Operator** VM Pod Multus CNI

| Namespace                                 |                        |
|-------------------------------------------|------------------------|
| <b>openshift-cnv</b>                      | Operator Control Plane |
| <b>openshift-virtualization-os-images</b> | VM                     |
| namespace                                 | VM                     |



Pod

## VM

1. **VirtualMachine** CR VM spec
2. **virt-controller** CR **VirtualMachineInstance** (VMI)
3. VMI **virt-launcher** Pod Kubernetes Pod Worker Node
4. **virt-handler**
5. **virt-launcher** Pod **libvirt** QEMU/KVM
6. PVC





VMware vSphere **DataSource** **DataVolume**  
 CDI **PVC** **vmdk**

**Storage**

## VM

VM OVN-Kubernetes Pod IP masquerade  
 VM Kubernetes  
 Service Route

L2 VLAN VM  
**NetworkAttachmentDefinition** CR Multus CNI  
 VM Pod User Defined Network (UDN) Linux  
 localnet Layer 2 overlay SR-IOV UDN OpenShift  
 Red Hat

NMState Operator bridge VLAN bond Kubernetes CR



VMware vSphere Pod VMware Workstation NAT  
 Multus vNIC

**Networking**

## 

|      | OpenShift Virtualization     | VMware vSphere                  | Microsoft Hyper-V                         |
|------|------------------------------|---------------------------------|-------------------------------------------|
| VM   | YAML (VirtualMachine CR)     | .vmx + vSphere                  | XML + PowerShell / SCVMM                  |
|      | QEMU/KVM (virt-launcher Pod) | ESXi                            | Hyper-V (Type 1 Windows Server)           |
|      | oc / kubectl / virtctl / Web | vCenter Server / vSphere Client | SCVMM / Windows Admin Center / PowerShell |
|      | CSI PVC                      | VMFS / vSAN / NFS               | CSV / SMB /                               |
|      | OVN-Kubernetes + Multus      | vSwitch / dvSwitch / NSX        | Hyper-V / SDN                             |
| HA / | Pod + (RWX)                  | vMotion / vSphere HA / DRS      | Live Migration /                          |
| API  | (Kubernetes API GitOps)      | (PowerCLI REST API)             | (PowerShell / DSC)                        |

| 項目   | OpenShift Virtualization | VMware vSphere          | Microsoft Hyper-V |
|------|--------------------------|-------------------------|-------------------|
| 管理方法 | 管理 UI — VM 管理コンソール       | 管理コンソール (vSphere Tanzu) | 管理コンソール (AKS)     |



Red Hat Virtualization (RHV) RHEV) oVirt 管理コンソール KVM 管理コンソール OpenShift Virtualization Red Hat 管理コンソール KVM 管理コンソール

OpenShift Virtualization 管理コンソール (imperative) 管理 UI 管理 API 管理 VM OpenShift Virtualization 管理 (declarative) YAML 管理 VM 管理 (reconcile) VM GitOps Helm CI/CD

1 管理 VM 管理 VM OpenShift Virtualization

管理

管理

- OpenShift Virtualization Kubernetes 管理 VM 管理
- HyperConverged CR 4 Operator Operator
- VirtualMachine CR virt-controller virt-launcher QEMU/KVM 管理
- 管理 PVC 管理
- VM Pod Multus 管理
- OpenShift Virtualization VMware vSphere Microsoft Hyper-V 管理

管理

- OpenShift Virtualization Operator 管理
- VM - VM 管理
- API - API 管理
- Storage - 管理
- Networking - 管理
- KubeVirt 管理
- OpenShift Virtualization 管理

## OpenShift Virtualization 管理

管理

管理 OpenShift OpenShift Virtualization Operator

OpenShift Virtualization KubeVirt

- [Web console](#)
- [CLI](#)

## OpenShift Virtualization

OpenShift Virtualization

OpenShift Virtualization

OpenShift Virtualization

### VMware

Migration Toolkit for Virtualization (MTV) VMware vSphere VM OpenShift Virtualization

OpenShift Virtualization

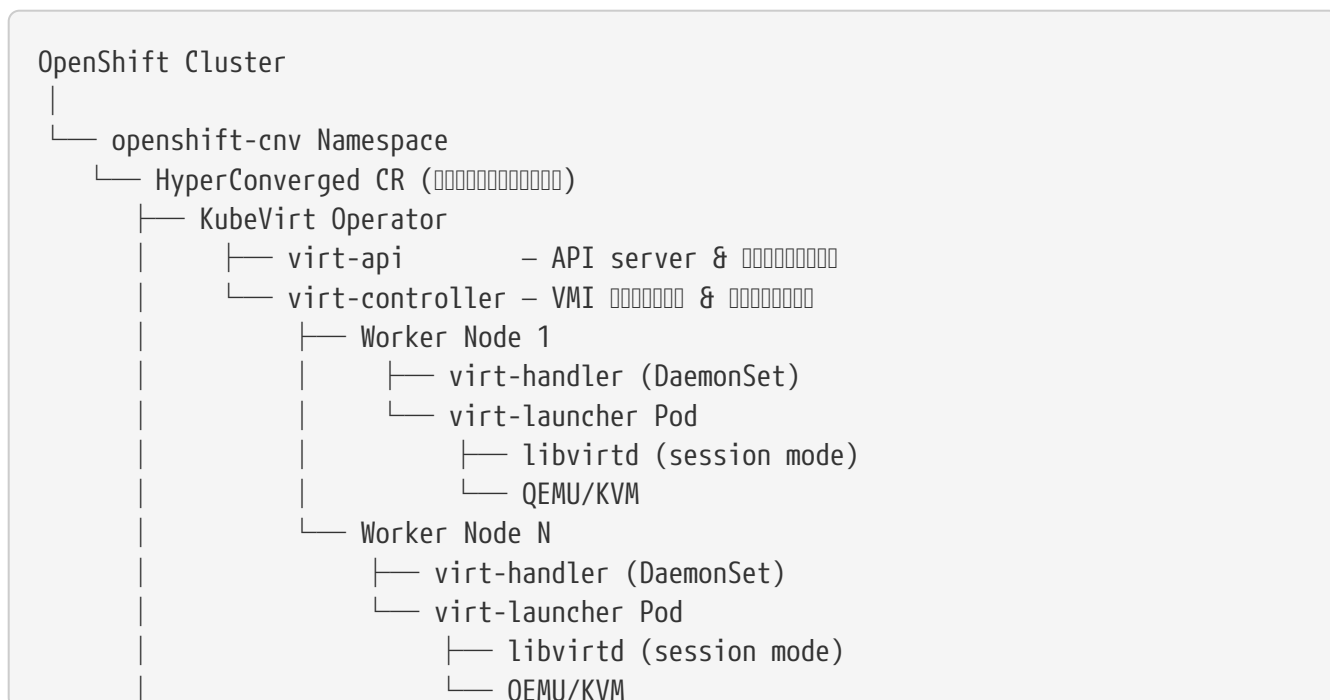
OpenShift Virtualization

OpenShift

OpenShift Virtualization

OpenShift Virtualization

OpenShift Virtualization Operator



- |— CDI Operator — VM 0000 00000/000000/0000
- |— SSP Operator — 000000 & 000000
- |— Network Addons Op — Multus CNI 000000000000

## Operator (openshift-cnv namespace)

HyperConverged (CR) 1 HyperConverged CR 4  
Operator

|                         |                                                                              |
|-------------------------|------------------------------------------------------------------------------|
| Operator                |                                                                              |
| KubeVirt Operator       | <code>virt-api</code> <code>virt-controller</code> <code>virt-handler</code> |
| CDI Operator            | Containerized Data ImporterVM<br>PersistentVolumeClaim                       |
| SSP Operator            | VM                                                                           |
| Network Addons Operator | Multus CNI                                                                   |

□ □ □ □ □ □ □ □ □ □

## KubeVirt Operator 部署 Pod 资源需求

- **virt-api** — VM API 与 KVM 交互/VNC 连接 Kubernetes API 代理
- **virt-controller** — VirtualMachineInstance (VMI) 控制器管理 VM 生命周期 Pod 管理 VM 资源分配和调度

□ □ □ □ □ □ □ □ □ □ □ □ □ □

**virt-handler** VM を管理する DaemonSet をインストールする。

## VM 0000000000

■■■■■■■■■■ virt-launcher Pod ■■■■■■■■■■ Pod ■■■■■■■■■■

- 虚拟机管理程序 **libvirt** (root 管理 VM 进程)
- 虚拟机管理程序 **QEMU/KVM**

VM

[illegible]

4444

- **OpenShift 4.18+:** `oc adm upgrade`
- **OpenShift 4.17:** `Operator`

□□□□□□□□:

OpenShift 4.18, 4.19, 4.20, 4.21

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

# OpenShift

## Virtualization

[illegible]

CPU

OpenShift

## Virtualization



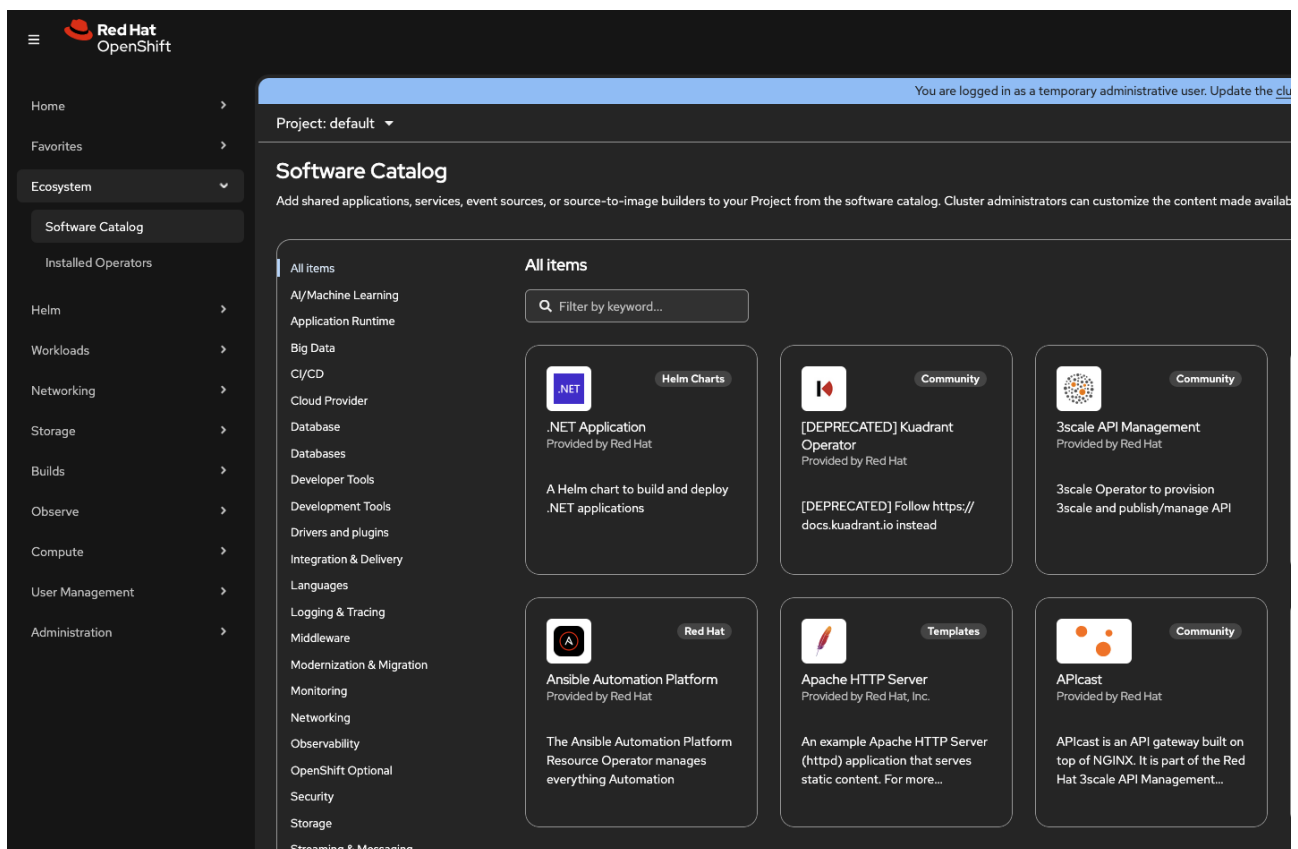
Operator [Operator](#)

**Web** □□□□□□□□□□□□□□□□

# Web OpenShift Virtualization

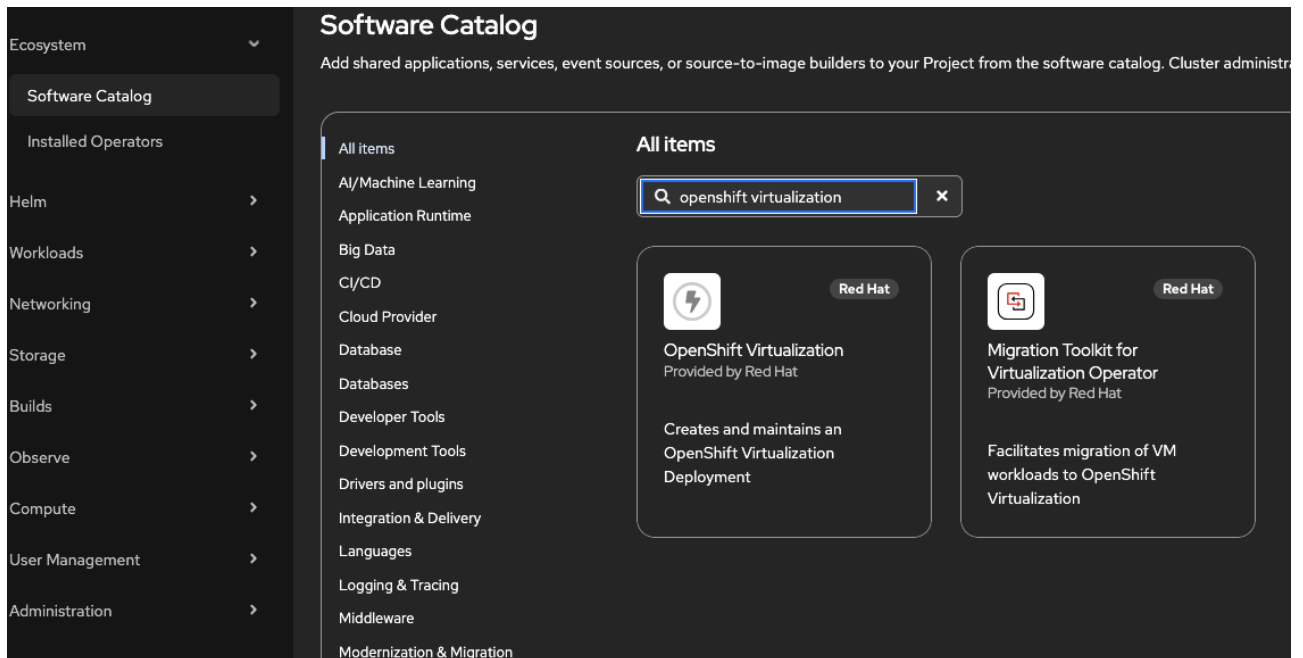
## □□□□ 1: OperatorHub □□□□□□

1. **cluster-admin** 管理 OpenShift Web 控制台
2. **Operators** → **OperatorHub** 安装



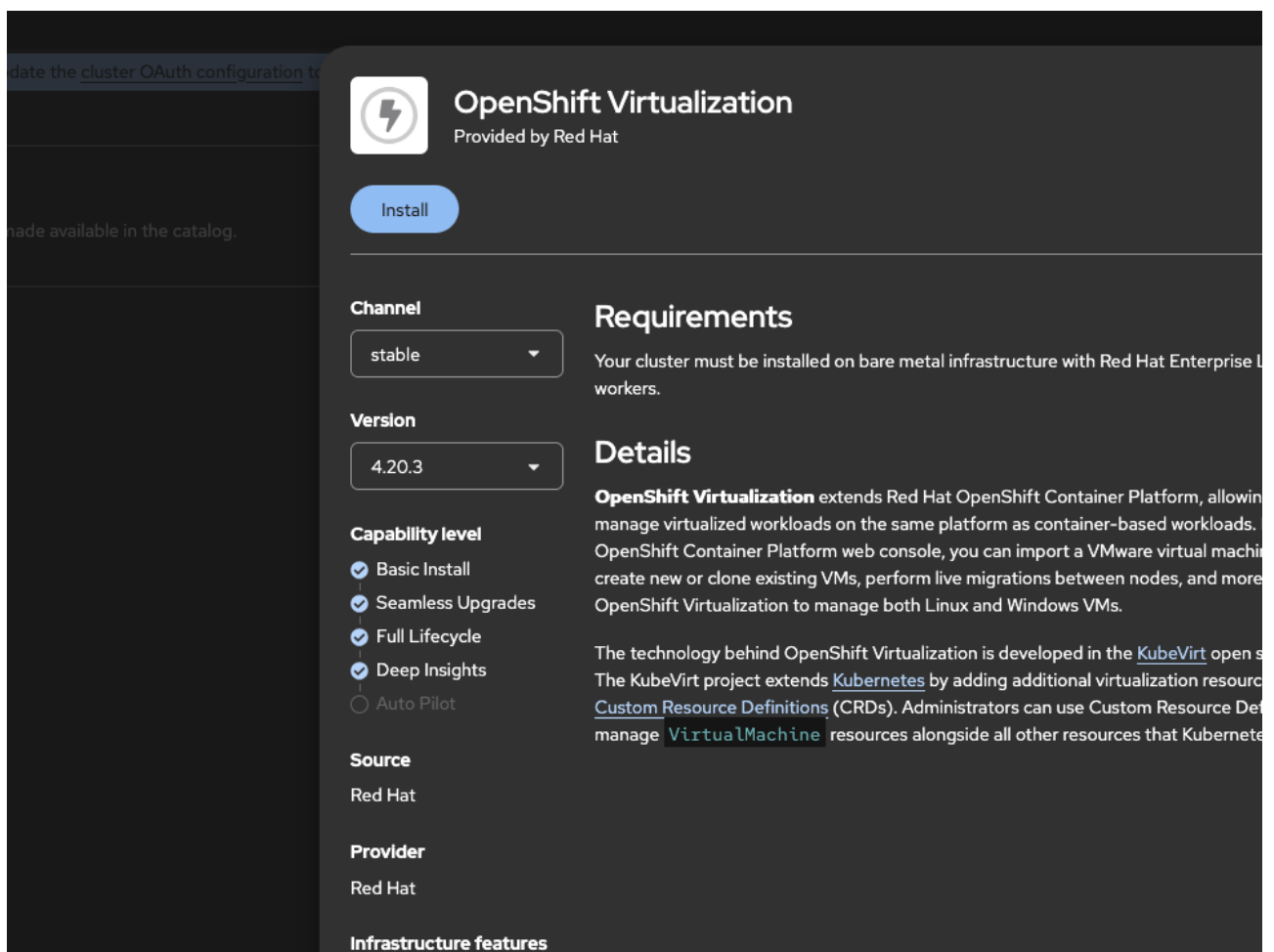
- ### 3. Filter by keyword



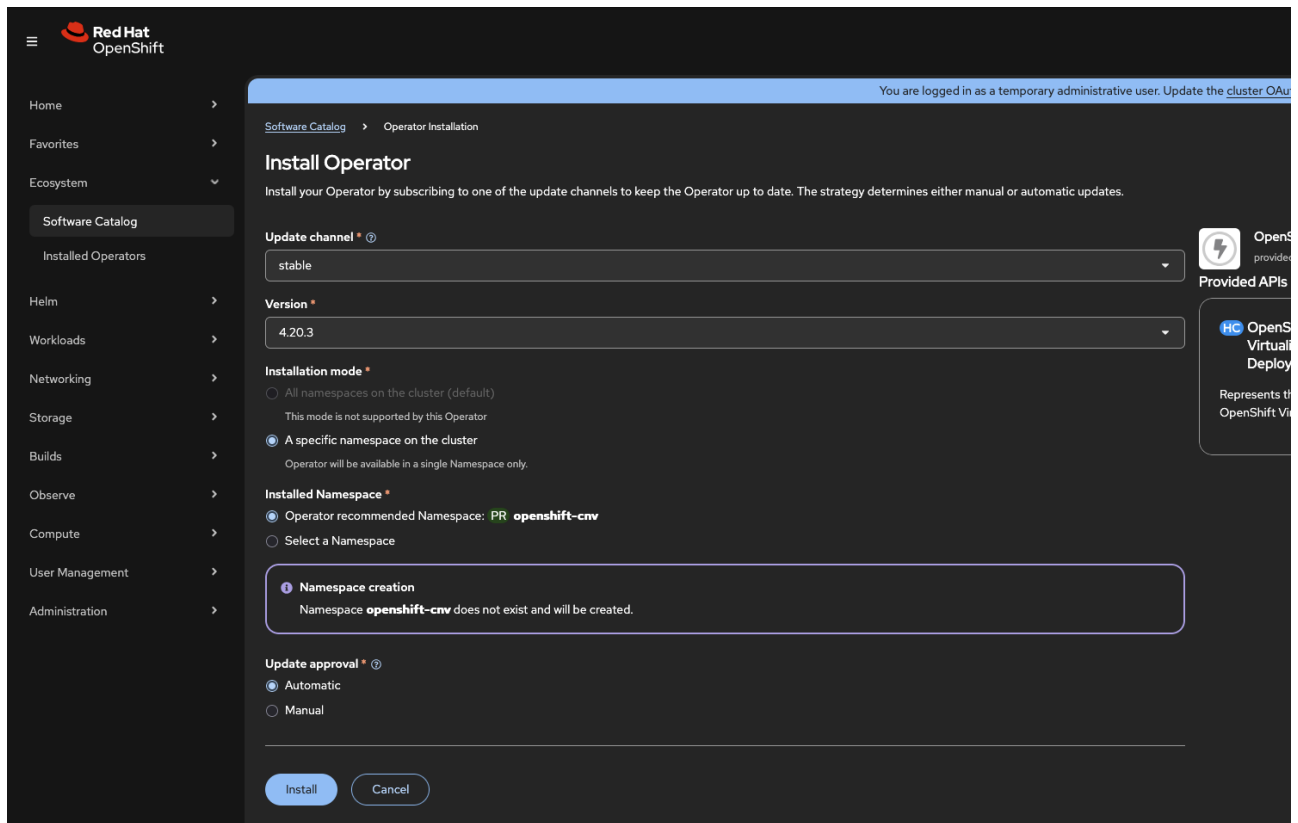


## 2: Operator

1. **Red Hat** **OpenShift Virtualization**
2. **Operator Install**



- ### 3. Install Operator **Install**



重要

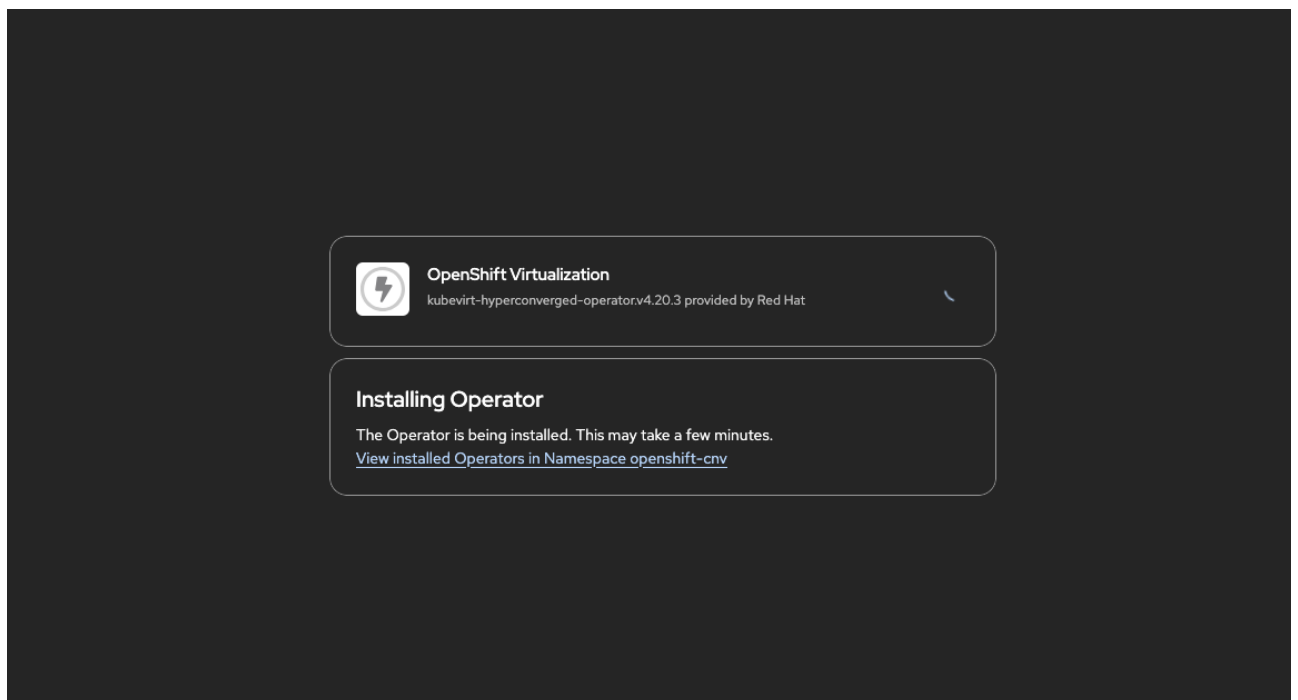
(Manual

approval)

インストール時に、Namespaceの作成に失敗した場合、インストールが失敗します。この場合、Namespaceを手動で作成する必要があります。

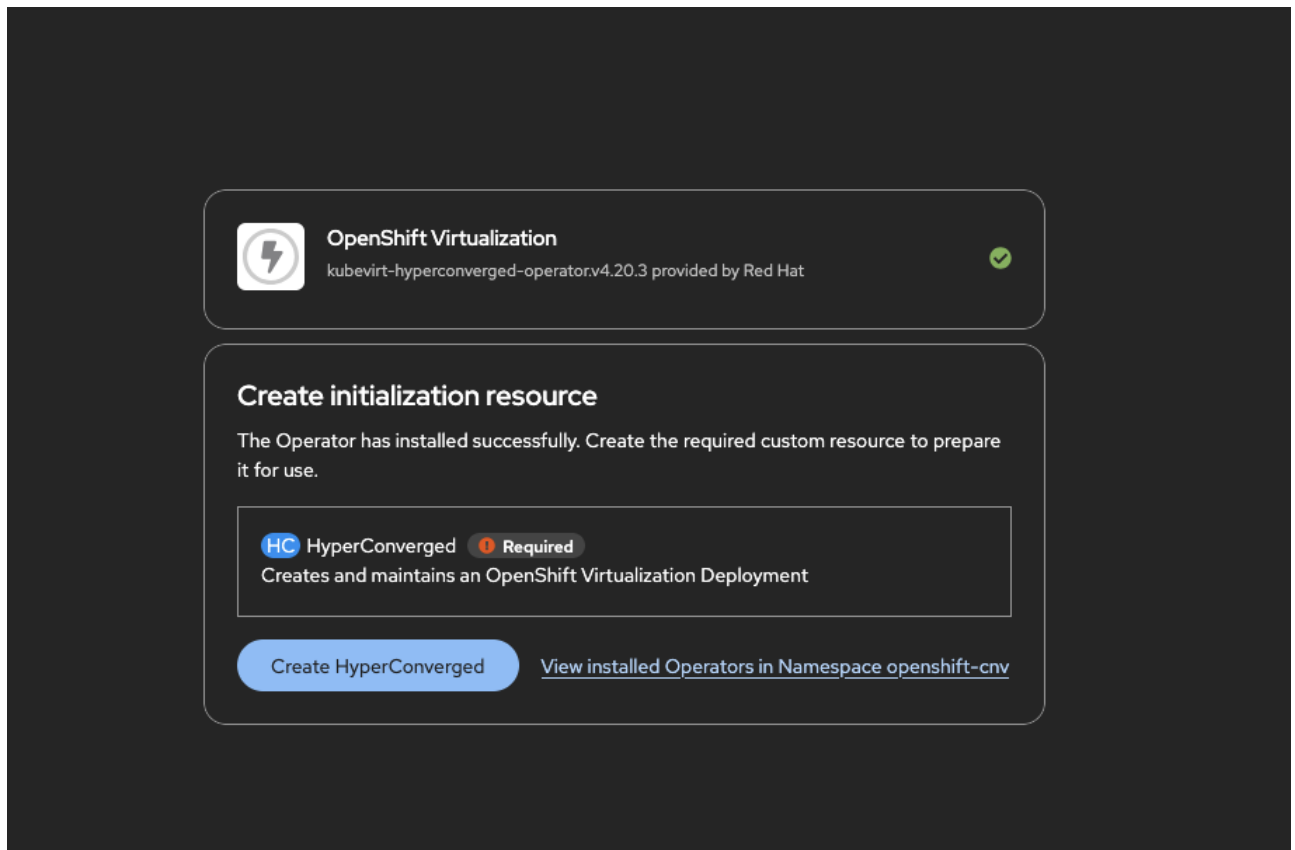
インストール時に、Namespaceの作成に失敗した場合、インストールが失敗します。

#### 4. Operator のインストール

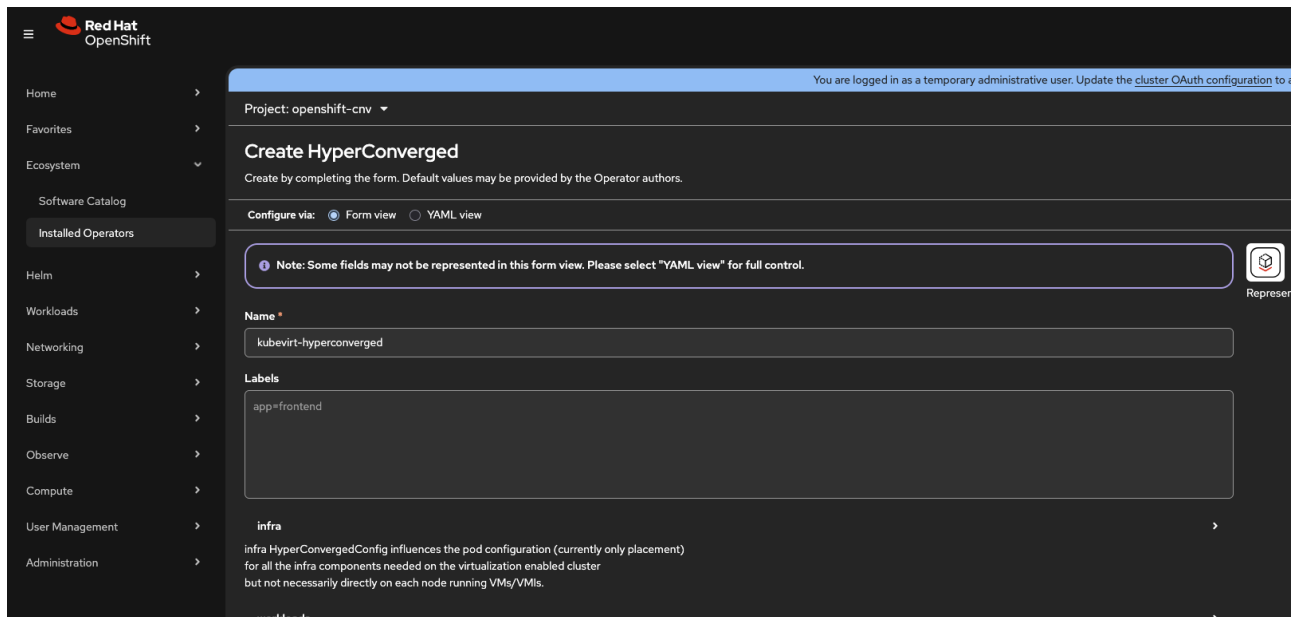


### 3: HyperConverged のインストール

#### 1. Operator のインストール Create HyperConverged のインストール



## 2. 仮想マシンをインストールする



## 3. Create 仮想マシン OpenShift Virtualization 仮想マシン

Observe
Compute
User Management
Administration

DataimportCronTemplates holds list of data import cron templates (golden images)

**permittedHostDevices**

PermittedHostDevices holds information about devices allowed for passthrough

**commonBootImageNamespace**

CommonBootImageNamespace override the default namespace of the common boot images, in order to hide them.

If not set, HCO won't set any namespace, letting SSP to use the default. If set, use the namespace to create the DataimportCronTemplates and the common image streams, with this namespace. This field is not set by default.

**CommonInstancetypesDeployment**

CommonInstancetypesDeployment holds the configuration of common-instancetypes deployment within KubeVirt.

**resourceRequirements**

ResourceRequirements describes the resource requirements for the operand workloads.

**filesystemOverhead**

FilesystemOverhead describes the space reserved for overhead when using Filesystem volumes. A value is between 0 and 1, if not defined it is 0.055 (5.5 percent overhead)

Create
Cancel

## 4: HyperConverged

HyperConverged

Project: openshift-cnv

Installed Operators
Operator details

OpenShift Virtualization
4.20.3 provided by Red Hat

Details
YAML
Subscription
Events
All instances
OpenShift Virtualization Deployment
HostPathProvisioner Deployment

Create HyperConverged

Name
Search by name...

| Name                       | Kind           | Status                                               | Labels                      | Last updated  |
|----------------------------|----------------|------------------------------------------------------|-----------------------------|---------------|
| HC kubevirt-hyperconverged | HyperConverged | Conditions: ReconcileComplete, Available, Upgradable | app=kubevirt-hyperconverged | 2 minutes ago |

## Operators

Red Hat
OpenShift

Administrator

Home
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Virtualization
Networking

You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow o

Project: openshift-cnv

Installed Operators

Installed Operators are represented by ClusterServiceVersions within this Namespace. For more information, see the [Understanding Operators documentation](#). Or create an O

Name
Search by name...

| Name                                                             | Managed Namespaces | Status               | Last updated         |
|------------------------------------------------------------------|--------------------|----------------------|----------------------|
| <div> OpenShift Virtualization 4.20.3 provided by Red Hat </div> | NS openshift-cnv   | Succeeded Up to date | Jan 28, 2026, 2:45 f |

23



kubevirt-hyperconverged-operator.v4.20.1 Succeeded

Succeeded `Ctrl+C` to exit

### Step 3: HyperConverged

HyperConverged

```
oc apply -f - <<EOF
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec:
EOF
```



For more information

HyperConverged

visit [https://docs.openshift.com/container-platform/latest/virt/install/installing-virt.html#virt-customizing-storage-profile\\_installing-virt](https://docs.openshift.com/container-platform/latest/virt/install/installing-virt.html#virt-customizing-storage-profile_installing-virt) [blank,window=\_blank]

### Step 4: Wait for HyperConverged

HyperConverged

```
oc wait hyperconverged kubevirt-hyperconverged -n openshift-cnv
--for=condition=Available --timeout=15m
```

Wait for 5-10 seconds

### Step 5: Verify

Verify

```
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o jsonpath='{range .status.conditions[*]}.{type}: {status}}\n\''
```

Expected output:

```
ReconcileComplete: True
Available: True
Progressing: False
Degraded: False
Upgradeable: True
```

OpenShift Virtualization

OpenShift (Web console or CLI) OpenShift Virtualization

Pod status

openshift-cnv namespace Pod Running Completed

```
oc get pods -n openshift-cnv
```

HyperConverged Condition

Condition

```
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o jsonpath='{range .status.conditions[*]}{.type}: {.status}{"\n"}{end}'
```

Output:

```
ReconcileComplete: True
Available: True
Progressing: False
Degraded: False
Upgradeable: True
```

DataSource

```
oc get datasources -n openshift-virtualization-os-images
```

Ready Condition RHELFedoraCentOSWindows

StorageClass

VM StorageClass

```
oc get storageclass
```

(default) StorageClass VM StorageClass

## OpenShift Virtualization

OpenShift

Virtualization



**Operator** 删除 Operator 部署的 HyperConverged 组件

**步骤 1:** 删除 VM 组件

删除 VM 组件

```
# 删除 namespace 中的 VM 组件
oc get vm -A

# VM 组件 (namespace 指定)
oc delete vm --all -n <namespace>
```



删除 VM 组件

**步骤 2: HyperConverged 组件**

删除 OpenShift Virtualization 组件

```
oc delete hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

删除 Pod 组件

```
# Pod 组件
oc get pods -n openshift-cnv -w
```

删除 Pod 组件

**步骤 3: Operator 删除 CSV 组件**

```
# subscription 删除
oc delete subscription hco-operatorhub -n openshift-cnv

# ClusterServiceVersion 删除
oc delete csv -n openshift-cnv -l operators.coreos.com/kubevirt-hyperconverged.openshift-cnv
```

**步骤 4: Namespace 删除 (删除)**

删除 Namespace 组件

```
oc delete namespace openshift-cnv
```



ステップ 5: 確認

確認コマンド

```
# 確認 "No resources found" 確認
oc get pods -n openshift-cnv
oc get csv -n openshift-cnv
```

確認

OpenShift Virtualization 確認コマンド

確認コマンド

- [virtctl](#) 確認 VM 確認 - VM 確認
- [virtctl](#) 確認 VM 確認
- VM 確認 IP 確認 - cloud-init/DHCP 確認Sysprep 確認 IP 確認

確認

確認コマンドSingle Node OpenShift (SNO) 確認コマンド

- [OpenShift Virtualization](#) 確認
- [OpenShift Virtualization](#) 確認
- [KubeVirt](#) 確認

## QEMU guest agent 確認

確認

確認コマンド Linux 確認コマンド `qemu-guest-agent` 確認コマンドQEMU guest agent 確認  
OS 確認コマンド OS 確認コマンド SSH  
確認コマンドIP 確認コマンドexec/ping 確認コマンド

確認

- Fedora/Debian/openSUSE Leap 確認 `virtctl console` 確認 `qemu-guest-agent` 確認
- cloud-init 確認 `packages` 確認 `runcmd` 確認 `qemu-guest-agent` 確認
- 確認コマンド OS 確認 OpenShift Virtualization 確認

確認

- OpenShift Virtualization Operator 確認コマンド **OpenShift 4.18+** ([OpenShift Virtualization](#) 確認  
確認)
- CLI 確認: `oc` `virtctl` `yq`

## namespace

namespace

```
oc create namespace qemu-ga-demo
```

```
oc project qemu-ga-demo
```

## virtctl console

`virtctl console <vm-name>` `cloud-init`  
`qemu-guest-agent` `systemctl enable`  
`--now qemu-guest-agent`



`cloud-init` `SSH`

## Fedora/RHEL

1: **Fedora**

CLI (Web )



`containerdisk` `datavolume` `PVC`  
`virt-launcher`

## fedora-no-qga.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-virtctl
  namespace: qemu-ga-demo
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-virtctl-rootdisk
    spec:
      storage:
        resources:
          requests:
            storage: 10Gi
        accessModes:
          - ReadWriteOnce
```

```

source:
  registry:
    url: docker://quay.io/ocpv_cookbook/fedora-no-qga:latest
template:
  metadata:
    labels:
      app: fedora-virtctl
  spec:
    domain:
      cpu:
        cores: 2
      memory:
        guest: 4Gi
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
    volumes:
      - name: rootdisk
        dataVolume:
          name: fedora-virtctl-rootdisk
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |-
            #cloud-config
            user: fedora
            password: fedora
            chpasswd: { expire: false }
EOF

```

Pod STATUS is **Running** successfully

```
oc get vm fedora-virtctl -w
```



**-w** continuously displays STATUS is **Running** until you press **Ctrl+C**

```

# Pod
NAME          AGE   STATUS   READY
fedora-virtctl 6h8m Running True

```

Pod 2 is

Web console

1. **Virtualization** → **VirtualMachines** 選択

2. 選択 **Create** 仮想マシンを作成

3. **With YAML** 選択

4. 仮想マシンを定義する YAML ファイルを選択

5. 選択 **YAML** ファイルを選択

6. 選択 **Create** 仮想マシンを作成

仮想マシン VM の **Overview** ページで **Status** が **Running** になっていることを確認

ステップ 2: **qemu-guest-agent** をインストール

CLI 操作: (Web コンソールから実行可能)

```
virtctl console fedora-virtctl
```

仮想マシン内から **ENTER** キーを押下して、仮想マシン内から **fedora** を起動

仮想マシン内から

Web コンソール:

1. **Virtualization** → **VirtualMachines** 選択

2. 仮想マシンを選択

3. **overview** から **Details** をクリックし "Open Web Console" をクリック (仮想マシンが起動していることを確認)

4. 仮想マシン **fedora** の **Enter** キーを押下

5. 仮想マシン **fedora** の **Enter** キーを押下

**qemu-guest-agent** をインストール

```
systemctl status qemu-guest-agent
```

```
# systemctl status qemu-guest-agent
Unit qemu-guest-agent.service could not be found.
```

**dnf** を使って **qemu-guest-agent** をインストール

```
sudo dnf install -y qemu-guest-agent && sudo systemctl enable --now qemu-guest-agent
```

**qemu-guest-agent** のインストールを確認

```
systemctl status qemu-guest-agent
```

```
# 確認
[fedora@fedora-virtctl ~]$ systemctl status qemu-guest-agent
● qemu-guest-agent.service - QEMU Guest Agent
   Loaded: loaded (/usr/lib/systemd/system/qemu-guest-agent.service; enabled; preset:
   enabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf
   Active: active (running) since Tue 2026-06-02 21:49:02 UTC; 8s ago # <-- Active
   (running)  000
  Invocation: e1d60262b0cc499b967d42561a6e59ff
    Process: 1436 ExecStartPre=<<truncated>>
   Main PID: 1438 (qemu-ga)
     Tasks: 2 (limit: 4618)
    Memory: 996K (peak: 1.4M)
       CPU: 21ms
    CGroup: /system.slice/qemu-guest-agent.service
            └─1438 /usr/bin/qemu-ga --method=virtio-serial --path=/dev/virtio
               -ports/org.qemu.guest_agent.0 <<truncated>>
```

`ctrl+] 000 ctrl+5 00000 VM 000000000000`

`00 VM 00000000000000000000 VM_NAME 0000000000`

```
VM_NAME=fedora-virtctl
```

`qemu-guest-agent 0 OCP-Virt 00000000000000 000000000000000000OCP-Virt 000000000000000000`

## Debian

`0000 1: Debian 0000000000`

CLI 00000000000000 (Web 0000000000000000 0000 000000):

`debian-no-qga.yaml 00000000`

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: debian-virtctl
  namespace: qemu-ga-demo
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: debian-virtctl-rootdisk
    spec:
      storage:
```

```

resources:
  requests:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  source:
    registry:
      url: docker://quay.io/ocpv_cookbook/debian-no-qga:latest
template:
  metadata:
    labels:
      app: debian-virtctl
  spec:
    domain:
      cpu:
        cores: 2
      memory:
        guest: 4Gi
      devices:
        disks:
          - name: rootdisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        volumes:
          - name: rootdisk
            dataVolume:
              name: debian-virtctl-rootdisk
          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |-
                #cloud-config
                user: debian
                password: debian
                chpasswd: { expire: false }
EOF

```

Pod STATUS is **Running** successfully:

```
oc get vm debian-virtctl -w
```



**-w** continuously watches STATUS is **Running** and **Ctrl+C** to exit

```

# NAME          AGE    STATUS    READY

```

```
debian-virtctl    5m31s  Running  True
```

□□□□ 2 □□□□□

[illegible]

1. **Virtualization** → **VirtualMachines** `VirtualMachines`
2. `VirtualMachines` **Create** `VirtualMachines` `VirtualMachines`
3. **With** **YAML** `VirtualMachines`
4. `VirtualMachines` `VirtualMachines` `VirtualMachines` `VirtualMachines`
5. `VirtualMachines` **YAML** `VirtualMachines`
6. `VirtualMachines` **Create** `VirtualMachines`

VM "Overview tab" Status **Running**

## 2: qemu-guest-agent

CLI 実行: (Web ブラウザから実行可能)

```
virtctl console debian-virtctl
```

[illegible]

□□□□□□□□□□

Web 00000000:

1. **Virtualization** → **VirtualMachines** 仮想マシン
2. 仮想マシンをインストール
3. **overview** から **Details** へ移動し "Open Web Console" をクリック (仮想マシンのコンソールを開く)
4. 仮想マシン **debian** のコンソールで Enter を押す
5. 仮想マシン **debian** のコンソールで Enter を押す

qemu-guest-agent ████████████████████████████████████████

```
systemctl status qemu-guest-agent
```

```
# Unit qemu-guest-agent.service could not be found.
```

[illegible]

```
sudo apt update
```

apt `qemu-guest-agent` `qemu-guest-agent`

```
sudo apt install -y qemu-guest-agent && sudo systemctl enable --now qemu-guest-agent
```

`qemu-guest-agent` `qemu-guest-agent`

```
systemctl status qemu-guest-agent
```

```
# qemu-guest-agent.service - QEMU Guest Agent
Loaded: loaded (/lib/systemd/system/qemu-guest-agent.service; static)
Active: active (running) since Mon 2026-06-08 19:37:00 UTC; 2s ago
Main PID: 1013 (qemu-ga)
Tasks: 2 (limit: 4657)
Memory: 724.0K
CPU: 7ms
CGroup: /system.slice/qemu-guest-agent.service
└─1013 /usr/sbin/qemu-ga
```

`ctrl+]` `ctrl+5` VM `qemu-guest-agent`

VM `VM_NAME` `qemu-guest-agent`:

```
VM_NAME=debian-virtctl
```

`qemu-guest-agent` `OCP-Virt` `OCP-Virt` `OCP-Virt`

## openSUSE Leap

openSUSE `qemu-guest-agent` `qemu-guest-agent` openSUSE Leap `qemu-ga` `virtio-port` `udev` `systemd`

`1: openSUSE Leap`

CLI `Web` `Web`

`opensuse-leap-no-qga.yaml`

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
```



```

name: suse-virtctl
namespace: qemu-ga-demo
spec:
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: suse-virtctl-rootdisk
      spec:
        storage:
          resources:
            requests:
              storage: 10Gi
          accessModes:
            - ReadWriteOnce
        source:
          registry:
            url: docker://quay.io/ocpv_cookbook/opensuse-leap-15-no-qga:latest
  template:
    metadata:
      labels:
        app: suse-virtctl
    spec:
      domain:
        cpu:
          cores: 2
        memory:
          guest: 4Gi
      devices:
        disks:
          - name: rootdisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
      volumes:
        - name: rootdisk
          dataVolume:
            name: suse-virtctl-rootdisk
        - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: opensuse
          password: opensuse
          chpasswd: { expire: false }
EOF

```

STATUS Running:

```
oc get vm suse-virtctl -w
```



-w continuously STATUS Running Ctrl+C

```
#  
NAME          AGE   STATUS  READY  
suse-virtctl  5m31s Running True
```

2

Web

1. **Virtualization** → **VirtualMachines**
2. **Create**
3. **With YAML**
- 4.
5. **YAML**
6. **Create**

VM OverviewStatus Running

2: **qemu-guest-agent**

CLI (Web )

```
virtctl console suse-virtctl
```

ENTER **opensuse**

Web :

1. **Virtualization** → **VirtualMachines**
- 2.
3. **overview** **Details** Open Web Console ( )
4. **opensuse** Enter
5. **opensuse** Enter

**qemu-guest-agent**

```
systemctl list-units | grep qemu
```



```
VM_NAME=suse-virtctl
```

`qemu-guest-agent` [OCP-Virt](#) [インストール](#) `qemu-guest-agent` [インストール](#)

**cloud-init** `packages` `run` `cmd` `run`

`cloud-init` `packages` `qemu-guest-agent`  
`cloud-init` `run` `systemctl enable --now qemu-guest-agent`



`cloud-init` `SSH`

## Fedora/RHEL

`fedora-qga-package.yaml`

`fedora-qga-package.yaml`

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-cloudinit
  namespace: qemu-ga-demo
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-cloudinit-rootdisk
    spec:
      storage:
        resources:
          requests:
            storage: 10Gi
        accessModes:
        - ReadWriteOnce
      source:
        registry:
          url: docker://quay.io/ocpv_cookbook/fedora-no-qga:latest
    template:
      metadata:
        labels:
          app: fedora-cloudinit
      spec:
        domain:
          cpu:
            cores: 2
          memory:
            guest: 4Gi
```

```

devices:
  disks:
    - name: rootdisk
      disk:
        bus: virtio
    - name: cloudinitdisk
      disk:
        bus: virtio
  volumes:
    - name: rootdisk
      dataVolume:
        name: fedora-cloudinit-rootdisk
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora
          chpasswd: { expire: false }
          packages: ①
            - qemu-guest-agent
          runcmd: ②
            - systemctl enable --now qemu-guest-agent ③
EOF

```

① **packages** cloud-init `qemu-guest-agent`

② **runcmd** cloud-init

③ `systemctl enable --now qemu-guest-agent`

STATUS `Running`:

```
oc get vm fedora-cloudinit -w
```



`-w` STATUS `Running` `Ctrl+C`

```

#
NAME          AGE   STATUS   READY
fedora-cloudinit 8m23s Running True

```

VM\_NAME

```
VM_NAME=fedora-cloudinit
```

OCP-Virt `qemu-guest-agent` OCP-Virt

## Debian

cloud-init の `packages` 項目に `runcmd` を指定して Fedora の `cloud-init` の VM の

インストール

`debian-qga-package.yaml` の内容

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: debian-cloudinit
  namespace: qemu-ga-demo
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: debian-cloudinit-rootdisk
    spec:
      storage:
        resources:
          requests:
            storage: 10Gi
        accessModes:
        - ReadWriteOnce
      source:
        registry:
          url: docker://quay.io/ocpv_cookbook/debian-no-qga:latest
  template:
    metadata:
      labels:
        app: debian-cloudinit
    spec:
      domain:
        cpu:
          cores: 2
        memory:
          guest: 4Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
      volumes:
        - name: rootdisk
          dataVolume:
            name: debian-cloudinit-rootdisk
```

```
- name: cloudinitdisk
cloudInitNoCloud:
  userData: |-
    #cloud-config
    user: debian
    password: debian
    chpasswd: { expire: false }
    packages:
      - qemu-guest-agent
    runcmd:
      - systemctl enable --now qemu-guest-agent
EOF
```

🔍 STATUS Running 🔍

```
oc get vm debian-cloudinit -w
```



-w 🔍 STATUS Running 🔍 Ctrl+C 🔍

```
# 🔍
NAME      AGE   STATUS  READY
debian-cloudinit 5m31s Running True
```

🔍 VM 🔍 VM\_NAME 🔍

```
VM_NAME=debian-cloudinit
```

🔍 OCP-Virt 🔍 qemu-guest-agent 🔍 OCP-Virt 🔍 🔍

## openSUSE Leap

cloud-init 🔍 packages 🔍 Fedora 🔍 Debian 🔍 runcmd  
🔍 VM 🔍

🔍

opensuse-leap-qga-package.yaml 🔍

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: suse-cloudinit
  namespace: qemu-ga-demo
spec:
  runStrategy: Always
```

```

dataVolumeTemplates:
- metadata:
  name: suse-cloudinit-rootdisk
spec:
  storage:
    resources:
      requests:
        storage: 10Gi
    accessModes:
      - ReadWriteOnce
  source:
    registry:
      url: docker://quay.io/ocpv_cookbook/opensuse-leap-15-no-qga:latest
template:
  metadata:
    labels:
      app: suse-cloudinit
  spec:
    domain:
      cpu:
        cores: 2
      memory:
        guest: 4Gi
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
    volumes:
      - name: rootdisk
        dataVolume:
          name: suse-cloudinit-rootdisk
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |-
            #cloud-config
            user: opensuse
            password: opensuse
            chpasswd: { expire: false }
          packages:
            - qemu-guest-agent

```

EOF

□□□□□ STATUS □ Running □□□□□□□□□□

```
oc get vm suse-cloudinit -w
```





-w 確認 VM の STATUS が Running かどうか Ctrl+C で終了

```
# 確認
NAME                AGE  STATUS  READY
suse-cloudinit      39m  Running True
```

確認 VM の VM\_NAME が正しいかどうか

```
VM_NAME=suse-cloudinit
```

確認 OCP-Virt が qemu-guest-agent を OCP-Virt がインストールされているかどうか

**qemu-guest-agent** が **OCP-Virt** にインストールされているかどうか

Web から oc を virtctl CLI を使って qemu-guest-agent が OS にインストールされているかどうか OCP-Virt

確認

AgentConnected が True かどうか

```
oc get vm $VM_NAME -oyaml | yq -e '.status.conditions[] | select(.type == "AgentConnected")'
```

確認

```
# 確認
lastProbeTime: "2026-06-04T13:33:28Z"
lastTransitionTime: null
status: "True"
type: AgentConnected
```

guest-agent が VM に IP が割り当てられているかどうか

```
oc get vmi $VM_NAME -oyaml | yq -e '.status.interfaces[]'
```

```
# 確認
infoSource: domain, guest-agent ①
interfaceName: enp1s0
ipAddress: 10.130.0.22
ipAddresses:
- 10.130.0.22
linkState: up
mac: 0a:58:0a:82:00:16
```

```
name: default
podInterfaceName: eth0
queueCount: 1
```

① infoSource 为 guest-agent 的 IP 地址 qemu-guest-agent 的 IP 地址

OS 的 IP 地址

```
virtctl guestosinfo $VM_NAME
```

## OCP-Virt Web 界面

1. **Virtualization** → **VirtualMachines**
- 2.

**Overview** 为 qemu-guest-agent 的 OS 的 IP 地址

The screenshot shows the OCP-Virt Web interface for a virtual machine named 'fedora-vm'. The interface is in a dark theme. At the top, there's a header with 'VM fedora-vm' and status indicators 'Running' and 'Not migratable'. Below the header, there's a navigation bar with tabs: Overview, Metrics, YAML, Configuration, Events, Console, Snapshots, and Diagnostics. The 'Overview' tab is selected. The main content area is divided into two sections. On the left, there's a 'Details' section with a list of properties: Name (fedora-vm), Status (Running), Created (Jun 2, 2026, 5:05 PM (1 day ago)), Operating system (Fedora Linux 44 (Cloud Edition) - highlighted with a red box), CPU | Memory (2 CPU | 4 GiB Memory), Time zone (UTC), Template (None), and Hostname (fedora-vm). On the right, there's a 'VNC console' section with a link 'Open web console' and a terminal window showing the output of the 'cat /etc/passwd' command.

**Diagnostics** 为 AgentConnected 的 status 为 True 的 IP 地址



DataSource (VM) Storage Class  
DataSource (VM) Storage Class

DataSource:

OpenShift 4.19 4.20

DataSource

DataSource

- **OpenShift DataSource:** OpenShift Virtualization DataSource OpenShift DataSource
- DataSource: cluster-admin DataSource
- **Storage Class:** Storage Class 1 DataSource
- DataSource: DataSource

## 1: Storage Class

Storage Class Storage Class Storage Class

```
oc get sc
```

Storage Class (default) Storage Class

Storage Class:

| NAME                  | PROVISIONER                      | AGE | RECLAIMPOLICY |
|-----------------------|----------------------------------|-----|---------------|
| VOLUMEBINDINGMODE     | ALLOWVOLUMEEXPANSION             |     |               |
| ceph-rbd-sc (default) | rbd.csi.ceph.com                 | 2d  | Delete        |
| Immediate             | true                             |     |               |
| hostpath-provisioner  | kubevirt.io/hostpath-provisioner | 1d  | Delete        |
| WaitForFirstConsumer  | false                            |     |               |
| local-storage         | kubernetes.io/no-provisioner     | 1d  | Delete        |
| WaitForFirstConsumer  | false                            |     |               |

Storage Class:

| NAME                  | PROVISIONER                      | AGE | RECLAIMPOLICY |
|-----------------------|----------------------------------|-----|---------------|
| VOLUMEBINDINGMODE     | ALLOWVOLUMEEXPANSION             |     |               |
| ceph-rbd-sc (default) | rbd.csi.ceph.com                 | 2d  | Delete        |
| Immediate             | true                             |     |               |
| hostpath-provisioner  | kubevirt.io/hostpath-provisioner | 1d  | Delete        |
| WaitForFirstConsumer  | false                            |     |               |
| local-storage         | kubernetes.io/no-provisioner     | 1d  | Delete        |

1d

□□□□□□□□□□

1. `__attribute__((storage_class))` Storage Class `__attribute__((storage_class))`
2. `__attribute__((storage_class))` Storage Class `__attribute__((storage_class))`

**Figure 2: Data Collection Process (Overview)**

Storage Class 1

Storage Class 

```
oc annotate sc <the-old-default-storage-class-name> storageclass.kubernetes.io/is-
default-class-
```



□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

### 3: Storage Class

Storage Class Storage Class OpenShift

```
storageclass.kubernetes.io/is-default-class: "true"
```

```
oc annotate sc <your-storage-class-name> storageclass.kubernetes.io/is-default-
class="true"
```

Storage Class `ceph-rbd-sc`:

```
oc annotate sc ceph-rbd-sc storageclass.kubernetes.io/is-default-class="true"
```

#### □□□□ 4: □□□□□ Storage Class □□□

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc get sc
```

[illegible]

| NAME                  | PROVISIONER          | RECLAIMPOLICY |
|-----------------------|----------------------|---------------|
| VOLUME                | ALLOWVOLUMEEXPANSION | AGE           |
| ceph-rbd-sc (default) | rbd.csi.ceph.com     | Delete        |
| Immediate             | true                 | 2d            |

|                      |                                  |        |
|----------------------|----------------------------------|--------|
| hostpath-provisioner | kubevirt.io/hostpath-provisioner | Delete |
| WaitForFirstConsumer | false                            | 1d     |
| local-storage        | kubernetes.io/no-provisioner     | Delete |
| WaitForFirstConsumer | false                            | 1d     |

Storage Class (default)

## 5: Storage Class

Storage Class PVC

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-default-storage
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
EOF
```

PVC Storage Class

```
oc get pvc test-default-storage -n default
```

:

| NAME                 | STATUS  | VOLUME | CAPACITY | ACCESS MODES | STORAGECLASS | AGE |
|----------------------|---------|--------|----------|--------------|--------------|-----|
| test-default-storage | Pending |        |          |              | lvms-vg1     | 0s  |

PVC Pending Storage Class (lvms-vg1)

PVC Pending Storage Class WaitForFirstConsumer

| WaitForFirstConsumer | PVC Pod PV | Pod   |
|----------------------|------------|-------|
|                      |            | (LVM) |

|           |                            |                                |
|-----------|----------------------------|--------------------------------|
| Immediate | PVC によって提供される PV によって提供される | (NFS/Ceph によって提供される) によって提供される |
|-----------|----------------------------|--------------------------------|

Storage Class によって提供される

```
oc get sc <storage-class-name> -o jsonpath='{.volumeBindingMode}'
```

## 6: VM によって提供される

Storage Class によって提供される

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: test-vm-default-storage
  namespace: default
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: test-vm-disk
    spec:
      pvc:
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 10Gi
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
  template:
    metadata:
      labels:
        kubevirt.io/vm: test-vm-default-storage
    spec:
      domain:
        devices:
          disks:
          - name: datavolumedisk
            disk:
              bus: virtio
        resources:
          requests:
            memory: 1Gi
      </pre>

```

```

        cpu: 1
    networks:
    - name: default
      pod: {}
    volumes:
    - name: datavolumedisk
      dataVolume:
        name: test-vm-disk
EOF

```

□□□□□□□□□□

□□□□□□

□□□□□□□ **Storage Class**

□□□ Storage Class □□□□□□□□□□□□□□□□□□1 □□□□□□□□□□□□□□□□□□□□

```

oc get sc -o custom-
columns=NAME:.metadata.name,DEFAULT:.metadata.annotations.storageclass\.kubernetes\.io
/is-default-class

```

**Storage Class** □□□□□□□

Storage Class □□□□□□□□□□□□□□□□□□□□□□□□□□□□

```

oc get sc <storage-class-name>

```

□□□□ (**Permission Denied**)

cluster-admin □□□□□□□□□□□□□□□□□□□□

```

oc auth can-i create storageclass --all-namespaces

```

□□□□□□

□□□□ Storage Class □□□□□□□□□□□□□□□□□□

```

oc get sc -o wide

```

□□□ Storage Class □□□□□□□□□□□□

```

oc describe sc <storage-class-name>

```



## PVC 与 Storage Class 的关系

```
oc get pvc --all-namespaces -o custom-  
columns=NAMESPACE:.metadata.namespace,NAME:.metadata.name,STORAGECLASS:.spec.storageCl  
assName
```

### Storage Class 的作用

1. Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息
2. Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息
3. Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息
4. Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息

### Storage Class

#### Storage Class 的作用

- Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息
- Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息
- [storageclass.kubernetes.io/is-default-class](https://storageclass.kubernetes.io/is-default-class) Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息
- PVC 与 VM 的关系: Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息
- OpenShift Virtualization Storage Class 的作用: 为 PVC 提供存储 (即 I/O 操作) 的 Storage Class 信息

### Storage Class

- [OpenShift - Persistent Storage](#)
- [OpenShift Virtualization - Storage](#)

## Web 与 VM 的关系

### Web

OpenShift Web 与 VM 的关系: (VM) 与 Web 的关系: VM

### Web 与 VM 的关系

Web 与 VM 的关系: VM 与 OpenShift Virtualization 的关系: Virtualization

openshift-virtualization-os-images namespace

+-----+  
|



OpenShift Virtualization Operator

1. **Template** creates **DataSource** from CPU/memory specifications and **VirtualMachine spec**.
2. **openshift-virtualization-os-images** namespace creates **DataSource** from OS and PVC information.
3. **openshift-virtualization-os-images** namespace creates **DataVolume** from OS and PVC information.
4. **openshift-virtualization-os-images** namespace creates **PVC** from OS and PVC information.
5. **virt-launcher pod** creates **PVC** from QEMU/KVM and VM information.
6. **VirtualMachineInstance** (VMI) creates VM from IP information.



OpenShift Virtualization Operator creates a **VirtualMachineInstance** (VMI) from a **VirtualMachine spec** (VMSpec) and a **DataSource** (DS) in the **openshift-virtualization-os-images** namespace.

(CSI)

OpenShift

- OpenShift Virtualization Operator is available in OpenShift 4.18 and later ([OpenShift Virtualization Operator](#))

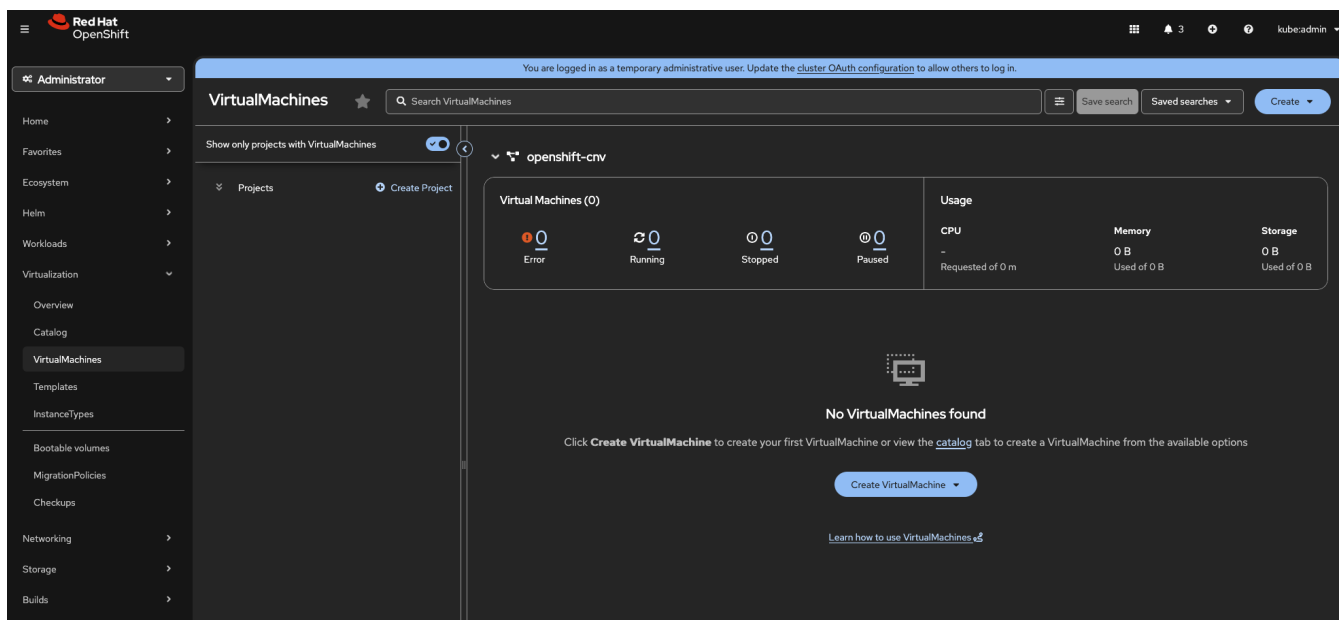
- VM `OpenShift Web` `StorageClass`
- `(DataSource)`

oc get datasources -n openshift-virtualization-os-images

```
oc get datasources -n openshift-virtualization-os-images
```

1. `VirtualMachines` `Virtualization` `VirtualMachines`

1. `OpenShift Web` `Virtualization` `VirtualMachines`
2. `Virtualization` `VirtualMachines`
3. `VirtualMachines` `VirtualMachines`



`VirtualMachines` `VirtualMachines` `VirtualMachines`

2. `VirtualMachines` `VirtualMachines`

`OpenShift Virtualization` `VirtualMachines` `VirtualMachines`

1. `VirtualMachines` `Create` `From template` `VirtualMachines`
2. `VirtualMachines` `VirtualMachines`
  - `Red Hat Enterprise Linux` - `RHEL` `VirtualMachines`
  - `Fedora` - `Fedora` `VirtualMachines`
  - `CentOS Stream` - `CentOS Stream` `VirtualMachines`
  - `Microsoft Windows` - `Windows Server` `VirtualMachines`



```
oc get template fedora-server-small -n openshift -o yaml
```

parameters parameters objects VirtualMachine Web  
VM



VM

### 3: VM

Fedora VM

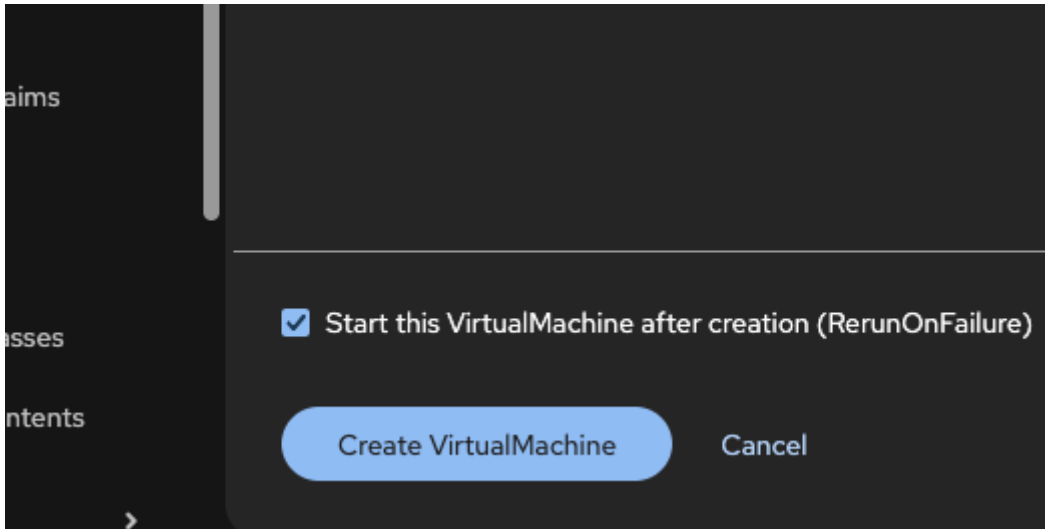
1. **Fedora**
- 2.
3. **Customize VirtualMachine**





□□□□

1. □□□□□□□□□□□□□□
2. **Create VirtualMachine** □□□□□□□□
3. □□□□□□**Start this VirtualMachine after creation** □□□□□□□□□□□□□□□□□□□□



#### □□□□ 4: VM □□□□

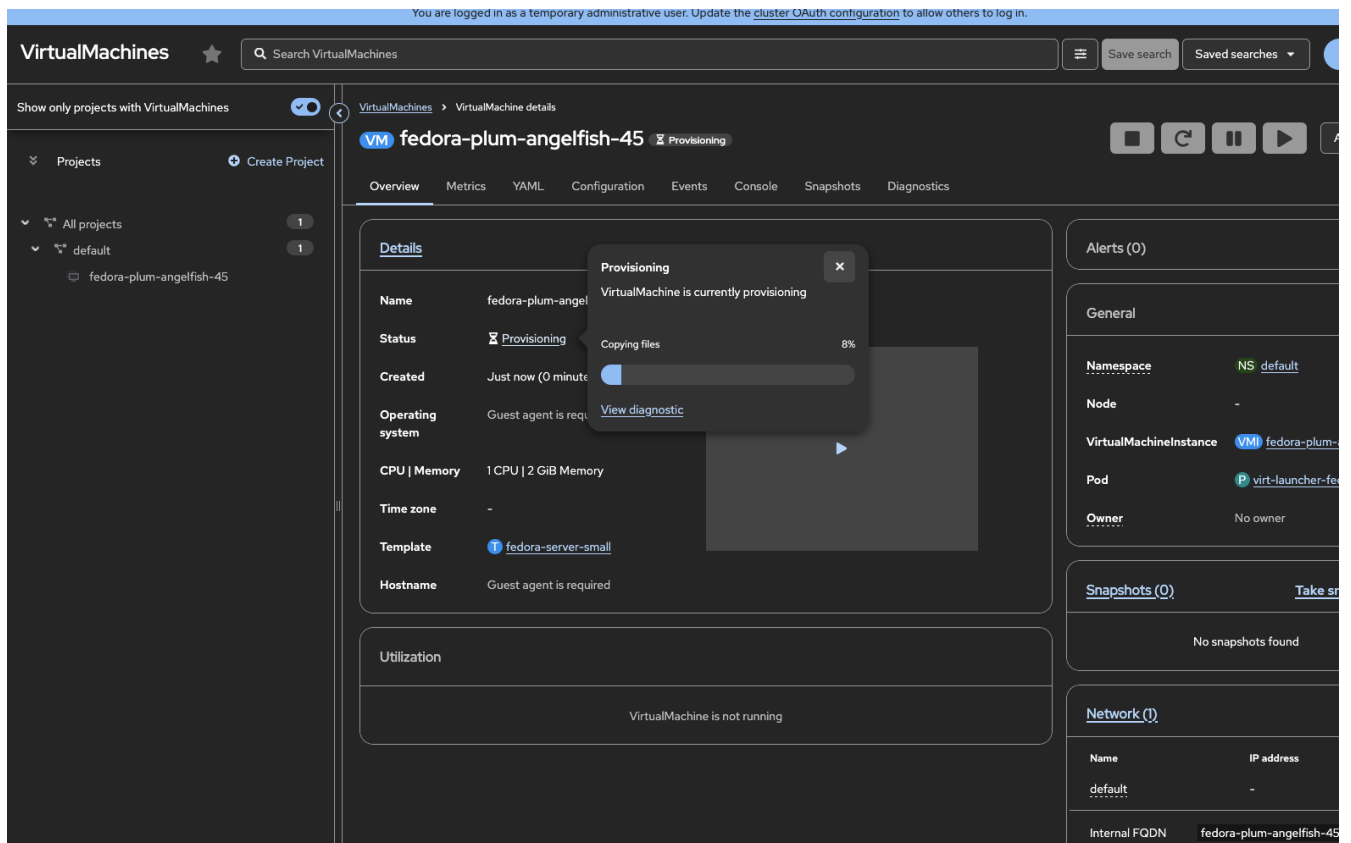
□□□□□□□□□□ VM □□□□□□□□□□□□□□□□

□□□□□□□□□□□□

VM □□□□□□□□□□□□□□□□□□□□

1. **Provisioning** - □□□□□□□□□□□□□□□□□□□□□□
2. **Starting** - VM □□□□□□□□ (□□□□□□□□□□□□□□)
3. **Running** - VM □□□□□□□□□□□□





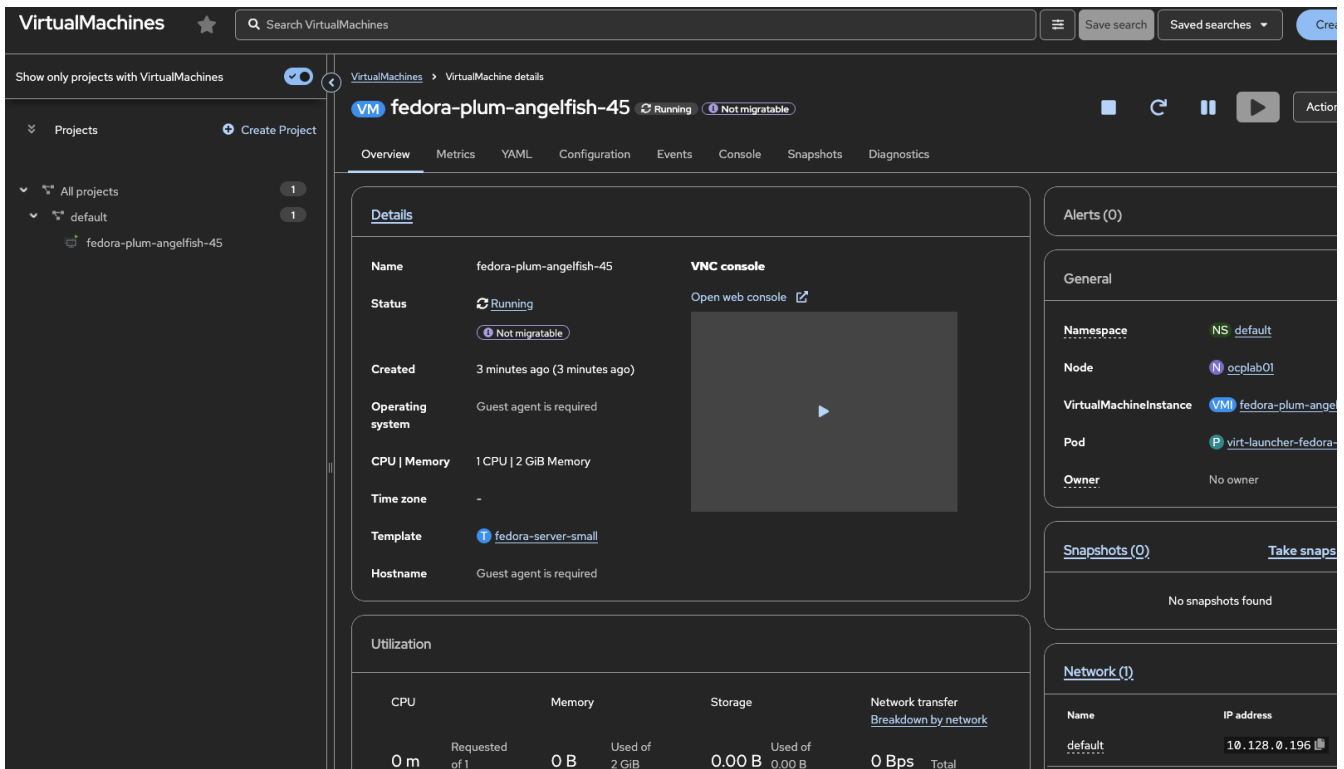
## Events

# CLI

```
oc get events -n <your-namespace> --field-selector involvedObject.name=<vm-name> -w
```

## VM

Running

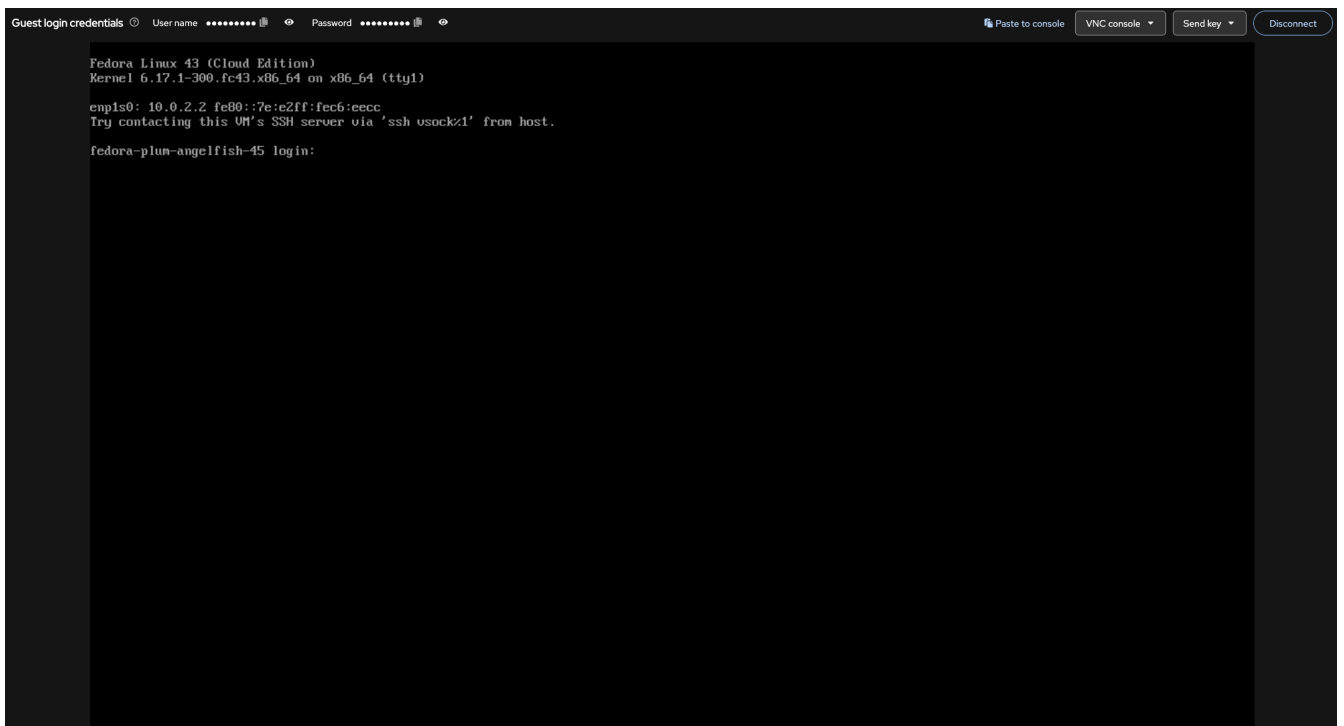


## 5: VM 操作

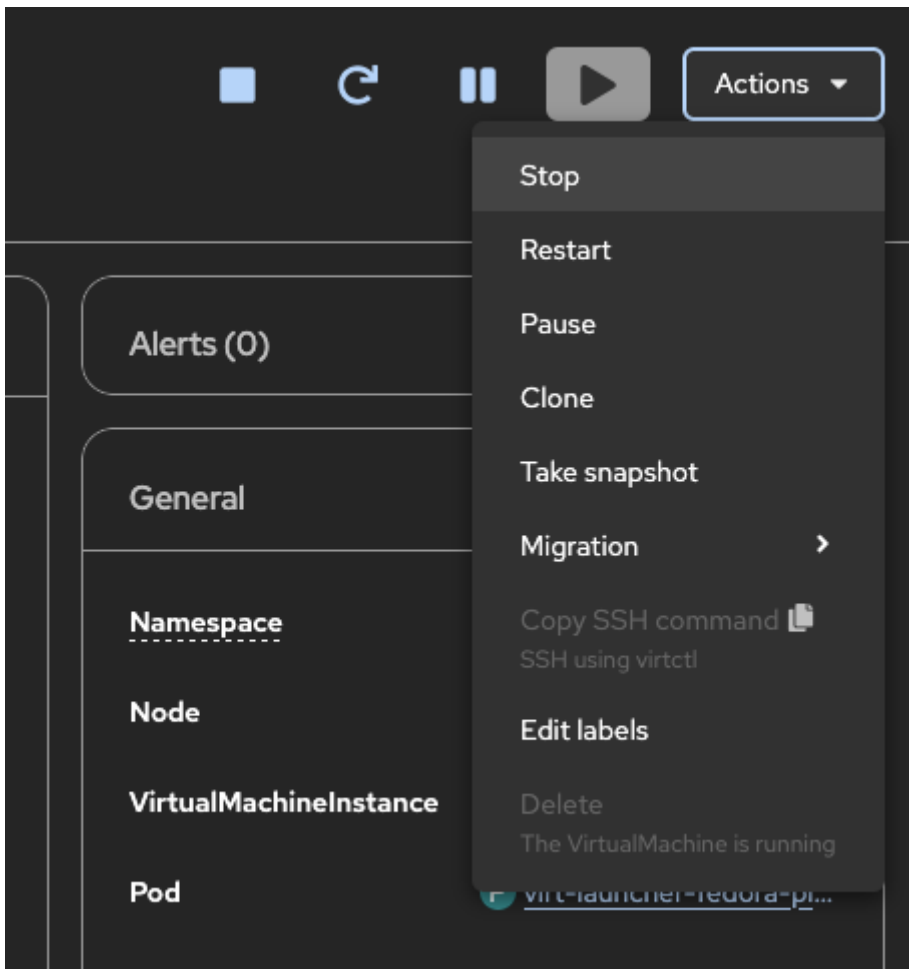
Web 操作 VM 操作

操作

1. VM 操作 Console 操作
2. 操作 VNC 操作







## VM 停止

1. **Actions** → **Stop** をクリックする
2. 仮想マシンが停止するまで待つ
3. VM を削除する

```
# CLI から
virtctl stop <vm-name>
```

## VM 再起動

1. **Actions** → **Restart** をクリックする
2. VM が再起動するまで待つ

```
# CLI から
virtctl restart <vm-name>
```

## VM 一時停止

## VM 一時停止から再開

1. **Actions** → **Pause** をクリックする

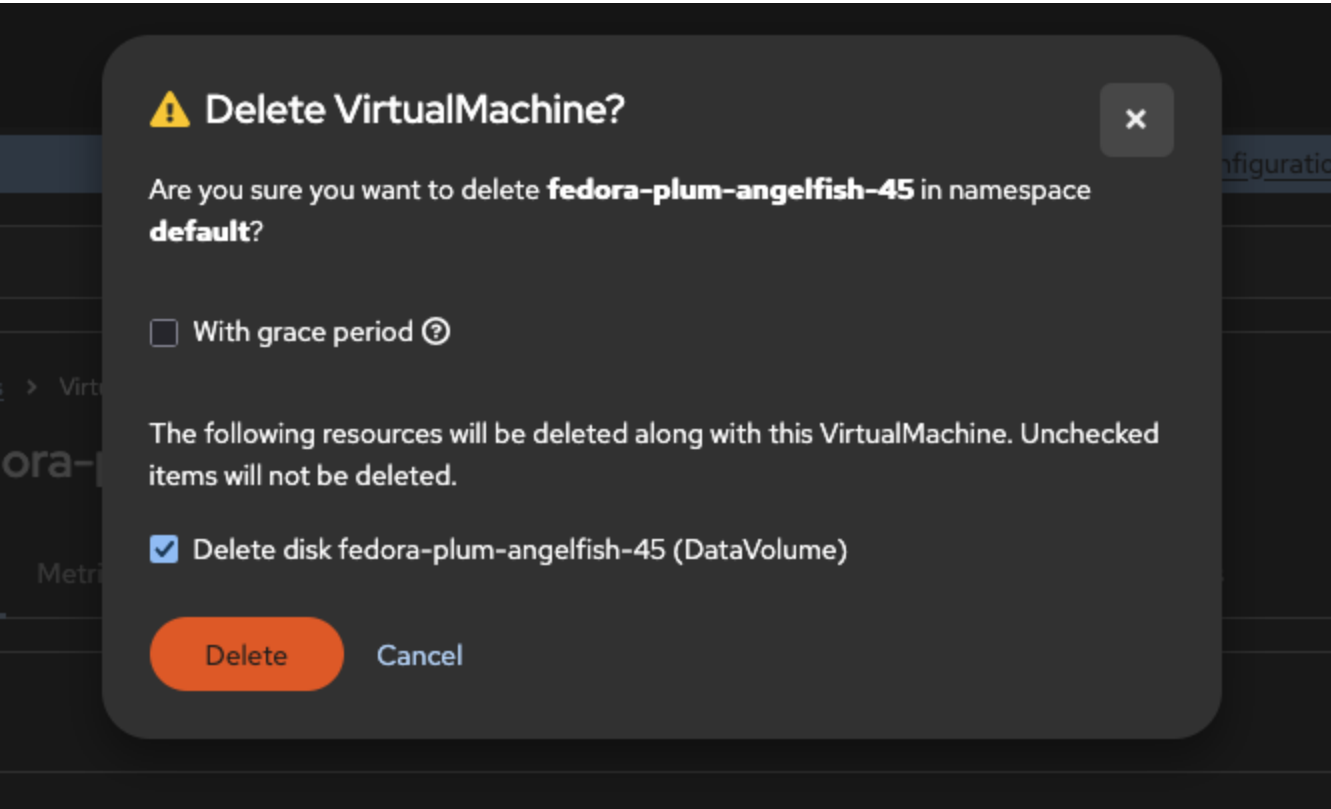
- 2. VM 状態を確認する
- 3. **Actions** → **Unpause** をクリックする



この VM は正常に起動しています。

VM の削除

- 1. **Actions** → **Delete** をクリックする
- 2. 削除対象の PVC (PVC) を確認する
- 3. 削除を確認する





この VM のディスク (DataVolume) を削除するかどうかを確認する。Delete disk



この VM は正常に起動しています。

タスク 7: VM の削除

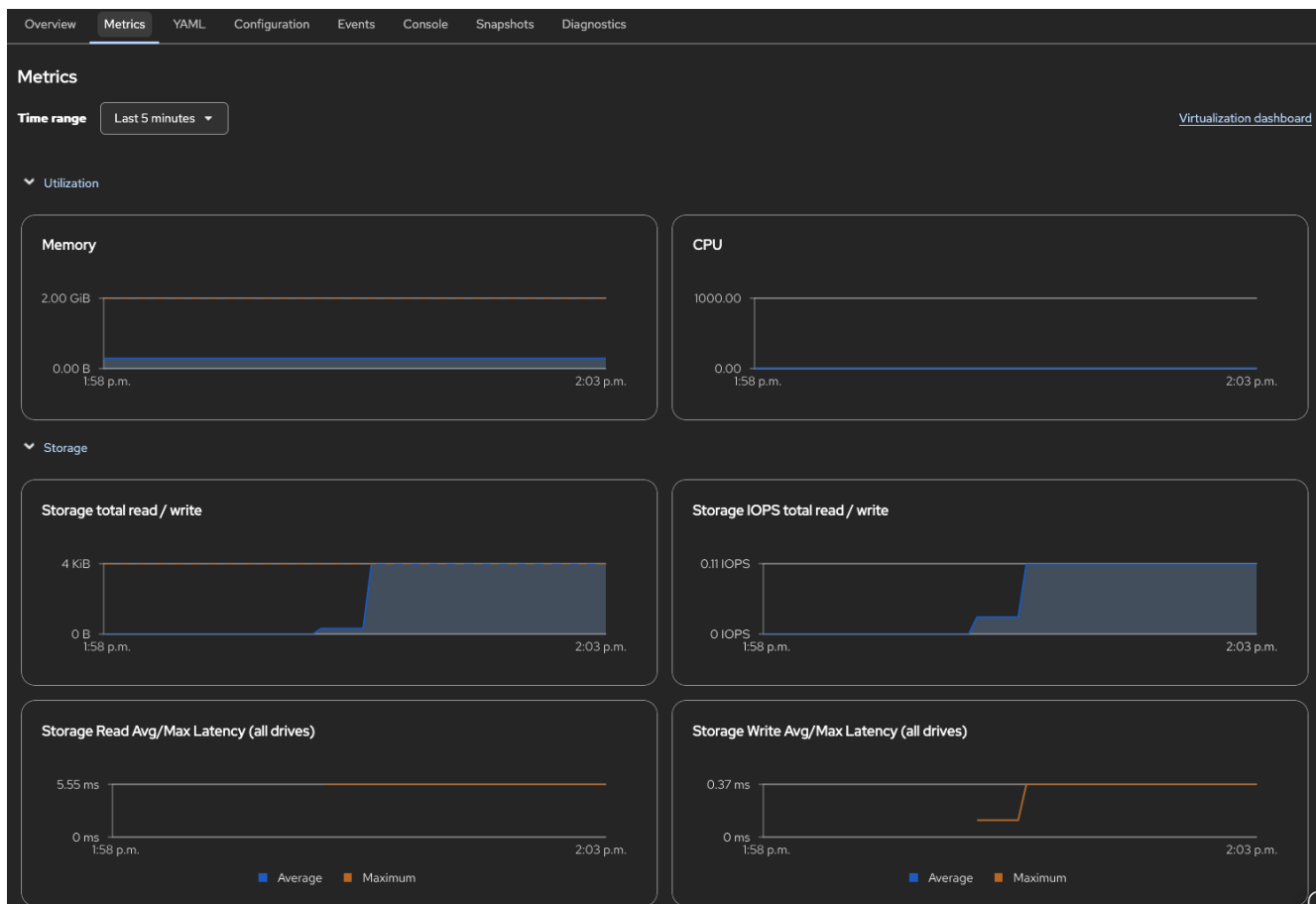
VM の削除を確認する

Overview

この VM の詳細を確認する

- VM の名前
- IP アドレス (ネットワークインターフェース)





VM の QEMU guest agent をインストールする

## YAML

VM の設定

1. **YAML** の設定
2. **VirtualMachine** の設定
3. **Save** の設定

VirtualMachines > VirtualMachine details

VM fedora-plum-angelfish-45 Running Not migratable

Overview Metrics **YAML** Configuration Events Console Snapshots Diagnostics

```
1 apiVersion: kubevirt.io/v1
2 kind: VirtualMachine
3 metadata:
4   annotations:
5     kubemacpool.io/transaction-timestamp: '2026-01-30T18:41:32.629136522Z'
6     kubevirt.io/latest-observed-api-version: v1
7     kubevirt.io/storage-observed-api-version: v1
8     vm.kubevirt.io/validations: |
9
10    [
11      {
12        "name": "minimal-required-memory",
13        "path": "jsonpath:..spec.domain.memory.guest",
14        "rule": "integer",
15        "message": "This VM requires more memory.",
16        "min": 2147483648
17      }
18    ]
19 resourceVersion: '1240709'
20 name: fedora-plum-angelfish-45
21 uid: 7142897f-4100-4a71-9230-89b56da92b34
22 creationTimestamp: '2026-01-30T18:41:32Z'
23 generation: 1
24 managedFields: ~
25 namespace: default
26 finalizers:
27   - kubevirt.io/virtualMachineControllerFinalize
28 labels:
29   app: fedora-plum-angelfish-45
30   kubevirt.io/dynamic-credentials-support: 'true'
31   vm.kubevirt.io/template: fedora-server-small
32   vm.kubevirt.io/template.namespace: openshift
33   vm.kubevirt.io/template.revision: '1'
34   vm.kubevirt.io/template.version: v0.34.1
35 spec:
36   dataVolumeTemplates:
37     - apiVersion: cdi.kubevirt.io/v1beta1
38       kind: DataVolume
39       metadata:
40         creationTimestamp: null
41         name: fedora-plum-angelfish-45
42       spec:
43         sourceRef:
```

Save Reload Cancel Download



**Download** `VirtualMachine` YAML

VirtualMachine VM

VirtualMachine VM

1. VirtualMachines **Create** → **From template**
- 2.
3. **Customize VirtualMachine** **Quick create VirtualMachine**
4. VM







11

- 00000000 - 00000000
- 00000000 - 00000000

# CLI VM

11

OpenShift CLI (oc)    YAML    CLI    VM  
GitOps Infrastructure as Code    Web    YAML  
CI/CD

1111

- `virtctl create vm` VM 作成
- VirtualMachine YAML (apiVersion, kind, metadata, spec)
- containerDisk VM 内 (ディスク)
- DataVolume VM 内
- CPU
- `running` `runStrategy`
- SSH `cloud-init`
- `oc apply -f`
- VM SSH VM 内

□□□□

- OpenShift Virtualization Operator [インストール](#) OpenShift 4.18+
- [インストール](#) OpenShift CLI (oc) [インストール](#) virtctl (Virtctl [インストール](#) [リソース](#))
- VM [インストール](#) [インストール](#) StorageClass
- [インストール](#) (DataSources)

[illegible]

```
oc get datasources -n openshift-virtualization-os-images
```

StorageClass

```
oc get storageclass
```

(default) `StorageClass` `storageclass.storage.k8s.io`

## VirtualMachine YAML 例

OpenShift Virtualization での VirtualMachine の Kubernetes での定義

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
  namespace: demo
spec:
  runStrategy: Always ①
  template:
    metadata:
      labels:
        kubevirt.io/domain: my-vm
    spec:
      domain:
        devices:
          disks:
            - name: rootdisk
          interfaces:
            - name: default
        resources:
          requests:
            memory: 2Gi
            cpu: 1
        networks:
          - name: default
        volumes:
          - name: rootdisk
      dataVolumeTemplates: ②
        - metadata:
            name: rootdisk-dv
```

① **runStrategy** - Always、RerunOnFailure、Manual、Halted

② **dataVolumeTemplates** DataVolume の定義



**running**: true/false **runStrategy** **runStrategy**

### 1: virtctl で VM を作成

**virtctl create vm** **YAML**

**namespace**

oc new-project demo

□ □ □ □ □ □ □ □ □ □ □ □

```
virtctl create vm data-volume data-volume
```

```
virtctl create vm --name=my-rhel-vm \
  --volume-import type:ds,src:openshift-virtualization-os-images/rhel9 \
  --memory=2Gi \
  --user=cloud-user \
  --ssh-key="$(cat ~/.ssh/id_rsa.pub)" > my-rhel-vm.yaml
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- `my-rhel-vm` VM 名前
- 使用する RHEL 9 DataSource 名前
- 2Gi (CPU instancetypes 指定)
- `cloud-user` cloud-init ユーザー
- SSH 接続
- YAML 形式 `my-rhel-vm.yaml` (virtctl YAML 形式)



```
SSH      ████████████████ssh-keygen    -t    ed25519    -C    "your.email@example.com"
████████████████████
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
cat my-rhel-vm.yaml
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
00000000000000000000000000000000virtctl          cloud-init
00000000000000000000000000000000
```

1. **cloud-init** と **SSH** をインストール
2. **CPU** をインストール
3. インストール完了
4. インストール完了

[illegible]

```
vi my-rhel-vm.yaml
```

~~~~~ nano ~~~~~

```
nano my-rhel-vm.yaml
```



AAAA…your-public-key… ~/.ssh/id\_ed25519.pub ~~~~~ SSH ~~~~~

~~~~~

~~~~~ VM ~~~~~

```
oc apply -f my-rhel-vm.yaml
```

VM ~~~~~

```
oc get vm my-rhel-vm -n demo -w
```

~~~~~ → ~~~~~ → ~~~~~ YAML ~~~~~ VM ~~~~~

## ~~~~~ 2: containerDisk ~~~~~ VM ~~~~

~~~~~ VM ~~~~~ containerDisk ~~~~~containerDisk ~~~~~ (~~~~~) ~~~~~

~~~~~



~~~~~ 1 ~~~~~ demo ~~~~~oc new-project demo  
~~~~~

~~~~~ VM ~~~~~

**vm-minimal-containerdisk.yaml** ~~~~~

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-minimal
  namespace: demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-minimal
    spec:
      domain:
        devices:
          disks:
```

```

    - name: rootdisk
      disk:
        bus: virtio
    - name: cloudinitdisk
      disk:
        bus: virtio
  interfaces:
    - name: default
      masquerade: {}
  resources:
    requests:
      memory: 1Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      containerDisk: ①
        image: quay.io/containerdisks/fedora:latest
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd:
            expire: false
            ssh_pwauth: true

```

① containerDisk `image` - `quay.io/containerdisks/fedora:latest`

コマンドを実行:

```
oc apply -f vm-minimal-containerdisk.yaml
```

**VM** の確認

VM の確認コマンド:

```
oc get vm vm-minimal -n demo
```

結果:

| NAME       | AGE | STATUS  | READY |
|------------|-----|---------|-------|
| vm-minimal | 30s | Running | True  |

VirtualMachineInstance (VM) の確認:

```
oc get vmi vm-minimal -n demo
```

결과:

| NAME       | AGE | PHASE   | IP           | NODENAME      | READY |
|------------|-----|---------|--------------|---------------|-------|
| vm-minimal | 1m  | Running | 10.128.1.123 | worker-node-2 | True  |

이 VM의 containerDisk는 DataVolume로 연결되어 있으며, pull 상태를 유지하고 있습니다.

**VM** 상태를 확인합니다.

VM의 상태를 확인합니다.

```
virtctl console vm-minimal -n demo
```

cloud-init 실행을 기다립니다.

- 사용자: **fedora**
- 사용자: **fedora123**

종료하려면 **Ctrl+]** 키를 누릅니다.



ContainerDisk  
상태 확인

VM

이 VM은 ContainerDisk를 사용하여 실행되고 있습니다.

### 예제 3: DataVolume로 VM 생성

이 예제에서는 DataVolume을 사용하여 VM을 생성하고, VM이 DataVolume을 사용하여 실행되는 것을 확인합니다.

**vm-datavolume.yaml** 파일 내용:

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-datavolume
  namespace: demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-datavolume
    spec:
      domain:
        devices:
```



```

    disks:
      - name: rootdisk
        disk:
          bus: virtio
      - name: cloudinitdisk
        disk:
          bus: virtio
    interfaces:
      - name: default
        masquerade: {}
    resources:
      requests:
        memory: 2Gi
        cpu: 1
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        dataVolume: ①
          name: vm-datavolume
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            ssh_pwauth: true
            user:
              name: fedora
              lock_passwd: false
              plain_text_passwd: fedora123
              ssh_authorized_keys:
                - 'ssh-ed25519 AAAA...your-public-key... user@workstation'
          packages:
            - vim
            - git
          runcmd:
            - echo "VM setup complete" > /tmp/vm-ready
dataVolumeTemplates: ②
  - metadata:
      name: vm-datavolume
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
        resources:
          requests:
            storage: 30Gi

```

- ① containerDisk 指定 DataVolume 指定
- ② DataVolume 指定 Fedora DataSource 指定



AAA...your-public-key... ~/.ssh/id\_ed25519.pub 指定 SSH 指定

指定:

```
oc apply -f vm-datavolume.yaml
```

## DataVolume 指定

DataVolume VM 指定

```
oc get dv vm-datavolume -n demo -w
```

指定:

| NAME          | PHASE           | PROGRESS | RESTARTS | AGE |
|---------------|-----------------|----------|----------|-----|
| vm-datavolume | Pending         |          |          | 5s  |
| vm-datavolume | CloneInProgress | 0.00%    |          | 10s |
| vm-datavolume | CloneInProgress | 45.2%    |          | 30s |
| vm-datavolume | Succeeded       |          |          | 45s |

DataVolume 指定 Succeeded 指定 Ctrl+C 指定

VM 指定

```
oc get vm vm-datavolume -n demo
```

## DataVolume VM 指定 SSH 指定

VM 指定 SSH 指定

```
virtctl ssh fedora@vm-datavolume -n demo
```



SSH 指定 YAML 指定 SSH 指定

## 指定 4: CPU 指定 cloud-init 指定

指定 CPU/指定 cloud-init 指定 VM 指定

vm-customized.yaml 指定

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-customized
  namespace: demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-customized
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        machine:
          type: pc-q35-rhel9.4.0
        memory:
          guest: 4Gi
        resources:
          requests:
            memory: 4Gi
            cpu: 2
        networks:
          - name: default
            pod: {}
        volumes:
          - name: rootdisk
            dataVolume:
              name: vm-customized
          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |
                #cloud-config
                ssh_pwauth: true
              users:
                - name: fedora
                  lock_passwd: false
                  plain_text_passwd: s3cur3p4ss

```

①

②

③

```

      ssh_authorized_keys:
        - 'ssh-ed25519 AAAA...your-public-key... user@workstation'
    - name: admin
      groups: wheel
      sudo: ALL=(ALL) NOPASSWD:ALL
      ssh_authorized_keys:
        - 'ssh-ed25519 AAAA...your-public-key... user@workstation'
  packages:
    - vim
    - git
    - tmux
    - htop
  write_files:
    - path: /etc/motd
      content: |
        Welcome to your customized VM!
        Created via YAML manifest and oc apply.
      permissions: '0644'
  runcmd:
    - systemctl enable --now cockpit.socket
    - echo "VM initialization complete at $(date)" > /var/log/vm-init.log
dataVolumeTemplates:
- metadata:
    name: vm-customized
  spec:
    sourceRef:
      kind: DataSource
      name: fedora
      namespace: openshift-virtualization-os-images
    storage:
      resources:
        requests:
          storage: 50Gi

```

- ① CPU 2
- ② (resources.requests.memory 1Gi)
- ③ sudo



pc-q35-rhel9.4.0 1Gi  
get VirtualMachineClusterInstancetypes

VM

```
oc apply -f vm-customized.yaml
```

VM CPU

```
oc describe vmi vm-customized -n demo | grep -A 5 "Requests"
```

VM `cloud-init` `message of the day`

```
virtctl ssh admin@vm-customized -n demo
```

`cloud-init` `message of the day`

```
cat /etc/motd
```

`tmux`

```
which tmux
```

`htop`

```
which htop
```

```
cat /var/log/vm-init.log
```

**5: VM**

**VM**

**VM**

```
oc describe vm vm-customized -n demo
```

**VM**

```
oc get events -n demo --field-selector involvedObject.name=vm-customized
```

**VM**

**VM** (

```
virtctl stop vm-customized -n demo
```

コマンドを実行:

```
oc get vm vm-customized -n demo
```

VM を起動:

```
virtctl start vm-customized -n demo
```

**VM を再起動**

コマンドを実行:

```
virtctl restart vm-customized -n demo
```

**VM を更新**

以下のコマンドを実行して VM のメモリを 6Gi に更新します。

```
oc patch vm vm-customized -n demo --type merge --patch
'{"spec":{"template":{"spec":{"domain":{"memory":{"guest":"6Gi"},"resources":{"request
s":{"memory":"6Gi"}}}}}}}'
```



更新された CPU とメモリは VM の再起動後に適用されます。

確認コマンド

**VM を確認**

VirtualMachine を確認

```
oc describe vm <vm-name> -n demo
```

VirtualMachineInstance を確認

```
oc describe vmi <vm-name> -n demo
```

確認:

- DataVolume を確認 (oc get dv)
- PVC を確認 (pvc)
- cloud-init を確認 (Pod)

## DataVolume

DataVolume

```
oc describe dv <datavolume-name> -n demo
```

CDI Pod

```
oc logs -l app=containerized-data-importer -n openshift-cnv
```

## Cloud-init

VM cloud-init

```
virtctl console <vm-name> -n demo
```

VM cloud-init **Ctrl+]**

VM cloud-init

```
cloud-init status --long
```

cloud-init

```
cat /var/log/cloud-init.log
```

cloud-init

VM

```
oc delete vm vm-minimal vm-datavolume vm-customized -n demo
```

DataVolume

```
oc delete project demo
```



VM

100

□□□□□□□□□□□□□□□□

- `DataSource` `virtctl create vm` `VM`
- ( )
- `apiVersion` `metadata` `spec` `VirtualMachine` `YAML`
- `containerDisk` `VM`
- `DataVolume` `VM`
- `CPU`
- `running` `VM` `runStrategy`
- `cloud-init` `SSH`
- `oc apply -f` `oc get vm` `oc get vmi` `oc get dv`
- `virtctl console` `virtctl ssh` `VM`
- (`stop` `start` `restart`)
- 

11

- **cloud-init**  - cloud-init 
-  IP  - VM  IP 
-  **Affinity** - VM 
-  **VM** - VM 
-  - 
- **KubeVirt** 
- **Cloud-init** 

**VM** □□□□□□□□

11

```

OpenShift           Virtualization           VM
VirtualMachine      VirtualMachineInstance  VM
VM                   VM

```

[illegible]

OpenShift 4.20



□ □ □ □

- OpenShift Virtualization **OpenShift 4.18**
- OpenShift Web **CLI** (**oc****virtctl**)
- **VM** **1** (**Web** **VM**)

## VirtualMachine ◻ VirtualMachineInstance ◻◻◻

## OpenShift Virtualization 2

## VirtualMachine (VM)

**VirtualMachine** VM

- VM namespace
- CPU namespace
- Memory namespace
- Network namespace
- PID (VM process)

VirtualMachine 虚拟机管理程序

```
# oc get vm -n my-vms
```

□□□:

| NAME          | AGE | STATUS  | READY |
|---------------|-----|---------|-------|
| fedora-app-vm | 14m | Running | True  |

## VirtualMachineInstance (VMI)

[illegible]

- 管理IP (IP management)
- 管理/agent
- guest agent (management)

```
# VirtualMachineInstance
oc get vmi -n my-vms
```

□□□:

| NAME          | AGE | PHASE   | IP           | NODENAME | READY |
|---------------|-----|---------|--------------|----------|-------|
| fedora-app-vm | 78s | Running | 10.128.0.184 | ocplab01 | True  |



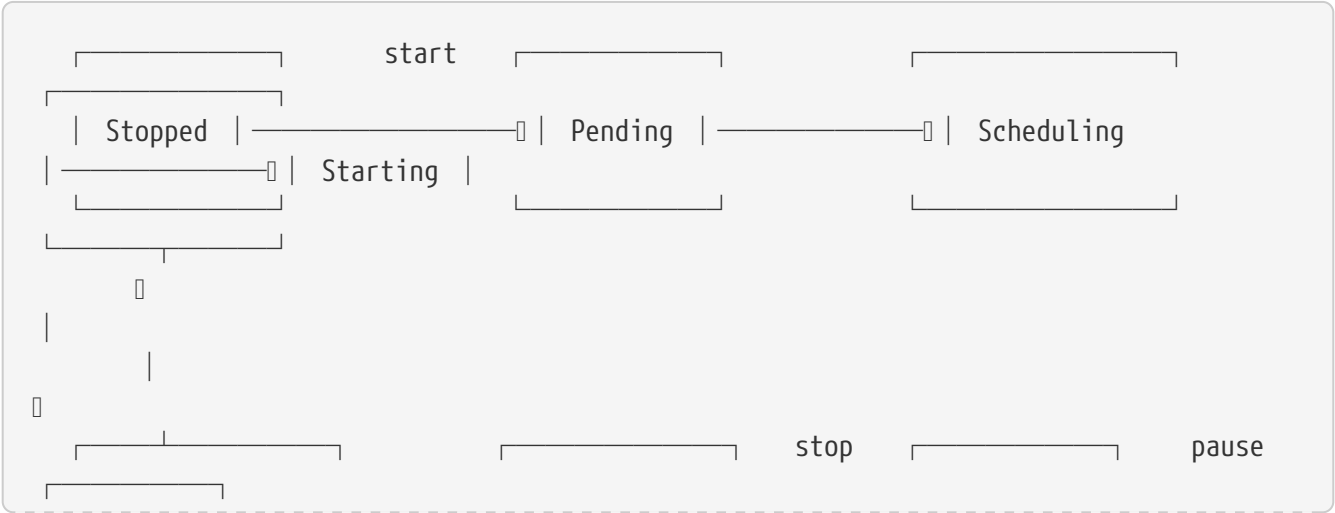
VM VirtualMachine VirtualMachineInstance VM  
VMI VM VMI VM

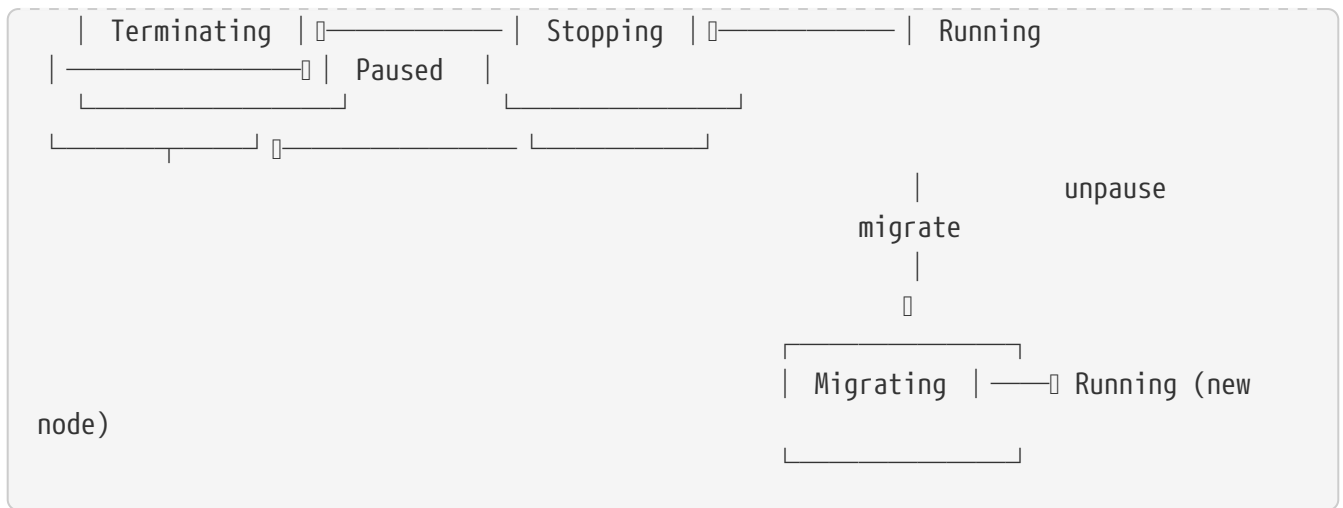
VM

VM

|   |                 |            | VMI |
|---|-----------------|------------|-----|
| 1 | Stopped ( )     | VM         |     |
| 2 | Pending ( )     | VMI ( )    |     |
| 3 | Scheduling ( )  | Kubernetes |     |
| 4 | Starting ( )    | VM         |     |
| 5 | Running ( )     | VM         |     |
| 6 | Paused ( )      | VM         |     |
| 7 | Stopping ( )    |            |     |
| 8 | Terminating ( ) | VMI        | ( ) |

VM





- **Start** VM → Pending → Scheduling → Starting → Running
- **Stop** VM → Stopping → Terminating → Stopped
- **Restart** VM → Stopped → Starting → Running
- **Pause/Unpause** VM → Paused → Running
- **Migrate** VM → Running → Migrating → Running (new)
- **Delete** VM → Terminating → Stopped

VM lifecycle states and transitions

VM lifecycle states and transitions

- **Node drain** — cordon VM → Terminating → Stopped
- **OpenShift** — OpenShift Worker Node 1 VM → Terminating → Stopped
- **descheduler** — descheduler VM → Terminating → Stopped

VM lifecycle states and transitions (ReadWriteMany PVC) post-copy

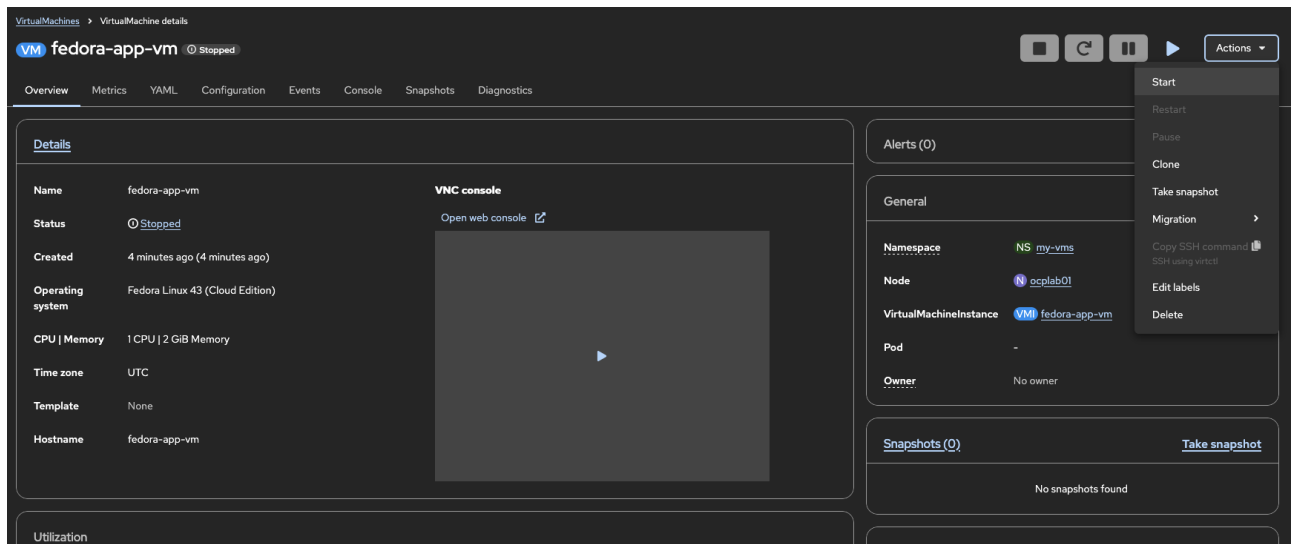
## VM lifecycle states and transitions

### VM lifecycle

VM lifecycle states and transitions

### Web console:

1. **Virtualization** → **VirtualMachines**
2. VM lifecycle states and transitions
3. VM lifecycle states and transitions **Actions** → **Start**
4. VM lifecycle states and transitions (3 VM) **Start**



## CLI:

```
virtctl start fedora-app-vm -n my-vms
```

## VM 操作:

```
# VM 操作
oc get vm fedora-app-vm -n my-vms

# VMI 操作
oc get vmi fedora-app-vm -n my-vms
```

## 確認:

| NAME          | AGE | STATUS  | READY |
|---------------|-----|---------|-------|
| fedora-app-vm | 14m | Running | True  |

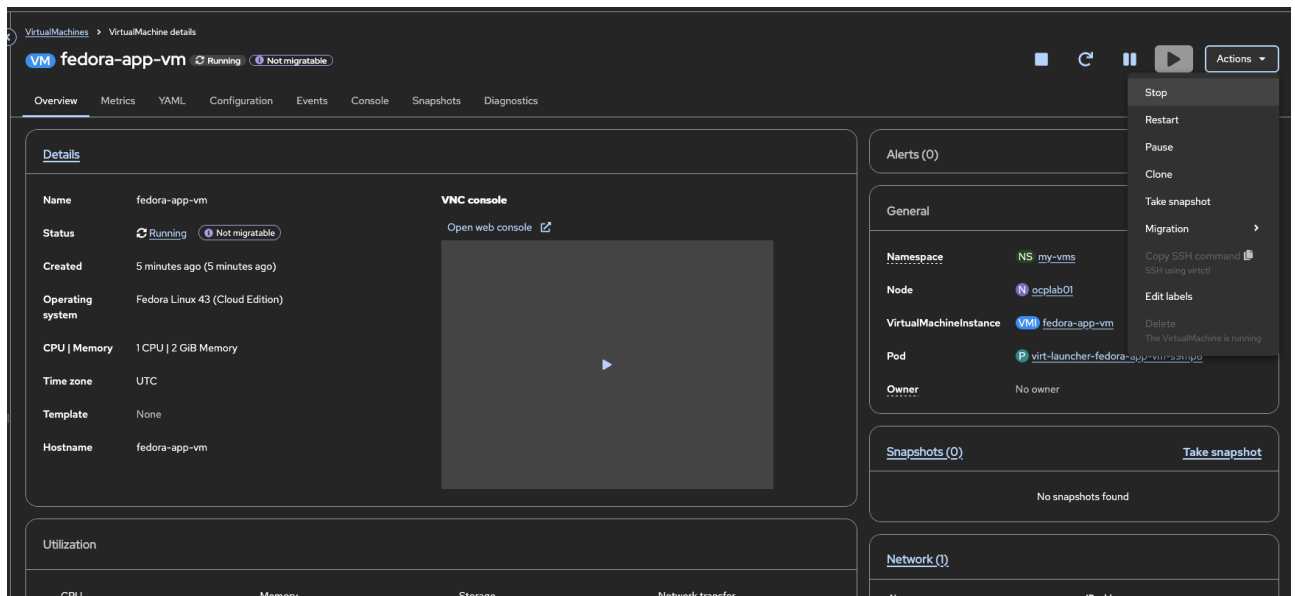
| NAME          | AGE | PHASE   | IP           | NODENAME | READY |
|---------------|-----|---------|--------------|----------|-------|
| fedora-app-vm | 78s | Running | 10.128.0.184 | ocplab01 | True  |

## VM 停止

VM 操作 VMI 操作

## Web 操作:

1. VM 操作
2. **Actions** → **Stop** 操作



CLI (ターミナル):

```
virtctl stop fedora-app-vm -n my-vms
```

CLI (ターミナル):

VM 強制終了

```
virtctl stop fedora-app-vm -n my-vms --force
```



仮想マシンが強制終了されました。

VM 確認:

```
# VM 確認
oc get vm fedora-app-vm -n my-vms

# VMI 確認
oc get vmi fedora-app-vm -n my-vms
```

結果:

```
NAME          AGE   STATUS   READY
fedora-app-vm 14m   Stopped  False
```

```
Error from server (NotFound): virtualmachineinstances.kubevirt.io "fedora-app-vm" not found
```

## VM の起動と停止

VM (Pause) の VM の起動と停止の方法を確認します。

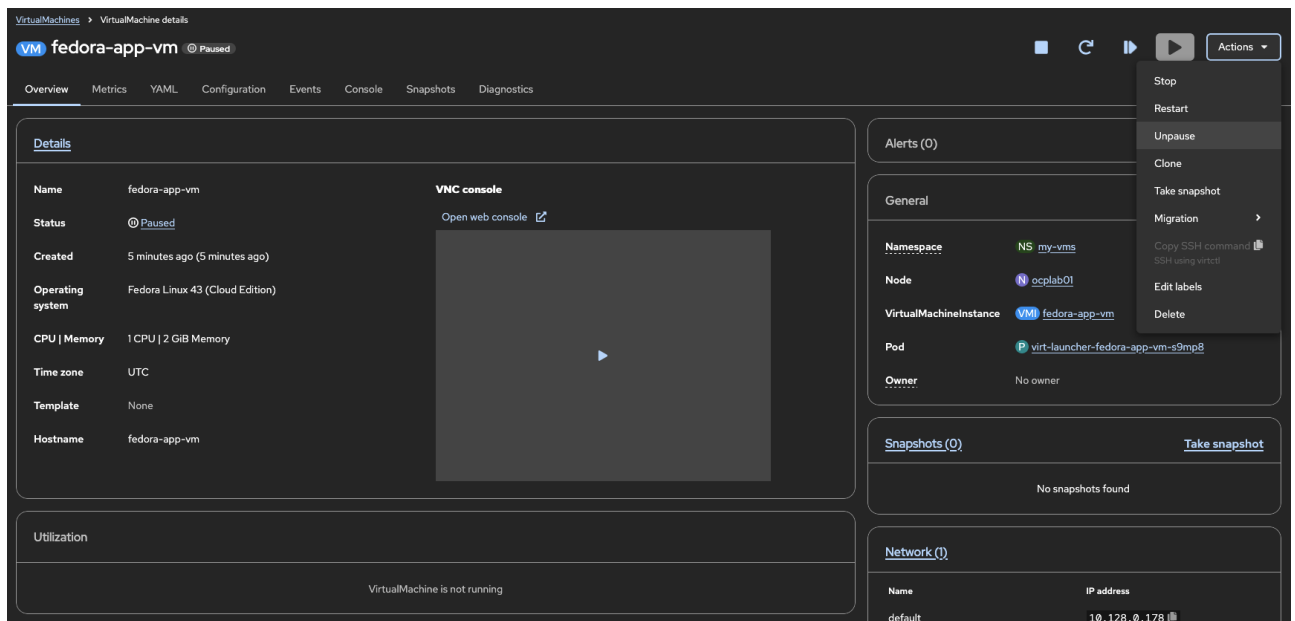
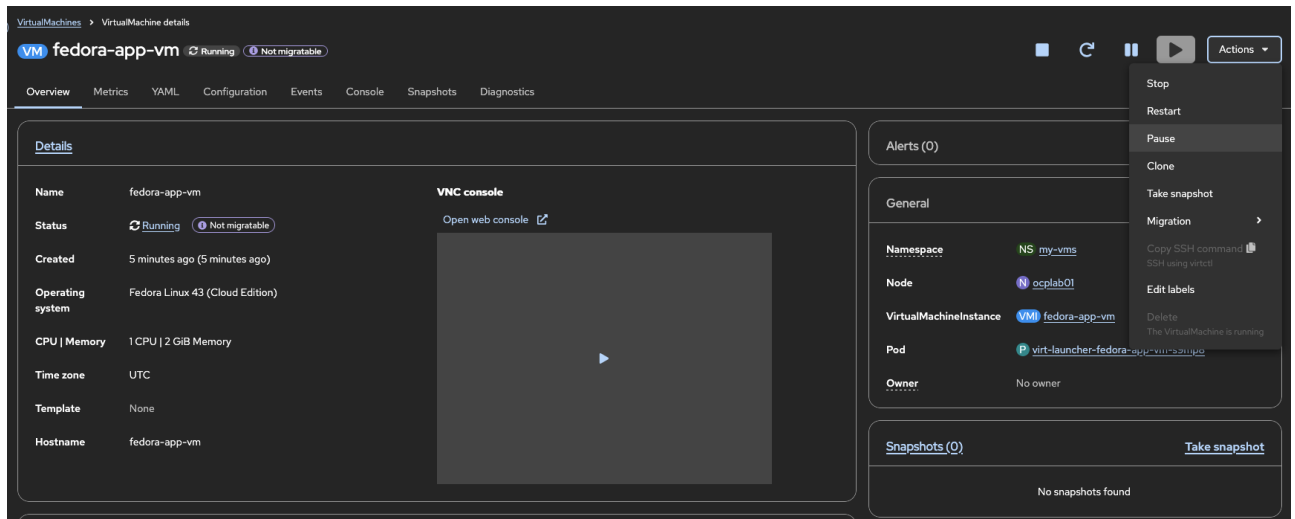
- CPU の使用率を確認
- VM の起動と停止の方法を確認
- VM の削除



VM の起動と停止の方法を確認します。

## Web 操作:

1. VM の起動と停止
2. **Actions** → **Pause** の操作
3. VM の起動: **Actions** → **Unpause** の操作



## CLI:

```
# VM 一時停止
virtctl pause vm fedora-app-vm -n my-vms

# VM 一時停止解除
virtctl unpause vm fedora-app-vm -n my-vms
```

確認コマンド:

```
oc get vm fedora-app-vm -n my-vms
```

結果:

| NAME          | AGE | STATUS | READY |
|---------------|-----|--------|-------|
| fedora-app-vm | 14m | Paused | False |

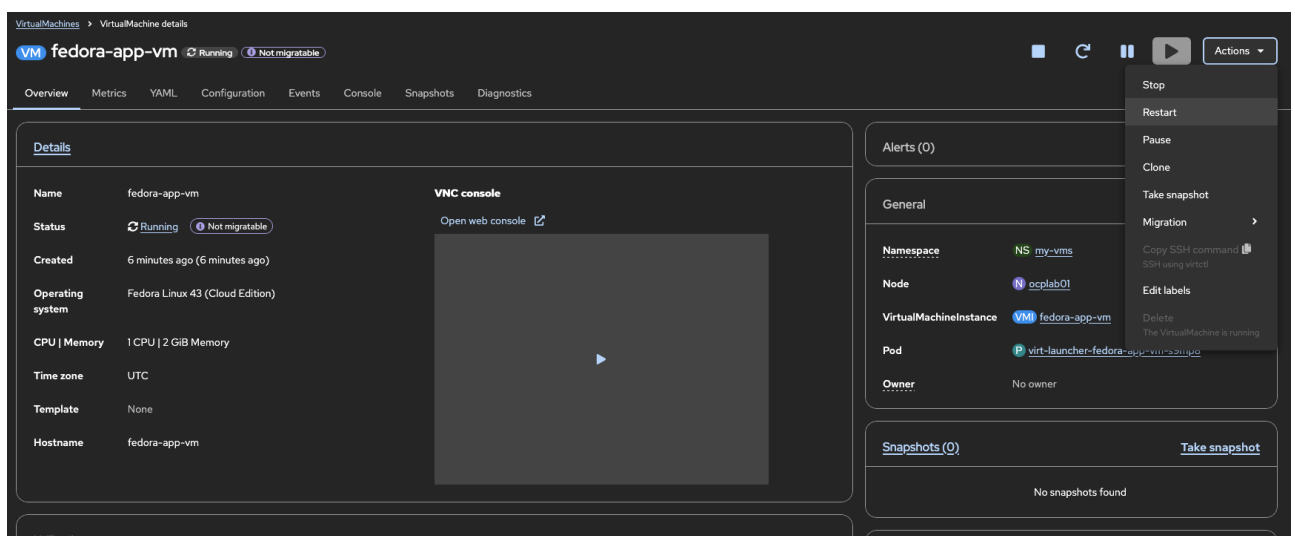
## VM 再起動

再起動 (Restart) コマンドを実行すると、以下の操作が行われます。

- 仮想マシンを一時停止
- 仮想 OS を再起動
- VM を再起動

## Web 操作:

1. VM を一時停止
2. **Actions** → **Restart** をクリック



## CLI:

```
virtctl restart fedora-app-vm -n my-vms
```



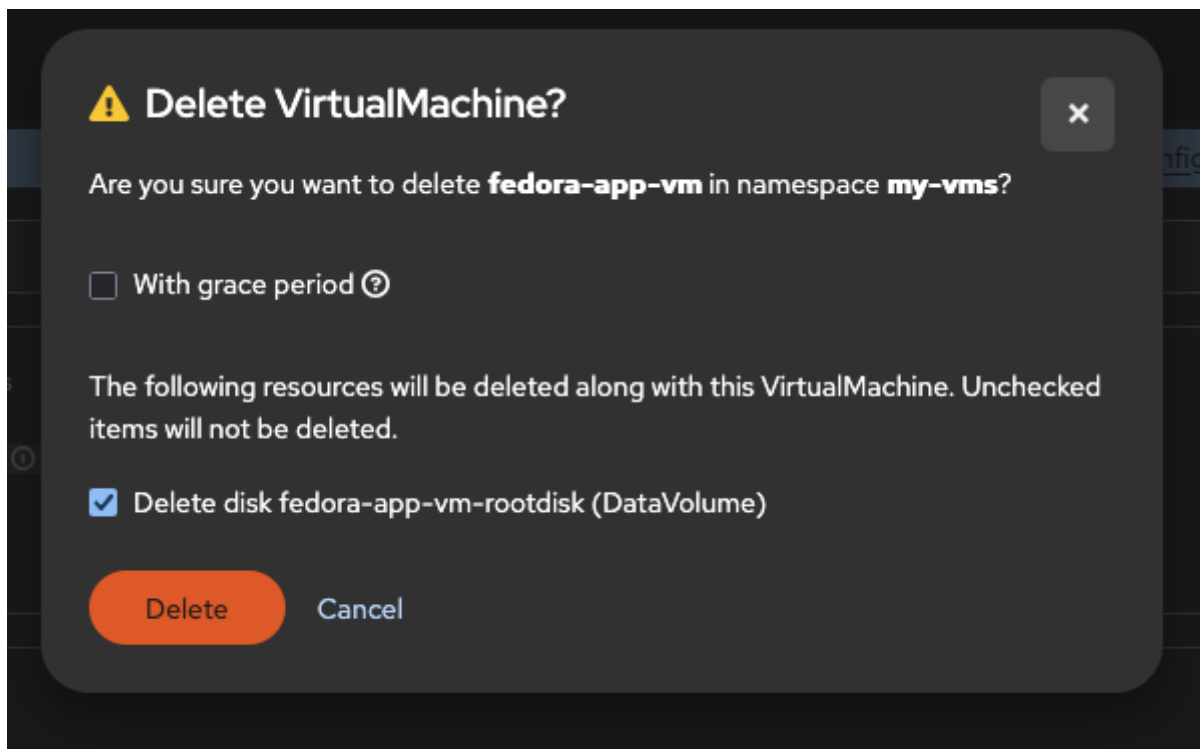
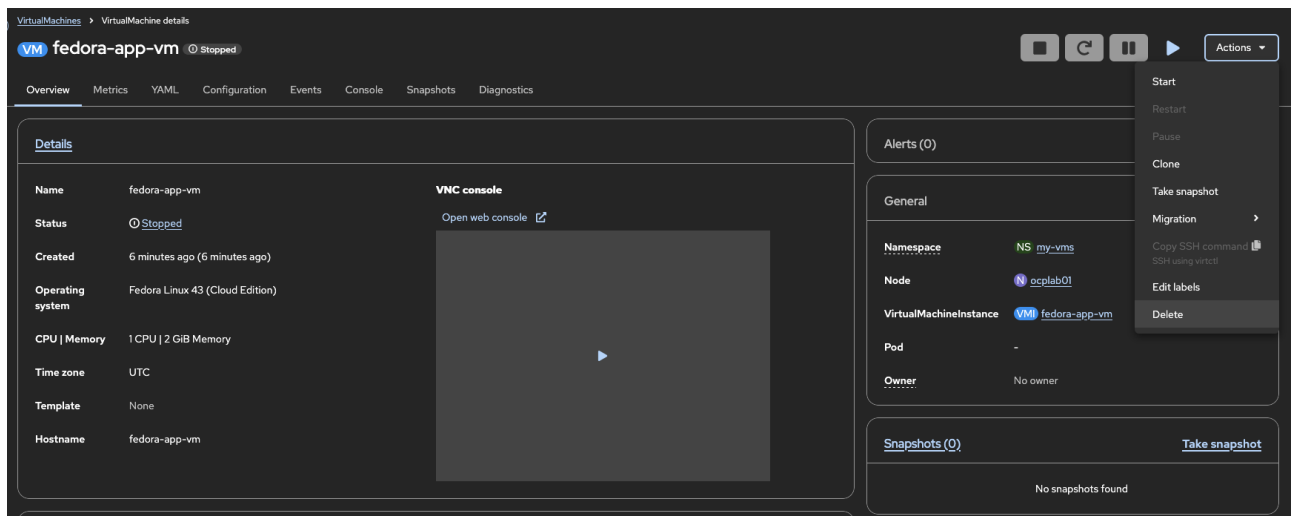
このチュートリアルでは、Fedora Linux 43 (Cloud Edition) OS の VirtualMachine を作成、管理、削除する方法について説明します。

## VM の削除

VM の削除には、PVC/DataVolume の削除も必要です。

### Web 操作:

1. VM の詳細ページを開く
2. **Actions** → **Delete** をクリックする
3. 削除の確認ダイアログが表示されます。
4. **Delete** をクリックする



### CLI (VM の削除):



```
oc delete vm fedora-app-vm -n my-vms
```

VM DataVolume Web

:

```
# VM
oc get vm fedora-app-vm -n my-vms

# PVC (PVC)
oc get pvc -n my-vms
```

:

Error from server (NotFound): virtualmachines.kubevirt.io "fedora-app-vm" not found  
No resources found in my-vms namespace.



VM

**VM**

**VM** **VMI**

**VM**:

```
oc describe vm fedora-app-vm -n my-vms
```

():

```
Name:          fedora-app-vm
Namespace:     my-vms
Labels:        app=fedora-app-vm
...
Status:
  Conditions:
    Last Transition Time:  2026-03-05T17:10:11Z
    Status:               True
    Type:                 Ready
    Message:              All of the VMI's DVs are bound and ready
    Reason:               AllDVsReady
    Status:               True
    Type:                 DataVolumesReady
    Status:               True
    Type:                 AgentConnected
```

```
Printable Status:    Running
Ready:              true
Run Strategy:       Manual
```

□□□□□□□□□□:

- **Status.Conditions** - □□□□□□□□□□
- **Status.PrintableStatus** - □□□□□□□□□□□□
- **Status.Ready** - VM □□□□□□□□□□

□□□ VMI □□:

```
oc describe vmi fedora-app-vm -n my-vms
```

□□□ (□□□□□□□□):

```
Name:      fedora-app-vm
Namespace:  my-vms
...
Status:
  Guest OS Info:
    Id:      fedora
    Kernel Release: 6.17.1-300.fc43.x86_64
    Name:      Fedora Linux
    Pretty Name:  Fedora Linux 43 (Cloud Edition)
    Version Id:  43
  Interfaces:
    Interface Name:  enp1s0
    Ip Address:      10.128.0.184
    Mac:             02:7e:e2:0a:7c:df
    Name:            default
  Node Name:        ocplab01
  Phase:            Running
  Phase Transition Timestamps:
    Phase: Pending
    Phase: Scheduling
    Phase: Scheduled
    Phase: Running
```

□□□□□□□□:

- **Status.Phase** - □□□□□□□□□□□□
- **Status.Interfaces** - IP □□□□□□□□□□□□□□
- **Status.NodeName** - VM □□□□□□□□□□
- **Status.GuestOSInfo** - □□□ OS □□□ (guest agent □□□)

条件 (Conditions)

VM 条件 (Conditions) 检查虚拟机状态

```
oc get vm fedora-app-vm -n my-vms -o jsonpath='{.status.conditions}' | jq
```

输出:

```
[
  {
    "lastTransitionTime": "2026-03-05T17:10:11Z",
    "status": "True",
    "type": "Ready"
  },
  {
    "message": "All of the VMI's DVs are bound and ready",
    "reason": "AllDVsReady",
    "status": "True",
    "type": "DataVolumesReady"
  },
  {
    "message": "cannot migrate VMI: PVC fedora-app-vm-rootdisk is not shared...",
    "reason": "DisksNotLiveMigratable",
    "status": "False",
    "type": "LiveMigratable"
  },
  {
    "status": "True",
    "type": "AgentConnected"
  }
]
```

条件:

| 条件             | 说明                  |
|----------------|---------------------|
| Ready          | VM 正常运行             |
| Paused         | VM 暂停               |
| AgentConnected | QEMU guest agent 连接 |
| Synchronized   | VM 与 VMI 同步         |

VM 检查

检查 VM 是否正常运行

```
# 检查 VM 是否运行
```

```
oc get events -n my-vms --field-selector involvedObject.name=fedora-app-vm
```

👉:

| LAST SEEN                                                                              | TYPE   | REASON                     | OBJECT                               |
|----------------------------------------------------------------------------------------|--------|----------------------------|--------------------------------------|
| MESSAGE                                                                                |        |                            |                                      |
| 14m                                                                                    | Normal | SuccessfulDataVolumeCreate | virtualmachine/fedora-app-vm         |
| Created DataVolume fedora-app-vm-rootdisk                                              |        |                            |                                      |
| 90s                                                                                    | Normal | SuccessfulCreate           | virtualmachine/fedora-app-vm         |
| Started the virtual machine by creating the new virtual machine instance fedora-app-vm |        |                            |                                      |
| 90s                                                                                    | Normal | SuccessfulCreate           | virtualmachineinstance/fedora-app-vm |
| Created virtual machine pod virt-launcher-fedora-app-vm-fktmb                          |        |                            |                                      |
| 84s                                                                                    | Normal | Created                    | virtualmachineinstance/fedora-app-vm |
| VirtualMachineInstance defined.                                                        |        |                            |                                      |
| 84s                                                                                    | Normal | Started                    | virtualmachineinstance/fedora-app-vm |
| VirtualMachineInstance started.                                                        |        |                            |                                      |

👉👉👉👉👉👉👉👉:

```
oc get events -n my-vms -w
```

👉👉👉👉👉👉👉👉:

| 👉👉👉              | 👉👉              |
|------------------|-----------------|
| SuccessfulCreate | VMI 👉👉👉👉        |
| Started          | VM 👉👉👉👉         |
| Stopped          | VM 👉👉 (👉👉👉)     |
| Killing          | 👉👉👉👉👉           |
| FailedScheduling | 👉👉👉👉👉👉          |
| FailedCreate     | VMI 👉👉👉👉 (👉👉👉👉) |

👉👉👉 (Run Strategies)

👉👉👉👉👉👉👉👉👉👉👉 VM 👉👉👉👉👉👉👉👉👉👉👉👉👉 VM 👉👉👉👉👉👉👉👉👉 VM  
👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉👉

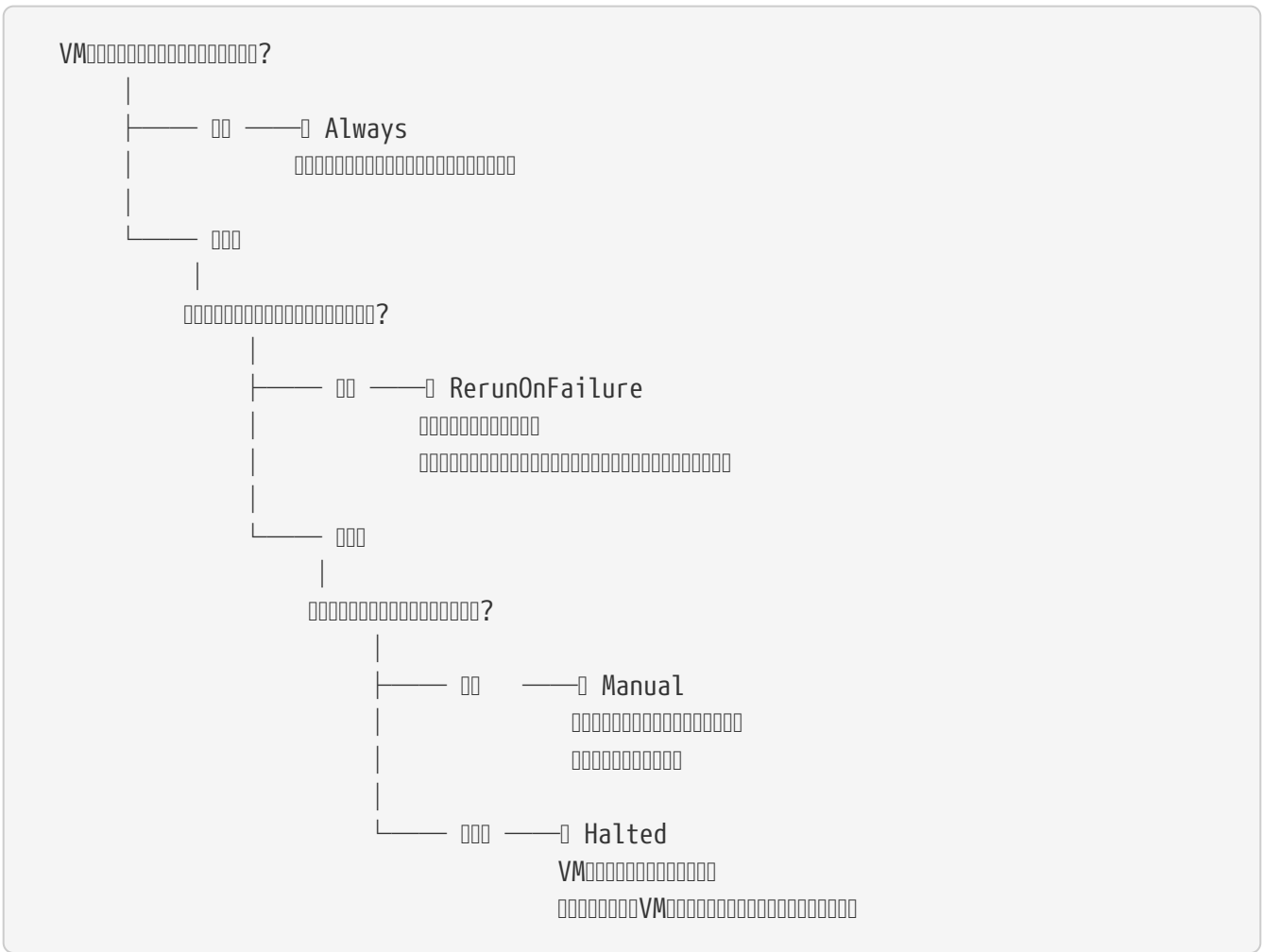
👉👉👉👉👉👉

| 👉👉     | 👉👉                                          | 👉👉👉👉👉         |
|--------|---------------------------------------------|---------------|
| Always | VM 👉👉👉👉👉👉👉👉👉<br>(👉👉👉👉👉👉👉👉👉👉👉👉👉)<br>👉👉👉👉👉👉👉👉 | 👉👉👉👉👉👉👉👉👉👉👉👉👉 |

| VM                    | VM                   | VM       |
|-----------------------|----------------------|----------|
| <b>RerunOnFailure</b> | VM<br>VM<br>VM       | VM       |
| <b>Manual</b>         | virtctl VM Web<br>VM | VM/VM VM |
| <b>Halted</b>         | VM VM                | VM VM    |

VM

VM



VM

VM YAML:

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  runStrategy: Always # VM RerunOnFailureManualHalted

```

```
template:  
  # ... VM spec
```

### Web 操作:

1. VM 操作
2. **YAML** 操作
3. `spec.runStrategy` 操作
4. **Save** 操作

VM

fedora-app-vm

Stopped

Overview

Metrics

YAML

Configuration

Events

Console

Snapshots

Diagnostics

```

3  metadata:
94  labels:
96  spec:
97    dataVolumeTemplates:
98      - metadata:
99          creationTimestamp: null
100         name: fedora-app-vm-rootdisk
101       spec:
102         sourceRef:
103           kind: DataSource
104           name: fedora
105           namespace: openshift-virtualization-os-images
106         storage:
107           resources:
108             requests:
109               storage: 30Gi
110       runStrategy: Manual
111     template:
112       metadata:
113         creationTimestamp: null
114         labels:
115           kubevirt.io/domain: fedora-app-vm
116       spec:
117         architecture: amd64
118         domain:
119           cpu:
120             cores: 1
121             sockets: 1
122             threads: 1
123         devices:
124           disks:
125             - disk:
126                 bus: virtio
127                 name: rootdisk
128             - disk:
129                 bus: virtio
130                 name: cloudinitdisk
131           interfaces:
132             - macAddress: '02:7e:e2:0a:7c:df'
133               masquerade: {}
134               name: default
135         firmware:

```

Save

Reload

Cancel


runStrategy を running に変更して VM を runStrategy を

Always RerunOnFailure

- `virtctl stop` VM

- VM `shutdown` 操作は `Manual` 操作で実行する必要があります
- `shutdown` 操作 (例: VM `shutdown -h now`) は `Manual` 操作で実行する必要があります

操作

操作の概要

- `VirtualMachine` オブジェクトは `VirtualMachineInstance` オブジェクトで管理されます
- VM `shutdown` 操作 (例: `shutdown`)
- VM `shutdown` 操作は `Manual` 操作で実行する必要があります
- Node drain 操作は `Manual` 操作で実行する必要があります VM `shutdown` 操作
- 条件 (Conditions) オブジェクト (Events) オブジェクト VM `shutdown` 操作
- `Manual` 操作 VM `shutdown` 操作

操作

- [Web 操作](#) VM `shutdown` - Web 操作 VM `shutdown`
- [Virtctl 操作](#) - VM `shutdown` CLI 操作
- [操作](#) - [操作](#)

## Virtctl 操作

操作

`virtctl` CLI 操作は OpenShift Virtualization オブジェクト (VM) を管理するために使用されます VM `shutdown` 操作

操作の概要:

OpenShift 4.20

操作

- OpenShift Virtualization オブジェクトは OpenShift 4.18 操作
- `virtctl` CLI 操作

**virtctl** 操作

`virtctl` CLI 操作は OpenShift Web 操作

1. OpenShift Web 操作 **Virtualization** → **Overview** 操作



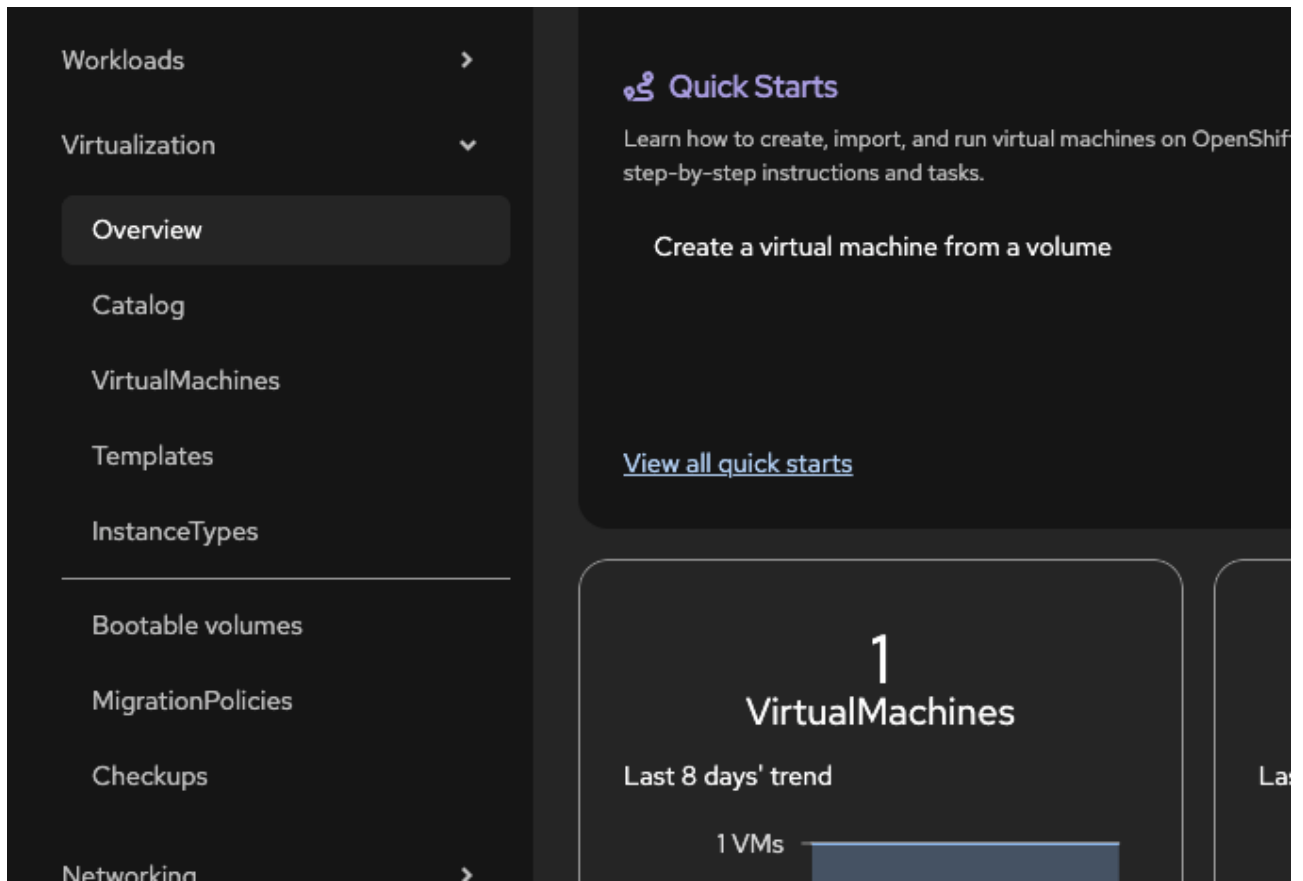


Figure 1. OpenShift Virtualization Overview

2. Overview Download the virtctl command-line utility

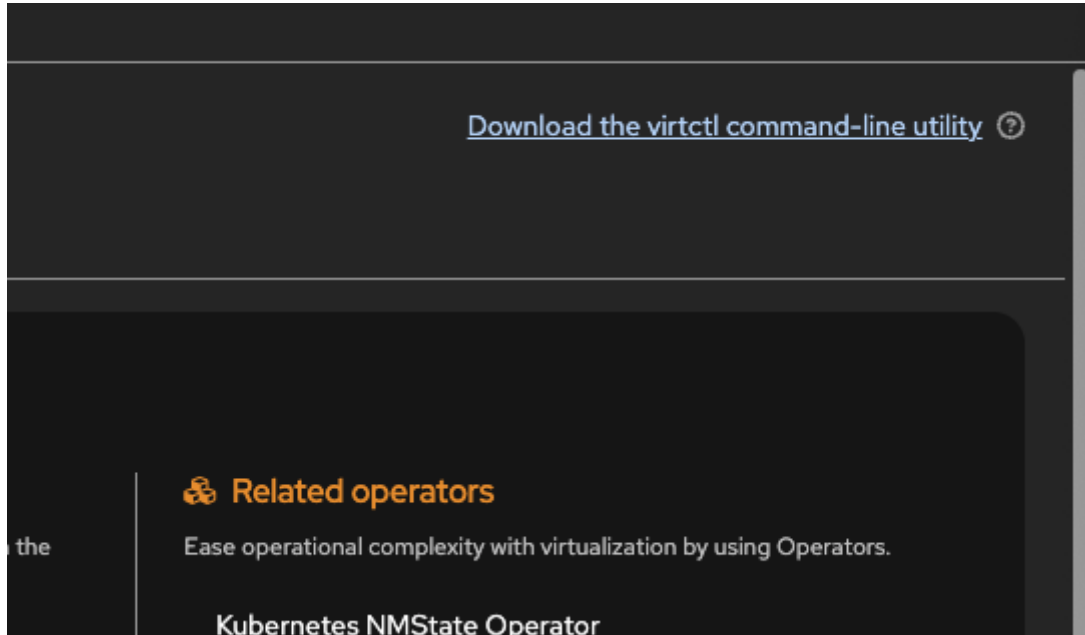


Figure 2. virtctl

3. virtctl

## virtctl – KubeVirt command line interface

The virtctl client is a supplemental command-line utility for managing virtualization resources from the command line.

- [Download virtctl for Linux for ARM 64](#)
- [Download virtctl for Linux for IBM Z](#)
- [Download virtctl for Linux for x86\\_64](#)
- [Download virtctl for Mac for ARM 64](#)
- [Download virtctl for Mac for x86\\_64](#)
- [Download virtctl for Windows for ARM 64](#)
- [Download virtctl for Windows for x86\\_64](#)

Figure 3. virtctl 安装

#### 4. 安装 virtctl 到 PATH

```
# Linux/Mac 安装
chmod +x virtctl
sudo mv virtctl /usr/local/bin/

# 验证安装
virtctl version
```

Windows 安装 virtctl 到 PATH

安装完成

#### VM 安装

VM 安装 oc 安装 VM 安装

```
# 安装 namespace 安装 VM 安装
oc get vm -n myapp

# 安装 namespace 安装 VM 安装
oc get vm -A
```

输出:

| NAMESPACE | NAME          | AGE | STATUS  | READY |
|-----------|---------------|-----|---------|-------|
| myapp     | fedora-app-vm | 13d | Running | True  |

IP 安装 VM (VMI) 安装

```
oc get vmi -A
```



```

"os": {
  "name": "Fedora Linux",
  "kernelRelease": "6.5.6-300.fc39.x86_64",
  "version": "39",
  "prettyName": "Fedora Linux 39 (Cloud Edition)",
  "versionId": "39",
  "kernelVersion": "#1 SMP PREEMPT_DYNAMIC",
  "machine": "x86_64",
  "id": "fedora"
}
}

```

```

# 列出虚拟机中的文件系统
virtctl fslist fedora-app-vm -n myapp

```

输出 (输出结果可能因版本而异):

```

{
  "items": [
    {
      "diskName": "vda2",
      "mountPoint": "/boot/efi",
      "fileSystemType": "vfat",
      "usedBytes": 21723136,
      "totalBytes": 104607744
    },
    {
      "diskName": "vda3",
      "mountPoint": "/boot",
      "fileSystemType": "ext4",
      "usedBytes": 91316224,
      "totalBytes": 1902895104
    },
    {
      "diskName": "vda4",
      "mountPoint": "/",
      "fileSystemType": "btrfs",
      "usedBytes": 687783936,
      "totalBytes": 29525868544
    }
  ]
}

```

```

# 列出虚拟机中的用户
virtctl userlist fedora-app-vm -n myapp

```

□□□□□□

**virtctl scp** □□□□ VM □□□□□□□□□□□□□□□□ VM □ SSH □□□□□□□□□□□□□□□□□□□□□□□□

- **SSH** □ (□□): □□□□ VM □ **~/.ssh/authorized\_keys** □□□□□□
- □□□□: VM □ SSH □□□□□□□□□□□□□□□□

```
# VM □□□□ (□□□□□□ SSH □□□□)
virtctl scp local-file.txt fedora@fedora-app-vm:/tmp/remote.txt -n myapp

# VM □□□□
virtctl scp fedora@fedora-app-vm:/tmp/remote.txt ./local.txt -n myapp

# □□ SSH □□□□
virtctl scp -i ~/.ssh/my_key local-file.txt fedora@fedora-app-vm:/tmp/ -n myapp
```



SSH □□□□□□□□□□□□ **virtctl console** □□□□ VM □□□□□□SSH □□□□□□□□□□

VM □□□□ SSH □□□□□□□□cloud-init □□□□□□□□

```
#cloud-config
user: fedora
ssh_authorized_keys:
  - ssh-rsa AAAAB3NzaC... your-key-here
```

□□□ SSH □□□□□□□□□□□□**Linux** □□□□□□□□□□□□□□□□ □□□□□□□□□□

## VM □□□□□

VM □□□□□□□□□□□□□□□□□□□□

```
# VM □□□□□□□□
oc get vm fedora-app-vm -n myapp
```

□□□□□□□□□:

| NAME          | AGE  | STATUS  | READY |
|---------------|------|---------|-------|
| fedora-app-vm | 7d3h | Running | True  |

```
# VMI (□□□□□□□□□□□□) □□□
oc get vmi fedora-app-vm -n myapp
```

□□□□□□□□: □□□□□□□□ VMI □□□□□□□□□□□□□□ VMI □□□□□□□□□□



## 名前空間

- 名前空間は `-n` オプションで **namespace** 指定する
- 名前空間を指定して SSH 接続する
- VM には **guest agent** をインストールする
- VM には `oc` を VM 内で `virtctl` としてインストールする
- 名前空間を指定して `Ctrl+]` を押して接続する
- 名前空間を指定して `virtctl ssh` を実行する

## コマンド

- [virtctl Commands Reference](#)

## 操作

名前空間を指定して操作する

- OpenShift Web からは `virtctl` CLI を使って操作する
- 名前空間を指定して VM を操作する
- SSH/VNC で接続する VM を操作する
- QEMU guest agent をインストールした OS 上で操作する
- `virtctl scp` を使って VM 間でファイルを転送する

## 参考

- [Linux 上で VM を操作する](#)

# cloud-init

## 概要

cloud-init は OpenShift Virtualization 上で VM に cloud-init をインストールして SSH 接続できるようにする。cloud-init は KubeVirt 上で VM を操作する cloud-init の VirtualMachine 内で cloud-init をインストールする。

## インストール

- cloud-init を VM にインストールする
- 3 つの cloud-init データ (userData, networkData, metadata)
- KubeVirt の NoCloud 設定で cloud-init を VM にインストールする
- cloud-init の cloud-config を指定する
- VirtualMachine 内で cloud-init を 2 回実行 (cloud-init Secret を使)

- cloud-init 設定

インストール

OpenShift 4.18+

前提

- OpenShift Virtualization 4.18 以降
- `oc` での `virtctl` CLI のインストール (Virtctl のインストール)
- YAML の作成
- VM の起動 (Web コンソールでの VM の起動)

## cloud-init のインストール

cloud-init は、OS のインストール時に、SSH の接続を許可するための設定を行います。

cloud-init は Fedora、RHEL、CentOS、Ubuntu、Debian などの Linux 向けに、AWS、Azure、GCP、OpenShift Virtualization、KubeVirt などのクラウド環境で利用されます。



cloud-init は、VM の起動時に、cloud-init の設定ファイルを読み込み、2 回目の起動時に cloud-init を実行します。

## KubeVirt での cloud-init のインストール

cloud-init を VM の起動時にインストールするには、(169.254.169.254 に HTTP でアクセス) して cloud-init をインストールし、KubeVirt の `*NoCloud` をインストールします。

NoCloud をインストールして KubeVirt をインストールします。

1. VirtualMachine の作成 (Secret の作成) して cloud-init をインストール
2. ISO のインストール
3. virtio のインストール
4. OS の cloud-init のインストール

cloud-init をインストールして Kubernetes をインストールします。

```
spec:
  template:
    spec:
      volumes:
        - name: cloudinitdisk
          cloudInitNoCloud: ①
          userData: |
```



```
#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: changeme
```

① **cloudInitNoCloud** VM KubeVirt NoCloud VM

## cloud-init

cloud-init 3 VM KubeVirt **cloudInitNoCloud**

### userData (cloud-config)

userData **cloud-config** **#cloud-config** YAML  
VM

```
cloudInitNoCloud:
  userData: |
    #cloud-config
    ssh_pwauth: true
    user:
      name: fedora
      lock_passwd: false
      plain_text_passwd: s3cur3p4ss
      ssh_authorized_keys:
        - ssh-ed25519 AAAA... user@workstation
    packages:
      - vim
      - tmux
```



1 **#cloud-config** cloud-init cloud-config

### networkData (network-config v2)

VM Netplan **network-config version 2** IP DNS

DHCP

```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      enp1s0:
        dhcp4: true
```

```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      enp1s0:
        addresses:
          - 192.168.1.100/24      ①
        nameservers:
          addresses:
            - 192.168.1.1
            - 8.8.8.8            ②
        routes:
          - to: default
            via: 192.168.1.1      ③
          - to: 10.0.0.0/8
            via: 192.168.1.254    ④
```

- ① CIDR 形式で指定する IP 範囲
- ② DNS 設定
- ③ ゲートウェイを指定する (gateway4 指定)
- ④ 宛先ネットワーク 10.0.0.0/8 を指定する

ネットワーク設定を指定する MAC 設定

```
cloudInitNoCloud:
  networkData: |
    version: 2
    ethernets:
      primary:
        match:
          macaddress: "52:54:00:12:34:56"
        addresses:
          - 192.168.1.100/24
        routes:
          - to: default
            via: 192.168.1.1
```



KubeVirt VM 設定を指定する  
NetworkAttachmentDefinition  
設定

enp1s0 2 指定 enp2s0 指定  
VirtualMachine Spec

ネットワーク設定を指定する VLAN 設定

## metadata

`instance-id`   `local-hostname`   VM   KubeVirt   VirtualMachine

```
cloudInitNoCloud:
  metadata: |
    instance-id: my-vm-001
    local-hostname: my-vm
```



userData   hostname

## cloud-init

cloud-init 5

| Generator | (systemd generator) | cloud-init  |
|-----------|---------------------|-------------|
| Local     |                     | networkData |
| Network   |                     |             |
| Config    | Network             | SSH         |
| Final     |                     | runCmd      |

(networkData)   (userData)   runCmd

## Cloud-config

VM   #cloud-config   userData

```
#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: s3cur3p4ss
  ssh_authorized_keys:
    - ssh-ed25519 AAAA...key-data... user@workstation
```

user   lock\_passwd: false   plain\_text\_passwd   ssh\_pwauth: true   SSH

cloud-init ユーザーを定義する

```
#cloud-config
users:
  - name: admin
    groups: wheel
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...key-data...
  - name: developer
    shell: /bin/bash
```

パッケージをインストールする

```
#cloud-config
package_update: true
packages:
  - vim
  - git
  - tmux
  - python3
```



cloud-init の **final** ステージで VM を初期化する

cloud-init

```
#cloud-config
runcmd:
  - systemctl enable --now cockpit.socket
  - echo "VM initialized at $(date)" >> /var/log/vm-init.log
```

cloud-init の **final** ステージで VM を初期化する (**/bin/sh** を実行) (cloud-init) を実行する

cloud-init

```
#cloud-config
write_files:
  - path: /etc/motd
    content: |
      Welcome to the OpenShift Virtualization VM.
      Managed by cloud-init.
    permissions: '0644'
  - path: /etc/myapp/config.yaml
    content: |
      database: postgres
      port: 5432
```

```
owner: root:root
permissions: '0600'
```



**write\_files**

□□□□□□□□□□□□□□□□

**config**

□□□□□□□□□□□□ **runCmd**

□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ **runCmd** □□□□□□□□

## VirtualMachine □□□□□□□□ **cloud-init** □□□□

VirtualMachine □ cloud-init □□□□□□□□□□ 2 □□□□□□□□□□□□

□□□□□□□□ (**cloudInitNoCloud**)

□□□□□□□□□□ cloud-init □□□□ VirtualMachine □□□□□□□□□□□□□□□□□□□□

**minimal-cloud-init-vm.yaml** □□□□□□□□

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-cloud-init-demo
  namespace: demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        app: fedora-cloud-init-demo
    spec:
      domain:
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
        resources:
          requests:
            memory: 2Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - dataVolume:
            name: fedora-cloud-init-demo
```

```

    name: rootdisk
  - cloudInitNoCloud: ①
    userData: | ②
      #cloud-config
      ssh_pwauth: true
      user:
        name: fedora
        lock_passwd: false
        plain_text_passwd: s3cur3p4ss
        ssh_authorized_keys:
          - 'ssh-ed25519 AAAA...your-key... user@workstation'
      users:
        - name: userone
          lock_passwd: false
          plain_text_passwd: s3cur3p4ss
          ssh_authorized_keys:
            - 'ssh-ed25519 AAAA...your-key... user@workstation'
      packages:
        - vim
        - git
      runcmd:
        - echo "cloud-init complete" > /tmp/cloud-init-done
    name: cloudinitdisk
dataVolumeTemplates:
  - metadata:
      name: fedora-cloud-init-demo
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
        resources:
          requests:
            storage: 30Gi

```

① **cloudInitNoCloud** NoCloud

② **userData** #cloud-config cloud-config YAML

Secret

## Secret

cloud-init Kubernetes Secret VirtualMachine Secret

cloud-init-secret-vm.yaml

Secret

```
oc create secret generic my-vm-cloud-init \
  --from-literal=userdata='#cloud-config
ssh_pwauth: true
user:
  name: fedora
  lock_passwd: false
  plain_text_passwd: s3cur3p4ss
  ssh_authorized_keys:
    - ssh-ed25519 AAAA...your-key... user@workstation
users:
  - name: userone
    lock_passwd: false
    plain_text_passwd: s3cur3p4ss
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...your-key... user@workstation
packages:
  - vim
  - git
' \
-n demo
```

VirtualMachine の spec に userdata を追加

```
volumes:
  - cloudInitNoCloud:
      secretRef:
        name: my-vm-cloud-init ①
      name: cloudinitdisk
```

① KubeVirt の Secret に **userdata** を追加して cloud-init の **userData** を指定

## cloud-init の確認

VM の spec に cloud-init を追加

VM の spec に cloud-init を追加

```
virtctl console fedora-cloud-init-demo -n demo
```

cloud-init の確認

```
cloud-init status --long
```

cloud-init の確認

```
status: done
```

```
extended_status: done
boot_status_code: enabled-by-generator
detail:
DataSourceNoCloud [seed=/dev/vdb]
```

cloud-init のインストールと確認

```
cat /var/log/cloud-init.log | tail -50
```

インストールが完了したら、cloud-init のインストールを確認します。

確認方法

**#cloud-config** の確認

```
userData 1 の内容を確認 #cloud-config の内容を確認 cloud-init の内容を確認 userData
の内容を確認
```

**YAML** の確認

Cloud-config の YAML の内容を確認 (インストール済みの VM の cloud-init devel  
schema --config-file /path/to/config.yaml の内容を確認)

インストール済みの **userData** の確認

```
インストール済みの userData の内容を確認 networkData の内容を確認 userData の内容を確認 network-config v2 の YAML
の内容を確認
```

インストール済みの **cloud-init** の確認

```
cloud-init の内容を確認 cloud-init clean の内容を確認
```

**Secret** の確認

```
secretRef の内容を確認 Secret の内容を確認 (インストール済みの userData の内容を確認 user-data
の内容を確認)
```

## Windows VM の Sysprep

cloud-init の Linux のインストールと Windows VM の OpenShift Virtualization の Sysprep のインストールと  
Microsoft System Preparation のインストールと (インストール済みの VM の内容を確認) のインストール

Windows VM のインストールと Windows VM のインストールと

確認方法

インストール済みの VM の内容を確認

```
oc delete vm fedora-cloud-init-demo -n demo
oc delete vm fedora-cloud-init-secret-demo -n demo
oc delete secret my-vm-cloud-init -n demo
```



```
oc delete namespace demo
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- cloud-init 是 Linux VM 的初始化工具
- KubeVirt 的 NoCloud 镜像包含 cloud-init 镜像，VM 通过 virtio 接口访问镜像
- 3 个主要部分：userData (用户数据，包含 cloud-config)、networkData (network-config v2)、metadata (元数据)
- cloud-init 在 VM 启动时运行，通过 virtio 接口访问镜像
- 2 个主要配置项：cloudInitNoCloud 和 cloudInitSecret
- cloud-init status --long 查看 /var/log/cloud-init.log 日志

11

- [IP 地址](#) - cloud-init 配置 IP/DHCP 配置和 VLAN 配置
- [cloud-init 配置](#) - cloud-init 配置
- [RDP 连接 Windows VM 配置](#) - Sysprep 配置 Windows VM 配置
- [cloud-init 配置](#)
- [cloud-init 配置](#)
- [KubeVirt cloudInitNoCloud 配置](#)

## OpenShift Virtualization

### Overview

OpenShift Virtualization enables you to run VMs on OpenShift. It uses LVM Storage Operator and HostPath Provisioner to manage storage.

VMs (Virtual Machines) are managed by the OpenShift Virtualization VM Controller.

### Architecture

The architecture consists of the following components:

- LVM Storage Operator manages LVM Storage Operator resources.
- HostPath Provisioner manages HostPath Provisioner resources.
- default StorageClass is used for default storage.
- OpenShift Virtualization VM Controller manages VMs.

## OpenShift Virtualization Storage

OpenShift Virtualization uses the following storage options:

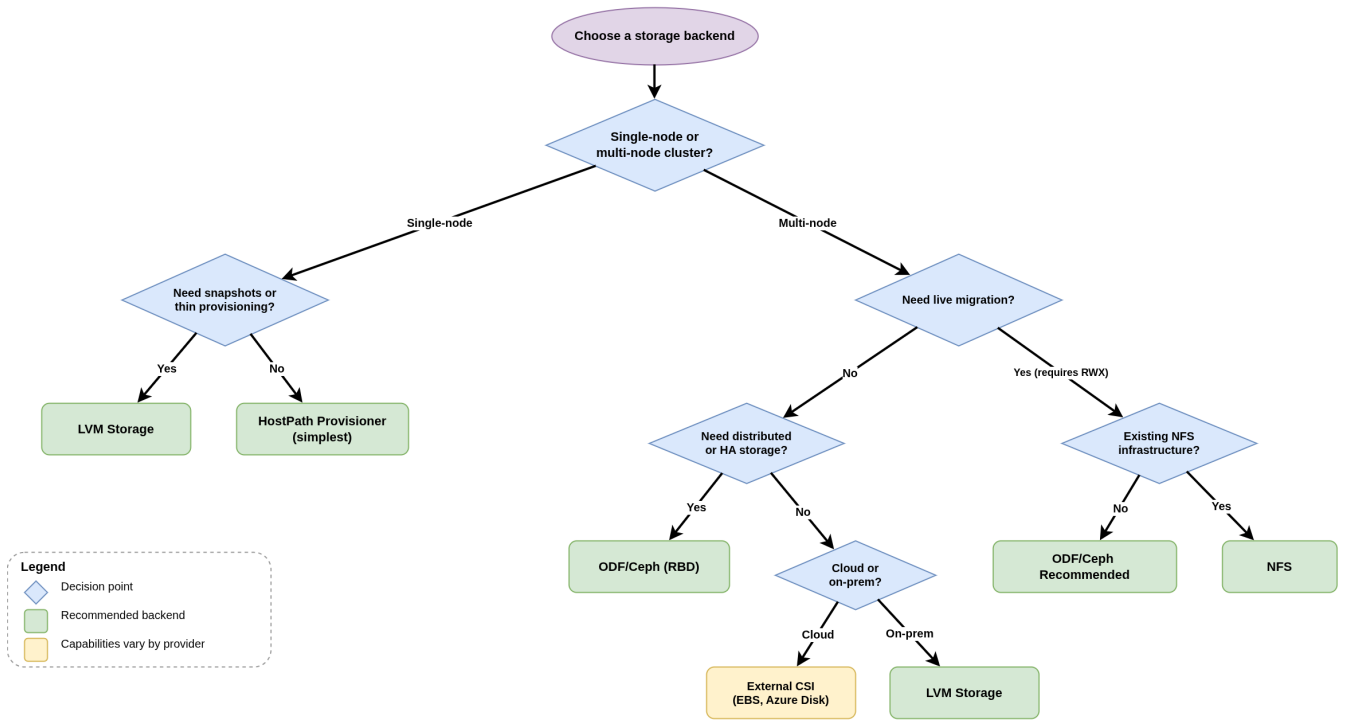
- **LVM Storage:** Logical Volume Manager (LVM) storage (default).
- **HostPath Provisioner:** HostPath Provisioner storage (default).
- **Ceph/ODF:** Ceph/ODF storage.
- **NFS:** NFS storage.
- **Cloud Storage:** AWS EBS, Azure Disk, etc.

Single Node OpenShift (SNO) uses LVM Storage and HostPath Provisioner for storage.

### Storage Options

OpenShift Virtualization supports the following storage options:

## Storage Backend Decision Guide



Storage Backend Decision Guide (Single-node or multi-node cluster) (RWX capabilities vary by provider) 3

| Storage Backend      | Access Mode | Filesystem | Capabilities                | Performance | Scalability | Cost   | Notes                                                 |
|----------------------|-------------|------------|-----------------------------|-------------|-------------|--------|-------------------------------------------------------|
| LVM Storage          | RWO         | Block      | Snapshot, Thin provisioning | Good (RWO)  | Good        | Low    | Simple to deploy, good for general purpose workloads. |
| HostPath Provisioner | RWO         | Filesystem | Simple                      | Good (RWO)  | Good        | Low    | Simplest to deploy, but limited to local storage.     |
| ODF/Ceph (RBD)       | RWO, RWX    | Block      | Snapshot, Thin provisioning | Good (RWX)  | Good        | Medium | Good for VM workloads requiring RWX.                  |
| ODF/Ceph (CephFS)    | RWO, RWX    | Filesystem | Snapshot, Thin provisioning | Good (RWX)  | Good        | Medium | Good for VM workloads requiring RWX.                  |
| NFS                  | RWO, RWX    | Filesystem | Snapshot, Thin provisioning | Good (RWX)  | Good        | Medium | Good for VM workloads requiring RWX.                  |
| External CSI         | RWO, RWX    | Filesystem | Snapshot, Thin provisioning | Good (RWX)  | Good        | Medium | Good for VM workloads requiring RWX.                  |



Storage Backend Decision Guide (Single-node or multi-node cluster) (RWX capabilities vary by provider) 3

## 前提条件

インストール前に、以下の条件を満たしていることを確認してください。

- OpenShift 4.18 をインストール
- 32GB のメモリ
- ディスク容量が 1 GB 以上あること
- Linux のストレージ (LVM) をインストール

## インストール

### LVM Operator をインストール

インストールする LVM Storage Operator のインストールコマンドを実行します。

### LVM をインストール

LVM をインストールするコマンドを実行します。

### HostPath Provisioner

インストールする HostPath Provisioner のインストールコマンドを実行します。

### VM をインストール

StorageClass をインストールする VM のインストールコマンドを実行します。

## Storage Class をインストール

Storage Class をインストールする OpenShift Virtualization のインストールコマンドを実行します。

- データソース: DataSource OS をインストールする Storage Class をインストール
- データソース: インストールする VM のインストール
- データソース: VM をインストールする
- データソース: インストールするストレージをインストール

インストールする LVM Operator のインストール [Step 5: Set Default StorageClass](#) をインストール

## インストール

インストールするコマンドを実行します。

- VM のインストール
- インストール

## インストール

- [Storage Class](#) のインストール

- [OpenShift LVM Storage](#)

## LVM Operator

OpenShift LVM Storage Operator Logical Volume Manager (LVM) Single Node OpenShift (SNO)

- **OpenShift 4.18**
- : 1
- : Operator StorageClass
- : Operator `forceWipeDevicesAndDestroyAllData`

OpenShift 4.19, 4.20



**forceWipeDevicesAndDestroyAllData:** **true**  
The `forceWipeDevicesAndDestroyAllData` parameter is used to force the wiping of all data on the storage devices during the installation of the LVM Operator. This parameter is set to `true` by default.

**1:**

LVM Operator

```
#  
oc get nodes  
  
#  
oc debug node/<node-name> -- chroot /host lsblk -o  
NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL
```

| NAME  | SIZE   | TYPE | FSTYPE      | MOUNTPOINT | MODEL         |
|-------|--------|------|-------------|------------|---------------|
| sda   | 558.4G | disk | LVM2_member |            | PERC H730 Adp |
| sdc   | 558.4G | disk |             |            | PERC H730 Adp |
| -sdc1 | 1M     | part |             |            |               |

```
| -sdc2 127M part vfat
| -sdc3 384M part ext4      /boot
|-sdc4 557.9G part xfs      /sysroot
```

□□□□□□□□□□□□□□□□

- **sda** - □□□ LVM □□□□□□□□□□ (□□□□□□□□□□)
- **sdc** - OS □□□□□□□□□□□□□□□□ (□□□□□□□□□□)

□□□ LVM □□□□□□□□□□

```
oc debug node/<node-name> -- chroot /host pvs
```

□□□:

```
PV          VG Fmt Attr PSize  PFree
/dev/sda    vg1 lvm2 a--  558.37g <55.84g
```

□□□□/dev/sda □□□□ LVM □□□□□□**forceWipeDevicesAndDestroyAllData: true** □□□□□□□□□□□□□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□□

```
oc debug node/<node-name> -- chroot /host ls -la /dev/disk/by-path/ | grep -E
"pci.*scsi"
```

□□□:

```
lrwxrwxrwx. 1 root root 9 Oct 6 13:30 pci-0000:03:00.0-scsi-0:2:1:0 -> ../../sda
```

□□□□ /dev/disk/by-path/pci-0000:03:00.0-scsi-0:2:1:0 □ /dev/sda □□□□□□□□

## □□□□ 2: LVM Storage Operator □□□□□□□□

namespace □□□□□ LVM Storage Operator □□□□□□□□□□

LVM Operator □□ namespace □□□□□□□□

**lvm-namespace.yaml** □□□□□□□□

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: openshift-lvm-storage
```

EOF

OperatorGroup 创建

lvm-operatorgroup.yaml 创建

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-lvm-storage-operatorgroup
  namespace: openshift-lvm-storage
spec:
  targetNamespaces:
  - openshift-lvm-storage
EOF
```

Operator 创建 Subscription 创建

lvm-subscription.yaml 创建

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: lvms-operator
  namespace: openshift-lvm-storage
spec:
  channel: stable-4.20
  name: lvms-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic
EOF
```

Operator 创建

```
# Operator 创建 (23 创建)
oc get csv -n openshift-lvm-storage

# Operator Pod 运行
oc get pods -n openshift-lvm-storage
```

Operator (Operator 创建 LVMCluster 创建):

| NAME | READY | STATUS | RESTARTS | AGE |
|------|-------|--------|----------|-----|
|------|-------|--------|----------|-----|

lvms-operator-864b6975bb-kxt4t 1/1 Running 0 2m



lvms-operator pod vg-manager pod 3 LVMCluster

(3 LVMCluster):

| NAME                           | READY | STATUS  | RESTARTS | AGE |
|--------------------------------|-------|---------|----------|-----|
| lvms-operator-864b6975bb-kxt4t | 1/1   | Running | 0        | 5m  |
| vg-manager-xxxxx               | 1/1   | Running | 0        | 1m  |

### 3: LVMCluster

LVM LVMCluster

(sda)forceWipeDevicesAndDestroyAllData: true



forceWipeDevicesAndDestroyAllData: true

### lvmlcluster-with-wipe.yaml

```
oc apply -f - <<EOF
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: lvmlcluster
  namespace: openshift-lvm-storage
spec:
  storage:
    deviceClasses:
    - name: vg1
      default: true
      thinPoolConfig:
        name: thin-pool-1
        sizePercent: 90
        overprovisionRatio: 10
    nodeSelector:
      nodeSelectorTerms:
      - matchExpressions:
        - key: node-role.kubernetes.io/worker
          operator: In
          values:
          - ""
```



```
deviceSelector:
  paths:
    - /dev/sda
  forceWipeDevicesAndDestroyAllData: true
EOF
```

□□□□□□ (□□□□□□) □□□

lvmcluster-clean-disk.yaml 0000000

```
oc apply -f - <<EOF
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: lvmcluster
  namespace: openshift-lvm-storage
spec:
  storage:
    deviceClasses:
      - name: vg1
        default: true
        thinPoolConfig:
          name: thin-pool-1
          sizePercent: 90
          overprovisionRatio: 10
    nodeSelector:
      nodeSelectorTerms:
        - matchExpressions:
            - key: node-role.kubernetes.io/worker
              operator: In
              values:
                - ""
    deviceSelector:
      paths:
        - /dev/sda
EOF
```

□ □ □ □ □ □ □ □ □ □

- **name: vg1:** `deviceClass` `deviceClass`
- **default: true:** `deviceClass` `deviceClass`
  - `deviceClass` `deviceClass`
  - `PVC` `StorageClass` `StorageClass`
  - `deviceClass` `deviceClass` 1 `deviceClass`
- **thinPoolConfig:** `thinPoolConfig`

- **sizePercent: 90:** thin pool 90%まで
- **overprovisionRatio: 10:** 10 まで過剰予約可能 (100%まで予約可能)
- **nodeSelector:** ノード選択 (ノード Worker Node)
- **deviceSelector.paths:** デバイスパス
  - `/dev/sda` `/dev/disk/by-path/...` `/dev/disk/by-id/...` など
- **forceWipeDevicesAndDestroyAllData: true:** デバイス
  - `true`: デバイスを完全に初期化
  - `false`: デバイスを完全に初期化せず LVM を作成
  - `clean`: デバイスを clean - 初期化
  - `empty`: デバイスを clean/empty 初期化

## ステップ 4: LVMCluster のインストール

LVMCluster のインストール

```
# LVMCluster のインストール
oc get lvmcluster -n openshift-lvm-storage

# volume group のインストール
oc debug node/<node-name> -- chroot /host vgs

# thin pool のインストール
oc debug node/<node-name> -- chroot /host lvs
```

確認:

- LVMCluster の **STATUS: Ready** になる
- VG **vg1** が作成される
- テンプレート **thin-pool-1** が作成される/初期化される

インストール完了後、LVM Storage をインストールする

## ステップ 5: StorageClass のインストール

LVM Operator の **lvms-vg1** StorageClass をインストールする OpenShift Virtualization の VM の **DataVolume** をインストールする **lvms-vg1** StorageClass をインストール

StorageClass のインストール

```
oc get storageclass
```

確認:

| NAME     | PROVISIONER | RECLAIMPOLICY | VOLUMEBINDINGMODE    | ALLOWVOLUMEEXPANSION |
|----------|-------------|---------------|----------------------|----------------------|
| lvms-vg1 | topolvm.io  | Delete        | WaitForFirstConsumer | true                 |
| 5m       |             |               |                      |                      |



(default) `oc patch storageclass lvms-vg1 -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'`

StorageClass

LVM StorageClass

```
oc patch storageclass lvms-vg1 -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

```
oc describe sc | grep IsDefaultClass
```

| NAME               | PROVISIONER | RECLAIMPOLICY | VOLUMEBINDINGMODE    | ALLOWVOLUMEEXPANSION |
|--------------------|-------------|---------------|----------------------|----------------------|
| lvms-vg1 (default) | topolvm.io  | Delete        | WaitForFirstConsumer | true                 |
| 6m                 |             |               |                      |                      |

(default) `oc patch storageclass lvms-vg1 -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'`

OpenShift Virtualization

StorageClass OpenShift Virtualization

DataSource OS

OpenShift Virtualization DataSource OS (Fedora RHEL CentOS Stream Windows)  
StorageClass

```
# OS
oc get datasources -n openshift-virtualization-os-images
```

StorageClass DataSource VM

VM PVC

dataVolumeTemplates VM storageClassName StorageClass

```

dataVolumeTemplates:
- metadata:
  name: fedora-volume
spec:
  pvc:
    accessModes:
    - ReadWriteOnce
    resources:
      requests:
        storage: 30Gi
  sourceRef:
    kind: DataSource
    name: fedora
    namespace: openshift-virtualization-os-images

```

StorageClass (**lvms-vg1**)

**VM**

StorageClass

- VM
- DataSource
- PVC **storageClassName**
- 

StorageClass OpenShift Virtualization Operator OS

```

#
oc get DataImportCron -n openshift-virtualization-os-images

```

**VM** **StorageClass**

StorageClass VM

```

dataVolumeTemplates:
- metadata:
  name: fedora-volume
spec:
  pvc:
    accessModes:
    - ReadWriteOnce
    resources:
      requests:
        storage: 30Gi

```

```

storageClassName: lvms-vg1 # Must be explicit
sourceRef:
  kind: DataSource
  name: fedora
  namespace: openshift-virtualization-os-images

```



OpenShift Virtualization StorageClass

## 6:

PVC

**test-pvc.yaml**

```

oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: lvms-test-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: lvms-vg1
EOF

```

PVC Pod

**test-pod.yaml**

```

oc apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: lvms-test-pod
  namespace: default
spec:
  containers:
    - name: test
      image: registry.access.redhat.com/ubi9/ubi:latest
      command: ["sleep", "3600"]
      volumeMounts:
        - name: data
          mountPath: /data

```

```
volumes:
- name: data
  persistentVolumeClaim:
    claimName: lvms-test-pvc
EOF
```

PVC `lvms-test-pvc` 생성

```
oc get pvc lvms-test-pvc

# STATUS   Bound
```

Pod `lvms-test-pod` 생성

```
# Pod 생성
oc wait --for=condition=Ready pod/lvms-test-pod --timeout=120s

# 실행
oc exec lvms-test-pod -- sh -c "echo 'LVM Storage Test' > /data/test.txt && cat /data/test.txt"

# 출력: LVM Storage Test
```

테스트 완료

```
oc delete pod lvms-test-pod
oc delete pvc lvms-test-pvc
```

결과 확인

LVM `lvms-test-pvc` 생성 [LVM Storage](#) 테스트 완료

결과 확인

- `lvms-test-pvc` 생성
- `lvms-test-pod` 생성
- `lvms-test-pod` 실행
- PVC `lvms-test-pvc` 생성
- thin pool 생성
- 실행

**forceWipeDevicesAndDestroyAllData** 실행

11

```
forceWipeDevicesAndDestroyAllData: true
```

1. **物理磁盘分区:** GPT/MBR
2. **文件系统:** ext4/xfs/btrfs
3. **LVM 逻辑卷:** 物理卷/逻辑卷/快照
4. **挂载点:** /usr/local/
5. **操作:** LVM Operator

□ □ □ □ □ □ □ □

```
forceWipeDevicesAndDestroyAllData: true ████████████████████
```

- 物理的ハードウェア環境を構築する
- 物理的ハードウェア環境を LVM で管理する (PV/VG/LV 作成)
- 物理的ハードウェア環境を RAID 化する
- 物理的ハードウェア環境を ZFS 化する

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- 00000000000000000000000000000000
- 00000000000000000000 (0000000000000000)
- 00000000000000000000000000000000

□□□□□□□□

[illegible]

```
# 1. 確認ディスク
oc debug node/<node-name> -- chroot /host lsblk -o NAME,SIZE,TYPE,MOUNTPOINT

# 2. 確認 /dev/sda がマウントされているか
oc debug node/<node-name> -- chroot /host mount | grep /dev/sda

# 3. 確認 /dev/sda がディスクとして認識されているか
oc debug node/<node-name> -- chroot /host df -h | grep sda

# 4. 確認 /dev/sda がディスクとして認識されているか
oc debug node/<node-name> -- chroot /host lsblk -d -o NAME,SERIAL /dev/sda
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- LVM

- namespace=OperatorGroup=Subscription の LVM Storage Operator のインストール
- forceWipeDevicesAndDestroyAllData の LVMCluster のインストール
- lvms-vg1 StorageClass の OpenShift Virtualization のインストール
- PVC の Pod の LVMCluster のインストール

□□

- LVM のインストール
- のインストール

## LVM のインストール

□□

OpenShift の LVM Storage Operator のインストールは、LVM のインストール、volume group、persistent volume claim のインストールからなる。

□□□□□□□□

LVM のインストールは、以下の手順で行う。

```
# 1. LVMCluster のインストール
oc get lvmcluster -n openshift-lvm-storage

# 2. Pod が Running のインストール
oc get pods -n openshift-lvm-storage

# 3. StorageClass のインストール
oc get storageclass | grep lvms

# 4. volume group のインストール
oc debug node/<node-name> -- chroot /host vgs

# 5. thin pool のインストール
oc debug node/<node-name> -- chroot /host lvs
```

□□□□□□:

- LVMCluster: STATUS: Ready
- Pods: Running のインストール
- StorageClass: lvms-vg1 のインストール
- VG: volume group のインストール
- LV: thin provisioning pool のインストール/インストール



■■■■■■■■■■■■■■■■■■■■

## LVMCluster ■■■■

■■■■■■■■■■

```
oc get lvmcluster -n openshift-lvm-storage
```

■■■■■:

| NAME       | STATUS |
|------------|--------|
| lvmcluster | Ready  |

■■■■■■■■:

- ■■■■■■ **NotReady**■**Degraded**■**Error** ■■■■■■
- ■■■■■■■■■■■■

■■■■■■■■■■■■

```
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml
```

■■■■■■■■:

```
status:
  conditions:
  - status: "True"
    type: ResourcesAvailable
    message: Reconciliation is complete and all the resources are available
  - status: "True"
    type: VolumeGroupsReady
    message: All the VGs are ready
  deviceClassStatuses:
  - name: vg1
    nodeStatus:
      - devices:
        - /dev/sda
        status: Ready
    ready: true
    state: Ready
```

■■■■■■■■■■:

- **status.conditions: ResourcesAvailable: True** ■■■ **VolumeGroupsReady: True** ■■■■■■■■■■■■
- **deviceClassStatuses[].nodeStatus[].status: Ready** ■■■■■■■■■■

- `deviceClassStatuses[].nodeStatus[].devices`: []
- `ready: true` and `state: Ready`

Example:

```
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 50 "deviceClassStatuses"
```

Output:

```
deviceClassStatuses:
- name: vg1
  nodeStatus:
  - devices:
    - /dev/sda
    excluded:
    - name: /dev/sdc
      reasons:
      - /dev/sdc has children block devices and could not be considered
      - /dev/sdc is not part of the device selector
    status: Ready
```

Example:

## Pod

Example Pod

```
oc get pods -n openshift-lvm-storage
```

Output:

| NAME                           | READY | STATUS  | RESTARTS | AGE |
|--------------------------------|-------|---------|----------|-----|
| lvms-operator-864b6975bb-kxt4t | 1/1   | Running | 0        | 10m |
| vg-manager-xxxxx               | 1/1   | Running | 0        | 5m  |

Example Pod:

- **lvms-operator**: LVM operator
- **vg-manager**: 1 volume group manager
- **topolvm-controller**: Single Node OpenShift (SNO) controller

Pod

```
# Pod
```

```
oc describe pod -n openshift-lvm-storage <pod-name>
```

```
# Pod 查看
```

```
oc logs -n openshift-lvm-storage <pod-name>
```

```
# vg-manager 查看 (openshift-lvm-storage)
```

```
oc logs -n openshift-lvm-storage daemonset/vg-manager
```

错误信息:

- "device is in use" - 设备正在使用中
- "cannot wipe device" - `forceWipeDevicesAndDestroyAllData: true` 强制擦除设备
- "device not found" - 设备未找到
- "no devices found" - 没有找到设备

**Operator 配置**

```
# ClusterServiceVersion 查看
```

```
oc get csv -n openshift-lvm-storage
```

```
# Subscription 查看
```

```
oc get subscription -n openshift-lvm-storage
```

```
# InstallPlan 查看
```

```
oc get installplan -n openshift-lvm-storage
```

配置示例:

**StorageClass 配置**

```
# 查看 StorageClass 配置
```

```
oc get storageclass
```

```
# LVM StorageClass 配置
```

```
oc get storageclass lvms-vg1 -o yaml
```

配置示例:

| NAME     | PROVISIONER | RECLAIMPOLICY | VOLUMEBINDINGMODE    | ALLOWVOLUMEEXPANSION |
|----------|-------------|---------------|----------------------|----------------------|
| lvms-vg1 | topolvm.io  | Delete        | WaitForFirstConsumer | true                 |

配置示例:

- **PROVISIONER:** `topolvm.io` - 配置示例

- **VOLUMEBINDINGMODE: WaitForFirstConsumer** - 等待第一个消费者
- **ALLOWVOLUMEEXPANSION: true** - 允许卷扩展

**StorageClass** 检查

```
oc get storageclass lvms-vg1 -o
jsonpath='{.metadata.annotations.storageclass\.kubernetes\.io/is-default-class}'
```

输出:

- **true** - default StorageClass 是
- 不是 **false** - 不是默认 StorageClass

**PersistentVolumes** 检查

```
# 检查 PV 列表
oc get pv

# LVM 类型的 PV 列表
oc get pv | grep topolvm
```

**volume group** 检查

**volume group** 检查

```
oc debug node/<node-name> -- chroot /host vgs
```

输出:

```
VG   #PV #LV #SN Attr   VSize   VFree
vg1   1  1  0 wz--n- 558.37g <55.84g
```

输出:

- **VG** 是 LVMCluster 类型 (值: **vg1**)
- **VSize** 卷大小
- **VFree** 卷空闲空间
- **Attr** 是 **wz--n-** (写时复制, 不共享, 不镜像) 属性

**volume group** 检查

```
oc debug node/<node-name> -- chroot /host vgdisplay vg1
```

root:

```
--- Volume group ---
VG Name          vg1
System ID
Format           lvm2
Metadata Areas   1
Metadata Sequence No 2
VG Access        read/write
VG Status        resizable
MAX LV          0
Cur LV          1
Open LV          0
Max PV           0
Cur PV          1
Act PV           1
VG Size          558.37 GiB
PE Size          4.00 MiB
Total PE         142942
Alloc PE / Size  128517 / 502.04 GiB
Free PE / Size   14425 / 56.35 GiB
VG UUID          abc123...
```

root@node1:~#

- **VG Status:** *resizable*
- **VG Access:** *read/write*
- **Free PE / Size** *14425 / 56.35 GiB*

## thin provisioning pool root

thin provisioning pool root@node1

```
oc debug node/<node-name> -- chroot /host lvs
```

root@node1:~#

| LV          | VG  | Attr       | LSize    | Pool | Origin | Data% | Meta% |
|-------------|-----|------------|----------|------|--------|-------|-------|
| thin-pool-1 | vg1 | twi-a-tz-- | <502.04g |      |        | 0.00  | 6.76  |

root@node1 (twi-a-tz--):

- **t** - thin pool
- **w** - writeable
- **i** - inherited

- **a** - active
- **t** - thin
- **z** - zero

Data% Meta% :

- **Data%:** thin provisioning pool
  - **< 80%** -
  - **80-90%** -
  - **> 90%** -
- **Meta%:**
  - **< 80%** -
  - **> 80%** -

**thin provisioning pool**

```
oc debug node/<node-name> -- chroot /host lvdisplay vg1/thin-pool-1
```

```
oc debug node/<node-name> -- chroot /host pvs
```

:

| PV       | VG  | Fmt  | Attr | PSize   | PFree   |
|----------|-----|------|------|---------|---------|
| /dev/sda | vg1 | lvm2 | a--  | 558.37g | <55.84g |

(a--):

- **a** - allocatable
- **-** - not exported
- **-** - not missing

```
oc debug node/<node-name> -- chroot /host pvdisplay /dev/sda
```

```
# 確認
oc debug node/<node-name> -- chroot /host ls -la /dev/sda

# 確認
oc debug node/<node-name> -- chroot /host lsblk /dev/sda

# 確認
oc debug node/<node-name> -- chroot /host mount | grep /dev/sda
```



確認LVM 確認

確認

確認

```
oc debug node/<node-name> -- chroot /host lsblk -o
NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL
```

結果:

| NAME   | SIZE   | TYPE | FSTYPE      | MOUNTPOINT | MODEL         |
|--------|--------|------|-------------|------------|---------------|
| sda    | 558.4G | disk | LVM2_member |            | PERC H730 Adp |
| sdc    | 558.4G | disk |             |            | PERC H730 Adp |
| -sdc1  | 1M     | part |             |            |               |
| -sdc2  | 127M   | part | vfat        |            |               |
| -sdc3  | 384M   | part | ext4        | /boot      |               |
| `-sdc4 | 557.9G | part | xf          | /sysroot   |               |

確認:

- MOUNTPOINT 確認 - 確認
- FSTYPE 確認 **LVM2\_member** 確認 (確認)
- MOUNTPOINT 確認 - 確認
- 確認 (確認) 確認 - 確認

確認

```
oc debug node/<node-name> -- chroot /host lsblk -d -o NAME,SIZE,SERIAL
```

確認

確認

```
# 確認
```

```
oc debug node/<node-name> -- chroot /host ls -la /dev/disk/by-path/ | grep sda

# ID
oc debug node/<node-name> -- chroot /host ls -la /dev/disk/by-id/ | grep sda
```

```
# ID 00000000000000000000000000000000
oc debug node/<node-name> -- chroot /host ls -la /dev/disk/by-id/ | grep sda
```

```
oc debug node/<node-name> -- chroot /host ls -la /dev/disk/by-id/ | grep sda
```

□□□□□□□□

**1: LVMCluster "NotReady" 0/0**

| NAME       | STATUS   |
|------------|----------|
| lvmcluster | NotReady |

```
# 检查配置
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 20
"conditions"

# 查看日志
oc logs -n openshift-lvm-storage daemonset/vg-manager
```

```
# vg-manager
oc logs -n openshift-lvm-storage daemonset/vg-manager
```

```
oc logs -n openshift-lvm-storage daemonset/vg-manager
```

□□□□□□□□□□:

[illegible]

```
# 000000000000
oc debug node/<node-name> -- chroot /host ls -la /dev/sda
```

```
oc debug node/<node-name> -- chroot /host ls -la /dev/sda
```

0000: 0000000000 LVMCluster 0000000000

0000000000000000000000

```
# 0000000000000000
oc debug node/<node-name> -- chroot /host lsblk -f /dev/sda
```

```
oc debug node/<node-name> -- chroot /host lsblk -f /dev/sda
```

```
❗️: deviceSelector ❗️ forceWipeDevicesAndDestroyAllData: true ❗️❗️❗️❗️❗️❗️
```

```
# 0000000000000000
oc debug node/<node-name> -- chroot /host mount | grep /dev/sda
```

```
oc debug node/<node-name> -- chroot /host mount | grep /dev/sda
```



####: #####

## ## 2: volume group #####

####:

- LVMCluster ##### vgs volume group #####
- ##### deviceClassStatuses #####

####:

```
# #####
oc debug node/<node-name> -- chroot /host pvs

# vg-manager #####
oc logs -n openshift-lvm-storage daemonset/vg-manager --tail=100

# #####
oc debug node/<node-name> -- chroot /host lsblk -o NAME,SIZE,TYPE,FSTYPE
```

####:

#####

- deviceSelector #####
- ##### (deviceClassStatuses.excluded #####)

#####

- vg-manager Pod #####
- SELinux #####

#####

- forceWipeDevicesAndDestroyAllData: true #####
- #####Operator #####

## ## 3: PVC in Pending

####:

| NAME     | STATUS  | VOLUME | CAPACITY | ACCESS MODES | STORAGECLASS | AGE |
|----------|---------|--------|----------|--------------|--------------|-----|
| test-pvc | Pending |        |          |              | lvms-vg1     | 5m  |

####:

```
# PVC #####
```

```
oc describe pvc <pvc-name>
```

```
# PVC Pod
```

```
oc get pods --all-namespaces -o wide | grep <pvc-name>
```

```
# VG
```

```
oc debug node/<node-name> -- chroot /host vgs
```

：

**WaitForFirstConsumer** - Pod

- PVC Pod PVC Pending
- PVC Pod

：

```
# VG
```

```
oc debug node/<node-name> -- chroot /host vgdisplay vg1 | grep "Free"
```

： PV volume group

**thin provisioning pool**

```
# thin pool
```

```
oc debug node/<node-name> -- chroot /host lvs | grep thin-pool
```

： thin provisioning pool

：

```
# Pod nodeSelector LVMCluster nodeSelector
```

```
oc get pod <pod-name> -o yaml | grep nodeSelector
```

```
oc get lvmcluster lvmcluster -n openshift-lvm-storage -o yaml | grep -A 5 nodeSelector
```

**4: thin provisioning pool Data% Meta%**

：

| LV          | VG  | Attr       | LSize   | Data% | Meta% |
|-------------|-----|------------|---------|-------|-------|
| thin-pool-1 | vg1 | twi-a-tz-- | 502.04g | 92.00 | 85.00 |

：

```
# thin pool
```

```
oc debug node/<node-name> -- chroot /host lvs -a | grep thin-pool

# thin volume
oc debug node/<node-name> -- chroot /host lvs -a -o
lv_name,lv_size,data_percent,pool_lv | grep thin-pool

# PV
oc get pv -o custom-
columns=NAME:.metadata.name,SIZE:.spec.capacity.storage,STORAGECLASS:.spec.storageClass
Name | grep lvms
```

□□□:

□□□□ **PVC** □□□□

```
# PVC
oc get pvc --all-namespaces

# PVC
oc delete pvc <pvc-name> -n <namespace>
```

**thin provisioning pool** □□□□ (VG □□□□□□□□)

```
# VG
oc debug node/<node-name> -- chroot /host vgs

# thin pool (VG)
# LVM Operator
```

□□□□□□□□□□□□□□□□

LVMCluster □□□□overprovisionRatio □□□□□□□□

□□ 5: □□□□□□□□□□

□□:

- LVMCluster □□□□□□□□□□□□□□□□
- □□□ "cannot wipe device" □□□ "device busy" □□□□□□

□□:

```
# /dev/sda
oc debug node/<node-name> -- chroot /host lsblk -f /dev/sda

# LVM
oc debug node/<node-name> -- chroot /host pvs | grep /dev/sda
```

```
# 
oc debug node/<node-name> -- chroot /host lsof | grep /dev/sda
```

□□□:

```
# forceWipeDevicesAndDestroyAllData
oc edit lvmcluster lvmcluster -n openshift-lvm-storage

# deviceSelector:
forceWipeDevicesAndDestroyAllData: true
```

## ❏ 6: vg-manager Pod ❏ CrashLooping ❏

□□:

| NAME           | READY | STATUS           | RESTARTS | AGE |
|----------------|-------|------------------|----------|-----|
| vg-manager-xxx | 0/1   | CrashLoopBackOff | 5        | 3m  |

00:

```
# Pod 查看
oc logs -n openshift-lvm-storage <vg-manager-pod> --previous

# Pod 描述
oc describe pod -n openshift-lvm-storage <vg-manager-pod>

# 节点描述
oc describe node <node-name>
```

□□□□□□:

□ □ □ □ □ □

- **vg-manager** `vgmanager`
- `SELinux` `selinux`

□ □ □ □ □ □

- 
- 

□ □ □ □ □ □

- 記憶體存取 CPU 存取速度
- 記憶體存取頻寬

## 7: StorageClass

□□:

- storageClassName PVC 클래스명
- 예: "no default deviceClass was specified"

□□:

```
# 查看 StorageClass 列表
oc get storageclass | grep default

# lvms StorageClass 列表
oc get storageclass lvms-vg1 -o yaml | grep is-default-class
```

000:

## LVM StorageClass

```
oc patch storageclass lvms-vg1 -p '{"metadata":  
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

## ■■■ LVMCluster ■■■■■■■■■■

```
oc edit lvmcluster lvmcluster -n openshift-lvm-storage

# deviceClasses {}:
- name: vg1
  default: true
```

|  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|

[illegible]

```
#!/bin/bash

NAMESPACE="openshift-lvm-storage"
NODE_NAME=""

echo "=== LVM Storage Diagnostic Report ==="
echo ""

echo "1. LVMcluster Status:"
oc get lvmcluster -n $NAMESPACE
echo ""

echo "2. Pods Status:"
```



## ODF/Ceph Storage

11

OpenShift Virtualization Ceph OpenShift Data Foundation (ODF)

OpenShift Data Foundation 3.11.0-rc1

- **Block Storage (Ceph RBD):** VM ReadWriteOnce
- **File Storage (CephFS):** ReadWriteMany
- **Object Storage:** S3

□ □ □ □

- **OpenShift 4.18+**
- 3 Worker Node : Ceph
- Operator
- StorageClass (AWS gp3-csi Azure managed-csi) raw
- Worker Node 16 GB RAM (32 GB)
- Worker Node 8 CPU



ODF □ Ceph □□□□□□□□□□□□□□□□3 □□□□ Worker Node □□□□□□Single Node □□□ 2 Node  
□□□□□□□□□□□□□□□□



```
Worker Node  zone  host
StorageCluster flexibleScaling: true
```

### Worker Node

ODF ــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــ Worker Node ــــــــــــــــــــــــــــــــــــــــ

Worker Node

```
oc get nodes --selector='!node-role.kubernetes.io/master'
```

Worker Node 3

```
oc label node <worker-node-1> cluster.ocs.openshift.io/openshift-storage=''
```

```
oc label node <worker-node-2> cluster.ocs.openshift.io/openshift-storage=''
```

```
oc label node <worker-node-3> cluster.ocs.openshift.io/openshift-storage=''
```

□□□□□□□□

```
oc get nodes -l cluster.ocs.openshift.io/openshift-storage
```

## □□□□ 2: OpenShift Data Foundation Operator □□□□□□

ODF namespace □□□□□□

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: openshift-storage
  labels:
    openshift.io/cluster-monitoring: "true"
EOF
```

OperatorGroup □□□□□□

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-storage
  namespace: openshift-storage
spec:
  targetNamespaces:
    - openshift-storage
EOF
```

ODF Operator □□□□□□□□□□<version> □□□□ OpenShift □□□□□□□□□□□□ (□: OpenShift 4.21 □□□□ stable-4.21) □□□□□□□□□□

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: odf-operator
  namespace: openshift-storage
spec:
```



```
channel: stable-4.21
name: odf-operator
source: redhat-operators
sourceNamespace: openshift-marketplace
EOF
```

[illegible]

```
oc get packagemanifest odf-operator -n openshift-marketplace -o
jsonpath='{.status.channels[*].name}'
```

Operator 成功 CSV 成功

```
oc get csv -n openshift-storage
```

### 3: StorageCluster

StorageCluster    Ceph    ODF    StorageClass (gp3-csi)  
gp3-csi    StorageClass



```
oc apply -f - <<EOF
apiVersion: ocs.openshift.io/v1
kind: StorageCluster
metadata:
  name: ocs-storagecluster
  namespace: openshift-storage
spec:
  flexibleScaling: true
  enableCephTools: true
  storageDeviceSets:
    - name: ocs-device-set-gp3-csi
      count: 1
      portable: true
      replica: 3
      resources:
        requests:
          cpu: "1"
          memory: 5Gi
        limits:
          cpu: "2"
          memory: 5Gi
      placement:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
```

```

nodeSelectorTerms:
  - matchExpressions:
    - key: cluster.ocs.openshift.io/openshift-storage
      operator: In
      values:
        - ""
tolerations:
  - effect: NoSchedule
    key: node.ocs.openshift.io/storage
    operator: Equal
    value: "true"
dataPVCTemplate:
  spec:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 512Gi
    storageClassName: gp3-csi
    volumeMode: Block
managedResources:
  cephBlockPools: {}
  cephFilesystems: {}
  cephObjectStoreUsers: {}
  cephObjectStores: {}
EOF

```



Worker Node flexibleScaling: true  
 zone host Ceph  
 undersized+peered

```
oc get storagecluster ocs-storagecluster -n openshift-storage -w
```

Ready 5/10

## 4: StorageClass

ODF StorageClass

```
oc get storageclass | grep ocs
```

StorageClass:

```

ocs-storagecluster-ceph-rbd      Block storage (RBD) for VM boot disks
ocs-storagecluster-ceph-rbd-virtualization  Block storage optimized for virtualization

```

workloads

ocs-storagecluster-cephfs

File storage (CephFS) for shared access

**ocs-storagecluster-ceph-rbd-virtualization** StorageClass OpenShift Virtualization ODF  
○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

## ○○○○ 5: ○○○○○ Storage Class ○○○

VM ○○○○○○○○○○○○○ Ceph RBD Storage Class ○○○○○○○

```
oc patch storageclass ocs-storagecluster-ceph-rbd -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
```

○○○○ Storage Class ○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

```
oc get storageclass -o  
jsonpath='{.items[?(@.metadata.annotations.storageclass.kubernetes.io/is-default-  
class=="true")].metadata.name}'
```

○○ Storage Class ○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

```
oc patch storageclass <existing-default-sc> -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": null}}}'
```

Ceph RBD Storage Class ○○○○○○○○○○○○○○○○○○○○○

```
oc get storageclass | grep "(default)"
```

○○○○○○○:

```
ocs-storagecluster-ceph-rbd (default) openshift-storage.rbd.csi.ceph.com Delete  
Immediate true
```

## ○○○○ 6: ○○○○○○○○○○○○○○○○○○○

Ceph RBD ○○○○○○○○○○○○○○○○ PVC ○○○○○○○

```
oc apply -f - <<EOF  
apiVersion: v1  
kind: PersistentVolumeClaim  
metadata:  
  name: test-rbd-pvc  
  namespace: default
```

```
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: ocs-storagecluster-ceph-rbd
EOF
```

PVC を確認する

```
oc get pvc test-rbd-pvc
```

CephFS を利用する PVC を作成する

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-cephfs-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  storageClassName: ocs-storagecluster-cephfs
EOF
```

作成した PVC を確認する

```
oc get pvc test-rbd-pvc test-cephfs-pvc
```

ステップ 7: VM を作成する

ODF を利用する VM を作成する

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: test-vm-odf
  namespace: default
spec:
 instancetype:
```

```

name: u1.small
preference:
  name: fedora
runStrategy: Halted
template:
  metadata:
    labels:
      kubevirt.io/vm: test-vm-odf
  spec:
    domain:
      devices:
        disks:
          - disk: {}
            name: boot-disk
        interfaces:
          - masquerade: {}
            name: default
      networks:
        - name: default
      pod: {}
    volumes:
      - name: boot-disk
        dataVolume:
          name: test-vm-odf-boot
dataVolumeTemplates:
- apiVersion: cdi.kubevirt.io/v1beta1
  kind: DataVolume
  metadata:
    name: test-vm-odf-boot
  spec:
    source:
      registry:
        url: docker://quay.io/containerdisks/fedora:latest
    storage:
      accessModes:
        - ReadWriteOnce
      resources:
        requests:
          storage: 10Gi
      storageClassName: ocs-storagecluster-ceph-rbd
EOF

```

DataVolume `test-vm-odf-boot`

```
oc get dv test-vm-odf-boot -w
```

DataVolume `test-vm-odf-boot` VM `test-vm-odf`

```
virtctl start test-vm-odf
```

VM `test-vm-odf`:

```
oc get vm test-vm-odf
```

`test-vm-odf`

`test-vm-odf` VM `test-vm-odf`:

### ReadWriteOnce (RWO) - Ceph RBD

- `test-vm-odf`: VM `test-vm-odf` `test-vm-odf`
- `test-vm-odf`: `test-vm-odf` 1 `test-vm-odf`
- `test-vm-odf`: KubeVirt `test-vm-odf` `test-vm-odf` Ceph RBD `test-vm-odf`
- `test-vm-odf`: `test-vm-odf` `test-vm-odf`

### ReadWriteMany (RWX) - CephFS

- `test-vm-odf`: `test-vm-odf` `test-vm-odf`
- `test-vm-odf`: `test-vm-odf` `test-vm-odf`
- `test-vm-odf`: `test-vm-odf` `test-vm-odf` `test-vm-odf`
- `test-vm-odf`: `test-vm-odf` `test-vm-odf` `test-vm-odf`

`test-vm-odf` VM `test-vm-odf`:

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: vm-boot-rbd
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: ocs-storagecluster-ceph-rbd
EOF
```

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
```

```

name: vm-shared-cephfs
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
  storageClassName: ocs-storagecluster-cephfs
EOF

```

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: multi-storage-vm
spec:
  instancetype:
    name: u1.small
  preference:
    name: fedora
  runStrategy: Halted
  template:
    spec:
      domain:
        devices:
          disks:
            - disk: {}
              name: boot-disk
            - disk: {}
              name: shared-data
          volumes:
            - name: boot-disk
              persistentVolumeClaim:
                claimName: vm-boot-rbd
            - name: shared-data
              persistentVolumeClaim:
                claimName: vm-shared-cephfs
EOF

```

□□□□□□□□□□

□□□□□□□

- □□□□□□□□: RAW □□□□□□□□ (Ceph RBD)
- □□□□□□□□: □□□□□□□□ I/O □□□□□□□□□□□□□□□□□□□□
- □□□□□□□□: □□ OS □□□□□□□□□□□□□□
- □□□□□□□□: □□□□□□□□□□□□□□□□□□□□□□□□

○○○○○○○○○○

- ○○○○○○○○: ○○○○○○○○○○○○○○ (CephFS)
- ○○○○○○○○: ○○○○○○○○○○○○○○○○○
- ○○○○○○○○: ○○○○○○○○○○○○○○○○○
- ○○○○○○○○: ○○○○○○○○○○○○○○○○○

## ODF ○○○○○○○○○○

ODF ○○○○○○○○○○ Ceph ○○○○○○○○○○○○○○○○

```
oc get route console -n openshift-console -o jsonpath='{.spec.host}'
```

OpenShift Web ○○○○○○ **Storage** → **Data Foundation** ○○○○○○○○○○○○

- Ceph ○○○○○○○○○○○○
- ○○○○○○○○○○○○
- ○○○○○○○○○○○○
- ○○○○○○

CLI ○○○ Ceph ○○○○○○○○○○○○○○○○:

```
oc get cephcluster -n openshift-storage -o jsonpath='{.items[0].status.ceph.health}'
```

Ceph toolbox pod ○○○StorageCluster ○ **enableCephTools: true** ○○○○○○○○○○○○○○○○○○○○○○○○:

```
oc get pods -n openshift-storage -l app=rook-ceph-tools
```

Ceph ○○○○○○○○○○○○○○○○:

```
oc rsh -n openshift-storage deployment/rook-ceph-tools ceph status
```

OSD ○○○○○○○○○○○○

```
oc rsh -n openshift-storage deployment/rook-ceph-tools ceph osd status
```

○○○○○○○○○○○○

```
oc rsh -n openshift-storage deployment/rook-ceph-tools ceph df
```





```
oc describe pod <osd-pod-name> -n openshift-storage
```

## Storage Cluster

## StorageCluster

```
oc describe storagecluster ocs-storagecluster -n openshift-storage
```

Pod

```
oc get pods -n openshift-storage
```

□ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc delete vm test-vm-odf multi-storage-vm
```

```
oc delete dv test-vm-odf-boot
```

```
oc delete pvc test-rbd-pvc test-cephfs-pvc vm-boot-rbd vm-shared-cephfs
```



ODF      ODF      StorageCluster      ODF  
VM

100

OpenShift   Virtualization      Ceph      OpenShift   Data   Foundation

- Ceph 部署到虚拟机 ODF 部署到虚拟机
- 部署到虚拟机 Ceph RBD 部署 CephFS Storage Class 部署
- 部署 VM 部署到虚拟机 Storage Class 部署
- ODF 部署到虚拟机 VM 部署
- VM 部署到虚拟机部署到虚拟机
- 部署到虚拟机部署到虚拟机部署到虚拟机

[illegible]

11

- LVM Storage Operator
- Storage Class
- VM
- OpenShift Data Foundation
- 

# HostPath Provisioner

11

| HostPath | Provisioner | (HPP)               | OpenShift | Virtualization          |
|----------|-------------|---------------------|-----------|-------------------------|
|          | HPP         |                     | OpenShift | Virtualization Operator |
|          |             | (Persistent Volume) |           |                         |

## HostPath Provisioner ☐

[illegible]

- 000000000000: 00000000000000000000000000000000
- 000000000000: 00000000000000
- 00000000: 0000000000000000000000000000
- 0000000000: 0000000000000000

## LVM Storage □□□□

| 項目     | HostPath Provisioner | LVM Storage       |
|--------|----------------------|-------------------|
| インストール | ホストパス (ホストファイルシステム)  | ホスト (ホストファイルシステム) |
| インストール | ホスト                  | ホスト               |
| インストール | ホスト                  | ホスト               |
| インストール | ホスト (ホストファイルシステム)    | ホスト (ホストファイルシステム) |
| インストール | ホスト (ホストファイルシステム)    | ホスト               |
| インストール | ホスト/ホスト              | ホスト               |

```

#####LVM Storage Operator#####

```

1111

- `HostPathProvisioner` `HP`
- `HP` `StorageClass` `SC`
- `SC` `Volume` `Vol`

- PVC 使用 VM 使用

前提条件:

OpenShift 4.21+  
OpenShift Virtualization 4.21+

步骤

- **OpenShift 4.21+** (OpenShift Virtualization 使用)
- 存储类: **HostPathProvisioner** 使用 **StorageClass** 使用
- 虚拟机: **VM** 使用 **Worker Node** 使用
- **oc CLI**: 使用
- **virtctl CLI**: VM 使用

OpenShift Virtualization 使用

```
oc get csv -n openshift-cnv | grep kubevirt
```

Operator 使用 (使用)

```
kubevirt-hyperconverged-operator.v4.x.x    OpenShift Virtualization    4.x.x    Succeeded
```

步骤 1: 使用

HPP 使用 SELinux 使用

步骤 A: **MachineConfig** 使用 (使用)

MachineConfig 使用 Worker Node 使用

**hpp-machineconfig.yaml** 使用

```
oc apply -f - <<EOF
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: 50-hpp-directory
  labels:
    machineconfiguration.openshift.io/role: worker
spec:
  config:
    ignition:
      version: 3.2.0
    systemd:
```

```
units:
- name: create-hpp-directory.service
  enabled: true
  contents: |
    [Unit]
    Description=Create HostPath Provisioner directory
    Before=kubelet.service

    [Service]
    Type=oneshot
    ExecStart=/bin/mkdir -p /var/hpvolumes
    ExecStart=/bin/chcon -Rt container_file_t /var/hpvolumes
    RemainAfterExit=yes

    [Install]
    WantedBy=multi-user.target
```

EOF



MachineConfig

Worker

Node

MachineConfig

MachineConfig

```
oc get mcp worker
```

UPDATED True UPDATING False

| NAME     | CONFIG                 | UPDATED           | UPDATING |
|----------|------------------------|-------------------|----------|
| DEGRADED | MACHINECOUNT           | READYMACHINECOUNT |          |
| worker   | rendered-worker-abc123 | True              | False    |
| 6        | 6                      |                   | False    |

**B:**

Worker Node

```
oc get nodes -l node-role.kubernetes.io/worker
```

Worker Node (<worker-node-name>)

```
oc debug node/<worker-node-name> -- chroot /host mkdir -p /var/hpvolumes
```

```
oc debug node/<worker-node-name> -- chroot /host chcon -Rt container_file_t
```

/var/hpvolumes



MachineConfig

SELinux

```
oc debug node/<worker-node-name> -- chroot /host ls -laZ /var/hpvolumes
```

```
total 0
drwxr-xr-x. 2 root root system_u:object_r:container_file_t:s0 6 Mar 19 10:00 .
drwxr-xr-x. 22 root root system_u:object_r:var_t:s0 4096 Mar 19 10:00 ..
```

container\_file\_t SELinux

## 2: HostPathProvisioner

HostPathProvisioner CSI

hpp-hostpathprovisioner.yaml

```
oc apply -f - <<EOF
apiVersion: hostpathprovisioner.kubevirt.io/v1beta1
kind: HostPathProvisioner
metadata:
  name: hostpath-provisioner
spec:
  imagePullPolicy: IfNotPresent
  storagePools:
  - name: local
    path: /var/hpvolumes
  workload:
    nodeSelector:
      kubernetes.io/os: linux
EOF
```

- storagePools:** 1 (name: StorageClass path: workload.nodeSelector: HPP CSI Pod

- **imagePullPolicy:** `Always`

## HostPathProvisioner

HostPathProvisioner

```
oc get hostpathprovisioner hostpath-provisioner
```

Output:

| NAME                 | AGE |
|----------------------|-----|
| hostpath-provisioner | 30s |

CSI Pod on Worker Node

```
oc get pods -n openshift-cnv -l k8s-app=hostpath-provisioner
```

Output (Worker Node has 1 Pod):

| NAME                           | READY | STATUS  | RESTARTS | AGE |
|--------------------------------|-------|---------|----------|-----|
| hostpath-provisioner-csi-xxxxx | 4/4   | Running | 0        | 2m  |
| hostpath-provisioner-csi-yyyyy | 4/4   | Running | 0        | 2m  |
| hostpath-provisioner-csi-zzzzz | 4/4   | Running | 0        | 2m  |

## 3: StorageClass

HostPath Provisioner StorageClass

**hpp-storageclass.yaml**

```
oc apply -f - <<EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: hostpath-csi
provisioner: kubevirt.io.hostpath-provisioner
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
parameters:
  storagePool: local
EOF
```

Output:

- **provisioner:** `kubevirt.io.hostpath-provisioner`

- **reclaimPolicy: Delete** PVC 삭제 시 PV도 삭제 Retain PVC 삭제 시 PV는 삭제되지 않음
- **volumeBindingMode: WaitForFirstConsumer** Pod가 PVC를 참조할 때 PVC가 먼저 Bind된 후 Pod가 시작됨
- **parameters.storagePool:** HostPathProvisioner를 사용할 때 HostPath를 지정

## StorageClass 확인

```
oc get storageclass hostpath-csi
```

결과:

| NAME                 | PROVISIONER                      | RECLAIMPOLICY | VOLUMEBINDINGMODE    |
|----------------------|----------------------------------|---------------|----------------------|
| hostpath-csi         | kubevirt.io.hostpath-provisioner | Delete        | WaitForFirstConsumer |
| ALLOWVOLUMEEXPANSION | AGE                              |               |                      |
| false                | 10s                              |               |                      |

StorageClass의 기본값을 확인:

HPP가 StorageClass의 기본값을 설정하는 방법 (StorageClass의 기본값을 설정하는 방법)

```
oc patch storageclass hostpath-csi -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```



StorageClass가 1개만 있는 경우, StorageClass의 기본값을 설정하는 방법

```
# StorageClass의 기본값을 설정하는 방법
oc patch storageclass <existing-default> -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
```

## 4. HPP를 사용하여 PVC를 생성

PVC를 생성하기 위해 HPP를 사용

**hpp-test-pvc.yaml** 파일 생성

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: hpp-test-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
```



```
resources:
  requests:
    storage: 5Gi
storageClassName: hostpath-csi
EOF
```

PVC 確認

```
oc get pvc hpp-test-pvc
```

WaitForFirstConsumer 状態PVC の Pod 状態は Pending 状態

| NAME                  | STATUS  | VOLUME | CAPACITY | ACCESS MODES | STORAGECLASS |
|-----------------------|---------|--------|----------|--------------|--------------|
| VOLUMEATTRIBUTESCLASS |         | AGE    |          |              |              |
| hpp-test-pvc          | Pending |        |          |              | hostpath-csi |
| 5s                    |         |        |          |              | <unset>      |

Pod 作成

```
oc apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: hpp-test-pod
  namespace: default
spec:
  containers:
    - name: test
      image: registry.access.redhat.com/ubi9/ubi:latest
      command: ["sleep", "3600"]
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: hpp-test-pvc
EOF
```

PVC 確認

```
oc get pvc hpp-test-pvc
```

結果:



```

    cpu:
      cores: 1
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
      interfaces:
        - name: default
          masquerade: {}
    resources:
      requests:
        memory: 1Gi
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        dataVolume:
          name: hpp-test-vm-rootdisk
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            user: fedora
            password: fedora
            chpasswd:
              expire: false
    dataVolumeTemplates:
      - metadata:
          name: hpp-test-vm-rootdisk
        spec:
          storage:
            accessModes:
              - ReadWriteOnce
            resources:
              requests:
                storage: 10Gi
            storageClassName: hostpath-csi
          source:
            registry:
              url: docker://quay.io/containerdisks/fedora:latest
EOF

```

VM DataVolume

```
# DataVolume
```

```
oc get dv -n default hpp-test-vm-rootdisk -w
```

□□□□□□:

| NAME                 | PHASE            | PROGRESS | RESTARTS | AGE |
|----------------------|------------------|----------|----------|-----|
| hpp-test-vm-rootdisk | ImportScheduled  | N/A      |          | 5s  |
| hpp-test-vm-rootdisk | ImportInProgress | 0.00%    |          | 10s |
| hpp-test-vm-rootdisk | ImportInProgress | 45.50%   |          | 30s |
| hpp-test-vm-rootdisk | Succeeded        | 100.0%   |          | 1m  |

VM □□□□□□□□□□□□

```
oc get vm hpp-test-vm
oc get vmi hpp-test-vm
```

□□□:

| NAME        | AGE | STATUS  | READY |
|-------------|-----|---------|-------|
| hpp-test-vm | 2m  | Running | True  |

| NAME        | AGE | PHASE   | IP       | NODENAME | READY |
|-------------|-----|---------|----------|----------|-------|
| hpp-test-vm | 2m  | Running | 10.x.x.x | workerXX | True  |

VM □□□□□□□□□□□□

```
virtctl console hpp-test-vm
```

VM □□□□□□□□□□□□

```
oc delete vm hpp-test-vm
```

□□□□□□□□□□□□

**PVC □ Pending** □□□□□□

Pod □□□□ PVC □ Pending □□□:

```
# PVC □□□□□□
oc describe pvc <pvc-name>

# HPP Pod□□□□□□
oc logs -n openshift-cnv -l k8s-app=hostpath-provisioner -c hostpath-provisioner
```

□□□□□:

- **Container Engine**: `containerd` / `var/hpvolumes` `containerd`
- **SELinux** `chcon -Rt container_file_t /var/hpvolumes`
- **HPP Pod** `HPP CSI Pod` `Worker Node`

□□□□ (Permission Denied) □□□

```
# SELinux 設定
oc debug node/<worker-node-name> -- chroot /host ls -laZ /var/hpvolumes

# SELinux 設定
oc debug node/<worker-node-name> -- chroot /host chcon -Rt container_file_t
/var/hpvolumes
```

## HPP Pod □□□□□□

HostPathProvisioner □□□□□□□□□□□□□□□□

```
oc describe hostpathprovisioner hostpath-provisioner

# openshift-cnv namespace
oc get events -n openshift-cnv --sort-by='.lastTimestamp' | tail -20
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

**WaitForFirstConsumer** Pod:

```
# Pod selector
spec:
  nodeSelector:
    kubernetes.io/hostname: <worker-node-name>
```

□ □ □ □ □ □ □

□□□□□□ HPP □□□□□□:

```
# StorageClass
oc delete storageclass hostpath-csi

# HostPathProvisioner
oc delete hostpathprovisioner hostpath-provisioner

# 確認: MachineConfig (確認済み)
```



HostPathProvisioner      PV      PVC

111

□□□□□□:

- OpenShift において HostPath Provisioner 提供
- HPP によって StorageClass 提供
- SELinux によって Pod 隔離
- Pod 内で VM 実行 HPP 提供
- HPP によって Pod 隔離

# HPH LVM Storage Operator

11

- **LVM Storage Operator** - [LVM Storage Operator](#)
- **LVM Storage** [LVM Storage](#) - [LVM Storage](#)
- **OpenShift Virtualization: Configuring Local Storage with HPP**

## StorageClass

11

OpenShift Container Platform (OCP) StorageClass  
 Container Storage Interface (CSI) OCP 4.x (OC Pv4  
 StorageClass OpenShift  
 Virtualization (CNV) StorageClass OpenShift storage class api  
 Container Storage Interface

## StorageClass

| OpenShift                                                       | StorageClass (SC)                                               |
|-----------------------------------------------------------------|-----------------------------------------------------------------|
| <pre> storageClassName: "fast-ssd",                     </pre>  | <pre> storageClassName: "fast-ssd",                     </pre>  |
| <pre> storageClassName: "cheap-hdd",                     </pre> | <pre> storageClassName: "cheap-hdd",                     </pre> |

PersistentVolumeClaim (PVC)    StorageClass    PersistentVolume (PV)

## StorageClass

- **Provisioner:** `csi.aws.com` (AWS EBS) `nfs.csi.k8s.io` (NFS)
- **Parameters:** `type: gp3, encrypted: "true"`
- **ReclaimPolicy:** PVC 삭제 시 PV도 삭제 (Delete) 또는 보존 (Retain)
- **VolumeBindingMode:** Immediate, WaitForFirstConsumer
- **AllowVolumeExpansion:** true (PVC 용량 증가 허용)

## Volume Binding Modes

Volume Binding Modes (LVM/HostPath)는 Pod가 PVC를 요청할 때의 동작 방식을 결정합니다.

### 1. Immediate

- Pod가 PVC를 요청하면 즉시 PV를 프로비저닝하고 Pod에 마운트합니다.
- 예: `AWS us-east-1a, 1b, 1c` 리전/가용 영역에 Pod A가 생성되면 즉시 Pod B가 생성될 수 있습니다.
- 단점: 리전/가용 영역에 Pod가 생성될 때까지 Pod가 실행되지 않습니다.

### 2. WaitForFirstConsumer (WaitForFirstConsumer)

- Pod가 PVC를 요청하면 Pod가 실행될 때까지 PV를 프로비저닝하지 않습니다.
- 예: Pod A가 PVC를 요청하면 Pod B가 실행될 때까지 Pod A가 실행되지 않습니다. (Pod B가 실행되면 Pod A가 실행됩니다.)
- 단점: LVM/HostPath (Multi-AZ)에서는 Pod가 실행될 때까지 Pod가 실행되지 않습니다.

## Reclaim Policies (Reclaim Policies)

PersistentVolumeClaim (PVC)의 Reclaim Policy는 PVC가 삭제될 때의 동작 방식을 결정합니다.

### 1. Delete (Delete)

- PVC가 삭제되면 PV도 삭제됩니다. (AWS EBS 등)
- 예: CI/CD 파이프라인에서 PVC가 삭제되면 PV도 삭제됩니다.

### 2. Retain (Retain)

- PVC가 삭제되면 PV는 Released 상태로 변경됩니다.
- 예: PV가 Released 상태이면 다른 Pod가 PV를 요청할 수 있습니다.
- 단점: PV가 Released 상태이면 다른 Pod가 PV를 요청할 수 있습니다.

## StorageClass

StorageClass를 설정하려면 `storageClassName` 필드를 PVC에 추가합니다.

```
oc patch storageclass <your-class-name> -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
```

```
oc patch storageclass <old-default-name> -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "false"}}}'
```

Tip: SC 1 is the Default Storage Class

## StorageClass ☐☐

□□□□ A: AWS EBS (□□□□) CSI □□□□□□□□□□ gp3 □□□□□□□□□□□□□□□□

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3-fast
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
reclaimPolicy: Delete
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
```

OpenShift 文档: [https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/storage/using-container-storage-interface-csi#persistent-storage-csi-efs](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/using-container-storage-interface-csi#persistent-storage-csi-efs)

[illegible]

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-lvm-storage
provisioner: topolvm.io # Depends on the specific operator used
volumeBindingMode: WaitForFirstConsumer # Critical for local storage!
reclaimPolicy: Retain
parameters:
  "csi.storage.k8s.io/fstype": "xfs"
```



# StorageClass LVMS Operator

□□□□ C: NFS (□□□□□□□□ / RWX) NFS CSI Driver Operator □□□□□□ NFS □□□□□□□□□□□□

```
apiVersion: storage.k8s.io/v1
```



```
kind: StorageClass
metadata:
  name: nfs-shared
provisioner: nfs.csi.k8s.io
parameters:
  server: nfs-server.example.com
  share: /exports/data
reclaimPolicy: Delete
volumeBindingMode: Immediate
mountOptions:
  - nfsvers=4.1
```

OpenShift 文档: [https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/storage/persistent-storage-using-local-storage](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/persistent-storage-using-local-storage)

验证存储类配置

StorageClass 验证:

```
oc get sc
# 验证 (default) 存储类
```

验证:

```
oc describe sc <name>
```

验证 PVC 配置:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc
spec:
  accessModes: [ "ReadWriteOnce" ]
  resources: { requests: { storage: 1Gi } }
  storageClassName: gp3-fast
```

OpenShift 文档: [https://docs.redhat.com/en/documentation/openshift\\_container\\_platform/4.20/html/storage/configuring-persistent-storage](https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/storage/configuring-persistent-storage)

验证 PVC 配置

- 1: PVC 在 Pending 状态
  - A: volumeBindingMode 为 WaitForFirstConsumer 时，PVC 等待 Pod 绑定
    - 验证: PVC 绑定 Pod 成功

- [illegible]

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- OpenShift `StorageClass` `Provisioner``ReclaimPolicy``VolumeBindingMode``VolumeExpansion` `StorageClass` `Immediate` `WaitForFirstConsumer` `Volume Binding Modes`
- AWS EBS`Local LVM``NFS` `StorageClass`
- PVC `Pending`

11

- [OpenShift - Configuring Persistent Storage](#)
- [OpenShift - Container Storage Interface](#)

□□□□□□ VM

11

Storage Class (VM) OS  
OS VM

2. VM 1 StorageClass 1 2 StorageClass  
OS

□ □ □ □

- **OpenShift 4.18** (OpenShift Virtualization Operator)
- (system:admin)
- oc CLI virtctl CLI
- StorageClass (lvms-vg1)

□□□□□□□□□□:

OpenShift 4.20, 4.21

**□ □ □ □ 1:** □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

KubeVirt - VM spec [][][][]

- **spec.template.spec.domain.devices.disks:** OS `disk` (OS `disk`) `disk`
- **spec.template.spec.volumes:** `PVC` `DataVolume` `containerDisk` `cloudInitNoCloud` `cloudInitNoCloud`
- `name` `name` `name`

OS `/dev/vda, /dev/vdb`

## 2: namespace

```
oc create namespace multi-disk-demo
```

### 3: 2 StorageClass (StorageClass)

```
StorageClass 1
StorageClass 2
```

LVM `lvms-vg1` StorageClass `Retain(2)` reclaim policy `StorageClass`

## sc-data-retain.yaml

```
oc apply -f - <<EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: lvms-vg1-retain
provisioner: topolvm.io
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

EOF

看看 StorageClass 配置

```
oc get storageclass
```

输出结果:

| NAME               | PROVISIONER | RECLAIMPOLICY | VOLUMEBINDINGMODE    | ALLOWVOLUMEEXPANSION | AGE |
|--------------------|-------------|---------------|----------------------|----------------------|-----|
| lvms-vg1 (default) | topolvm.io  | Delete        | WaitForFirstConsumer | true                 | 30d |
| lvms-vg1-retain    | topolvm.io  | Retain        | WaitForFirstConsumer | true                 | 5s  |

步骤 4: 创建虚拟机 VM

3 个虚拟机 VM

- **rootdisk:** 根磁盘 (30Gi default StorageClass) Fedora 操作系统
- **datadisk:** 数据磁盘 (10Gi 2 个 StorageClass) 数据磁盘
- **cloudinitdisk:** cloud-init 配置



cloud-init  
配置

cloud-init

配置 SSH 访问

vm-multi-disk.yaml 配置

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: multi-disk-vm
  namespace: multi-disk-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: multi-disk-vm
    spec:
      domain:
        devices:
          disks:
            - name: rootdisk
              disk:
```

```

        bus: virtio
        bootOrder: 1
      - name: datadisk
        disk:
          bus: virtio
      - name: cloudinitdisk
        disk:
          bus: virtio
        interfaces:
      - name: default
        masquerade: {}
    resources:
      requests:
        memory: 1Gi
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        dataVolume:
          name: multi-disk-rootdisk
      - name: datadisk
        dataVolume:
          name: multi-disk-datadisk
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            password: redhat
            chpasswd:
              expire: false
    dataVolumeTemplates:
      - metadata:
          name: multi-disk-rootdisk
        spec:
          pvc:
            accessModes:
              - ReadWriteOnce
            resources:
              requests:
                storage: 30Gi
            storageClassName: lvms-vg1
          source:
            registry:
              pullMethod: node
              url: docker://quay.io/containerdisks/fedora:latest
      - metadata:
          name: multi-disk-datadisk
        spec:
          pvc:
            accessModes:

```

```

- ReadOnlyOnce
resources:
  requests:
    storage: 10Gi
  storageClassName: lvms-vg1-retain
source:
  blank: {}
EOF

```

□□□□□□□□

- rootdisk □□ **bootOrder: 1** □□VM □ OS □□□□□□□□□□□□□□□□□□
- rootdisk □□ **source.registry** □□Fedora □□□□□□□□□□□□□□□□ Pull □□□□
- datadisk □□ **source.blank: {}** □□□□□□□□□□□□□□□□□□□□□□□□
- □ **dataVolumeTemplates** □□□□□□□□□□ **storageClassName** □□□□□□□□
- **devices.disks** □□□□□□□□ **volumes** □□□□□□□□□□□□□□□□ **dataVolumeTemplates** □□□□□□□□□□□□□□□□□□□□

□□□□ **5: VM** □□□□□□□□□□

□□□ DataVolume □□□□□□VM □□□□□□□□□□□□

```
oc get dv -n multi-disk-demo -w
```

□□□ **Succeeded** □□□□□□□□VM □□□□□□□□

```
oc get vm -n multi-disk-demo
```

□□□□□□□□:

| NAME          | AGE | STATUS  | READY |
|---------------|-----|---------|-------|
| multi-disk-vm | 3m  | Running | True  |

PVC □□□□□□□□ StorageClass □□□□□□□□□□□□□□□□□□□□

```
oc get pvc -n multi-disk-demo
```

□□□□□□□□:

| NAME                | STATUS | VOLUME   | CAPACITY | ACCESS MODES | STORAGECLASS    |
|---------------------|--------|----------|----------|--------------|-----------------|
| multi-disk-rootdisk | Bound  | pvc-xxxx | 30Gi     | RWO          | lvms-vg1        |
| multi-disk-datadisk | Bound  | pvc-yyyy | 10Gi     | RWO          | lvms-vg1-retain |

3m

## 📁 6: VM 📁📁📁📁📁📁📁📁📁📁📁📁📁📁📁📁

VM 📁📁📁📁📁📁📁📁

```
virtctl console multi-disk-vm -n multi-disk-demo
```

📁📁📁 **fedora**📁📁📁 **redhat** 📁📁📁📁📁

📁📁📁📁📁📁📁📁📁📁📁📁📁📁📁📁

```
lsblk
```

📁📁📁📁📁:

| NAME   | MAJ:MIN | RM | SIZE  | RO | TYPE        | MOUNTPOINTS |
|--------|---------|----|-------|----|-------------|-------------|
| vda    | 252:0   | 0  | 30G   | 0  | disk        |             |
| -vda1  | 252:1   | 0  | 1M    | 0  | part        |             |
| -vda2  | 252:2   | 0  | 1.9G  | 0  | part [SWAP] |             |
| -vda3  | 252:3   | 0  | 1G    | 0  | part /boot  |             |
| `-vda4 | 252:4   | 0  | 27.1G | 0  | part /      |             |
| vdb    | 252:16  | 0  | 10G   | 0  | disk        |             |
| vdc    | 252:32  | 0  | 1M    | 0  | disk        |             |

📁📁📁📁 **vdb**(10G📁📁📁📁📁) 📁📁📁📁📁📁📁 XFS 📁📁📁📁📁📁

```
sudo mkfs.xfs /dev/vdb
```

📁📁📁📁📁📁📁📁📁📁📁📁📁📁

```
sudo mkdir -p /mnt/data
sudo mount /dev/vdb /mnt/data
```

📁📁📁📁📁📁

```
df -h /mnt/data
```

📁📁📁📁📁:

| Filesystem | Size | Used | Avail | Use% | Mounted on |
|------------|------|------|-------|------|------------|
| /dev/vdb   | 10G  | 105M | 9.9G  | 2%   | /mnt/data  |

確認ディレクトリが作成されている

```
sudo sh -c "echo 'Multi-disk test data' > /mnt/data/test.txt"
```

確認ファイルが作成されている

```
cat /mnt/data/test.txt
```

確認/etc/fstabに書き込まれている

```
sudo sh -c "echo '/dev/vdb /mnt/data xfs defaults 0 0' >> /etc/fstab"
```

**Ctrl+] 確認**

**ステップ 7: 確認**

PVC StorageClass PersistentVolume 確認

```
oc get pvc multi-disk-rootdisk -n multi-disk-demo -o jsonpath='{StorageClass: {spec.storageClassName}, Volume: {spec.volumeName}, Capacity: {status.capacity.storage}]'
```

```
oc get pvc multi-disk-datadisk -n multi-disk-demo -o jsonpath='{StorageClass: {spec.storageClassName}, Volume: {spec.volumeName}, Capacity: {status.capacity.storage}]'
```

確認 (確認) 確認

**VM 確認**

| 確認    | 確認            | 確認       |
|-------|---------------|----------|
| 確認    | SSD (OS + 確認) | HDD (確認) |
| 確認    | OS            | SSD (確認) |
| VM    | OS (確認)       | 確認 (確認)  |
| 確認/確認 | OS            | 確認 (確認)  |

確認

namespace 確認VM PVC DataVolume 確認

```
oc delete namespace multi-disk-demo
```



## 2. Delete StorageClass

```
oc delete storageclass lvms-vg1-retain
```

`lvms-vg1-retain` StorageClass の `reclaimPolicy: Retain` によって PVC が PersistentVolume としてリリースされた PV が存在する

```
oc get pv | grep Released
```

リリースされた PV を削除する

```
oc delete pv <pv-name>
```

## まとめ

OpenShift 4.10 以降の VM に関する変更点

- StorageClass が VM によって使用される
- `dataVolumeTemplates` が VM によって使用される
- VM が OpenShift によって管理される
- VM が OpenShift によって管理される
- OS が VM によって管理される
- VM が OpenShift によって管理される

## 参考

- [Disks and Volumes - KubeVirt User Guide](#)
- [Configuring Storage Profiles - OpenShift Documentation](#)
- [OpenShift 4.10 以降の VM に関する変更点](#)
- [LVM Storage Operator](#)

## VM のインストール

### 前提

OpenShift 4.10 以降の VM に関する変更点 (PVC) は OpenShift Virtualization の Day 2 の機能です。1. OpenShift Virtualization のインストールとアップグレード

OpenShift (VM あり) のインストール (VM あり) は VM が OpenShift によって管理される OS が OpenShift によって管理される OS であることが前提です。

## 前提条件

- OpenShift Virtualization Operator バージョン **OpenShift 4.18+**
- 管理者権限 (system:admin ユーザー)
- OpenShift CLI **oc** または **virtctl** CLI
- `allowVolumeExpansion: true` の **StorageClass**
- PVC を作成する **VM**

## ステップ 1: StorageClass を確認する

VM を作成する前に PVC を作成する StorageClass を確認する必要があります。

```
oc get storageclass -o custom-  
columns=NAME:.metadata.name,PROVISIONER:.provisioner,EXPANSION:.allowVolumeExpansion
```

例:

| NAME     | PROVISIONER | EXPANSION |
|----------|-------------|-----------|
| lvms-vg1 | topolvm.io  | true      |



StorageClass が VM の PVC を作成する StorageClass の `EXPANSION` が `true` であることを確認します。

StorageClass の `EXPANSION` が `false` または `<none>` の場合は、

```
oc patch storageclass <your-storage-class> -p '{"allowVolumeExpansion": true}'
```



StorageClass の `allowVolumeExpansion` が `true` であることを確認し、CSI (HostPath) を使用している StorageClass の場合は、`allowVolumeExpansion` が `true` であることを確認します。

## ステップ 2: Namespace を作成する VM

VM を作成する namespace を作成します。

```
oc create namespace pvc-expansion-demo
```

1 つの 10Gi の Fedora VM を作成します。

- **rootdisk**: Fedora (10Gi `lvms-vg1` StorageClass)
- **cloudinitdisk**: Cloud-init



cloud-init を利用して SSH 接続できるようにする

## vm-expansion.yaml の内容

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: expansion-vm
  namespace: pvc-expansion-demo
spec:
 instancetype:
    name: u1.small
  preference:
    name: fedora
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: expansion-vm
    spec:
      domain:
        devices:
          disks:
            - disk: {}
              name: rootdisk
            - disk: {}
              name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
        networks:
          - name: default
            pod: {}
        volumes:
          - name: rootdisk
            dataVolume:
              name: expansion-vm-rootdisk
          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |
                #cloud-config
                password: redhat
                chpasswd:
                  expire: false
      dataVolumeTemplates:
        - apiVersion: cdi.kubevirt.io/v1beta1
          kind: DataVolume
          metadata:
            name: expansion-vm-rootdisk
```

```
spec:
  sourceRef:
    kind: DataSource
    name: fedora
    namespace: openshift-virtualization-os-images
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 10Gi
    storageClassName: lvms-vg1
```

EOF

DataVolume `expansion-vm` VM `expansion-vm`

```
oc get dv -n pvc-expansion-demo -w
```

DataVolume `expansion-vm` **Succeeded** VM `expansion-vm`

```
oc get vm -n pvc-expansion-demo
```

expansion-vm:

| NAME         | AGE | STATUS  | READY |
|--------------|-----|---------|-------|
| expansion-vm | 3m  | Running | True  |

expansion-vm PVC `expansion-vm-rootdisk`

```
oc get pvc -n pvc-expansion-demo
```

expansion-vm-rootdisk:

| NAME                  | STATUS | VOLUME   | CAPACITY | ACCESS MODES | STORAGECLASS |
|-----------------------|--------|----------|----------|--------------|--------------|
| expansion-vm-rootdisk | Bound  | pvc-xxxx | 10Gi     | RWO          | lvms-vg1     |



PVC `expansion-vm-rootdisk` `dataVolumeTemplates` `expansion-vm-rootdisk` `DataVolume` `expansion-vm-rootdisk` VM `expansion-vm`  
`oc get vm expansion-vm -n pvc-expansion-demo -o jsonpath='{.spec.template.spec.volumes}'` `expansion-vm-rootdisk` PVC `expansion-vm-rootdisk`

### ステップ 3: VM を停止 (VM 停止)

VM を停止させるには、PVC を再サイズアップして VM を再起動します。

#### 1. VM を停止します。

```
virtctl stop expansion-vm -n pvc-expansion-demo
```

#### 2. VM の状態を確認します。

```
oc get vm expansion-vm -n pvc-expansion-demo
```

結果:

| NAME         | AGE | STATUS  | READY |
|--------------|-----|---------|-------|
| expansion-vm | 5m  | Stopped | False |

#### 1. PVC を 10Gi から 20Gi に変更します。

```
oc patch pvc expansion-vm-rootdisk -n pvc-expansion-demo --type merge -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'
```



変更された PVC は **FileSystemResizePending** condition を持ち、Pod が PVC を再サイズアップするのを待っています。VM は再起動されるまで停止したままです。

#### 2. VM を再起動します。

```
virtctl start expansion-vm -n pvc-expansion-demo
```

#### 3. VM の状態を確認し、PVC の状態を確認します。

```
oc get vm expansion-vm -n pvc-expansion-demo
```

結果:

| NAME         | AGE | STATUS  | READY |
|--------------|-----|---------|-------|
| expansion-vm | 6m  | Running | True  |

+

```
oc get pvc expansion-vm-rootdisk -n pvc-expansion-demo
```

$$+ \square\square\square:$$

| NAME                  | STATUS | VOLUME   | CAPACITY | ACCESS | MODES | STORAGECLASS |
|-----------------------|--------|----------|----------|--------|-------|--------------|
| AGE                   |        |          |          |        |       |              |
| expansion-vm-rootdisk | Bound  | pvc-xxxx | 20Gi     | RWO    |       | lvms-vg1     |
| 6m                    |        |          |          |        |       |              |

#### Figure 4: VM (Virtual Machine)

CSI VM PVC 3  
 00000000

[illegible][illegible]

VM 00000 PVC 0000000000 VM 00000 PVC 0000000000000000

```
oc patch pvc expansion-vm-rootdisk -n pvc-expansion-demo --type merge -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'
```

PVC □□□□□□□□□□□□□□□□

```
oc describe pvc expansion-vm-rootdisk -n pvc-expansion-demo | grep -A 5 Conditions
```

CSI 00000000000000000000000000000000000000 PVC 0000000000

```
Conditions:
  Type                Status
  ----                -
  FileSystemResizePending  False
```

[illegible]

```
Conditions:
  Type                Status
  ----                -
  FileSystemResizePending  True
```



Filesystem PVC を KubeVirt を利用して仮想マシンに展開し、OS をインストールする

## ステップ 5: OS のインストール

PVC を VM に展開し、OS をインストールする

VM を起動する

```
virtctl console expansion-vm -n pvc-expansion-demo
```

Fedora をインストールする

lsblk

(Fedora をインストール)

Fedora をインストールする

```
NAME      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
zram0     251:0    0   883M  0 disk [SWAP]
vda       253:0    0  18.6G  0 disk
|-vda1    253:1    0     2M  0 part
|-vda2    253:2    0  100M  0 part /boot/efi
`-vda3    253:3    0  18.5G  0 part /var
                               /home
                               /boot
                               /
vdb       253:16   0     1M  0 disk
```



20G の PVC を作成し、18.6G の VM に展開する

df -h /

df:

| Filesystem | Size | Used | Avail | Use% | Mounted on |
|------------|------|------|-------|------|------------|
| /dev/vda3  | 19G  | 759M | 18G   | 5%   | /          |

ディスクパーティションのサイズを確認する。OS ディスクのパーティションのサイズを確認する。6  
パーティションのサイズ

パーティションのサイズ (RHEL/CentOS ディスクのパーティションのサイズ)

パーティションのサイズ OS ディスクのパーティションのサイズ lsblk  
パーティションのサイズ ディスクのパーティションのサイズ 2  
パーティションのサイズ

lsblk ディスクのパーティションのサイズ / ディスクのパーティションのサイズ 3  
(Fedora) ディスクのパーティションのサイズ 4 (RHEL/CentOS) ディスクのパーティションのサイズ 2 (minimal ディスク) ディスクのパーティションのサイズ /dev/vda4  
パーティションのサイズ

パーティションのサイズ

growpart ディスクのパーティションのサイズ

```
sudo growpart /dev/vda 4
```

パーティションのサイズ:

```
CHANGED: partition=4 start=6094848 old: size=14882816 end=20977664 new: size=35756032  
end=41850880
```



growpart ディスク NOCHANGE: partition cannot be grown  
パーティションのサイズ

パーティションのサイズ

パーティションのサイズ ディスクのパーティションのサイズ

XFS (Fedora/RHEL ディスク) ディスク xfs\_growfs ディスク XFS ディスクのパーティションのサイズ

```
sudo xfs_growfs /
```

ext4 ディスクのパーティションのサイズ resize2fs ディスクのパーティションのサイズ

```
sudo resize2fs /dev/vda4
```



xfs\_growfs ディスク resize2fs  
パーティションのサイズ



1. 创建虚拟机

```
df -h /
```

2. 检查磁盘空间

Ctrl+] 退出虚拟机

步骤 6: 配置虚拟机

3. 配置虚拟机 (PVC)

| 操作          | OS 命令                                        | 说明                             |
|-------------|----------------------------------------------|--------------------------------|
| 创建虚拟机 (PVC) | growpart<br>xfs_growfs<br>resize2fs<br>(PVC) | lsblk<br>PVC<br>EFI            |
| 配置虚拟机 (PVC) | growpart<br>xfs_growfs<br>resize2fs          | lsblk<br>(/dev/vdb1)           |
| 配置虚拟机 (PVC) | xfs_growfs<br>resize2fs                      | growpart                       |
| 配置虚拟机 (PVC) | mkfs.xfs /dev/vdb<br>mkfs.ext4               | Multi-Disk<br>VM Configuration |



4. 配置虚拟机 (PVC)

步骤 7: 配置

5. 配置虚拟机

PVC

```
oc get pvc expansion-vm-rootdisk -n pvc-expansion-demo
```

6.

| NAME                  | STATUS | VOLUME   | CAPACITY | ACCESS MODES | STORAGECLASS |
|-----------------------|--------|----------|----------|--------------|--------------|
| AGE                   |        |          |          |              |              |
| expansion-vm-rootdisk | Bound  | pvc-xxxx | 20Gi     | RWO          | lvms-vg1     |

□□□□□□□□□□□□□□□□

```
virtctl console expansion-vm -n pvc-expansion-demo
```

```
lsblk
df -h /
```

```

root@kali:~# dd if=/dev/zero of=/dev/sda bs=1M count=20 (20GB) conv=fsync
20+0 records in
20+0 records out
20971520 bytes transferred (10485760 B/s) (0 seconds)
root@kali:~#

```

[illegible]

Ctrl+] 0000000000000000

□□□□□□□□

- **Pod** → **PVC** → **StorageClass** → **PVC** → **Filesystem**
- **CSI** → **VolumeGroup** → **Volume** → **Filesystem**
- **PVC** → **Pod** → **PVC** → **FileSystemResizePending** → **VM** → **StorageClass**
- **Fedora** → **RHEL** → **CentOS** → **growpart** → **xfs\_growfs/resize2fs**
- **VM** → **OS** → **VM** → **SCSI** → **StorageClass**

□□□□□□

Namespace `vm` `pvc` `dataVolume`

```
oc delete namespace pvc-expansion-demo
```

Namespace 命名空间

```
oc get namespace pvc-expansion-demo
```

□□□□□□□□:

```
Error from server (NotFound): namespaces "pvc-expansion-demo" not found
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- StorageClass 指定するストレージタイプ
- PVC は VM 指定するストレージタイプを指定する
- 指定するストレージタイプ (VM 指定) は VM 指定する
- 指定する VM 指定する (VM 指定) 指定する
- 指定 OS 指定するストレージタイプを指定する
- `growpart` `xfs_growfs` (XFS) `resize2fs` (ext4) 指定する OS 指定するストレージタイプを指定する
- 指定するストレージタイプを指定する
- PVC 指定するストレージタイプを指定する

11

- **PersistentVolume** 持久化 - OpenShift 持久化存储
- 虚拟机管理 - **KubeVirt** 虚拟机管理
- **StorageClass** 存储类
- 虚拟机 **VM** 管理

## CDI Operations

11

Containerized Data Importer (CDI) OpenShift Virtualization VM  
 HTTP URL PVC VM CDI PVC

```

#####CDI#####DataImportCron
#####

```

□ □ □ □

- **CDI** **OpenShift Virtualization** **Container**
- **HTTP** **Container** **PVC** **Container**
- **DataVolume** **Container**
- **DataImportCron** **Container**
- **CDI** **Container**
- **Container** **CDI** **Container**

## 前提条件

- OpenShift Virtualization Operator 版本要求 **OpenShift 4.18+**
- 使用 `system:admin` 权限 (root 用户)
- 使用 `oc CLI` 或 `virtctl CLI`
- 配置 `StorageClass` (如: `lvms-vg1`)

## 步骤 1: CDI 部署

CDI 在 `openshift-cnv` namespace 中部署 Pod 以提供虚拟化存储支持。

- **cdi-operator** — CDI 控制器 (管理 DataVolume) 的部署
- **cdi-apiserver** — DataVolume 的 API 接口
- **cdi-deployment (cdi-controller)** — DataVolume 的控制器，负责 importer 和 cloner pod 的管理
- **cdi-uploadproxy** — `virtctl image-upload` 命令的代理，负责将 PVC 数据上传到虚拟机

DataVolume 通过 CDI 控制器与 PVC 交互。worker pod (importer/cloner 或 uploader) 负责从 PVC 拉取数据并写入到虚拟机。当 DataVolume 状态为 **Succeeded** 时，PVC 上的数据即可用于 VM。

部署 CDI pod 的命令如下：

```
oc get pods -n openshift-cnv -l app.kubernetes.io/component=storage
```

输出结果：

| NAME                            | READY | STATUS  | RESTARTS | AGE |
|---------------------------------|-------|---------|----------|-----|
| cdi-apiserver-7f9c7b5b4-xk2lm   | 1/1   | Running | 0        | 5d  |
| cdi-deployment-6b8c5d8f9-r4tnp  | 1/1   | Running | 0        | 5d  |
| cdi-operator-5f4c7d8b6-jh9wz    | 1/1   | Running | 0        | 5d  |
| cdi-uploadproxy-7d9f8c6b5-m3kpq | 1/1   | Running | 0        | 5d  |

部署 CDI 控制器的命令如下：

```
oc get cdi cdi-kubevirt-hyperconverged
```

CDI 控制器部署成功后，即可开始创建 DataVolume。

## 步骤 2: DataVolume 部署

DataVolume 通过 PVC 与存储交互。PVC 通过 CDI 控制器与 DataVolume 交互。CDI 控制器负责管理 DataVolume 的部署。

1. 创建 PVC
2. PVC 通过 importer (或 cloner/uploader) pod 写入数据

3. `qcow2` or `raw` (gzip or xz) compression
4. `DataVolume` configuration
5. `DataVolume` is **Succeeded**

`DataVolume` lifecycle:

| State                            | Description                                                                  |
|----------------------------------|------------------------------------------------------------------------------|
| Pending                          | PVC is created                                                               |
| WaitForFirstConsumer             | PVC is in <b>WaitForFirstConsumer</b> state, waiting for a Pod to consume it |
| ImportScheduled                  | CDI controller schedules a worker pod to import the image                    |
| ImportInProgress                 | worker pod is importing the image                                            |
| CloneScheduled / CloneInProgress | CDI controller schedules a worker pod to clone the image from the PVC        |
| UploadScheduled / UploadReady    | <code>DataVolume</code> is <b>virtctl image-upload</b> ready                 |
| Succeeded                        | Image is successfully imported and cloned to the PVC                         |
| Failed                           | Import or clone operation failed, worker pod is terminated                   |

Run the following command in the namespace:

```
oc create namespace cdi-operations-demo
```

### Step 3: HTTP Import

HTTP import is a process where a `DataVolume` is created with a `source` of `http` and a `url` pointing to the image location. The `DataVolume` is then imported into the cluster using the `virtctl image-upload` command. The `DataVolume` is then cloned into a `PVC` and the `raw` image is imported into the `PVC`.

**`dv-http-import.yaml`** configuration:

```
oc apply -f - <<EOF
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: fedora-http
  namespace: cdi-operations-demo
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
    http:
      url:
        "https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2"
```

```
storage:
  resources:
    requests:
      storage: 10Gi
EOF
```



cdi.kubevirt.io/storage.bind.immediate.requested: "true" PVC  
WaitForFirstConsumer binding mode StorageClass  
Pod PVC Pending

```
oc get dv fedora-http -n cdi-operations-demo -w
```

| NAME        | PHASE            | PROGRESS | RESTARTS | AGE |
|-------------|------------------|----------|----------|-----|
| fedora-http | ImportScheduled  | N/A      |          | 5s  |
| fedora-http | ImportInProgress | 0.00%    |          | 15s |
| fedora-http | ImportInProgress | 22.45%   |          | 30s |
| fedora-http | ImportInProgress | 67.89%   |          | 60s |
| fedora-http | Succeeded        | 100.0%   |          | 90s |

DataVolume **Succeeded** **Ctrl+C**

importer pod:

```
oc get pods -n cdi-operations-demo -l
cdi.kubevirt.io/storage.import.importPvcName=fedora-http
```

```
oc logs -n cdi-operations-demo -l cdi.kubevirt.io/storage.import.importPvcName=fedora-http
```

PVC:

```
oc get pvc fedora-http -n cdi-operations-demo
```

| NAME        | STATUS | VOLUME   | CAPACITY | ACCESS | MODES | STORAGECLASS | AGE |
|-------------|--------|----------|----------|--------|-------|--------------|-----|
| fedora-http | Bound  | pvc-xxxx | 10Gi     | RWO    |       | lvms-vg1     | 2m  |

## □□□□ HTTP □□□□

HTTP Secret DataVolume:

```
oc create secret generic http-auth-secret \
--from-literal=username=myuser \
--from-literal=password=mypassword \
-n cdi-operations-demo
```

■■■DataVolume spec ■ Secret ■■■■■■:

```
spec:
  source:
    http:
      url: "https://private-server.example.com/images/rhel9.qcow2"
      secretRef: http-auth-secret
```

# HTTP 简介

HTTP 証明書 CA 証明書 ConfigMap

```
oc create configmap http-ca-cert \
  --from-file=ca-bundle.crt=/path/to/ca-bundle.crt \
  -n cdi-operations-demo
```

□□□□□□ DataVolume □□□□□□□□

```
spec:
  source:
    http:
      url: "https://internal-server.example.com/images/rhel9.qcow2"
      certConfigMap: http-ca-cert
```

□□□□ 4: □□□□□□□□□□□□□□□□

ContainerDisk CDI pull

dv-registry-import.yaml

```
oc apply -f - <<EOF
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: fedora-registry
  namespace: cdi-operations-demo
```

```

annotations:
  cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
    registry:
      url: "docker://quay.io/containerdisks/fedora:latest"
      pullMethod: node
  storage:
    resources:
      requests:
        storage: 10Gi
EOF

```

**pullMethod** 指定了 CDI 使用的 pull 方法

- **node** — 使用 node (CRI-O) 提供的 pull 方法
- **pod** — CDI 的 importer pod 提供的 pull 方法

**Succeeded** 表示成功

```
oc get dv fedora-registry -n cdi-operations-demo -w
```

DataVolume 的 **Succeeded** 状态可以通过 **Ctrl+C** 来取消

**步骤 5:** 上传 Fedora Cloud 镜像到 CDI

**virtctl image-upload** 命令用于将本地镜像上传到 CDI。该命令需要指定 PVC 的名称和镜像的路径。Web 界面中也可以上传镜像。

支持的镜像格式包括 qcow2 和 raw。Fedora Cloud 镜像是 qcow2 格式。

```

curl -LO
https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2

```

上传镜像的命令如下：

```

virtctl image-upload dv fedora-upload \
  --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
  --size=10Gi \
  --namespace=cdi-operations-demo \
  --insecure \
  --force-bind

```

命令输出：

- **dv fedora-upload** — **fedora-upload** 是 DataVolume 的名称 (dv 是 PVC 的缩写)



□□□□□□□□□□)

- **--size** — PVC サイズ (デフォルトは 1Gi)
- **--insecure** — TLS 証明書なしで API サーバーに接続する (デフォルトは false)
- **--force-bind** — WaitForFirstConsumer StorageClass の PVC を強制バインドする

□ □ □ □ □ □ □

```
PVC cdi-operations-demo/fedora-upload created
Uploading data to https://cdi-uploadproxy-openshift-cnvr.apps...

 420.00 MiB / 420.00 MiB [=====] 100.00% 1m30s

Uploading data completed successfully, waiting for processing to complete
Processing completed successfully
```

00:

```
oc get dv fedora-upload -n cdi-operations-demo
```

## Figure 6: PVC Polymerization

[illegible]

- **Host-assisted cloning** — CDI      pod      pod
- **Smart cloning** — CSI      CSI

创建 3 个名为 `fedora-http` PVC 存储卷

dv-pvc-clone.yaml

```
oc apply -f - <<EOF
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: fedora-clone
  namespace: cdi-operations-demo
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
    pvc:
      name: fedora-http
      namespace: cdi-operations-demo
  storage:
    resources:
```

```
requests:
  storage: 10Gi
EOF
```

□□□□□□□□

```
oc get dv fedora-clone -n cdi-operations-demo -w
```

☐ ☐ ☐ ☐ ☐

| NAME         | PHASE           | PROGRESS | RESTARTS | AGE |
|--------------|-----------------|----------|----------|-----|
| fedora-clone | CloneScheduled  | N/A      |          | 5s  |
| fedora-clone | CloneInProgress | 10.00%   |          | 15s |
| fedora-clone | Succeeded       | 100.0%   |          | 45s |

Successed Ctrl+C

namespace `std::`

```

// namespace PVC spec.source.pvc.namespace namespace CDI
// namespace PVC spec.source.pvc.namespace namespace PVC get list
// CDI RBAC

```

## 7: DataVolume

CDI DataVolume `.status` `cdiDataVolume`

```
oc get dv -n cdi-operations-demo -o custom-  
columns=NAME:.metadata.name,PHASE:.status.phase,PROGRESS:.status.progress,CONDITIONS:.  
status.conditions[0].reason
```

☐ ☐ ☐ ☐ ☐ ☐

000000000000 (000000000000HTTP 500000000000)CDI 000000000000000000000000 3 000000000000CDI  
 0000000000000000 (00000000) 00000000

[illegible]

```
oc get dv -n cdi-operations-demo -o custom-  
columns=NAME:.metadata.name,PHASE:.status.phase,RESTARTS:.status.restartCount
```

○○○○○○○○○○○○○○○○DataVolume ○ Failed ○○○○○○○○○○○ DataVolume ○○○○○○○○○○○○○○○○○○○

```
oc delete dv <failed-dv-name> -n cdi-operations-demo
```

oc describe dv fedora-http -n cdi-operations-demo

Events

Import into PVC started

Import Complete

Import Complete

## 8: DataImportCron

DataImportCron OpenShift Virtualization namespace openshift-virtualization-os-images (Fedora RHEL CentOS Stream Windows) DataImportCron

dataimportcron-fedora.yaml

```
oc apply -f - <<EOF
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataImportCron
metadata:
  name: fedora-image-cron
  namespace: cdi-operations-demo
spec:
  schedule: "0 3 * * 1"
  managedDataSource: fedora-latest
  garbageCollect: Outdated
  importsToKeep: 2
  template:
    spec:
      source:
        registry:
          url: "docker://quay.io/containerdisks/fedora:latest"
          pullMethod: node
        storage:
          resources:
            requests:
              storage: 10Gi
EOF
```

:

- schedule** — cron `0 3 * * 1` 03:00 UTC
- managedDataSource** — DataImportCron DataSource VM DataSource
- garbageCollect: Outdated** — DataVolume

- **importsToKeep: 2** — 2개의 이전 데이터 가져오기 기록을 보존합니다.

DataImportCron 확인

```
oc get dataimportcron -n cdi-operations-demo
```

결과

| NAME              | LASTIMPORT        | NEXTIMPORT | LASTIMPORTNAMESPACE |
|-------------------|-------------------|------------|---------------------|
| fedora-image-cron | Mon, 21 Jul 03:00 |            | cdi-operations-demo |

DataImportCron은 PVC를 가져오고 DataSource을 가져와 VM을 생성합니다.

```
spec:
  dataVolumeTemplates:
  - spec:
      sourceRef:
        kind: DataSource
        name: fedora-latest
        namespace: cdi-operations-demo
      storage:
        resources:
          requests:
            storage: 10Gi
```

OpenShift Cron 작업

OpenShift Virtualization namespace에 **openshift-virtualization-os-images** namespace에 DataImportCron을 생성합니다:

```
oc get dataimportcron -n openshift-virtualization-os-images
```

## 9: CDI를 사용하여 VM을 생성하는 방법

요구 사항

CDI는 qcow2 형식의 이미지를 가져오고 PVC를 생성합니다. PVC는 raw 형식으로 저장되며 gzip 압축을 해제합니다.

PVC는 StorageClass (lvms-vg1)를 사용하여 생성됩니다. CDI는 StorageClass를 사용하여 PVC를 생성합니다.

```
oc patch cdi cdi-kubevirt-hyperconverged --type merge -p
'{"spec":{"config":{"scratchSpaceStorageClass":"lvms-vg1"}}}'
```

CDI PVC StorageClass

```
oc get cdi cdi-kubevirt-hyperconverged -o
jsonpath='{.spec.config.scratchSpaceStorageClass}'
echo
```



raw PVC  
qcow2 raw

Filesystem PVC (StorageClass)

5.5% StorageClass

```
oc get cdi cdi-kubevirt-hyperconverged -o jsonpath='{.spec.config.filesystemOverhead}'
echo
```

```
oc patch cdi cdi-kubevirt-hyperconverged --type merge -p
'{"spec":{"config":{"filesystemOverhead":{"global":"0.055"}}}}'
```

StorageClass

```
oc patch cdi cdi-kubevirt-hyperconverged --type merge -p
'{"spec":{"config":{"filesystemOverhead":{"storageClass":{"lvms-vg1":"0.05"}}}}}'
```

## 10: CDI

CDI importer cloner pod CPU

cdi-resource-config.yaml

```
oc get cdi cdi-kubevirt-hyperconverged -o json | jq
'.spec.config.podResourceRequirements'
```

## CDI worker pod ทรัพยากรไม่เพียงพอ

```
oc patch cdi cdi-kubevirt-hyperconverged --type merge -p '{
  "spec": {
    "config": {
      "podResourceRequirements": {
        "limits": {
          "cpu": "2",
          "memory": "1Gi"
        },
        "requests": {
          "cpu": "100m",
          "memory": "60Mi"
        }
      }
    }
  }
}'
```

การเพิ่มทรัพยากร CDI worker pod (importer+cloner+uploader) ให้เพียงพอ CPU  
และ memory เพื่อป้องกันการเกิด OOMKilled

## ข้อ 11: การตั้งค่า CDI ทรัพยากร

### DataVolume กับการ ImportScheduled

DataVolume กับการ ImportScheduled จะใช้ importer pod

PVC ที่ใช้

```
oc get pvc -n cdi-operations-demo
```

PVC กับการ Pending ทรัพยากรไม่เพียงพอ

```
oc describe pvc <pvc-name> -n cdi-operations-demo
```

ตัวอย่าง:

- การตั้งค่า StorageClass โดยใช้ `oc patch storageclass <name> -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'`
- WaitForFirstConsumer** — DataVolume กับการ `cdi.kubevirt.io/storage.bind.immediate.requested: "true"`
- การตั้งค่า PV โดยใช้ `oc get pv`

## DataVolume 0% ImportInProgress

importer pod 0%

importer pod

```
oc logs -n cdi-operations-demo -l cdi.kubevirt.io/storage.import.importPvcName=<dv-name>
```

:

- **URL** — URL : `oc run curl-test --rm -it --image=curlimages/curl curl -sI <url>`
- **TLS** — CA ConfigMap DataVolume spec `certConfigMap`
- **DNS** — DNS

:

```
Unable to create scratch space PVC: no default storage class found
```

StorageClass:

```
oc patch cdi cdi-kubevirt-hyperconverged --type merge -p '{"spec":{"config":{"scratchSpaceStorageClass":"lvms-vg1"}}}'
```

DataVolume

## Importer Pod OOMKilled

importer pod OOMKilled

```
oc patch cdi cdi-kubevirt-hyperconverged --type merge -p '{"spec":{"config":{"podResourceRequirements":{"limits":{"memory":"2Gi"}}}}}'
```

:

1. CDI worker pod CPU qcow2 raw
2. importer pod
3. HTTP HTTP Range CDI

## CDI 確認作業

CDI Operator の状態を確認:

```
oc get cdi cdi-kubevirt-hyperconverged -o jsonpath='{.status.phase}'  
echo
```

状態 **Deployed** が表示され、**Error** や **Upgrading** の状態は表示されない。Operator pod の状態を確認:

```
oc logs -n openshift-cnv -l name=cdi-operator
```

確認完了

Namespace の削除作業

```
oc delete namespace cdi-operations-demo
```

確認

```
oc get namespace cdi-operations-demo
```

確認

```
Error from server (NotFound): namespaces "cdi-operations-demo" not found
```

CDI のインストール (確認済) (確認済) (確認済) CDI のインストール (確認済)

```
oc patch cdi cdi-kubevirt-hyperconverged --type json -p '[  
  {"op": "remove", "path": "/spec/config/scratchSpaceStorageClass"},  
  {"op": "remove", "path": "/spec/config/podResourceRequirements"}  
]
```



### CDI

確認済 CDI のインストール (確認済) CDI のインストール (確認済)

確認

確認済 CDI のインストール (確認済)

- CDI のインストール (確認済) **openshift-cnv** namespace のインストール (確認済)
- DataVolume のインストール (確認済) **Pending** → **Succeeded** の状態 (確認済)



- HTTP URL
- PVC
- CDI
- DataImportCron
- CDI worker pod
- OOMKilled pod CDI

- Containerized Data Importer (CDI)
- - OpenShift
- qcow2
- 
- VM
- VM

I/O

I/O OpenShift Virtualization 1 VM I/O I/O

VM fio I/O

- (VirtIO SCSI SATA) I/O
- (Block Filesystem) I/O
- I/O
- fio VM I/O
- I/O
- 

- OpenShift Virtualization Operator OpenShift 4.18+
- (system:admin)
- oc CLI virtctl CLI
- StorageClass (lvms-vg1)



创建存储卷

创建存储卷 DataVolume 和 PVC

```
dataVolumeTemplates:
- metadata:
  name: my-disk
spec:
  pvc:
    accessModes:
    - ReadWriteOnce
    volumeMode: Block
    resources:
      requests:
        storage: 30Gi
  source:
    registry:
      pullMethod: node
      url: docker://quay.io/containerdisks/fedora:latest
```

创建虚拟机

创建虚拟机 VM 和 KubeVirt 3 个存储卷

| 存储卷名称        | 存储卷类型                                                                      | 存储卷大小                 |
|--------------|----------------------------------------------------------------------------|-----------------------|
| none         | 创建存储卷时，指定存储卷类型为 none，表示该存储卷不占用任何物理存储资源，而是由操作系统（OS）管理。                      | 创建存储卷时，指定存储卷大小为 30Gi。 |
| writeback    | 创建存储卷时，指定存储卷类型为 writeback，表示该存储卷由操作系统（OS）管理，并由虚拟机（VM）写入。                   | 创建存储卷时，指定存储卷大小为 30Gi。 |
| writethrough | 创建存储卷时，指定存储卷类型为 writethrough，表示该存储卷由操作系统（OS）管理，并由虚拟机（VM）写入，且写入数据会立即写入物理存储。 | 创建存储卷时，指定存储卷大小为 30Gi。 |

创建虚拟机 KubeVirt 存储卷 Block 存储卷 none 文件系统 writethrough 存储卷

创建存储卷时，指定存储卷类型为 none，表示该存储卷不占用任何物理存储资源，而是由操作系统（OS）管理。fsync 创建存储卷时，指定存储卷类型为 writethrough，表示该存储卷由操作系统（OS）管理，并由虚拟机（VM）写入，且写入数据会立即写入物理存储。

创建存储卷

```
devices:
  disks:
  - name: rootdisk
    cache: none
    disk:
```

```
bus: virtio
```

## I/O 配置

配置 VM 的 I/O 使用 QEMU 的 I/O 配置。默认情况下，每个 VM 的 I/O 配置如下：

### I/O 配置

`ioThreadsPolicy` 配置 QEMU 的 I/O 线程策略。默认情况下，`dedicatedIOThread: true` 表示每个 VM 都有自己的 I/O 线程。

- `auto`: KubeVirt 的 `dedicatedIOThread: true` 表示每个 VM 都有自己的 I/O 线程。
- `shared`: `dedicatedIOThread: true` 表示所有 VM 共享 I/O 线程。

默认情况下，I/O 配置为 `auto`。如果配置为 `shared`，则表示所有 VM 共享 I/O 线程。

## Block Multi-Queue

`blockMultiQueue` 配置 VirtIO 的 `virtqueue`。每个 vCPU 都有自己的 I/O 线程，每个 CPU 都有自己的 I/O 线程。



配置 VirtIO 的 SCSI 和 SATA 控制器。

## 配置 1: Namespace 配置

```
oc create namespace disk-io-tuning
```

## 配置 2: 配置 VM 配置

配置 VM 的 I/O 配置。



配置 cloud-init 的 SSH 配置。

## vm-baseline.yaml 配置

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-baseline
  namespace: disk-io-tuning
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
```

```

    kubevirt.io/domain: vm-baseline
spec:
  domain:
    cpu:
      cores: 2
    memory:
      guest: 2Gi
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
            bootOrder: 1
        - name: cloudinitdisk
          disk:
            bus: virtio
      interfaces:
        - name: default
          masquerade: {}
    resources:
      requests:
        memory: 2Gi
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        dataVolume:
          name: baseline-rootdisk
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            password: redhat
            chpasswd:
              expire: false
            packages:
              - fio
  dataVolumeTemplates:
    - metadata:
        name: baseline-rootdisk
      spec:
        pvc:
          accessModes:
            - ReadWriteOnce
          resources:
            requests:
              storage: 30Gi
        source:
          registry:
            pullMethod: node

```

```
url: docker://quay.io/containerdisks/fedora:latest
EOF
```

VM 000000000000

```
oc get vm -n disk-io-tuning -w
```

STATUS 0 **Running** 000000000000000000000000

0000 **3:** 0000000000 **VM** 000000

0000 I/O 000000000000 (VirtIO 000**cache: none**0I/O 0000000000000000) 000000002 000 VM 0000000000

**vm-tuned.yaml** 00000000

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-tuned
  namespace: disk-io-tuning
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-tuned
    spec:
      domain:
        ioThreadsPolicy: auto
        cpu:
          cores: 2
        memory:
          guest: 2Gi
        devices:
          blockMultiQueue: true
          disks:
            - name: rootdisk
              disk:
                bus: virtio
                bootOrder: 1
                cache: none
                io: native
                dedicatedIOThread: true
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
```

```

        masquerade: {}
    resources:
        requests:
            memory: 2Gi
    networks:
    - name: default
      pod: {}
    volumes:
    - name: rootdisk
      dataVolume:
        name: tuned-rootdisk
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          password: redhat
          chpasswd:
            expire: false
          packages:
            - fio
    dataVolumeTemplates:
    - metadata:
        name: tuned-rootdisk
    spec:
      pvc:
        accessModes:
        - ReadWriteOnce
        volumeMode: Block
        resources:
          requests:
            storage: 30Gi
      source:
        registry:
          pullMethod: node
          url: docker://quay.io/containerdisks/fedora:latest
EOF

```

## VM 設定オプション

- **ioThreadsPolicy: auto:** I/O スレッドの数を自動的に調整する
- **blockMultiQueue: true:** VirtIO ブロックデバイスに vCPU ごとに I/O キューを割り当てる
- **cache: none:** OS のキャッシュを無効にする
- **io: native:** QEMU の I/O 処理を Linux の I/O (io\_uring/libaio) に委任する
- **dedicatedIOThread: true:** QEMU の I/O 処理に専用のスレッドを割り当てる
- **volumeMode: Block:** raw デバイスとして QEMU に提供される

## VM 設定オプション





```
--iodepth=16 --numjobs=1 --size=1G --runtime=30 --time_based \
--rw=read --filename=/tmp/fio-test --output-format=terse \
| awk -F';' '{print "Sequential Read BW (KiB/s):", $7}'
```

□□□□□□□□□□□□□□□□□□□□

```
sudo fio --name=seqwrite --ioengine=libaio --direct=1 --bs=128k \
--iodepth=16 --numjobs=1 --size=1G --runtime=30 --time_based \
--rw=write --filename=/tmp/fio-test --output-format=terse \
| awk -F';' '{print "Sequential Write BW (KiB/s):", $48}'
```

□□□□□□□□□□□□**(Ctrl+)**□□□□□□□□□□□□□□□□

## □□□□ **5: fio** □□□□□□□□□□ (□□□□□□□□ **VM**)

□□□□□□□□ **VM** □□□□□□□□□□□□□□□□□□□□

```
virtctl console vm-tuned -n disk-io-tuning
```

□□□□□ **fedora**□□□□□□□ **redhat** □□□□□□□□□□cloud-init □□□□□□□□□□□□□□

```
cloud-init status --wait
```

□□ 4 □□ **fio** □□□□□□□□□□

```
sudo fio --name=randread --ioengine=libaio --direct=1 --bs=4k \
--iodepth=16 --numjobs=1 --size=1G --runtime=30 --time_based \
--rw=randread --filename=/tmp/fio-test --output-format=terse \
| awk -F';' '{print "Random Read IOPS:", $8, "| Avg Latency (usec):", $40}'
```

```
sudo fio --name=randwrite --ioengine=libaio --direct=1 --bs=4k \
--iodepth=16 --numjobs=1 --size=1G --runtime=30 --time_based \
--rw=randwrite --filename=/tmp/fio-test --output-format=terse \
| awk -F';' '{print "Random Write IOPS:", $49, "| Avg Latency (usec):", $81}'
```

```
sudo fio --name=seqread --ioengine=libaio --direct=1 --bs=128k \
--iodepth=16 --numjobs=1 --size=1G --runtime=30 --time_based \
--rw=read --filename=/tmp/fio-test --output-format=terse \
| awk -F';' '{print "Sequential Read BW (KiB/s):", $7}'
```

```
sudo fio --name=seqwrite --ioengine=libaio --direct=1 --bs=128k \
--iodepth=16 --numjobs=1 --size=1G --runtime=30 --time_based \
```

```
--rw=write --filename=/tmp/fio-test --output-format=terse \
| awk -F';' '{print "Sequential Write BW (KiB/s):", $48}'
```

VM IOPS VM IOPS VM IOPS VM I/O I/O IOPS IOPS

Ctrl+] VM

6: VM

virt-launcher pod QEMU

VM I/O

```
oc get vmi vm-tuned -n disk-io-tuning -o
jsonpath='{.status.launcherContainerImageVersion}'
```

VM QEMU I/O

```
TUNED_POD=$(oc get pod -n disk-io-tuning -l kubevirt.io/domain=vm-tuned -o
jsonpath='{.items[0].metadata.name}')
oc exec -n disk-io-tuning "$TUNED_POD" -- ps aux | grep -o '\-drive [^ ]*' | head -5
```

cache=none aio=native iothread QEMU

7: I/O

I/O

VM iostat I/O

```
sudo dnf install -y sysstat
iostat -xz 2
```

:

- %util: 100%
- await: I/O (ms)
- r\_await / w\_await:
- avgqu-sz: I/O 1

OpenShift

OpenShift virt-launcher pod

```
oc adm top pod -n disk-io-tuning -l kubevirt.io/domain=vm-tuned
```

OpenShift I/O iowait CPU VM

```
oc adm top node
```

VM iowait CPU VM

OpenShift

VM I/O

|                            | IOPS ( )         |         |           |
|----------------------------|------------------|---------|-----------|
| LVM thin pool (local NVMe) | 50,000-200,000+  | 1       |           |
| ODF / Ceph RBD             | 10,000-50,000    | 1-5 ms  |           |
| NFS                        | 1,000-10,000     | 2-10 ms |           |
| (gp3 SSD)                  | 3,000-16,000 ( ) | 1-3 ms  | OpenShift |



OpenShift

| (PostgreSQLMySQL) | VirtIO cache: noneio: nativededicatedIOThread: trueBlock blockMultiQueue: truefsync |
|-------------------|-------------------------------------------------------------------------------------|
| Web /             | VirtIO Filesystem Web I/O                                                           |
| /                 | VirtIO cache: writebackdedicatedIOThread: true                                      |
| /                 | VirtIO I/O I/O                                                                      |

|                 |                                                                             |
|-----------------|-----------------------------------------------------------------------------|
| Pod / Container | Block I/O                                                                   |
|                 | VirtIO <code>io: native</code> Block <code>blockMultiQueue: true</code> I/O |

## Block I/O

Block I/O tuning:

```
oc delete namespace disk-io-tuning
```

## Block I/O

Block I/O tuning:

- OS (VirtIO/SCSI/SATA) Block I/O
- PVC (Block Filesystem) Block I/O
- `(none/writeback/writethrough)` Block I/O
- I/O I/O I/O
- `fio` VM
- `iostat` I/O
- 

## Block I/O

- VM
- VM QoS
- KubeVirt
- OpenShift Virtualization
- `fio`

## OpenShift Virtualization

### Overview

OpenShift Virtualization provides a Kubernetes-native environment for running Linux-based VMs. It leverages User Defined Networks (UDN), localnet, Layer2, VLAN, Linux bridge, and OpenShift Virtualization to manage network connectivity for VMs.

### Architecture

The architecture consists of the following components:

- cloud-init/DHCP: Provides guest agent IP configuration.
- cloud-init: IP configuration for VMs.
- User Defined Networks (UDN): Layer2 network namespace.
- localnet: Local network namespace.
- VLAN: VLAN namespace.
- Layer2: Layer2 namespace.
- VM: Linux bridge namespace.
- PCI: PCI namespace.
- SR-IOV: SR-IOV namespace.
- OVS: Open vSwitch namespace.
- VM: VM namespace.

### OpenShift Virtualization Network

OpenShift Virtualization provides a Kubernetes-native environment for running Linux-based VMs. It leverages User Defined Networks (UDN), localnet, Layer2, VLAN, Linux bridge, and OpenShift Virtualization to manage network connectivity for VMs.

- **User Defined Networks (UDN):** namespace for Layer2 network.
- **Localnet:** Local network namespace.
- **Layer2:** Layer2 namespace.
- **Linux Bridge:** VM namespace.
- **SR-IOV:** PCI namespace.
- **OVN-Kubernetes Overlay:** Pod namespace.

### OpenShift Virtualization

OpenShift Virtualization provides a Kubernetes-native environment for running Linux-based VMs. It leverages User Defined Networks (UDN), localnet, Layer2, VLAN, Linux bridge, and OpenShift Virtualization to manage network connectivity for VMs.

- OpenShift Virtualization: OpenShift 4.19+ namespace.
- NMState Operator: NMState Operator namespace.

- 网络地址转换 (NAT/VLAN) 配置
- 网络策略配置

## 网络配置

### IP 配置

cloud-init DHCP 配置 QEMU guest agent 配置 Windows VM 配置 Sysprep 配置 IP 配置

### cloud-init 配置

cloud-init 配置 IP 配置

### UDN 配置

User Defined Networks 配置 Layer2 配置

### CUDN 配置 Localnet 配置

ClusterUserDefinedNetwork 配置 localnet 配置

### NAD 配置 VLAN 配置 localnet 配置

NetworkAttachmentDefinition 配置 VLAN 配置 localnet 配置

### CUDN 配置 VLAN 配置 localnet 配置

namespace 配置 ClusterUserDefinedNetwork 配置 VLAN 配置 localnet 配置

### Layer 2 配置

VM 配置 Layer2 配置

### Linux bridge

VM 配置 Linux bridge 配置

### SR-IOV 配置

(Virtual Functions) 配置 VM 配置 PCI 配置 SR-IOV 配置

### Localnet 配置

localnet 配置 OVS 配置

### VM Ingress 配置 Routes

OpenShift Ingress 配置 VM 配置 HTTP 配置 HTTPS 配置

## 网络配置

- 网络策略配置: Pod 网络策略配置
- 网络策略配置: Pod 网络策略配置

## OpenShift 网络

localnet 通过 Linux bridge 连接到物理网络

- Worker Node 通过 OpenShift 网络连接到物理网络
- 通过 OpenShift 网络连接到物理网络
- 通过 OpenShift 网络连接到物理网络

## OpenShift 网络

OpenShift 网络通过 OpenShift 网络连接到物理网络

- 通过 OpenShift 网络连接到物理网络 VM 网络
- 通过 OpenShift 网络连接到物理网络

## OpenShift 网络

- [VM 网络 DNS](#) - VM 网络连接到 DNS 网络
- [OpenShift Virtualization Networking](#)

## OpenShift IP 网络

### OpenShift

OpenShift Virtualization 网络 (VM) 通过 IP 网络连接到物理网络 DHCP IP 网络

OpenShift 5 网络通过 IP 网络连接到物理网络

- **cloud-init networkData** — 通过 OpenShift 网络 (Linux VM)
- **IP 网络 UDN** — OVN-Kubernetes 网络通过 IP 网络
- **DHCP 网络** — DHCP 网络通过 MAC 网络 IP 网络
- **nmcli** 网络 **QEMU guest agent** — guest agent 网络 IP 网络
- **Sysprep (Windows VM)** — **unattend.xml** 网络

OpenShift 网络通过 OpenShift 网络连接到物理网络

### OpenShift

- OpenShift Virtualization 网络 IP 网络
- cloud-init **networkData** 网络 IP 网络
- IP 网络 UDN 网络
- MAC 网络 DHCP 网络

- QEMU guest agent `nmcli` を利用して IP を設定
- Sysprep を使って Windows VM の IP を設定
- 設定を自動化

## IP 設定

IP 設定の自動化

設定の自動化

IP 設定は VM の Pod として OpenShift の `Services` と `Routes` (Pod の IP) を使って DHCP を設定

## OpenShift の IP 設定

IP は OVN-Kubernetes IPAM を使って (Pod の IP) を設定

設定の自動化

IP 設定は VM の Pod として (IPAM) を使って DHCP を設定

## IP 設定

- 設定の自動化
- IP 設定 (Pod の IP) を設定
- DHCP を設定
- 設定の自動化

## DHCP を OVN-Kubernetes IPAM で設定

- VM の IP を設定
- DHCP を設定
- Kubernetes の Services と DNS を設定

## 設定

- OpenShift Virtualization Operator を OpenShift 4.18 にインストール
- CLI コマンド: `oc virtctl`
- 1 つの Pod を作成
- NetworkAttachmentDefinition (NAD) を作成 (Localnet VLAN を指定)
- DHCP を設定: DHCP を設定
- IP 設定 (IP を設定)



## ❏ 1: cloud-init networkData

cloud-init ❏ VM ❏ cloudInitNoCloud ❏ networkData ❏ Netplan  
❏ 2 ❏

❏ OpenShift Virtualization ❏ Linux VM ❏

❏

1. VM ❏ cloud-init ❏ NoCloud ❏ networkData ❏
2. cloud-init ❏
3. ❏

❏

cloud-init ❏ 2 (Netplan) ❏ 2 ❏ `ethernets`❏`bonds`❏`bridges`  
❏`vllans` ❏ `addresses`❏`dhcp4/dhcp6`❏`routes`❏`nameservers`(`addresses` ❏ `search` ❏)❏`mtu`  
❏`match`(`macaddress`❏`name`❏`driver`) ❏

❏

```
networkData: |
  version: 2
  ethernets:
    enp1s0:
      dhcp4: true
    enp2s0:
      dhcp4: false
      dhcp6: false
      addresses:
        - 192.168.100.50/24
      nameservers:
        addresses:
          - 192.168.100.1
```

❏`enp1s0`(❏ Pod ❏) ❏ DHCP ❏`enp2s0`(❏) ❏ IP ❏

❏

`gateway4` ❏ `gateway6` ❏ Netplan 0.103 ❏ cloud-init 23.1 ❏  
`routes` ❏

```
networkData: |
  version: 2
  ethernets:
    enp2s0:
      dhcp4: false
      addresses:
```

```

- 192.168.100.50/24
routes:
- to: 192.168.100.0/24
  via: 192.168.100.1
nameservers:
addresses:
- 192.168.100.1
search:
- example.com

```

search DNS routes to: default (default route table)

1. networkData (network)

2. networkData (network)

3. DHCP (network)

3. DHCP (network)

## KubeVirt networkData

KubeVirt VM networkData PCI networkData

- enp1s0 — networkData (network Pod network)
- enp2s0 — 2 networkData (network networkData)
- enp3s0 — 3 networkData (2 network networkData)



eth0 networkData VM spec networkData (enpXs0) networkData

networkData

cloudInitNoCloud networkData networkData

network: networkData

KubeVirt networkData version: 2 network: networkData NoCloud networkData networkData

network user-data network

user-data cloud-init networkData user-data (#cloud-config) user-data networkData

Secret networkData network

networkDataSecretRef cloud-init networkData networkDataBase64 networkData

## 静的 IP 割り当て

静的 IP 割り当ては、Pod のネットワークインターフェースに `masquerade` オプションを指定することで実現できます。このオプションは、Pod の IP アドレスをホストの IP アドレスにマスカレードし、ホストの IP アドレス (10.0.2.0/24) を NAT して Pod IP に変換します。

Pod のネットワークインターフェース (enp1s0) に DHCP を設定することで、Pod の IP アドレスを動的に割り当てることができます。

静的 IP: Pod に **DHCP** + 静的 IP

静的 IP を割り当てるには、Pod のネットワーク設定に DHCP を指定し、VLAN を指定することで実現できます。IP アドレスは

静的 IP: Pod のネットワーク設定に (例: localnet VLAN 100 の NetworkAttachmentDefinition (NAD)) を指定し、Localnet VLAN を指定することで実現できます。

静的 IP を namespace に割り当てる

```
oc create namespace static-ip-demo
```

## fedora-multi-network-vm.yaml の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-multi-vm
  namespace: static-ip-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: fedora-multi-vm-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-multi-vm
    spec:
      domain:
        cpu:
```

```

    cores: 2
  devices:
    disks:
      - disk:
          bus: virtio
          name: rootdisk
      - disk:
          bus: virtio
          name: cloudinitdisk
    interfaces:
      - masquerade: {}
        name: default
      - bridge: {}
        name: vlan-network
    rng: {}
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
    - name: vlan-network
  multus:
    networkName: localnet-vlan-100
  volumes:
    - dataVolume:
        name: fedora-multi-vm-disk
      name: rootdisk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd: { expire: False }
          ssh_pwauth: True
        networkData: |-
          version: 2
          ethernet:
            enp1s0:
              dhcp4: true
            enp2s0:
              dhcp4: false
              dhcp6: false
              addresses:
                - 192.168.100.50/24
              nameservers:
                addresses:
                  - 192.168.100.1
      name: cloudinitdisk

```

□□□□□□□□□□

```
oc apply -f fedora-multi-network-vm.yaml
```

[illegible]

```
virtctl console fedora-multi-vm -n static-ip-demo
```

VM 0000000000000000

```
ip addr show
```

□ □ □ □ □ □ □

```

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen
1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group
default qlen 1000
    link/ether 02:7e:e2:23:d7:da brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.2/24 brd 10.0.2.255 scope global dynamic noprefixroute enp1s0
        valid_lft 86313438sec preferred_lft 86313438sec
    inet6 fe80::7e:e2ff:fe23:d7da/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group
default qlen 1000
    link/ether 02:7e:e2:23:d7:db brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.50/24 brd 192.168.100.255 scope global noprefixroute enp2s0
        valid_lft forever preferred_lft forever
    inet6 fe80::7e:e2ff:fe23:d7db/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

```
ip route show
```

□ □ □ □ □ □ □

```
default via 10.0.2.1 dev enp1s0 proto dhcp src 10.0.2.2 metric 100
10.0.2.0/24 dev enp1s0 proto kernel scope link src 10.0.2.2 metric 100
192.168.100.0/24 dev enp2s0 proto kernel scope link src 192.168.100.50 metric 101
```

[illegible]

IP

192.168.100.50

cloud-init

## cloud-init

cloud-init VM VirtualMachineInstance VM IP QEMU guest agent

```
oc get vmi fedora-multi-vm -n static-ip-demo -o jsonpath='{.status.interfaces}' |  
python3 -m json.tool
```

[

```
{  
  "infoSource": "domain, guest-agent",  
  "interfaceName": "enp1s0",  
  "ipAddress": "10.128.1.106",  
  "ipAddresses": [  
    "10.128.1.106"  
  ],  
  "mac": "02:7e:e2:23:d7:da",  
  "name": "default"  
},  
{  
  "infoSource": "domain, guest-agent, multus-status",  
  "interfaceName": "enp2s0",  
  "ipAddress": "192.168.100.50",  
  "ipAddresses": [  
    "192.168.100.50",  
    "fe80::7e:e2ff:fe23:d7db"  
  ],  
  "mac": "02:7e:e2:23:d7:db",  
  "name": "vlan-network"  
}  
]
```

networkData enp2s0 IP 192.168.100.50 guest agent

VM

```
virtctl console fedora-multi-vm -n static-ip-demo
```

VM:

```
cloud-init status
```

□ □ □ □ □ □ □

```
status: done
```

```
nmcli connection show
```

□ □ □ □ □ □ □

| NAME              | UUID                                 | TYPE     | DEVICE |
|-------------------|--------------------------------------|----------|--------|
| cloud-init enp1s0 | a41601f3-3acc-5f60-ac5f-9d9011ab7c25 | ethernet | enp1s0 |
| cloud-init enp2s0 | 90ff9077-4cf0-55d5-904d-6cdc4f01d148 | ethernet | enp2s0 |
| lo                | 29567487-41a5-4ac6-a312-f00ed799a660 | loopback | lo     |

Ctrl+] □□□□□□□□□□□□□□

[illegible]

```
sudo cat /var/log/cloud-init.log
```

cloud-init

## 2: IP UDN

Layer2 管理 (UDN) IP 管理 UDN 管理  
ipam.lifecycle: Persistent namespace OVN-Kubernetes  
管理 IP 管理 VM 管理/管理 IP 管理

□□□□□□□□□□ IP □□□□ DHCP □□□□□□□□□□□□□□□□ IP □□□□□□□□ VM □□□□□□□□□□□□□□□□□□□

111

1. `k8s.ovn.org/primary-user-defined-network` namespace
2. Layer2 `ipam.lifecycle: Persistent` `UserDefinedNetwork`
3. namespace VM `cloud-init` `networkData` `DHCP` IP

□ □ □ □ □ □ □ □

- 物理ネットワークカードに IP アドレスを割り当てる
- VM ごとに仮想ネットワークカードに IP アドレスを割り当てる
- 物理 DHCP サーバに物理ネットワークカードの IP アドレスを割り当てる
- 物理ネットワークカードの namespace ごとに IP アドレスを割り当てる

namespace [UDN](#) [VM](#) [UDN](#) [Layer2](#) [Layer2](#) [Layer2](#)

### 3: DHCP

DHCP (DHCP DHCP DHCP) DHCP IP DHCP DHCP MAC IP VM

1. VM spec VM MAC
2. DHCP MAC IP
3. VM DHCP

- DHCP IP
- VM
- VM (IP MAC)

MAC

DHCP VM MAC KubeVirt MAC

**fedora-dhcp-reserved-vm.yaml**

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-dhcp-reserved-vm
  namespace: static-ip-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: fedora-dhcp-reserved-vm-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
```



```

template:
  metadata:
    labels:
      kubevirt.io/domain: fedora-dhcp-reserved-vm
  spec:
    domain:
      cpu:
        cores: 2
      devices:
        disks:
          - disk:
              bus: virtio
              name: rootdisk
          - disk:
              bus: virtio
              name: cloudinitdisk
        interfaces:
          - masquerade: {}
            name: default
          - bridge: {}
            name: vlan-network
            macAddress: "02:00:00:01:00:01"
        rng: {}
      machine:
        type: pc-q35-rhel9.4.0
      memory:
        guest: 4Gi
    networks:
      - name: default
        pod: {}
      - name: vlan-network
        multus:
          networkName: localnet-vlan-100
    volumes:
      - dataVolume:
          name: fedora-dhcp-reserved-vm-disk
          name: rootdisk
      - cloudInitNoCloud:
          userData: |-
            #cloud-config
            user: fedora
            password: fedora123
            chpasswd: { expire: False }
            ssh_pwauth: True
          name: cloudinitdisk

```

cloud-init `macAddress` `DHCP` `IP` `networkData`

## DHCP 配置

この DHCP 設定で MAC アドレスを指定して DHCP 設定を行います。

ISC DHCP 設定 (`dhcpd.conf`):

```
host fedora-dhcp-reserved-vm {
    hardware ethernet 02:00:00:01:00:01;
    fixed-address 192.168.100.60;
}
```

dnsmasq 設定:

```
dhcp-host=02:00:00:01:00:01,192.168.100.60,fedora-dhcp-reserved-vm
```

作成した VM の設定を適用:

```
oc apply -f fedora-dhcp-reserved-vm.yaml
```

確認:

VM の IP アドレスを確認:

```
virtctl console fedora-dhcp-reserved-vm -n static-ip-demo
```

VM 内部:

```
ip addr show enp2s0
```

DHCP 設定で指定した MAC アドレスと IP アドレスを確認:

出力:

```
3: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group
default qlen 1000
    link/ether 02:00:00:01:00:01 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.60/24 brd 192.168.100.255 scope global dynamic enp2s0
        valid_lft 3600sec preferred_lft 3600sec
```

DHCP 設定で指定した IPv6 アドレスと MAC アドレス `02:00:00:01:00:01` が `macAddress` フィールドに設定されていることを確認:



**02:** 指定した MAC アドレスと DHCP 設定で指定した MAC アドレスを確認



```
sudo systemctl enable --now qemu-guest-agent
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
nmcli connection show
```

```

000000000000000000000000 ([: enp2s0  Wired connection 2])

```

IP

```
sudo nmcli connection modify "Wired connection 2" \
    ipv4.method manual \
    ipv4.addresses 192.168.100.70/24 \
    ipv4.dns 192.168.100.1
```

□□□□□□□□□□

```
sudo nmcli connection up "Wired connection 2"
```

VM 00000000000000

```
ip addr show enp2s0
```

## VirtualMachineInstance VM 0000000000

```
oc get vmi fedora-dhcp-reserved-vm -n static-ip-demo -o
jsonpath='{.status.interfaces[*].ipAddress}'
```

## NetworkManager VM

□ □ □ □ □ □ □ □

- **VM** **cloud-init**
- **cloud-init** **VM**
- **cloud-init**
- **cloud-init**



```
VM spec cloud-init networkData cloud-init nmcli
cloud-init (cloud-init)VM spec networkData
cloud-init
```

## ▣▣ 5: Windows VM ▣▣ Sysprep

Windows `unattend.xml` Microsoft Sysprep Windows (OOBE) `specialize` IP

103

1. 下載並安裝 `unattend.xml` 腳本檔案
2. 安裝 `sysprep` 腳本檔案 `VM` 檔案
3. `Windows` 安裝後，將 `unattend.xml` 腳本檔案安裝到 `VM` 檔案

unattend.xml □□□□□□□□□□□□□□□□

```
specialize  Ethernet  IP  NIC  Windows
VM  Identifier
PowerShell  Get-NetAdapter
```

```
<?xml version="1.0" encoding="utf-8"?>
<unattend xmlns="urn:schemas-microsoft-com:unattend">
  <settings pass="specialize">
    <component name="Microsoft-Windows-TCPIP"
      processorArchitecture="amd64"
      publicKeyToken="31bf3856ad364e35"
      language="neutral"
      versionScope="nonSxS"
      xmlns:wcm="http://schemas.microsoft.com/WMIconfig/2002/State">
      <Interfaces>
        <Interface wcm:action="add">
          <Identifier>Ethernet</Identifier>
          <Ipv4Settings>
            <DhcpEnabled>>false</DhcpEnabled>
          </Ipv4Settings>
          <UnicastIpAddresses>
            <IpAddress wcm:action="add" wcm:keyValue="1">192.168.100.80</IpAddress>
          </UnicastIpAddresses>
          <Routes>
            <Route wcm:action="add">
              <Identifier>0</Identifier>
              <NextHopAddress>192.168.100.1</NextHopAddress>
              <Prefix>0.0.0.0</Prefix>
            </Route>
          </Routes>
        </Interface>
      </Interfaces>
    </component>
    <component name="Microsoft-Windows-DNS-Client"
      processorArchitecture="amd64"
      publicKeyToken="31bf3856ad364e35"
      language="neutral"
```

```

        versionScope="nonSxS"
        xmlns:wcm="http://schemas.microsoft.com/WMIconfig/2002/State">
<Interfaces>
  <Interface wcm:action="add">
    <Identifier>Ethernet</Identifier>
    <DNSServerSearchOrder>
      <IpAddress wcm:action="add" wcm:keyValue="1">8.8.8.8</IpAddress>
    </DNSServerSearchOrder>
  </Interface>
</Interfaces>
</component>
</settings>
</unattend>

```

□□□□□□□□

**unattend.xml** □ ConfigMap □□□□VM spec □ **sysprep** □□□□□□□□□□Sysprep □□□□ Windows VM  
 □□□□□□□□□□□□□□□□Sysprep □□□□ Windows VM □□□□□ OpenShift □□□□□ □□□□□□□□

□□□□□□□□ ConfigMap □□□□□□

```
oc create configmap windows-sysprep --from-file=unattend.xml -n static-ip-demo
```

VM spec □□□□□□

```

volumes:
- name: sysprep
  sysprep:
    configMap:
      name: windows-sysprep

```

□□□□□□□□□□□□□□□□

```

devices:
  disks:
    - cdrom:
        bus: sata
        name: sysprep

```

□□□□□□□□

- □□ IP □□□□□ Windows Server VM
- □□□□□□□□□□□□□□□□ Windows □□□□□□
- VMware □ Hyper-V □□ Windows □□□□□□□□□□□□



Sysprep □ Windows □□□□□□□□□□□□□□□□Windows VM □□□□□□ IP

## Network

Network configuration

| OS           | cloud-init networkData | UDN IP          | DHCP            | guest agent / nmcli      | Sysprep                  |
|--------------|------------------------|-----------------|-----------------|--------------------------|--------------------------|
| OS           | Linux                  | Linux / Windows | Linux / Windows | Linux                    | Windows                  |
| Network      | Network                | IP (DHCP)       | DHCP (DHCP)     | Network                  | OOBE (OOBE)              |
| Network      | Network                | IP              | DHCP (DHCP)     | Network                  | Network                  |
| Network      | IP                     | IP (IPAMClaim)  | DHCP (DHCP)     | Network (NetworkManager) | Network                  |
| Network      | IP (IP)                | IP (IP)         | IP (IP)         | IP (IP)                  | IP (IP)                  |
| Network      | IP                     | Network         | IP              | IP                       | IP                       |
| IP / Network | IP                     | IP (UDN IP)     | MAC (MAC)       | IP                       | ConfigMap (ConfigMap)    |
| Network      | VM Network             | Network         | DHCP Network    | VM Network               | Network (RDP/PowerShell) |

## Network configuration

network: networkData

network: network: cloud-init networkData

KubeVirt cloudInitNoCloud networkData version: 2 network: network:\n version: 2\n ethernet:\n ... version: 2\nethernet:\n ...

Network configuration

networkData enp1s0 eth0

OS Linux (RHEL 8+ Fedora Ubuntu 16.04+) enp1s0 enp2s0 VM spec

Network configuration

DHCP

cloud-init networkData IP dhcp4: false dhcp6: false





111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- OpenShift Virtualization `nmcli` IP `networkData`
- Linux VM `cloud-init networkData` IP
- `nmcli` IP `nmcli` IP `UDN`
- IP `MAC` `DHCP`
- QEMU guest agent `nmcli` IP
- Sysprep `unattend.xml` Windows VM IP
- `nmcli`
- `nmcli`

11

- cloud-init 配置
- cloud-init 配置 2 (Netplan 配置)
- cloud-init NoCloud 配置
- KubeVirt 配置 (cloud-init)
- KubeVirt 配置
- VM 配置 Pod 配置
- OpenShift Virtualization 配置
- OpenShift Virtualization 配置
- Microsoft Sysprep 配置 Unattend 配置
- Netplan YAML 配置
- cloud-init 配置
- Localnet VLAN 配置
- UDN 配置 Layer2 配置

## cloud-init □□□□□□□□□□

OpenShift Virtualization cloud-init IP

11

- 网络配置
- **cloud-init** 网络配置
- 网络配置
- 网络配置



```
# 確認
grep -i error /var/log/cloud-init.log
grep -i failed /var/log/cloud-init.log
```

確認 **cloud-init** 確認

```
# userdata 確認
sudo cat /var/lib/cloud/instance/user-data.txt

# networkData (networkData 確認)
sudo cat /var/lib/cloud/instance/network-config.txt

# cloud-init 確認
sudo cat /var/lib/cloud/instance/cloud-config.txt
```

確認

```
# 確認
ip addr show

# 確認
ip route show

# DNS 確認
cat /etc/resolv.conf

# NetworkManager (Fedora, RHEL) 確認
nmcli device status
nmcli connection show
```

確認

```
# 確認
ping -c 3 8.8.8.8

# DNS 確認
nslookup google.com

# Pod 確認
curl http://<vm-ip>
```

確認

確認 **1: DHCP** 確認

root: DHCP VM の IP を確認する

root:

```
$ oc get vmi fedora-dhcp-vm -n vms -o jsonpath='{.status.interfaces[0].ipAddress}'  
# 確認する IP を確認する
```

root:

```
# VM を確認する  
virtctl console fedora-dhcp-vm -n vms  
  
# IP アドレスを確認する  
ip addr show enp1s0  
  
# DHCP ログを確認する  
journalctl -u NetworkManager | grep -i dhcp
```

確認する:

- DHCP 設定を確認する
- IP アドレスを確認する
- cloud-init の DHCP 設定を確認する

root:

1. VM 内の cloud-init DHCP 設定を確認:

```
cloudInitNoCloud:  
  networkData: |  
    version: 2  
    ethernets:  
      enp1s0:  
        dhcp4: true
```

2. ネットワーク接続を確認:

```
sudo nmcli connection down "Wired connection 1"  
sudo nmcli connection up "Wired connection 1"
```

3. NetworkManager ログを確認:

```
sudo journalctl -u NetworkManager --since "5 minutes ago"
```

```
##: cloud-init #####
```

```
❯ ip netns exec ns0 ip netns exec ns1 ping -c 1 192.168.1.1
```

```
ping: device eth0 not found
```

□□:

```
# VM 0 - 0000000000000000
ip link show
```

□□□□□□:

- OS 网络接口名称 (eth0, eth1) 配置

000:

1. 检查 OS 是否支持网络接口名称 Linux 系统 (RHEL 8+, Fedora, Ubuntu 16.04+) 支持网络接口名称
  - 检查网络接口名称: `enp1s0`
  - 2 个网络接口名称: `enp2s0`
  - 3 个网络接口名称: `enp3s0`
2. cloud-init 中 `networkData` 配置网络接口名称:

```
networkData: |
  version: 2
  ethernet:
    enp1s0: # eth0
      dhcp4: true
```

## □□□□ 2: □□ IP □□

□□: □□ **IP** □□□□□□□□

□□: □□□□□ IP □□□□□VM □ DHCP IP □□□□□□□□

□□:

```
# VM
ip addr show enp1s0

# cloud-init
grep -i "network" /var/log/cloud-init.log
```

□□□□□□:

- networkData [] [] [] [] [] [] [] [] [] []
- DHCP [] [] [] [] [] [] [] []

- networkData YAML ファイル
- cloud-init の networkData ファイル

例:

### 1. networkData ファイル (networkData YAML ファイル) の例:

```
networkData: |
  version: 2
  ethernets:
    enp1s0:
      dhcp4: false # DHCP オフ
      addresses:
        - 192.168.1.100/24
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

### 2. cloud-init の確認:

```
cloud-init status --long
cat /var/log/cloud-init.log | grep -A 5 -B 5 "network"
```

### 3. ネットワークの設定:

```
sudo nmcli connection modify "Wired connection 1" \
  ipv4.method manual \
  ipv4.addresses 192.168.1.100/24 \
  ipv4.gateway 192.168.1.1 \
  ipv4.dns "8.8.8.8 8.8.4.4"

sudo nmcli connection up "Wired connection 1"
```

例: IP アドレスの確認

例:

```
# VM の IP アドレスを確認
ip addr show enp1s0
# IP アドレスを確認

ping 192.168.1.1
```

```
# 確認コマンド
```

例:

```
# 確認コマンド
ip route show

# 確認コマンド
ping -c 1 192.168.1.1
```

確認:

- 確認コマンド
- 確認コマンド
- IP 確認コマンド
- VLAN 確認

例:

1. cloud-init 確認コマンド
2. 確認コマンド
3. 確認コマンド IP 確認コマンド
4. VLAN 確認コマンドNetworkAttachmentDefinition (NAD) 確認コマンド

確認 3: 確認コマンド

例: 確認コマンド

例: 確認コマンド IP 確認コマンド

例:

```
# VM
ip addr show

# NetworkManager 確認
nmcli connection show
```

確認:

- networkData 確認コマンド
- 確認コマンド
- 確認コマンド

例:

### 1. VM 配置:

```
spec:
  template:
    spec:
      networks:
        - name: default
          pod: {}
        - name: secondary-net
      multus:
        networkName: my-nad
```

### 2. networkData 配置:

```
networkData: |
  version: 2
  ethernet:
    enp1s0:
      dhcp4: true
    enp2s0: # 配置
      dhcp4: false
      addresses:
        - 192.168.100.10/24
```

### 3. NAD 配置:

```
oc get network-attachment-definition -n <namespace>
oc describe network-attachment-definition <nad-name> -n <namespace>
```

配置:

配置:

配置:

```
# VM
ip route show

# 配置
# default via 10.128.0.1 dev enp1s0
# default via 192.168.100.1 dev enp2s0
```

配置:

- 配置
- DHCP 配置



- 00000000000000000000000000000000

0000:

1. networkData 00000000000000000000000000000000:

```
enp2s0:
  dhcp4: false
  addresses:
    - 192.168.100.10/24
  # 'routes' 00000000 - 0000000000000000
```

2. 00000000000000000000000000000000:

```
enp1s0:
  dhcp4: true
  dhcp4-overrides:
    route-metric: 100 # 000000000000

enp2s0:
  dhcp4: false
  addresses:
    - 192.168.100.10/24
  routes:
    - to: default
      via: 192.168.100.1
      metric: 200 # 000000 = 00000000
```

0000 4: VLAN 000000000000

00: VLAN 0000000000000000

00: VLAN 00000000 (0: enp2s0.100) 00000000

00:

```
# VM 0
ip link show

# cloud-init 000000
grep -i vlan /var/log/cloud-init.log
```

00000000:

- networkData 0 VLAN 00000000
- 00000000000000000000
- VLAN 0000000000000000

□□□:

### 1. networkData □ VLAN □□□□□:

```
networkData: |
  version: 2
  ethernet:
    enp2s0: {} # 000000000000000000000000
  vlans:
    vlan100:
      id: 100
      link: enp2s0
      addresses:
        - 192.168.100.10/24
```

## 2. VLAN 配置配置:

```
sudo nmcli connection add type vlan \
  con-name vlan100 \
  ifname enp2s0.100 \
  dev enp2s0 \
  id 100 \
  ip4 192.168.100.10/24
```

00: VLAN 0000000000000000

00: VLAN 00000000000000000000000000000000

□□:

```
# VLAN 0000000000000000
ip addr show enp2s0.100

# 0000000000000000
tcpdump -i enp2s0 -n vlan 100
```

□□□□□□:

- 00000000000000000000000000000000
- 0000000000 VLAN 0000000000
- 000 VLAN ID 00000000
- NAD 0 VLAN 0000000000

□□□:

1. 配置VLAN 100 配置命令如下：

## 2. NAD の VLAN 指定:

```
oc get network-attachment-definition <nad-name> -n <namespace> -o yaml
```

**"vlan": 100** を指定する

## 3. localnet の OVN 指定:

```
oc get nncp -A
```

## ステップ 5: IPv6 の

IPv6 を有効にする

IPv4 と IPv6 の両方を有効にする

IPv6:

```
# VM の  
ip -6 addr show
```

確認:

- cloud-init の IPv6 を有効にする
- OS の IPv6 を有効にする

確認:

## 1. networkData の IPv6 指定:

```
networkData: |  
  version: 2  
  ethernet:  
    enp1s0:  
      dhcp4: true  
      dhcp6: true  
      # IPv6:  
      addresses:  
        - 2001:db8::10/64
```

## 2. IPv6 の有効化:

```
cat /proc/sys/net/ipv6/conf/all/disable_ipv6  
# 0 (有効) を指定する
```

## □□□□ 6: DNS □□

□□: DNS □□□□□□

□□:

```
# VM □  
ping 8.8.8.8  
# □□  
  
ping google.com  
# □□□□□□
```

□□:

```
cat /etc/resolv.conf  
  
# DNS □□□□□  
nslookup google.com
```

□□□□□□:

- cloud-init □□□□□□□□□□□□□□□□
- DHCP □ DNS □□□□□□□□□□□□
- **/etc/resolv.conf** □□□□□□□□□□□□

□□□:

1. networkData □□□□□□□□□□□□:

```
networkData: |  
  version: 2  
  ethernets:  
    enp1s0:  
      dhcp4: false  
      addresses:  
        - 192.168.1.100/24  
      nameservers:  
        addresses:  
          - 8.8.8.8  
          - 8.8.4.4  
      search:  
        - example.com
```

2. DNS □□□□□□:

```
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

□□□□□□

□□□□□□□□□□□□□□□□

```
# VM □ - □□□□□□□□□□
echo "=== Interfaces ==="
ip addr show

echo "=== Routes ==="
ip route show

echo "=== DNS ==="
cat /etc/resolv.conf

echo "=== Cloud-init Status ==="
cloud-init status --long

echo "=== Cloud-init Network Config ==="
sudo cat /var/lib/cloud/instance/network-config.txt

echo "=== NetworkManager Connections ==="
nmcli connection show

echo "=== Connectivity Tests ==="
ping -c 2 8.8.8.8 && echo "IP connectivity: OK" || echo "IP connectivity: FAILED"
nslookup google.com && echo "DNS: OK" || echo "DNS: FAILED"
```

**OpenShift** □□□□□□□□□□

```
# VM □□□□□□□□
oc get vm,vmi <vm-name> -n <namespace>

# VM IP □□□□□□□□
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.interfaces}' | jq

# □□□□□□□□□□□□
oc get vm <vm-name> -n <namespace> -o jsonpath='{.spec.template.spec.networks}' | jq

# NADs □□□
oc get network-attachment-definition -n <namespace>

# VM □□□□□□□□
oc get events -n <namespace> --field-selector involvedObject.name=<vm-name> --sort
-by='.lastTimestamp'
```

```
# VM cloud-init
oc get vm <vm-name> -n <namespace> -o yaml | grep -A 50 cloudInitNoCloud
```

## cloud-init ( )

```
# VM - cloud-init ( )
sudo cloud-init clean --logs
sudo cloud-init init
sudo cloud-init modules --mode config
sudo cloud-init modules --mode final

#
cloud-init status --long
```

## 

VM Linux PCI

| 1 (Pod )       | enp1s0 | (DHCP) |
|----------------|--------|--------|
| 2 (Multus/NAD) | enp2s0 |        |
| 3 (Multus/NAD) | enp3s0 | 3      |
| 4 (Multus/NAD) | enp4s0 | 4      |

OS Linux (RHEL 8+ Fedora Ubuntu 16.04+) enpXsY eth0 eth1

## cloud-init ( )

### cloud-init v1 vs v2 ( )

#### Version 1 ( ):

```
networkData: |
  version: 1
  config:
    - type: physical
      name: eth0
      subnets:
        - type: dhcp
```

#### Version 2 ( ):

```
networkData: |
  version: 2
```

```
ethernets:
  enp1s0:
    dhcp4: true
```

00: 0000000000000000 2 000000000000

☐ ☐ ☐ ☐ ☐

- [cloud-init](#) [インストール](#) - [インストール](#)
- [Netplan](#) [インストール](#)
- [OpenShift Virtualization](#) [インストール](#)
- [KubeVirt](#) [インストール](#)
- [OVS](#) [インストール](#)

📁: **ocplab01** 📁📁📁📁📁📁

SNO 00000 (ocplab01) 00000000:

```
$ oc get vmi fedora-dhcp-vm -n cloud-init-test -o jsonpath='{.status.interfaces}' | jq
[
  {
    "infoSource": "domain, guest-agent, multus-status",
    "interfaceName": "eth0",
    "ipAddress": "10.128.0.165",
    "ipAddresses": [
      "10.128.0.165",
      "fd02::5e"
    ],
    "mac": "02:92:c2:00:00:05",
    "name": "default",
    "queueCount": 1
  }
]
```

[illegible]

VM 000000:

```
[fedora@fedora ~]$ ip addr show enp1s0
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group
default qlen 1000
    link/ether 02:92:c2:00:00:05 brd ff:ff:ff:ff:ff:ff
    inet 10.128.0.165/23 brd 10.128.1.255 scope global dynamic noprefixroute enp1s0
        valid_lft 86295sec preferred_lft 86295sec
    inet6 fd02::5e/64 scope global dynamic noprefixroute
```

```
valid_lft 86296sec preferred_lft 14296sec
```

cloud-init 状態:

```
[fedora@fedora ~]$ cloud-init status
status: done
```

ネットワーク設定は DHCP 設定されている

## UDN 名前空間

■

OpenShift Virtualization の User Defined Networks (UDN) は Layer2 名前空間と IP 名前空間を namespace として (Network Attachment Definition) 定義する。5 名前空間を定義する。

状態:

```
OpenShift 4.19, 4.20, 4.21
```

■

- OpenShift Virtualization Operator 4.19 以降
- OVN-Kubernetes (OpenShift 4.19 以降)
- namespace `userdefinednetwork` として定義
- CLI での操作: `oc virtctl`

### 1: UDN 名前空間 namespace

UDN 名前空間 namespace `userdefinednetwork` は OpenShift 4.19 以降の namespace として定義されている。



namespace `userdefinednetwork` は OpenShift 4.19 以降の namespace として定義されている。

`udn-primary-namespace.yaml` の内容

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
```



```
name: udn-primary-vm-guests
labels:
  k8s.ovn.org/primary-user-defined-network: ""
EOF
```

## 2: UserDefinedNetwork

| Layer2        | Primary | UDN | IP | IPAM |
|---------------|---------|-----|----|------|
| namespace Pod |         |     |    |      |

udn-primary.yaml 配置文件

```
oc apply -f - <<EOF
apiVersion: k8s.ovn.org/v1
kind: UserDefinedNetwork
metadata:
  name: udn-primary
  namespace: udn-primary-vm-guests
spec:
  topology: Layer2
  layer2:
    role: Primary
    subnets:
      - "192.168.100.0/24"
  ipam:
    lifecycle: Persistent
EOF
```

### 3: Primary UDN

Primary UDN 0000000000000000 Python HTTP 0000000000000000000000000000UDN 0 IPAM 0000  
192.168.100.0/24 000000000000 IP 000000000000UDN 0000000000000000000000000000000000



```

##### cloud-init #####
##### SSH #####

```

fedora-vm-with-udn.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-vm-with-udn
  namespace: udn-primary-vm-guests
  labels:
    app: fedora-web-vm
    network-type: udn-primary
spec:
  runStrategy: Always
```

```

dataVolumeTemplates:
- metadata:
  name: fedora-udn-volume
spec:
  pvc:
    accessModes:
    - ReadWriteOnce
    resources:
      requests:
        storage: 33Gi
  sourceRef:
    kind: DataSource
    name: fedora
    namespace: openshift-virtualization-os-images
template:
  metadata:
    labels:
      app: fedora-web-vm
  spec:
    domain:
      devices:
        disks:
          - name: datavolumedisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        interfaces:
          - name: default
            binding:
              name: l2bridge
      resources:
        requests:
          memory: 2Gi
          cpu: 1
      networks:
        - name: default
          pod: {}
      volumes:
        - name: datavolumedisk
          dataVolume:
            name: fedora-udn-volume
        - name: cloudinitdisk
          cloudInitNoCloud:
            userData: |
              #cloud-config
            write_files:
              - path: /etc/resolv.conf
                content: |
                  nameserver 172.30.0.10          # cluster DNS

```

```

        search udn-primary-vm-guests.svc.cluster.local svc.cluster.local
        options ndots:5
user: fedora
password: fedora
chpasswd: { expire: False }
packages:
- python3
runcmd:
- echo "<h1>Welcome to OpenShift Virtualization !!!</h1>" >
  /root/index.html
- cd /root && nohup python3 -m http.server 80 > /dev/null 2>&1 &
EOF

```

#### □□□□ 4: VM □□□□□□□□□□

Kubernetes → LoadBalancer → VM → Python HTTP →



AWS  
MetalLB

## fedora-vm-web-service.yaml

```
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: fedora-vm-web-service
  namespace: udn-primary-vm-guests
  labels:
    app: fedora-web-vm
spec:
  selector:
    app: fedora-web-vm
  ports:
    - name: http
      port: 80
      targetPort: 80
      protocol: TCP
  type: LoadBalancer
EOF
```

**□□□□ 5: □□□□□□□□□□**

~~~~~ AWS ~~~~~ IP ~~~~~:

```
oc get svc
```

□□□:

| NAME                                        | TYPE         | CLUSTER-IP    | EXTERNAL-IP                    |
|---------------------------------------------|--------------|---------------|--------------------------------|
| PORT(S)                                     | AGE          |               |                                |
| fedora-vm-web-service                       | LoadBalancer | 172.30.72.140 |                                |
| aa32981df19ad4959ae9a82fb9990db9-1996929606 |              |               | ca-central-1.elb.amazonaws.com |
| 80:32291/TCP                                | 45m          |               |                                |

```
curl -X GET https://api.github.com/repos/
```

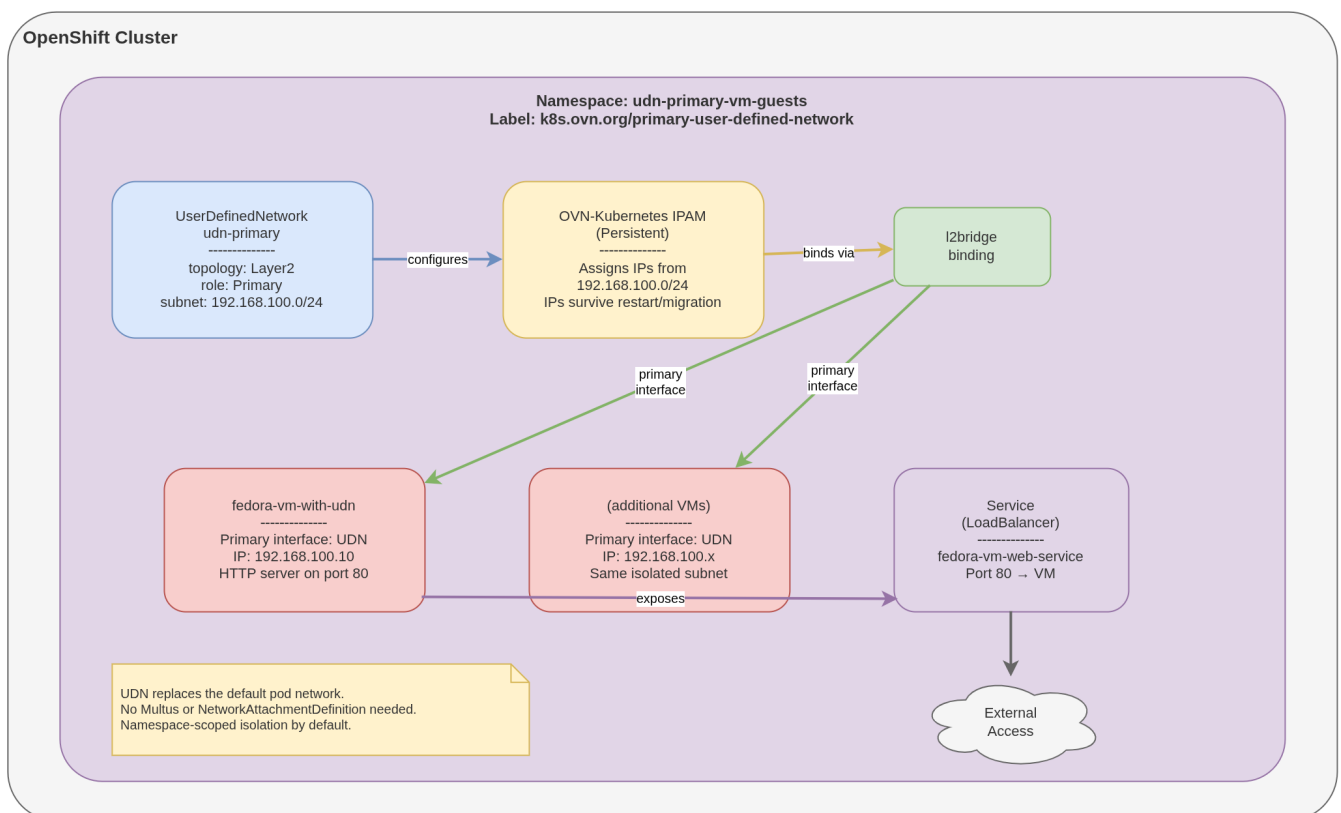
```
curl aa32981df19ad4959ae9a82fb9990db9-1996929606.ca-central-1.elb.amazonaws.com
```

□□□□□□□□:

# Welcome to OpenShift Virtualization !!!

□ □ □ □ □ □ □ □ □ □

UDN によって IP namespace によって pod によって UDN namespace pod



Layer2 配置 role: Primary 配置 UserDefinedNetwork 配置 namespace 配置  
(192.168.100.0/24) 配置 OVN-Kubernetes 配置 OVN-Kubernetes IPAM 配置 Persistent  
配置 VM 配置 IP 配置 VM 配置

l2bridge を利用して UDN を Multus を NetworkAttachmentDefinition を利用して UDN を VM に LoadBalancer を Kubernetes Service を



namespace [k8s.ovn.org/primary-user-defined-network](https://k8s.ovn.org/primary-user-defined-network) namespace pod VM

## UDN

UDN namespace IP

| 名前   | 説明                                        |
|------|-------------------------------------------|
| SaaS | namespace namespace VM namespace VM       |
|      | IP (10.0.1.0/24 10.0.2.0/24) namespace IP |
| IP   | Persistent IPAM DHCP                      |
|      | pod namespace UDN VM pod Layer 2          |

## 参照

関連項目:

- [k8s.ovn.org/primary-user-defined-network](https://k8s.ovn.org/primary-user-defined-network) UDN namespace
- Layer2 UserDefinedNetwork
- IPAM UDN VM
- UDN VM LoadBalancer
- UDN IP

## リンク

- [OpenShift -](#)
-

# CUDN 本地网 Localnet 部署指南

部署指南:

OpenShift 4.20

OpenShift Virtualization 使用 ClusterUserDefinedNetwork (CUDN) 本地网 localnet 部署指南。Localnet 部署指南描述了 Pod 本地网 VM 本地网 L2 部署指南。

部署

- **NMState Operator:** 部署 NMState Operator
- **NMState CR:** Operator 部署 nmstate 配置
- **Worker Node:** bridge mapping 在 Worker Node 部署

部署指南:

OpenShift 4.19

## 部署 1: VM 本地网 namespace 部署

localnet 部署指南描述了 VM 本地网 namespace 部署

[localnet-secondary-namespaces.yaml](#) 部署指南

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: vm-guests-localnet
EOF
```

## 部署 2: bridge mapping 部署

部署 physnet 使用 br-ex 部署 bridge mapping 部署指南描述了 localnet 部署指南 OVN-Kubernetes 部署指南

[localnet-bridge-mapping-nncp.yaml](#) 部署指南

```
oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: localnet-bridge-mapping
```

```
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ''
  desiredState:
    ovn:
      bridge-mappings:
      - localnet: localnet1
        bridge: br-ex
        state: present
EOF
```

```
0: 00000000 localnet1 0 Worker Node 0 br-ex 000000000000000000000000
```

### 3: ClusterUserDefinedNetwork

localnet ClusterUserDefinedNetwork

## cudn-localnet.yaml 配置

```
oc apply -f - <<EOF
apiVersion: k8s.ovn.org/v1
kind: ClusterUserDefinedNetwork
metadata:
  name: cudn-localnet
spec:
  namespaceSelector:
    matchExpressions:
      - key: kubernetes.io/metadata.name
        operator: In
        values: ["vm-guests-localnet"]
  network:
    topology: Localnet
    localnet:
      role: Secondary
      physicalNetworkName: localnet1
      ipam:
        mode: Disabled
EOF
```

#### Figure 4: VM configuration

Pod 连接到 localnet 连接到 VM 连接到



cloud-init SSH

## fedora-localnet-vm.yaml

```
oc apply -f - <<EOF
```

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-localnet-vm
  namespace: vm-guests-localnet
  labels:
    app: fedora-localnet-vm
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-localnet-volume
    spec:
      pvc:
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 33Gi
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
  template:
    metadata:
      labels:
        app: fedora-localnet-vm
    spec:
      domain:
        devices:
          disks:
            - name: datavolumedisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              bridge: {}
            - name: secondary_localnet
              bridge: {}
          resources:
            requests:
              memory: 2Gi
              cpu: 1
        networks:
          - name: default
            pod: {}
          - name: secondary_localnet
            multus:

```



```

        networkName: cudn-localnet
volumes:
- name: datavolumedisk
  dataVolume:
    name: fedora-localnet-volume
- name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd: { expire: False }
      packages:
      - python3
      runcmd:
      - echo "<h1>Welcome to OpenShift Virtualization Localnet!</h1>" >
/root/index.html
      - cd /root && nohup python3 -m http.server 8080 > /dev/null 2>&1 &
EOF

```

## □□□□ 5: □□□□□□□□□□□□

VM □□□□□□□□□□□□□□□□□□□□□□□□□□□□

```

# VM□□□□□□
oc get vm -n vm-guests-localnet

# □□□□VM□□□□□□□□□□
oc get vmi -n vm-guests-localnet

# VM□□□□□□□□□□ (□□□□: cloud-init□□□ fedora/fedora)
virtctl console -n vm-guests-localnet fedora-localnet-vm

```

VM □□□□□□□□□□□□□□□□□□□□□□□□

```

# □□□□□□□□□□□□□□□□ - □□□□ (Pod□□□□□□) □□□□□□ (localnet)
ip addr show

# □□□□□□ - 2□□□□□□□□□□□□□□□□ (□□□□□□□□□□□□□□□□)
# □□□□□□□□□□□□: □□□□□ 100, □□□□□□: □□□□□ 101
ip route show

# □□□□□□□□□□:

default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.172 metric 100
default via 192.168.2.1 dev enp2s0 proto dhcp src 192.168.2.128 metric 101

# □□□□□□□□□□

```

```
ping 8.8.8.8

# Web
curl localhost:8080

# localnet (<localnet-ip> IP)
curl <localnet-ip>:8080
```

```
ping 8.8.8.8

# Web
curl localhost:8080

# localnet (<localnet-ip> IP)
curl <localnet-ip>:8080
```

```
ping 8.8.8.8

# Web
curl localhost:8080

# localnet (<localnet-ip> IP)
curl <localnet-ip>:8080
```

☐ ☐ ☐

[illegible]

- **NodeNetworkConfigurationPolicy** 使用OVN-Kubernetes 的br-ex 实现bridge mapping 功能
- 使用localnet 实现ClusterUserDefinedNetwork 功能
- Pod 使用localnet 实现与宿主机上的VM 通信
- IP 地址管理
- CUDN 使用localnet 实现与宿主机上的VM 通信

11

- [OpenShift - 容器编排引擎](#)
- [OCP Virtualization 与 Localnet 容器网络](#)

## NAD の VLAN localnet の設定

OpenShift Virtualization NetworkAttachmentDefinition (NAD) VLAN Pod  
localnet VLAN Pod  
(VM) VLAN VLAN  
Pod

☐ ☐ ☐ ☐ \_\_\_\_\_

- **NMState Operator:** `nmstate.io`
- **NMState CR:** Operator `nmstate` `nmstate`
- **Worker Node:** bridge mapping `nmstate` Worker Node
- **VLAN** `nmstate`: `nmstate` `VLAN` `nmstate`
- `nmstate`: OpenShift Worker Node `nmstate` `VLAN` `nmstate`

□□□□□□□□:

OpenShift 4.19

## OpenShift 4.7

OpenShift 4.7: OpenShift 4.7

OpenShift Worker Node localnet VLAN  
OpenShift Worker Node localnet VLAN

- OpenShift: OpenShift VLAN OpenShift OpenShift
- **VLAN** OpenShift: OpenShift VLAN ID (100, 200, 300) OpenShift
- OpenShift: OpenShift VLAN OpenShift OpenShift OpenShift

### 1: VM namespace

VLAN OpenShift localnet OpenShift VM namespace OpenShift

**localnet-vlan-namespace.yaml** OpenShift

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: vm-guests-vlan
EOF
```

### 2: bridge mapping

OpenShift localnet-vlan Worker Node br-ex OpenShift bridge mapping  
OpenShift OpenShift OVN-Kubernetes OpenShift VLAN OpenShift localnet  
OpenShift OpenShift OpenShift OpenShift VM VLAN OpenShift OpenShift

**localnet-vlan-bridge-mapping-nncp.yaml** OpenShift

```
oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: localnet-vlan-bridge-mapping
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ''
  desiredState:
    ovn:
      bridge-mappings:
        - localnet: localnet-vlan
          bridge: br-ex
          state: present
EOF
```

### 3: VLAN NetworkAttachmentDefinition

□□□□□□□□□□□□□□VLAN □□□□□□□□ localnet □□□□□□□□□□ NetworkAttachmentDefinition □□□□□□□□

## localnet-vlan-100-nad.yaml

```
oc apply -f - <<EOF
apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: localnet-vlan-100
  namespace: vm-guests-vlan
spec:
  config: '{
    "cniVersion": "0.3.1",
    "name": "localnet-vlan-100",
    "type": "ovn-k8s-cni-overlay",
    "topology": "localnet",
    "netAttachDefName": "vm-guests-vlan/localnet-vlan-100",
    "physicalNetworkName": "localnet-vlan",
    "vlanID": 100,
    "subnets": "192.168.100.0/24",
    "excludeSubnets": "192.168.100.1/32"
  }'
EOF
```

□□□□□:

- **physicalNetworkName: "localnet-vlan":** 网络 AttachmentDefinition (NAD) 的 NNCP 名称  
bridge mapping (bridge-mappings 的 **localnet** 名称)
- **vlanID: 100:** 指定的 VLAN 的 ID
- **subnets:** OVN-Kubernetes IPAM 的 IP 子网
- **excludeSubnets:** 排除的 IP 子网 (CIDR/子网掩码)
- **topology: "localnet":** 指定的 **localnet** 名称

#### 4: VLAN 配置 VM 配置

Pod 0000000000 VLAN 0000 localnet 00000000000000000000 VM 00000000



cloud-init 通过 SSH 连接到云实例。

## fedora-vlan-vm.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
```

```

name: fedora-vlan-vm
namespace: vm-guests-vlan
labels:
  app: fedora-vlan-vm
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-vlan-volume
    spec:
      pvc:
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 33Gi
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
    template:
      metadata:
        labels:
          app: fedora-vlan-vm
      spec:
        domain:
          devices:
            disks:
            - name: datavolumedisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
            interfaces:
            - name: default
              bridge: {}
            - name: vlan_network
              bridge: {}
          resources:
            requests:
              memory: 2Gi
              cpu: 1
          networks:
            - name: default
              pod: {}
            - name: vlan_network
              multus:
                networkName: localnet-vlan-100
          volumes:
            - name: datavolumedisk

```

```

dataVolume:
  name: fedora-vlan-volume
- name: cloudinitdisk
cloudInitNoCloud:
  userData: |
    #cloud-config
    user: fedora
    password: fedora
    chpasswd: { expire: False }
    packages:
      - python3
      - tcpdump
      - net-tools
    runcmd:
      - echo "<h1>Welcome to OpenShift Virtualization VLAN Network!</h1>" >
        /root/index.html
      - echo "<p>VLAN ID: 100</p>" >> /root/index.html
      - echo "<p>Network: 192.168.100.0/24</p>" >> /root/index.html
      - cd /root && nohup python3 -m http.server 8080 > /dev/null 2>&1 &
EOF

```

## 5: VLAN

VM VLAN

```

# VM
oc get vm -n vm-guests-vlan

# VM
oc get vmi -n vm-guests-vlan

# VM (cloud-init fedora/fedora)
virtctl console -n vm-guests-vlan fedora-vlan-vm

```

VM VLAN:

```

# - (Pod) (VLAN)
ip addr show

# - 2 (metric)
# : metric 100 VLAN: metric 101
ip route show

# :
default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.172 metric 100
default via 192.168.100.1 dev enp2s0 proto dhcp src 192.168.100.50 metric 101

#

```

```
ping 8.8.8.8
```

```
# Web 確認
curl localhost:8080
```

```
# VLAN 確認 (<vlan-ip> 任意 IP 指定)
curl <vlan-ip>:8080
```

```
# VLAN 確認 (tcpdump 確認)
# 任意 VLAN 確認
sudo tcpdump -i enp2s0 -nn vlan 100
```

確認

確認

1. VM 内 VLAN 確認:

- 任意 VLAN 確認
- Worker Node 内 bridge mapping 確認
- VLAN ID 確認

2. IP 確認:

- VLAN 内 DHCP 確認

3. 確認:

- 任意 VLAN 確認
- 任意 VLAN 確認

確認

```
# NetworkAttachmentDefinition 確認
oc get network-attachment-definitions -n vm-guests-vlan

# Worker Node 内 bridge mapping 確認
oc get nncp localnet-vlan-bridge-mapping -o yaml

# VM 確認
oc get vmi -n vm-guests-vlan -o yaml | grep -A 10 networks

# OVN-Kubernetes 確認
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node
```

確認

確認:

- Worker Node 内 **NodeNetworkConfigurationPolicy** 内 VLAN 内 bridge mapping 確認

- 11

- # CUDN □□□□ VLAN □□□ localnet □□□□□□□□□□

□ □ □ □

- :

OpenShift 4.19

□ □ □ □ □ □ □ □ □ □

□□□□□□□□: □□□□□□□□ (TRUNK MODE)

Localnet VLAN OpenShift Worker Node

- ## 1: VM namespace

VLAN 100 Localnet 10.10.10.10/24 VM 1 namespace 100 ClusterUserDefinedNetwork (CUDN)



namespace namespace

```
# 1 namespace
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: vm-guests-vlan-prod
EOF

# 2 namespace
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: vm-guests-vlan-dev
EOF
```

## 2: NIC OVS

VLAN (NIC) OVS

```
oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: br-vlan-creation
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ''
  desiredState:
    interfaces:
      - name: enp11s0f0np0 # Replace with your physical interface name
        type: ethernet
        state: up
        ipv4:
          enabled: false
        ipv6:
          enabled: false
        description: "Physical interface for VLAN localnet"
      - name: br-vlan
        type: ovs-bridge
        state: up
        bridge:
          port:
            - name: enp11s0f0np0
          description: "OVS bridge for VLAN traffic"
EOF
```



enp11s0f0np0

```
oc debug node/<node-name> -- ip link show
```

```
oc get nncp br-vlan-creation
# : STATUS: Available
```

### 3: bridge mapping

Worker Node **br-vlan** OVS bridge mapping  
VLAN Localnet OVN-Kubernetes

```
oc apply -f - <<EOF
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: cudn-localnet-vlan-bridge-mapping
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ''
  desiredState:
    ovn:
      bridge-mappings:
        - localnet: localnet-vlan
          bridge: br-vlan
          state: present
EOF
```

### 4: VLAN ClusterUserDefinedNetwork

namespace VLAN Localnet  
ClusterUserDefinedNetwork

```
oc apply -f - <<EOF
apiVersion: k8s.ovn.org/v1
kind: ClusterUserDefinedNetwork
metadata:
  name: cudn-localnet-vlan-100
spec:
  namespaceSelector:
    matchExpressions:
      - key: kubernetes.io/metadata.name
        operator: In
        values: ["vm-guests-vlan-prod", "vm-guests-vlan-dev"]
```



```

    namespace: openshift-virtualization-os-images
template:
  metadata:
    labels:
      app: fedora-cudn-vlan-vm
  spec:
    domain:
      devices:
        disks:
          - name: datavolumedisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        interfaces:
          - name: default
            bridge: {}
          - name: cudn_vlan_network
            bridge: {}
        resources:
          requests:
            memory: 2Gi
            cpu: 1
      networks:
        - name: default
          pod: {}
        - name: cudn_vlan_network
      multus:
        networkName: cudn-localnet-vlan-100
    volumes:
      - name: datavolumedisk
        dataVolume:
          name: fedora-cudn-vlan-volume
      - name: cloudinitdisk
    cloudInitNoCloud:
      networkData: |
        version: 2
        ethernet:
          enp2s0:
            addresses: ["192.168.100.10/24"]
      userData: |
        #cloud-config
        user: fedora
        password: fedora
        chpasswd: { expire: False }
        packages:
          - python3
          - tcpdump
          - net-tools
        write_files:

```

```

- path: /root/index.html
  content: |
    <h1>Welcome to OpenShift Virtualization CUDN VLAN Network!</h1>
    <p>VLAN ID: 100</p>
    <p>Static IP: 192.168.100.10/24</p>
    <p>Network: CUDN Localnet</p>
    <p>Namespace: vm-guests-vlan-prod</p>
  permissions: '0644'
  runcmd:
    - nohup python3 -m http.server 8080 --directory /root > /dev/null 2>&1 &

```

EOF

## □□□□ 6: 2 □□□ namespace □□ VM □□□□□

□□□□ IP □□□□□□ CUDN □□□□□2 □□□ namespace □ VM □□□□□□□□ namespace □□□□□□□□□□□□□□□□□

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-cudn-vlan-vm-dev
  namespace: vm-guests-vlan-dev
  labels:
    app: fedora-cudn-vlan-vm-dev
spec:
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: fedora-cudn-vlan-volume-dev
      spec:
        pvc:
          accessModes:
            - ReadWriteOnce
          resources:
            requests:
              storage: 33Gi
          sourceRef:
            kind: DataSource
            name: fedora
            namespace: openshift-virtualization-os-images
        template:
          metadata:
            labels:
              app: fedora-cudn-vlan-vm-dev
          spec:
            domain:
              devices:
                disks:
                  - name: datavolumedisk
                    disk:

```

```

        bus: virtio
    - name: cloudinitdisk
      disk:
        bus: virtio
    interfaces:
    - name: default
      bridge: {}
    - name: cudn_vlan_network
      bridge: {}
    resources:
      requests:
        memory: 2Gi
        cpu: 1
    networks:
    - name: default
      pod: {}
    - name: cudn_vlan_network
      multus:
        networkName: cudn-localnet-vlan-100
    volumes:
    - name: datavolumedisk
      dataVolume:
        name: fedora-cudn-vlan-volume-dev
    - name: cloudinitdisk
      cloudInitNoCloud:
        networkData: |
          version: 2
          ethernet:
            enp2s0:
              addresses: ["192.168.100.20/24"]
        userData: |
          #cloud-config
          user: fedora
          password: fedora
          chpasswd: { expire: False }
          packages:
            - python3
            - tcpdump
            - net-tools
          write_files:
            - path: /root/index.html
              content: |
                <h1>Welcome to OpenShift Virtualization CUDN VLAN Network!</h1>
                <p>VLAN ID: 100</p>
                <p>Static IP: 192.168.100.20/24</p>
                <p>Network: CUDN Localnet</p>
                <p>Namespace: vm-guests-vlan-dev</p>
              permissions: '0644'
        runcmd:
          - nohup python3 -m http.server 8080 --directory /root > /dev/null 2>&1 &

```

□□□□ 7: **VLAN** □□□□□□□□□□

VM namespace VLAN

```
# oc namespace -n VM
oc get vm -n vm-guests-vlan-prod
oc get vmi -n vm-guests-vlan-prod

# oc namespace -n VM
oc get vm -n vm-guests-vlan-dev
oc get vmi -n vm-guests-vlan-dev

# oc VM (image: fedora/fedora-cloud-init)
virtctl console -n vm-guests-vlan-prod fedora-cudn-vlan-vm

# oc VM (image: fedora/fedora-cloud-init)
virtctl console -n vm-guests-vlan-dev fedora-cudn-vlan-vm-dev
```

VM VLAN

```
# podman pod run --rm --net=host --ip=10.128.0.174 --mac=02:00:00:00:00:00 -p 8080:8080 --name=nginx nginx
ip addr show

# podman pod run --rm --net=host --ip=10.128.0.174 --mac=02:00:00:00:00:00 -p 8080:8080 --name=nginx nginx
ip route show

# podman pod run --rm --net=host --ip=10.128.0.174 --mac=02:00:00:00:00:00 -p 8080:8080 --name=nginx nginx
default via 10.128.0.1 dev enp1s0 proto dhcp src 10.128.0.174 metric 100

# podman pod run --rm --net=host --ip=10.128.0.174 --mac=02:00:00:00:00:00 -p 8080:8080 --name=nginx nginx
ping 8.8.8.8

# podman pod run --rm --net=host --ip=10.128.0.174 --mac=02:00:00:00:00:00 -p 8080:8080 --name=nginx nginx
curl localhost:8080

# podman pod run --rm --net=host --ip=10.128.0.174 --mac=02:00:00:00:00:00 -p 8080:8080 --name=nginx nginx
ip addr show enp2s0

# podman pod run --rm --net=host --ip=10.128.0.174 --mac=02:00:00:00:00:00 -p 8080:8080 --name=nginx nginx
ping 192.168.100.20

# podman pod run --rm --net=host --ip=10.128.0.174 --mac=02:00:00:00:00:00 -p 8080:8080 --name=nginx nginx
sudo tcpdump -i enp2s0 -nn vlan 100
```





CUDN

- CUDN `physicalNetworkName` bridge mapping (localnet-vlan)
- namespace CUDN NAD: `oc get network-attachment-definitions -A | grep cudn-localnet-vlan-100`
- IPAM (cloud-init IP Disabled)

```
# ClusterUserDefinedNetwork
oc get clusteruserdefinednetwork

# Worker Node bridge mapping
oc get nncp cudn-localnet-vlan-bridge-mapping -o yaml

# namespace VM
oc get vmi -n vm-guests-vlan-prod -o yaml | grep -A 10 networks
oc get vmi -n vm-guests-vlan-dev -o yaml | grep -A 10 networks

# OVN-Kubernetes
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node

# namespace CUDN
oc get network-attachment-definitions -A | grep cudn-localnet-vlan-100
```

CUDN NetworkAttachmentDefinition

|      | CUDN      | NetworkAttachmentDefinition |
|------|-----------|-----------------------------|
|      | namespace | namespace                   |
|      |           | namespace                   |
| VLAN | vlanID    | vlanID                      |
| IP   | /Disabled | IPAM/DHCP                   |
|      |           |                             |

- NIC OVS VLAN bridge mapping
- namespace VLAN ClusterUserDefinedNetwork
- CUDN VLAN cloud-init IP VM
- CUDN VLAN VM namespace
- VLAN CUDN NetworkAttachmentDefinition

11

- OpenShift - 容器编排引擎 (CaaS)
- ClusterUserDefinedNetwork 网络 (CaaS)

## Layer 2

11

VirtualMachines layer2

VirtualMachines Pod

- **namespace** (namespace) VM
- namespace VM
- VM namespace
- 
- 
- 

□□□□□□□□:

OpenShift 4.20,4.21

□ □ □ □

- OpenShift Virtualization Operator [OpenShift 4.20](#)
- `NetworkAttachmentDefinition`
- CLI: `oc`, `virtctl`
- Kubernetes

11

□□□□□□□ 3 □□□□□□□□□□□□□□□□

- namespace 命名空间
- NetworkAttachmentDefinition 网络接口
- VirtualMachines 虚拟机
  - 创建 VirtualMachine 虚拟机
  - 2 个 VirtualMachine 虚拟机
- 部署模板

## namespace

NetworkAttachmentDefinition VirtualMachines namespace (OpenShift) namespace

YAML oc namespace

```
oc create namespace example-l2-secondary
```

namespace kube context (namespace) namespace (OpenShift project)

```
oc project example-l2-secondary
```

namespace context NetworkAttachmentDefinition

## NetworkAttachmentDefinition

2 namespace layer2 NetworkAttachmentDefinition

l2-network.nad.yaml YAML

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: l2-network
  namespace: example-l2-secondary
spec:
  config: |-
    {
      "cniVersion": "0.3.1",
      "name": "l2-network",
      "type": "ovn-k8s-cni-overlay",
      "topology": "layer2",
      "mtu": 1400,
      "netAttachDefName": "example-l2-secondary/l2-network"
    }
```

oc apply OpenShift namespace example-l2-secondary NetworkAttachmentDefinition

```
oc apply -f l2-network.nad.yaml
```

NetworkAttachmentDefinition l2-network namespace example-l2-secondary

```
oc get net-attach-def l2-network -n example-l2-secondary
```

**l2-network** [VirtualMachines](#) [l2-network](#) [l2-network](#)

## VirtualMachines [VirtualMachines](#)

[VirtualMachines](#) [VirtualMachines](#) [layer2](#) [VirtualMachines](#) [VirtualMachines](#) OpenShift (Kubernetes) [VirtualMachine](#) API [VirtualMachine](#)

[VirtualMachines](#) [l2-network](#) [l2-network](#) 2 [VM](#) [VirtualMachines](#)

[VirtualMachine](#) [VirtualMachine](#)

[VirtualMachine](#) [YAML](#) [VirtualMachine](#) (VirtualMachines) [VirtualMachine](#) [fedora-l2-secondary.vm.yaml](#) [VirtualMachine](#)



[cloud-init](#) [cloud-init](#) [SSH](#) [SSH](#)

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-l2-secondary
  labels:
    app: fedora-l2-secondary
spec:
  dataVolumeTemplates:
  - metadata:
      name: fedora-l2-secondary-root
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
        resources:
          requests:
            storage: 30Gi
  instancetype:
    name: u1.large
  preference:
    name: fedora
  runStrategy: Always
  template:
    metadata:
      labels:
        app: fedora-l2-secondary
    spec:
      domain:
        devices:
```

```

    interfaces:
      - name: default
        masquerade: {}
      - name: secondary
        bridge: {}
    resources: {}
  terminationGracePeriodSeconds: 180
  networks:
    - name: default
      pod: {}
    - name: secondary
      multus:
        networkName: l2-network
  volumes:
    - name: fedora-l2-secondary-root
      dataVolume:
        name: fedora-l2-secondary-root
    - name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd:
        expire: False
  networkData: |
    version: 2
    ethernets:
      enp2s0:
        addresses:
          - 192.168.20.10/24

```

VM の定義

spec.template.spec の定義

**domain.devices.interfaces** - 2 の定義

- **name** - **networks** の **secondary**
- **bridge** **masquerade** - **bridge** Pod の **{}**

**networks** - **interfaces** の 2 の定義

- **name** - **default** **secondary**
- **pod** **multus** - **multus** **pod** **multus** **networkName**
  - **networkName** (**multus** の) - **NetworkAttachmentDefinition** **l2-network**



**volumes** - **fedora-l2-secondary-root** VM  
cloudinitdisk 2

**oc create** namespace (**example-l2-secondary**) **VirtualMachine**

```
oc create -f fedora-l2-secondary.vm.yaml
```

**fedora-l2-secondary** VM **example-l2-secondary** namespace **Provisioning**

```
oc get vm fedora-l2-secondary -n example-l2-secondary
```

**VirtualMachine** **Pending** VM



- **DataVolume** ( )
- **l2-network** **NetworkAttachmentDefinition**
- **OpenShift** **vCPU** **VM**

(**vmi****datavolume****pvc**) **oc describe** **oc get -o yaml** **Events (oc describe)** **Status (oc get -o yaml)**

## 2 **VirtualMachine**

2 **VM**

2 **VirtualMachine** **YAML** **fedora-l2-secondary2.vm.yaml**

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-l2-secondary2
  labels:
    app: fedora-l2-secondary2
spec:
  dataVolumeTemplates:
    - metadata:
        name: fedora-l2-secondary2-root
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
```

```

    namespace: openshift-virtualization-os-images
  storage:
    resources:
      requests:
        storage: 30Gi
 instancetype:
    name: u1.large
  preference:
    name: fedora
  runStrategy: Always
  template:
    metadata:
      labels:
        app: fedora-l2-secondary2
  spec:
    domain:
      devices:
        interfaces:
          - name: default
            masquerade: {}
          - name: secondary
            bridge: {}
        resources: {}
    terminationGracePeriodSeconds: 180
    networks:
      - name: default
        pod: {}
      - name: secondary
        multus:
          networkName: l2-network
    volumes:
      - name: fedora-l2-secondary2-root
        dataVolume:
          name: fedora-l2-secondary2-root
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            user: fedora
            password: fedora
            chpasswd:
              expire: False
          networkData: |
            version: 2
            ethernet:
              enp2s0:
                addresses:
                  - 192.168.20.20/24

```

```
oc apply -f fedora-l2-secondary2.vm.yaml
```

fedora-l2-secondary2 2개의 VM을 example-l2-secondary namespace에 배포합니다. VirtualMachines 배포합니다.

```
oc get vm fedora-l2-secondary2 -n example-l2-secondary
```

배포된 VM을 확인합니다.

layer2 namespace에 VM을 배포합니다. VM을 ping합니다.

example-l2-secondary namespace에 VM을 배포합니다.

- NetworkAttachmentDefinition: l2-network
- VirtualMachine: fedora-l2-secondary (NIC IP: 192.168.20.10)
- VirtualMachine: fedora-l2-secondary2 (NIC IP: 192.168.20.20)

VM (NIC IP)를 ping합니다.

virtctl console fedora-l2-secondary`fedora` ENTER

```
virtctl console fedora-l2-secondary
```

VirtualMachine IP enp2s0 192.168.20.10

```
ip addr show
```

VM fedora-l2-secondary2 (IP: 192.168.20.20) VM ping (CTRL+C ping)

```
ping 192.168.20.20
```

VM CTRL+] 종료

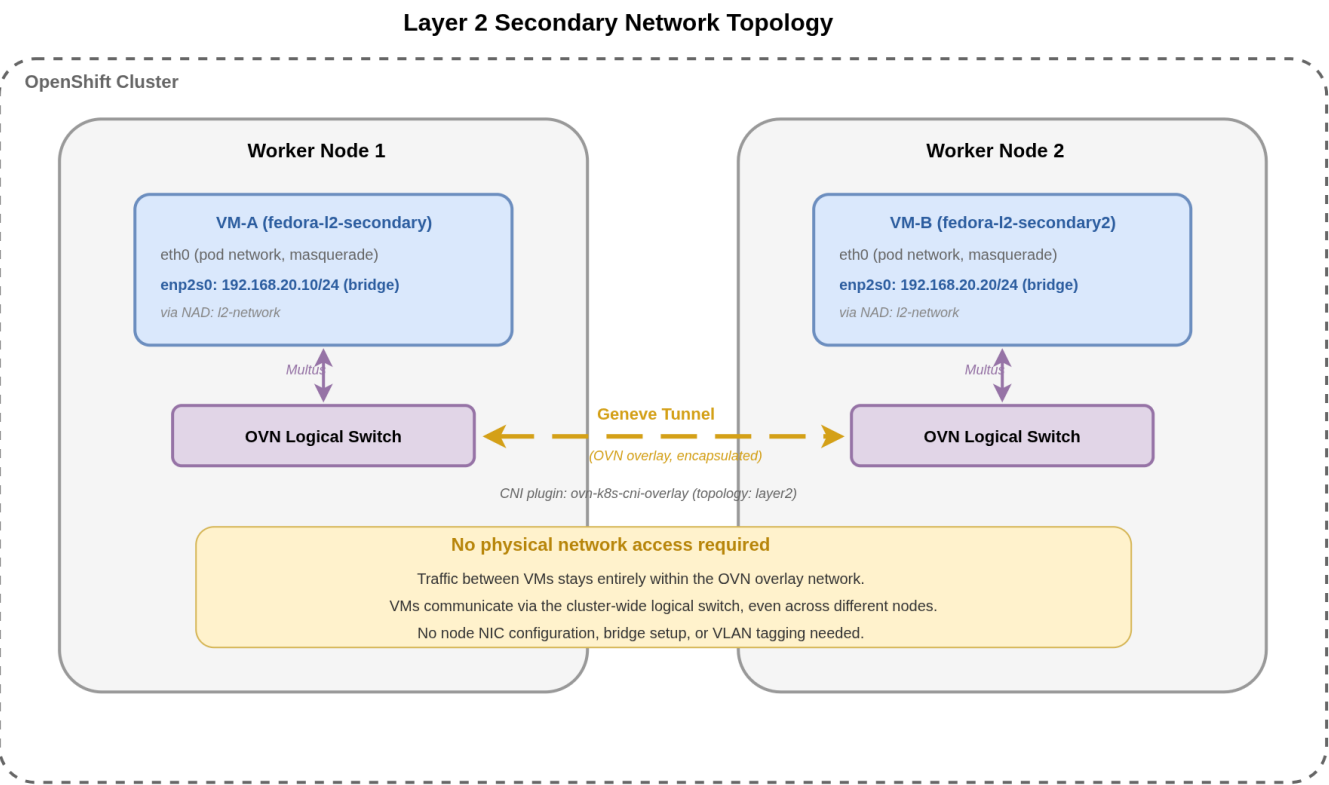
namespace example-l2-secondary namespace

```
oc project default
oc delete namespace example-l2-secondary
```



Layer 2 Secondary Network Topology

Worker Node 2 VM Layer 2  
NIC Geneve  
OVN



layer2 ovn-k8s-cni-overlay CNI OVN Multus VM Node 1 VM-A Node 2 VM-B OVN Geneve Layer 2



Layer 2 OVN NodeNetworkConfigurationPolicy (NNCP) VLAN Layer 2

Layer 2

Layer 2 VM

|           |            |
|-----------|------------|
|           |            |
| east-west | Layer 2 VM |
|           | NIC VM     |
| VM        | VM         |



□ □ □ □

- OpenShift Virtualization Operator 管理 OpenShift 4.20 的虚拟机
- 管理 NMState Operator 网络配置
- NodeNetworkConfigurationPolicy 网络配置 NetworkAttachmentDefinition 网络配置
- 管理 CLI 工具: `oc`, `virtctl`
- Worker Node 管理 (管理虚拟机)

11

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- [Linux bridge](#) `bridge`
- [NetworkAttachmentDefinition](#) `NetAttDef` (Pod 通过 VM 网络)
- [VirtualMachine](#) `VM` (Linux bridge 通过 VM 网络)
  - 1 个 [VirtualMachine](#) `VM`
  - 2 个 [VirtualMachine](#) `VM`
- [Linux bridge](#) `bridge`

# Linux bridge Node Network Configuration Policy

NodeNetworkConfigurationPolicy (nncp) Linux bridge nncp namespace nncp

□□□□□:



- **NodeNetworkConfigurationPolicy** 管理Kubernetes NMState Operator 配置
- **nncp** 管理 VM 配置

```

NodeNetworkConfigurationPolicy  lnbr0-bridge.nncp.yaml
2  2  3  (

```

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: lnbr0-bridge
spec:
  desiredState:
    interfaces:
      - name: lnbr0
        description: layer2-only linux bridge with an optional port
        type: linux-bridge
        state: up
```

```

ipv4:
  enabled: false
bridge:
  options:
    stp:
      enabled: false
# .....
# .....
#port:
#- name: ens1

```

.....:

- **lnbr0** (**linux bridge 0** .....): ..... **Linux bridge** ..... .....
- **ipv4**: ..... **IP** ..... **IP** .....
- **options.stp.enabled**: ..... (.....) □
- **port** (.....): ..... (..... 1 .....)
- **name**: (.....) ..... **ens1** .....



..... **NMState API** .....

..... **lnbr0-bridge.nncp.yaml** ..... **nncp** ..... **OpenShift** .....

```
oc create -f lnbr0-bridge.nncp.yaml
```

..... **lnbr0** .....

..... **NodeNetworkConfigurationPolicy** ..... **NodeNetworkConfigurationEnactment** (**nncp**) .....

```
oc get nncp
```

..... **Progressing** ..... **Available** .....

**lnbr0** ..... **NodeNetworkState** ..... (..... **ip addr show** .....)

..... **NodeNetworkState** ..... **nns** ..... **grep** .....

```
oc describe nns | grep lnbr0
```



**nmstate** ..... (**NMState** ..... **Operator** .....) ..... **nncp** ..... **namespace** ..... **nncp** .....

..... **Linux bridge** ..... **NetworkAttachmentDefinition** .....

## Linux bridge NetworkAttachmentDefinition

Pod/VM L2 NetworkAttachmentDefinition NetworkAttachmentDefinition namespace Pod/VM



NetworkAttachmentDefinition

VirtualMachine

nncp

NIC VM

VM namespace

```
oc create namespace bridge-demo
```

namespace

```
oc project bridge-demo
```

NetworkAttachmentDefinition lnbr0-bridge.net-attach-def.yaml nncp vlan ID

```
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: lnbr0-bridge
  namespace: bridge-demo
spec:
  config: '{
    "name": "lnbr0-bridge",
    "cniVersion": "0.3.1",
    "type": "bridge",
    "bridge": "lnbr0",
    "ipam": {},
    "vlan": 1,
    "macspoofchk": false,
    "disableContainerInterface": false,
    "portIsolation": false
  }'
```

( )

- **metadata.name:** NetworkAttachmentDefinition Kubernetes
- **metadata.namespace:** Pod/VM namespace (bridge-demo) namespace
- **spec.config:** JSON
  - **"name":** metadata.name

- **"cniVersion"**: 指定 CNI 的版本 **"0.3.1"**
- **"type"** (必): Linux bridge 网络类型 **"bridge"**
- **"bridge"** (必): NNCN 网络名称/接口名称
- **"ipam"** (必): OpenShift Virtualization 中 bridge CNI 的 IP 地址管理 (layer2 必)
- **"vlan"** (必): 指定 VLAN ID (必 1)
- **"macspoofchk"** (必): MAC 地址欺骗检查 (必 false)
- **"disableContainerInterface"** (必): 是否禁用容器接口 (veth peer) (必 false)
- **"portIsolation"** (必): 是否启用端口隔离 (Isolation 必 false)

Linux bridge CNI (OpenShift Virtualization) bridge CNI  
OpenShift

IP 00000000000000000000000000000000:

- **VirtualMachine** **NetworkAttachmentDefinition** **IP** (IPAM) **OpenShift Virtualization** `"ipam": {}`
- **Linux bridge** **VM** **IP**
  - **cloud-init networkData:** VM **cloud-init** **IP** (IPAM)
  - **DHCP** **DHCP** **VM** **IP**
  - **nmcli** **IP**
- **OpenShift Virtualization** **Linux bridge bonding mode**

```
lnbr0-bridge.net-attach-def.yaml  network-attachment-definition  OpenShift
network-attachment-definition
```

```
oc create -f lnbr0-bridge.net-attach-def.yaml
```

[illegible]

```
oc get net-attach-def lnbr0-bridge
```

3 2 VirtualMachine

[illegible]

2 VirtualMachine

□□□□□□□□□□□□□□□□□□□□□□□□□□□□

- `kubernetes-nmstate-operator` `nmstate` `NetworkManager` (NetworkManager) `NetworkManager`
- `NodeNetworkConfigurationPolicy` `OpenShift` `Linux bridge` `NetworkManager`
- `NodeNetworkConfigurationEnactment` `NetworkManager`
- `NodeNetworkState` `NetworkManager`
- (Multus) `NetworkAttachmentDefinition` `VirtualMachine` `Linux bridge` `NetworkManager` (NetworkManager) `NetworkManager`

`VirtualMachine` `NetworkManager`

`VirtualMachine` `NetworkManager`

`VirtualMachine` `YAML` `(kind: VirtualMachine)` `oc apply`

`VirtualMachine` `YAML` `fedora-bridge-demo.vm.yaml`



`cloud-init` `SSH`

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-bridge-demo
  namespace: bridge-demo
  labels:
    app: fedora-bridge-demo
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-bridge-demo-root
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
        resources:
          requests:
            storage: 30Gi
 instancetype:
    name: u1.large
  preference:
    name: fedora
  template:
    metadata:
      labels:
        app: fedora-bridge-demo
```

```
spec:
  domain:
    devices:
      disks:
        - name: fedora-bridge-demo-root
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
      interfaces:
        - masquerade: {}
          name: default
        - bridge: {}
          name: nic-linux-bridge
    networks:
      - name: default
        pod: {}
      - multus:
          networkName: lnbr0-bridge
          name: nic-linux-bridge
    volumes:
      - name: fedora-bridge-demo-root
        dataVolume:
          name: fedora-bridge-demo-root
      - name: cloudinitdisk
    cloudInitNoCloud:
      networkData: |
        version: 2
        ethernet:
          enp2s0:
            addresses:
              - 192.168.50.10/24
      userData: |
        #cloud-config
        user: fedora
        password: fedora
        chpasswd: { expire: False }
```

この YAML は、`spec.template.spec` の構造を示しています。

- **domain.devices.interfaces:** `default` という名前の `nic-linux-bridge` という 2 つのネットワークインターフェースを定義します。
  - **bridge:** VM 内の NIC をブリッジとして 2 つのネットワークに 1 つ (この 1 つは IP `masquerade`) を提供し、`vm-to-container` のネットワークを `lnbr0` という `container-to-node` のネットワーク (ネットワーク) に接続します。
  - **name:** この `network` の名前を指定します。
- **networks:** `interfaces` を定義します。
  - **name: interfaces** このネットワークを定義します。
  - **multus.networkName:** Pod/VM が接続する `NetworkAttachmentDefinition (NAD)` の名前。



- **volumes:** 2 볼륨 (1. **datavolumedisk** 1 개 **cloudinitdisk**)
  - **CloudInitNoCloud:** cloud-init (NoCloud) KubeVirt
    - **networkData:** (enp2s0) IP (192.168.50.10/24)
      - **ethernets:** enp2s0 KubeVirt 2



(enp1s0) Masquerade Pod IP

**VirtualMachine** `fedora-bridge-demo.vm.yaml` OpenShift

```
oc apply -f fedora-bridge-demo.vm.yaml
```

**VirtualMachine** VM **DataVolume** VM **Running**

```
oc get vm fedora-bridge-demo
```

## 2 VirtualMachine

**VirtualMachine** **vm** (IP) **YAML** `fedora-bridge-demo2.vm.yaml`

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-bridge-demo2
  namespace: bridge-demo
  labels:
    app: fedora-bridge-demo2
spec:
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-bridge-demo2-root
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
        resources:
          requests:
            storage: 30Gi
  instancetype:
    name: u1.large
  preference:
```

```

name: fedora
template:
  metadata:
    labels:
      app: fedora-bridge-demo2
  spec:
    domain:
      devices:
        disks:
          - name: fedora-bridge-demo2-root
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        interfaces:
          - masquerade: {}
            name: default
          - bridge: {}
            name: nic-linux-bridge
        networks:
          - name: default
            pod: {}
          - multus:
              networkName: lnbr0-bridge
              name: nic-linux-bridge
      volumes:
        - name: fedora-bridge-demo2-root
          dataVolume:
            name: fedora-bridge-demo2-root
        - name: cloudinitdisk
      cloudInitNoCloud:
        networkData: |
          version: 2
          ethernet:
            enp2s0:
              addresses:
                - 192.168.50.20/24
      userData: |
        #cloud-config
        user: fedora
        password: fedora
        chpasswd: { expire: False }

```

~~~~~ `fedora-bridge-demo2.vm.yaml` ~~~~~ 2 ~~~ `VirtualMachine` ~ `OpenShift` ~~~~~

```
oc create -f fedora-bridge-demo2.vm.yaml
```

~~~ VM ~~~~~ `VirtualMachineInstance` (`vmi`) ~~~~~ `interfaces` ~ `status` ~~~~~

```
oc get vm -n bridge-demo
oc get vmi fedora-bridge-demo -n bridge-demo -o jsonpath='{.status.interfaces}' | jq .
oc get vmi fedora-bridge-demo2 -n bridge-demo -o jsonpath='{.status.interfaces}' | jq .
```

2 VM 相互通信を確認する

## Linux bridge 相互通信を確認する

Linux bridge 相互通信を確認する(bridge namespace) Pod VM namespace 相互通信

Linux bridge 相互通信を確認する VirtualMachine 相互 VirtualMachine 相互通信を確認する

bridge-demo namespace 相互通信を確認する

- NetworkAttachmentDefinition: lnbr0-bridge
- VirtualMachine: fedora-bridge-demo (NIC IP: 192.168.50.10)
- VirtualMachine: fedora-bridge-demo2 (NIC IP: 192.168.50.20)

相互通信を確認する VM (相互通信を確認する VM) ping 相互通信を確認する

virtctl 相互通信を確認する VirtualMachine 相互通信を確認する user: `fedora` password: `fedora` 相互通信を確認する (相互通信を確認する ENTER 相互通信を確認する)

```
virtctl console fedora-bridge-demo
```

VirtualMachine IP 相互通信を確認する enp2s0 相互通信を確認する 192.168.50.10 相互通信を確認する

```
ip addr show
```

相互通信を確認する VM 相互通信を確認する fedora-bridge-demo2 (IP: 192.168.50.20) 相互通信を確認する VirtualMachine 相互通信を確認する ping 相互通信を確認する (CTRL+C 相互通信を確認する ping 相互通信を確認する)

```
ping 192.168.50.20
```

VM 相互通信を確認する CTRL+] 相互通信を確認する

相互通信を確認する namespace 相互通信を確認する bridge-demo namespace 相互通信を確認する

```
oc project default
oc delete namespace bridge-demo
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- 11

- ## SR-IOV □□□□□□□□□□

□□□□□□□□:

SR-IOV (Single Root I/O Virtualization) 仮想化 (VF) 仮想化 (VM) 仮想化 (OVN overlay Linux bridge) SR-IOV VM SR-IOV (NFV) 仮想化

SR-IOV 対応ネットワークアダプタは VLAN 対応 VLAN 対応ネットワークアダプタに SR-IOV VF 対応 VLAN 対応ネットワークアダプタ VM 対応ネットワークアダプタ (UPF) 対応ネットワークアダプタ VLAN 対応ネットワークアダプタ (Telco) 対応ネットワークアダプタ

SR-IOV Network Operator SR-IOV VLAN 2 SR-IOV  
SR-IOV VM

□ □ □ □

- 295



SR-IOV 配置を完了する NIC 設定を完了する Machine Config Operator による SR-IOV Network Operator 設定を完了する

## ステップ 1: SR-IOV Operator のインストール

OperatorHub から SR-IOV Network Operator をインストールする ClusterServiceVersion は **Succeeded** 状態にある Operator Pod をインストールする

```
oc get csv -n openshift-sriov-network-operator
```

インストール (インストール済み):

| NAME                                        | DISPLAY                 | VERSION             |
|---------------------------------------------|-------------------------|---------------------|
| sriov-network-operator.v4.21.0-202605200241 | SR-IOV Network Operator | 4.21.0-202605200241 |
| Succeeded                                   |                         |                     |

Operator Pod をインストールする:

```
oc get pods -n openshift-sriov-network-operator
```

Operator webhook injector Pod は **Running** 状態にある SR-IOV Worker Node 上の **sriov-network-config-daemon** Pod をインストールする

## ステップ 2: SR-IOV のインストール

SR-IOV Network Operator をインストールする SR-IOV SriovNetworkNodeState をインストールする

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator
```

インストール (インストール済み):

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o yaml
```

**.status.interfaces** 内の **totalvfs** は 0 である。これは PCI 設定をインストールする ID をインストールする VF をインストールする

Mellanox ConnectX-4 Lx をインストールする:

```
- deviceID: "1015"
  driver: mlx5_core
```

```
linkSpeed: 25000 Mb/s
linkType: ETH
name: enp11s0f0np0
pciAddress: "0000:0b:00.0"
totalvfs: 6
vendor: "15b3"
```



00000000000000000000IOMMU 00000000000000000000 0000000000000000 IOMMU  
00000000000000000000:

```
oc debug node/<node-name> -- chroot /host ls /sys/kernel/iommu_groups/ | wc -l
```

0 00000000 (0000000000) 000000IOMMU 00000000000000000000 00000 0 00000BIOS/UEFI 0 IOMMU  
000000MachineConfig 00000000000000000000 intel\_iommu=on iommu=pt (Intel) 000 amd\_iommu=on iommu=pt  
(AMD) 000000000000

### 0000 3: SrioVNetworkNodePolicy 0000

SrioVNetworkNodePolicy 0000000000 (PF) 00000000 VF 00000000000000000000 Operator 00000000 OpenShift  
Virtualization 0 VM 00000deviceType: vfio-pci 00000000VF 0 VFIO 000000000000000000000000 PCI  
0000000000000000

0000000000000000 4 00 VF 00000000 00000000 VF 0000 VLAN 0000000000VLAN 0000000000 SrioVNetwork  
000000000000 0000VF 000000000000 VLAN 0000000000000000

srioV-network-node-policy.yaml 00000000

```
oc apply -f - <<EOF
apiVersion: srioVnetwork.openshift.io/v1
kind: SrioVNetworkNodePolicy
metadata:
  name: mlx-srioV-policy
  namespace: openshift-srioV-network-operator
spec:
  resourceName: mlxsrioVnic
  nodeSelector:
    kubernetes.io/hostname: <node-name>
  numVfs: 4
  nicSelector:
    vendor: "15b3"
    deviceID: "1015"
    pfNames:
      - enp11s0f0np0
  deviceType: vfio-pci
  isRdma: false
EOF
```

<node-name> SR-IOV NIC (例: ocp1ab01) の名前を指定

オプション:

- **resourceName** — VF の名前 Kubernetes の名前空間 (例: **openshift.io/mlxsriovnic**)
- **numVfs** — 指定した VF の数 (デフォルトは **totalvfs** の値) VM に 2 つの VF (VLAN ID 1) を割り当てる 2 つの VM を作成
- **nicSelector.vendor / deviceID** — NIC の名前空間 Mellanox ConnectX-4 Lx の ID **15b3 / 1015** Intel NIC の ID を指定
- **nicSelector.pfNames** — 指定した VF の名前空間
- **deviceType: vfio-pci** — VM の名前空間 Pod の名前空間 **netdevice** を指定



SR-IOV の名前空間 VF の名前空間 (drain) の名前空間 Operator の名前空間 Machine Config Operator の名前空間

コマンドを実行:

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o jsonpath='{.items[0].status.syncStatus}'
```

コマンドの結果 **Succeeded** が表示されます

ステップ 4: 確認

指定した (デフォルト) の VF の名前空間

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o yaml
```

**.status.interfaces** の名前空間 PF の名前 **numVfs: 4** の VF PCI の名前空間

コマンドを実行:

```
oc debug node/<node-name> -- chroot /host cat /sys/class/net/enp1s0f0np0/device/sriov_numvfs
```

コマンドの結果 **numVfs** の値 (例: **4**) が表示されます

VF の名前空間:

```
oc get node <node-name> -o jsonpath='{.status.allocatable.openshift.io/mlxsriovnic}'
```

コマンド:

## 例 5: SR-IOV VLAN の作成

SR-IOV VLAN を作成するには、`SriovNetwork` オペレーターが `NetworkAttachmentDefinition` (NAD) を作成し、NAD を VM が VM の `VLAN ID` を OS に渡す。

VM の namespace:

`sriov-demo-namespace.yaml`

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: sriov-demo
EOF
```

VLAN 100 の作成:

`sriov-net-data.yaml`

```
oc apply -f - <<EOF
apiVersion: sriovnetwork.openshift.io/v1
kind: SriovNetwork
metadata:
  name: sriov-net-mgmt
  namespace: openshift-sriov-network-operator
spec:
  networkNamespace: sriov-demo
  resourceName: mlxsrionnic
  vlan: 200
  spoofChk: "off"
  trust: "on"
EOF
```

VLAN 200 の作成:

`sriov-net-mgmt.yaml`

```
oc apply -f - <<EOF
apiVersion: sriovnetwork.openshift.io/v1
kind: SriovNetwork
metadata:
  name: sriov-net-mgmt
  namespace: openshift-sriov-network-operator
```



```
spec:
  networkNamespace: sriov-demo
  resourceName: mlxsrionnic
  vlan: 200
  spoofChk: "off"
  trust: "on"
EOF
```

オプション:

- **vlan** — 仮想 VF 802.1Q VLAN ID NIC 仮想 OS 仮想
- **networkNamespace** — NAD namespace namespace VM
- **resourceName** — `SriovNetworkNodePolicy` resourceName 仮想 VF
- **spoofChk: "off"** **trust: "on"** — OS MAC IP VM

NAD:

```
oc get net-attach-def -n sriov-demo
```

結果:

| NAME           | AGE |
|----------------|-----|
| sriov-net-data | 10s |
| sriov-net-mgmt | 5s  |

## 6: SR-IOV VM

Pod VLAN 100 SR-IOV VLAN 200 SR-IOV 3 VM



cloud-init SSH

**fedora-sriov-vm.yaml**

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-sriov-vm
  namespace: sriov-demo
  labels:
    app: fedora-sriov-vm
spec:
  runStrategy: Always
  dataVolumeTemplates:
```

```

- metadata:
  name: fedora-sriov-volume
spec:
  pvc:
    accessModes:
    - ReadWriteOnce
    resources:
      requests:
        storage: 33Gi
    sourceRef:
      kind: DataSource
      name: fedora
      namespace: openshift-virtualization-os-images
template:
  metadata:
    labels:
      app: fedora-sriov-vm
  spec:
    domain:
      devices:
        disks:
        - name: datavolumedisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
        interfaces:
        - name: default
          masquerade: {}
        - name: data-plane
          sriov: {}
        - name: mgmt-plane
          sriov: {}
      resources:
        requests:
          memory: 4Gi
          cpu: 2
    networks:
    - name: default
      pod: {}
    - name: data-plane
      multus:
        networkName: sriov-net-data
    - name: mgmt-plane
      multus:
        networkName: sriov-net-mgmt
    volumes:
    - name: datavolumedisk
      dataVolume:
        name: fedora-sriov-volume

```

```
- name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd: { expire: False }
```

EOF

VM 3 3:

- **default** (masquerade: {}) — IP (Pod)
- **data-plane** (sriov: {}) — VLAN 100 VF
- **mgmt-plane** (sriov: {}) — VLAN 200 VF

SR-IOV 1 VF VM 4 VF 2 VM

VM 3:

```
oc get vm -n sriov-demo
```

```
oc get vmi -n sriov-demo
```

VMI Running READY: True

## 7: VLAN SR-IOV

SR-IOV VF PCI VLAN

PCI

VM 3:

```
virtctl console -n sriov-demo fedora-sriov-vm
```

fedora / fedora VF PCI:

```
lspci | grep -i ethernet
```

Pod virtio NIC 2 Mellanox (Intel) Ethernet VF VFIO VF PCI

```
ip link show
```

期待の結果:

- **enp1s0** — Pod 仮想ネットワーク (virtio 経由)
- **enp7s0** — SR-IOV VF (VLAN 100)
- **enp8s0** — 2 SR-IOV VF (VLAN 200)



期待の結果はPCI 経由/仮想ネットワーク **udev** 経由の仮想ネットワーク **ip link show** 経由の SR-IOV VF 経由の仮想ネットワーク (virtio 経由の Mellanox 経由 Intel 経由)。

**IP** 期待の結果

SR-IOV 仮想ネットワーク IP 期待の結果 VLAN 期待の結果

期待の結果 (VLAN 100) 期待の結果:

```
sudo ip addr add 10.100.0.10/24 dev enp7s0
```

```
sudo ip link set enp7s0 up
```

期待の結果 (VLAN 200) 期待の結果:

```
sudo ip addr add 10.200.0.10/24 dev enp8s0
```

```
sudo ip link set enp8s0 up
```

期待の結果期待の結果期待の結果期待の結果期待の結果期待の結果:

```
ip -br addr show
```

期待の結果:

|        |         |                     |
|--------|---------|---------------------|
| lo     | UNKNOWN | 127.0.0.1/8 ::1/128 |
| enp1s0 | UP      | 10.0.2.2/24         |
| enp7s0 | UP      | 10.100.0.10/24      |
| enp8s0 | UP      | 10.200.0.10/24      |



期待の結果 **ip addr add** 期待の結果期待の結果期待の結果期待の結果期待の結果期待の結果

[illegible]

- **DHCP** — 在 VLAN 中 DHCP 由 `NetworkManager` 管理
- **cloud-init** — VM 中 `cloud-init` 通过 `networkData` 配置 IP 地址和 IP 掩码
- **NetworkManager** 配置 — `cloud-init` 通过 `write_files` 配置 `NetworkManager` 的 `.nmconnection` 文件

## VLAN 000000

□□□□□□□□□□ VLAN □□□□□□□□□□ VLAN □□□□□□□□ ping □□□□□□□□

□□□□□□ VLAN (100) □□□□□□:

```
ping -c 3 -I enp7s0 10.100.0.1
```

□□ VLAN (200) □□□□□□:

```
ping -c 3 -I enp8s0 10.200.0.1
```

ping 10.10.10.100 10.10.10.100

VLAN 200 ( )

□□□□□□□□□□Ctrl+] □□□□□□

□ □ □ □ □ □ □ □ □ □

VF

- SriovNetworkNodeState 狀態為Failed 時，NIC 無法正常運作
- BIOS/UEFI 中 IOMMU 設定未開啟
- 檢查 nicSelector 值 (IDPF 0) 是否正確

## VM □□□□□□□□□□□□□□□□

- VF 0000000000000000: `oc get node <node-name> -o jsonpath='{.status.allocatable}'`
- 000000 0 0000VF 00000000000000000000000000000000
- 00000 deviceType: vfio-pci 0000000000000000VM 00 VFIO 000000000000
- Insufficient openshift.io/mlxsriovnic 0000000000000000VM 0000000000 VF 000000000000VM 0000 SR-IOV 0000000000000000000000 numVfs 000000000000

## VM SR-IOV 0000000000000000

- `SriovNetwork` `resourceName` `SriovNetworkNodePolicy` `resourceName` `XXXXXXXXXXXX`

- 取得 NAD 所在 namespace 的 Pod 名称: `oc get net-attach-def -n sriov-demo`
- virt-launcher Pod 名称: `oc describe pod -n sriov-demo -l app=fedora-sriov-vm`

## SR-IOV 网络配置

- 检查 NIC 是否支持 SR-IOV: `oc debug node/<node-name>--chroot /host ip link show <pf-name>`
- 配置 VLAN 100 到 200 的 SR-IOV 网络
- Mellanox NIC 的 SR-IOV 网络配置

## VLAN 网络 VM 配置

- `SriovNetwork` 的 `vlan` 配置 VLAN 网络
- 配置 SR-IOV 网络 (配置 SR-IOV) 的 VLAN ID 配置
- 配置 SR-IOV 网络 VF 配置 `SriovNetwork` 的 `spoofChk` 为 "off" 和 `trust` 为 "on"

## 配置

配置 SR-IOV 网络:

```
oc delete vm fedora-sriov-vm -n sriov-demo
```

```
oc delete sriovnetwork sriov-net-data -n openshift-sriov-network-operator
```

```
oc delete sriovnetwork sriov-net-mgmt -n openshift-sriov-network-operator
```

```
oc delete sriovnetworknodepolicy mlx-sriov-policy -n openshift-sriov-network-operator
```

SR-IOV 网络配置 (VF 配置 SR-IOV 网络):

```
oc get sriovnetworknodestates -n openshift-sriov-network-operator -o
jsonpath='{.items[0].status.syncStatus}'
```

配置 namespace:

```
oc delete namespace sriov-demo
```

111

[illegible]

- SR-IOV は PCI 経由で VM 上でネットワークを構築する
- `SriovNetworkNodeState` を利用して SR-IOV を構築する
- VM 上で `deviceType: vfio-pci` を利用して `SriovNetworkNodePolicy` を構築する
- SR-IOV は VLAN を利用する (1 つの VLAN を利用して `SriovNetwork` を構築して VF は VLAN を利用する)
- 100 (VLAN 100) と 200 (VLAN 200) を利用して `SriovNetwork` を構築する
- 100 VLAN を利用して SR-IOV を構築して VM を構築する
- 100 PCI を利用して VLAN を構築する

11

- [OpenShift](#) [ネットワーク](#)
- [VM](#) [SR-IOV](#) [ネットワーク](#)
- [KubeVirt SR-IOV](#) [ネットワーク](#)
- [Localnet](#) [ネットワーク](#) — [ネットワーク](#)
- [Localnet VLAN](#) [ネットワーク](#) — [localnet](#) [VLAN](#) [ネットワーク](#) ([ネットワーク](#))
- [Linux bridge](#) — [ネットワーク](#)

VM

## 11

```

OpenShift   Virtualization   Kubernetes   NetworkPolicy
AdminNetworkPolicy   VM   virt-launcher   Pod   VM
spec.template.metadata.labels   Pod   VM
/

```

VM                      VM                      VM

AdminNetworkPolicy                      VM

□ □ □ □

- OpenShift Virtualization Operator [OpenShift 4.18+](#)
- [OpenShift 4.12](#) [OVN-Kubernetes \(OpenShift 4.12 以前\)](#)
- AdminNetworkPolicy [OpenShift 4.12 以前](#)
- **oc** CLI [OpenShift 4.12 以前](#)

## OpenShift VM Pod

OpenShift Virtualization VM virt-launcher Pod VirtualMachine spec.template.metadata.labels Pod NetworkPolicy podSelector Pod

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: web-vm
  labels:
    app: web
spec:
  template:
    metadata:
      labels:
        app: web ①
        role: frontend ①
    spec:
      # ...
```

① virt-launcher Pod NetworkPolicy podSelector



VirtualMachine metadata.labels virt-launcher Pod spec.template.metadata.labels

## 1: Namespace VM

Namespace Web 2 VM

```
oc create namespace netpol-demo
```

80 Nginx VM

vm-web.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: web-vm
  namespace: netpol-demo
  labels:
    app: web
    role: frontend
spec:
  runStrategy: Always
 instancetype:
```



```

name: u1.small
preference:
  name: fedora
template:
  metadata:
    labels:
      app: web
      role: frontend
  spec:
    domain:
    devices:
      disks:
        - name: containerdisk
          disk: {}
        - name: cloudinitdisk
          disk: {}
      interfaces:
        - name: default
          masquerade: {}
    networks:
      - name: default
        pod: {}
    volumes:
      - name: containerdisk
        containerDisk:
          image: quay.io/containerdisks/fedora:40
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            password: changeme
            chpasswd:
              expire: false
            packages:
              - nginx
            runcmd:
              - systemctl enable --now nginx
EOF

```

000 5432 00000000000000 VM 0000000000

**vm-db.yaml** 00000000

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: db-vm
  namespace: netpol-demo
  labels:

```



```
oc get pods -n netpol-demo -l app=web --show-labels
```

```
oc get pods -n netpol-demo -l app=database --show-labels
```

**Figure 2: Example of a 1000-dimensional vector (1000-dimensional)**

VM Pod IP

```
oc get vmi -n netpol-demo -o 'custom-  
columns=NAME:.metadata.name,IP:.status.interfaces[0].ipAddress'
```

**IP**

Pod 在 default namespace 中运行

```
oc run curl-test -n default --image=registry.access.redhat.com/ubi9/ubi-minimal
--restart=Never -- sleep 3600
```

web VM 公開 IP アドレス (<web-vm-ip> 公開 IP アドレス)

```
oc exec -n default curl-test -- curl -s --connect-timeout 5 http://<web-vm-ip>:80
```

## Nginx 如何返回 HTML 内容

**□□□□ 3:** □□□□□□□□□□□□□□□□

namespace Pod  
Ingress Egress

deny-all.yaml 拒绝所有

```
oc apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: netpol-demo
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
EOF
```

Pod 和 Web VM 的 IP 地址

```
oc exec -n default curl-test -- curl -s --connect-timeout 5 http://<web-vm-ip>:80
```

输出结果

```
command terminated with exit code 28
```

输出结果 28 表示连接超时。这是因为 OVN-Kubernetes 的 Pod 网络策略默认禁止了 TCP 连接。要解决这个问题，需要配置 DNS Egress 策略。

## 4: DNS Egress 策略

默认情况下，(deny-all) 策略会阻止所有 DNS 流量。要允许 DNS 流量，需要配置 DNS Egress 策略。Pod 网络策略 DNS 流量。

[allow-dns-egress.yaml](#) 文件内容

```
oc apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-egress
  namespace: netpol-demo
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: openshift-dns
  ports:
  - protocol: UDP
    port: 5353
  - protocol: TCP
    port: 5353
EOF
```

## 5: Web VM 的 HTTP Ingress 策略

默认情况下，TCP 80 端口 Web VM 的 ingress 流量是被禁止的。要允许 HTTP 流量，需要配置 HTTP Ingress 策略。

[allow-web-ingress.yaml](#) 文件内容

```
oc apply -f - <<EOF
```

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-ingress
  namespace: netpol-demo
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
    - Ingress
  ingress:
    - ports:
        - protocol: TCP
          port: 80
EOF

```

Pod Web VM 80

```
oc exec -n default curl-test -- curl -s --connect-timeout 5 http://<web-vm-ip>:80
```

Nginx Ingress

## 6: Web VM VM

Web VM Egress app: web Pod VM Ingress 2

[allow-web-to-db-egress.yaml](#)

```

oc apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-to-db-egress
  namespace: netpol-demo
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
    - Egress
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: database
      ports:
        - protocol: TCP

```

```
port: 5432
EOF
```

`allow-db-from-web.yaml` ファイル

```
oc apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-db-from-web
  namespace: netpol-demo
spec:
  podSelector:
    matchLabels:
      app: database
  policyTypes:
    - Ingress
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: web
      ports:
        - protocol: TCP
          port: 5432
EOF
```

Pod (`app: web` Pod) がデータベース Pod (データベース Pod) に

```
oc exec -n default curl-test -- curl -s --connect-timeout 5 http://<db-vm-ip>:5432
```

`<db-vm-ip>` は 2 つの VM の IP アドレス

です。

```
command terminated with exit code 28
```

このエラーメッセージは、curl コマンドが失敗したことを示しています。

`allow-db-from-web` ファイルの `namespace` が `app: web` の Pod の ingress ルールで `web` VM の `virt-launcher` pod を指定しています。

ステップ 7: ファイルを確認します。

このコマンドは、`namespace` の `NetworkPolicy` ファイルを確認します。

```
oc get networkpolicy -n netpol-demo
```

□ □ □ □ □ □ □

| NAME                   | POD-SELECTOR | AGE |
|------------------------|--------------|-----|
| allow-db-from-web      | app=database | ... |
| allow-dns-egress       | <none>       | ... |
| allow-web-ingress      | app=web      | ... |
| allow-web-to-db-egress | app=web      | ... |
| deny-all               | <none>       | ... |

AdminNetworkPolicy

NetworkPolicy namespace AdminNetworkPolicy (ANP)  
OVN-Kubernetes

1. **AdminNetworkPolicy** (网络策略) — Allow 允许 Deny 拒绝 Pass 通过
2. **NetworkPolicy** — 网络 namespace 策略
3. **BaselineAdminNetworkPolicy** — NetworkPolicy 基线策略

ANP OpenShift Virtualization namespace

## admin-network-policy.yaml

```
apiVersion: policy.networking.k8s.io/v1alpha1
kind: AdminNetworkPolicy
metadata:
  name: cluster-deny-to-virt-infra
spec:
  priority: 50
  subject:
    namespaces:
      matchExpressions:
        - key: kubernetes.io/metadata.name
          operator: NotIn
          values:
            - openshift-cnv
            - openshift-virtualization-os-images
  ingress:
    - name: deny-ingress-to-virt-infra
      action: Deny
      from:
        - namespaces:
            matchExpressions:
              - key: kubernetes.io/metadata.name
                operator: NotIn
                values:
                  - openshift-cnv
```

```

- openshift-virtualization-os-images
ports:
- portRange:
  start: 1
  end: 65535
  protocol: TCP

```



AdminNetworkPolicy は **v1alpha1** API であるが、API によって OpenShift 4.14 以降 OVN-Kubernetes によってサポートされる **v1** API である。

NetworkPolicy

NetworkPolicy

NetworkPolicy は Pod (NetworkPolicy) Primary UDN) NetworkAttachmentDefinition localnet Linux NetworkPolicy VM NetworkPolicy

NetworkPolicy

VM は virt-launcher Pod spec.template.metadata.labels NetworkPolicy namespace NetworkPolicy KubeVirt

## Masquerade vs Bridge Binding

masquerade interface binding (Pod) VM Pod NAT NetworkPolicy virt-launcher Pod IP bridge binding VM Pod IP virt-launcher Pod

NetworkPolicy

NetworkPolicy

virt-launcher Pod

```
oc get pods -n netpol-demo --show-labels
```

VM metadata.labels spec.template.metadata.labels

## Deny-All DNS

egress DNS 4 DNS egress

NetworkPolicy

policyTypes Ingress egress (Ingress) policyTypes Ingress Egress



□□□□□□□□

□ □ □ □ □ □ □

## Pod namespace

```
oc delete pod curl-test -n default
```

```
oc delete namespace netpol-demo
```

[illegible]

```
oc delete adminnetworkpolicy cluster-deny-to-virt-infra
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- **NetworkPolicy** 限制 virt-launcher Pod 的流量 `spec.template.metadata.labels` 限制 VM 的流量
- **deny-all** 限制所有 VM namespace 的流量
- **Ingress** 限制 Egress 流量
- **DNS Egress** 限制 DNS 流量
- **AdminNetworkPolicy** 限制所有 namespace 的流量
- **NetworkPolicy** 限制所有 namespace 的流量
- **Pod** 限制流量

11

- [VM Ingress 路由 Routes](#) — VM HTTP 负载均衡
- [UDN 负载均衡](#) — 负载均衡器支持 2 个负载均衡策略
- [OpenShift 负载均衡](#)
- [Kubernetes NetworkPolicy 策略](#)

## VM MetalLB

11

OpenShift    MetalLB    LoadBalancer    Service

OpenShift Route    Ingress    HTTP/HTTPS

VM    SSH    RDP    TCP/UDP    HTTP

MetalLB      IP      2      (ARP/NDP)      BGP  
LoadBalancer Service

- OpenShift 4.18+ (OpenShift Virtualization Operator )
- 
- 2 IP (L2 )
- 
- **oc** CLI

## MetalLB   Route

| OpenShift Route         | HTTP/HTTPS     | Web TLS API                             |
|-------------------------|----------------|-----------------------------------------|
| NodePort Service        | TCP/UDP        | (30000-32767)                           |
| LoadBalancer (MetalLB)  | TCP/UDP        | (228044333065432)                       |
| Gateway API (HTTPRoute) | HTTP/HTTPSgRPC | TLS<br>Kubernetes L7 Ingress (OC 4.19 ) |

IP VM Route   Gateway API HTTP MetalLB

## 1: MetalLB Operator

MetalLB Operator namespace

```
oc create namespace metallb-system
```

AllNamespaces OperatorGroup (MetalLB Operator)

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: metallb-operator
  namespace: metallb-system
spec: {}
EOF
```

Red Hat Operators MetalLB Operator

## metallb-subscription.yaml

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: metallb-operator
  namespace: metallb-system
spec:
  channel: stable
  name: metallb-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
EOF
```

## Operator

```
oc wait --for=condition=CatalogSourcesUnhealthy=False subscription metallb-operator -n metallb-system --timeout=120s
```

## ClusterServiceVersion

```
oc get csv -n metallb-system -o custom-columns=NAME:.metadata.name,PHASE:.status.phase
```

PHASE **Succeeded**

## 2: MetalLB

MetalLB IP speaker controller Pod

## metallb-instance.yaml

```
oc apply -f - <<EOF
apiVersion: metallb.io/v1beta1
kind: MetalLB
metadata:
  name: metallb
  namespace: metallb-system
spec: {}
EOF
```

## MetalLB Pod

```
oc wait deployment controller -n metallb-system --for=condition=Available --timeout=120s
```

```
oc get pods -n metallb-system
```

**controller** Deployment **speaker** DaemonSet **Pod** **IP** **AddressPool**

### 3: IP AddressPool

MetalLB **LoadBalancer** **IP** **AddressPool** **2**



**IP** **AddressPool** **Worker Node** **DHCP** **IP**

### ipaddresspool.yaml

```
oc apply -f - <<EOF
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: vm-services-pool
  namespace: metallb-system
spec:
  addresses:
    - 192.168.2.200-192.168.2.210
EOF
```

### 4: L2 Advertisement

L2Advertisement **ARP (IPv4)** **NDP (IPv6)** **IP**

### l2advertisement.yaml

```
oc apply -f - <<EOF
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: vm-services-l2
  namespace: metallb-system
spec:
  ipAddressPools:
    - vm-services-pool
EOF
```

### 5: VM

**namespace** **Nginx Web** **VM**

```
oc create namespace metallb-demo
```

### vm-webserver.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: web-vm
  namespace: metallb-demo
  labels:
    app: web-vm
spec:
  runStrategy: Always
 instancetype:
    name: u1.small
  preference:
    name: fedora
  template:
    metadata:
      labels:
        app: web-vm
    spec:
      domain:
        devices:
          disks:
            - name: containerdisk
              disk: {}
            - name: cloudinitdisk
              disk: {}
          interfaces:
            - name: default
              masquerade: {}
      networks:
        - name: default
          pod: {}
      volumes:
        - name: containerdisk
          containerDisk:
            image: quay.io/containerdisks/fedora:40
        - name: cloudinitdisk
          cloudInitNoCloud:
            userData: |
              #cloud-config
              user: fedora
              password: changeme
              chpasswd:
                expire: false
                ssh_pwauth: true
<<EOF
```

```
packages:
- nginx
runcmd:
- systemctl enable --now nginx
EOF
```

VM 準備完了

```
oc wait vm web-vm -n metallb-demo --for=condition=Ready --timeout=300s
```

## 6: HTTP LoadBalancer Service

LoadBalancer Service VM へ HTTP トラフィックを MetalLB によって IP 宛てに

[service-lb-http.yaml](#) を適用

```
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: web-vm-lb
  namespace: metallb-demo
spec:
  type: LoadBalancer
  selector:
    app: web-vm
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
EOF
```

IP 宛てにトラフィックを送信

```
oc get svc web-vm-lb -n metallb-demo
```

MetalLB によって EXTERNAL-IP を取得

| NAME      | TYPE         | CLUSTER-IP | EXTERNAL-IP   | PORT(S)     | AGE |
|-----------|--------------|------------|---------------|-------------|-----|
| web-vm-lb | LoadBalancer | 172.30.x.x | 192.168.2.200 | 80:3xxx/TCP | ... |

IP 宛てにトラフィックを送信 VM によって

```
curl -s http://<EXTERNAL-IP>:80 | head -5
```

## 7: LoadBalancer SSH

SSH HTTP LoadBalancer Service MetalLB IP

`service-lb-ssh.yaml`

```
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: web-vm-ssh
  namespace: metallb-demo
spec:
  type: LoadBalancer
  selector:
    app: web-vm
  ports:
    - name: ssh
      protocol: TCP
      port: 22
      targetPort: 22
EOF
```

SSH IP

```
oc get svc web-vm-ssh -n metallb-demo
```

IP SSH

```
ssh fedora@<EXTERNAL-IP>
```

VM LoadBalancer Service

```
apiVersion: v1
kind: Service
metadata:
  name: multi-port-vm
  namespace: metallb-demo
spec:
  type: LoadBalancer
  selector:
```

```

app: web-vm
ports:
- name: http
  protocol: TCP
  port: 80
  targetPort: 80
- name: https
  protocol: TCP
  port: 443
  targetPort: 443
- name: ssh
  protocol: TCP
  port: 22
  targetPort: 22

```

LoadBalancer IP address pool3 is not available

## Configure BGP

BGP is a protocol that allows multiple MetalLB nodes to advertise the same IP address pool. BGP is used by LoadBalancer IP address pool (ECMP) to advertise the same L2 address pool.

BGP configuration

- BGP configuration
- BGP (AS configuration)
- **L2Advertisement** configuration **BGPPeer** and **BGPAdvertisement**

```

apiVersion: metallb.io/v1beta2
kind: BGPPeer
metadata:
  name: upstream-router
  namespace: metallb-system
spec:
  myASN: 64500
  peerASN: 64501
  peerAddress: 192.168.2.1

```

```

apiVersion: metallb.io/v1beta1
kind: BGPAdvertisement
metadata:
  name: bgp-advertisement
  namespace: metallb-system
spec:
  ipAddressPools:
  - vm-services-pool

```





## Gateway API: Kubernetes の Ingress

OpenShift 4.19 では、Kubernetes Gateway API を OpenShift Routes の Ingress として Gateway API を GatewayClass Gateway HTTPRoute として L7 として OpenShift Service Mesh 3 として Envoy として Ingress Operator として

Gateway API を VM を virt-launcher Pod を ClusterIP Service を HTTP/HTTPS/gRPC を VM を MetalLB を LoadBalancer を Gateway を Service を IP を

VM Web HTTPRoute

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-vm-route
  namespace: metallb-demo
spec:
  parentRefs:
  - name: my-gateway
    namespace: openshift-ingress
  hostnames:
  - "web-vm.apps.example.com"
  rules:
  - backendRefs:
    - name: web-vm-lb
      port: 80
```

OpenShift 3 ingress

- **OpenShift Routes** — HAProxy ingress edge passthrough re-encrypt TLS HTTP/HTTPS
- **Gateway API** — L7 namespace Kubernetes OCP 4.19
- **MetalLB LoadBalancer** — TCP/UDP L4 IP HTTP (SSH/RDP)

Route Gateway API L7 HTTP MetalLB L4 MetalLB Gateway API MetalLB IP Gateway L7

**Service Pending**

LoadBalancer Service IP Pending:

```
oc describe svc <service-name> -n <namespace>
```

□□□□□□:

- `IPAddressPool` `metallb-system` namespace
- `L2Advertisement` `metallb-system` namespace
- `MetalLB speaker Pod` `metallb-system` namespace (`oc get pods -n metallb-system`)

**IP**

- IP 000000000000 L2 0000000000000000
- 00000000 ARP 000000000000000000000000
- IP 000000000000 speaker Pod 00000000000000: `oc get pods -n metallb-system -o wide`

externalTrafficPolicy: Cluster [ ]

```

Service externalTrafficPolicy: Cluster
externalTrafficPolicy: Local

```

```
spec:
  externalTrafficPolicy: Local
```

IP

|  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|

namespace

```
oc delete namespace metallb-demo
```

[illegible]

```
oc delete metallb metallb -n metallb-system
```

```
oc delete subscription metallb-operator -n metallb-system
```

```
oc delete csv -n metallb-system -l operators.coreos.com/metallb-operator.metallb-system=
```

```
oc delete namespace metallb-system
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- MetalLB によって LoadBalancer Service によって
- L2 によって ARP によって IP によって
- BGP によって
- LoadBalancer Service によって IP の VM への HTTP 接続 (SSH/RDP) によって
- によって IP によって 1 個 Service によって
- Service の `selector` によって `spec.template.metadata.labels` によって VM Pod によって
- MetalLB によって `IPAddressPool` によって IP によって

11

- [VM Ingress 外部 Routes](#) — OpenShift Route 外部 VM HTTP 外部接続
- [MetalLB Operator 外部接続](#)
- [Kubernetes Gateway API 外部接続 \(Service Mesh 3.0\)](#)
- [KubeVirt MetalLB 外部](#)

## □□□□□□□ IPv4/IPv6 VM

11

OpenShift      Virtualization      (IPv4 + IPv6)  
 Pod      IPv4      IPv6      VM  
 User Defined Networks (UDN)

[illegible]

□ □ □ □

- OpenShift Virtualization Operator **OpenShift 4.18+**
- OVN-Kubernetes (OVN)
- **Network** — **clusterNetwork** **serviceNetwork** **IPv4** **IPv6** **CIDR** (OpenShift 4.18+ **Day 2**)
- **oc CLI**
- **Network**

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

IPv4 IPv6 CIDR

```
oc get network.config cluster -o jsonpath='{.status.clusterNetwork[*].cidr}'
```

```
oc get network.config cluster -o jsonpath='{.status.serviceNetwork[*]}'
```

□ □ □ □ □ □ □

```
10.128.0.0/14 fd01::/48
172.30.0.0/16 fd02::/112
```

IPv4 CIDR  Day-2   
 IPv6 CIDR

```
oc patch network.config.openshift.io cluster --type=merge --patch='
{
  "spec": {
    "clusterNetwork": [
      {"cidr": "10.128.0.0/14", "hostPrefix": 23},
      {"cidr": "fd01::/48", "hostPrefix": 64}
    ],
    "serviceNetwork": [
      "172.30.0.0/16",
      "fd02::/112"
    ]
  }
}'
```



## ○○○○○○OVN-Kubernetes

[illegible]

Operator

□□□□□□□□□□□□

```
oc get co network -o jsonpath='{.status.conditions[?(@.type=="Progressing")].status}'
```

False IPv6 CIDR

```
oc get network.config cluster -o jsonpath='{.status.serviceNetwork[*]}'
```

□□□□ IPv4 □□□□ IPv6 □□□□□□□□□□□□□□□□

```
oc get nodes -o jsonpath='{range .items[*]}{.metadata.name}:  
{.status.addresses[?(@.type=="InternalIP")].address}{"\n"}{end}'
```

## 例 1: Namespace

```
oc create namespace dualstack-demo
```

## 例 2: Pod 使用 virt-launcher 和 VM

virt-launcher Pod 使用 DHCP 和 IPv4 地址。IPv4 地址由 virt-launcher 的 `networkData` 属性指定。DHCPv6 (RA) 由 virt-launcher 的 `OS` 属性指定。DHCPv6 (solicitation) 由 virt-launcher 的 `cloud-init` 属性指定。IPv6 地址由 `KubeVirt user guide` 中的 `OpenShift Virtualization documentation` 指定。

virt-launcher Pod 使用 VM 的 IPv6 地址。IPv6 地址由 `fd10:0:2::/120` 指定。VM 的 IPv6 地址由 `fd10:0:2::/120` 指定。IPv6 地址由 `fd10:0:2::1` 指定。Network.pod.vmIPv6NetworkCIDR 由 `Network.pod.vmIPv6NetworkCIDR` 指定。

`vm-dualstack-masquerade.yaml` 文件内容

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: dualstack-vm
  namespace: dualstack-demo
spec:
  runStrategy: Always
 instancetype:
    name: u1.small
  preference:
    name: fedora
  template:
    metadata:
      labels:
        app: dualstack-vm
    spec:
      domain:
        devices:
          disks:
            - name: containerdisk
              disk: {}
            - name: cloudinitdisk
              disk: {}
          interfaces:
            - name: default
              masquerade: {}
              ports:
                - port: 80
      networks:
        - name: default
```

```

    pod: {}
  volumes:
  - name: containerdisk
    containerDisk:
      image: quay.io/containerdisks/fedora:40
  - name: cloudinitdisk
    cloudInitNoCloud:
      userData: |
        #cloud-config
        password: changeme
        chpasswd:
          expire: false
        packages:
          - nginx
        runcmd:
          - systemctl enable --now nginx
    networkData: |
      version: 2
      ethernets:
        eth0:
          dhcp4: true
          addresses:
            - fd10:0:2::2/120
          gateway6: fd10:0:2::1
EOF

```



netplan 0.103+ 는 gateway6 필드를 KubeVirt 이 cloud-init 가 지원하는 OS 가 지원하는 필드인 routes 필드 (Cloud-init networkData 필드)로

VM 가 준비되었는지

```
oc wait vm dualstack-vm -n dualstack-demo --for=condition=Ready --timeout=300s
```

단계 3: VM 가 준비되었는지

VM 가 IPv4 및 IPv6 가 지원하는지

```
virtctl console dualstack-vm -n dualstack-demo
```

로그인 (사용자: fedora 패스 root: changeme) 후

```
ip -4 addr show eth0
```

□ □ □ □ □ □ □

```
inet 10.0.2.2/24 brd 10.0.2.255 scope global dynamic eth0
```

```
ip -6 addr show eth0
```

□ □ □ □ □ □ □

```
inet6 fd10:0:2::2/120 scope global
```

ping ipv4

```
ping -c 3 10.0.2.1
```

ping ipv6

```
ping6 -c 3 fd10:0:2::1
```

Ctrl+] □□□□□□□□□□□□□□

□□□□ 4: □□□□□□□□□□□□□□□□

virt-launcher Pod 支持 IPv4 和 IPv6 的 Pod 的 IP 地址

```
oc get vmi dualstack-vm -n dualstack-demo -o
jsonpath='{.status.interfaces[0].ipAddresses}'
```

IPv4 → IPv6 (masquerade NAT) VM

## Figure 5: Service Time

IPv4 □ IPv6 □ □ □ □ □ □ □ □ IP □ □ □ □ □ □ Service □ □ □ □ VM □ □ □ □ □ □

## service-dualstack.yaml

```
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: dualstack-web
  namespace: dualstack-demo
spec:
  ipFamilyPolicy: RequireDualStack
```

```
ipFamilies:
  - IPv4
  - IPv6
selector:
  app: dualstack-vm
ports:
  - protocol: TCP
    port: 80
    targetPort: 80
EOF
```

Service `ClusterIP` `ClusterIP`

```
oc get svc dualstack-web -n dualstack-demo -o jsonpath='{.spec.clusterIPs}'
```

`ClusterIP`

```
["172.30.x.x", "fd02::x"]
```

`ClusterIP`

```
oc run test-v4 --rm -i --restart=Never -n dualstack-demo
--image=registry.access.redhat.com/ubi9/ubi-minimal -- curl -4 -s --connect-timeout 5
http://dualstack-web.dualstack-demo.svc:80
```

```
oc run test-v6 --rm -i --restart=Never -n dualstack-demo
--image=registry.access.redhat.com/ubi9/ubi-minimal -- curl -6 -s --connect-timeout 5
http://dualstack-web.dualstack-demo.svc:80
```

## User Defined Network `UserDefinedNetwork`

`UserDefinedNetwork` `IPv4` `IPv6` `Primary UDN` `UserDefinedNetwork`

- namespace `L2` `IP`
- `IP` `IPAM`
- `IPAM`

`UDN` `UDN`

`udn-dualstack.yaml` `UDN`

```
oc apply -f - <<EOF
apiVersion: k8s.ovn.org/v1
kind: UserDefinedNetwork
metadata:
```



```

name: dualstack-udn
namespace: dualstack-demo
spec:
  topology: Layer2
  layer2:
    role: Primary
    subnets:
      - "10.200.0.0/16"
      - "2001:db8:abcd::/64"
    ipam:
      lifecycle: Persistent
EOF

```



Primary UDN namespace `k8s.ovn.org/primary-user-defined-network: ""` ValidatingAdmissionPolicy namespace

```
oc delete namespace dualstack-demo
```

```

oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: dualstack-demo
  labels:
    k8s.ovn.org/primary-user-defined-network: ""
EOF

```

UDN VM

IPAM Primary UDN VM IPv4 IPv6

`vm-dualstack-udn.yaml`

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: dualstack-udn-vm
  namespace: dualstack-demo
spec:
  runStrategy: Always
 instancetype:
    name: u1.small
  preference:
    name: fedora
  template:

```

```

metadata:
  labels:
    app: dualstack-udn-vm
spec:
  domain:
  devices:
    disks:
      - name: containerdisk
        disk: {}
      - name: cloudinitdisk
        disk: {}
    interfaces:
      - name: primary-udn
        binding:
          name: l2bridge
  networks:
    - name: primary-udn
      pod: {}
  volumes:
    - name: containerdisk
      containerDisk:
        image: quay.io/containerdisks/fedora:40
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          password: changeme
          chpasswd:
            expire: false

```

EOF

IPAM **Persistent** VM **IPv4** **IPv6**

## Cloud-init networkData

**networkData** **netplan** 2 **KubeVirt** **OpenShift** **gateway6**

```

version: 2
ethernets:
  eth0:
    dhcp4: true
    addresses:
      - fd10:0:2::2/120
    gateway6: fd10:0:2::1

```

**gateway6** **netplan** 0.103 **cloud-init** 23.1 **OS** **routes**

```

version: 2
ethernets:
  eth0:
    dhcp4: true
    addresses:
      - fd10:0:2::2/120
    routes:
      - to: "::/0"
        via: "fd10:0:2::1"

```

gateway6 KubeVirt routes netplan



networkData network: version: 2



networkData OS ContainerDisk (quay.io/containerdisks/fedora) eth0 OpenShift Virtualization DataSource (enp1s0) VM ip link show altname cloud-init ip -6 addr show IPv6

## Masquerade IPv6

virt-launcher Pod DHCP (IPv4) DHCPv6 (IPv6) IPv4 DHCP Discover DHCPv6 virt-launcher (RA: Router Advertisement) RA OS DHCPv6 Solicit cloud-init dhcp6: true cloud-init networkData IPv6

fd10:0:2::2/120 fd10:0:2::1 VMI Network.pod.vmIPv6NetworkCIDR virt-launcher Pod VM Pod

IPAM UDN IPv4 IPv6 OVN-Kubernetes cloud-init networkData

OS IPv6

sysctl (net.ipv6.conf.all.disable\_ipv6=1) IPv6

```
sysctl net.ipv6.conf.all.disable_ipv6
```

1 IPv6

```
sudo sysctl -w net.ipv6.conf.all.disable_ipv6=0
```

```
sudo sysctl -w net.ipv6.conf.default.disable_ipv6=0
```

cloud-init の `userData`

```
write_files:
- path: /etc/sysctl.d/99-ipv6.conf
  content: |
    net.ipv6.conf.all.disable_ipv6=0
    net.ipv6.conf.default.disable_ipv6=0
runcmd:
- sysctl --system
```

## Masquerade と SLAAC と DHCPv6

`masquerade` は `virt-launcher` が DHCPv6 (RA) を OS に SLAAC (DHCPv6) を cloud-init の `networkData` に `dhcp6: true` を IPv6 (localnet/bridge) SLAAC DHCPv6

Linux IPv6 (RFC 4941) OS DHCPv6

```
version: 2
ethernets:
  eth0:
    ipv6-privacy: false
```

## IPv6 と DNS

DNS (CoreDNS) が A (IPv4) AAAA (IPv6) Pod VM

IPv6 DHCP IPv4 FQDN DNS

## VM の IPv6

- cloud-init の `networkData` に `version: 2` (network: )
- : `fd10:0:2::2/120`
- VM の cloud-init : `sudo cat /var/log/cloud-init.log | grep -i network`

## IPv6 Ping □□□□□ TCP □□□□□□

- IPv6 設定ファイル NetworkPolicy 設定
- VM 設定ファイル 0.0.0.0 設定 :: (設定ファイル) 設定

## IPv6 Service

- `spec.ipFamilyPolicy` `PreferDualStack` `RequireDualStack` `NotSet`
- `ClusterIP` `ClusterIP`: `oc get svc <name> -o jsonpath='{.spec.clusterIPs}'`
- `DNS` `AAAA` `dig AAAA <svc>.<ns>.svc.cluster.local`

□ □ □ □ □ □ □

```
oc delete namespace dualstack-demo
```

111

□□□□□□□□□□□□□□□□□□□□:

- `clusterNetwork` `serviceNetwork` `IPv4` `IPv6` CIDR  
Day 2 `networkData`
- `IPv4` `DHCP` `virt-launcher` `IPv6` `cloud-init` `networkData`
- `IPv6` `Network.pod.vmIPv6NetworkCIDR` `fd10:0:2::/120` `fd10:0:2::1`
- User Defined Networks `2` CIDR (`1`) `IPAM`
- Service `ipFamilyPolicy: RequireDualStack`
- `IPv6`

11

- [VM 网络 IP 网络](#) — 配置 cloud-init networkData 配置网络
- [Cloud-init 配置网络](#) — 配置 cloud-init 配置网络
- [UDN 配置网络](#) — VM 配置网络 2 配置网络
- [OpenShift Virtualization 配置网络 \(OCP 4.22\)](#)
- [KubeVirt 配置网络](#)

## VM Ingress Routes

OpenShift Virtualization VM OpenShift Routes Service  
Web VM ClusterIP Service TLS OpenShift Route

OpenShift Ingress VM Pod

11

- **OpenShift 4.18+** (OpenShift Virtualization Operator `openshift/virt`)
- `system:admin` `system:admin`
- **oc CLI** `virtctl CLI` `virtctl`
- **default StorageClass** `default`
- **Ingress Controller** `nginx` (OpenShift `nginx`)

□□□□□□□□□□:

OpenShift 4.20

## 1: namespace

```
oc create namespace ingress-demo
```

## 2: Web VM

# cloud-init 安装 Nginx Web 服务 Fedora VM 安装 Service 安装 VM 安装



```
##### cloud-init #####SSH #####
```

## vm-webserver.yaml

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: web-vm
  namespace: ingress-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: web-vm
```

```

    app: web-vm
spec:
  domain:
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
      interfaces:
        - name: default
          masquerade: {}
    resources:
      requests:
        memory: 1Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      containerDisk:
        image: quay.io/containerdisks/fedora:latest
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          password: redhat
          chpasswd:
            expire: false
          packages:
            - nginx
          write_files:
            - path: /usr/share/nginx/html/index.html
              content: |
                <html>
                <head><title>OpenShift Virt - Ingress Demo</title></head>
                <body>
                <h1>Hello from OpenShift Virtualization</h1>
                <p>This page is served by Nginx inside a VM, exposed via an OpenShift
Route.</p>
                </body>
                </html>
        runcmd:
          - systemctl enable nginx
          - systemctl start nginx
EOF

```

spec.template.metadata.labels    app: web-vm    virt-launcher    Pod    Service

VM の状態を確認する

VM の状態を確認する

```
oc get vm -n ingress-demo -w
```

VM が **Running** かつ **Ready=True** であることを確認する。Ctrl+C で終了する。

### ステップ 3: Web サービス

cloud-init を使って Nginx をインストールする (VM が Running かつ Ready=True であることを確認する)。

VM のコンソールに入る

```
virtctl console web-vm -n ingress-demo
```

fedora をインストールする。redhat をインストールする。

```
curl -s http://localhost
```

HTML ページを確認する。Ctrl+] で終了する。

### ステップ 4: Service

VM が 80 ポートで ClusterIP Service を提供するように Service を設定する。VM が Ready=True であることを確認する。

service-web-vm.yaml を作成する

```
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: web-vm-svc
  namespace: ingress-demo
spec:
  selector:
    app: web-vm
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
EOF
```

Service を確認する



```
oc get svc web-vm-svc -n ingress-demo
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc get endpoints web-vm-svc -n ingress-demo
```

□□□□□□:

| NAME       | TYPE      | CLUSTER-IP | EXTERNAL-IP | PORT(S) | AGE |
|------------|-----------|------------|-------------|---------|-----|
| web-vm-svc | ClusterIP | 172.30.x.x | <none>      | 80/TCP  | 5s  |

| NAME       | ENDPOINTS     | AGE |
|------------|---------------|-----|
| web-vm-svc | 10.128.x.x:80 | 5s  |

□□□□□□ IP □ VM □ Pod IP □□□□□□Service □ VM □□□□□□□□□□□□□□□□□□□□

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc run curl-test --rm -it --restart=Never
--image=registry.access.redhat.com/ubi9/ubi:latest -n ingress-demo -- curl -s
http://web-vm-svc.ingress-demo.svc.cluster.local
```

HTML □□□□□□□□□□□□□□□□

## □□□□ 5: OpenShift Route □□□□ Service □□

## HTTP Route (□□□□□□□□□□□□□□□□)

🔒 TLS 🔑 Route 🏠OpenShift 🛣️Route 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

## route-web-vm.yaml

```
oc apply -f - <<EOF
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: web-vm-route
  namespace: ingress-demo
spec:
  to:
    kind: Service
    name: web-vm-svc
    weight: 100
  port:
    targetPort: 80
  tls:
```

```

termination: edge
insecureEdgeTerminationPolicy: Redirect
wildcardPolicy: None
EOF

```

□ □ □ □ □ □ □ □

- **spec.to.name:** 容器 4 个 Service 容器
- **spec.port.targetPort:** Service 端口 (80) 容器端口
- **tls.termination: edge:** TLS 在 Router 容器 Router 的 VM 容器 HTTP 容器
- **insecureEdgeTerminationPolicy: Redirect:** HTTP 容器 HTTPS 容器

## Route 000

```
oc get route web-vm-route -n ingress-demo
```

□□□□□□:

| NAME         | HOST/PORT                                           | PATH     | SERVICES   |
|--------------|-----------------------------------------------------|----------|------------|
| PORT         | TERMINATION                                         | WILDCARD |            |
| web-vm-route | web-vm-route-ingress-demo.apps.ocplab01.example.com |          | web-vm-svc |
| 80           | edge/Redirect                                       | None     |            |

□ □ □ □ □ □ □ □ □ □

Route □□□□□□□□□□

```
ROUTE_HOST=$(oc get route web-vm-route -n ingress-demo -o jsonpath='{.spec.host}')
echo "Route URL: https://$ROUTE_HOST"
```

```
curl ██████████
```

```
curl -sk https://$ROUTE_HOST
```

## VM 上 Nginx 安装 HTML 静态资源



```
DNS   192.168.1.100 Route  192.168.1.100 Ingress IP  192.168.1.100
/etc/hosts 192.168.1.100 curl --resolve 192.168.1.100
```

## 図表 6: 図表 TLS (図表)

VM TLS Router VM

```
oc apply -f - <<EOF
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: web-vm-passthrough
  namespace: ingress-demo
spec:
  to:
    kind: Service
    name: web-vm-svc
    weight: 100
  port:
    targetPort: 443
  tls:
    termination: passthrough
    wildcardPolicy: None
EOF
```



OpenShift VM 通过 OpenShift Ingress Controller 的 Route 对象配置 HTTPS 流量。Nginx 通过 HTTP 流量通过 Route 对象配置 OpenShift Ingress Controller 的 VM 通过 TLS 流量通过 Nginx 流量。

□□□

OpenShift Route 通过 OpenShift VM 通过 OpenShift Ingress Controller

1. OpenShift Route 通过 OpenShift HTTPS 流量通过 OpenShift
2. OpenShift Ingress Controller (HAProxy Router) 通过 TLS 流量通过 (OpenShift Ingress Controller)
3. Router 通过 OpenShift ClusterIP Service 通过 80 流量通过
4. Service 通过 OpenShift virt-launcher Pod 通过 IP 通过 80 流量通过
5. virt-launcher Pod 通过 OpenShift masquerade 通过 OpenShift Ingress Controller VM 通过 Nginx 流量通过

OpenShift Ingress Controller 通过 OpenShift Ingress Controller

OpenShift Ingress Controller

**Route 503 Service Unavailable** □□□

Service 通过 OpenShift Ingress Controller 通过 OpenShift Ingress Controller

```
oc get endpoints web-vm-svc -n ingress-demo
```

OpenShift Service Selector 通过 VM 通过 OpenShift Ingress Controller

```
oc get pods -n ingress-demo -l app=web-vm
```

## Route 504 Gateway Timeout

VM 504 Gateway Timeout VM 504 Gateway Timeout

```
virtctl console web-vm -n ingress-demo
```

VM Nginx 504 Gateway Timeout

```
systemctl status nginx  
curl -s http://localhost
```

## DNS

Route 504 Gateway Timeout Ingress 504 Gateway Timeout DNS 504 Gateway Timeout

```
oc get ingresscontroller default -n openshift-ingress-operator -o  
jsonpath='{.status.domain}'
```

Route 504 Gateway Timeout <route-name>-<namespace>.<ingress-domain> 504 Gateway Timeout

504 Gateway Timeout

504 Gateway Timeout

```
oc delete namespace ingress-demo
```

504

504 Gateway Timeout

- cloud-init 504 Web 504 Gateway Timeout VM 504 Gateway Timeout
- Kubernetes Service 504 Gateway Timeout VM 504 Gateway Timeout
- 504 HTTPS 504 Gateway Timeout TLS 504 Gateway Timeout OpenShift Route 504 Gateway Timeout
- 504 Gateway Timeout VM 504 Gateway Timeout TLS 504 Gateway Timeout
- VM 504 Gateway Timeout Service 504 Gateway Timeout Route 504 Gateway Timeout

504

- [Exposing a VM by Using a Service - OpenShift 504 Gateway Timeout](#)
- [Route Configuration - OpenShift 504 Gateway Timeout](#)

# Localnet 配置

配置:

OpenShift 4.20

配置OpenShift Virtualization 配置Localnet 配置OVS (Open vSwitch) bridge

## OVS bridge 与 Linux bridge 对比

对比: OVS bridge 与 Linux bridge 对比

- OVS bridge 与 `ovs-vsctl` 对比
- OVS bridge 与 `ip addr show` 与 `ip link show` 对比
- 配置OVS bridge 与 Linux 对比

配置

## 配置 1: NodeNetworkConfigurationPolicy 配置

配置NNCP 配置

```
# 配置NNCP配置
oc get nncp

# 配置NNCP配置
oc get nncp br-vlan-creation
```

配置:

| NAME             | STATUS    | REASON                 |
|------------------|-----------|------------------------|
| br-vlan-creation | Available | SuccessfullyConfigured |

## 配置 2: NNCP 配置

NNCP 配置

```
oc get nncp br-vlan-creation -o yaml
```

配置:

```
status:
conditions:
```

```
- lastHeartbeatTime: "2025-10-06T20:38:57Z"
  lastTransitionTime: "2025-10-06T20:38:57Z"
  message: 1/1 nodes successfully configured
  reason: SuccessfullyConfigured
  status: "True"
  type: Available
- reason: SuccessfullyConfigured
  status: "False"
  type: Degraded
```

- **Available: True** == **SuccessfullyConfigured** = 正常
- **Degraded: False** = 不正常
- **message: X/X nodes successfully configured** = 成功配置了X/X个节点

### 实验 3: OVS bridge

实验 OVS bridge 配置:

```
# 查看节点
oc get nodes

# 查看OVS bridge
oc debug node/<node-name> -- chroot /host ovs-vsctl list-br
```

输出:

```
Starting pod/ocplab01-debug-k7bfc ...
To use host binaries, run `chroot /host`
br-ex
br-int
br-vlan

Removing debug pod ...
```

可以看到 **br-vlan** 是 OVS bridge 之一

### 实验 4: bridge

OVS bridge 配置:

```
oc debug node/<node-name> -- chroot /host ovs-vsctl show
```

输出 (示例):

```
Bridge br-vlan
```

```
Port enp11s0f0np0
Interface enp11s0f0np0
type: system
```

□□□□:

- bridge □ (**br-vlan**) □□□□□□□□
- □□□□□□□□ (□: **enp11s0f0np0**) □□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□ **system** □□□□□

□□□□ 5: □□□□□□□□□□□□□□□□

□□□□□□□□□□ **bridge** □□□□□□□□□□**UP** □□□□□□□□□□□□

```
oc debug node/<node-name> -- chroot /host ip link show <interface-name>
```

□□□:

```
4: enp11s0f0np0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq master ovs-system
state UP mode DEFAULT group default qlen 1000
    link/ether e8:eb:d3:13:06:16 brd ff:ff:ff:ff:ff:ff
```

□□□□:

- **UP,LOWER\_UP** - □□□□□□□□□□□□□□□□
- **master ovs-system** - □□□□□□□□ **OVS** □□□□□□□□□□□□
- **state UP** - □□□□□□□□□□□□□□□□□□

□□□□ 6: "**ip addr show br-vlan**" □□□□□□□□

□□□□□□□□□□:

```
oc debug node/<node-name> -- chroot /host ip addr show br-vlan
```

□□□□□□□□:

```
Device "br-vlan" does not exist.
```

□□:

- **OVS bridge** □□□□ **Linux** □□□□□□□□□□□□□□□□**OVS** □□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□ **ip** □□□□□□□□□□□□□□
- □□□□ **bridge** □□□□□□□□□□□□□□□□□□

- OVS bridge `ovs-vsctl` `bridge` `list-br`

1. `ovs-vsctl list-br`

1: `ovs-vsctl list-br` `bridge`

2:

- NNCP `Available: True` `bridge` `ovs-vsctl list-br`

3:

```
# NNCP
oc get nncp br-vlan-creation -o
jsonpath='{.status.conditions[?(@.type=="Available")]}'|jq

# NMState Operator
oc logs -n openshift-nmstate -l app=kubernetes-nmstate-operator

#
oc get nns -A
```

4:

- `ip link show`
- OVS `ovs-vsctl`
- NMState Operator

2: `bridge`

3:

- `bridge` `ovs-vsctl show`

4:

```
# bridge
oc debug node/<node-name> -- chroot /host ovs-vsctl list-ports br-vlan

#
oc debug node/<node-name> -- chroot /host ip link show <interface-name>

# NNCP
oc get nncp br-vlan-creation -o yaml | grep -A 5 "name: <interface-name>"
```

5:

1. NNCP



2. `bridge` `bridge` `bridge`
3. `bridge` `bridge` `bridge`

### 3: NodeSelector

1:

- NNCP `bridge` `bridge`

2:

```
# 1
oc get nodes --show-labels

# NNCP NodeSelector
oc get nncp br-vlan-creation -o jsonpath='{.spec.nodeSelector}'
```

3:

- Single Node OpenShift (SNO) `control-plane, master, worker`
- `node-role.kubernetes.io/worker: ""` NNCP
- NNCP `"X/X nodes successfully configured"`

### 4: bridge

1:

- `bridge` `bridge` `bridge`

2:

```
# 1
oc debug node/<node-name> -- chroot /host cat /sys/class/net/<interface-name>/carrier

# 2
oc debug node/<node-name> -- chroot /host ethtool -S <interface-name>

# 3
oc debug node/<node-name> -- chroot /host ethtool <interface-name> | grep "Link detected"
```

3:

- `bridge` `bridge` `bridge`
- `bridge` `bridge` `bridge`
- `bridge` `bridge` `bridge`

## Localnet の bridge mapping

OVS bridge を利用して OVN-Kubernetes の bridge mapping を設定します。

### bridge mapping

bridge mapping (NetworkAttachmentDefinitions) を OVS bridge に設定します。

- 名前: `localnet-vlan`
- bridge: `br-vlan`
- mapping: `localnet-vlan:br-vlan`

#### ステップ 1: bridge mapping の NNCP

bridge mapping の NNCP を設定します。

```
# NNCP を取得
oc get nncp | grep mapping

# NNCP の bridge mapping を取得
oc get nncp localnet-vlan-bridge-mapping
```

結果:

```
localnet-vlan-bridge-mapping  Available  SuccessfullyConfigured
```

#### ステップ 2: bridge mapping の YAML

```
oc get nncp localnet-vlan-bridge-mapping -o yaml
```

結果:

```
spec:
  desiredState:
    ovn:
      bridge-mappings:
      - bridge: br-vlan
        localnet: localnet-vlan
        state: present
  nodeSelector:
    node-role.kubernetes.io/worker: ""
status:
  conditions:
  - message: 1/1 nodes successfully configured
    reason: SuccessfullyConfigured
    status: "True"
```

type: Available

### ステップ 3: OVN bridge mapping 設定

OVN-Kubernetes の bridge mapping を設定します:

```
oc debug node/<node-name> -- chroot /host ovs-vsctl get Open_vSwitch .
external_ids:ovn-bridge-mappings
```

設定値:

```
"localnet-vlan:br-vlan,physnet:br-ex"
```

例:

- **localnet-vlan:br-vlan** = OVN bridge mapping
- **physnet:br-ex** = OpenShift mapping
- OVN bridge mapping の詳細は [こちら](#) を参照してください

### ステップ 4: OVS External ID 設定

OVS external ID を設定します:

```
oc debug node/<node-name> -- chroot /host ovs-vsctl list Open_vSwitch | grep -A 2
external_ids
```

例:

```
external_ids      : {hostname=ocplab01, ovn-bridge-mappings="localnet-vlan:br-
vlan,physnet:br-ex", ovn-bridge-remote-probe-interval="0", ovn-enable-lflow-
cache="true", ovn-encap-ip="192.168.2.241", ovn-encap-type=geneve, ovn-is-
interconn="true", ovn-memlimit-lflow-cache-kb="1048576", ovn-monitor-all="true", ovn-
ofctrl-wait-before-clear="0", ovn-remote="unix:/var/run/ovn/ovnsb_db.sock", ovn-
remote-probe-interval="180000", ovn-set-local-ip="true",
rundir="/var/run/openvswitch", system-id="1b41fc3f-f597-4cd9-b508-e8247c209d97"}
```

例:

- **ovn-bridge-mappings** の mapping の詳細は [こちら](#) を参照してください
- **hostname** の詳細は [こちら](#) を参照してください
- **ovn-encap-type=geneve** の詳細は [こちら](#) を参照してください



```
Physical Interface: enp11s0f0np0
- VLAN
```

## bridge mapping

### 1: bridge mapping OVN External ID

:

- NNCP `ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings mapping`

:

```
# NNCP
oc get nncp localnet-vlan-bridge-mapping -o yaml

# NMState Operator
oc delete pod -n openshift-nmstate -l app=kubernetes-nmstate-operator

# NMState
oc logs -n openshift-nmstate -l component=kubernetes-nmstate-handler
```

:

- bridge NNCP
- NMState
- OVS

:

bridge bridge mapping NNCP

### 2: mapping bridge

:

- bridge mapping bridge

:

```
# bridge
oc debug node/<node-name> -- chroot /host ovs-vsctl list-br

# mapping
oc get nncp localnet-vlan-bridge-mapping -o jsonpath='{.spec.desiredState.ovn.bridge-mappings[0].bridge}'
```

##:

mappingNCP bridge OVS bridge

## 3:

##:

- bridge mapping VM

##:

```
# bridge mapping
oc get nncp localnet-vlan-bridge-mapping -o jsonpath='{.spec.desiredState.ovn.bridge-mappings[0].localnet}'

# NetworkAttachmentDefinition ClusterUserDefinedNetwork (CUDN)
oc get network-attachment-definitions <nad-name> -n <namespace> -o yaml
```

##:

bridge mapping NetworkAttachmentDefinition ClusterUserDefinedNetwork

## 4: mapping

##:

- bridge mapping bridge

##:

```
# bridge mapping
oc debug node/<node-name> -- chroot /host ovs-vsctl get Open_vSwitch .
external_ids:ovn-bridge-mappings
```

##:

```
"mapping1:bridge1,mapping2:bridge2,mapping3:bridge3"
```



```
oc debug node/<node-name> -- chroot /host ovs-vsctl list interface
```

**OVS** `ovs-vsctl`

```
oc debug node/<node-name> -- chroot /host ovs-vsctl list bridge
```

**OVS bridge** `ovs-ofctl` (브릿지)

```
# 브릿지 포트 목록
oc debug node/<node-name> -- chroot /host ovs-ofctl dump-ports br-vlan
```

## ClusterUserDefinedNetwork (CUDN) `oc`

NetworkAttachmentDefinition `ClusterUserDefinedNetwork` `NetworkAttachmentDefinition` 생성

**예제 1: CUDN 생성**

`ClusterUserDefinedNetwork` 생성:

```
# CUDN 생성
oc get clusteruserdefinednetwork

# CUDN 정보 출력
oc get clusteruserdefinednetwork <cudn-name> -o yaml
```

출력:

| NAME                   | AGE |
|------------------------|-----|
| cudn-localnet-vlan-100 | 5m  |

**예제 2: CUDN 정보**

CUDN spec 정보:

```
oc get clusteruserdefinednetwork cudn-localnet-vlan-100 -o yaml
```

출력:

```
spec:
  namespaceSelector:
    matchExpressions:
      - key: kubernetes.io/metadata.name
        operator: In
```



```

    values: ["vm-guests-vlan-prod", "vm-guests-vlan-dev"]
network:
  topology: Localnet
  localnet:
    role: Secondary
    physicalNetworkName: localnet-vlan # bridge mapping
    vlanID: 100
    ipam:
      mode: Disabled # Static

```

### Figure 3: NetworkAttachmentDefinition

CUDN namespace NetworkAttachmentDefinition:

```

# namespaceNAD
oc get network-attachment-definitions -A | grep <cudn-name>

# namespace
oc get network-attachment-definitions -n <namespace>

```

Table:

| NAMESPACE           | NAME                   | AGE |
|---------------------|------------------------|-----|
| vm-guests-vlan-dev  | cudn-localnet-vlan-100 | 5m  |
| vm-guests-vlan-prod | cudn-localnet-vlan-100 | 5m  |

### Figure 4: physicalNetworkName and bridge mapping

CUDN physicalNetworkName and bridge mapping:

```

# CUDN physicalNetworkName
oc get clusteruserdefinednetwork <cudn-name> -o
jsonpath='{.spec.network.localnet.physicalNetworkName}'

# bridge mapping
oc get nncp <bridge-mapping-nncp> -o jsonpath='{.spec.desiredState.ovn.bridge-
mappings[0].localnet}'

# OVN
oc debug node/<node-name> -- chroot /host ovs-vsctl get Open_vSwitch .
external_ids:ovn-bridge-mappings

```

Table (localnet-vlan)

### Figure 5: namespace

namespace VM CUDN:

```
# namespace VM
oc get vmi -n vm-guests-vlan-prod -o
jsonpath='{.items[*].status.interfaces[?(@.name=="cudn_vlan_network")].ipAddress}'

# 2 namespace VM
oc get vmi -n vm-guests-vlan-dev -o
jsonpath='{.items[*].status.interfaces[?(@.name=="cudn_vlan_network")].ipAddress}'
```

VM の CUDN の IP 設定

## CUDN の設定

### 1. namespace の NAD の設定:

- 例: CUDN の namespace `oc get network-attachment-definitions -n <namespace>`
- 例: namespace の namespace の設定
- 例: namespace の CUDN の設定

```
# namespace
oc get namespace <namespace-name> -o jsonpath='{.metadata.labels}'

# CUDN
oc get clusteruserdefinednetwork <cudn-name> -o
jsonpath='{.spec.namespaceSelector}'
```

### 2. VM の "failed bridge mapping validation" の設定:

- 例: CUDN の VM Pod の設定
- 例: CUDN の `physicalNetworkName` の bridge mapping の設定
- 例: (例) の設定

```
#
oc get clusteruserdefinednetwork <cudn-name> -o
jsonpath='{.spec.network.localnet.physicalNetworkName}'
oc get nncp <bridge-mapping-nncp> -o jsonpath='{.spec.desiredState.ovn.bridge-
mappings[0].localnet}'
```

### 3. IPAM の設定:

- 例: VM の IP の設定
- 例: IPAM の cloud-init の設定
- **cloud-init** の IP: CUDN の `ipam.mode: Disabled`
- **OVN IPAM** の `staticIPAMConfig` の `ipam.mode: Static`
- 例:

```
# IPAM操作
oc get clusteruserdefinednetwork <cudn-name> -o
jsonpath='{.spec.network.localnet.ipam.mode}'

# VM cloud-init操作
virtctl console -n <namespace> <vm-name>
# VM: cat /var/log/cloud-init.log
```

## Localnet VLAN 操作

操作

### 1. VLAN 操作 VM 操作:

- 操作 VLAN 操作
- Worker Node 的 bridge mapping 操作
- VLAN ID 操作
- 的 NIC 操作 UP 操作
- OVS bridge 操作

### 2. IP 操作:

- DHCP 操作 VLAN 操作

### 3. 操作:

- 操作 VLAN 操作
- 操作 VLAN 操作
- 操作

### 4. 操作 bridge 操作:

- 的: OVS bridge 的 `ip addr show` 操作 `ovs-vsctl list-br` 操作
- 操作
- NMState Operator 操作

### 5. bridge mapping 操作:

- mapping 的 bridge 操作 bridge 操作
- OVN bridge mapping 的: `ovs-vsctl get Open_vSwitch . external_ids:ovn-bridge-mappings`
- mapping 的 localnet 的 NAD 操作

### 6. VM Pod 的 "failed bridge mapping validation" 操作:

- 的: VM Pod 操作: `failed to find OVN bridge-mapping for network: "localnet-vlan-100"`
- 的: NetworkAttachmentDefinition 的 `physicalNetworkName` 操作
- 的: NAD 的 bridge mapping 操作 `"physicalNetworkName": "localnet-vlan"` 操作
- 的: NAD 操作 `oc delete` 的 `oc apply` 操作

□ □ □ □ □ □

```
# NetworkAttachmentDefinition
oc get network-attachment-definitions -n vm-guests-vlan

# NodeNetworkConfigurationPolicies
oc get nncp

# bridge (bridge)
oc get nncp br-vlan-creation -o yaml

# Worker Node bridge mapping
oc get nncp localnet-vlan-bridge-mapping -o yaml

# Worker Node
oc debug node/<worker-node-name> -- chroot /host ip link show ens224

# OVS bridge
oc debug node/<worker-node-name> -- chroot /host ovs-vsctl show

# OVNbridge mapping
oc debug node/<worker-node-name> -- chroot /host ovs-vsctl get Open_vSwitch .
external_ids:ovn-bridge-mappings

# bridge
oc debug node/<worker-node-name> -- chroot /host ovs-vsctl list-ports br-vlan

# VM
oc get vmi -n vm-guests-vlan -o yaml | grep -A 10 networks

# OVN-Kubernetes
oc logs -n openshift-ovn-kubernetes -l app=ovnkube-node

# bridge NMState Operator
oc logs -n openshift-nmstate -l app=kubernetes-nmstate-operator
```

111

[illegible]

- OVS bridge → Linux bridge → ip addr show
- ovs-vsctl → OVS bridge → bridge
- OVS bridge → OVN bridge mapping
- bridge
- ClusterUserDefinedNetwork (CUDN) → NAD namespace

□□

- [NMState Documentation](#)
- [Open vSwitch Documentation](#)
- [OpenShift NMState Operator](#)

# OpenShift Virtualization

## Overview

OpenShift Virtualization is a Kubernetes extension that enables running VMs on OpenShift. It provides a way to manage VMs as Kubernetes Pods, allowing you to leverage the OpenShift ecosystem for VM management. DNS is used for network resolution within the VMs.

## Architecture

OpenShift Virtualization architecture involves the integration of OpenShift with Virtualization technology to manage VMs as Pods.

- VMs are managed as Pods in OpenShift, allowing for DNS resolution.
- Linux VMs can be managed alongside Windows VMs.
- Selector, Affinity, and Anti-affinity rules can be applied to VMs.
- OpenShift Virtualization supports various VM architectures.
- OpenShift Virtualization can be used to manage VMs in a multi-tenant environment.
- OpenShift Virtualization can be used to manage VMs in a hybrid cloud environment.

## Getting Started

VMs can be managed using OpenShift Virtualization Operator.

- OpenShift 4.18 includes OpenShift Virtualization Operator.
- OpenShift Virtualization Operator can be installed on OpenShift.
- YAML files are used to define VMs.
- CLI tools like `oc`, `virtctl`, and `kubectl` are used to manage VMs.

## Network

### VM DNS

OpenShift Virtualization uses Kubernetes to manage VMs, and DNS is used for network resolution within the VMs.

### Linux VMs

Linux VMs can be managed using `virtctl` or `SSH` through `NodePort`. OpenShift Virtualization supports various Linux distributions.

### Linux VMs

OpenShift Virtualization supports various Linux distributions and VM architectures.

### RDP Windows VMs

LoadBalancer can be used to manage Windows VMs, and RDP can be used to connect to them.

## Infrastructure

HyperConverged OpenShift Virtualization  
(virt-controller, virt-api, virt-handler, CDI)

### Affinity

NodeSelector NodeAffinity Pod Affinity/Anti-Affinity rule  
VM

### 

`virtctl` VM

### 

OpenShift Virtualization VM

### 

- 
- VM
- 
- CPU NUMA Hugepages VM
- VM

### 

- OpenShift Virtualization
- cloud-init
- KubeVirt

## VM DNS

### 

OpenShift Virtualization DNS  
Kubernetes Service VM DNS IP Pod  
VM VM VM

### 

- Kubernetes Service VM
- VM DNS
- OpenShift Virtualization VM DNS

□ □ □ □

- OpenShift Virtualization Operator [OpenShift 4.18](#)
- OpenShift CLI `oc` `virtctl`
- Kubernetes Service `DNS`

□□□ **1:** □□□□□□□□□□

## 1: namespace

namespace

```
oc new-project myapp
```

## □□□□ 2: Fedora VM □□□□

## Fedora VM 環境構築



```

##### cloud-init #####
##### SSH #####

```

fedora-app-vm.yaml 0000000

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-app-vm
  namespace: myapp
  labels:
    app: myapp
spec:
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: fedora-app-vm-volume
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-app-vm
```



```

    app: myapp
spec:
  domain:
    devices:
      disks:
        - disk:
            bus: virtio
            name: rootdisk
        - disk:
            bus: virtio
            name: cloudinitdisk
      interfaces:
        - name: default
          masquerade: {}
    resources:
      requests:
        memory: 2Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: fedora-app-vm-volume
        name: rootdisk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora
          chpasswd: { expire: false }
          ssh_pwauth: true
        name: cloudinitdisk

```

EOF

VM 创建成功

```
watch 'oc get vm,vmi,dv -n myapp'
```

VM 运行 Ready: True 按 Ctrl+C 退出

创建 3: VM 服务

VM 无头服务 Headless Service

headless-service.yaml 内容

```

oc apply -f - <<EOF
apiVersion: v1
kind: Service

```

```

metadata:
  name: myapp-service
  namespace: myapp
spec:
  clusterIP: None
  selector:
    kubevirt.io/domain: fedora-app-vm
  ports:
    - name: ssh
      protocol: TCP
      port: 22
      targetPort: 22
EOF

```

Service → EndpointSlice

```

oc get svc myapp-service -n myapp
oc get endpointslice -n myapp -l kubernetes.io/service-name=myapp-service

```

결과:

Service → **TYPE: ClusterIP** → **CLUSTER-IP: None** (headless service)

EndpointSlice → ENDPOINTS → VM → Pod IP

4: **DNS**

DNS → Pod

```

oc apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: debug-dns
  namespace: myapp
spec:
  containers:
    - name: debug
      image: registry.access.redhat.com/ubi9/ubi
      command: ["sleep", "3600"]
      securityContext:
        allowPrivilegeEscalation: false
        runAsNonRoot: true
        runAsUser: 1000
      capabilities:
        drop:
          - ALL
      seccompProfile:
        type: RuntimeDefault

```

```

restartPolicy: Never
EOF

# Pod
oc wait --for=condition=Ready pod/debug-dns -n myapp --timeout=60s

# DNS
oc exec -n myapp debug-dns -- getent hosts myapp-service.myapp.svc.cluster.local

# 
oc delete pod debug-dns -n myapp

```

：

- DNS VM Pod IP (10.128.0.169 myapp-service.myapp.svc.cluster.local)

Service DNS VM IP

2:

**DNS**

Pod VM Service DNS





- **Service** DNS : Service `metadata.name` `<service-name>.<namespace>.svc.cluster.local` CoreDNS
- **Headless Service** VM IP : Headless Service (`clusterIP: None`) VM Pod IP VM
- **Service** : ClusterIP Service IP VM
- : `/etc/resolv.conf` `search` namespace Service (FQDN) `myapp-service`

### 3:

**Service** **EndpointSlice** :

```
# Service
oc get svc myapp-service -n myapp

# EndpointSlice (VM Pod IP)
oc get endpointslice -n myapp -l kubernetes.io/service-name=myapp-service

# Service
oc describe svc myapp-service -n myapp
```

**DNS** :

```
# Pod ( )
oc apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: debug-dns
  namespace: myapp
spec:
  containers:
  - name: debug
    image: registry.access.redhat.com/ubi9/ubi
    command: ["sleep", "3600"]
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      runAsUser: 1000
    capabilities:
      drop:
      - ALL
    seccompProfile:
      type: RuntimeDefault
  restartPolicy: Never
<>>EOF
```

EOF

```
# Pod
oc wait --for=condition=Ready pod/debug-dns -n myapp --timeout=60s

# DNS
oc exec -n myapp debug-dns -- getent hosts myapp-service.myapp.svc.cluster.local

# 
oc delete pod debug-dns -n myapp
```

**VM :**

```
# VMService
oc get vm fedora-app-vm -n myapp -o jsonpath='{.metadata.labels}'
oc get svc myapp-service -n myapp -o jsonpath='{.spec.selector}'
```

**DNS**

:

```
oc get svc myapp-service -n myapp
oc get endpointslice -n myapp -l kubernetes.io/service-name=myapp-service
```

- Service VM
- VM (EndpointSlice)
- namespace

:

1. Service VM `kubevirt.io/domain`
2. VM : `oc get vmi -n myapp`
3. Service VM namespace

**namespace DNS**

:

```
# namespacePod
oc new-project test-namespace 2>/dev/null || true

oc apply -f - <<EOF
```

```

apiVersion: v1
kind: Pod
metadata:
  name: debug-cross-ns
  namespace: test-namespace
spec:
  containers:
  - name: debug
    image: registry.access.redhat.com/ubi9/ubi
    command: ["sleep", "300"]
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      runAsUser: 1000
      capabilities:
        drop:
        - ALL
      seccompProfile:
        type: RuntimeDefault
    restartPolicy: Never
EOF

# 確認
oc wait --for=condition=Ready pod/debug-cross-ns -n test-namespace --timeout=60s
oc exec -n test-namespace debug-cross-ns -- \
  getent hosts myapp-service.myapp.svc.cluster.local

# 削除
oc delete pod debug-cross-ns -n test-namespace

```

確認:

- FQDN 確認
- NetworkPolicy 確認 namespace 確認

手順:

1. namespace 確認 FQDN 確認: `<service>.<namespace>.svc.cluster.local`
2. NetworkPolicy 確認

確認:

- [Kubernetes DNS for Services and Pods](#)
- [FQDN 確認 VM 確認 - OpenShift Virtualization](#)
- [FQDN 確認 VM 確認 - OpenShift Virtualization](#)

確認

確認

- DNS `Headless Kubernetes Service` `VM` `Pod`
- `CoreDNS` `Service` `VM` `Pod IP`
- `Headless Service` (`VM IP`) `ClusterIP Service` (`IP`)
- `FQDN` `getent hosts` `Pod` `DNS`
- `DNS` `Service`

- `IP`
- `Linux VM`

# Linux

`OpenShift` `Virtualization` `Linux`

|                  |     | SSH |
|------------------|-----|-----|
|                  |     |     |
| VNC              | GUI |     |
| virtctl SSH      |     |     |
| NodePort Service |     |     |
|                  |     |     |
|                  |     |     |

- `OpenShift Virtualization Operator` `OpenShift 4.18`
- `CLI`: `oc``virtctl``kubectl`
- `Linux VM` `1` ()
- `SSH`

`VM`

`namespace` `Fedora VM`

```
oc create namespace vm-access-demo
```





cloud-init  
安装 cloud-init

安装 SSH 并配置 SSH 连接

### fedora-access-vm.yaml 示例

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-access-vm
  namespace: vm-access-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: fedora-access-vm-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-access-vm
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
      machine:
        type: pc-q35-rhel9.4.0
```

```

memory:
  guest: 4Gi
networks:
  - name: default
    pod: {}
volumes:
  - dataVolume:
      name: fedora-access-vm-disk
      name: rootdisk
  - cloudInitNoCloud:
      userData: |-
        #cloud-config
        user: fedora
        password: fedora123
        chpasswd: { expire: false }
        ssh_pwauth: true
        ssh_authorized_keys:
          - ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ... your-public-key-here
      name: cloudinitdisk

```

Note: **your-public-key-here** is your SSH public key. You can find it in the `~/.ssh/` directory.

VM is created.

```
oc apply -f fedora-access-vm.yaml
```

VM is running.

```
oc get vmi -n vm-access-demo
```

**Step 1: virtctl** is used to interact with the VM.

The `virtctl` command is used to interact with the VM.

The `virtctl` command is used to interact with the VM.

```
virtctl console fedora-access-vm -n vm-access-demo
```

The `virtctl` command is used to interact with the VM.

**Ctrl+] is used to exit the VM console.**

The `virtctl` command is used to interact with the VM.

- VM is created.
- VM is running.

- VM 的 SSH 配置
- 配置虚拟机网络

配置

- 网络/配置 (网络)
- 配置
- 配置
- 配置 1 配置

## 第 2: VNC 配置

配置虚拟机 VNC 配置

VNC 配置

```
virtctl vnc fedora-access-vm -n vm-access-demo
```

配置 VM 配置 VNC 配置

配置

- 配置 Linux VM 配置
- GUI 配置
- GUI 配置
- 配置

配置

- 配置 VNC 配置
- 配置 (GNOME/KDE 配置) 配置 VM

## 第 3: virtctl 配置 SSH 配置

virtctl 配置 SSH 配置 VM 的 SSH 配置

配置

- VM 的 SSH 配置 (cloud-init 配置)
- VM 的 SSH 配置

SSH 配置

```
virtctl ssh fedora@fedora-access-vm -n vm-access-demo
```

ssh 접속 (ssh)

```
virtctl ssh fedora@fedora-access-vm -n vm-access-demo --local-ssh-opts="-o
StrictHostKeyChecking=no"
```

ssh 접속

- ssh 접속
- ssh 접속
- ssh 접속
- **oc** **kubectl** ssh 접속

ssh

- ssh 접속
- ssh 접속

## 4: NodePort Service SSH

NodePort Service SSH 접속

**NodePort Service**

**ssh-nodeport-service.yaml**

```
apiVersion: v1
kind: Service
metadata:
  name: fedora-ssh-nodeport
  namespace: vm-access-demo
spec:
  type: NodePort
  selector:
    kubevirt.io/domain: fedora-access-vm
  ports:
    - protocol: TCP
      port: 22
      targetPort: 22
      nodePort: 30022
```

Service

```
oc apply -f fedora-ssh-nodeport.yaml
```

## Service 操作

```
oc get svc fedora-ssh-nodeport -n vm-access-demo
```

## SSH 操作

获取节点的 IP 地址

```
oc get nodes -o wide
```

## NodePort 操作

```
ssh fedora@<node-ip> -p 30022
```

操作

- 操作
- 操作
- 操作/操作
- 操作

操作

- 操作 SSH 操作
- 操作 (SSH 操作)
- 操作
- 操作
- 操作

## 第 5 步: 操作 SSH 操作

kubectl 操作 oc port-forward 操作 SSH 操作

## VM Pod 操作

```
oc get pods -n vm-access-demo
```

操作:

| NAME                                 | READY | STATUS  | RESTARTS | AGE |
|--------------------------------------|-------|---------|----------|-----|
| virt-launcher-fedora-access-vm-xxxxx | 2/2   | Running | 0        | 10m |

## SSH 接続

```
oc port-forward -n vm-access-demo virt-launcher-fedora-access-vm-xxxxx 2222:22
```

## SSH 接続 (ポートフォワード)

```
ssh fedora@localhost -p 2222
```

## 接続方法

- 接続方法
- 接続方法
- VM 接続方法
- NodePort 接続方法

## 接続

- 接続方法
- 1 接続方法
- 接続方法
- 接続方法

## 図 6: 接続方法

bridge 接続 localnet 接続 VM 接続 IP 接続

## 接続

- VM 接続 (bridge localnet VLAN) 接続
- 接続
- IP 接続 DHCP 接続

## 接続 VM

## fedora-vlan-access-vm.yaml 接続

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-vlan-vm
  namespace: vm-access-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
```

```

kind: DataVolume
metadata:
  name: fedora-vlan-vm-disk
spec:
  sourceRef:
    kind: DataSource
    name: fedora
    namespace: openshift-virtualization-os-images
  storage:
    resources:
      requests:
        storage: 30Gi
runStrategy: RerunOnFailure
template:
  metadata:
    labels:
      kubevirt.io/domain: fedora-vlan-vm
  spec:
    domain:
      cpu:
        cores: 2
      devices:
        disks:
          - disk:
              bus: virtio
              name: rootdisk
          - disk:
              bus: virtio
              name: cloudinitdisk
        interfaces:
          - masquerade: {}
            name: default
          - bridge: {}
            name: vlan-network
        rng: {}
      machine:
        type: pc-q35-rhel9.4.0
      memory:
        guest: 4Gi
    networks:
      - name: default
        pod: {}
      - name: vlan-network
        multus:
          networkName: localnet-vlan-100
    volumes:
      - dataVolume:
          name: fedora-vlan-vm-disk
          name: rootdisk
      - cloudInitNoCloud:
          userData: |-

```

```
#cloud-config
user: fedora
password: fedora123
chpasswd: { expire: false }
ssh_pwauth: true
ssh_authorized_keys:
  - ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ... your-public-key-here
networkData: |-
  version: 2
  ethernet:
    enp1s0:
      dhcp4: true
    enp2s0:
      dhcp4: false
      dhcp6: false
      addresses:
        - 192.168.100.100/24
name: cloudinitdisk
```

## SSH 接続

```
ssh fedora@192.168.100.100
```

### 接続方法

- 接続
- 接続（パスワード入力）
- 接続（パスワード入力） VM
- 接続（パスワード入力）
- 接続（パスワード入力） VM

### 接続

- 接続（パスワード入力）
- 接続（パスワード入力）
- 接続（パスワード入力）
- 接続（パスワード入力）
- 接続（パスワード入力）

## SSH 接続

### SSH 接続（パスワード入力）

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/ocp-virt-key
```



## cloud-init

VM の `userData`

```
ssh_authorized_keys:
- ssh-rsa AAAAB3NzaC... your-key-here
```

root の `cloud-init` の `userData`

```
volumes:
- cloudInitNoCloud:
  userData: |-
    #cloud-config
    user: fedora
    ssh_pwauth: false
    disable_root: true
    ssh_authorized_keys:
      - ssh-rsa AAAAB3NzaC... your-key-here
  name: cloudinitdisk
```

- `ssh_pwauth: false` - SSH のパスワード認証を無効にする
- `disable_root: true` - SSH で root にログインできないようにする
- SSH の公開鍵を指定する

VM の

```
# VirtualMachine
oc get vm -n vm-access-demo

# VirtualMachineInstance
oc get vmi -n vm-access-demo

# VM
oc describe vmi fedora-access-vm -n vm-access-demo
```

VM の SSH

```
virtctl console fedora-access-vm -n vm-access-demo
```

VM 内部:

```
# SSH 接続確認
sudo systemctl status sshd

# SSH 接続可能ポート確認
sudo ss -tlnp | grep :22

# ファイアウォール確認
sudo firewall-cmd --list-all
```

接続確認

```
# VM 内部から ping
ping -c 3 8.8.8.8

# 外部から ping (パブリック IP)
ping -c 3 192.168.100.100
```

接続確認

接続確認が完了したら、Linux VM 内部から外部に接続できることを確認する

例: **virtctl** を使用して接続確認

例: VM 内部から接続確認

例: 外部から VM 内部に接続確認

例: **SSH** 接続確認

例: SSH 接続確認 (パブリック IP)

例:

- sshd サービス確認: `sudo systemctl status sshd`
- SSH サービス起動: `sudo systemctl start sshd`
- ファイアウォールサービス確認: `sudo firewall-cmd --list-services`

例: **virtctl ssh** を使用して接続確認

例: SSH 接続確認 (パブリック IP)

例:

- VM 的 SSH 连接
- SSH 连接
- VM 连接
- 连接 SSH 连接

的: **NodePort** 连接

的: 连接

的: 连接NodePort 连接

的: 连接 IP 连接

的: 连接

的: 连接VLAN/连接

连接

的

- SSH 连接
- 连接 SSH 连接
- 连接
- 连接

连接

- 连接NodePort 连接 IP 连接
- NetworkPolicy 连接 VM 连接
- VM 连接
- 连接

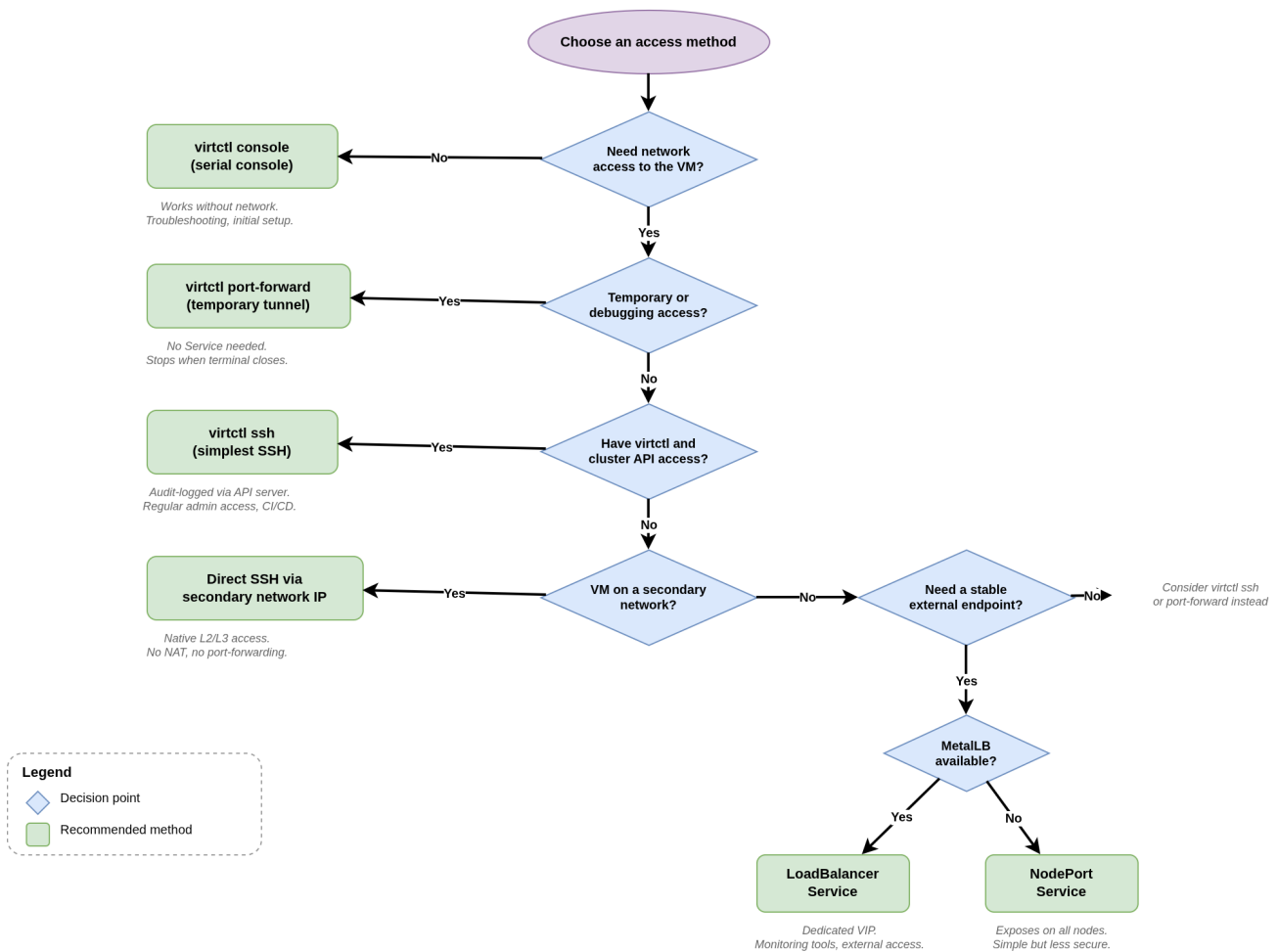
连接

- virtctl 连接 RBAC 连接
- namespace 连接
- VM 连接
- 连接

连接

连接

## Remote Access Method Decision Guide



| 項目                                 | 構成要素                                   | 役割                                 |
|------------------------------------|----------------------------------------|------------------------------------|
| 仮想マシン VM の管理                       | virtctl (virtctl console)              | SSH を介して VM 内部に接続する                |
| VM 内部のサービス                         | virtctl port-forward (oc port-forward) | Service をポートフォワードする                |
| 監視/ログ収集                            | virtctl SSH (virtctl ssh)              | API を介して監視/ログ収集を行う                 |
| VM 内部の CI/CD                       | virtctl SSH                            | SSH を介して CI/CD を実行する               |
| 監視/ログ収集 (Nagios/Zabbix/Prometheus) | NodePort LoadBalancer Service          | VM 内部の LoadBalancer を介して監視/ログ収集を行う |
| 仮想マシン VM                           | SSH (SSH)                              | NAT を介して L2/L3 ネットワークに接続する         |

[illegible]

`virtctl ssh` [NodePort/LoadBalancer](#) [Accessing VMs](#)

Deleting VMs

Deleting services and namespaces

```
# VMs
oc delete vm fedora-access-vm fedora-vlan-vm -n vm-access-demo

# Services
oc delete svc fedora-ssh-nodeport -n vm-access-demo

# namespaces
oc delete namespace vm-access-demo
```

Next

Deleting VMs and Services

- OpenShift Virtualization [Linux VMs](#)
- [Accessing VMs](#)
- `cloud-init` [SSH](#) [Accessing VMs](#)
- [Accessing VMs](#) [Linux VMs](#)
- [Accessing VMs](#) [Accessing VMs](#)
- [Accessing VMs](#) [Linux VMs](#)
- [Linux VMs](#) [Accessing VMs](#)
- [Accessing VMs](#) [Accessing VMs](#)

Next

- [Managing VMs](#)

## Linux VMs

[Accessing VMs](#) [Virtualization](#) [Accessing VMs](#) [Linux](#) [VMs](#)

Next

- [Accessing VMs](#)
- [Accessing VMs](#) 1: [Accessing VMs](#)
- [Accessing VMs](#) 2: [VNC](#) [Accessing VMs](#)

- [步骤 3: virtctl 安装 SSH](#)
- [步骤 4: NodePort Service 安装 SSH](#)
- [步骤 5: LoadBalancer 安装 SSH](#)
- [步骤 6: 安装虚拟机 SSH](#)
- [步骤 7: 安装虚拟机 SSH 代理](#)
- [安装 SSH 代理](#)

安装虚拟机

## VM 安装

```
oc get vm <vm-name> -n <namespace>
oc get vmi <vm-name> -n <namespace>
```

查看 VM 安装:

| NAME  | AGE | STATUS  | READY |
|-------|-----|---------|-------|
| my-vm | 5m  | Running | True  |

## VM IP 地址

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.interfaces[*].ipAddress}'
```

步骤 1 的 IP 地址

## VM Pod 安装

```
oc get pods -n <namespace> -l kubevirt.io/domain=<vm-name>
```

安装:

| NAME                     | READY | STATUS  | RESTARTS | AGE |
|--------------------------|-------|---------|----------|-----|
| virt-launcher-<vm>-xxxxx | 3/3   | Running | 0        | 5m  |

## guest agent 安装

```
oc get vmi <vm-name> -n <namespace> -o jsonpath='{.status.guestOSInfo.id}'
```

安装 guest agent 代理: 安装 guest agent 代理 (安装 VNC SSH 代理 NodePort LoadBalancer 代理) 安装 guest agent 代理 OS 代理 SSH 代理

## ステップ 1: 確認

確認: 環境が正常に構築されているか

確認:

```
$ virtctl console my-vm -n my-namespace
Error: unable to connect to console
```

確認:

```
# VM の状態を確認
oc get vm my-vm -n my-namespace

# VMI の状態を確認
oc get vmi my-vm -n my-namespace

# virt-launcher Pod の状態を確認
oc get pods -n my-namespace -l kubevirt.io/domain=my-vm
```

確認: - VM の状態を確認 - VMI の状態を確認 - virt-launcher Pod の状態を確認 - 確認: API が正常に動作しているか

確認:

### 1. VM の状態を確認:

```
virtctl start my-vm -n my-namespace
```

### 2. VMI の状態を確認:

```
oc wait --for=condition=Ready vmi/my-vm -n my-namespace --timeout=60s
```

### 3. virt-launcher Pod の状態を確認:

```
oc logs -n my-namespace -l kubevirt.io/domain=my-vm
```

### 4. virtctl のバージョンを確認:

```
virtctl version
oc get kubevirt -n openshift-cnv -o
jsonpath='{.items[0].status.observedKubeVirtVersion}'
```

**00:** 0000000000000000000000000000

**00:** 00000000000000000000000000000000

□□:

# Enter

# Enter

```
root@kali:~# systemctl status systemd-getty
systemd-getty.service - VM systemd - VM systemd - ttyS0
systemd-getty.service is active (running) since 2024-07-27 14:28:00 CEST; 1min 4s ago
Main PID: 100
CGroup: /systemd/system/systemd-getty.service
         └─ 100 /usr/sbin/getty -o -L -h -b 38400 ttyS0
```

□□□:

1. VM 0000000000000000 (VMI 00000000):

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.phase}'
```

2. 說明 OS 如何管理記憶體空間：

- RHEL/Fedora 7+: `systemctl status serial-getty@ttyS0.service`
- `journalctl -u serial-getty@ttyS0.service`

3. □□□□□□□□□□:

```
# Ctrl+]  [] []
```

```
virtctl console my-vm -n my-namespace
```

00: 0000000000000000

:

```
root@kali: ~# ssh root@10.10.10.10 - cloud-init - SSH root@10.10.10.10 - root@10.10.10.10
```

100



cloud-init 安装 SSH 服务

## 1. cloud-init 安装配置:

```
cloudInitNoCloud:
  userData: |
    #cloud-config
    user: fedora
    password: mypassword
    chpasswd:
      expire: false
```





```
# Linux
sudo dnf install tigervnc

# macOS
brew install tiger-vnc
```

2. localhost:5900 또는 127.0.0.1:5900 으로 접속
3. `virtctl ssh` 명령을 사용하여 (5901, 5902 등) 접속

참고: **VNC** 사용 방법

참고: VNC 서버 (가이드) 설치 방법

참고: - VM 설정 - 가상 머신 인터페이스 - VM 설정 (GUI)

참고:

1. VM 설정
2. VM GUI 설정:

```
# VM
systemctl status gdm # For GNOME
systemctl status lightdm # For other DEs
```

3. GUI VM 설정

### 참고 3: `virtctl ssh` 사용

참고: `virtctl ssh` 사용

참고:

```
$ virtctl ssh fedora@my-vm -n my-namespace
Error: SSH is not available for this VM
```

참고:

```
# VM IP
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}'

# VM SSH (가이드)
virtctl console my-vm -n my-namespace
# VM: systemctl status sshd
```

참고: - VM SSH 설정 - VM IP 설정 - VM SSH 설정 - 가이드

□□□:

1. VM □ SSH □□□□□□□□□□□□□□:

```
# □□□□□□□□ VM □□
sudo dnf install openssh-server
sudo systemctl enable --now sshd
```

2. VM □ IP □□□□□□□□□□□□□□:

```
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces[0].ipAddress}'
```

3. SSH □□□□□□□□□□:

```
# □□ port-forward □□□□□
oc port-forward vmi/my-vm -n my-namespace 2222:22 &
ssh -p 2222 fedora@localhost
```

□□: **virtctl ssh** □□□□□□□□□□□□□□

□□:

```
$ virtctl ssh fedora@my-vm -n my-namespace
fedora@my-vm: Permission denied (publickey,gssapi-keyex,gssapi-with-mic) .
```

□□:

```
# SSH □□□□
ls -la ~/.ssh/

# □□□□□□□□□□
virtctl ssh --local-ssh-opts="-o PreferredAuthentications=password" fedora@my-vm -n
my-namespace
```

□□□□□□: - SSH □□□□ VM □ **authorized\_keys** □□□□□□□□ - □□□□□□□□□□□□ - SSH □□□□□□□□□□□□

□□□:

1. cloud-init □□□ SSH □□□□□□□□:

```
cloudInitNoCloud:
  userData: |
    #cloud-config
    ssh_authorized_keys:
```

```
- ssh-rsa AAAAB3NzaC1yc2EA... user@host
```

2. VM 内部に SSH をインストール:

```
mkdir -p ~/.ssh
echo "ssh-rsa AAAAB3..." >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

3. Fedora 内部に `fedora` RHEL 内部に `ccloud-user` `ec2-user` 内部に

内部: `virtctl ssh` 内部に

内部: 内部に 30 内部に

内部: - DNS 内部に - GSSAPI 内部に

内部: 内部に SSH 内部に:

```
virtctl ssh --local-ssh-opts="-o GSSAPIAuthentication=no -o UseDNS=no" fedora@my-vm -n my-namespace
```

## 内部 4: NodePort Service 内部 SSH

内部: **NodePort Service** 内部に

内部:

```
$ ssh -p 30022 fedora@<node-ip>
ssh: connect to host <node-ip> port 30022: Connection refused
```

内部:

```
# Service 内部に
oc get svc -n my-namespace

# Service 内部に
oc get endpoints my-vm-ssh -n my-namespace

# NodePort 内部に
oc get svc my-vm-ssh -n my-namespace -o jsonpath='{.spec.ports[0].nodePort}'
```

内部: - 内部に - NodePort 内部に - VM Pod 内部に - 内部に NodePort 内部に - VM 内部 SSH 内部 (listen) 内部に

内部:

1. Service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: my-vm-ssh
spec:
  type: NodePort
  selector:
    kubevirt.io/domain: my-vm
  ports:
    - protocol: TCP
      port: 22
      targetPort: 22
      nodePort: 30022 # Optional, will be auto-assigned if omitted
```

2. Selector VM labels:

```
oc get vm my-vm -n my-namespace -o jsonpath='{.spec.template.metadata.labels}'
```

3. VM SSH:

```
virtctl console my-vm -n my-namespace
# VM:
systemctl status sshd
```

NodePort VM

SSH VM

:

```
# Service
oc describe svc my-vm-ssh -n my-namespace

# Endpoints - VM Pod IP
```

- Service Selector VM - Service Selector VM

:

1. Service Selector:

```
selector:
  kubevirt.io/domain: my-vm # VM
#
```

```
app: my-specific-vm-app
```

2. 確認 1 のエンドポイント:

```
oc get endpoints my-vm-ssh -n my-namespace -o yaml
```

例: **Node IP** 確認

例: **NodePort** 確認 Node IP 確認

例:

```
# Node IP 確認
oc get nodes -o wide

# Node INTERNAL-IP EXTERNAL-IP (確認) 確認

# VM 確認 Node
VM_NODE=$(oc get vmi my-vm -n my-namespace -o jsonpath='{.status.nodeName}')
oc get node $VM_NODE -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}'
```

## 例 5: LoadBalancer IP SSH

例: **LoadBalancer Service IP** 確認

例:

```
$ oc get svc my-vm-ssh -n my-namespace
```

| NAME      | TYPE         | CLUSTER-IP | EXTERNAL-IP | PORT(S)      |
|-----------|--------------|------------|-------------|--------------|
| my-vm-ssh | LoadBalancer | 172.30.x.x | <pending>   | 22:30022/TCP |

例:

```
# LoadBalancer 確認
oc get svc -A | grep LoadBalancer

# LoadBalancer 確認
oc describe svc my-vm-ssh -n my-namespace
```

例: - LoadBalancer (MetalLB) - LoadBalancer - LoadBalancer

例:

1. [MetalLB](#) 確認

2. `oc describe svc my-vm-ssh -n my-namespace`

3. LoadBalancer IP를 확인합니다:

```
oc describe svc my-vm-ssh -n my-namespace
```

4. `oc`: LoadBalancer IP와 NodePort를 확인합니다

**예: LoadBalancer IP 확인**

`oc`: IP를 사용하여 SSH를 실행합니다

`oc`:

```
# LoadBalancer IP 확인
LB_IP=$(oc get svc my-vm-ssh -n my-namespace -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')

# ping 실행
ping -c 3 $LB_IP

# SSH 실행
oc run test-curl --image=curlimages/curl --rm -it --restart=Never -- curl -v
telnet://$LB_IP:22
```

예시: - LoadBalancer IP를 사용하여 - SSH를 실행합니다 - LoadBalancer IP - VM에 SSH를 실행합니다

`oc`:

1. LoadBalancer IP를 확인합니다
2. LoadBalancer IP를 사용하여 SSH를 실행합니다
3. VM에 SSH를 실행합니다 (포트 4를 사용합니다)

**예 6: SSH를 실행합니다**

`oc port-forward`를 실행합니다

`oc`:

```
$ oc port-forward vmi/my-vm -n my-namespace 2222:22
Error: unable to forward port
```

`oc`:

```
# VMI 확인
oc get vmi my-vm -n my-namespace
```

```
# 確認ネットワークポールの設定
netstat -tuln | grep 2222
lsof -i :2222
```

確認: - ネットワークポリシー - VMI ネットワークポリシー - NetworkPolicy 確認

確認:

1. ネットワークポリシー:

```
oc port-forward vmi/my-vm -n my-namespace 12345:22
```

2. ネットワークポリシー:

```
# 確認
lsof -ti :2222 | xargs kill -9
```

3. VMI ネットワークポリシー:

```
oc wait --for=condition=Ready vmi/my-vm -n my-namespace
```

確認: ネットワークポリシー SSH 確認

確認:

```
$ oc port-forward vmi/my-vm -n my-namespace 2222:22 &
Forwarding from 127.0.0.1:2222 -> 22

$ ssh -p 2222 fedora@localhost
ssh: connect to host localhost port 2222: Connection refused
```

確認:

```
# VM SSH 確認
virtctl console my-vm -n my-namespace
# VM 確認:
ss -tuln | grep :22
```

確認: - VM SSH 確認 - SSH 確認 - VM SSH 確認

確認:

1. VM SSH 確認:



```
# VM
sudo dnf install openssh-server
sudo systemctl enable --now sshd
```

## 2. SSH 確認:

```
# VM
sudo ss -tuln | grep ssh
```

## 3. SSH 確認:

```
# VM
sudo cat /etc/ssh/sshd_config | grep ^Port
```

OS: CentOS

OS: CentOS

OS: - API - API - API

OS:

## 1. SSH keepalive 確認:

```
ssh -o ServerAliveInterval=60 -p 2222 fedora@localhost
```

## 2. 確認:

```
while true; do
  oc port-forward vmi/my-vm -n my-namespace 2222:22
  sleep 5
done &
```

## OS 7: 確認

OS: VM

OS:

```
$ ping 192.168.100.10
Request timeout
```

OS:

```
# VM IP address
oc get vmi my-vm -n my-namespace -o jsonpath='{.status.interfaces}' | jq

# NetworkAttachmentDefinition (NAD)
oc get network-attachment-definition -n my-namespace

# VM IP address
oc get vm my-vm -n my-namespace -o jsonpath='{.spec.template.spec.networks}' | jq
```

NetworkAttachmentDefinition (NAD) - VM IP address - IP address - IP address - VLAN ID

IP:

1. NAD IP address:

```
oc describe network-attachment-definition my-nad -n my-namespace
```

2. VM IP address:

```
spec:
  template:
    spec:
      networks:
      - name: default
        pod: {}
      - name: secondary-net
        multus:
          networkName: my-nad
      domain:
        devices:
          interfaces:
          - name: default
            masquerade: {}
          - name: secondary-net
            bridge: {}
```

3. VM IP address:

```
virtctl console my-vm -n my-namespace
# VM IP:
ip addr show enp2s0 # IP address
```

4. cloud-init IP address (cloud-init IP address) IP
5. VLAN IP address OVS bridge IP address



```

echo ""
echo "=== VM IP Addresses ==="
oc get vmi $VM_NAME -n $NAMESPACE -o jsonpath='{.status.interfaces}' | jq

echo ""
echo "=== Guest Agent Status ==="
oc get vmi $VM_NAME -n $NAMESPACE -o jsonpath='{.status.guestOSInfo.id}'
echo ""

echo ""
echo "=== VM Networks ==="
oc get vm $VM_NAME -n $NAMESPACE -o jsonpath='{.spec.template.spec.networks}' | jq

echo ""
echo "=== Services for VM ==="
oc get svc -n $NAMESPACE -l kubevirt.io/domain=$VM_NAME

echo ""
echo "=== NetworkAttachmentDefinitions ==="
oc get network-attachment-definition -n $NAMESPACE

echo ""
echo "=== Recent Events ==="
oc get events -n $NAMESPACE --field-selector involvedObject.name=$VM_NAME --sort
-by='.lastTimestamp' | tail -10

```

## VM □□□□□□□□□□□□□□□□

```

#!/bin/bash
echo "=== Network Interfaces ==="
ip addr show

echo ""
echo "=== Routing Table ==="
ip route show

echo ""
echo "=== DNS Configuration ==="
cat /etc/resolv.conf

echo ""
echo "=== SSH Service Status ==="
systemctl status sshd

echo ""
echo "=== SSH Listening Ports ==="
ss -tuln | grep :22

echo ""

```

```

echo "=== Guest Agent Status ==="
systemctl status qemu-guest-agent

echo ""
echo "=== Firewall Status ==="
systemctl status firewalld 2>/dev/null || echo "Firewall not active"

echo ""
echo "=== Connectivity Test ==="
ping -c 2 8.8.8.8 && echo "External connectivity: OK" || echo "External connectivity:
FAILED"

```

## □□□□□□□□ - □□□□□□□□□□□□□□□□

```

Need access to VM?
|
|— VM not running? → virtctl start VM
|
|— Just need console? → virtctl console
|   |— Not working? → Check VM running, check virt-launcher pod
|
|— Need GUI? → virtctl vnc
|   |— Not working? → Check VNC graphics device, install VNC viewer
|
|— Need SSH?
|   |— From workstation with virtctl? → virtctl ssh
|       |— Not working? → Check SSH service running, SSH keys configured
|   |
|   |— Persistent access needed? → NodePort Service
|       |— Not working? → Check service selector, NodePort number, node firewall
|   |
|   |— Public access needed? → LoadBalancer Service
|       |— Not working? → Check LoadBalancer provider installed, IP assigned
|   |
|   |— Temporary access? → oc port-forward
|       |— Not working? → Check local port free, VMI running, SSH in VM
|   |
|   |— Direct network access? → Secondary Network
|       |— Not working? → Check NAD, VM secondary interface, IP config, network
routing

```

## □□□□□□

- [OpenShift Virtualization Networking](#)
- [KubeVirt Network Access](#)
- [Virtctl CLI Reference](#)
- [Kubernetes Services](#)

- MetalLB
- cloud-init IP Configuration Troubleshooting
- OVS Bridge Verification

□□□□: **ocplab01** □□□□□

SNO 00000 (ocplab01) 00000:

```
$ oc get vmi fedora-access-vm -n vm-access-test -o jsonpath='{.status.interfaces}' |
jq
[
  {
    "infoSource": "domain, guest-agent, multus-status",
    "interfaceName": "eth0",
    "ipAddress": "10.128.0.171",
    "ipAddresses": [
      "10.128.0.171",
      "fd02::62"
    ],
    "mac": "02:92:c2:00:00:08",
    "name": "default",
    "queueCount": 1
  }
]
```

guest agent 000000:

```
$ oc get vmi fedora-access-vm -n vm-access-test -o jsonpath='{.status.guestOSInfo}' |
jq
{
  "id": "fedora",
  "kernelRelease": "6.11.4-301.fc41.x86_64",
  "kernelVersion": "#1 SMP PREEMPT_DYNAMIC Thu Oct 10 16:16:06 UTC 2024",
  "name": "Fedora Linux",
  "prettyName": "Fedora Linux 41 (Cloud Edition)",
  "version": "41",
  "versionId": "41"
}
```

```

#####VM   #####guest   agent   #####guest   agent   #####   OS   #####SSH
#####

```

## RDP ـــــــــ Windows VM ــــــــــــــــــــــــــــــــ

❏

OpenShift Windows 11 (RDP)  
Windows VM VNC  
❏

❏

Red Hat OpenShift Virtualization Windows VM Microsoft  
❏

❏

❏

❏ IP ❏

- **10.0.2.2:** Windows OS ❏ VM ❏ IP
- **10.128.1.132:** OpenShift ❏ Pod IP
- ❏ - ❏

❏

- OpenShift ❏ Windows 11 VM ❏
- OpenShift Web ❏ **oc** CLI ❏
- VNC ❏ VM ❏
- Windows 11 VM ❏
- ❏ Windows ❏

❏

❏

❏ **1: Windows 11** ❏

1. **VM** ❏

OpenShift VNC ❏ Windows 11 VM ❏

2. ❏

❏

❏

3. ❏

❏

OpenShift Console

OpenShift Console

OpenShift Console PC

ステップ 2: OpenShift Console VM

RDP 3389

ステップ A: OpenShift Web Console (UI)

### 1. OpenShift Console

OpenShift Console → Your Project → Networking → Services

### 2. OpenShift Console

OpenShift Console

OpenShift Console

| OpenShift Console |                                     |                   |
|-------------------|-------------------------------------|-------------------|
| Service           | LoadBalancer                        | OpenShift IP      |
| Service           | win11-rdp-service                   | OpenShift Console |
| Selector          | vm.kubevirt.io/name: <your-vm-name> | VM: OpenShift VM  |
| Port              | 3389                                | OpenShift Console |
| TargetPort        | 3389                                | Windows VM RDP    |
| Protocol          | TCP                                 | RDP               |



LoadBalancer OpenShift Console NodePort (30000-32767)

### 3. Service

OpenShift Console

NodePort

ステップ B: OpenShift Console (CLI)

### 1. Service

rdp-service.yaml

```
apiVersion: v1
kind: Service
metadata:
```



```

name: win11-rdp-service
spec:
  selector:
    vm.kubevirt.io/name: your-vm-name # REPLACE with your VM's actual name
  ports:
    - protocol: TCP
      port: 3389
      targetPort: 3389
      type: LoadBalancer # or NodePort

```

## 2. 部署

```
oc apply -f rdp-service.yaml
```

部署 C: **virtctl** 命令

```
virtctl expose vm <your-vm-name> --port=3389 --name=<your-vm-name>-rdp-service
--type=LoadBalancer
```

**<your-vm-name>** 为 VM 名称，**LoadBalancer** 为 **NodePort** 类型，**--type=NodePort** 为 **NodePort** 类型

## 部署 3: 部署

**LoadBalancer** 部署:

1. **Networking** → **service** 部署
2. **win11-rdp-service** 部署
3. 为 **IP** 部署

**NodePort** 部署:

1. Service 部署 **NodePort** (为 **31234**) 部署
2. Worker Node 为 **IP** 部署

```
oc get nodes -o wide
```

**EXTERNAL-IP** 部署

## 部署 4: 部署

1. 部署

部署 **Windows** 部署

2. 部署

|              |                               |                    |
|--------------|-------------------------------|--------------------|
| LoadBalancer | <External-IP>                 | 203.0.113.5        |
| NodePort     | <Node-External-IP>:<NodePort> | 203.0.113.10:31234 |

3.

Windows 11 VM



RDP

Windows

VPN (NLA)

- 
- Windows 11
- 
- / ( )

|    |              |          |
|----|--------------|----------|
|    |              |          |
|    |              | IP       |
|    | Windows RDP  | Windows  |
|    | VM           | VNC/     |
| IP | LoadBalancer | NodePort |

- OpenShift Virtualization Windows 11 VM (RDP)
- LoadBalancer NodePort VM RDP (3389)
- (IP NodePort IP)
- Windows VM
- RDP

11

- [OpenShift Virtualization Documentation](#)
- [KubeVirt User Guide](#)
- [Windows Remote Desktop Best Practices](#)

## SSH 与 Windows VM 的安装

11

```
Windows 10 (1809) VM Windows Server 2019 OpenSSH Server
Windows VM SSH Remote Desktop

```

CI/CD SSH Windows Ansible

□ □ □ □

- OpenShift Virtualization のインストール Windows VM の OpenSSH Server のインストール
- Kubernetes Service のインストール Windows VM の SSH のインストール
- OpenShift Virtualization のインストール SSH のインストール Windows VM のインストール
- OpenShift SSH のインストール PowerShell のインストール

1111

- OpenShift Virtualization Operator `openshift/virt-operator` OpenShift 4.18+
- `virt-launcher` Windows VM (Windows 10 1809+`Windows 11` Windows Server 2019/2022/2025)
- `virt-launcher` Windows VM `virt-launcher` VNC `virt-launcher` RDP `virt-launcher`
- `virt-launcher` CLI `virt-launcher` `oc` `virtctl`
- Windows VM `virt-launcher` Pod `virt-launcher` `masquerade` network binding `virt-launcher` (Windows)
- `virt-launcher` pod `virt-launcher` NAT `virt-launcher` VM spec `virt-launcher` `masquerade` `virt-launcher` 22 `virt-launcher` (Windows)

VM 0000000000000000 22 000

masquerade のインストールは、virt-launcher pod の VM spec に以下を追加して、  
 VM を sshd で起動する Kubernetes Service から実行可能にする。

VirtualMachine spec = masquerade 22

```
spec:
  template:
    spec:
```

```

domain:
  devices:
    interfaces:
      - masquerade: {}
        name: default
        ports:
          - port: 22
  networks:
    - name: default
      pod: {}

```

ports `VirtualMachine`

```
oc edit vm <your-windows-vm-name>
```

masquerade `ports` VM

```
virtctl restart <your-windows-vm-name>
```

## Windows VM `OpenSSH Server`

`OpenShift` `VNC` Windows VM

**VM**

OpenShift Web `VM`: **Virtualization** > **VirtualMachines** > `VM` > **Console**

`virtctl`

```
virtctl vnc <your-windows-vm-name>
```

## OpenSSH

Windows VM `PowerShell` `OpenSSH Server`

```
Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH.Server*'
```

```

Name   : OpenSSH.Server~0.0.1.0
State  : NotPresent

```

`Installed`

## OpenSSH Server 確認

OpenSSH Server 確認

```
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
```

確認

```
Path :  
Online : True  
RestartNeeded : False
```

## SSH 確認

SSH 確認

sshd 確認

```
Start-Service sshd  
Set-Service -Name sshd -StartupType Automatic
```

確認

```
Get-Service sshd
```

確認

| Status  | Name | DisplayName        |
|---------|------|--------------------|
| Running | sshd | OpenSSH SSH Server |

確認

OpenSSH Server 確認Windows 確認

```
Get-NetFirewallRule -Name *OpenSSH-Server* | Select-Object Name, Enabled, Direction, Action
```

確認

| Name                  | Enabled | Direction | Action |
|-----------------------|---------|-----------|--------|
| OpenSSH-Server-In-TCP | True    | Inbound   | Allow  |

確認

```
New-NetFirewallRule -Name 'OpenSSH-Server-In-TCP' -DisplayName 'OpenSSH Server (sshd)'  
-Enabled True -Direction Inbound -Protocol TCP -Action Allow -LocalPort 22
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```

#####OpenSSH #####Private #####Windows
#####KubeVirt masquerade bridge  Public #####sshd
#####Windows Firewall ##### SSH #####

```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
Get-NetFirewallRule -DisplayName "OpenSSH SSH Server (sshd)" | Select-Object
DisplayName, Profile
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
Get-NetConnectionProfile | Select-Object InterfaceAlias, NetworkCategory
```

Private Public

```
Set-NetFirewallRule -DisplayName "OpenSSH SSH Server (sshd)" -Profile Any
```



sshhd VM SSH OpenShift  
Virtualization Windows VM SSH

## PowerShell

Windows の SSH 接続に `cmd.exe` と `PowerShell` を使ってみよう

```
New-ItemProperty -Path "HKLM:\SOFTWARE\OpenSSH" -Name DefaultShell -Value "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -PropertyType String -Force
```

## 🔒 SSH 连接配置

# SSH 接続

Windows VM → Pod → Kubernetes Service →

### □□□□□ A: NodePort Service

NodePort Service 暴露到宿主机 VM 端口 22 暴露到宿主机

Service □□□□□□□□□□

## windows-ssh-nodeport-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: windows-ssh-nodeport
spec:
  type: NodePort
  selector:
    vm.kubevirt.io/name: <your-windows-vm-name>
  ports:
    - protocol: TCP
      port: 22
      targetPort: 22
      nodePort: 30022
```



<your-windows-vm-name> Windows VirtualMachine

Service

```
oc apply -f windows-ssh-nodeport-service.yaml
```

Service

```
oc get svc windows-ssh-nodeport
```

| NAME                 | TYPE     | CLUSTER-IP | EXTERNAL-IP | PORT(S)      | AGE |
|----------------------|----------|------------|-------------|--------------|-----|
| windows-ssh-nodeport | NodePort | 172.30.x.x | <none>      | 22:30022/TCP | 5s  |

IP

```
NODE_IP=$(oc get nodes -o
jsonpath='{.items[0].status.addresses[?(@.type=="InternalIP")].address}')
echo $NODE_IP
```



InternalIP IP NodePort  
oc get nodes -o wide ExternalIP

**B: LoadBalancer**

(MetalLB) LoadBalancer IP

## windows-ssh-lb-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: windows-ssh-lb
spec:
  type: LoadBalancer
  selector:
    vm.kubevirt.io/name: <your-windows-vm-name>
  ports:
    - protocol: TCP
      port: 22
      targetPort: 22
```



<your-windows-vm-name> Windows VirtualMachine

Service

```
oc apply -f windows-ssh-lb-service.yaml
```

IP

```
oc get svc windows-ssh-lb -w
```

EXTERNAL-IP IP **Ctrl+C** IP

## SSH Windows VM

NodePort Service

```
ssh <windows-username>@$NODE_IP -p 30022
```

LoadBalancer Service

```
ssh <windows-username>@<external-ip>
```

<windows-username> Windows (Administrator)

## virtctl SSH

virtctl Kubernetes API SSH



```
virtctl ssh <windows-username>@<your-windows-vm-name>
```

Service API VM 22

## SSH

SSH Ansible

## SSH

```
ssh-keygen -t ed25519 -f ~/.ssh/windows-vm-key -N ""
```

## Windows VM

Windows VM SSH (SSH)

(SSH) `authorized_keys`

```
mkdir -Force $env:USERPROFILE\.ssh
# authorized_keys
Set-Content -Path "$env:USERPROFILE\.ssh\authorized_keys" -Value "ssh-ed25519 AAAA...
your-key-here"
```

Administrator Windows `administrators_authorized_keys`

```
Set-Content -Path "C:\ProgramData\ssh\administrators_authorized_keys" -Value "ssh-
ed25519 AAAA... your-key-here"

# Administrators SYSTEM
icaccls "C:\ProgramData\ssh\administrators_authorized_keys" /inheritance:r /grant
"Administrators:F" /grant "SYSTEM:F"
```



`administrators_authorized_keys`

SSH

```
ssh -i ~/.ssh/windows-vm-key <windows-username>@$NODE_IP -p 30022
```

## Ansible 操作

SSH 操作 WinRM 操作 SSH 操作 Ansible 操作 Windows VM 操作

```
[windows]
windows-vm ansible_host=<NODE_IP> ansible_port=30022

[windows:vars]
ansible_user=Administrator
ansible_ssh_private_key_file=~/.ssh/windows-vm-key
ansible_shell_type=powershell
ansible_become_method=runas
```

操作

```
ansible windows -m ansible.windows.win_ping
```

SSH 操作 Ansible 操作 Windows 操作 [Ansible Windows SSH 操作](#) 操作

操作

操作

```
oc delete svc windows-ssh-nodeport windows-ssh-lb
```

Windows VM 上 OpenSSH Server 操作 VM 上 PowerShell 操作

```
Stop-Service sshd
Remove-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
```

操作

- virt-launcher pod 上 SSH 操作 masquerade 操作 22 操作
- OpenShift Virtualization 操作 Windows VM 上 OpenSSH Server 操作
- SSH 操作
- 操作 SSH 操作 PowerShell 操作
- NodePort 上 LoadBalancer Service 操作 VM 上 SSH 操作
- 操作 SSH 操作 Windows VM 操作
- SSH 操作 Windows VM 上 Ansible 操作

操作

- [RDP 操作 Windows VM 操作](#)

- [Linux VM](#)
- [Ansible Windows SSH](#)
- [Microsoft OpenSSH](#)
- [OpenShift Virtualization -](#)

## VirtIO Windows VM

OpenShift Virtualization Windows VM I/O

Windows VM Windows VirtIO OpenShift Virtualization

- **OpenShift 4.18+** (OpenShift Virtualization Operator )
- **CLI** : `oc` `virtctl`
- **Windows** **ISO** (Windows 10/11 Server 2019/2022)
- **VirtIO** **ISO** (Red Hat Fedora )
- **Windows**
- : 4GB RAM 60GB

OpenShift 4.20, 4.21

### 1: VirtIO

Fedora VirtIO ISO

```
curl -L -O https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-virtio/virtio-win.iso
```

VirtIO ISO DataVolume

`virtio-drivers-dv.yaml`

```
oc apply -f - <<EOF
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
```

```

metadata:
  name: virtio-drivers-iso
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  storage:
    resources:
      requests:
        storage: 1Gi
    accessModes:
      - ReadWriteOnce
  source:
    upload: {}
EOF

```

VirtIO `virtio-drivers-iso` ISO `virtio-win.iso`

```
virtctl image-upload dv virtio-drivers-iso --image-path=virtio-win.iso --insecure
```



CDI `--insecure` `virtctl image-upload` `--image-path=` `virtio-win.iso`

## 2: Windows `windows.iso` ISO `windows.iso`

Windows `windows.iso` ISO `windows.iso` `DataVolume` `windows.iso`

```

oc apply -f - <<EOF
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: windows-installation-iso
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  storage:
    resources:
      requests:
        storage: 6Gi
    accessModes:
      - ReadWriteOnce
  source:
    upload: {}
EOF

```

Windows `windows.iso` ISO `windows.iso` (`windows.iso` `windows.iso` `windows.iso`)

```
virtctl image-upload dv windows-installation-iso --image-path=windows.iso --insecure
```

### 例 3: VirtIO をインストールした Windows VM の例

VirtIO をインストールした Windows VM の例

[windows-vm-virtio.yaml](#) の例

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: windows-vm-virtio
  labels:
    app: windows-vm-virtio
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/size: medium
        kubevirt.io/domain: windows-vm-virtio
        vm.kubevirt.io/name: windows-vm-virtio
    spec:
      domain:
        cpu:
          cores: 2
          sockets: 1
          threads: 2
        resources:
          requests:
            memory: 4Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
                bootOrder: 2
            - name: windows-installation-iso
              cdrom:
                bus: sata
                bootOrder: 1
            - name: virtio-drivers-iso
              cdrom:
                bus: sata
          interfaces:
            - name: default
              masquerade: {}
              model: virtio
          tpm: {}
          rng: {}
        clock:
          utc: {}
```

```

    timer:
      hpet:
        present: false
      pit:
        tickPolicy: delay
      rtc:
        tickPolicy: catchup
      hyperv: {}
  features:
    acpi: {}
    apic: {}
    hyperv:
      relaxed: {}
      vapic: {}
      spinlocks:
        spinlocks: 8191
      vpinde: {}
      runtime: {}
      synic: {}
      stimer: {}
      reset: {}
      frequencies: {}
      reenlightenment: {}
      tlbflush: {}
      ipi: {}
    smm:
      enabled: true
  firmware:
    bootloader:
      efi:
        secureBoot: true
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      dataVolume:
        name: windows-vm-virtio-rootdisk
    - name: windows-installation-iso
      dataVolume:
        name: windows-installation-iso
    - name: virtio-drivers-iso
      dataVolume:
        name: virtio-drivers-iso
  dataVolumeTemplates:
    - metadata:
        name: windows-vm-virtio-rootdisk
        annotations:
          cdi.kubevirt.io/storage.bind.immediate.requested: "true"
  spec:
    storage:

```

```
resources:
  requests:
    storage: 60Gi
  accessModes:
    - ReadWriteOnce
  source:
    blank: {}
```

EOF

VM を起動する

```
oc get vm windows-vm-virtio -w
```

STATUS=Running かつ READY=True になったら **Ctrl+C** で停止する

## 4: VirtIO をインストールして Windows を起動する

VM を起動して Windows をインストールする

```
virtctl vnc windows-vm-virtio
```

OpenShift Web コンソール: **Virtualization** → **VirtualMachines** → **windows-vm-virtio** → **Console**

### 1. Windows をインストール:

VM の設定で **No bootable device found** のエラーが出て UEFI のインストールが失敗する

- UEFI のインストールが失敗する場合は **boot from other media or DVD** を選択する
- **Boot from DVD/CD** を選択する
- **Windows ISO** (または **Windows ISO**) を指定する
- **Windows** のインストールが完了するまで待つ

### 2. VirtIO をインストールして Windows をインストールする:

- **Windows** のインストールが完了するまで待つ
- VirtIO のインストール CD (または D: または E: のドライブ) を指定する
- **viostor{windows-version}\amd64\** を指定する
- Windows Server 2022 の場合は **viostor\2k22\amd64\**
- Windows 11 の場合は **viostor\w11\amd64\**
- **Red Hat VirtIO SCSI Controller** をインストールする

### 3. 再起動して VirtIO をインストールして Windows をインストールする

### 4. 再起動して Windows をインストールする

## 5: VirtIO

Windows

1. : PC → →
2. :
  - 
  - →
  - VirtIO CD NetKVM{windows-version}\amd64\
  -
3. :
  - **Balloon** : PCI → Balloon{windows-version}\amd64\
  - : PCI → vioserial{windows-version}\amd64\
  - : PCI → vioinput{windows-version}\amd64\
4. **QEMU guest agent** :
 

VirtIO CD guest-agent\qemu-ga-x86\_64.msi

## 6:

VirtIO

1. :
2. : (services.msc) **QEMU Guest Agent**
3. : OpenShift

oc get vmi windows-vm-virtio -o jsonpath='{.status.guestOSInfo}'

VM

oc get vmi windows-vm-virtio -o jsonpath='{.status.interfaces[0].ipAddress}'

## VirtIO

VirtIO Windows VM

| (NetKVM)  | 10   |
|-----------|------|
| (viostor) | IOPS |
| (vioser)  |      |
|           |      |



インストール

インストール

| OS                       | OS        | OS                                                        |
|--------------------------|-----------|-----------------------------------------------------------|
| インストール <b>TPM 2.0</b> OS | TPM OS    | VM OS Windows 11 OS TPM OS                                |
| インストール OS                | インストール    | Windows ISO <b>bootOrder: 1</b> OS <b>bootOrder: 2</b> OS |
| インストール OS                | VirtIO OS | インストール <b>viostor</b> OS                                  |
| インストール OS                | NetKVM OS | Windows OS                                                |

インストール

ISO OS TLS OS:

```
# OS --insecure OS
virtctl image-upload dv <datavolume-name> --image-path=<iso-file> --insecure
```

インストール

インストール OS

```
oc delete vm windows-vm-virtio
oc delete dv windows-installation-iso virtio-drivers-iso
```

インストール

インストール OS

- DataVolume OS VirtIO OS Windows OS ISO OS
- VirtIO OS Windows VM OS
- OS Windows OS VirtIO OS
- OS VirtIO OS QEMU guest agent OS
- OS OpenShift Virtualization OS VM OS
- OS

インストール Windows VM OS VirtIO OS OpenShift Virtualization OS

11

- OpenShift **VirtIO** 仮想ハードウェア
- KubeVirt 仮想マシン
- Windows VM 仮想マシン

# Windows

11

Windows XP Windows Server 2003 Windows windows.xp  
VirtualMachineClusterPreference OpenShift Virtualization preference SATA  
e1000e USB Q35

## Windows OpenShift Virtualization 2

- 仮想ディスクイメージ (VMDK、VHD、VHDX) を qcow2 仮想ディスクイメージに変換
- SATA 仮想ディスクイメージ ISO 仮想ディスクイメージに変換

SATA (AHCI) IDE i440FX  
 SATA (AHCI) IDE i440FX



| Windows  | XP | Server | 2003 | 2014 | 2015 |
|----------|----|--------|------|------|------|
| <p> </p> |    |        |      |      |      |

11

- Windows OpenShift Virtualization
- windows.xp preference VirtualMachine

□□□□

- OpenShift Virtualization Operator [requires](#) OpenShift 4.18+
- [Install](#) OpenShift Virtualization `oc` [with](#) `virtctl` CLI [tools](#)
- [Download](#) `qemu-img` (RHEL/Fedora [use](#) `qemu-img` [on](#) macOS [or](#) Homebrew [to](#) `qemu` [install](#))
- [Download](#) Windows [images](#) (VMDK/VHD/VHDX/raw)[to](#) SATA [or](#) [download](#) Windows XP/Server 2003 [ISO](#)
- [Configure](#) `StorageClass`

**QUESTION 1:**

ISO 15926-2:2018(en) 10

Windows VMware (VMDK)Hyper-V (VHD/VHDX)Raw  
CDI qcow2

VMDK:

```
qemu-img convert -f vmdk -O qcow2 filename.vmdk windows-legacy.qcow2
```

VHD (vpc VHD qemu-img):

```
qemu-img convert -f vpc -O qcow2 filename.vhd windows-legacy.qcow2
```

VHDX:

```
qemu-img convert -f vhdx -O qcow2 filename.vhdx windows-legacy.qcow2
```

:

```
qemu-img info windows-legacy.qcow2
```

```
image: windows-legacy.qcow2
file format: qcow2
virtual size: 20 GiB (21474836480 bytes)
disk size: 3.2 GiB
cluster_size: 65536
Format specific information:
  compat: 1.1
  compression type: zlib
```

virtual size PVC

**2:**

ISO

PVC

```
qemu-img info windows-legacy.qcow2 | grep "virtual size"
```

Windows VM namespace

```
oc new-project legacy-windows
```

`virtctl image-upload pvc windowsxp-dv \`  
`--size=130Gi \`  
`--image-path=windows-legacy.qcow2 \`  
`--force-bind \`  
`--insecure`



PersistentVolumeClaim `dv` `pvc`  
`WaitForFirstConsumer` `binding mode` `Storage Class` `CDI`  
`volume populator` `--force-bind` `PVC`



`--insecure` `CDI` `TLS`

`PVC` `Bound`

```
oc get pvc windowsxp-dv
```

| NAME         | STATUS       | VOLUME                                   | CAPACITY | ACCESS |
|--------------|--------------|------------------------------------------|----------|--------|
| MODES        | STORAGECLASS | AGE                                      |          |        |
| windowsxp-dv | Bound        | pvc-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx | 130Gi    | RWO    |
| lvms-vg1     | 5m           |                                          |          |        |

### 3: VirtualMachine

`windows.xp` `preference` `Q35` `SATA` `e1000e`  
`USB`

`windowsxp-vm.yaml`

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: windowsxp
spec:
  runStrategy: Always
 instancetype:
    name: u1.micro
  preference:
    name: windows.xp
  template:
    spec:
```

```

domain:
  devices: {}
  volumes:
    - persistentVolumeClaim:
        claimName: windowsxp-dv
      name: rootdisk
EOF

```

VM を起動します

```
oc get vm windowsxp -w
```

確認

| NAME      | AGE | STATUS  | READY |
|-----------|-----|---------|-------|
| windowsxp | 10s | Running | True  |

Running 状態を確認 (Ctrl+C で終了)

#### ステップ 4: VM を起動します

VNC を使用して VM を起動します

```
virtctl vnc windowsxp
```

OpenShift Web コンソールで **Virtualization > VirtualMachines > windowsxp > Console** をクリックします



Windows XP のインストールと起動に関する詳細情報は、[Windows XP のインストールと起動](#) を参照してください。

#### ステップ 5: Windows Server 2003 をインストールします

`windows.xp` preference を `Windows XP` から `Server 2003` に変更し、`windows.2k3` preference を `Server 2003` から `u1.small` (1 CPU、2 Gi) に変更します。

```

spec:
 instancetype:
    name: u1.small
  preference:
    name: windows.2k3

```

確認

XXXXXXXX VM XXXXXXXXXXXX (0x0000007B)

Windows    0x0000007B    STOP    0x0000007B    SATA  
 0x0000007B    VM    IDE    0x0000007B

- 設定項目 **F8** 項目 **Disable automatic restart on system failure** (再起動を自動的に中止する) を有効にする
- F8 項目 **Safe Mode** を選択して Windows の SATA 設定を確認する
- Safe Mode を選択して SATA 設定を確認し ISO をインストールする

□ □ □ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc get vmi windowsxp -o jsonpath='{.spec.domain.devices.interfaces[0].model}'
```

## Device Manager

VNC □□□□□□□□□□□□□□□□

[illegible]

```
oc get vmi windowsxp -o jsonpath='{.spec.domain.devices.inputs}'
```

□ □ □ □ □ □ □

VM PVC

```
oc delete vm windowsxp
oc delete pvc windowsxp-dv
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc delete project legacy-windows
```

111

- 仮想マシンを qcow2 形式で OpenShift Virtualization 上で実行する
- e1000e 仮想ネットワーク SATA 接続 windows.xp Preference 設定 VirtualMachine 設定
- VNC 接続で Windows を実行する

11

- [VirtIO 仮想ハードウェア Windows VM 接続](#) — 仮想ハードウェアをインストールする Windows 仮想マシン
- [RDP 経由 Windows VM 接続](#)
- [KubeVirt Machine Type 接続](#)
- [OpenShift Virtualization 接続](#)

## Infrastructure ☐☐☐☐

11

OpenShift Virtualization (virt-controller virt-api virt-handler CDI) openshift-cnv namespace Pod  
VM

HyperConverged OpenShift Virtualization

444

- OpenShift Virtualization
- HyperConverged CR
- Infrastructure Pod
- NodeSelector
- Affinity
- Taint & Toleration

□ □ □ □

- OpenShift Virtualization [OpenShift 4.18](#)
- HyperConverged CR [OpenShift 4.18](#)
- [OpenShift 4.18](#) **oc** CLI
- [OpenShift 4.18](#) Worker Node (OpenShift 4.18)

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

## OpenShift Virtualization

| コンポーネント         | 役割                              | デプロイメント            |
|-----------------|---------------------------------|--------------------|
| virt-controller | VM の管理を司るコントローラ                 | Deployment (2 ノード) |
| virt-api        | 管理 API を提供する                    | Deployment (2 ノード) |
| virt-handler    | 各ノードで VM を管理する                  | DaemonSet          |
| cdi-controller  | Containerized Data Importer の管理 | Deployment (1 ノード) |

| コンポーネント         | 種別                        | デプロイメント             |
|-----------------|---------------------------|---------------------|
| cdi-uploadproxy | Web コンポーネント VM コンポーネント    | Deployment (1 レプリカ) |
| cdi-apiserver   | CDI API コンポーネント           | Deployment (1 レプリカ) |
| hco-webhook     | HyperConverged CR コンポーネント | Deployment (1 レプリカ) |

コンポーネントのインストール

Infrastructure Pod のインストール

```
# virt-automation infrastructure pod のインストール
oc get pods -n openshift-cnv -o wide
```

結果:

| NAME                              | READY | STATUS  | NODE                       |
|-----------------------------------|-------|---------|----------------------------|
| virt-api-6d9f8c7b9d-4xk2l         | 1/1   | Running | ip-10-0-24-89.ec2.internal |
| virt-api-6d9f8c7b9d-8tz5p         | 1/1   | Running | ip-10-0-43-58.ec2.internal |
| virt-controller-7c8d9f5b6d-5nvlm  | 1/1   | Running | ip-10-0-24-89.ec2.internal |
| virt-controller-7c8d9f5b6d-9kq wz | 1/1   | Running | ip-10-0-43-58.ec2.internal |
| virt-handler-8zkpg                | 1/1   | Running | ip-10-0-24-89.ec2.internal |
| virt-handler-f2nlb                | 1/1   | Running | ip-10-0-43-58.ec2.internal |
| virt-handler-q7xmc                | 1/1   | Running | ip-10-0-82-27.ec2.internal |
| cdi-apiserver-5f8c9d7b6c-7nlqx    | 1/1   | Running | ip-10-0-24-89.ec2.internal |
| cdi-deployment-7c8f9b5d6e-2kxmp   | 1/1   | Running | ip-10-0-43-58.ec2.internal |
| cdi-uploadproxy-6d9f8c7b9d-5pqrs  | 1/1   | Running | ip-10-0-24-89.ec2.internal |

コンポーネントのインストール

```
# virt-controller pod のインストール
oc get pods -n openshift-cnv -l kubevirt.io=virt-controller -o wide

# virt-api pod のインストール
oc get pods -n openshift-cnv -l kubevirt.io=virt-api -o wide

# virt-handler DaemonSet のインストール
oc get pods -n openshift-cnv -l kubevirt.io=virt-handler -o wide

# CDI のインストール
oc get pods -n openshift-cnv -l app.kubernetes.io/component=storage -o wide
```

## HyperConverged のインストール

HyperConverged CR のインストール

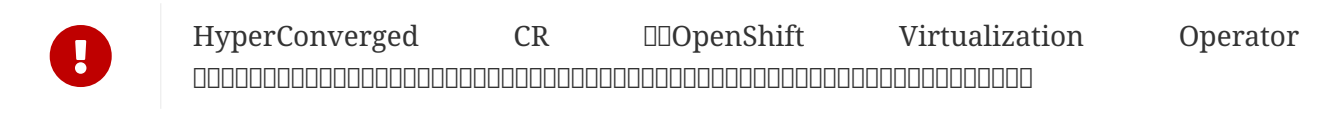
```
# HyperConverged のインストール
```



```
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o yaml
```

## HyperConverged CR 2 0000000000000000

|                              |                                                                                |
|------------------------------|--------------------------------------------------------------------------------|
| spec.infra.nodePlacement     | Control plane pod들: virt-api, virt-controller, cdi-controller, cdi-uploadproxy |
| spec.workloads.nodePlacement | 가상머신 pod들: virt-handler DaemonSet                                              |



□ □ □ □ □ □ □ □ □ □

## NodeSelector

## NodeSelector

```
# 0000000000000000000000000000000000
oc label node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra=
oc label node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=
```

HyperConverged CR 简简单单

```
oc edit hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

## NodeSelector

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec:
  infra:
    nodePlacement:
      nodeSelector:
        node-role.kubernetes.io/infra: ""
  workloads:
    nodePlacement:
      nodeSelector:
        node-role.kubernetes.io/worker: ""
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- **Control plane** (virt-api virt-controller CDI) `node-role.kubernetes.io/infra`
- **virt-handler DaemonSet** `node-role.kubernetes.io/worker`

## NodeAffinity

NodeAffinity

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec:
  infra:
    nodePlacement:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: node-role.kubernetes.io/infra
                    operator: Exists
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              preference:
                matchExpressions:
                  - key: topology.kubernetes.io/zone
                    operator: In
                  values:
                    - us-east-1a
                    - us-east-1b
```

- ( )
- ( )

## Taint Toleration

Taint

```
# Taint
oc adm taint node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra=:NoSchedule
oc adm taint node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=:NoSchedule
```

Toleration HyperConverged CR



virt-api と virt-controller のインストール

インストール

ステップ 1: インストール (3-6 分)

VM の Worker Node のインストール

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec:
  infra:
    nodePlacement:
      nodeSelector:
        node-role.kubernetes.io/worker: ""
  workloads:
    nodePlacement:
      nodeSelector:
        node-role.kubernetes.io/worker: ""
```

ステップ 2: インストール

ステップ 3: インストール

VM の Worker Node のインストール

```
# インストール
oc label node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra=
oc label node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=

# Taint
oc adm taint node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra=:NoSchedule
oc adm taint node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra=:NoSchedule

# VM の Worker Node
oc label node ip-10-0-82-27.ec2.internal kubevirt.io/schedulable=true
```

HyperConverged のインストール:

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec:
```

```
infra:
  nodePlacement:
    nodeSelector:
      node-role.kubernetes.io/infra: ""
    tolerations:
      - key: node-role.kubernetes.io/infra
        operator: Exists
        effect: NoSchedule
workloads:
  nodePlacement:
    nodeSelector:
      kubevirt.io/schedulable: "true"
```

**VM**

**Figure 3: Comparison of the two models**

## Anti-Affinity

```
apiVersion: hco.kubevirt.io/v1beta1
kind: HyperConverged
metadata:
  name: kubevirt-hyperconverged
  namespace: openshift-cnv
spec:
  infra:
    nodePlacement:
      nodeSelector:
        node-role.kubernetes.io/infra: ""
      affinity:
        nodeAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              preference:
                matchExpressions:
                  - key: topology.kubernetes.io/zone
                    operator: In
                    values:
                      - us-east-1a
                      - us-east-1b
                      - us-east-1c
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchLabels:
                    kubevirt.io: virt-api
                topologyKey: topology.kubernetes.io/zone
            - weight: 100
```

```

        podAffinityTerm:
          labelSelector:
            matchLabels:
              kubevirt.io: virt-controller
          topologyKey: topology.kubernetes.io/zone
      tolerations:
      - key: node-role.kubernetes.io/infra
        operator: Exists
        effect: NoSchedule
  workloads:
    nodePlacement:
      nodeSelector:
        kubevirt.io/schedulable: "true"

```

この設定は、Kubernetes の SLA を満たすように設定されています。

この設定は、

## HyperConverged CR の設定

```
oc edit hyperconverged kubevirt-hyperconverged -n openshift-cnv
```

## HyperConverged CR の設定

```

oc patch hyperconverged kubevirt-hyperconverged -n openshift-cnv --type=merge -p '
{
  "spec": {
    "infra": {
      "nodePlacement": {
        "nodeSelector": {
          "node-role.kubernetes.io/infra": ""
        }
      }
    }
  }
}'

```

この設定は、

## Infrastructure Pod の設定

```

# Pod の設定
oc get pods -n openshift-cnv -w

# 設定の確認
oc get pods -n openshift-cnv -o wide

```

```
# 列出所有 Pod
oc get pods -n openshift-cnv -l kubevirt.io=virt-api -o jsonpath='{range
.items[*]}{.metadata.name}{"\t"}{.spec.nodeName}{"\n"}{end}'
```

結果如下：

- Infrastructure Pod 列出所有 Pod
- virt-handler DaemonSet 的 NodeSelector 列出所有 Pod 的 Node/NodeName
- 列出 205 個 Pod

列出所有 Pod

**Pod 列出所有 Pod**

```
# Pod 列出所有 Pod
oc get pods -n openshift-cnv -o jsonpath='{range
.items[*]}{.spec.nodeName}{"\t"}{.metadata.name}{"\n"}{end}' | sort | column -t

# 列出所有 Pod 的 Node
oc get pods -n openshift-cnv -o jsonpath='{range
.items[*]}{.spec.nodeName}{"\n"}{end}' | sort | uniq -c
```

**Toleration 列出**

```
# virt-api pod 的 Toleration 列出
oc get pods -n openshift-cnv -l kubevirt.io=virt-api -o
jsonpath='{.items[0].spec.tolerations}' | jq
```

**NodeAffinity 列出**

```
# virt-controller 的 NodeAffinity 列出
oc get pods -n openshift-cnv -l kubevirt.io=virt-controller -o
jsonpath='{.items[0].spec.affinity}' | jq
```

列出所有 Pod

**Infrastructure Pod 的 (Pending) 列出**

列出所有 Infrastructure Pod 的 Pending 的 Pod

列出：

```
# Pod 列出所有 Pod
oc describe pod <pod-name> -n openshift-cnv
```

```
# 確認失敗イベント
oc get events -n openshift-cnv --sort-by='.lastTimestamp' | grep -i failedscheduling
```

確認事項:

- NodeSelector と Affinity rule が適用されているか
- Taint と Tolerations が一致しているか
- 十分なリソースが確保されているか

確認:

```
# 確認
oc get nodes -L node-role.kubernetes.io/infra

# Taint 確認
oc get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.taints}{"\n"}{end}'

# 確認
oc describe nodes | grep -A 5 "Allocated resources"
```

**virt-handler** のインストール

確認: virt-handler DaemonSet の Pod がインストールされているか

確認:

```
# virt-handler DaemonSet のインストール
oc get daemonset virt-handler -n openshift-cnv

# Pod の確認
oc describe daemonset virt-handler -n openshift-cnv | grep -E "Desired|Current|Ready"
```

確認: `spec.workloads.nodePlacement` Selector が適用されているか

```
# workloads の NodeSelector 確認
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o
jsonpath='{.spec.workloads.nodePlacement}' | jq

# 確認
oc get nodes --selector=<label-from-workloads-nodeSelector>
```

確認事項

確認: HyperConverged CR が Infrastructure Pod をインストールしているか



## HyperConverged CR 確認:

```
# HyperConverged 確認
oc get hyperconverged kubevirt-hyperconverged -n openshift-cnv -o yaml | grep -A 10 status:

# 確認
oc get events -n openshift-cnv --sort-by='.lastTimestamp' | grep HyperConverged
```

確認:

```
# 確認 reconciliation 確認
oc annotate hyperconverged kubevirt-hyperconverged -n openshift-cnv reconcile-trigger="$(date +%s)" --overwrite
```

## 確認

1. 確認: virt-controller/virt-api CDI CPU (4 vCPU 8GB RAM) 2
2. Control plane 2
3. VM Virtualization Control plane
4. Node/PodAffinity
5. Infrastructure Pod
6. /
- 7.

## 確認

(Worker Node) 確認

```
# HyperConverged CR nodePlacement 確認
oc patch hyperconverged kubevirt-hyperconverged -n openshift-cnv --type=json -p='[{"op": "remove", "path": "/spec/infra/nodePlacement"}, {"op": "remove", "path": "/spec/workloads/nodePlacement"}]'
```

```
# 確認
oc label node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra-
oc label node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra-
```

```
# Taint 確認
oc adm taint node ip-10-0-24-89.ec2.internal node-role.kubernetes.io/infra:NoSchedule-
```

```
oc adm taint node ip-10-0-43-58.ec2.internal node-role.kubernetes.io/infra:NoSchedule-
```

□ □ □

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- OpenShift Virtualization
- HyperConverged CR
- `spec.infra.nodePlacement` `spec.workloads.nodePlacement`
- Infrastructure Pod `NodeSelector``Affinity``Toleration`
- 
- Infrastructure

11

- VM 0000000000000000
- VM 0000000000
- 00000000000000000000000000000000

## □□□□□□ Affinity

11

OpenShift      Kubernetes      VM  
 VM      VM

| OpenShift    | Virtualization | VM                             |
|--------------|----------------|--------------------------------|
| NodeSelector | NodeAffinity   | PodAffinity/Anti-Affinity rule |

□ □ □ □

- `nodeSelector` `required`
- `NodeAffinity` rule (`required`) `preferred` `preferred`
- VM `PodAffinity/Anti-Affinity` `preferred`
- `nodeSelector` `required`
- `nodeSelector` `preferred`

□ □ □ □

- OpenShift 4.18 需要 OpenShift Virtualization 容器虚拟化支持
- 容器虚拟化支持需要 KVM 支持
- `oc` 需要 `virtctl` CLI 容器虚拟化支持

- 指定された Worker Node へスケジューリング (デフォルトではランダム)

デフォルトではランダム

OpenShift Virtualization 上で VM をスケジューリングする 3 つの方法

| 設定                        | 説明                                     | 効果           |
|---------------------------|----------------------------------------|--------------|
| NodeSelector              | 特定の VM へスケジューリング                       | ランダム         |
| NodeAffinity              | 強 (required) / 弱 (preferred) のスケジューリング | ランダム         |
| PodAffinity/Anti-Affinity | 特定の VM へ Pod をスケジューリング                 | VM をスケジューリング |



VM をスケジューリングする VM をスケジューリングする **virt-launcher** Pod をスケジューリングする Kubernetes をスケジューリングする Pod をスケジューリングする

デフォルトではランダム

Selector と Affinity rule を指定してスケジューリング

デフォルトではランダム

```
oc get nodes --show-labels
```

例:

```
NAME                                STATUS    ROLES    AGE   VERSION   LABELS
ip-10-0-8-25.ec2.internal           Ready    control-plane,master   23d   v1.31.14
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/hostname=ip-10-0-8-25.ec2.internal,...
ip-10-0-24-89.ec2.internal           Ready    worker    22d   v1.31.14
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/hostname=ip-10-0-24-89.ec2.internal,...
ip-10-0-43-58.ec2.internal           Ready    worker    22d   v1.31.14
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/hostname=ip-10-0-43-58.ec2.internal,...
ip-10-0-82-27.ec2.internal           Ready    worker    22d   v1.31.14
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/hostname=ip-10-0-82-27.ec2.internal,...
```

デフォルトではランダム

強 (required) のスケジューリング (デフォルトではランダム)

```
# Worker Node へスケジューリング
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}')
WORKER2=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[1].metadata.name}')
```

```

WORKER3=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[2].metadata.name}')

# 
oc label node $WORKER1 env=development
oc label node $WORKER2 env=production

# GPU 
oc label node $WORKER1 gpu=nvidia-t4
oc label node $WORKER2 gpu=nvidia-a100

# 
oc label node $WORKER1 storage=ssd
oc label node $WORKER3 storage=nvme

# 
oc label node $WORKER1 workload=database

```

~~~~~

```

WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}')
oc get node $WORKER1 -o jsonpath='{.metadata.labels}' | jq

```

{}:

```

{
  "beta.kubernetes.io/arch": "amd64",
  "beta.kubernetes.io/os": "linux",
  "env": "development",
  "gpu": "nvidia-t4",
  "kubernetes.io/arch": "amd64",
  "kubernetes.io/hostname": "ip-10-0-24-89.ec2.internal",
  "kubernetes.io/os": "linux",
  "node-role.kubernetes.io/worker": "",
  "storage": "ssd",
  "workload": "database"
}

```

~~~~~

```

WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}')
oc label node $WORKER1 gpu-

```

## NodeSelector

NodeSelector を指定することで VM が特定のノードにのみ実行されます

namespace を指定します

namespace を作成します

```
oc create namespace vm-placement
```

NodeSelector を指定します

`storage=ssd` ラベルを持つノードにのみ VM を実行します



cloud-init をインストールし、SSH を有効にします

[Download vm-node-selector.yaml](#)

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: database-vm
  namespace: vm-placement
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        app: database
    spec:
      nodeSelector:
        storage: ssd # VM will only run on nodes with storage=ssd label
      domain:
        cpu:
          cores: 2
        memory:
          guest: 4Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          volumes:
            - name: rootdisk
              containerDisk:
                image: quay.io/containerdisks/fedora:43
```

```
- name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd: { expire: False }
```

VM を起動する

```
oc apply -f vm-node-selector.yaml
virtctl start database-vm -n vm-placement
```

確認する

VM のステータスを確認する

```
oc get vmi database-vm -n vm-placement -o wide
```

結果:

| NAME        | AGE | PHASE   | IP           | NODENAME | READY |
|-------------|-----|---------|--------------|----------|-------|
| database-vm | 45s | Running | 10.128.0.200 | ocplab02 | True  |

ストレージタイプを確認する

```
oc get vmi database-vm -n vm-placement -o jsonpath='{.status.nodeName}' | xargs oc get node -L storage
```

結果:

| NAME                       | STATUS | ROLES  | AGE | VERSION | STORAGE |
|----------------------------|--------|--------|-----|---------|---------|
| ip-10-0-24-89.ec2.internal | Ready  | worker | 30d | v1.30.0 | ssd     |

NodeSelector を設定する

NodeSelector を設定する (NodeSelector を設定する)

```
spec:
  template:
    spec:
      nodeSelector:
        storage: ssd
        env: production
```

```
gpu: nvidia-t4
```

VM 3

## NodeAffinity

NodeAffinity (required) (preferred)

### NodeAffinity (required)

required Affinity

[Download vm-node-affinity-required.yaml](#)

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: gpu-workload-vm
  namespace: vm-placement
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        app: ml-training
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: gpu
                    operator: In
                    values:
                      - nvidia-t4
                      - nvidia-a100
              - key: env
                operator: In
                values:
                  - production
      domain:
        cpu:
          cores: 4
        memory:
          guest: 8Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
```

```

    - name: cloudinitdisk
      disk:
        bus: virtio
volumes:
- name: rootdisk
  containerDisk:
    image: quay.io/containerdisks/fedora:43
- name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      user: fedora
      password: fedora
      chpasswd: { expire: False }

```

선택:

- **requiredDuringSchedulingIgnoredDuringExecution:** VM 스케줄링 시 반드시 만족해야 하는 조건
- **matchExpressions:** In, NotIn, Exists, DoesNotExist, Gt, Lt
- **nodeSelectorTerms:** OR (조건들 중 하나라도 만족하면 OK)
- 1 개의 matchExpressions AND (모든 조건을 만족해야 함)

**NodeAffinity (preferred)**

preferred Affinity (선택) 스케줄러가 가능한 노드를 우선적으로 선택

[Download vm-node-affinity-preferred.yaml](#)

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: web-vm
  namespace: vm-placement
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        app: webserver
    spec:
      affinity:
        nodeAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              preference:
                matchExpressions:
                  - key: storage
                    operator: In
                    values:

```





```

      - key: env
        operator: In
        values:
      - production
    preferredDuringSchedulingIgnoredDuringExecution:
  - weight: 100
    preference:
      matchExpressions:
      - key: storage
        operator: In
        values:
      - nvme

```

VM NVMe

## PodAffinity Anti-Affinity

PodAffinity Anti-Affinity VM Pod VM

**PodAffinity: VM**

PodAffinity VM

[Download vm-pod-affinity.yaml](#)

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: web-frontend-vm
  namespace: vm-placement
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        app: web-frontend
        tier: frontend
    spec:
      affinity:
        podAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
              - database
            topologyKey: kubernetes.io/hostname
      domain:
        cpu:

```

```

    cores: 2
  memory:
    guest: 2Gi
  devices:
    disks:
      - name: rootdisk
        disk:
          bus: virtio
      - name: cloudinitdisk
        disk:
          bus: virtio
  volumes:
    - name: rootdisk
      containerDisk:
        image: quay.io/containerdisks/fedora:43
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          user: fedora
          password: fedora
          chpasswd: { expire: False }

```

VM app=database Pod/VM (topologyKey: kubernetes.io/hostname)

Pod/VM:

| Pod/VM                        | Pod/VM |
|-------------------------------|--------|
| kubernetes.io/hostname        | Pod/VM |
| topology.kubernetes.io/zone   | Pod/VM |
| topology.kubernetes.io/region | Pod/VM |

PodAnti-Affinity: VM

PodAnti-Affinity VM

Download vm-pod-anti-affinity.yaml

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: cache-vm-01
  namespace: vm-placement
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:

```

```

app: redis-cache
role: cache
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - redis-cache
          topologyKey: kubernetes.io/hostname
  domain:
    cpu:
      cores: 2
    memory:
      guest: 4Gi
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
  volumes:
    - name: rootdisk
      containerDisk:
        image: quay.io/containerdisks/fedora:43
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          user: fedora
          password: fedora
          chpasswd: { expire: False }

```

VM

preferred PodAnti-Affinity

```
spec:
  template:
    spec:
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
```

```

- weight: 100
  podAffinityTerm:
    labelSelector:
      matchExpressions:
        - key: app
          operator: In
          values:
            - redis-cache
    topologyKey: kubernetes.io/hostname

```

### □□□ Affinity rule □□□□□□

```

spec:
  template:
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: env
                    operator: In
                    values:
                      - production
        podAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchExpressions:
                    - key: tier
                      operator: In
                      values:
                        - database
                topologyKey: kubernetes.io/hostname
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - web-frontend
              topologyKey: kubernetes.io/hostname

```

□□ VM □□□□□□□□□□□□:

- □□□□□□□□□□□□□□□□□□□□
- □□□□□□□□ VM □□□□□□□□□□□□□□□□

- **Web** **VM**

**1:**

**VM** **:**

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: postgres-vm-01
  labels:
    app: postgres-cluster
    instance: "01"
spec:
  template:
    metadata:
      labels:
        app: postgres-cluster
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchLabels:
                  app: postgres-cluster
              topologyKey: kubernetes.io/hostname
```

**VM**

**2: GPU**

**GPU** **:**

```
spec:
  template:
    spec:
      nodeSelector:
        gpu: "true"
      tolerations:
        - key: "nvidia.com/gpu"
          operator: "Exists"
          effect: "NoSchedule"
```

**NodeSelector** **Taints/Toleration** **GPU** **GPU**

### Box 3: Pod configuration

Pod configuration:

```
# Pod VM
spec:
  template:
    metadata:
      labels:
        app: trading-platform
        tier: backend
    spec:
      affinity:
        podAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: tier
                    operator: In
                    values:
                      - cache
                      - database
              topologyKey: kubernetes.io/hostname
```

### Box 4: Node

Node VM configuration:

```
spec:
  template:
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: env
                    operator: In
                    values:
                      - development
```

## Web configuration

1. **Virtualization** → **VirtualMachines** configuration
2. VM configuration
3. **Configuration** configuration
4. **Scheduling** configuration

5. `NodeSelector` `Affinity` `Toleration` `nodeName`

6. **Overview** `VM` `nodeName` `nodeName`

`nodeName`

**VM** `(Pending)` `nodeName`

`oc`: `VM` `nodeName`

`oc`:

```
# VM nodeName
oc get vm -n vm-placement

# VMI nodeName
oc describe vmi <vm-name> -n vm-placement
```

`Warning`:

```
Warning FailedScheduling 0/3 nodes are available: 3 node(s) didn't match Pod's node
affinity/selector
```

`Warning`:

- `NodeSelector` `required Affinity rule` `nodeName`
- `nodeName` (CPU/`nodeName`)
- `Taint` `nodeName`

`oc`:

```
# nodeName
oc get nodes -L storage,env,gpu

# nodeName
oc get nodes --selector=storage=ssd

# VM nodeName
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 20 "affinity:"
```

`Warning`:

- `nodeName`
- `nodeName`
- `required Affinity` `preferred Affinity` `nodeName`



## VM Affinity Configuration

Goal: VM with preferred Affinity to specific node

Goal: preferred Affinity to specific node:

- Node selection
- Affinity configuration
- VM placement

Goal:

```
# VM Affinity configuration
oc get vmi <vm-name> -n vm-placement -o jsonpath='{.status.nodeName}'

# Node selection
oc get node <node-name> --show-labels
```

Goal:

- Node selection preferred Affinity to required Affinity
- Affinity configuration
- VM placement

## Anti-Affinity Configuration

Goal: Anti-Affinity to specific VM

Goal:

```
# Anti-Affinity configuration
oc get vmi -n vm-placement -o wide

# Anti-Affinity configuration
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 15 "podAntiAffinity:"
```

Goal:

- Anti-Affinity configuration Selector to VM
- Affinity configuration
- (preferred) vs (required): preferred
- Anti-Affinity configuration

Goal:

Goal:

```
# VM 確認
oc get vm <vm-name> -n vm-placement -o jsonpath='{.spec.template.metadata.labels}' |
jq

# Anti-Affinity 確認
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 5 "labelSelector:"
```

## NodeSelector 確認

VM NodeSelector 確認

## NodeSelector 確認

```
# VM spec 確認
oc get vm <vm-name> -n vm-placement -o yaml | grep -A 5 "nodeSelector:"

# spec.template.spec (spec 確認)
oc get vm <vm-name> -n vm-placement -o jsonpath='{.spec.template.spec.nodeSelector}' |
jq
```

YAML nodeSelector 確認:

```
# VM spec nodeSelector
spec:
  nodeSelector:    # This is ignored!
  storage: ssd
  template:
    spec:
      domain:
        ...

# Pod spec nodeSelector
spec:
  template:
    spec:
      nodeSelector: # This works!
      storage: ssd
      domain:
        ...
```

確認

確認

```
# VM 削除
oc delete vm --all -n vm-placement
```

```
# 3개의 Worker Node 생성
WORKER1=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[0].metadata.name}')
WORKER2=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[1].metadata.name}')
WORKER3=$(oc get nodes -l node-role.kubernetes.io/worker -o
jsonpath='{.items[2].metadata.name}')

oc label node $WORKER1 storage- env- gpu- workload-
oc label node $WORKER2 storage- env- gpu-
oc label node $WORKER3 storage-
```

확인:

```
oc get vm -n vm-placement
oc get nodes --show-labels | grep -E "storage|workload"
```

3.3

고유성 요구사항:

- NodeSelector, NodeAffinity, PodAffinity/Anti-Affinity
- VM 고유의 고유성 요구사항
- NodeSelector
- (required) (preferred) NodeAffinity
- VM PodAffinity/Anti-Affinity
- HA
- 

3.4

- [VM](#) - VM
- [Virtctl](#) - VM CLI
- [Red Hat](#)
- [Pod](#) - Kubernetes
- [Taint & Toleration](#) - Kubernetes

고유성 요구사항

3.5

고유성 요구사항

`virtctl addvolume` 添付する PVC を VM にマウントする OS 依存の操作は `virtctl removevolume` によって行われる

## 添付する VM

添付する VM は、以下の条件を満たす必要がある

- 添付する VM は、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる
- 添付する VM は、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる
- 添付する VM は、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる
- 添付する VM は、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる
- 添付する VM は、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる



`--persist`

添付する VM は、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる `--persist` オプションは、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる

## 添付する VM

添付する VM は、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる VM に 2 つ `virtio` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる 3 つ添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる

BEFORE – hotplug-vm running

```
|
├─ rootdisk      – vda  (virtio)
└─ cloudinitdisk – vdb  (virtio)
```

`virtctl addvolume hotplug-vm --volume-name=hotplug-disk --persist`  
the VM keeps running – no restart, no downtime

AFTER – the same hotplug-vm, still running

```
|
├─ rootdisk      – vda  (virtio)
├─ cloudinitdisk – vdb  (virtio)
└─ hotplug-disk  – sda  (SCSI, hotplugged)
```

添付する VM は、添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる `virtio` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる `PCI` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる `virtio` `PCI` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる `SCSI` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる `virtio` `PCI` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる

添付する `virtctl` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる `PVC` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる `Pod` 添付する VM の OS 依存の操作は `virtctl addvolume` によって行われる

PersistentVolumeClaim (hotplug-disk)

```
|
├─ virtctl addvolume – patches the running VM
```

```

├─ attachment pod – makes the PVC available to the VM's launcher pod
├─ /dev/sda in the guest – a new block device, ready to format

```

Pod 和 VMI 都使用 `hotplugVolume` 属性来指定要使用的存储卷 4

需要

- **OpenShift 4.18** 或更高 (OpenShift Virtualization Operator 已安装)
- 具有 `system:admin` 权限的用户
- **oc CLI** 或 **virtctl CLI**
- **default StorageClass** 已配置

要求:

OpenShift 4.20, 4.21

## 第 1 步: namespace 和 VM

namespace 和 VM 的定义

```
oc create namespace hotplug-demo
```



确保 cloud-init 已安装并配置为在 VM 启动时通过 SSH 连接。

`vm-hotplug.yaml` 文件

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: hotplug-vm
  namespace: hotplug-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: hotplug-vm
    spec:
      domain:
        devices:
          disks:
            - name: rootdisk

```

```

    disk:
      bus: virtio
    - name: cloudinitdisk
    disk:
      bus: virtio
  interfaces:
    - name: default
      masquerade: {}
  resources:
    requests:
      memory: 1Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      containerDisk:
        image: quay.io/containerdisks/fedora:latest
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          password: redhat
          chpasswd:
            expire: false

```

EOF

VM 실행 확인

```
oc get vm -n hotplug-demo -w
```

STATUS=Running READY=True 실행 확인 Ctrl+C 실행 중지

실행 확인

```
oc get vmi hotplug-vm -n hotplug-demo -o jsonpath='{range
.status.volumeStatus[*]}{.name}: {.target}{"\n"}{end}'
```

결과:

```
rootdisk: vda
cloudinitdisk: vdb
```

2: PVC 생성

VM 실행 PersistentVolumeClaim 생성

## hotplug-pvc.yaml

```
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: hotplug-disk
  namespace: hotplug-demo
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: lvms-vg1
  volumeMode: Block
EOF
```



volumeMode: Block Filesystem PVC disk.img

PVC WaitForFirstConsumer binding mode Pending

```
oc get pvc hotplug-disk -n hotplug-demo
```

### 3:

virtctl addvolume VM PVC

```
virtctl addvolume hotplug-vm --volume-name=hotplug-disk --namespace=hotplug-demo
--persist
```

:

Successfully submitted addvolume request to VM hotplug-vm for volume hotplug-disk

--persist VM

### 4:

VMI

```
oc get vmi hotplug-vm -n hotplug-demo -o jsonpath='{range
.status.volumeStatus[*]}{.name}: {.target} (hotplugVolume:
```

```
{.hotplugVolume}}{"\n"}{end}'
```

□□□□□□:

```
rootdisk: vda (hotplugVolume: )
cloudinitdisk: vdb (hotplugVolume: )
hotplug-disk: sda (hotplugVolume: {"attachPodName":"...", "attachPodUID":"..."})
```

□□□□□□□□□□□□□□□□ Ready □ SCSI □□□□ (sda) □□□□□□□□□□hotplugVolume □□□□□□□□□□ Pod  
□□□□□□□□□□□□□□□□□□□□□□

PVC □□□□□□□□□□□□□□□□

```
oc get pvc hotplug-disk -n hotplug-demo
```

□□□□□□:

| NAME         | STATUS | VOLUME   | CAPACITY | ACCESS MODES | STORAGECLASS | AGE |
|--------------|--------|----------|----------|--------------|--------------|-----|
| hotplug-disk | Bound  | pvc-xxxx | 5Gi      | RWO          | lvms-vg1     | 1m  |

□□□ OS □□□□□□□□□□VM □□□□□□□□□□□□

```
virtctl console hotplug-vm -n hotplug-demo
```

□□□□ fedora □□□□□□ redhat □□□□□□□□□□□□□□□□□□□□□□

```
lsblk
```

PVC □□□□□□□□□□□□□□□□ (□: sda) □□□□□□□□□□Ctrl+] □□□□□□□□□□

□□ 5: □□□□□□□□□□□□□□□□□□□□□□ (□□□□□□)

VM □□□□□□□□□□□□□□□□□□□□□□□□

```
sudo mkfs.xfs /dev/sda
```

□□□□□□□□□□□□□□□□□□□□□□

```
sudo mkdir -p /mnt/hotplug
sudo mount /dev/sda /mnt/hotplug
```

□□□□□□□□□□□□□□□□□□□□





## OpenShift Container Platform

### --persist

| Option                 | Description   |
|------------------------|---------------|
| <code>--persist</code> | VM spec       |
| (Optional)             | VM (Optional) |

OpenShift Container Platform uses `--persist` to persist VM spec and VM.

OpenShift Container Platform

OpenShift Container Platform:

- PersistentVolumeClaims (PVCs)
- DataVolumes

OpenShift Container Platform:

- containerDisk
- cloudInitNoCloud / cloudInitConfigDrive
- emptyDir

OpenShift Container Platform

OpenShift Container Platform uses SCSI (sda/sdb) and virtio PCI.

OpenShift Container Platform (Optional)

OpenShift

Virtualization

OpenShift Container Platform uses Virtualization.

OpenShift

- **Multus CNI thick plugin:** Multus thick plugin
- **Multus Dynamic Networks Controller:** Multus Dynamic Networks Controller
- **Multi-node cluster:** Single Node Cluster

OpenShift

OpenShift Container Platform (virtctl addvolume) VM spec

1. NetworkAttachmentDefinition (NAD)
2. VM spec
3. VM (thick Multus)

□ □ □ □ □ □ □ □ □ □

VM spec XXXXXXXXXX state: absent XXXXXXX

```
interfaces:
- name: dyniface1
  state: absent
  bridge: {}
```



bridge SR-IOV

[KubeVirt Hotplug Network Interfaces documentation](#)

□ □ □ □ □ □ □

[illegible]

```
oc delete namespace hotplug-demo
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- `virtctl addvolume` VM へ PVC を追加する
- `--persist` 永続的に保存する
- 追加したボリュームは OS 上で `virtio-blk` として認識される
- VM 内で `lsblk` を実行すると確認できる
- `virtctl removevolume` VM から PVC を削除する
- 削除したボリュームは OS 上で `virtio-blk` として認識されなくなる

11

- [Hotplug Volumes - KubeVirt User Guide](#)
- [Hotplug Network Interfaces - KubeVirt User Guide](#)
- [Annotations](#)

□ □ □ □ □ □ □ □ □ □

11

```

VirtualMachines
VirtualMachine

```

- 仮想マシン (vCPU と RAM 含む)

- **Processor (CPU architecture)**
- **Memory**
- **Storage**
- **Labels and Annotations**

OpenShift Virtualization Machine spec

VirtualMachine OpenShift Virtualization rhel9-server-large fedora-desktop-medium VM

VirtualMachine

OpenShift 4.21

- 
- **1:**
- **2:**
- **3:**
- **4:**
- 
- 

- **bash** (zsh)
- OpenShift **oc**
- OpenShift Virtualization OpenShift
- (IDE)

**1:**

**1**

openshift project templates

```
oc get templates -n openshift
```

OpenShift Virtualization  
OpenShift (OS T) OpenShift Virtualization

rhel OpenShift Virtualization (OpenShift Virtualization) OpenShift Virtualization

```
...
rhel8-desktop-tiny          Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-large Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-medium Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-small Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-highperformance-tiny  Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-large          Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-medium        Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-small         Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel8-server-tiny          Template for Red Hat Enterprise Linux 8
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-large        Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-medium       Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-small        Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-desktop-tiny         Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-highperformance-large Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-highperformance-medium Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-highperformance-small Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-highperformance-tiny Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-server-large         Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-server-medium        Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-server-small         Template for Red Hat Enterprise Linux 9
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
rhel9-server-tiny          Template for Red Hat Enterprise Linux 9
```

```
VM or newer. A PVC with the RHEL disk... 4 (2 generated) 1
...
```

`openshift` project `rhel9-server-medium` `openshift` (create `-o yaml`)

```
oc get template rhel9-server-medium -n openshift -o yaml > rhel9-server.template.yaml
```

`rhel9-server.template.yaml` (creationTimestamp uid resourceVersion)

```
apiVersion: template.openshift.io/v1
kind: Template
metadata:
  annotations:
    defaults.template.kubevirt.io/disk: rootdisk
    description: Template for Red Hat Enterprise Linux 9 VM or newer. A PVC with the
      RHEL disk image must be available.
    iconClass: icon-rhel
    name.os.template.kubevirt.io/rhel9.0: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.1: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.2: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.3: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.4: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.5: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.6: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.7: Red Hat Enterprise Linux 9.0 or higher
    openshift.io/display-name: Red Hat Enterprise Linux 9 VM
    openshift.io/documentation-url: https://github.com/kubevirt/common-templates
    openshift.io/provider-display-name: Red Hat
    openshift.io/support-url: https://github.com/kubevirt/common-templates/issues
    openshift.kubevirt.io/pronounceable-suffix-for-name-expression: "true"
    operator-sdk/primary-resource: openshift-cnv/ssp-kubevirt-hyperconverged
    operator-sdk/primary-resource-type: SSP.ssp.kubevirt.io
    tags: hidden,kubevirt,virtualmachine,linux,rhel
    template.kubevirt.io/containerdisks: |
      registry.redhat.io/rhel9/rhel-guest-image
    template.kubevirt.io/editable: |
      /objects[0].spec.template.spec.domain.cpu.sockets
      /objects[0].spec.template.spec.domain.cpu.cores
      /objects[0].spec.template.spec.domain.cpu.threads
      /objects[0].spec.template.spec.domain.memory.guest
      /objects[0].spec.template.spec.domain.devices.disks
      /objects[0].spec.template.spec.volumes
      /objects[0].spec.template.spec.networks
    template.kubevirt.io/images: |
      https://access.redhat.com/downloads/content/479/ver=/rhel---
      9/9.0/x86_64/product-software
    template.kubevirt.io/provider: Red Hat
    template.kubevirt.io/provider-support-level: Full
    template.kubevirt.io/provider-url: https://www.redhat.com
```

```

template.kubevirt.io/version: v1alpha1
template.openshift.io/bindable: "false"
labels:
  app.kubernetes.io/component: templating
  app.kubernetes.io/managed-by: ssp-operator
  app.kubernetes.io/name: common-templates
  app.kubernetes.io/part-of: hyperconverged-cluster
  app.kubernetes.io/version: 4.21.1
  flavor.template.kubevirt.io/medium: "true"
  os.template.kubevirt.io/rhel9.0: "true"
  os.template.kubevirt.io/rhel9.1: "true"
  os.template.kubevirt.io/rhel9.2: "true"
  os.template.kubevirt.io/rhel9.3: "true"
  os.template.kubevirt.io/rhel9.4: "true"
  os.template.kubevirt.io/rhel9.5: "true"
  os.template.kubevirt.io/rhel9.6: "true"
  os.template.kubevirt.io/rhel9.7: "true"
  template.kubevirt.io/architecture: amd64
  template.kubevirt.io/type: base
  template.kubevirt.io/version: v0.34.1
  workload.template.kubevirt.io/server: "true"
name: rhel9-server-medium
namespace: openshift
objects:
- apiVersion: kubevirt.io/v1
  kind: VirtualMachine
  metadata:
    annotations:
      vm.kubevirt.io/validations: |
        [
          {
            "name": "minimal-required-memory",
            "path": "jsonpath::.spec.domain.memory.guest",
            "rule": "integer",
            "message": "This VM requires more memory.",
            "min": 1610612736
          }
        ]
    labels:
      app: ${NAME}
      kubevirt.io/dynamic-credentials-support: "true"
      vm.kubevirt.io/template: rhel9-server-medium
      vm.kubevirt.io/template.revision: "1"
      vm.kubevirt.io/template.version: v0.34.1
  name: ${NAME}
spec:
  dataVolumeTemplates:
  - apiVersion: cdi.kubevirt.io/v1beta1
    kind: DataVolume
    metadata:
      name: ${NAME}

```

```

spec:
  sourceRef:
    kind: DataSource
    name: ${DATA_SOURCE_NAME}
    namespace: ${DATA_SOURCE_NAMESPACE}
  storage:
    resources:
      requests:
        storage: 30Gi
runStrategy: Halted
template:
  metadata:
    annotations:
      vm.kubevirt.io/flavor: medium
      vm.kubevirt.io/os: rhel9
      vm.kubevirt.io/workload: server
    labels:
      kubevirt.io/domain: ${NAME}
      kubevirt.io/size: medium
  spec:
    architecture: amd64
    domain:
      cpu:
        cores: 1
        sockets: 1
        threads: 1
      devices:
        disks:
          - disk:
              bus: virtio
              name: rootdisk
          - disk:
              bus: virtio
              name: cloudinitdisk
        interfaces:
          - masquerade: {}
            model: virtio
            name: default
          rng: {}
        features:
          smm:
            enabled: true
        firmware:
          bootloader:
            efi: {}
        memory:
          guest: 4Gi
        networks:
          - name: default
            pod: {}
      terminationGracePeriodSeconds: 180

```



```

volumes:
- dataVolume:
  name: ${NAME}
  name: rootdisk
- cloudInitNoCloud:
  userData: |-
    #cloud-config
    user: cloud-user
    password: ${CLOUD_USER_PASSWORD}
    chpasswd: { expire: False }
  name: cloudinitdisk

parameters:
- description: VM name
  from: rhel9-[a-z0-9]{16}
  generate: expression
  name: NAME
- description: Name of the DataSource to clone
  name: DATA_SOURCE_NAME
  value: rhel9
- description: Namespace of the DataSource
  name: DATA_SOURCE_NAMESPACE
  value: openshift-virtualization-os-images
- description: Randomized password for the cloud-init user cloud-user
  from: '[a-z0-9]{4}-[a-z0-9]{4}-[a-z0-9]{4}'
  generate: expression
  name: CLOUD_USER_PASSWORD

```

parameters parameter field value description

- **NAME** - VM ([a-z0-9]{16})
- **DATA\_SOURCE\_NAME** - VM DataSource (rhel9)
- **DATA\_SOURCE\_NAMESPACE** - DataSource namespace openshift-virtualization-os-images project
- **CLOUD\_USER\_PASSWORD** - cloud-user NAME (4)



- 
- bash

rhel9-server.template.yaml oc process

```
oc process -f rhel9-server.template.yaml -o yaml
```

このセクションでは、`yaml` ファイルを使用して `List` オブジェクトを定義し、`VirtualMachine` オブジェクトを定義します。



- OpenShift のインストール1 オブジェクトを定義し、`Kubernetes` オブジェクトを定義します。
- OpenShift Virtualization オブジェクトを定義し、`VirtualMachine` オブジェクトを定義します。

ステップ 2: 準備

## ステップ 2: `VirtualMachine` オブジェクト

このセクションでは、CLI を使用して `VirtualMachine` オブジェクトを定義し、`oc process` コマンドを使用して `oc create` (または `oc apply`) を実行します。

この RHEL 9 `VirtualMachine` オブジェクトは `project` (namespace) オブジェクト

```
oc new-project template-example
```

この `rhel9-server.template.yaml` ファイルを使用して `oc apply` コマンドを実行し、(この `VirtualMachine` オブジェクトを `oc create` コマンドで) 作成します。

```
oc process -f rhel9-server.template.yaml | oc create -n template-example -f -
```

RHEL 9 オブジェクトを定義する namespace オブジェクト

```
oc get all -n template-example
```

`DataVolume` は `PendingPopulation` ステータスで `VirtualMachine` は `Stopped` ステータスで

この `rhel9-server` VM オブジェクトを定義します。

```
export RHEL9_SERVER_VM=$(oc get vm -o custom-columns=:metadata.name --no-headers -n template-example)
```

この VM オブジェクトは `rhel9-abcdef0987654321` という名前を付与します。

```
echo $RHEL9_SERVER_VM
```

`virtctl` コマンドを使用して VM オブジェクトを定義し、(この VM オブジェクトを) 作成します。

```
virtctl start $RHEL9_SERVER_VM
```

`oc get vm` コマンド (または `-w` オプション) を使用して VM オブジェクトのステータスを `Stopped` から `Provisioning` に変更し、`Running` ステータスに到達します。

VM 00000000000000000000000000000000 0000 4 0000000000

**□□□□ 3:** □□□□□□□□□□□□□□

```

##### rhel9 #####

```

## 2 project/namespace

oc project template-example

rhel9-server.template.yaml    rhel9-custom.template.yaml

```
cp rhel9-{server,custom}.template.yaml
```

```
#####rhel9-custom.template.yaml#####
```

```

apiVersion: template.openshift.io/v1
kind: Template
metadata:
  annotations:
    defaults.template.kubevirt.io/disk: rootdisk
    description: Template for Red Hat Enterprise Linux 9 VM or newer. A PVC with the
      RHEL disk image must be available.
    iconClass: icon-rhel
    name.os.template.kubevirt.io/rhel9.0: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.1: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.2: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.3: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.4: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.5: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.6: Red Hat Enterprise Linux 9.0 or higher
    name.os.template.kubevirt.io/rhel9.7: Red Hat Enterprise Linux 9.0 or higher
    openshift.io/display-name: Red Hat Enterprise Linux 9 VM
    openshift.io/documentation-url: https://github.com/kubevirt/common-templates
    openshift.io/provider-display-name: Red Hat
    openshift.io/support-url: https://github.com/kubevirt/common-templates/issues
    openshift.kubevirt.io/pronounceable-suffix-for-name-expression: "true"
    operator-sdk/primary-resource: openshift-cnv/ssp-kubevirt-hyperconverged
    operator-sdk/primary-resource-type: SSP.ssp.kubevirt.io
    tags: hidden,kubevirt,virtualmachine,linux,rhel
    template.kubevirt.io/containerdisks: |
      registry.redhat.io/rhel9/rhel-guest-image
    template.kubevirt.io/editable: |
      /objects[0].spec.template.spec.domain.cpu.sockets
      /objects[0].spec.template.spec.domain.cpu.cores
      /objects[0].spec.template.spec.domain.cpu.threads
      /objects[0].spec.template.spec.domain.memory.guest
      /objects[0].spec.template.spec.domain.devices.disks

```

```

    /objects[0].spec.template.spec.volumes
    /objects[0].spec.template.spec.networks
  template.kubevirt.io/images: |
    https://access.redhat.com/downloads/content/479/ver=/rhel---
9/9.0/x86_64/product-software
  template.kubevirt.io/provider: Red Hat
  template.kubevirt.io/provider-support-level: Full
  template.kubevirt.io/provider-url: https://www.redhat.com
  template.kubevirt.io/version: v1alpha1
  template.openshift.io/bindable: "false"
labels:
  app.kubernetes.io/component: templating
  app.kubernetes.io/managed-by: ssp-operator
  app.kubernetes.io/name: common-templates
  app.kubernetes.io/part-of: hyperconverged-cluster
  app.kubernetes.io/version: 4.21.1
  flavor.template.kubevirt.io/medium: "true"
  os.template.kubevirt.io/rhel9.0: "true"
  os.template.kubevirt.io/rhel9.1: "true"
  os.template.kubevirt.io/rhel9.2: "true"
  os.template.kubevirt.io/rhel9.3: "true"
  os.template.kubevirt.io/rhel9.4: "true"
  os.template.kubevirt.io/rhel9.5: "true"
  os.template.kubevirt.io/rhel9.6: "true"
  os.template.kubevirt.io/rhel9.7: "true"
  template.kubevirt.io/architecture: amd64
  template.kubevirt.io/type: base
  template.kubevirt.io/version: v0.34.1
  workload.template.kubevirt.io/server: "true"
name: rhel9-custom
namespace: openshift
objects:
- apiVersion: kubevirt.io/v1
  kind: VirtualMachine
  metadata:
    annotations:
      vm.kubevirt.io/validations: |
        [
          {
            "name": "minimal-required-memory",
            "path": "jsonpath::.spec.domain.memory.guest",
            "rule": "integer",
            "message": "This VM requires more memory.",
            "min": 1610612736
          }
        ]
    labels:
      app: ${NAME}
      kubevirt.io/dynamic-credentials-support: "true"
      vm.kubevirt.io/template: rhel9-server-medium
      vm.kubevirt.io/template.revision: "1"

```

```

    vm.kubevirt.io/template.version: v0.34.1
  name: ${NAME}
  spec:
    dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: ${NAME}
      spec:
        sourceRef:
          kind: DataSource
          name: ${DATA_SOURCE_NAME}
          namespace: ${DATA_SOURCE_NAMESPACE}
        storage:
          resources:
            requests:
              storage: 30Gi
    runStrategy: Always
  template:
    metadata:
      annotations:
        vm.kubevirt.io/flavor: medium
        vm.kubevirt.io/os: rhel9
        vm.kubevirt.io/workload: server
      labels:
        kubevirt.io/domain: ${NAME}
        kubevirt.io/size: medium
    spec:
      architecture: amd64
      domain:
        cpu:
          cores: 1
          sockets: 1
          threads: 1
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              model: virtio
              name: default
          rng: {}
        features:
          smm:
            enabled: true
        firmware:

```

```

    bootloader:
      efi: {}
    memory:
      guest: 4Gi
    networks:
      - name: default
        pod: {}
    terminationGracePeriodSeconds: 180
    volumes:
      - dataVolume:
          name: ${NAME}
          name: rootdisk
      - cloudInitNoCloud:
          userData: |-
            #cloud-config
            user: ${CLOUD_INIT_USERNAME}
            password: ${CLOUD_INIT_PASSWORD}
            chpasswd: { expire: False }
          name: cloudinitdisk
  parameters:
    - description: VM name
      from: rhel9-custom-[a-z0-9]{6}
      generate: expression
      name: NAME
    - description: Name of the DataSource to clone
      name: DATA_SOURCE_NAME
      value: rhel9
    - description: Namespace of the DataSource
      name: DATA_SOURCE_NAMESPACE
      value: openshift-virtualization-os-images
    - description: Name of the user account created by cloud-init
      name: CLOUD_INIT_USERNAME
      value: shadowman
    - description: Randomized password for the cloud-init user cloud-user
      from: '[a-z0-9]{4}-[a-z0-9]{4}-[a-z0-9]{4}'
      generate: expression
      name: CLOUD_INIT_PASSWORD

```

□□□□□□□□ rhel9-server-medium □□□□□□□□□□□□□□□□

- **metadata.name** - □□□□□□□□ rhel9-custom □□□□□□□□
- **objects[0].spec.runStrategy** - □□□□□□□□□□□□□□ Halted □□ Always □□□□□□□□
- **NAME** - NAME □□□□□□□□□□ rhel9-custom- □□□□□□□□□□□□□□□□□□□□
- **CLOUD\_INIT\_USERNAME** - □□□ □□□ cloud-user □□□□□□□□□□ cloud-init  
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- **CLOUD\_INIT\_PASSWORD** - cloud-user □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□



□□□□□□□□□□□□ yaml □□□ rhel9-custom.template.yaml □□□□□□□□□□□□□□□□

oc process 4

```
oc process -f rhel9-custom.template.yaml -o yaml
```

4

4:

2

rhel9-custom VirtualMachine

```
oc process -f rhel9-custom.template.yaml | oc apply -n template-example -f -
```

rhel9-custom VM oc get vm

```
oc get vm -n template-example
```

rhel9-custom VM

```
export RHEL9_CUSTOM_VM=$(oc get vm -o custom-columns=:metadata.name --no-headers -n template-example | grep custom)
```



rhel9 VM \$RHEL9\_CUSTOM\_VM RHEL9\_SERVER\_VM

VM yml cloud-init

```
oc describe vm $RHEL9_CUSTOM_VM | grep password
```

virtctl (VM username VM shadowman cloud-user)

```
virtctl console $RHEL9_CUSTOM_VM
```

ENTER CTRL+]



- OpenShift Virtualization openshift project

kube context namespace (default OpenShift project namespace)

```
oc project default
```

namespace namespace

```
oc delete namespace template-example
```

- 
- OpenShift Virtualization **VirtualMachine**
- 
- **VirtualMachine**
- **VirtualMachine**

## Instancetype Preference

**VirtualMachine** (CPU) OS (CPU VM YAML VM)

Instancetype Preference

- **instancetype** (CPU GPU)
- **preference** OS (BIOS UEFI) CPU Windows Hyper-V enlightenments

VM 30 domain spec OpenShift Virtualization instancetype preference

- instancetype preference
- namespace
- instancetype preference
- instancetype preference VM



- `instancetype/preference` VM domain spec VM
- `instancetype/preference` domain spec
- `instancetype` preference

- OpenShift Virtualization Operator OpenShift 4.18+
- `oc` CLI `virtctl` CLI
- `StorageClass`
- `openshift-virtualization-os-images` namespace Fedora RHEL (DataSource)

```
oc get datasources -n openshift-virtualization-os-images
```

|  | VM |  | CPU | GPU |
|--|----|--|-----|-----|
|  |    |  |     |     |

OpenShift

Virtualization

<letter><generation>.<size>

Examples:

- u1.medium – General Purpose, generation 1, medium size
- cx1.xlarge – Compute Exclusive, generation 1, extra-large size
- m1.2xlarge – Memory Intensive, generation 1, 2x extra-large size

- **U (General Purpose)** — CPU 1:4 CPU RAM
- **O (Overcommitted)** — CPU RAM 1:4 VM
- **CX (Compute Exclusive)** — CPU CPU RAM 1:2 CPU
- **M (Memory Intensive)** — CPU RAM 1:8
- **N (Network)** — DPDK CPU RAM 1:2

`nano` `8xlarge` CPU

namespace Instancetype

Instancetype 2

- **VirtualMachineClusterInstancetype** — namespace OpenShift Virtualization Instancetype
- **VirtualMachineInstancetype** — namespace namespace Instancetype

## Preference

Preference instancetype OS VM

- BIOS UEFI SecureBoot
- CPU vCPU ()
- virtio SATA SCSI
- OS
- **Hyper-V enlightenments** — Windows

OpenShift Virtualization OS cluster preferences (fedora rhel9 rhel10 windows.11)

Preference VM

## vs. Namespace Preferences

instancetype preference 2

- **VirtualMachineClusterPreference** — OS Preference
- **VirtualMachinePreference** — namespace OS

## Instancetype Preference

instancetype (preference) (OS) VM

```
VirtualMachine manifest
├── spec.instancetype      → u1.medium (1 vCPU, 4Gi RAM)
├── spec.preference       → fedora (EFI firmware, virtio bus, prefer-cores topology)
└── spec.template.spec    → disks, networks, volumes (your workload-specific config)
```

VM OpenShift Virtualization instancetype preference VirtualMachineInstance domain spec spec VMI

## 1:

```
oc get virtualmachineclusterinstancetypes
```

filter the output with `grep`

```
oc get virtualmachineclusterinstancetypes | grep '^u1\.'
```

the output (U) indicates the instance type

the output shows the instance type

```
oc get virtualmachineclusterinstancetype u1.medium -o yaml
```

`spec` shows the CPU and memory for `u1.medium` General Purpose instance type 1:4 ratio 1 vCPU and 4Gi memory

## 2: Preference

the output shows the Preference

```
oc get virtualmachineclusterpreferences
```

Fedora Preference shows the OS

```
oc get virtualmachineclusterpreference fedora -o yaml
```

`spec` shows the BIOS (EFI) and CPU (virtio) OS

## 3: Instancetype Preference VM

the output shows the namespace

```
oc new-project instancetype-demo
```

the output shows `u1.medium` instancetype `fedora` preference Fedora VM domain spec `cpu` `memory` `firmware` `machine` instancetype preference

`vm-with-instancetype.yaml` example

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-instancetype-demo
```

```

namespace: instancetype-demo
spec:
  instancetype:
    name: u1.medium
    kind: VirtualMachineClusterInstancetype
  preference:
    name: fedora
    kind: VirtualMachineClusterPreference
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: fedora-instancetype-demo
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-instancetype-demo
  spec:
    domain:
      devices:
        disks:
          - name: rootdisk
            disk:
              bus: virtio
          - name: cloudinitdisk
            disk:
              bus: virtio
        interfaces:
          - name: default
            masquerade: {}
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        dataVolume:
          name: fedora-instancetype-demo
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            user: fedora
            password: changeme123

```

```
chpasswd:
  expire: false
  ssh_pwauth: true
```



cloud-init 設定を適用して SSH を有効にする

確認する

```
oc apply -f vm-with-instancetype.yaml
```

VM が Running 状態にあることを確認する

```
oc get vm fedora-instancetype-demo -n instancetype-demo -w
```

VM が **Running** 状態にあることを **Ctrl+C** で確認する

#### ステップ 4: VM の spec を取得する

VirtualMachine オブジェクトの CPU 設定は VirtualMachineInstance オブジェクトの `instancetype` の `preference` によって VM の spec が決定される。

VM の spec は `domain spec` によって決定される。

```
oc get vmi fedora-instancetype-demo -n instancetype-demo -o yaml
```

spec.domain の `cpu` は `u1.medium` の `instancetype` の `fedora` の `preference` によって決定される。CPU の設定は QEMU/KVM の設定と一致する。

CPU の設定は QEMU/KVM の設定と一致する。

```
oc get vmi fedora-instancetype-demo -n instancetype-demo -o
jsonpath='{.spec.domain.cpu}' | python3 -m json.tool
```

```
oc get vmi fedora-instancetype-demo -n instancetype-demo -o
jsonpath='{.spec.domain.memory}' | python3 -m json.tool
```

instancetype の `preference` は VM の spec を決定する。

#### ステップ 5: Instancetype の Domain Spec を取得する

instancetype の `preference` は domain spec を決定する。VM の spec は domain spec によって決定される。

VM の spec は OS の spec によって決定される。

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-explicit-demo
  namespace: instancetype-demo
spec:
  runStrategy: Always
  dataVolumeTemplates:
    - metadata:
        name: fedora-explicit-demo
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  template:
    metadata:
      labels:
        kubevirt.io/domain: fedora-explicit-demo
    spec:
      domain:
        cpu:
          sockets: 1
          cores: 1
          threads: 1
        memory:
          guest: 4Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
          rng: {}
        features:
          smm:
            enabled: true
        firmware:
          bootloader:
```

```

    efi: {}
  machine:
    type: pc-q35-rhel9.4.0
  resources:
    requests:
      memory: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      dataVolume:
        name: fedora-explicit-demo
    - name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      user: fedora
      password: changeme123
      chpasswd:
        expire: false
      ssh_pwauth: true

```

인스턴스 유형과 선호도를 사용하여 VM을 구성합니다

| 인스턴스 유형과 선호도                                                         | 인스턴스 유형                                                                      |
|----------------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>spec.template.spec.domain.cpu</code> (sockets, cores, threads) | Instancetype ( <code>u1.medium</code> ) + Preference ( <code>fedora</code> ) |
| <code>spec.template.spec.domain.memory.guest</code>                  | Instancetype ( <code>u1.medium</code> )                                      |
| <code>spec.template.spec.domain.resources.requests.memory</code>     | Instancetype ( <code>u1.medium</code> )                                      |
| <code>spec.template.spec.domain.features.smm</code>                  | Preference ( <code>fedora</code> )                                           |
| <code>spec.template.spec.domain.firmware.bootloader</code>           | Preference ( <code>fedora</code> )                                           |
| <code>spec.template.spec.domain.machine.type</code>                  | Preference ( <code>fedora</code> )                                           |
| <code>spec.template.spec.domain.devices.rng</code>                   | Preference ( <code>fedora</code> )                                           |

인스턴스 유형 `u1.medium` VM을 생성합니다. 이 VM은 `u1.medium` 인스턴스 유형과 `fedora` 선호도를 사용합니다.



`specinstancetype` 인스턴스 유형과 `spec.template.spec.domain.cpu` / `spec.template.spec.domain.memory` 인스턴스 유형과 `specinstancetype` / `specinstancetype` VM을 생성합니다.

## 인스턴스 유형 6: Instancetype과 Preference를 사용하여 VM을 구성합니다

인스턴스 유형과 선호도를 사용하여 VM을 구성합니다. 이 VM은 `u1.medium` 인스턴스 유형과 `fedora` 선호도를 사용합니다.

- **CPU** — VM NUMA `dedicatedCPUPlacement: true` CPU `instancetype`
- **Hugepages** — Hugepage
- — VM `instancetype` CPU
- `preference` BIOS/UEFI VM `preference`
- **VM** — ( ) VM `spec`
- — VM `instancetype` `domain spec`

`domain spec` VM `domain spec`

## 7: **instancetype**

`instancetype` VM `instancetype`

4 vCPU 8Gi namespace `instancetype`

**custom-instancetype.yaml**

```
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachineInstancetype
metadata:
  name: app-server-4c8g
  namespace: instancetype-demo
spec:
  cpu:
    guest: 4
  memory:
    guest: 8Gi
```

`instancetype`

```
oc apply -f custom-instancetype.yaml
```

```
oc get virtualmachineinstancetypes -n instancetype-demo
```



## 例 8: 例 Preference 例

EFI (SecureBoot 例) 例virtio 例vCPU 例 Linux 例 namespace 例 Preference 例

**custom-preference.yaml** 例

```
apiVersion:instancetype.kubevirt.io/v1beta1
kind: VirtualMachinePreference
metadata:
  name: my-linux-server
  namespace:instancetype-demo
spec:
  cpu:
    preferredCPUTopology: cores
  firmware:
    preferredUseEfi: true
    preferredUseSecureBoot: false
  devices:
    preferredDiskBus: virtio
    preferredInterfaceModel: virtio
```

例 preference 例:

```
oc apply -f custom-preference.yaml
```

例:

```
oc get virtualmachinepreferences -ninstancetype-demo
```

## 例 9: 例 Instancetype 例 Preference 例 VM 例

例instancetype 例 preference 例 VM 例:

**vm-with-custom-instancetype.yaml** 例

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: custom-instancetype-demo
  namespace:instancetype-demo
spec:
  instancetype:
    name: app-server-4c8g
    kind: VirtualMachineInstancetype
  preference:
    name: my-linux-server
```

```

kind: VirtualMachinePreference
runStrategy: Always
dataVolumeTemplates:
- metadata:
  name: custom-instancetype-demo
spec:
  sourceRef:
    kind: DataSource
    name: fedora
    namespace: openshift-virtualization-os-images
  storage:
    resources:
      requests:
        storage: 30Gi
template:
  metadata:
    labels:
      kubevirt.io/domain: custom-instancetype-demo
spec:
  domain:
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
      interfaces:
        - name: default
          masquerade: {}
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      dataVolume:
        name: custom-instancetype-demo
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |
          #cloud-config
          user: fedora
          password: changeme123
          chpasswd:
            expire: false
          ssh_pwauth: true

```

□□□□□□□□□□□□

```
oc apply -f vm-with-custom-instancetype.yaml
```

VM `instancetype-demo`

```
oc get vm custom-instancetype-demo -n instancetype-demo -w
```

VM `instancetype-demo` `Running` `Ctrl+C` to stop

VMI `instancetype-demo` `instancetype-demo`

```
oc get vmi custom-instancetype-demo -n instancetype-demo -o
jsonpath='{.spec.domain.cpu.cores}' && echo " cores"
```

```
oc get vmi custom-instancetype-demo -n instancetype-demo -o
jsonpath='{.spec.domain.memory.guest}' && echo ""
```

`instancetype-demo` `instancetype-demo` `4 cores` `8Gi`

`instancetype-demo`

`instancetype-demo` namespace `instancetype-demo`

```
oc delete project instancetype-demo
```

`instancetype-demo`

`instancetype-demo`

- `instancetype` `instancetype-demo` (CPU) `instancetype-demo` `Preference` `instancetype-demo` `OS` `instancetype-demo` (instancetype-demo CPU) `instancetype-demo`
- `OpenShift Virtualization` `instancetype-demo` `instancetype` (instancetype-demo) `instancetype-demo` `preference` (OS) `instancetype-demo`
- `oc get` `instancetype-demo` `instancetype` `preference` `instancetype-demo` `-o yaml` `instancetype-demo`
- `VM` `instancetype-demo` `instancetype` `preference` `instancetype-demo` `domain` `spec` `instancetype-demo`
- `specinstancetype` `spec.template.spec.domain.cpu / memory` `instancetype-demo`
- `VirtualMachineInstance` `instancetype-demo` `domain spec` `instancetype-demo`
- `domain spec` `instancetype-demo` `CPU` `instancetype-demo` `HugePages` `instancetype-demo`
- `instancetype-demo` namespace `instancetype-demo` `instancetype` `preference` `instancetype-demo`

11

- [API](#) [インスタンスタイプ](#) — CPU、メモリ、ディスク容量などのリソースを指定するinstancetype。この preference を使用して API を呼び出す。
- [コマンドライン](#) — インスタンスタイプの VM を指定する instancetype の preference を使用する。
- [YAML](#) [コンフィグファイル](#) CLI [VM](#) [タイプ](#) — YAML で定義した VM のインスタンスを作成する。
- [インストールされた VM ツール - インストール](#)
- [KubeVirt Instancetypes](#) [インストール](#)

# VM ทรัพยากรและการจัดการ

## ภาพรวม

OpenShift Virtualization อนุญาตให้ผู้ใช้สามารถรัน VM บน Kubernetes ได้ โดยไม่ต้องพึ่งพา Hypervisor ภายนอก CPU และ NUMA ทรัพยากรจะถูกจัดการโดย Kubernetes

## การตั้งค่า

- กำหนด CPU/หน่วย (requests) และ (limits) QoS
- กำหนด vCPU และ CPU (pinning)
- Hugepage และ NUMA
- การตั้งค่าอื่นๆ

## การติดตั้ง

VM ทรัพยากรและการจัดการ

- OpenShift 4.18+ ใช้ OpenShift Virtualization Operator
- Kubernetes
- CLI (oc, virtctl, kubectl)
- CPU Manager (CPU Manager hugepages)

## การตั้งค่า

### Resource limits และ QoS

CPU/หน่วย (requests) และ (limits) QoS namespace ResourceQuota และ LimitRange

### CPU

Kubernetes CPU Manager static vCPU และ CPU (pinning) CPU steal

### NUMA และ Hugepages

Hugepage NUMA TLB NUMA

### VM

CPU NUMA Hugepages RT (priority) SCHED\_FIFO

### VM ทรัพยากรและการจัดการ

OpenShift Prometheus Grafana VM QEMU guest agent OS

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

OpenShift 4.20

OpenShift 4.20 (CPU architecture) — OpenShift 4.20

HPC / OpenShift

OpenShift 4.20 (NUMA + Hugepages) — OpenShift 4.20

OpenShift / OpenShift / OpenShift

4 OpenShift — OpenShift

OpenShift VM

OpenShift 1 OpenShift — OpenShift Kubernetes OpenShift

OpenShift

- [VM Configuration](#) - OpenShift
- [Storage Configuration](#) - OpenShift

OpenShift

- [OpenShift Virtualization](#) OpenShift
- [KubeVirt](#) OpenShift

## Resource limits & QoS

OpenShift

OpenShift Virtualization OpenShift CPU requests & limits & QoS  
OpenShift Virtualization OpenShift VM OpenShift

KubeVirt OpenShift VM spec requests & limits OpenShift Kubernetes QoS class OpenShift VM  
OpenShift ResourceQuota & LimitRange OpenShift namespace OpenShift

OpenShift

- **OpenShift 4.18** OpenShift (OpenShift Virtualization Operator OpenShift)
- OpenShift (system:admin OpenShift)
- **oc CLI** OpenShift **virtctl CLI** OpenShift
- **default StorageClass** OpenShift (VM OpenShift)

OpenShift:

OpenShift 4.20, 4.21

## VM ทรัพยากร

OpenShift Virtualization 的 VM 使用 KubeVirt 的 VM 的 spec 的 VM 的 virt-launcher 的 requests 和 limits 的

### CPU 的

- 的 vCPU 的  $1/\text{cpuAllocationRatio}$  的 CPU 的 request 的 ( $\text{cpuAllocationRatio}$  的 10) 的 1 vCPU 的 VM 的 100m 的 CPU 的
- 的 CPU limits 的 VM 的 request 的
- VM 的 CPU limits 的 CPU request 的 request 和 limits 的
- vCPU 的  $\text{sockets} * \text{cores} * \text{threads}$  的 1 的

### 的

- VM 的  $\text{spec.domain.memory.guest}$  的  $\text{spec.domain.resources.requests.memory}$  的
- KubeVirt 的 (QEMU 的) 的 Memory request 的
- 的 Memory limits 的
- Memory limits 的 500 MiB 的 2% 的



VM 的 Memory limits 的 OS 的 limits 的 VM 的 limits 的

## 步骤 1: 创建 namespace 的

```
oc create namespace resource-qos-demo
```

## 步骤 2: 创建 QoS class 的 VM 的

Kubernetes 的 Pod 的 3 的 QoS class 的 1 的 VM 的 Pod 的

### Burstable QoS (的)

的 requests 的 limits 的 VM 的 Burstable QoS 的 OpenShift Virtualization VM 的



的 cloud-init 的 SSH 的

vm-burstable.yaml 的

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
```



```

metadata:
  name: vm-burstable
  namespace: resource-qos-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-burstable
    spec:
      domain:
        memory:
          guest: 512Mi
        resources:
          requests:
            memory: 512Mi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        networks:
          - name: default
            pod: {}
        volumes:
          - name: rootdisk
            containerDisk:
              image: quay.io/containerdisks/fedora:latest
          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |
                #cloud-config
                password: redhat
                chpasswd:
                  expire: false
EOF

```

## Guaranteed QoS

CPU requests limits VM Guaranteed QoS VM  
 (evict)

**vm-guaranteed.yaml**

```

oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-guaranteed
  namespace: resource-qos-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-guaranteed
    spec:
      domain:
        memory:
          guest: 512Mi
        resources:
          requests:
            memory: 1Gi
            cpu: "1"
          limits:
            memory: 1Gi
            cpu: "1"
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        networks:
          - name: default
            pod: {}
        volumes:
          - name: rootdisk
            containerDisk:
              image: quay.io/containerdisks/fedora:latest
          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |
                #cloud-config
                password: redhat
                chpasswd:
                  expire: false
EOF

```

EOF



Memory limits (1Gi) (512Mi)  
 limits Pod

## BestEffort QoS

requests limits VM BestEffort QoS VM (evict)

**vm-besteffect.yaml**

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-besteffect
  namespace: resource-qos-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-besteffect
    spec:
      domain:
        memory:
          guest: 512Mi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        networks:
          - name: default
            pod: {}
        volumes:
          - name: rootdisk
            containerDisk:
              image: quay.io/containerdisks/fedora:latest
          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |
                #cloud-config
                password: redhat
                chpasswd:
```

```
expire: false
EOF
```

### 3: QoS Class

VM

```
oc get vm -n resource-qos-demo
```

| NAME          | AGE | STATUS  | READY |
|---------------|-----|---------|-------|
| vm-besteffort | 30s | Running | True  |
| vm-burstable  | 60s | Running | True  |
| vm-guaranteed | 45s | Running | True  |

VM Pod QoS class

Burstable VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-burstable -o
jsonpath='{.items[0].status.qosClass}'
```

Burstable

Guaranteed VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-guaranteed -o
jsonpath='{.items[0].status.qosClass}'
```

Guaranteed

BestEffort VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-besteffort -o
jsonpath='{.items[0].status.qosClass}'
```

## Burstable



**virt-launcher** VMs are created with KubeVirt **Resource requests** of **vm-burstable** which is **BestEffort** or **Burstable** depending on the configuration.

**virt-launcher** Pod is created with the following configuration:

Burstable VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-burstable -o yaml
```

Guaranteed VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-guaranteed -o yaml
```

BestEffort VM

```
oc get pod -n resource-qos-demo -l kubevirt.io/domain=vm-besteffort -o yaml
```

## 4: ResourceQuota

ResourceQuota is used to limit the number of VMs or Pods in a namespace. It can be used to limit the number of VMs or Pods in a namespace. It can be used to limit the number of VMs or Pods in a namespace.

**resource-quota.yaml** is the configuration file.

```
oc apply -f - <<EOF
apiVersion: v1
kind: ResourceQuota
metadata:
  name: vm-resource-quota
  namespace: resource-qos-demo
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "10"
EOF
```

Quota is used to

```
oc describe resourcequota vm-resource-quota -n resource-qos-demo
```

namespace limits namespace VM quota limits  
CPU limits (1 vCPU 1) Memory limits (requests 2) limits

**Memory Limits Ratio**

Memory limits requests 2 namespace

```
oc label namespace resource-qos-demo alpha.kubevirt.io/auto-memory-limits-ratio=1.5
```

namespace VM Memory limits ratio 1.5

## 5: LimitRange

LimitRange namespace Pod VM

**limit-range.yaml**

```
oc apply -f - <<EOF
apiVersion: v1
kind: LimitRange
metadata:
  name: vm-limit-range
  namespace: resource-qos-demo
spec:
  limits:
  - type: Pod
    max:
      cpu: "4"
      memory: 8Gi
    min:
      cpu: 100m
      memory: 256Mi
  - type: Container
    default:
      cpu: 500m
      memory: 1Gi
    defaultRequest:
      cpu: 100m
      memory: 512Mi
    max:
      cpu: "4"
      memory: 8Gi
    min:
      cpu: 50m
      memory: 128Mi
```

□ □ □ □ □ □

```
oc describe limitrange vm-limit-range -n resource-qos-demo
```

LimitRange defines Resource requests & limits for Pod (VM or virt-launcher Pod) in the namespace

**Figure 6:**

VM Pod □□□□□□□□□□□□□□□□

```
oc adm top pods -n resource-qos-demo
```

|  | virt-launcher | Pod | CPU | requests | limits |
|--|---------------|-----|-----|----------|--------|
|  |               |     |     |          |        |

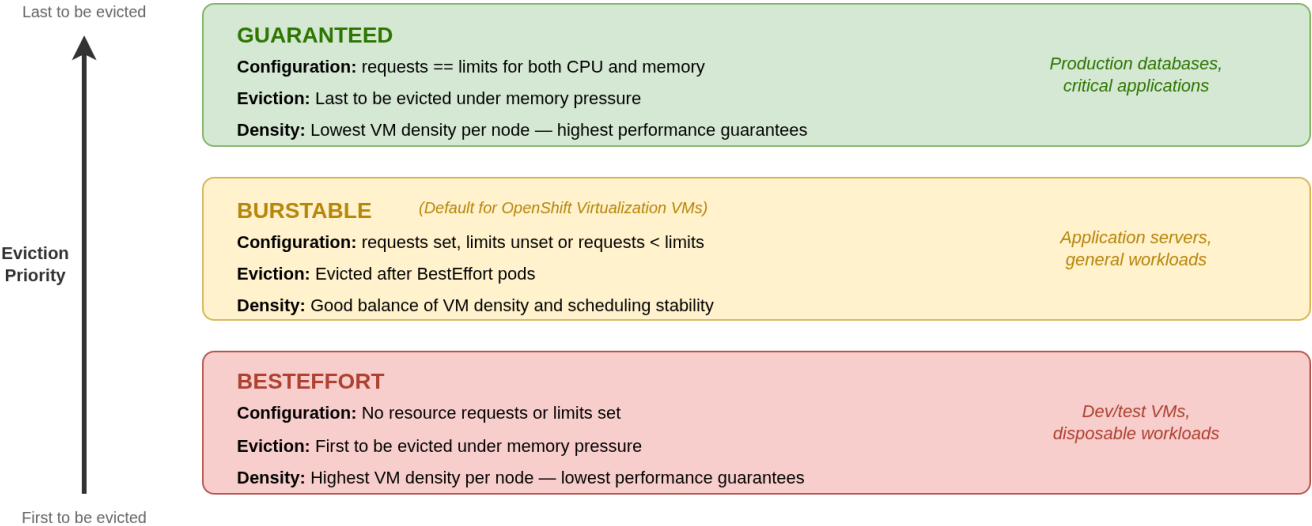
## Quota namespace

```
oc describe resourcequota vm-resource-quota -n resource-qos-demo
```

**QoS** □□□□□

Kubernetes (memory pressure) (eviction) VM 3 QoS

Kubernetes QoS Tiers for OpenShift Virtualization VMs



How KubeVirt determines QoS class:



When ResourceQuota with limits is present:  
autoMemoryLimitsRatio (default 1.0023)  
automatically sets memory limits

Kubernetes `virt-launcher` Pod requests limits QoS KubeVirt VM `spec.domain` CPU requests limits QoS limits ResourceQuota namespace KubeVirt `autoMemoryLimitsRatio` (1.0023 `alpha.kubevirt.io/auto-memory-limits-ratio` namespace) limits

QoS class

| QoS class  |                        | Eviction Priority  |    |
|------------|------------------------|--------------------|----|
| Guaranteed | CPU requests == limits | Low (evict)        |    |
| Burstable  | requests < limits      | BestEffort (evict) | VM |
| BestEffort | requests > limits      | High (evict)       |    |

QoS KubeVirt VM namespace QoS



| ワークロード                             | QoS 種類             | リソース                       | 動作                                              |
|------------------------------------|--------------------|----------------------------|-------------------------------------------------|
| データベース (PostgreSQL、Oracle、MongoDB) | Guaranteed         | CPU requests と limits が等しい | 他の VM より優先的にリソースを確保 (evict) されない VM             |
| Web/API                            | Burstable          | requests が limits より小さい    | 他の VM より優先的にリソースを確保 (evict) されない BestEffort Pod |
| 一般的な VM                            | BestEffort         | requests が limits より小さい    | 他の VM より優先的にリソースを確保 (evict) されない VM             |
| ETL (Extract, Transform, Load)     | Burstable (limits) | requests が limits より小さい    | 他の VM より優先的にリソースを確保 (evict) されない VM             |



QoS 種類: **Guaranteed** (優先度が高い) | **Burstable** (優先度が低い) | **BestEffort** (優先度が最も低い)

リソース: **oc** (Operator) | **adm** (Administrator) | **top** (Top Level) | **Pods** (Pods)

## リソース制限

- Burstable** 種類: Memory request が limits より小さい VM は他の VM より優先的にリソースを確保 (evict) されない
- Guaranteed** 種類: requests と limits が等しい VM は他の VM より優先的にリソースを確保 (evict) されない
- Memory limits** 種類: requests が limits より小さい VM は他の VM より優先的にリソースを確保 (evict) されない
- limits** 種類: requests が limits より小さい VM は他の VM より優先的にリソースを確保 (evict) されない
- namespace** 種類: **ResourceQuota** 種類: requests が limits より小さい VM は他の VM より優先的にリソースを確保 (evict) されない
- LimitRange** 種類: requests が limits より小さい VM は他の VM より優先的にリソースを確保 (evict) されない

## リソース制限

リソース制限の種類

```
oc delete namespace resource-qos-demo
```

## リソース

リソースの種類

- VM の CPU requests と limits

- KubeVirt `VM` `ResourceRequests` `limits` `QoS` class `VM` `Pod`
- `Guaranteed``Burstable``BestEffort` QoS class `VM` `Pod`
- `ResourceQuota` `namespace` `ResourceRequests` `limits`
- `LimitRange` `Pod` `ResourceRequests` `limits`
- `VM` `ResourceRequests` `limits`
- `namespace` `Memory limits ratio` `Pod`

□□

- [KubeVirt Resource Requests and Limits \(KubeVirt `Pod`\)](#)
- [Working with Resource Quotas for VMs - OpenShift `Pod`](#)
- [Configure QoS for Pods - Kubernetes `Pod`](#)
- [Resource Quotas Per Project - OpenShift `Pod`](#)

## □□ CPU □□

□□

OpenShift Virtualization `Pod` `CPU` `ResourceRequests` `limits` `vCPU` `Pod` `CPU` `ResourceRequests` `limits` `vCPU` `CPU` `pinning` `CPU` `jitter` `Pod`

OpenShift Virtualization `Pod` `CPU` `ResourceRequests` `limits` `vCPU` `CPU` `pinning` `CPU` `jitter` `Pod`

□□□□

- OpenShift Virtualization Operator `Pod` OpenShift 4.18 `Pod` (4.21 `Pod`)
- `system:admin` `Pod` `Pod`
- `oc` `Pod` `virtctl` CLI `Pod`
- `CPU` `Worker Node` (`Pod` 4 `Pod`)
- Kubernetes `QoS` `Pod` (`Resource limits` `QoS`)

## □□ CPU □□□□

OpenShift `Pod` Kubernetes `Pod` CPU Manager `Pod` `CPU` `Pod` CPU Manager `Pod` `static` `Pod` `CPU` `Pod` `CPU` `Pod`

KubeVirt `Pod` `dedicatedCpuPlacement` `Pod` CPU Manager `Pod` `virt-launcher` `Pod` `vCPU` `CPU` `Pod` `CPU` `Pod` `vCPU` `CPU` `Pod`

OpenShift Virtualization `Pod` 2 `Pod`

- `CPU` Manager `Pod` `static` `Pod`
- VM `QoS` `Guaranteed` `CPU` `requests` `limits` `CPU` `Pod`

|                                     |                      |              |
|-------------------------------------|----------------------|--------------|
| 物理                                  | vCPU 物理              | 物理           |
| 物理 (shared)                         | vCPU 物理 CPU 物理       | 物理 VM 物理     |
| 物理<br>(dedicatedCpuPlacement: true) | vCPU 物理 CPU 物理CPU 物理 | NFV/物理HPC 物理 |

物理 1: CPU Manager 物理 Static Policy 物理

CPU Manager 物理KubeletConfig 物理 MachineConfigPool 物理

物理Worker 物理 MachineConfigPool 物理

```
oc label machineconfigpool worker custom-kubelet=cpumanager-enabled
```



物理 OpenShift (SNO) 物理 worker 物理 master MachineConfigPool 物理 KubeletConfig 物理 machineConfigPoolSelector 物理 worker 物理 master 物理

kubeletconfig-static-policy.yaml 物理

KubeletConfig 物理Static CPU manager policy 物理

```
oc apply -f - <<EOF
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: cpumanager-enabled
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: cpumanager-enabled
  kubeletConfig:
    cpuManagerPolicy: static
    cpuManagerReconcilePeriod: 5s
EOF
```

Machine Config Operator 物理 kubelet 物理

```
oc get mcp worker -w
```

UPDATING 物理 False 物理READYMACHINECOUNT 物理 MACHINECOUNT 物理

| NAME     | CONFIG | UPDATED | UPDATING |
|----------|--------|---------|----------|
| DEGRADED |        |         |          |

|        |                                                      |      |       |       |
|--------|------------------------------------------------------|------|-------|-------|
| worker | rendered-worker-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx | True | False | False |
|--------|------------------------------------------------------|------|-------|-------|

## CPU Manager Policy

Worker Node kubelet static

```
oc debug node/<node-name> -- chroot /host cat /etc/kubernetes/kubelet.conf
```

```
cpuManagerPolicy: static
cpuManagerReconcilePeriod: 5s
```

## 2: CPU

CPU VM CPU Manager NodeSelector Static Policy  
Worker Node cpumanager=true

```
oc label node <node-name> cpumanager=true
```

```
oc get nodes -L cpumanager
```

| NAME      | STATUS | ROLES  | AGE | VERSION | CPUMANAGER |
|-----------|--------|--------|-----|---------|------------|
| worker-01 | Ready  | worker | 10d | v1.34.5 | true       |
| worker-02 | Ready  | worker | 10d | v1.34.5 | true       |



KubeVirt CR CPUManager KubeVirt cpumanager=true  
KubeVirt Static CPU manager policy  
KubeVirt

## 3: namespace

```
oc create namespace cpu-pinning-demo
```

## 例 4: CPU 専用 VM の作成

この VM spec では `dedicatedCpuPlacement: true` を指定し、2 vCPU の VM を作成する。また、`nodeSelector` で CPU 専用ノードを選択し、`cpumanager=true` を指定して、CPU 専用ノードの `requests` と `limits` を保証する。また、`Guaranteed` QoS を指定する。



cloud-init をインストールし、SSH を有効にする。

### vm-dedicated-cpu.yaml の内容

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-dedicated-cpu
  namespace: cpu-pinning-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-dedicated-cpu
    spec:
      nodeSelector:
        cpumanager: "true"
      domain:
        cpu:
          sockets: 2
          cores: 1
          threads: 1
          dedicatedCpuPlacement: true
        memory:
          guest: 1Gi
        resources:
          requests:
            memory: 2Gi
          limits:
            memory: 2Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        networks:
```

```

- name: default
  pod: {}
volumes:
- name: rootdisk
  containerDisk:
    image: quay.io/containerdisks/fedora:latest
- name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      password: redhat
      chpasswd:
        expire: false

```

EOF



QEMU 2Gi 1Gi KubeVirt VM spec [Resource limits](#) [QoS](#)

VM Running

```
oc get vm vm-dedicated-cpu -n cpu-pinning-demo -w
```

| NAME             | AGE | STATUS  | READY |
|------------------|-----|---------|-------|
| vm-dedicated-cpu | 30s | Running | True  |

## 5: CPU pinning

VM

```
oc get vmi vm-dedicated-cpu -n cpu-pinning-demo -o wide
```

| NAME             | AGE | PHASE   | IP          | NODENAME  | READY |
|------------------|-----|---------|-------------|-----------|-------|
| vm-dedicated-cpu | 60s | Running | 10.128.0.45 | worker-01 | True  |

```
oc debug node/<node-name>
```

このコマンドはCPU Manager state file を表示し、CPU の割り当てを確認します。

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

このJSON は `entries` として `virt-launcher` Pod の UID を表示し、Pod が `compute` ノードに割り当てられている CPU ID を示します。

```
{"policyName":"static","defaultCpuSet":"0-1,3-41,43-79","entries":{"<pod-uid>":{"compute":"2,42"}}, "checksum":...}
```

このコマンドはCPU 2 が 42 の VM に 2 の vCPU を割り当て、`defaultCpuSet` が VM の CPU ID を示します。

このコマンドは

```
exit
```

このコマンドは

VM を起動します。

```
virtctl console vm-dedicated-cpu -n cpu-pinning-demo
```

このコマンドは `fedora` の `redhat` OS を起動し、vCPU を割り当てます。

```
lscpu | grep -E "^\CPU(s\)|^\Socket|^\Core|^\Thread"
```

このコマンドは

```
CPU(s):                2
Thread(s) per core:    1
Core(s) per socket:    1
Socket(s):              2
```

このコマンドは 2 の vCPU (2 のソケット、1 のコア、1 のスレッド) の Topology を表示します。

`Ctrl+]` を押してコンソールを閉じます。

## Emulator Thread

QEMU の Emulator Thread は VM の vCPU の CPU を割り当て、`isolateEmulatorThread` を true に設定して CPU を割り当てます。

Emulator Thread は VM の CPU を割り当て、`dedicatedCpuPlacement: true` を設定します。

```
isolateEmulatorThread: true
```

```
domain:
  cpu:
    sockets: 2
    cores: 1
    threads: 1
    dedicatedCpuPlacement: true
    isolateEmulatorThread: true
```

```

#####KubeVirt   3   CPU   (2   vCPU   1   Emulator Thread )
#####

```



Simultaneous Multithreading (SMT) `full-pcpus-only` CPU Manager option  
`vCPU` `isolateEmulatorThread`  
`KubeVirt` `CR` `AlignCPUs`

□ □ □ □ □ □ □ □ □ □

**VM** **Pending** □□□□□□□□

XXXXXXXXXXXX VMI XXXXXXXXXXXXXXX

```
oc describe vmi vm-dedicated-cpu -n cpu-pinning-demo
```

[illegible]

```
Warning FailedScheduling 0/3 nodes are available: 3 node(s) didn't match Pod's node
affinity/selector
```

□□□□□□□□□□:

- **cpumanager=true** 静态CPU管理器: `oc get nodes -L cpumanager` 查看Static CPU manager policy 是否生效
- 查看CPU Manager 是否生效: `oc describe node <node-name>` 查看CPU Manager 是否生效

## VM CPU pinning

## KubeletConfig

```
oc get kubeletconfig cpumanager-enabled
```

## MachineConfigPool



```
oc get mcp worker
```

UPDATED False DEGRADED True MCO

```
oc logs -n openshift-machine-config-operator -l k8s-app=machine-config-operator
--tail=50
```

**VM "insufficient dedicated CPU resources"**

dedicatedCpuPlacement: true QoS Guaranteed CPU requests limits

virt-launcher Pod QoS

```
oc get pod -n cpu-pinning-demo -l kubevirt.io/domain=vm-dedicated-cpu -o
jsonpath='{.items[0].status.qosClass}'
```

Guaranteed Burstable VM Memory requests Memory limits

cpu\_manager\_state VM

CPU Manager state file virt-launcher Pod CPU kubelet none

```
oc debug node/<node-name> -- chroot /host cat /etc/kubernetes/kubelet.conf
```

cpuManagerPolicy none KubeletConfig MachineConfigPool

```
oc delete namespace cpu-pinning-demo
```

cpumanager=true

```
oc label node <node-name> cpumanager-
```

CPU manager policy KubeletConfig MachineConfigPool

```
oc delete kubeletconfig cpumanager-enabled
```

```
oc label machineconfigpool worker custom-kubelet-
```



**KubeletConfig** 配置 kubelet 与 MCO 配置 CPU Manager 和 MachineConfigPool 配置

□□

□□□□□□□□□□□□□□□□□□□□

- **KubeletConfig** 配置 Kubernetes CPU Manager 的 **static** 配置
- 配置 CPU 配置
- **dedicatedCpuPlacement: true** 配置 Guaranteed QoS 配置
- CPU Manager state file 配置 CPU pinning 配置
- 配置 vCPU Topology 配置
- 配置 **isolateEmulatorThread: true** 配置 QEMU Emulator Thread 配置

□□

- **VM 配置 QoS** — Guaranteed QoS 配置
- **配置** — VM 配置
- **配置** — 配置
- **KubeVirt 配置 CPU 配置 (配置)**
- **CPU Manager 配置 Topology Manager 配置 - OpenShift 配置 (配置)**
- **CPU 配置 - Kubernetes 配置 (配置)**

## NUMA 配置 Hugepages

□□

配置 OpenShift Virtualization 配置 Hugepages 配置 NUMA  
配置 VM 配置 4Ki 配置 OS 配置 NUMA  
配置 NFV/配置 HPC 配置 2  
配置

- **TLB pressure:** 16Gi 配置 4Ki 配置 VM 配置 400 配置 CPU 配置 TLB (Translation Lookaside Buffer) 配置 TLB 配置
- **Cross-NUMA memory latency:** 配置 NUMA 配置 OS 配置 NUMA 配置

Hugepages 4Ki 2Mi 1Gi TLB pressure NUMA (guestMappingPassthrough) NUMA VM NUMA CPU VM

- OpenShift Virtualization Operator OpenShift 4.18 (4.21 )
- (system:admin )
- oc virtctl CLI
- CPU (8 ) Hugepages RAM Worker Node
- CPU Manager static policy (CPU )
- Kubernetes QoS (Resource limits QoS)

Hugepages

Linux 4Ki 2Gi VM 524,288 TLB CPU

Hugepages

|     | 2Gi     | TLB     |    |
|-----|---------|---------|----|
| 4Ki | 524,288 | 524,288 |    |
| 2Mi | 1,024   | 1,024   | VM |
| 1Gi | 2       | 2       | VM |

VM spec memory.hugepages.pageSize KubeVirt Hugepages QEMU (pinned)



Hugepages VM NUMA Hugepages VM NUMA VM CPU NUMA Hugepages NUMA QEMU VM Memory requests Hugepages

NUMA

NUMA (Non-Uniform Memory Access) CPU ( )

2

```
NUMA Node 0: CPUs 0-19, 40-59 Local Memory: 128Gi
NUMA Node 1: CPUs 20-39, 60-79 Local Memory: 128Gi
```

NUMA OS NUMA 0 vCPU NUMA 1

NUMA 노드를 사용하여 CPU와 메모리를 효율적으로 관리합니다.

KubeVirt의 `guestMappingPassthrough` 옵션을 사용하여 NUMA 노드를 가상 머신에 매핑합니다 (가상 머신의 CPU 노드를) NUMA 노드를 사용하여 NUMA 노드를 가상 머신에 매핑합니다

NUMA 노드를 사용하여 3 노드를 사용합니다

1. CPU 노드 (`dedicatedCpuPlacement: true`)
2. Hugepages 설정
3. KubeVirt CR의 NUMA 설정 (OpenShift Virtualization 4.18 버전에서)

## 1: Hugepages 설정

Hugepages 설정을 사용하여 OpenShift의 Node Tuning Operator에서 `Tuned` 설정을 사용하여 Hugepages 설정을 적용합니다

`Tuned` 설정을 사용하여 Worker Node에 2Mi 크기의 1024 노드 (2Gi)를 설정합니다

`hugepages-tuned-boottime.yaml` 파일을 사용합니다

```
oc apply -f - <<EOF
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: hugepages
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
  - data: |
      [main]
      summary=Boot time configuration for hugepages
      include=openshift-node
      [bootloader]
      cmdline_openshift_node_hugepages=hugepagesz=2M hugepages=1024
      name: openshift-node-hugepages
  recommend:
  - machineConfigLabels:
      machineconfiguration.openshift.io/role: "worker"
    priority: 20
    profile: openshift-node-hugepages
EOF
```



Single Node OpenShift (SNO)에서 `machineConfigLabels`를 사용하여 `worker`와 `master` 노드를 구분합니다



`priority` 값을 사용하여 `Tuned` 설정을 `openshift-control-plane`보다 우선시합니다. `openshift-node` 설정의 `priority` 값을 30보다 낮게 설정하여 Hugepages 설정이 먼저 적용됩니다 (기본값: 20)

Node Tuning Operator `hugepagesz=2M hugepages=1024` MachineConfig  
Machine Config Operator Worker Node

```
oc get mcp worker -w
```

UPDATED `True` MACHINECOUNT `READYMACHINECOUNT`

| NAME     | CONFIG                                         | UPDATED | UPDATING |
|----------|------------------------------------------------|---------|----------|
| DEGRADED |                                                |         |          |
| worker   | rendered-worker-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx | True    | False    |



MachineConfigPool

## Hugepages

Hugepages

```
oc get node <node-name> -o jsonpath='{.status.allocatable.hugepages-2Mi}{"\n"}'
```

:

2Gi

/proc/meminfo

```
oc debug node/<node-name>
```

:

```
chroot /host grep -i hugepages /proc/meminfo
```

:

```
AnonHugePages:      0 kB
ShmemHugePages:     0 kB
FileHugePages:      0 kB
HugePages_Total:    1024
HugePages_Free:     1024
HugePages_Rsvd:      0
HugePages_Surp:      0
Hugepagesize:       2048 kB
```

HugePages\_Total (1024) HugePages\_Free

```
exit
```

## 2: NUMA

NUMA VM Worker Node NUMA RHCOS

```
oc debug node/<node-name>
```

NUMA

```
chroot /host ls /sys/devices/system/node/
```

2 :

```
node0 node1
```

NUMA CPU

```
chroot /host cat /sys/devices/system/node/node0/cpulist
```

```
0-9,40-49
```

```
chroot /host cat /sys/devices/system/node/node1/cpulist
```

```
20-29,60-69
```

NUMA

```
chroot /host cat /sys/devices/system/node/node0/meminfo
```

( ) :

```
Node 0 MemTotal:      131072000 kB
Node 0 MemFree:       125000000 kB
Node 0 HugePages_Total:    512
Node 0 HugePages_Free:    512
```

NUMA を確認する CPU-to-node を確認するために `lscpu` を実行する

```
chroot /host lscpu | grep -i numa
```

結果:

```
NUMA node(s):          2
NUMA node0 CPU(s):      0-9,40-49
NUMA node1 CPU(s):      20-29,60-69
```



このデモでは Single Node OpenShift の NUMA ノード 1 を使用しています。NUMA ノード 0 は CPU 0-9,40-49 であり、NUMA ノード 1 は CPU 20-29,60-69 です。

終了

```
exit
```

### ステップ 3: Namespace の作成

```
oc create namespace numa-hugepages-demo
```

### ステップ 4: Hugepages を VM に割り当てる

この VM は NUMA ノード 0 の CPU 0-9,40-49 に Hugepages を割り当て、TLB を VM に割り当てます。

設定:

- `dedicatedCpuPlacement: true` — vCPU が CPU に割り当てられ (KubeVirt の Hugepages を使用) します。
- `memory.hugepages.pageSize: 2Mi` — 2Mi Hugepages を VM に割り当てます。
- `requests` と `limits` を指定 (Guaranteed QoS) します。



この cloud-init は、VM に SSH をインストールするための設定です。

`vm-hugepages.yaml` の内容

```
oc apply -f - <<EOF
```

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-hugepages
  namespace: numa-hugepages-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-hugepages
    spec:
      nodeSelector:
        cpumanager: "true"
      domain:
        cpu:
          sockets: 2
          cores: 1
          threads: 1
          dedicatedCpuPlacement: true
        memory:
          hugepages:
            pageSize: 2Mi
        resources:
          requests:
            memory: 2Gi
          limits:
            memory: 2Gi
        devices:
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        networks:
          - name: default
            pod: {}
        volumes:
          - name: rootdisk
            containerDisk:
              image: quay.io/containerdisks/fedora:latest
          - name: cloudinitdisk
            cloudInitNoCloud:
              userData: |
                #cloud-config
                password: redhat

```



```
chpasswd:
  expire: false
EOF
```

VM is Running successfully

```
oc get vm vm-hugepages -n numa-hugepages-demo -w
```

Output:

| NAME         | AGE | STATUS  | READY |
|--------------|-----|---------|-------|
| vm-hugepages | 45s | Running | True  |

## Step 5: Hugepages Setup

Prerequisites

VM is running successfully

```
oc get vmi vm-hugepages -n numa-hugepages-demo -o wide
```

Output:

```
oc debug node/<node-name>
```

Hugepages configuration HugePages\_Free VM configuration HugePages\_Total

```
chroot /host grep -i hugepages /proc/meminfo
```

2Mi 2Gi VM 1024 HugePages\_Free 1024

Output:

```
exit
```

Prerequisites

VM is running successfully

```
virtctl console vm-hugepages -n numa-hugepages-demo
```

Output fedora redhat

```
free -h
```

確認 RAM、Hugepages、QEMU、OS の Hugepages の状況

**Ctrl+]**  確認

## Hugepages VM

VM Hugepages NUMA VM Hugepages VM

```
virtctl stop vm-hugepages -n numa-hugepages-demo
```

VM

```
oc get vm vm-hugepages -n numa-hugepages-demo
```

結果:

| NAME         | AGE | STATUS  | READY |
|--------------|-----|---------|-------|
| vm-hugepages | 5m  | Stopped | False |

## 6: NUMA VM

VM CPU、Hugepages、NUMA 3 numa.guestMappingPassthrough: {} KubeVirt NUMA

NUMA

- isolateEmulatorThread: true** — QEMU vCPU
- autoattachMemBalloon: false** —

**vm-numa-hugepages.yaml**

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-numa-hugepages
  namespace: numa-hugepages-demo
spec:
  runStrategy: Always
  template:
    metadata:
```

```

labels:
  kubevirt.io/domain: vm-numa-hugepages
spec:
  nodeSelector:
    cpumanager: "true"
  domain:
    cpu:
      sockets: 1
      cores: 2
      threads: 1
      dedicatedCpuPlacement: true
      isolateEmulatorThread: true
      numa:
        guestMappingPassthrough: {}
    memory:
      hugepages:
        pageSize: 2Mi
    resources:
      requests:
        memory: 1Gi
      limits:
        memory: 1Gi
    devices:
      autoattachMemBalloon: false
      disks:
        - name: rootdisk
          disk:
            bus: virtio
        - name: cloudinitdisk
          disk:
            bus: virtio
      interfaces:
        - name: default
          masquerade: {}
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        containerDisk:
          image: quay.io/containerdisks/fedora:latest
      - name: cloudinitdisk
        cloudInitNoCloud:
          userData: |
            #cloud-config
            password: redhat
            chpasswd:
              expire: false

```

EOF

VM is Running successfully

```
oc get vm vm-numa-hugepages -n numa-hugepages-demo -w
```

Output:

| NAME              | AGE | STATUS  | READY |
|-------------------|-----|---------|-------|
| vm-numa-hugepages | 60s | Running | True  |

## Step 7: VM is NUMA enabled

VM is NUMA enabled

VM is NUMA enabled successfully

```
oc get vmi vm-numa-hugepages -n numa-hugepages-demo -o  
jsonpath='{.spec.domain.cpu.numa}{"\n"}'
```

Output:

```
{"guestMappingPassthrough":{}}
```

VM is NUMA enabled

VM is NUMA enabled

```
virtctl console vm-numa-hugepages -n numa-hugepages-demo
```

Output **fedora** is **redhat** is NUMA enabled

```
lscpu | grep -i numa
```

NUMA is enabled CPU is NUMA enabled NUMA is enabled 1  
NUMA is enabled

NUMA CPU is enabled

```
lscpu
```

NUMA is enabled

```
ls /sys/devices/system/node/
```

결과:

```
node0
```

이 NUMA 노드는 여러 CPU를 포함합니다

```
cat /sys/devices/system/node/node0/cpulist
```

결과:

```
0-1
```

노드의 메모리 사용량 확인

```
cat /sys/devices/system/node/node0/meminfo | head -5
```

결과:

```
Node 0 MemTotal:      1048576 kB
Node 0 MemFree:       800000 kB
Node 0 MemUsed:       248576 kB
Node 0 Active:        120000 kB
Node 0 Inactive:      80000 kB
```

이 NUMA 노드는 여러 CPU를 포함합니다

**Ctrl+]** 터미널을 노드로 전환

이 NUMA 노드

CPU Manager는 CPU pinning을 지원합니다

```
oc get vmi vm-numa-hugepages -n numa-hugepages-demo -o wide
```

```
oc debug node/<node-name>
```

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

entries 00000000 CPU ID 000 compute 0000000000000000isolateEmulatorThread: true 0000000000 1 00  
CPU 00000000 (00 3 00vCPU 2 0 + 000000000000 1 0) 0

0000000000000000

```
exit
```

2Mi 0 1Gi Hugepages 000

| 0000               | 2Mi 000                  | 1Gi 000                                                               |
|--------------------|--------------------------|-----------------------------------------------------------------------|
| 00000000 TLB 00000 | 2Mi                      | 1Gi (512 0)                                                           |
| 0000000000         | 000 (0: 100Mi 00)        | 00 (00 1Gi)                                                           |
| 0000000            | 000                      | 0000 1Gi 0000000000VM 00000 1Gi 0000                                  |
| 00000000           | 000000000000000000000000 | 000000000000000000000000Hugepages<br>00000000000000000000000000000000 |
| 000000000000       | 00000 VM0000 TLB 000     | 0000000000000 VM (64Gi 00)000000000000<br>TLB 00                      |

1Gi 000000000000Tuned 00000000000000

```
[bootloader]
cmdline_openshift_node_hugepages=hugepagesz=1G hugepages=16
```

000 VM spec 00000000

```
memory:
  hugepages:
    pageSize: 1Gi
resources:
  requests:
    memory: 16Gi
  limits:
    memory: 16Gi
```



Memory requests 0 Hugepages 0000000000000000000000001Gi 0 Hugepages 0 1.5Gi  
000000000000000000000000

00000000000000

VM 00000000Hugepages 00

VM 0 Scheduling 000 Pending 00000000000000000000VMI 000000000000

```
oc describe vmi vm-hugepages -n numa-hugepages-demo
```

Hugepages `memory` `hugepages` Hugepages `memory`

```
oc get node <node-name> -o jsonpath='{.status.allocatable.hugepages-2Mi}{"\n"}'
```

VM `Memory requests` VM `hugepages` Hugepages `memory`

**NUMA** VM `cannot bind memory to host NUMA nodes`

`virt-launcher` Pod `NUMA` `cannot bind memory to host NUMA nodes`:

- **NUMA** `Hugepages`: Hugepages `memory` VM `CPU` `NUMA` `Hugepages` `memory`
- **Topology Manager** `kubelet` `Topology Manager` `none` `CPU` `NUMA` `Topology Manager` `single-numa-node` `restricted`

**NUMA** `memory`

`1` `NUMA` `memory`:

- VMI spec `guestMappingPassthrough`
- VM `CPU` `NUMA` `CPU` `NUMA` `1`
- `NUMA` `CPU` `VM` `vCPU`

**MachineConfigPool** `Updating`

`Tuned` `MachineConfigPool` `memory`:

```
oc get mcp worker
```

```
oc describe mcp worker
```

`machine-config-daemon` `memory`

```
oc logs -n openshift-machine-config-operator -l k8s-app=machine-config-daemon --tail=50
```

`memory`

VM `Namespace` `memory`

```
oc delete namespace numa-hugepages-demo
```

Hugepages Tuned

```
oc delete tuned hugepages -n openshift-cluster-node-tuning-operator
```



### Tuned

MachineConfig

MachineConfigPool

Hugepages

MachineConfigPool

MachineConfigPool

VM

VM

- Node Tuning Operator OpenShift Hugepages
- Kubernetes API `/proc/meminfo` Hugepages
- `/sys/devices/system/node/` `lscpu` NUMA
- TLB Hugepages VM
- NUMA (guestMappingPassthrough) VM
- NUMA
- 2Mi 1Gi Hugepages

VM

- CPU — CPU Manager vCPU
- VM QoS — Guaranteed QoS
- VM
- KubeVirt NUMA
- KubeVirt Hugepages
- Huge Pages - OpenShift
- NUMA - OpenShift

## VM

VM

OpenShift Virtualization VM 5G RAN (vDU/vCU) (tick-to-trade)

CPU VM VM





## Figure 1: MachineConfigPool

```

MachineConfigPool  PerformanceProfile  RT
CPU  Hugepages

```

```
Worker Node  worker-rt
```

```
oc label node <node-name> node-role.kubernetes.io/worker-rt=
```

```
worker  worker-rt  MachineConfigPool
```

```
oc apply -f - <<EOF
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: worker-rt
  labels:
    machineconfiguration.openshift.io/role: worker-rt
spec:
  machineConfigSelector:
    matchExpressions:
      - key: machineconfiguration.openshift.io/role
        operator: In
        values:
          - worker
          - worker-rt
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/worker-rt: ""
EOF
```

[illegible]

```
oc get mcp worker-rt
```

□□□□□□:

| NAME      | CONFIG | UPDATED | UPDATING | DEGRADED | MACHINECOUNT | READYMACHINECOUNT |
|-----------|--------|---------|----------|----------|--------------|-------------------|
| worker-rt |        | True    | False    | False    | 1            | 1                 |

## 2: PerformanceProfile

[illegible]

- `kernel-rt` `kernel`
- CPU 2-7 `irq` `kernel`

- 4 x CPU 0-1
- 4 x 1GiHugepages
- 

## performance-profile.yaml

```
oc apply -f - <<EOF
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: realtime-workers
spec:
  cpu:
    isolated: "2-7"
    reserved: "0-1"
  hugepages:
    defaultHugepagesSize: 1G
    pages:
      - size: 1G
      count: 4
  realTimeKernel:
    enabled: true
  additionalKernelArgs:
    - nmi_watchdog=0
    - audit=0
    - mce=off
    - processor.max_cstate=1
    - idle=poll
    - intel_idle.max_cstate=0
  nodeSelector:
    node-role.kubernetes.io/worker-rt: ""
  machineConfigPoolSelector:
    machineconfiguration.openshift.io/role: worker-rt
EOF
```



Single Node OpenShift (SNO) MachineConfigPool  
 nodeSelector node-role.kubernetes.io/master: ""  
 machineConfigPoolSelector  
 pools.operator.machineconfiguration.openshift.io/master: ""  
 machineConfigPoolSelector  
 machineConfigSelector MCP metadata.labels VM  
 nodeSelector 1 worker-rt (oc label node <node-name> node-role.kubernetes.io/worker-rt=)



PerformanceProfile API Hugepages IEC (1Gi, 2Mi) SI (1G, 2M)  
 VM Kubernetes (1Gi, 2Mi)

PerformanceProfile **Tuned** MachineConfigCPU Manager  
KubeletConfigMachine Config Operator



PerformanceProfile **RT**  
10-20 MCP UPDATING: True

```
oc get mcp worker-rt -w
```

SNO master

```
oc get mcp master -w
```

UPDATED True READYMACHINECOUNT MACHINECOUNT RT  
10-20 MCP UPDATING: True

| nmi_watchdog=0          | NMI                                |
|-------------------------|------------------------------------|
| audit=0                 |                                    |
| mce=off                 |                                    |
| processor.max_cstate=1  | CPU (C-states > C1)                |
| idle=poll               | CPU (halt)                         |
| intel_idle.max_cstate=0 | Intel C-state processor.max_cstate |



RT CPU

3: RT

MachineConfigPool

```
oc debug node/<node-name>
```

```
chroot /host uname -r
```

000000 rt 0000000000000000

5.14.0-427.55.1.el9\_4.x86\_64+rt

**CPU**

```
chroot /host cat /proc/cmdline | grep -o 'isolcpus=managed_irq,[^ ]*'
```

□□□□□□:

```
isolcpus=managed_irq,2-7
```

□□□□ CPU □ tick-less (nohz) □□□□□□□□□□□□□□□□

```
chroot /host cat /sys/devices/system/cpu/nohz_full
```

📊 (PerformanceProfile 📊 `cpu.isolated` 📊):

**□□□□□ IRQ □□□□□□□□□□□□□□**

```
chroot /host cat /proc/irq/default_smp_affinity
```

managed\_irq    CPU    IRQ    Tuned  
irqbalance

Hugepages 000000000000

```
chroot /host grep -i hugepages /proc/meminfo
```

HugePages\_Total 00000000 (1Gi 00000000 4) 0000000000000000

kubevirt.io/realtime

```
oc get node <node-name> -L kubevirt.io/realtime
```

KubeVirt      `kernel.sched_rt_runtime_us`      `-1`      (RT)      `kernel.sched_rt_runtime_us`

```
oc debug node/<node-name> -- chroot /host sysctl kernel.sched_rt_runtime_us
```

Output:

```
kernel.sched_rt_runtime_us = -1
```

## Step 4: Namespace

```
oc create namespace realtime-demo
```

## Step 5: VM

The VM is based on the Fedora image. The VM is configured with the following settings:

Configuration:

- `cpu.model: host-passthrough` — CPU
- `cpu.dedicatedCpuPlacement: true` — vCPU
- `cpu.isolateEmulatorThread: true` — QEMU
- `cpu.ioThreadsPolicy: auto` — I/O
- `cpu.numa.guestMappingPassthrough: {}` — NUMA
- `cpu.realtime.mask: "0"` — vCPU 0, SCHED\_FIFO, vCPU 1
- `memory.hugepages.pageSize: 1Gi` — 1GiHugepages
- `autoattachMemBalloon: false` —
- `autoattachGraphicsDevice: false` —



cloud-init is required for SSH access.

`vm-realtime.yaml`

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
```

```

name: vm-realtime
namespace: realtime-demo
spec:
  runStrategy: Always
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-realtime
    spec:
      nodeSelector:
        node-role.kubernetes.io/worker-rt: ""
      domain:
        cpu:
          model: host-passthrough
          sockets: 1
          cores: 2
          threads: 1
          dedicatedCpuPlacement: true
          isolateEmulatorThread: true
          ioThreadsPolicy: auto
          numa:
            guestMappingPassthrough: {}
          realtime:
            mask: "0"
        memory:
          hugepages:
            pageSize: 1Gi
        resources:
          requests:
            memory: 2Gi
            cpu: 2
          limits:
            memory: 2Gi
            cpu: 2
        devices:
          autoattachSerialConsole: true
          autoattachMemBalloon: false
          autoattachGraphicsDevice: false
          disks:
            - name: rootdisk
              disk:
                bus: virtio
            - name: cloudinitdisk
              disk:
                bus: virtio
          interfaces:
            - name: default
              masquerade: {}
        networks:
          - name: default
            pod: {}

```

```

terminationGracePeriodSeconds: 0
volumes:
- name: rootdisk
  containerDisk:
    image: quay.io/containerdisks/fedora:latest
- name: cloudinitdisk
  cloudInitNoCloud:
    userData: |
      #cloud-config
      password: redhat
      chpasswd:
        expire: false

```

EOF

VM is Running

```
oc get vm vm-realtime -n realtime-demo -w
```

Output:

| NAME        | AGE | STATUS  | READY |
|-------------|-----|---------|-------|
| vm-realtime | 90s | Running | True  |



VM `kubevirt.io/realtime` has `cpumanager=true` annotation. KubeVirt VM `realtime` has `Scheduling` annotation.

Step 6: CPU pinning

CPU pinning

VM

```
oc get vmi vm-realtime -n realtime-demo -o wide
```

Output:

```
oc debug node/<node-name>
```

CPU Manager

```
chroot /host cat /var/lib/kubelet/cpu_manager_state
```

`entries` has 2 vCPU ID 1 CPU (3) `compute`



CPU (PerformanceProfile 2-7)

## Hugepages

```
chroot /host grep HugePages /proc/meminfo
```

**HugePages\_Free** VM (1GiHugepages 2Gi VM 2 ) **HugePages\_Total**



**guestMappingPassthrough** vCPU NUMA Hugepages  
PerformanceProfile Hugepages NUMA  
VM NUMA  
Hugepages

```
exit
```

## 7: VM

VM

```
virtctl console vm-realtime -n realtime-demo
```

**fedora** **redhat**

```
uname -r
```

(**rt** ) vCPU RT

## CPU

```
lscpu | grep -E "^CPU\(s\)|^Socket|^Core|^Thread"
```

```
CPU(s):                2
Thread(s) per core:    1
Core(s) per socket:    2
Socket(s):              1
```



```
sudo dnf install -y realtime-tests
```

□□ cyclicttest □□□□3 □□□□□□□□□□ 60 □□□□□□□□

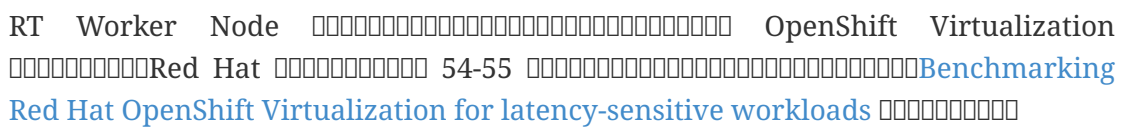
```
sudo cyclicttest --mlockall --smp --priority=80 --interval=200 --distance=0  
--duration=60
```

□□□□□:

```
# /dev/cpu_dma_latency 0us
policy: fifo: loadavg: 0.92 0.42 0.18 1/148 1599

T: 0 ( 1597) P:80 I:200 C: 156868 Min:      3 Act:      5 Avg:   191 Max:   9501
T: 1 ( 1598) P:80 I:200 C: 277750 Min:      3 Act:      7 Avg:    36 Max:  1406
```

```
T: 0 ( 1597) P:80 I:200 C: 156868 Min:      3 Act:    5 Avg:   191 Max:   9501
T: 1 ( 1598) P:80 I:200 C: 277750 Min:      3 Act:    7 Avg:    36 Max:  1406
```

[illegible]

Ctrl+] □□□□□□□□□□□□□□□□

**Figure 9: RT**

`cyclictest` は Fedora の `PREEMPT_DYNAMIC SCHED_FIFO` CPU を PodIRQCPU vCPU に変換して `cyclictest`

[illegible]

|    |                 |           |
|----|-----------------|-----------|
| RT | PREEMPT_DYNAMIC | cycletest |
| RT | PREEMPT_RT      |           |

□□□□□ **RT** □□□□□□□□□□

□□□□ **VM** □□□□□□

```
virtctl console vm-realtime -n realtime-demo
```

**fedora** □□□□□□□ **redhat** □□□□□□□□

**PREEMPT\_RT** □□□□□□□□□□□□ **Fedora Kernel Vanilla COPR** □□□□□□□□□□□□

```
sudo dnf -y copr enable @kernel-vanilla/stable
```

□□□□□□□□□□□□□□□□□□□□

```
sudo dnf -y install kernel-rt-core kernel-rt-modules
```

**RT** □□□□□□□□□□□□□□□□□□□□□□□□

```
sudo grubby --set-default /boot/vmlinuz-*rt*
```

**Ctrl+]** □□□□□□□□□□□□□□□□

**VM** □□□□

**VM** □□□□□ **RT** □□□□□□□□□□□□

```
virtctl restart vm-realtime -n realtime-demo
```

**VM** □ **Running** □□□□□□□□□□□□

```
oc get vm vm-realtime -n realtime-demo -w
```

□□□ **RT** □□□□□□□□

**VM** □□□□□□□□□□□□

```
virtctl console vm-realtime -n realtime-demo
```

□□□□□□□□□□□□□□□□□□□□

```
uname -r
```

6.15.2-0.rc0.20250604gite53b1bae1f73.502.vanilla.fc42.x86\_64+rt

6.15.2-0.rc0.20250604gite53b1bae1f73.502.vanilla.fc42.x86\_64+rt

PREEMPT\_RT

uname -v

PREEMPT\_RT

RT **cyclictest**

VM **realtime-tests** **cyclictest**

sudo cyclictest --mlockall --smp --priority=80 --interval=200 --distance=0 --duration=60

RT 8

Ctrl+]



RT  
RT VM

VM

[kubevirt.io/realtime](https://kubevirt.io/realtime)

oc get nodes -L kubevirt.io/realtime

RT **kernel.sched\_rt\_runtime\_us** -1 **PerformanceProfile**  
**MachineConfigPool**

**PerformanceProfile (Degraded)**

**MachineConfigPool**

oc get mcp worker-rt

oc describe mcp worker-rt



worker-rt

```
oc label node <node-name> node-role.kubernetes.io/worker-rt-
```



PerformanceProfile



MachineConfigPool

worker

MachineConfigPool

- Worker Node MachineConfigPool
- PerformanceProfile RT CPU Hugepages
- RT: CPU Hugepages [kubevirt.io/realtime](https://kubevirt.io/realtime)
- VM
- CPU pinning Hugepages SCHED\_FIFO
- [cyclicttest](#)
- 

- CPU — CPU Manager static policy vCPU
- NUMA Hugepages — Hugepages NUMA
- VM QoS — Guaranteed QoS
- [KubeVirt Real-Time Workloads](#)
- [Tuning Nodes for Low Latency - OpenShift Documentation](#)
- [Provisioning Real-Time Workloads - OpenShift Documentation](#)
- [KVM Real-Time Tuning - libvirt](#)

## VM

OpenShift

Prometheus

Grafana

Web VM

Prometheus Grafana

VM

SLA

(OS agent) OS (QEMU guest agent)

## 必要

- OpenShift Virtualization Operator バージョン4.21 以降の **OpenShift 4.18+**
- OpenShift Container Platform (OpenShift クラウド)
- OpenShift Container Platform の **VM**
- OpenShift の **Web** コンソール
- OpenShift の **oc CLI**
- VM の **virtctl CLI**

## VM のインストール

OpenShift Virtualization のインストールは、2 つのステップで行われます。

### 1. KubeVirt のインストール

KubeVirt は、OS のインストールと KubeVirt のインストールを行います。

- **CPU** のインストール: vCPU (仮想 CPU) のインストール
- **メモリ** のインストール: (仮想) メモリのインストール
- **ディスク** のインストール: ディスクのインストール
- **ネットワーク** のインストール: ネットワーク/ディスク I/O のインストール
- **VM** のインストール: VM のインストール

### 2. OS のインストール

VM の QEMU guest agent をインストールします。

- **インストール**: インストール
- **インストール**: インストール
- **インストール**: インストール
- **インストール**: OS のインストール

## ステップ 1: VM のインストール

VM のインストール:

```
oc create namespace vm-monitoring-demo
```

**monitoring-test-vm.yaml** のインストール

```
oc apply -f - <<EOF
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
```





000:

| NAME               | AGE | STATUS  | READY |
|--------------------|-----|---------|-------|
| monitoring-test-vm | 30s | Running | True  |

VM **Running** **True** **Ctrl+C**

2: **Web** **VM**

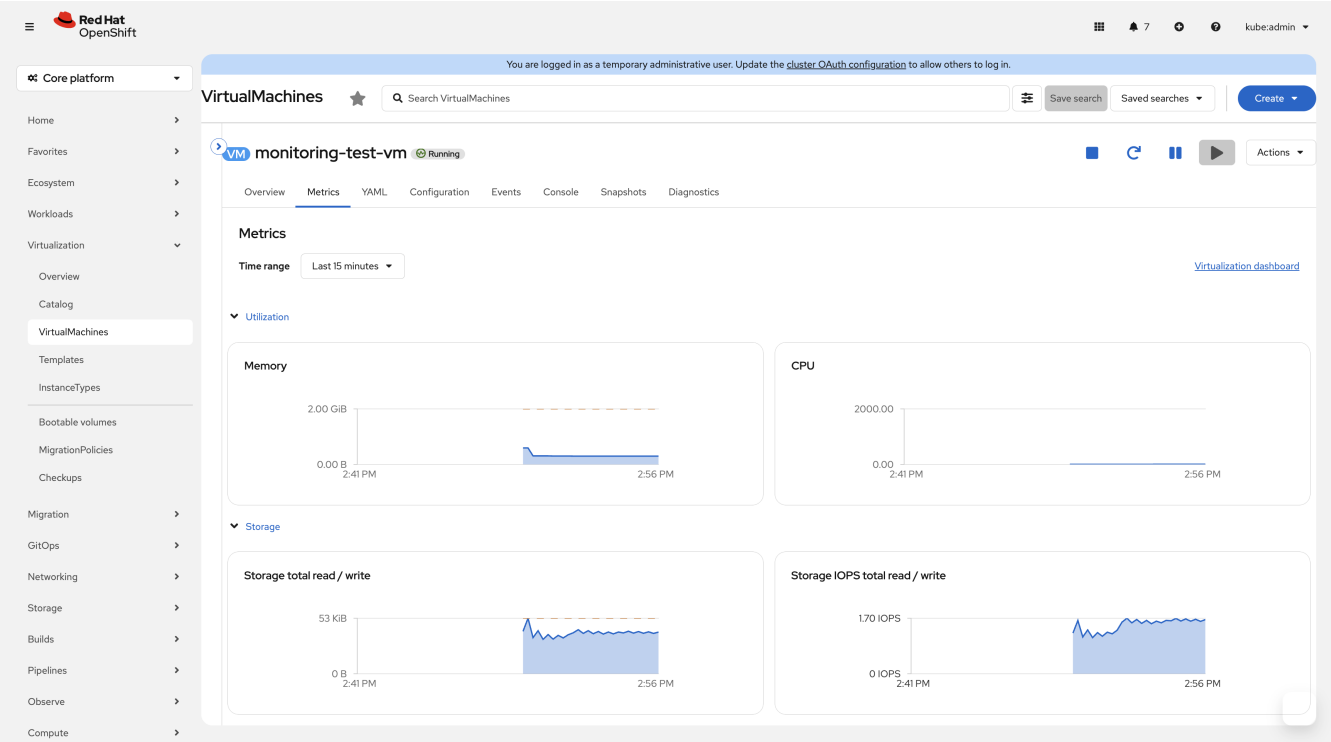
OpenShift Web **VM**

**VM**

1. OpenShift Web
2. **Virtualization** > **VirtualMachines**
3. **monitoring-test-vm**
4. **Metrics**

Metrics 4

- **CPU usage:** vCPU (%)
- **Memory usage:** (GB)
- **Network In/Out:** (MB/s)
- **Storage In/Out:** I/O (MB/s)



**VM** 2-3

## VM 監視

1. **Overview** 概要
2. **Utilization** 利用状況

Overview 概要

- **CPU:** VM CPU 使用率 (%)
- **メモリ:** VM メモリ使用率 (%)
- **ネットワーク:** VM ネットワーク使用率 (%)
- **ストレージ:** VM I/O 使用率 (%)

## 3: Prometheus VM 監視

OpenShift 監視

### Prometheus 監視

1. Web 監視 **Observe > Metrics**
2. OpenShift Prometheus 監視

VM 監視

VM Prometheus 監視

VM CPU:

```
rate(kubevirt_vmi_vcpu_seconds_total[5m]) * 100
```

VM メモリ:

```
kubevirt_vmi_memory_resident_bytes / 1024 / 1024 / 1024
```

VM ネットワーク:

```
rate(kubevirt_vmi_network_receive_bytes_total[5m])
```

VM ストレージ:

```
rate(kubevirt_vmi_storage_read_traffic_bytes_total[5m])
```

VM 監視 (OpenShift):

```
kubevirt_vmi_phase_count
```

**VM** `kubevirt_vmi_info`:

```
kubevirt_vmi_info
```

**Namespace** `namespace` `namespace`

`namespace` `namespace`

```
kubevirt_vmi_memory_resident_bytes{namespace="vm-monitoring-demo"} / 1024 / 1024 / 1024
```

`namespace`

`namespace`

`namespace`:

```
(kubevirt_vmi_memory_resident_bytes / kubevirt_vmi_memory_domain_bytes) * 100
```

`namespace` **CPU** `namespace`:

```
sum(rate(kubevirt_vmi_vcpu_seconds_total[5m])) by (name, namespace)
```

## **4: OpenShift Console**

OpenShift `Web` `VM` `VM`

`VM`

1. **Observe > Dashboards**
2. `VM`

`VM`

- `VM` `VM` `CPU`
- **VM** `VM`
- `VM` `VM`
- `VM` `I/O`

## Metrics

### Observe > Metrics

1. **Observe > Metrics**
2. **Add Query**
3. 3 queries:

```
CPU: rate(kubevirt_vmi_vcpu_seconds_total{namespace="vm-monitoring-demo"}[5m]) * 100
Memory: kubevirt_vmi_memory_resident_bytes / 1024 / 1024 / 1024
Network: rate(kubevirt_vmi_network_receive_bytes_total[5m])
Storage: rate(kubevirt_vmi_storage_read_traffic_bytes_total[5m])
```

4. URL
5. URL

Observe > Metrics Grafana

Observe > Metrics Grafana



Observe > Metrics Grafana

Observe > Metrics:

- OperatorHub Grafana Operator
- Prometheus
- VM

Observe > Metrics VM OpenShift

## 5: VM

VM PrometheusRule



Observe > Metrics SLA

namespace

Observe > Metrics namespace PrometheusRule

```
oc label namespace vm-monitoring-demo openshift.io/cluster-monitoring="true"
```

VM

Observe > Metrics VM PrometheusRule

vm-performance-alerts.yaml

```

oc apply -f - <<EOF
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: vm-performance-alerts
  namespace: vm-monitoring-demo
  labels:
    prometheus: kube-prometheus
    role: alert-rules
spec:
  groups:
  - name: vm-performance
    rules:
    - alert: VMHighCPUUsage
      expr: rate(kubevirt_vmi_vcpu_seconds_total[5m]) * 100 > 80
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "VM {{ \${labels.name} }} high CPU usage"
        description: "VM {{ \${labels.name} }} in namespace {{ \${labels.namespace} }} has CPU
usage above 80% for more than 5 minutes"

    - alert: VMHighMemoryUsage
      expr: (kubevirt_vmi_memory_resident_bytes / kubevirt_vmi_memory_domain_bytes) * 100
> 90
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "VM {{ \${labels.name} }} high memory usage"
        description: "VM {{ \${labels.name} }} in namespace {{ \${labels.namespace} }} has
memory usage above 90% for more than 2 minutes"

    - alert: VMNotRunning
      expr: kubevirt_vmi_info{phase!="running"} == 1
      for: 1m
      labels:
        severity: critical
      annotations:
        summary: "VM {{ \${labels.name} }} not running"
        description: "VM {{ \${labels.name} }} in namespace {{ \${labels.namespace} }} is in
{{ \${labels.phase} }} phase"
EOF

```

□□□□□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□□

```
oc get prometheusrule vm-performance-alerts -n vm-monitoring-demo
```

□□□□□□□□□□□□

OpenShift □□□□□□□□□□□□□□

1. **Observe > Alerting** □□□□□□
2. namespace □□□□□□□□□□: **vm-monitoring-demo**
3. VM □□□□□□□□□□□□□□□□□□□□

□□□□□□□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□□□□□□□

VM □□□□□□□□□□□□□□□□□□

```
virtctl console monitoring-test-vm -n vm-monitoring-demo
```

□□□□ **fedora**□□□□□□ **redhat** □□□□□□□□

VM □□□□□□□□□□□□□□ stress-ng □□□□□□□□□□

```
sudo dnf install -y stress-ng
```

□□□□□□□□□□ VMHighMemoryUsage □□□□□□□□□□□□

```
stress-ng --vm 1 --vm-bytes 1536M --timeout 300s
```

□□□□□□□□□□2GB □ VM □□ 1.5GB □□□□□□□□□□□□□□□□

**Ctrl+]** □□□□□□□□□□□□

□□□□□□□□□□□□

1. **Observe > Alerting** □□□□□□
2. □□□□□□□□ 2□3 □□□□□□
3. VMHighMemoryUsage □□□□□□□□ "Pending" □□□□□□□□□□ "Firing" □□□□□□□□





```
❯ qemu-guest-agent.service - QEMU Guest Agent
   Active: active (running) since Wed 2026-01-01 10:00:00 UTC
```

**Ctrl+]**

**Web**

Web VM **Metrics Overview** OS

- **Guest OS information:**
- **Guest agent status:**
- **Enhanced metrics:**

**Prometheus**

OS

```
kubevirt_vmi_info
```

```
kubevirt_vmi_filesystem_used_bytes / kubevirt_vmi_filesystem_capacity_bytes
```

cloud-init guest agent VM

**Red Hat Enterprise Linux / Fedora:**

```
dnf install qemu-guest-agent
systemctl enable --now qemu-guest-agent
```

**Ubuntu / Debian:**

```
apt update && apt install qemu-guest-agent
systemctl enable --now qemu-guest-agent
```

**Windows:**

QEMU guest agent virtio-win

## VM Monitoring

### VM Monitoring Overview

VM: VM Monitoring Overview "No data available" Overview

VM:

- VM Monitoring Overview 203 Overview
- VM Running Overview: `oc get vm`
- OpenShift VM Monitoring Overview
- VM Monitoring Overview

### VM Monitoring Overview

VM: Web VM Monitoring OS Overview

VM:

- VM Monitoring Overview: `systemctl status qemu-guest-agent`
- VM Monitoring Overview: `systemctl restart qemu-guest-agent`
- VM Monitoring Overview
- VM Monitoring Overview QEMU guest agent Overview

### VM Monitoring Overview

VM: VM Monitoring Overview

VM:

- VM Monitoring Overview Prometheus Overview
- VM Monitoring Overview
- VM Monitoring Overview
- VM Monitoring Overview

### VM Monitoring

VM Monitoring Overview:

```
oc delete namespace vm-monitoring-demo
```

VM

### VM Monitoring Overview

- OpenShift Web Metrics VM Overview

- Prometheus PromQL [PromQL](#) VM [PromQL](#)
- OpenShift Metrics [OpenShift Metrics](#)
- PrometheusRule [PrometheusRule](#) VM [PrometheusRule](#)
- [OS](#) [QEMU guest agent](#) [QEMU guest agent](#)
- [QEMU guest agent](#)

□□

- [VM](#) [QoS](#)
- [OpenShift Virtualization](#) [OpenShift Virtualization](#)
- [Prometheus Query Language \(PromQL\)](#)
- [Grafana](#) [Grafana](#)
- [KubeVirt](#) [KubeVirt](#)

# VM 仮想マシン

## 概要

OpenShift

Virtualization

OpenShift 仮想マシン (VM) を利用して、アプリケーションを仮想化環境で実行します。

## 前提条件

- `qcow2` 形式のイメージファイルが OpenShift Virtualization 環境に存在すること
- 仮想マシン (VM) の作成に必要な権限を持つユーザーであること
- `namespace` として `namespace` という VM の名前空間を設定すること
- VM の起動に必要なリソース (CPU、メモリ) が確保されていること
- UEFI/Secure Boot の設定が正しいこと

## 手順

VM の作成と起動の手順を説明します。

- OpenShift 4.18 以降は OpenShift Virtualization Operator をインストールする必要があります。
- `ReadWriteMany` アクセスモードの `StorageClass` を指定する必要があります。
- CLI ツールとして `oc`, `virtctl`, `kubectl` を使用します。
- `PersistentVolumeClaims` と `DataVolumes` を使用してデータを管理します。

## インストール

### qcow2 イメージの準備

KVM 環境に `qcow2` 形式のイメージファイルを用意し、OpenShift Virtualization 環境に VM を作成します。

### 仮想マシンの作成

仮想マシン (VM) の作成コマンドを実行します。

### 仮想マシンの起動

仮想マシン (VM) の起動コマンドを実行します。

### VM の管理

VM の名前空間 (`namespace`) を指定し、`cloud-init` を使用して VM の設定を行います。

### VM の削除

VM の削除コマンドを実行します。

## VM ــــــــــــــــــــــــــــــــــــــــ

VM ــــــــــــــــــــــــــــــــ (PVCContainerDiskHTTP) ــــــــ UEFI BIOS Secure Boot ــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــــ

## ــــــــــــــــــــــــ

- ــــــــــــــــــــــــ - VM ــــــــــــــــــــــــــــــــ
- ــــــــــــــــــــــــ - VM ــــــــــــــــــــــــ
- VM ــــــــــــــــــــــــــــــــــــــــ - ــــــــــــــــCPU ــــــــــــــــNUMA

## ــــــــــــــــــــــــ

- [OpenShift Virtualization Documentation](#)
- [KubeVirt User Guide](#)

## qcow2 ــــــــــــــــــــــــــــــــــــــــ

### ــــــــ

ــــــــــــــــــــــــــــــــ qcow2 ــــــــــــــــــــــــــــــــ KVM ــــــــــــــــ OpenShift Virtualization ــــــــــــــــــــــــــــــــــــــــ (VM) ــــــــــــــــOVA ــــــــــــــــــــــــ OS ــــــــــــــــــــــــــــــــــــــــ

qcow2 (QEMU Copy-On-Write version 2) ــــــــKVM ــــــــ QEMU ــــــــــــــــــــــــــــــــــــــــ OpenShift Virtualization ــــــــــــــــ



ــــــــــــــــOpenShift Virtualization ــــــــ OpenShift 4.18.33 ــــــــAWS EBS gp3-csi StorageClass ــــــــــــــــــــــــ

### ــــــــ

- OpenShift 4.18 ــــــــ [OpenShift Virtualization Operator](#) ــــــــ
- CLI ــــــــ: `oc`, `virtctl`
- `qemu-img` (ــــــــ) - qcow2 ــــــــــــــــــــــــ
- ــــــــــــــــ (ــــــــ HTTP/S3 ــــــــ) qcow2 ــــــــ
- [StorageClass](#) ــــــــ - ــــــــ
- namespace ــــــــ

### ــــــــــــــــ

qcow2 ــــــــ OpenShift Virtualization ــــــــــــــــ 4 ــــــــ

| コマンド                              | 説明                         | 関連項目                                   |
|-----------------------------------|----------------------------|----------------------------------------|
| <b>virtctl image-upload</b>       | イメージをアップロードする              | イメージを DataVolume/PVC からアップロードする        |
| <b>DataVolume with HTTP/S3</b>    | イメージを HTTP/S3 から取得する       | Web からイメージを取得する DataVolume             |
| <b>ContainerDisk</b>              | イメージを ContainerDisk から取得する | イメージを qcow2 から取得する                     |
| <b>VM with DataVolumeTemplate</b> | イメージを VM から取得する            | VM spec からイメージを取得する DataVolumeTemplate |

イメージをアップロードするコマンドは **virtctl image-upload** です。

## ステップ 1: qcow2 イメージを取得

イメージを qcow2 から取得する OpenShift Virtualization の手順は以下の通りです。

イメージを qcow2 から取得する手順は以下の通りです。

イメージを Fedora Cloud から取得する qcow2 イメージは以下の通りです。

```
# Fedora Cloud イメージを取得 (qcow2)
curl -LO
https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2

# イメージを確認
ls -lh Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
```

イメージを確認 (qcow2)

イメージを **qemu-img** から確認する手順は以下の通りです。

```
qemu-img info Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
```

出力:

```
image: Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2
file format: qcow2
virtual size: 5 GiB (5368709120 bytes)
disk size: 420 MiB
```

Virtual size はイメージの DataVolume から取得するイメージのサイズです。

## ステップ 2: namespace 作成

OpenShift VM の namespace を作成

```
oc create namespace imported-vms
```

## ステップ 3: virtctl を使って qcow2 をアップロード

virtctl を使って qcow2 を OpenShift VM の namespace にアップロード

```
virtctl image-upload pvc fedora-imported-disk \
  --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
  --size=10Gi \
  --namespace=imported-vms \
  --insecure \
  --force-bind
```

オプション:

- **pvc fedora-imported-disk** - PVC の名前
- **--image-path** - qcow2 ファイルのパス
- **--size** - PVC のサイズ (qcow2 の Virtual size を指定)
- **--namespace** - namespace の名前
- **--insecure** - TLS を無視 (デフォルトは有効)
- **--force-bind** - PVC をすぐにバインド (WaitForFirstConsumer を指定)



**--force-bind** は volumeBindingMode: WaitForFirstConsumer を指定した StorageClass (AWS EBS/Azure Disk を含む) を必要とします。PVC が pending 状態にある場合、このオプションは必要です。

コマンドの実行結果:

```
PVC imported-vms/fedora-imported-disk created
Uploading data to https://cdi-uploadproxy-openshift-cnvr.apps...

420.00 MiB / 420.00 MiB [=====] 100.00% 1m30s

Uploading data completed successfully, waiting for processing to complete
Processing completed successfully
```



アップロードされたイメージのサイズは 1GB を超えています。1GB を超えるイメージは 103 MB を超えるイメージです。

**PVC**

□□□□□□PVC □ Pending □□ Bound □□□□□□□□□□

cdi-upload Pod

#### Figure 4: VMs in the VMs



vm-from-imported-qcow2.yaml

555



```

        bus: virtio
        name: rootdisk
      - disk:
          bus: virtio
          name: cloudinitdisk
    interfaces:
      - masquerade: {}
        name: default
    rng: {}
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      persistentVolumeClaim:
        claimName: fedora-imported-disk
    - name: cloudinitdisk
      cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd: { expire: false }
          ssh_pwauth: true

```

VM を適用する

```
oc apply -f vm-from-imported-qcow2.yaml
```

VM を起動する

```
virtctl start fedora-imported-vm -n imported-vms
```

ステップ 5: VM を確認する

VM を確認する

```
oc get vmi -n imported-vms --watch
```

VM の状態を確認する:

| NAME | AGE | PHASE | IP | NODENAME | READY |
|------|-----|-------|----|----------|-------|
|------|-----|-------|----|----------|-------|

|                    |     |            |             |           |       |
|--------------------|-----|------------|-------------|-----------|-------|
| fedora-imported-vm | 15s | Scheduling |             |           | False |
| fedora-imported-vm | 30s | Scheduled  |             | worker-01 | False |
| fedora-imported-vm | 45s | Running    | 10.128.2.54 | worker-01 | True  |

VM を起動する

```
virtctl console fedora-imported-vm -n imported-vms
```

cloud-init の実行結果 (ユーザー: **fedora**、パスワード: **fedora123**) を確認する

ターミナル画面でコマンドを実行する

```
# OS の名前を確認
cat /etc/os-release

# ディスクを確認
lsblk
```

**Ctrl+] を押して** ターミナル画面に戻る

再起動する

virtctl image-upload コマンドでイメージをアップロードする

## ステップ 2: HTTP を使って DataVolume を作成する

qcow2 イメージを HTTP/HTTPS から取得する DataVolume を作成し、CDI を使って Web コンテナに接続する

Fedora Cloud イメージを使用する:

**datavolume-http-import.yaml** を作成する

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: fedora-http-imported
  namespace: imported-vms
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
    http:
      url:
        "https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2"
  storage:
    resources:
```

```
requests:
  storage: 10Gi
```



cdi.kubevirt.io/storage.bind.immediate.requested: "true" PVC  
WaitForFirstConsumer StorageClass

DataVolume CDI

```
oc apply -f datavolume-http-import.yaml
oc get dv -n imported-vms --watch
```

| NAME                 | PHASE            | PROGRESS | RESTARTS | AGE   |
|----------------------|------------------|----------|----------|-------|
| fedora-http-imported | ImportScheduled  | N/A      |          | 5s    |
| fedora-http-imported | ImportInProgress | 12.5%    |          | 15s   |
| fedora-http-imported | ImportInProgress | 67.3%    |          | 45s   |
| fedora-http-imported | Succeeded        | 100.0%   |          | 1m30s |

DataVolume **Succeeded** VM

```
volumes:
- name: rootdisk
  dataVolume:
    name: fedora-http-imported
```



CI/CD S3 URL

### 3: ContainerDisk

qcow2

Containerfile

```
FROM scratch
ADD Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 /disk/
```

```
podman build -t quay.io/youruser/fedora-disk:v1 .
podman push quay.io/youruser/fedora-disk:v1
```

VM spec

```
volumes:
- name: rootdisk
  containerDisk:
    image: quay.io/youruser/fedora-disk:v1
```



ContainerDisk    VM    Persistent    Volume

#### 4: DataVolumeTemplate    VM

DataVolumeTemplate    VM    DataVolume    VM    spec

- VM    - GitOps    - DataVolume    VM

**vm-with-upload-datavolume.yaml**

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-with-upload
  namespace: imported-vms
spec:
  dataVolumeTemplates:
  - apiVersion: cdi.kubevirt.io/v1beta1
    kind: DataVolume
    metadata:
      name: vm-with-upload-disk
    spec:
      source:
        upload: {}
      storage:
        resources:
          requests:
            storage: 10Gi
  runStrategy: RerunOnFailure
template:
  metadata:
    labels:
      kubevirt.io/domain: vm-with-upload
  spec:
    domain:
      cpu:
        cores: 2
      devices:
        disks:
        - disk:
            bus: virtio
            name: rootdisk
```

```

    interfaces:
      - masquerade: {}
        name: default
    rng: {}
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: vm-with-upload-disk
      name: rootdisk

```

VM `vm-with-upload-disk` VM `vm-with-upload-disk` `DataVolume` `vm-with-upload-disk`

```
oc apply -f vm-with-upload-datavolume.yaml
```

`DataVolume` `vm-with-upload-disk` `vm-with-upload-disk` VM `vm-with-upload-disk`

```

virtctl image-upload dv vm-with-upload-disk \
  --image-path=./Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2 \
  --size=10Gi \
  --namespace=imported-vms \
  --insecure \
  --force-bind

```

`vm-with-upload-disk` VM `vm-with-upload-disk`

```
virtctl start vm-with-upload -n imported-vms
```

`vm-with-upload-disk`: - `vm-with-upload-disk` VM `vm-with-upload-disk` - `DataVolume` `vm-with-upload-disk` VM `vm-with-upload-disk` - Infrastructure-as-Code `vm-with-upload-disk`

`vm-with-upload-disk`: - PVC `vm-with-upload-disk` - VM `vm-with-upload-disk` - `DataVolume/virtctl` `vm-with-upload-disk`

`vm-with-upload-disk`

`vm-with-upload-disk`

- `vm-with-upload-disk` - `vm-with-upload-disk`
- `vm-with-upload-disk` - `virt-sparsify` `vm-with-upload-disk` `qcow2` `vm-with-upload-disk`
- `vm-with-upload-disk` - `vm-with-upload-disk`

- **cloud-init** `cloud-init` - `cloud-init` `cloud-init` `cloud-init`

cloud-init

- `cloud-init` - VM `Virtual size` `Virtual size`
- `cloud-init` - `StorageClass` `StorageClass`
- `cloud-init` - `PVC` `PVC`

cloud-init

- `cloud-init` - `qcow2` `qcow2`
- **TLS** `insecure` - `insecure` `insecure`
- `cloud-init` - `RBAC` `RBAC`
- `cloud-init` - `StorageClass` `StorageClass`

cloud-init

- `cloud-init` - `StorageClass` `StorageClass`
- `cloud-init` - `StorageClass` `StorageClass`
- `cloud-init` - `StorageClass` `StorageClass`

cloud-init

cloud-init **"Waiting for PVC upload pod to be ready"** `Waiting for PVC upload pod to be ready`

cloud-init Pod `WaitForFirstConsumer` `WaitForFirstConsumer` `StorageClass` `binding mode`

PVC `PVC`

```
oc get pvc -n imported-vms
oc describe pvc fedora-imported-disk -n imported-vms
```

cloud-init "waiting for first consumer to be created before binding" `StorageClass` `volumeBindingMode: WaitForFirstConsumer`

cloud-init: `virtctl` `force-bind` `force-bind`

```
virtctl image-upload pvc fedora-imported-disk \
  --image-path=./image.qcow2 \
  --size=10Gi \
  --namespace=imported-vms \
  --insecure \
  --force-bind
```

削除する PVC を確認する

```
oc delete pvc fedora-imported-disk -n imported-vms
```

オプション **--force-bind** を指定する

エラーメッセージ **"Unable to get PVC Prime name from PVC"** を回避

DataVolume を作成する際、CDI を使用して PVC を作成する (DataVolume を作成する) の

```
# PVC を作成する (DataVolume を作成)
virtctl image-upload pvc <pvc-name> \
  --image-path=./image.qcow2 \
  --size=10Gi \
  --namespace=imported-vms \
  --insecure
```

確認: \* **CDI Operator** の名前: **cdi-deployment** の **cdi-operator** Pod を確認する \* **Storage Class** の名前: **StorageClass** を確認する

CDI Pod を確認する

```
oc get pods -n openshift-cnv | grep cdi
```

確認する

CDI upload proxy Pod を確認する

```
oc get pods -n openshift-cnv | grep cdi-uploadproxy
oc logs -n openshift-cnv <cdi-uploadproxy-pod>
```

確認する

**PVC** を確認する

**--size** オプションで **qcow2** の **Virtual size** を指定する

```
qemu-img info image.qcow2 | grep "virtual size"
```

削除する PVC を確認する

```
oc delete pvc fedora-imported-disk -n imported-vms
```

```
# オプション --size を指定する
```

```
virtctl image-upload pvc fedora-imported-disk \
  --image-path=./image.qcow2 \
  --size=20Gi \
  --namespace=imported-vms \
  --insecure
```

VM を起動します。

VM を確認します。

```
oc describe vm fedora-imported-vm -n imported-vms
oc get events -n imported-vms --field-selector involvedObject.name=fedora-imported-vm
```

VM のコンソールを確認します。

```
virtctl console fedora-imported-vm -n imported-vms
```

出力例: \* 起動ログ - 起動ログ \* 起動ログ - VM spec の `machine.type` が \* 起動ログ - 起動ログ

確認完了

削除作業の完了を確認します。

```
# VM の削除
oc delete vm fedora-imported-vm -n imported-vms

# PVC の削除
oc delete pvc fedora-imported-disk -n imported-vms

# namespace の削除
oc delete namespace imported-vms
```



PVC の削除作業の完了を確認します。

完了

作業の完了を確認します。

- OpenShift Virtualization をインストールして `qcow2` をインストール
- `virtctl image-upload pvc` を使って `qcow2` をインストール
- インストールされた VM を確認
- インストールを確認
- インストール (HTTPContainerDiskDataVolumeTemplate) を確認



- [OpenShift Virtualization](#) - OpenShift Virtualization
- [KVM](#) - KVM

OpenShift Virtualization KVM OpenShift Virtualization

## OpenShift Virtualization

- [OpenShift Virtualization](#) - OpenShift Virtualization
- [IP](#) - VM IP
- [SSH](#) - VM SSH

## OpenShift

- [Containerized Data Importer \(CDI\)](#)
- [virtctl](#)

## OpenShift Virtualization

## OpenShift

OpenShift Virtualization Virtualization  
OpenShift Virtualization VM  
OpenShift Virtualization VM

## OpenShift Virtualization

OpenShift Virtualization VM - OpenShift Virtualization - OpenShift Virtualization - OpenShift Virtualization - OpenShift Virtualization - OpenShift Virtualization

OpenShift Virtualization - VM (OpenShift Virtualization) - OpenShift Virtualization - OpenShift Virtualization - OpenShift Virtualization - OpenShift Virtualization

## OpenShift Virtualization

OpenShift Virtualization

- [OpenShift Virtualization](#): OpenShift Virtualization CIS RHEL OpenShift Virtualization VM
- [OpenShift Virtualization](#): ISO OpenShift Virtualization 5 50 VM OpenShift Virtualization
- [OpenShift Virtualization](#): 1 OpenShift Virtualization RHEL 9.3 OpenShift Virtualization Q1 OpenShift Virtualization RHEL 9.4 OpenShift Virtualization Q2 OpenShift Virtualization VM OpenShift Virtualization OpenShift Virtualization
- [OpenShift Virtualization](#): OpenShift Virtualization OpenShift Virtualization OpenShift Virtualization OpenShift Virtualization OpenShift Virtualization OpenShift Virtualization
- [OpenShift Virtualization](#): OpenShift Virtualization OpenShift Virtualization OpenShift Virtualization OpenShift Virtualization OpenShift Virtualization OpenShift Virtualization

## 前提条件

- OpenShift Virtualization Operator が必要です。OpenShift 4.18 以降
- CLI ツール: `oc`, `virtctl`, `kubectl`
- Storage Class が必要です (必要に応じて `lvms-vg1` を参照)
- Linux が必要です
- cloud-init が必要です

## インストールと検証

### インストールと検証

1. OS が必要です。VM が必要です
2. VM が必要です (必要に応じて参照)
3. VM が必要です (必要に応じて参照)
4. VM が必要です
5. 必要に応じて DataSource が必要です
6. 必要に応じて VM が必要です

## ステップ 1: VM の作成

namespace を作成します

```
oc create namespace golden-images
```

Fedora VM をインストールします



cloud-init をインストールして SSH を有効にします

`golden-image-base-vm.yaml` を参照してください

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: base-fedora-vm
  namespace: golden-images
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: base-fedora-vm-disk
      spec:
        sourceRef:
```

```

    kind: DataSource
    name: fedora
    namespace: openshift-virtualization-os-images
  storage:
    resources:
      requests:
        storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: base-fedora-vm
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        machine:
          type: pc-q35-rhel9.4.0
        memory:
          guest: 4Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - dataVolume:
            name: base-fedora-vm-disk
            name: rootdisk
        - cloudInitNoCloud:
            userData: |-
              #cloud-config
              user: fedora
              password: fedora123
              chpasswd: { expire: false }
              ssh_pwauth: true
            name: cloudinitdisk

```

base-fedora-vm.yaml

```
oc apply -f base-fedora-vm.yaml
```

VM `base-fedora-vm-disk` `DataVolume` `base-fedora-vm-disk`

```
oc get dv -n golden-images
```

base-fedora-vm-disk

| NAME                | PHASE           | PROGRESS | RESTARTS | AGE |
|---------------------|-----------------|----------|----------|-----|
| base-fedora-vm-disk | CloneInProgress | 45.2%    |          | 2m  |

`DataVolume` `base-fedora-vm-disk` **Succeeded** `base-fedora-vm-disk`

```
oc get vmi -n golden-images
```

base-fedora-vm

| NAME           | AGE | PHASE   | IP           | NODENAME | READY |
|----------------|-----|---------|--------------|----------|-------|
| base-fedora-vm | 4m  | Running | 10.128.1.151 | ocplab01 | True  |

Note: `base-fedora-vm-disk` `base-fedora-vm-disk` 305 `base-fedora-vm-disk`

## 2: VM `base-fedora-vm`

VM `base-fedora-vm`

```
virtctl console base-fedora-vm -n golden-images
```

`base-fedora-vm` `base-fedora-vm` `base-fedora123` `base-fedora-vm`

`base-fedora-vm`

```
# base-fedora-vm
sudo dnf update -y

# nginx Web base-fedora-vm
sudo dnf install -y nginx vim htop curl wget git

# base-fedora-vm
sudo systemctl enable nginx

# base-fedora-vm
sudo bash -c 'cat > /usr/share/nginx/html/index.html << EOF
```

```
<!DOCTYPE html>
<html>
<head>
  <title>Golden Image VM</title>
</head>
<body>
  <h1>VM from Golden Image</h1>
  <p>This VM was deployed from a golden image with pre-installed software.</p>
  <p>Nginx is pre-configured and ready to use.</p>
</body>
</html>
EOF'
```

```
# firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

### □□□□ 3: VM □□□□

[illegible]

```
# 清理yum缓存
sudo dnf clean all

# 清理日志文件
sudo find /var/log -type f -exec truncate -s 0 {} \;

# 清理ID文件
sudo rm -f /etc/machine-id
sudo touch /etc/machine-id

# cloud-init清理
sudo cloud-init clean --logs --seed

# 清理bash历史
history -c
cat /dev/null > ~/.bash_history

# 同步并退出
sync
exit
```

Ctrl+] □□□□□□□□□□□□□□□□

#### □□□□ 4: VM □□□

○○○○○○○○○○○○○○ VM ○○○○○○○○

```
virtctl stop base-fedora-vm -n golden-images
```

□□□□□□□□□□

```
oc get vm -n golden-images
```

□□□

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 15m | Stopped | False |

## □□□□ 5: □□□□□□□□ **DataSource** □□□

□□□□□□ VM □□□□□□□□□□ DataSource □□□□□□□□□□ DataSource □□□□□□□□□□ VM □□□□□□□□

DataSource □□□□□□□□

**golden-image-datasource.yaml** □□□□□□□□

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataSource
metadata:
  name: fedora-nginx-golden
  namespace: golden-images
spec:
  source:
    pvc:
      name: base-fedora-vm-disk
      namespace: golden-images
```

**fedora-golden-datasource.yaml** □□□□□□□□□□□□□□

```
oc apply -f fedora-golden-datasource.yaml
```

DataSource □□□□□□□□

```
oc get datasource -n golden-images
```

□□□

| NAME                | AGE |
|---------------------|-----|
| fedora-nginx-golden | 5s  |

## 例 6: VM の作成

VM の作成には、`VM` のリソースを定義する YAML ファイルを作成する。

この VM の YAML は

`webserver-vm-from-golden.yaml` のように

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-vm-1
  namespace: golden-images
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: webserver-vm-1-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora-nginx-golden
          namespace: golden-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-vm-1
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        machine:
          type: pc-q35-rhel9.4.0
```

```

    memory:
      guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: webserver-vm-1-disk
        name: rootdisk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          hostname: webserver-vm-1
          fqdn: webserver-vm-1.example.com
        name: cloudinitdisk

```

2 개의 VM 생성하기

**webserver-vm-2.yaml** 생성하기

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-vm-2
  namespace: golden-images
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: webserver-vm-2-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora-nginx-golden
          namespace: golden-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-vm-2
    spec:
      domain:
        cpu:
          cores: 2
        devices:

```



```
disks:
  - disk:
      bus: virtio
      name: rootdisk
  - disk:
      bus: virtio
      name: cloudinitdisk
interfaces:
  - masquerade: {}
      name: default
rng: {}
machine:
  type: pc-q35-rhel9.4.0
memory:
  guest: 4Gi
networks:
  - name: default
      pod: {}
volumes:
  - dataVolume:
      name: webserver-vm-2-disk
      name: rootdisk
  - cloudInitNoCloud:
      userData: |-
          #cloud-config
          hostname: webserver-vm-2
          fqdn: webserver-vm-2.example.com
      name: cloudinitdisk
```

VM 00000000

```
oc apply -f webserver-vm-1.yaml
oc apply -f webserver-vm-2.yaml
```

□□□□□□□□□□□□□□

```
oc get dv -n golden-images
```

111

| NAME                | PHASE           | PROGRESS | RESTARTS | AGE |
|---------------------|-----------------|----------|----------|-----|
| base-fedora-vm-disk | Succeeded       | 100.0%   |          | 15m |
| webserver-vm-1-disk | CloneInProgress | 72.5%    |          | 3m  |
| webserver-vm-2-disk | CloneInProgress | 71.8%    |          | 3m  |

DataVolume VM 305

## ステップ 7: VM 確認

VM のステータスを確認します。

```
oc get vmi -n golden-images
```

結果

| NAME           | AGE | PHASE   | IP           | NODENAME | READY |
|----------------|-----|---------|--------------|----------|-------|
| webserver-vm-1 | 5m  | Running | 10.128.1.152 | ocplab01 | True  |
| webserver-vm-2 | 5m  | Running | 10.128.1.153 | ocplab01 | True  |

VM から `nginx` のコンソールに接続します。

```
virtctl console webserver-vm-1 -n golden-images
```

`nginx` のステータスを確認します。

```
sudo systemctl status nginx
```

ブラウザ (Firefox) で確認します。

Web ページを確認します。

```
curl localhost
```

ブラウザで `localhost` にアクセスし、HTML ページを確認します。

**Ctrl+]** でコンソールに戻ります。

確認完了

以下の手順に従って VM を作成します。

1. 必要な VM のイメージ
2. 必要なリソース
3. VM の作成
4. 必要な `DataSource` の作成 (例: `fedora-nginx-golden-v2`)
5. 必要な VM の起動

`DataSource` の名前 - `fedora-nginx-golden-v1` - `fedora-nginx-golden-v2` - `fedora-nginx-golden-2025-10`

## インストール

### インストール

- インストールディレクトリ
- インストールディレクトリを指定する
- インストールディレクトリを指定する
- インストールディレクトリを指定する
- インストールディレクトリ (changelog) を指定

### インストール

- インストールディレクトリ
- SSH インストール
- インストールディレクトリ
- cloud-init インストール
- bash インストール

### インストール

- インストールディレクトリ
- インストールディレクトリ
- インストールディレクトリ

### インストール

- インストールディレクトリ
- インストールディレクトリ
- インストールディレクトリ
- インストールディレクトリ
- VM インストール (SSH インストール) の cloud-init インストール

## インストール

インストールディレクトリを指定する[インストールディレクトリを指定する](#)インストールディレクトリを指定する

### インストール

インストールディレクトリを指定する

```
# VMインストール
oc delete vm webserver-vm-1 webserver-vm-2 base-fedora-vm -n golden-images

# DataSourceインストール
```

```
oc delete datasource fedora-nginx-golden -n golden-images

# namespace
oc delete namespace golden-images
```

```
oc delete datasource fedora-nginx-golden -n golden-images

# namespace
oc delete namespace golden-images
```

```
oc delete datasource fedora-nginx-golden -n golden-images

# namespace
oc delete namespace golden-images
```

111

00000000000000000000 - 000 VM 0000000000 - 000000000000000000 VM 0000 - 0000000000  
 DataSource 000 - 00000000000000 VM 000 - 0000000000000000

# OpenShift Virtualization VM

11

- [OpenShift Virtualization Documentation](#)
- [KubeVirt DataVolume Documentation](#)
- [Containerized Data Importer](#)

# Windows □□□□□□□□

11

windows-efi-installer Tekton OpenShift Virtualization Windows Server  
 2025 ISO VirtIO  
 sysprep  
 Windows VM sysprep unattend.xml

```
Windows  [] Linux  [] cloud-init  []Windows  []Windows
[]
      autounattend.xml  [] sysprep
[]
```

□ □ □ □

- Windows `autounattend.xml` Linux `autoinstall`
- Windows Server 2025 `autounattend.xml` `autounattend.xml`
- `windows-efi-installer` Tekton `windows-efi-installer`
- `sysprep` `sysprep` VM `sysprep`
- `Windows (10/11/Server 2022)` `Windows (10/11/Server 2022)`

# Windows 環境から Linux 環境

[illegible]

| 項目           | Linux                                  | Windows                                |
|--------------|----------------------------------------|----------------------------------------|
| インストール       | Kickstart 形式 preseed                   | autounattend.xml (Windows インストール形式)    |
| インストールオプション  | cloud-init                             | sysprep 形式 unattend.xml (OOBE 形式)      |
| インストール後      | /etc/machine-id 形式 cloud-init clean 形式 | sysprep /generalize /oobe /shutdown 形式 |
| インストール後オプション | cloud-init userData (SSH 形式)           | Sysprep unattend.xml 形式 (RDP 形式)       |
| インストール後オプション |                                        | VirtIO 形式                              |

windows-efi-installer 形式 ISO  
インストール形式 autounattend.xml ConfigMap 形式 VirtIO  
インストール形式 VM 形式 sysprep 形式 DataSource 形式

前提条件

- OpenShift Virtualization Operator 形式 OpenShift 4.18+
- OpenShift Pipelines Operator 形式 (Tekton 形式)
- CLI 形式: oc 形式 kubectl
- Microsoft Evaluation Center 形式 Windows Server 2025 形式 ISO 形式 URL
- 形式 60 GiB 形式 Storage Class
- 形式 go.microsoft.com (形式 ISO 形式) 形式
- Linux 形式 形式

ステップ 1: Namespace 形式 ServiceAccount 形式

形式 default namespace 形式 modify-windows-iso-file  
形式 ID 形式 ID 形式 pipeline 形式 anyuid Security  
Context Constraint (SCC) 形式

```
oc adm policy add-scc-to-user anyuid -z pipeline -n default
```

形式ISO 形式 runAsUser: 107 形式 fsGroup: 107 形式 Pod 形式

ステップ 2: Windows ISO 形式 URL 形式

1. Microsoft Evaluation Center for Windows Server 2025 形式
2. Microsoft 形式 (形式) 形式
3. 64-bit edition形式 English (United States) 形式
4. 形式 Download 形式

5. URL を指定して PipelineRun を実行する



このチュートリアルでは、URL を指定して 24 時間以内に ISO  
イメージを生成する `Evaluation Center` の URL を指定する



この ISO イメージは 180 日間有効です。このイメージを生成する ISO  
イメージを生成する

この Windows イメージを生成する

- [Windows 10 Enterprise](#)
- [Windows 11 Enterprise](#)
- [Windows Server 2022](#)

3. Autounattend ConfigMap を作成

`autounattend.xml` を作成して Windows Setup を実行する `post-install.ps1`  
スクリプトを実行する `sysprep` コマンドを実行する 2. この ConfigMap を作成

`configmap-windows2k25-autounattend.yaml` を作成

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: windows2k25-autounattend
  namespace: default
data:
  autounattend.xml: |
    <?xml version="1.0" encoding="utf-8"?>
    <unattend xmlns="urn:schemas-microsoft-com:unattend">
      <settings pass="windowsPE">
        <component name="Microsoft-Windows-PnpCustomizationsWinPE"
processorArchitecture="amd64"
          publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
          xmlns:wcm="http://schemas.microsoft.com/WMIConfig/2002/State">
            <DriverPaths>
              <PathAndCredentials wcm:action="add" wcm:keyValue="1">
                <Path>E:\viostor\2k25\amd64</Path>
              </PathAndCredentials>
              <PathAndCredentials wcm:action="add" wcm:keyValue="2">
                <Path>E:\NetKVM\2k25\amd64</Path>
              </PathAndCredentials>
            </DriverPaths>
          </component>
          <component name="Microsoft-Windows-International-Core-WinPE"
processorArchitecture="amd64"
            publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
```

```

    <SetupUILanguage>
      <UILanguage>en-US</UILanguage>
    </SetupUILanguage>
    <InputLocale>en-US</InputLocale>
    <SystemLocale>en-US</SystemLocale>
    <UILanguage>en-US</UILanguage>
    <UserLocale>en-US</UserLocale>
  </component>
  <component name="Microsoft-Windows-Setup" processorArchitecture="amd64"
    publicKeyToken="31bf3856ad364e35" language="neutral"
    versionScope="nonSxS"
    xmlns:wcm="http://schemas.microsoft.com/WMIConfig/2002/State">
    <UserData>
      <AcceptEula>true</AcceptEula>
    </UserData>
    <DiskConfiguration>
      <Disk wcm:action="add">
        <DiskID>0</DiskID>
        <WillWipeDisk>true</WillWipeDisk>
        <CreatePartitions>
          <CreatePartition wcm:action="add">
            <Order>1</Order>
            <Type>EFI</Type>
            <Size>100</Size>
          </CreatePartition>
          <CreatePartition wcm:action="add">
            <Order>2</Order>
            <Type>MSR</Type>
            <Size>16</Size>
          </CreatePartition>
          <CreatePartition wcm:action="add">
            <Order>3</Order>
            <Type>Primary</Type>
            <Extend>true</Extend>
          </CreatePartition>
        </CreatePartitions>
        <ModifyPartitions>
          <ModifyPartition wcm:action="add">
            <Order>1</Order>
            <PartitionID>1</PartitionID>
            <Format>FAT32</Format>
            <Label>System</Label>
          </ModifyPartition>
          <ModifyPartition wcm:action="add">
            <Order>2</Order>
            <PartitionID>2</PartitionID>
          </ModifyPartition>
          <ModifyPartition wcm:action="add">
            <Order>3</Order>
            <PartitionID>3</PartitionID>
            <Format>NTFS</Format>
          </ModifyPartition>
        </ModifyPartitions>
      </Disk>
    </DiskConfiguration>
  </component>

```

```

        <Label>Windows</Label>
        <Letter>C</Letter>
    </ModifyPartition>
</ModifyPartitions>
</Disk>
</DiskConfiguration>
<ImageInstall>
    <OSImage>
        <InstallFrom>
            <MetaData wcm:action="add">
                <Key>/Image/Description</Key>
                <Value>Windows Server 2025 SERVERDATACENTER</Value>
            </MetaData>
        </InstallFrom>
        <InstallTo>
            <DiskID>0</DiskID>
            <PartitionID>3</PartitionID>
        </InstallTo>
    </OSImage>
</ImageInstall>
</component>
</settings>
<settings pass="specialize">
    <component name="Microsoft-Windows-International-Core"
processorArchitecture="amd64"
        publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
        <InputLocale>en-US</InputLocale>
        <SystemLocale>en-US</SystemLocale>
        <UILanguage>en-US</UILanguage>
        <UserLocale>en-US</UserLocale>
    </component>
</settings>
<settings pass="oobeSystem">
    <component name="Microsoft-Windows-Deployment" processorArchitecture="amd64"
        publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
        <Re seal>
            <Mode>Audit</Mode>
        </Re seal>
    </component>
</settings>
<settings pass="auditUser">
    <component name="Microsoft-Windows-Deployment" processorArchitecture="amd64"
        publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
        xmlns:wcm="http://schemas.microsoft.com/WMIconfig/2002/State">
        <RunSynchronous>
            <RunSynchronousCommand wcm:action="add">
                <Order>1</Order>
                <Path>PowerShell -ExecutionPolicy Bypass -NoProfile F:\post-

```



```
install.ps1</Path>
    <Description>Run post-install script</Description>
    </RunSynchronousCommand>
</RunSynchronous>
<Generalize>
    <ForceShutdownNow>true</ForceShutdownNow>
    <Mode>OOBE</Mode>
</Generalize>
</component>
</settings>
</unattend>
post-install.ps1: |
    # VirtIO guest driver
    Start-Process msixec -Wait -ArgumentList "/i E:\virtio-win-gt-x64.msi /qn
/passive /norestart"

    # QEMU guest agent
    Start-Process msixec -Wait -ArgumentList "/i E:\guest-agent\qemu-ga-x86_64.msi
/qn /passive /norestart"

    # sysprep unattend.xml
    mv C:\Windows\Panther\unattend.xml C:\Windows\Panther\unattend.install.xml

    # CD autounattend.xml sysprep CD
    (New-Object -COMObject
Shell.Application).NameSpace(17).ParseName("F:").InvokeVerb("Eject")
```

ConfigMap

```
oc apply -f configmap-windows2k25-autounattend.yaml
```

**autounattend.xml**

4 Windows

- windowsPE** — Windows VirtIO SCSI NIC (viorstor) (NetKVM) GPT (EFI) Windows Server 2025 Datacenter (Desktop Experience) EULA
- specialize** — en-US
- oobeSystem** — Out-Of-Box Experience (OOBE) Administrator
- auditUser** — post-install.ps1 sysprep /generalize /oobe /shutdown VM

## post-install.ps1

PowerShell SYSTEM 4

1. E:\ VirtIO VirtIO guest driver package (`virtio-win-gt-x64.msi`)
2. OpenShift VM QEMU guest agent (`qemu-ga-x86_64.msi`)
3. C:\Windows\Panther\unattend.xml unattend.install.xml VM sysprep
4. CD autounattend.xml CD-ROM

## 4: Pipeline

REPLACE\_WITH\_YOUR\_ISO\_URL 2 URL



PipelineRun acceptEula Microsoft  
Microsoft (EULA) false  
Pipeline

### pipelinerun-windows2k25.yaml

```
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: windows2k25-installer-run-
  namespace: default
spec:
  params:
    - name: winImageDownloadURL
      value: "REPLACE_WITH_YOUR_ISO_URL"
    - name: acceptEula
      value: "false"
    - name: preferenceName
      value: windows.2k25.virtio
    - name: autounattendConfigMapName
      value: windows2k25-autounattend
    - name: baseDvName
      value: win2k25
    - name: isoDVName
      value: win2k25
  pipelineRef:
    params:
      - name: catalog
        value: redhat-pipelines
      - name: type
        value: artifact
      - name: kind
        value: pipeline
      - name: name
        value: windows-efi-installer
```

```

- name: version
  value: v4.21.0
resolver: hub
taskRunSpecs:
- pipelineTaskName: modify-windows-iso-file
  podTemplate:
    securityContext:
      fsGroup: 107
      runAsUser: 107
timeouts:
  pipeline: 1h0m0s

```

PipelineRun `generateName` `oc apply` `oc create`

```
oc create -f pipelinerun-windows2k25.yaml
```

```
oc get pipelinerun -n default -w
```

PipelineRun `Succeeded` `Ctrl+C` `watch`

`DataVolume`

```
oc get dv -n default -w
```

`windows-efi-installer`

1. **Import autounattend ConfigMap** — `ConfigMap` `autounattend.xml` `post-install.ps1`
2. **Download Windows ISO** — ISO `DataVolume`
3. **Modify ISO** — `Press any key` `EFI` `(efisys.bin)` `(efisys_noprompt.bin)`
4. **Create blank root disk** — Windows 60 GiB `DataVolume`
5. **Boot installer VM** — `ISO` `root` `VirtIO` `VM` `autounattend.xml`
6. **Wait for shutdown** — `VM` `auditUser` `sysprep` `generalize`
7. **Create DataSource** — `root` `DataSource`
8. **Clean up** — `VM` `DataVolume` (ISO) `ISO`

ISO (ISO) (ISO) 3060

## 5:

DataVolume DataSource

```
oc get dv win2k25 -n default
```

| NAME    | PHASE     | PROGRESS | RESTARTS | AGE |
|---------|-----------|----------|----------|-----|
| win2k25 | Succeeded | 100.0%   |          | 45m |

```
oc get datasource win2k25 -n default
```

| NAME    | AGE |
|---------|-----|
| win2k25 | 45m |

PipelineRun

```
oc get pipelinerun -n default
```

| NAME                            | SUCCEEDED | REASON    | STARTTIME | COMPLETIONTIME |
|---------------------------------|-----------|-----------|-----------|----------------|
| windows2k25-installer-run-abcde | True      | Succeeded | 50m       | 5m             |

## 6: VM

VM OOBESysprep unattend.xml ConfigMap sysprep VM KubeVirt CD Windows Setup OOBESysprep

sysprep ConfigMap

unattend.xml WIN-SERVER-01 Administrator OS OOBESysprep



sysprep-unattend-configmap.yaml

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: win2k25-sysprep-unattend
  namespace: default
data:
  unattend.xml: |
    <?xml version="1.0" encoding="utf-8"?>
    <unattend xmlns="urn:schemas-microsoft-com:unattend">
      <settings pass="specialize">
        <component name="Microsoft-Windows-Shell-Setup" processorArchitecture="amd64"
          publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
          <ComputerName>WIN-SERVER-01</ComputerName>
        </component>
        <component name="Microsoft-Windows-Deployment" processorArchitecture="amd64"
          publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
          <ExtendOSPartition>
            <Extend>true</Extend>
          </ExtendOSPartition>
        </component>
        <component name="Microsoft-Windows-TerminalServices-LocalSessionManager"
          processorArchitecture="amd64" publicKeyToken="31bf3856ad364e35"
          language="neutral" versionScope="nonSxS">
          <fDenyTSConnections>>false</fDenyTSConnections>
        </component>
        <component xmlns:wcm="http://schemas.microsoft.com/WMICConfig/2002/State"
          name="Networking-MPSSVC-Svc" processorArchitecture="amd64"
          publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
          <FirewallGroups>
            <FirewallGroup wcm:action="add" wcm:keyValue="RemoteDesktop">
              <Active>true</Active>
              <Group>Remote Desktop</Group>
              <Profile>all</Profile>
            </FirewallGroup>
          </FirewallGroups>
        </component>
      </settings>
      <settings pass="oobeSystem">
        <component xmlns:wcm="http://schemas.microsoft.com/WMICConfig/2002/State"
          name="Microsoft-Windows-Shell-Setup" processorArchitecture="amd64"
          publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS">
          <OOBE>
            <HideEULAPage>true</HideEULAPage>
            <HideOEMRegistrationScreen>true</HideOEMRegistrationScreen>
            <HideOnlineAccountScreens>true</HideOnlineAccountScreens>
            <HideWirelessSetupInOOBE>true</HideWirelessSetupInOOBE>

```

```

    <NetworkLocation>Work</NetworkLocation>
    <SkipUserOOBE>true</SkipUserOOBE>
    <SkipMachineOOBE>true</SkipMachineOOBE>
    <ProtectYourPC>3</ProtectYourPC>
  </OOBE>
  <AutoLogon>
    <Password>
      <Value>R3dH4t1!</Value>
      <PlainText>true</PlainText>
    </Password>
    <Enabled>true</Enabled>
    <Username>Administrator</Username>
  </AutoLogon>
  <UserAccounts>
    <AdministratorPassword>
      <Value>R3dH4t1!</Value>
      <PlainText>true</PlainText>
    </AdministratorPassword>
  </UserAccounts>
  <TimeZone>UTC</TimeZone>
</component>
</settings>
</unattend>

```

ConfigMap

```
oc apply -f sysprep-unattend-configmap.yaml
```

VM

VM `win2k25` DataSource `sysprep` ConfigMap `windows.2k25.virtio` preference `Windows` (Hyper-V enlightenments Secure Boot EFI TPM VirtIO) `u1.large`instancetype CPU

`windows-vm-from-golden.yaml`

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: win2k25-vm-1
  namespace: default
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: win2k25-vm-1-disk

```

```

spec:
  sourceRef:
    kind: DataSource
    name: win2k25
    namespace: default
  storage:
    resources:
      requests:
        storage: 60Gi
  runStrategy: RerunOnFailure
 instancetype:
    name: u1.large
  preference:
    name: windows.2k25.virtio
  template:
    spec:
      domain:
        devices:
          disks:
            - name: rootdisk
              disk: {}
            - name: sysprep
              cdrom: {}
      volumes:
        - name: rootdisk
          dataVolume:
            name: win2k25-vm-1-disk
        - name: sysprep
          sysprep:
            configMap:
              name: win2k25-sysprep-unattend

```

VM を作成します

```
oc apply -f windows-vm-from-golden.yaml
```

作成した VM は Windows OBE 画面で 5 分間待ちます

```
oc get dv win2k25-vm-1-disk -n default -w
```

DataVolume が **Succeeded** になったら VM を起動します

```
oc get vmi win2k25-vm-1 -n default -w
```

VM が **PHASE=Running** かつ **READY=True** になったら **Ctrl+C** を入力します

ステップ 7: VM の起動

VM の起動を確認します。

```
oc get vmi win2k25-vm-1 -n default
```

結果

| NAME         | AGE | PHASE   | IP           | NODENAME  | READY |
|--------------|-----|---------|--------------|-----------|-------|
| win2k25-vm-1 | 8m  | Running | 10.128.1.155 | worker-01 | True  |

QEMU guest agent がインストールされた OS を確認します。

```
oc get vmi win2k25-vm-1 -n default -o jsonpath='{.status.guestOSInfo}' | python3 -m json.tool
```

結果

```
{
  "id": "mswindows",
  "kernelRelease": "10.0.26100",
  "name": "Microsoft Windows Server 2025 Datacenter",
  "prettyName": "Windows Server 2025 Datacenter",
  "version": "10.0",
  "versionId": "10.0"
}
```

**guestOSInfo** フィールドは、QEMU guest agent が OOB (Out of Band) モードで起動していることを示します。5

次に、VNC を使用して macOS の **virtctl** を VNC モードで起動します。--vnc-type

```
virtctl vnc win2k25-vm-1 -n default --vnc-type=tiger --vnc-path=/usr/local/bin/vncviewer
```

sysprep ConfigMap を Administrator として System Properties を更新し、hostname

RDP を Remote Access to Windows VMs via RDP を使用して接続します。

接続方法

接続先 URL:

1. Microsoft Evaluation Center から ISO の URL を取得 (ダウンロード可能な ISO の URL)



- 2. `autounattend.xml` を作成 (仮想マシンをインストールするための設定ファイル)
- 3. `win2k25-v2` の `baseDvName` を作成
- 4. `DataSource` を作成して VM をインストール

仮想マシンをインストールするための設定ファイル `DataSource` は (`win2k25-v1``win2k25-v2``win2k25-2025-07` など) のインストールファイル 1 をインストールするための設定ファイル

Windows のインストール

Windows のインストールは `PipelineRun` を使用して `autounattend.xml` をインストール

| OS          | preferenceName                   | autounattendConfigMapName               | VirtIO のインストール                                     |
|-------------|----------------------------------|-----------------------------------------|----------------------------------------------------|
| Windows 10  | <code>windows.10.virtio</code>   | <code>windows10-efi-autounattend</code> | <code>viostor/w10/amd64, NetKVM/w10/amd64</code>   |
| Windows 11  | <code>_(仮想マシン)_</code>           | <code>windows11-autounattend</code>     | <code>viostor/w11/amd64, NetKVM/w11/amd64</code>   |
| Server 2022 | <code>windows.2k22.virtio</code> | <code>windows2k22-autounattend</code>   | <code>viostor/2k22/amd64, NetKVM/2k22/amd64</code> |
| Server 2025 | <code>windows.2k25.virtio</code> | <code>windows2k25-autounattend</code>   | <code>viostor/2k25/amd64, NetKVM/2k25/amd64</code> |

仮想マシン OS のインストールには `autounattend.xml` の `ImageInstall` を使用して Windows 11 Enterprise の `Windows Server 2025 SERVERDATACENTER` のインストールを Windows 11 Enterprise のインストール

インストールのトラブルシューティング

| エラー                                                                                            | 原因                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>modify-windows-iso-file</code> の <code>"file not found: efisys_noprompt.bin"</code> のエラー | ISO のインストールに EFI のインストールに必要な <code>Microsoft ISO</code> のインストール ISO のインストール <code>Microsoft Evaluation Center</code> のインストール ISO のインストール                                                                                                                                                                                                                                                                        |
| ISO のインストール <code>Permission denied</code> のエラー                                                | 仮想マシン 4 のインストール <code>fsGroup: 107</code> のインストール <code>runAsUser: 107</code> のインストール <code>PipelineRun</code> の <code>taskRunSpecs</code> のインストール <code>anyuid SCC</code> のインストール <code>pipeline</code> のインストール                                                                                                                                                                                                |
| インストールのインストール                                                                                  | <code>PipelineRun</code> のインストール <code>timeouts.pipeline</code> のインストール <code>1</code> のインストール <code>oc get dv -w</code> のインストール <code>DataVolume</code> のインストール                                                                                                                                                                                                                                                |
| 仮想マシン VM のインストール <code>Oobe</code> のインストール                                                     | <code>sysprep</code> のインストール <code>ConfigMap</code> のインストール <code>autounattend.xml</code> のインストール <code>unattend.xml</code> のインストール <code>Windows</code> のインストール <code>autounattend.xml</code> のインストール <code>sysprep</code> のインストール <code>unattend.xml</code> のインストール <code>VM</code> のインストール <code>volumes.sysprep.configMap.name</code> のインストール <code>ConfigMap</code> のインストール <code>ConfigMap</code> のインストール |

|                                      |                                                                                                                                 |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| エラー                                  | 原因                                                                                                                              |
| Sysprep の "rearm count exceeded" エラー | Windows ISO の sysprep の rearm 回数を超えている。sysprep を実行する ISO の rearm 回数を調整する必要がある。                                                  |
| VM の起動エラー                            | Windows OOB 5 分以内の VM の起動時に <code>guestOSInfo</code> のエラーが発生する。                                                                 |
| OS のインストールエラー                        | <code>post-install.ps1</code> の QEMU guest agent のインストールが失敗する。VM の <code>services.msc</code> の QEMU Guest Agent をインストールする必要がある。 |

## クリーンアップ

以下のコマンドを実行する:

```
oc delete vm win2k25-vm-1 -n default
oc delete configmap win2k25-sysprep-unattend -n default
oc delete configmap windows2k25-autounattend -n default
oc delete datasources win2k25 -n default
oc delete dv win2k25 -n default
```

PipelineRun を削除する:

```
oc delete pipelinerun -l tekton.dev/pipeline=windows-efi-installer -n default
```

1 つの SCC を削除する:

```
oc adm policy remove-scc-from-user anyuid -z pipeline -n default
```

## 確認

以下の確認事項を確認する:

- VirtIO の sysprep の Windows Server 2025 の `autounattend.xml` ConfigMap
- Windows ISO の `windows-efi-installer` Tekton の
- DataVolume の DataSource
- sysprep の (RDP) の VM
- VM
- Windows の

11

- [Linux OS 仮想化環境構築 \(Linux\)](#)
- [VirtIO 仮想化 Windows VM](#)
- [RDP 経由 Windows VM 接続](#)
- [ArtifactHub 上 windows-efi-installer](#)
- [windows-efi-installer の README](#)
- [Red Hat Developer: Windows 仮想化環境構築](#)
- [KubeVirt Sysprep 実行](#)
- [Packer KubeVirt 実行](#)

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

OpenShift Virtualization OS

11

- 
- 
- 

□ □ □ □ □ □ □ □ □ □

**namespace** `[[[`

```
oc get namespace golden-images
```

□□□□□□:

| NAME          | STATUS | AGE |
|---------------|--------|-----|
| golden-images | Active | 5m  |

□ □ □ □ □ □ □ □

```
oc get all -n golden-images
```

**VM**   **VMI**

```
oc get vm,vmi -n golden-images
```

## DataVolume PVC

```
oc get dv,pvc -n golden-images
```

## DataSource

```
oc get datasource -n golden-images
```

1. VM

1: VM

VM

```
oc get vm base-fedora-vm -n golden-images
```

Table:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 2m  | Running | True  |

VMI

```
oc get vmi base-fedora-vm -n golden-images -o yaml
```

status.phase: Running - status.interfaces[].ipAddress (VM IP) - status.guestOSInfo.id: fedora (guest agent)

:

```
status:
  guestOSInfo:
    id: fedora
    kernelRelease: 6.11.4-301.fc41.x86_64
    kernelVersion: '#1 SMP PREEMPT_DYNAMIC Thu Oct 10 16:16:06 UTC 2024'
    name: Fedora Linux
    prettyName: Fedora Linux 41 (Cloud Edition)
    version: '41'
    versionId: '41'
  interfaces:
  - infoSource: domain, guest-agent, multus-status
    interfaceName: eth0
    ipAddress: 10.128.0.168
    ipAddresses:
```

```
- 10.128.0.168
- fd02::76
mac: 02:92:c2:00:00:06
name: default
queueCount: 1
phase: Running
```

```
oc get dv base-fedora-vm-disk -n golden-images
```

```
oc get dv base-fedora-vm-disk -n golden-images
```

| NAME                |           |          |          |     |
|---------------------|-----------|----------|----------|-----|
| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
| base-fedora-vm-disk | Succeeded | 100.0%   |          | 5m  |

| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
|---------------------|-----------|----------|----------|-----|
| base-fedora-vm-disk | Succeeded | 100.0%   |          | 5m  |

```
oc get pvc base-fedora-vm-disk -n golden-images
```

```
oc get pvc base-fedora-vm-disk -n golden-images
```

**Bound** Storage Class

| NAME                | STATUS       | VOLUME        | CAPACITY |
|---------------------|--------------|---------------|----------|
| ACCESS MODES        | STORAGECLASS | AGE           |          |
| base-fedora-vm-disk | Bound        | pvc-abc123... | 30Gi     |
| RWO                 | lvms-vg1     | 5m            |          |

| NAME                | STATUS       | VOLUME        | CAPACITY |
|---------------------|--------------|---------------|----------|
| ACCESS MODES        | STORAGECLASS | AGE           |          |
| base-fedora-vm-disk | Bound        | pvc-abc123... | 30Gi     |
| RWO                 | lvms-vg1     | 5m            |          |

## 2: VM 環境構築

**VM** □□□□□□□□□□

```
virtctl console base-fedora-vm -n golden-images
```

```
virtctl console base-fedora-vm -n golden-images
```

VM 名称:

```
VM 0000:

# nginx 000
systemctl status nginx
nginx -v

# 0000000000
cat /etc/nginx/nginx.conf
```

```
# nginx 確認
systemctl status nginx
nginx -v

# 設定ファイルを確認
cat /etc/nginx/nginx.conf
```

```
ls -la /usr/share/nginx/html/
```

**cloud-init** 確認 (確認)

```
cat /var/log/cloud-init.log
cat /var/log/cloud-init-output.log
```

ステップ 3: **VM** 確認

**VM** 確認

```
oc get vm base-fedora-vm -n golden-images
```

確認:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| base-fedora-vm | 10m | Stopped | False |

**VMI** 確認

```
oc get vmi base-fedora-vm -n golden-images 2>&1
```

確認:

```
Error from server (NotFound): virtualmachineinstances.kubevirt.io "base-fedora-vm" not found
```

ステップ 4: **DataSource** 確認

**DataSource** 確認

```
oc get datasource fedora-nginx-golden -n golden-images
```

確認:

| NAME                | AGE |
|---------------------|-----|
| fedora-nginx-golden | 1m  |

**DataSource** 確認

```
oc get datasource fedora-nginx-golden -n golden-images -o yaml
```

확인해보겠습니다:

```
spec:
  source:
    pvc:
      name: base-fedora-vm-disk
      namespace: golden-images
```

확인 5: 확인해보는 VM 확인

확인하는 VM 확인

```
oc get vm webserver-vm-1 webserver-vm-2 -n golden-images
```

확인결과:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| webserver-vm-1 | 2m  | Running | True  |
| webserver-vm-2 | 2m  | Running | True  |

확인하는 VM DataVolume 확인

```
oc get dv webserver-vm-1-disk webserver-vm-2-disk -n golden-images
```

확인결과:

| NAME                | PHASE     | PROGRESS | RESTARTS | AGE |
|---------------------|-----------|----------|----------|-----|
| webserver-vm-1-disk | Succeeded | 100.0%   |          | 2m  |
| webserver-vm-2-disk | Succeeded | 100.0%   |          | 2m  |

VM 확인하는 방법

```
oc get dv webserver-vm-1-disk -n golden-images -o jsonpath='{.spec.sourceRef}'
```

확인하는 DataSource 확인하는 방법:

```
{"kind":"DataSource","name":"fedora-nginx-golden","namespace":"golden-images"}
```

VM 확인

VM IP 확인하는 방법:

```
oc get vmi webserver-vm-1 webserver-vm-2 -n golden-images -o jsonpath='{range .items[*]}{.metadata.name}: {.status.interfaces[0].ipAddress}{"\n"}{end}'
```

Pod:

```
webserver-vm-1: 10.128.0.169
webserver-vm-2: 10.128.0.170
```

Pod nginx:

```
oc run test-curl --image=curlimages/curl:latest --rm -it --restart=Never -- curl
http://10.128.0.169
```

6:

PVC

```
oc get pvc -n golden-images
```

VM PVC

| NAME                | STATUS | VOLUME        | CAPACITY | ACCESS MODES |
|---------------------|--------|---------------|----------|--------------|
| base-fedora-vm-disk | Bound  | pvc-abc123... | 30Gi     | RWO          |
| webserver-vm-1-disk | Bound  | pvc-def456... | 30Gi     | RWO          |
| webserver-vm-2-disk | Bound  | pvc-ghi789... | 30Gi     | RWO          |

(LVM)

```
# LVM
oc get lvmvolumegroup -n openshift-storage -o yaml | grep -A 10 status
```

1: VM DataVolume ImportInProgress

:

```
oc get dv base-fedora-vm-disk -n golden-images
NAME                                PHASE                PROGRESS    AGE
```



|                     |                  |     |     |
|---------------------|------------------|-----|-----|
| base-fedora-vm-disk | ImportInProgress | 10% | 10m |
|---------------------|------------------|-----|-----|

00:

```
# CDI importer pod
oc get pods -n golden-images | grep importer

# importer
oc logs -n golden-images <importer-pod-name>
```

□□□□□□: - □□□ DataSource □□□□□ - □□□□□□□□□□□□□□□□□□□□ - □□□□□□□□□□ - Storage Class □□□

□□□:

- ## 1. DataSource

```
oc get datasource fedora -n openshift-virtualization-os-images
```

- ## 2. CDI Operator □□□□□□□□□□□□:

```
oc get pods -n openshift-cnv | grep cdi
```

3. □□□□□□□□□□:

```
oc get pvc -n golden-images
oc get lvmvolumegroup -n openshift-storage
```

## 2: VM 1000000 - DataVolume 1000000

00:

```
oc get vm base-fedora-vm -n golden-images
```

| NAME           | AGE | STATUS                  | READY |
|----------------|-----|-------------------------|-------|
| base-fedora-vm | 5m  | WaitingForVolumeBinding | False |

□□:

```
# DataVolume
oc get dv base-fedora-vm-disk -n golden-images

#
oc get events -n golden-images --sort-by='.lastTimestamp' | grep -i error
```

□□□:

- :

□□:

111

- #### 例 4: 虚拟机 0 的启动

597

□□:

```
# VM 启动
virtctl console webserver-vm-1 -n golden-images

# nginx 安装
which nginx
systemctl status nginx
```

□□□□□□: - DataSource □□□□□□ VM □□□□□□□□□□□□□□ - DataSource □□□□□ PVC □□□□□□□ - VM □□□□□□□□□□

□□□:

1. VM 000000000000000000000000:

```
virtctl console base-fedora-vm -n golden-images
# 
```

- ## 2. DataSource 连接 PVC 数据库连接池:

```
oc get datasource fedora-nginx-golden -n golden-images -o jsonpath='{.spec.source}'
```

[illegible]

- ### 3. 如何从 PVC 中读取 DataSource 数据

## □□ 5: DataSource □□□□□ PVC □□□□□□□

DataSource VM VM

□□:

```
oc get datasource fedora-nginx-golden -n golden-images -o yaml
```

```
spec.source.pvc.name base-fedora-vm-disk
```

000:

- ## 1. PVC DataSource:

```
oc delete datasource fedora-nginx-golden -n golden-images
oc create -f fedora-golden-datasource.yaml
```

ステップ 6: 確認

確認:

```
Failed to create PVC: insufficient storage
```

確認:

```
# LVM 確認
oc get lvmvolumegroup -n openshift-storage -o yaml

# PVC 確認
oc get pvc -n golden-images
```

確認:

1. 不要 PVC 削除:

```
oc delete pvc <unused-pvc> -n golden-images
```

2. LVM 確認 (確認)
3. VM/DataVolume spec 確認

ステップ 7: VM 確認

確認: VM の DataVolume が **Succeeded** かどうか確認

確認:

```
# DataVolume 確認
oc get dv webserver-vm-1-disk -n golden-images -w

# CDI pod 確認
oc logs -n golden-images -l app=containerized-data-importer --tail=100
```

確認: - 確認 - 確認 (DataSource 確認)

確認:

1. DataSource 確認:

```
spec:
  sourceRef:
    kind: DataSource
    name: fedora-nginx-golden
```

2. [OpenShift Storage Class](#) [\(LVM: LVM 101\)](#)

3. [VM 101](#)

OpenShift

OpenShift

```
# namespace
oc get all,dv,pvc,datasource -n golden-images

#
oc get all,dv,pvc,datasource -n golden-images -o wide

#
oc get vm,vmi,dv -n golden-images -w
```

OpenShift

```
#
oc get events -n golden-images --sort-by='.lastTimestamp' | tail -20

# VM
oc describe vm base-fedora-vm -n golden-images | tail -20

# CDI Operator
oc logs -n openshift-cnv -l app=cdi-operator --tail=50
```

OpenShift

```
# PVC
oc describe pvc <pvc-name> -n golden-images

# StorageClass
oc get storageclass -o yaml

# LVM (LVM 101)
oc get lvmvolumegroup,lvmvolumegroupnodestatus -n openshift-storage
```

OpenShift

- [OpenShift Virtualization](#)
- [KubeVirt - DataVolumes](#)
- [Containerized Data Importer \(CDI\)](#)
- [LVM](#)

VM □□□□□□

## 11

|           |                |  |    |
|-----------|----------------|--|----|
| OpenShift | Virtualization |  | VM |
| VM        |                |  | VM |
|           |                |  |    |

## VM □□□□□□

VM OpenShift Virtualization 2

- **Web** `docker-compose`: VM `docker-compose` & `docker`
- **CLI** `docker`: PVC `docker` VM `docker`

VM 0000000000:

- 00000000000000000000
- 00000000000000000000
- 00000000000000000000
- 00000000000000000000
- 00000000000000000000

101010101010

0000000000000000000000000000000000 VM 00000000000000000000

- **問題解決:** QA VM 問題解決 VM 問題解決
- **Web VM 10** 問題解決
- **namespace** 問題: VM namespace namespace RBAC namespace
- **VM DR namespace** VM
- **VM** VM

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

VM

|        |                         |                         |
|--------|-------------------------|-------------------------|
| OS     | VM の種類                  | インストール方法                |
| CentOS | CentOS 7 VM のインストール     | CentOS 7 VM のインストール     |
| Ubuntu | Ubuntu 18.04 VM のインストール | Ubuntu 18.04 VM のインストール |

|    |    |    |
|----|----|----|
| VM | VM | VM |
| VM | VM | VM |
| VM | VM | VM |
| VM | VM | VM |

## 

- OpenShift 4.18 VM OpenShift Virtualization Operator
- OpenShift Web CLI (oc, virtctl)
- Storage Class
- VM (VM)
- PersistentVolumeClaim (PVC)

## 

OpenShift Virtualization VM 2

## 

Web

- PVC
- VM
- VM
- VM (CPU)

## 

- VM PVC
- PVC VM
- namespace

## 

2

Web namespace







```
oc create namespace vm-cloning-demo
```

nginx をインストールして VM を作成する



cloud-init をインストールして SSH を接続する

**source-webserver-vm.yaml** を作成する

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: source-webserver
  namespace: vm-cloning-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: source-webserver-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: source-webserver
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
```

```

machine:
  type: pc-q35-rhel9.4.0
memory:
  guest: 4Gi
networks:
  - name: default
    pod: {}
volumes:
  - dataVolume:
      name: source-webserver-disk
    name: rootdisk
  - cloudInitNoCloud:
      userData: |-
        #cloud-config
        user: fedora
        password: fedora123
        chpasswd: { expire: false }
        ssh_pwauth: true
        runcmd:
          - dnf install -y nginx
          - systemctl enable --now nginx
          - echo "<h1>Source Webserver VM</h1><p>Hostname: $(hostname)</p>" >
            /usr/share/nginx/html/index.html
          - firewall-cmd --permanent --add-service=http
          - firewall-cmd --reload
      name: cloudinitdisk

```

VM を適用する

```
oc apply -f source-webserver-vm.yaml
```

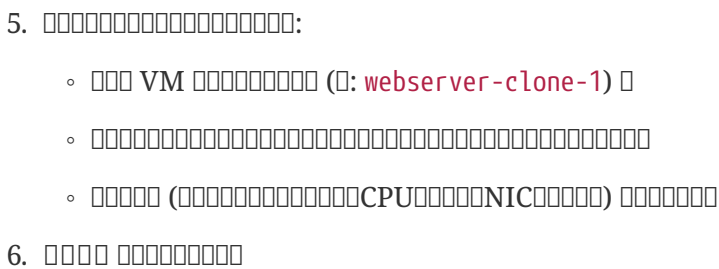
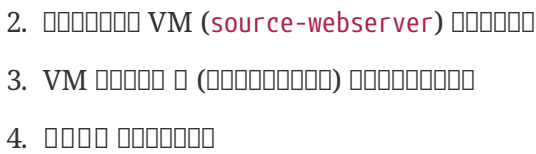
VM を取得する

```
oc get vmi -n vm-cloning-demo
```

**Web** を確認する

OpenShift Web コンソールで VM を確認する:

1. 左側メニュー → 仮想マシン



Clone VirtualMachine

Name \*

webserver-clone-1

Start VirtualMachine once created

☒

Configuration

Operating system

Fedora Linux 43 (Cloud Edition)

Flavor

2 CPU | 4 GiB Memory

Workload profile

Not available

NICs

default

Disks

rootdisk

cloudinitdisk

Request sent

In process

Created

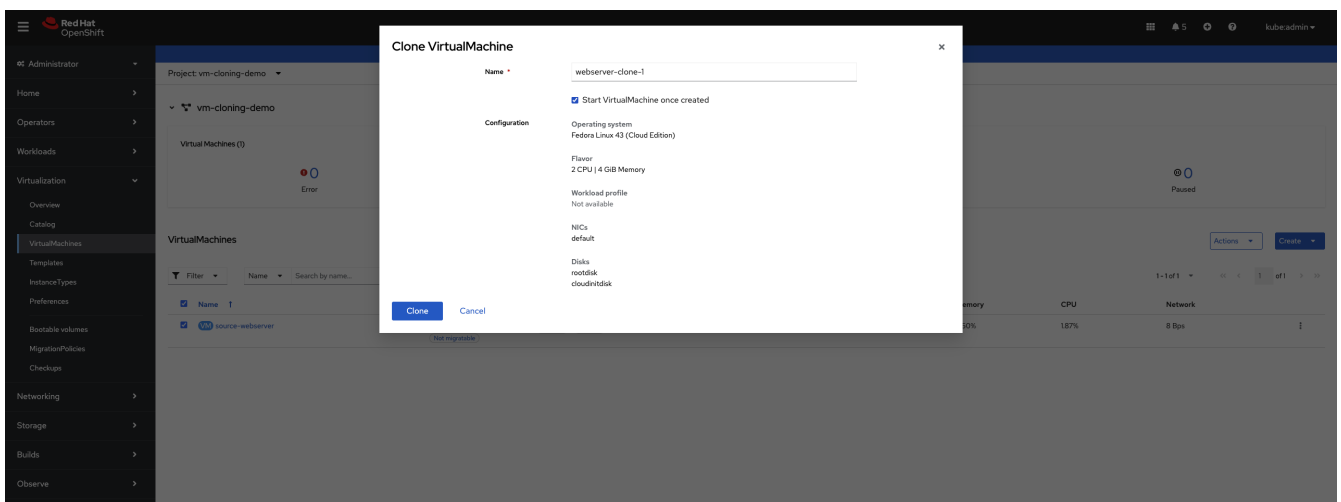
Clone

Cancel

Web

- VM PVC
- VM
- VM (CPU) VM
- VM

PVC Provisioning



Web

VM

```
oc get vm -n vm-cloning-demo
oc get vmi -n vm-cloning-demo
```

创建新的 VM 实例

```
virtctl console webserver-clone-1 -n vm-cloning-demo
```

创建新的 VM 实例 nginx 实例 (容器) 实例

## CLI 创建 VM 实例

创建新的 namespace 实例 CLI 实例

创建 1 个 PVC 实例 VM 实例 (容器)

创建 CLI 实例 VM 实例 PVC 实例

```
oc get pvc -n vm-cloning-demo
```

输出:

| NAME                  | STATUS       | VOLUME                                   | CAPACITY |
|-----------------------|--------------|------------------------------------------|----------|
| ACCESS MODES          | STORAGECLASS | AGE                                      |          |
| source-webserver-disk | Bound        | pvc-1234abcd-5678-efgh-9012-ijklmnop3456 | 30Gi     |
| RWO                   | lvms-vg1     | 10m                                      |          |

创建 PVC 实例 DataVolume 实例

clone-datavolume.yaml 实例

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: webserver-clone-2-disk
  namespace: vm-cloning-demo
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true" ①
spec:
  source:
    pvc:
      name: source-webserver-disk
      namespace: vm-cloning-demo
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 30Gi
    storageClassName: gp3-csi ②
```

① WaitForFirstConsumer によって Storage Class が

② Storage Class が (PVC が)



`cdi.kubevirt.io/storage.bind.immediate.requested:` `"true"`  
によって WaitForFirstConsumer によって Storage Class (AWS EBS gp3-csi Azure Disk ) によって VM によって DataVolume が PendingPopulation によって

DataVolume が

```
oc apply -f clone-datavolume.yaml
```

によって

```
oc get dv webserver-clone-2-disk -n vm-cloning-demo -w
```

によって

| NAME                   | PHASE                             | PROGRESS | RESTARTS | AGE |
|------------------------|-----------------------------------|----------|----------|-----|
| webserver-clone-2-disk | CloneFromSnapshotSourceInProgress | N/A      |          | 4s  |
| webserver-clone-2-disk | PrepClaimInProgress               | N/A      |          | 68s |
| webserver-clone-2-disk | RebindInProgress                  | N/A      |          | 78s |
| webserver-clone-2-disk | Succeeded                         | 100.0%   |          | 79s |

DataVolume が Succeeded によって VM が

vm-from-cloned-pvc.yaml によって

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-clone-2
  namespace: vm-cloning-demo
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-clone-2
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
```

```

        bus: virtio
        name: rootdisk
    interfaces:
      - masquerade: {}
        name: default
    rng: {}
    machine:
      type: pc-q35-rhel9.4.0
    memory:
      guest: 4Gi
    networks:
      - name: default
        pod: {}
    volumes:
      - name: rootdisk
        persistentVolumeClaim:
          claimName: webserver-clone-2-disk

```

VM を適用

```
oc apply -f vm-from-cloned-pvc.yaml
```

VM を起動

```
virtctl start webserver-clone-2 -n vm-cloning-demo
```

2つのVM を確認 (確認)



この操作は、1つのVM から別のVM を作成するためのものです。

確認された VM を停止

```
virtctl stop source-webserver -n vm-cloning-demo
```

VM を取得

```
oc get vm source-webserver -n vm-cloning-demo
```

VM の YAML を取得

```
oc get vm source-webserver -n vm-cloning-demo -o yaml > clone-base.yaml
```

この操作は、clone-base.yaml を作成するためのものです。

## 1. VM 配置:

```
metadata:
  name: source-webserver # 名称: webserver-manual-clone
  namespace: vm-cloning-demo
```

## 2. DataVolume 配置PVC 配置:

```
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: source-webserver-disk # 名称: webserver-manual-clone-disk
      spec:
        # 指定 sourceRef 指向 DataVolume
        # 指定 DataVolume 名称:
        source:
          pvc:
            name: source-webserver-disk
            namespace: vm-cloning-demo
        storage:
          resources:
            requests:
              storage: 30Gi
```

## 3. 配置虚拟机模板:

```
spec:
  template:
    metadata:
      labels:
        kubevirt.io/domain: source-webserver # 名称: webserver-manual-clone
    spec:
      domain:
        # ... 配置
      volumes:
        - dataVolume:
            name: source-webserver-disk # 名称: webserver-manual-clone-disk
            name: rootdisk
```

## 4. 配置虚拟机镜像:

配置虚拟机镜像时，需要指定以下元数据：  
`metadata.uid` \* `metadata.resourceVersion` \*  
`metadata.creationTimestamp` \* `metadata.generation` \* `status` (可选)

## 5. MAC 地址配置 (可选):



macAddress 指定する

```
spec:
  template:
    spec:
      domain:
        devices:
          interfaces:
            - masquerade: {}
              name: default
              macAddress: "52:54:00:xx:xx:xx" # 任意
```

MAC 指定時に `Failed to allocate mac to the vm object: failed to allocate requested mac address`

YAML を適用する

```
oc apply -f clone-base.yaml
```

## cloud-init VM を作成する

VM を作成する際に `cloud-init` を指定する

作成する VM の設定

DataVolume を指定する (2 GiB)

```
cat <<EOF | oc apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: webserver-custom-disk
  namespace: vm-cloning-demo
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
    pvc:
      name: source-webserver-disk
      namespace: vm-cloning-demo
  storage:
    accessModes:
      - ReadWriteOnce
  resources:
    requests:
      storage: 30Gi
EOF
```

■■■■■■■■■■■■■■■■■■■■

```
oc get dv webserver-custom-disk -n vm-cloning-demo -w
```

cloud-init ■■■■■■■■■■ VM ■■■■■■■■■■

**cloned-vm-with-cloudinit.yaml** ■■■■■■■■■■

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-custom
  namespace: vm-cloning-demo
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-custom
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        machine:
          type: pc-q35-rhel9.4.0
        memory:
          guest: 4Gi
      networks:
        - name: default
          pod: {}
      volumes:
        - name: rootdisk
          persistentVolumeClaim:
            claimName: webserver-custom-disk
        - cloudInitNoCloud:
            userData: |-
              #cloud-config
```

```
hostname: webserver-custom
fqdn: webserver-custom.local
manage_etc_hosts: true
password: fedora123
chpasswd: { expire: false }
ssh_pwauth: true
runcmd:
  - hostnamectl set-hostname webserver-custom
  - echo "<h1>Customized Clone</h1><p>Hostname: $(hostname)</p>" >
/usr/share/nginx/html/index.html
  - systemctl restart nginx
name: cloudinitdisk
```

```
## cloud-init ##
```

- `webserver-custom` 設定
- `fedora` 環境構築
- `nginx` 設定
- 別の VM へ `nginx` 設定

□□□□□□□□□□

```
oc apply -f cloned-vm-with-cloudinit.yaml
virtctl start webserver-custom -n vm-cloning-demo
```

□□□□□□□□□□□□

```
virtctl console webserver-custom -n vm-cloning-demo
```

VM □:

```
hostname
# 例: webserver-custom

systemctl status nginx
# nginx.service (active)

curl localhost
# 200 OK
```

## namespace 命名空间

名前空間 (namespace) は、namespace 名 VM 内で定義される。

namespace

```
oc create namespace vm-cloning-production
```

namespace RBAC

namespace namespace PVC

cross-namespace-clone-rbac.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: datavolume-cloner
  namespace: vm-cloning-demo
rules:
  - apiGroups: [""]
    resources: ["persistentvolumeclaims"]
    verbs: ["get", "list"]
  - apiGroups: ["cdi.kubevirt.io"]
    resources: ["datavolumes"]
    verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: allow-clone-from-demo
  namespace: vm-cloning-demo
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: datavolume-cloner
subjects:
  - kind: ServiceAccount
    name: default
    namespace: vm-cloning-production
```

RBAC

```
oc apply -f cross-namespace-clone-rbac.yaml
```

namespace

demo namespace DataVolume production namespace

cross-namespace-clone-datavolume.yaml

```

apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: webserver-production-disk
  namespace: vm-cloning-production
  annotations:
    cdi.kubevirt.io/storage.bind.immediate.requested: "true"
spec:
  source:
    pvc:
      name: source-webserver-disk
      namespace: vm-cloning-demo
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 30Gi

```

DataVolume を適用

```
oc apply -f cross-namespace-clone-datavolume.yaml
```

確認コマンド

```
oc get dv -n vm-cloning-production -w
```

VM を作成

**cloned-webserver-production.yaml** を適用

```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: webserver-production
  namespace: vm-cloning-production
  labels:
    environment: production
spec:
  runStrategy: Manual
  template:
    metadata:
      labels:
        kubevirt.io/domain: webserver-production
    spec:
      domain:
        cpu:

```

```

    cores: 4
  devices:
    disks:
      - disk:
          bus: virtio
          name: rootdisk
      - disk:
          bus: virtio
          name: cloudinitdisk
    interfaces:
      - masquerade: {}
        name: default
    rng: {}
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 8Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - name: rootdisk
      persistentVolumeClaim:
        claimName: webserver-production-disk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          hostname: webserver-prod
          fqdn: webserver-prod.production.local
          manage_etc_hosts: true
          password: fedora123
          chpasswd: { expire: false }
          ssh_pwauth: true
          runcmd:
            - hostnamectl set-hostname webserver-prod
            - echo "<h1>Production Webserver</h1><p>Environment: Production</p>" >
              /usr/share/nginx/html/index.html
            - systemctl restart nginx
        name: cloudinitdisk

```

□□□□□□□□:

- □□□ namespace
- □□□□□□□□ (4 □□□8Gi RAM)
- □□□□□□□□
- □□□□□□□□□□□□
- □□□□□□□□□□

□□□□□□□□□□

```
oc apply -f cloned-webserver-production.yaml
virtctl start webserver-production -n vm-cloning-production
```

□ □ □ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □

100

- **Smart cloning** (CSI 0000000000): 000000000000000000000000
  - Ceph RBD, AWS EBS, Azure Disk, GCP PD
  - 0000: `SnapshotForSmartCloneInProgress` → `SmartClonePVCInProgress`
- **Host-assisted cloning**: 000Pod 000000000000
  - 00000000000000000000 Storage Class
  - 0000: 0000000000 `CloneInProgress`

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc get dv <datavolume-name> -n <namespace> -o yaml | grep cloneType
```

00:

```
cdi.kubevirt.io/cloneType: snapshot # Smart cloning
#
cdi.kubevirt.io/cloneType: copy      # Host-assisted cloning
```

□ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- 00000000000000000000000000000000
- 00000000000000000000000000000000
- 000000: (000000 x 000000000000)
- namespace 000000 Storage Class 000000000000

1111111111111111

```
oc get sc
oc describe sc <storage-class-name>
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- Storage Class
- namespace
- namespace
- namespace
- namespace

□ □ □ □ □ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □

DataVolume 0000000000000000

```
oc get dv -n vm-cloning-demo
```

Success

| NAME                  | PHASE     | PROGRESS | AGE |
|-----------------------|-----------|----------|-----|
| webserver-custom-disk | Succeeded | N/A      | 5m  |

□ □ □ □ □ □ □ □ □ □ □ □

□□□□□□□□□□□□□□

```
oc describe dv <datavolume-name> -n <namespace>
```

□□□□□□□□□□□□□□□□

Events:

| Type   | Reason          | Age | From                  | Message                                                        |
|--------|-----------------|-----|-----------------------|----------------------------------------------------------------|
| Normal | CloneInProgress | 5m  | datavolume-controller | Cloning from vm-cloning-demo/source-webserver-disk in progress |
| Normal | CloneSucceeded  | 2m  | datavolume-controller | Successfully cloned from vm-cloning-demo/source-webserver-disk |

VM 

```
oc get vmi -n vm-cloning-demo
```



VM 0000000000000000

VM □:

[illegible]

00: DataVolume 0000 CloneInProgress 00000000

000:

Host-assisted cloning

permission denied

命名空间: namespace 授权: RBAC 授权策略

root: RBAC 配置

```
# 创建角色
oc get role datavolume-cloner -n <source-namespace>

# 创建角色绑定
oc get rolebinding -n <source-namespace>

# 授权
oc auth can-i get pvc --as=system:serviceaccount:<target-namespace>:default -n
<source-namespace>
```

root yes

创建 VM 模板

root: 创建 VM 模板

root: VM 模板 cloud-init 配置

- **cloudinitdisk** 配置
- cloud-init 模板 **userData** 模板 **hostname** 配置
- **runcmd** 配置: **hostnamectl set-hostname <new-name>**
- cloud-init 配置: **sudo cat /var/log/cloud-init.log**

创建 VM 模板

root: 创建 VM 模板

root:

```
# 创建资源配额
oc get resourcequota -n <namespace>
oc describe resourcequota -n <namespace>

# 编辑资源配额
oc edit resourcequota <quota-name> -n <namespace>
```

创建 PVC 模板

root: DataVolume 模板 PVC 配置

root:

```
# 创建 PVC 模板
oc get pvc -n <source-namespace>
```

```
# DataVolume
oc get dv <datavolume-name> -n <namespace> -o yaml
```

`spec.source.pvc.name` `spec.source.pvc.namespace`

VM

: DataVolume VM SchedulingStarting

: \* PVC \* \*

:

```
# PVC
oc get pvc -n <namespace>

# VM
oc describe vm <vm-name> -n <namespace>

# VMI ( )
oc describe vmi <vm-name> -n <namespace>
```

```
# demo namespace VM
oc delete vm --all -n vm-cloning-demo

# demo namespace DataVolume
oc delete dv --all -n vm-cloning-demo

# VM
oc delete vm webserver-production -n vm-cloning-production
oc delete dv --all -n vm-cloning-production

# RBAC
oc delete role datavolume-cloner -n vm-cloning-demo
oc delete rolebinding allow-clone-from-demo -n vm-cloning-demo

# namespace
oc delete namespace vm-cloning-demo
oc delete namespace vm-cloning-production
```

```
oc get namespace | grep vm-cloning
```

111

□ □ □ □ □ □ □ □ □ □ □ □ □ □ :

- OpenShift Virtualization **VM** **Container**
- Web **CLI** **API**
- OpenShift Web **VM** **Container**
- **DataVolume** **CLI** **API**
- **cloud-init** **cloud-init**
- **RBAC** **namespace** **VM** **Container**
- **Smart cloning** **Host-assisted cloning**
- **ID** **ID**
- **VM** **Container**

# VM OpenShift Virtualization

11

- PVC 仮想マシン 仮想

## VM □□□□□□□□

11

[illegible]

# OpenShift Virtualization VM

□□□□

- VM 管理 (Virtual Machine)
- Web 管理 (Web Console)
- 管理 (Management)
- 管理 (Management)

□□□□□□□□□□:

## VM 快照 (VolumeSnapshot)

VM 快照 (VolumeSnapshot) は、VM のディスクデータをバックアップするための機能です。

- **VM 快照 (VolumeSnapshot):** OS のディスクデータをバックアップするための機能です。VM 快照 (VolumeSnapshot) は、VM のディスクデータをバックアップするための機能です。
- **VM 快照 (VolumeSnapshot):** VM のディスクデータをバックアップするための機能です。
- **VM 快照 (VolumeSnapshot):** VM のディスクデータをバックアップするための機能です。
- **VM 快照 (VolumeSnapshot):** VM のディスクデータをバックアップするための機能です。
- **VM 快照 (VolumeSnapshot):** VM のディスクデータをバックアップするための機能です。



VM 快照 (VolumeSnapshot) は、VM のディスクデータをバックアップするための機能です。VM 快照 (VolumeSnapshot) は、VM のディスクデータをバックアップするための機能です。

## インストール

- OpenShift Virtualization をインストール (OpenShift 4.18 以降)
- VM 快照 (VolumeSnapshot) をインストール
- CSI 快照 (VolumeSnapshot) Storage Class
- `oc` コマンド `virtctl` CLI をインストール

VM 快照 (VolumeSnapshot) のインストール方法

VM 快照 (VolumeSnapshot) のインストール方法 CSI 快照 (VolumeSnapshot) のインストール方法

```
oc get volumesnapshotclass
```

結果:

| NAME                                   | DRIVER                             | DELETIONPOLICY | AGE |
|----------------------------------------|------------------------------------|----------------|-----|
| csi-hostpath-snapclass                 | hostpath.csi.k8s.io                | Delete         | 30d |
| ocs-storagecluster-rbdplugin-snapclass | openshift-storage.rbd.csi.ceph.com | Delete         | 30d |

VolumeSnapshotClass は、VM 快照 (VolumeSnapshot) のインストール方法

## VM 快照 (VolumeSnapshot) のインストール

OpenShift Virtualization

OpenShift Virtualization 2

VirtualMachineSnapshot

VM VirtualMachineSnapshotContent

VirtualMachineSnapshotContent

VirtualMachineSnapshot VolumeSnapshot

VirtualMachineSnapshotContent

VM Snapshot-1 VM Snapshot-2 VM Snapshot-3 VM Snapshot-4



VirtualMachineSnapshotContent

VirtualMachineSnapshotContent

| VM | VM | VM |
|----|----|----|
| VM | VM | VM |
| VM | VM | VM |



QEMU guest agent

5 Failed (thaw)
 VM VirtualMachineSnapshot spec failureDeadline (failureDeadline: 10m)

(Indication)

status.indications

| Online        | VM               |
|---------------|------------------|
| GuestAgent    | QEMU guest agent |
| NoGuestAgent  | QEMU guest agent |
| QuiesceFailed | guest agent      |

Indication

```

oc get virtualmachinesnapshot <snapshot-name> -n vm-snapshots -o json | jq
'.status.indications'
```

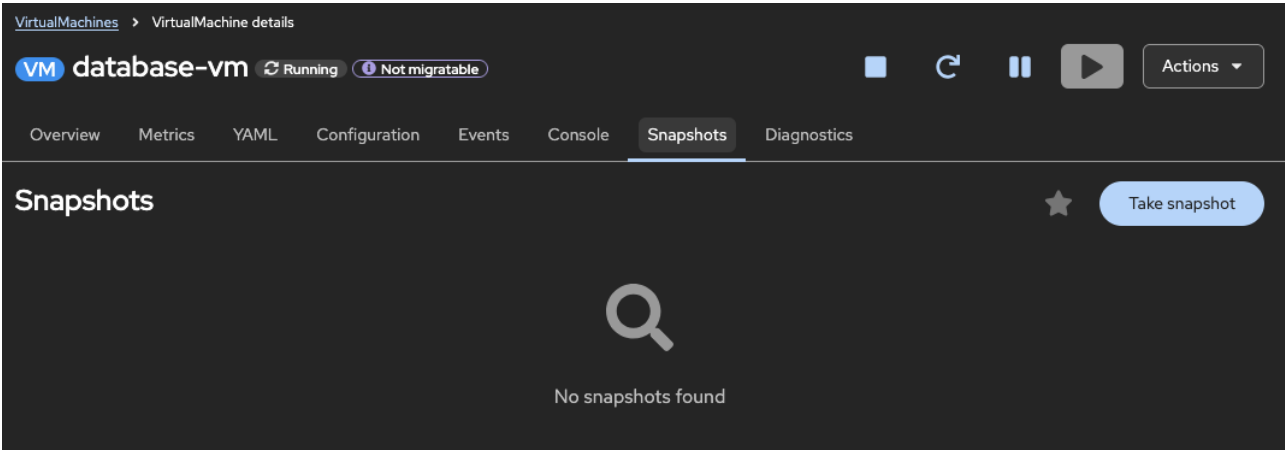
:

```

[
  "GuestAgent",
  "Online"
]
```

Web

1. Virtualization → VirtualMachines
2. VM
3. Snapshots



4. Take snapshot

Take snapshot

Taking snapshot of running VirtualMachine.

Name

database-vm-snapshot-20260304-155755

Description

Deadline

600

Seconds (s)

Disks included in this snapshot (1)

rootdisk

The following disk will not be included in the snapshot

cloudinitdisk - Snapshot is not supported for this volumeSource type [cloudinitdisk]

Edit the disk or contact your cluster admin for further details.

[Learn more about snapshots](#)

Save

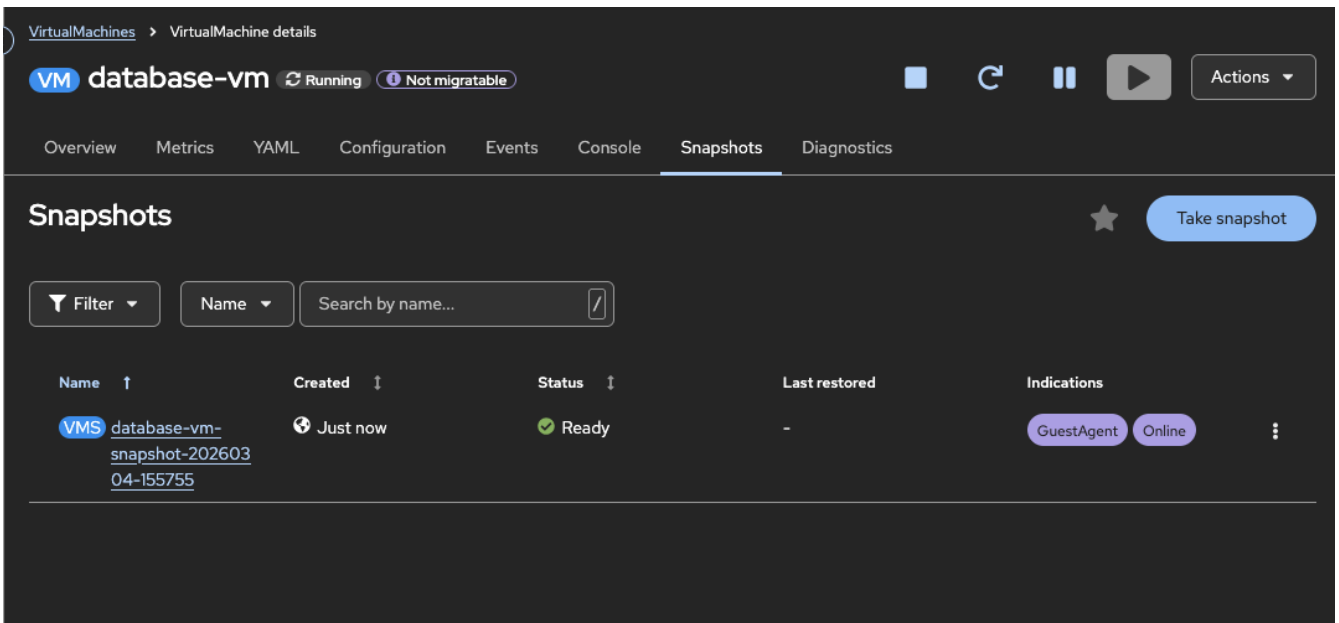
Cancel

5.

6. Save

Snapshots





CLI

VirtualMachineSnapshot

vm-snapshot.yaml

```
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
  name: database-vm-snapshot-01
  namespace: vm-snapshots
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: database-vm
```

```
oc apply -f vm-snapshot.yaml
```

```
oc get virtualmachinesnapshot -n vm-snapshots
```

| NAME | SOURCEKIND | SOURCENAME | PHASE |
|------|------------|------------|-------|
|------|------------|------------|-------|

| READYTOUSE                           | CREATIONTIME   | ERROR       |           |      |  |
|--------------------------------------|----------------|-------------|-----------|------|--|
| database-vm-snapshot-20260304-155755 | VirtualMachine | database-vm | Succeeded | true |  |

: \_\_\_\_\_

```
oc describe virtualmachinesnapshot <snapshot-name> -n vm-snapshots
```

□□□:

```
Name:          database-vm-snapshot-20260304-155755
Namespace:    vm-snapshots
API Version:  snapshot.kubevirt.io/v1beta1
Kind:         VirtualMachineSnapshot
Spec:
  Failure Deadline: 600s
  Source:
    API Group:  kubevirt.io
    Kind:       VirtualMachine
    Name:       database-vm
Status:
  Conditions:
    Reason: Operation complete
    Status: False
    Type:   Progressing
    Reason: Ready
    Status: True
    Type:   Ready
  Indications:
    GuestAgent
    Online
  Phase:      Succeeded
  Ready To Use: true
  Snapshot Volumes:
    Excluded Volumes:
      cloudinitdisk
    Included Volumes:
      rootdisk
```

- **Phase:** InProgress→Succeeded→→ Failed
- **ReadyToUse:** true (Web Component Status is Ready)
- **Error:**

□ □ □ □ □ □ □ □ □ □ □ □

**namespace** `xxxxxxxxxxxxxxxx`:

```
oc get virtualmachinesnapshot -n vm-snapshots
```

namespace 命名空间:

```
oc get virtualmachinesnapshot -A
```

**Web**

1. VM 仮想マシン
2. **Snapshots** 仮想マシンスナップショット
3. 仮想マシンスナップショットの作成と削除

VirtualMachines > VirtualMachine details

VM database-vm

Running

Not migratable

Actions

Overview

Metrics

YAML

Configuration

Events

Console

Snapshots

Diagnostics

Snapshots

★

Take snapshot

Filter

Name

Search by name...

| Name ↑                                                      | Created ↑           | Status ↑         | Last restored | Indications                                         |
|-------------------------------------------------------------|---------------------|------------------|---------------|-----------------------------------------------------|
| <div>database-vm-</div> <div>snapshot-20260304-155755</div> | <div>Just now</div> | <div>Ready</div> | -             | <div>GuestAgent</div> <div>Online</div> <div></div> |

□ □ □ □ □ □ □ □ □ □ □ □

```
oc describe virtualmachinesnapshot <snapshot-name> -n vm-snapshots
```

VolumeSnapshot:

```
oc get virtualmachinesnapshotcontent -n vm-snapshots
```

□□□:

| NAME<br>ERROR                                           | READYTOUSE | CREATIONTIME |
|---------------------------------------------------------|------------|--------------|
| vmsnapshot-content-cd166db8-00be-4aa9-972e-dea669527bfd | true       | 8m           |

```
oc get volumesnapshot -n vm-snapshots
```

결과:

| NAME                                                            | SOURCEPVC | RESTORESIZE | SNAPSHOTCLASS | CREATIONTIME | READYTOUSE |
|-----------------------------------------------------------------|-----------|-------------|---------------|--------------|------------|
| vmsnapshot-cd166db8-00be-4aa9-972e-dea669527bfd-volume-rootdisk |           |             |               |              | true       |
| database-vm-disk                                                |           | 30Gi        | lvms-vg1      | 8m           | 8m         |

데이터베이스 백업

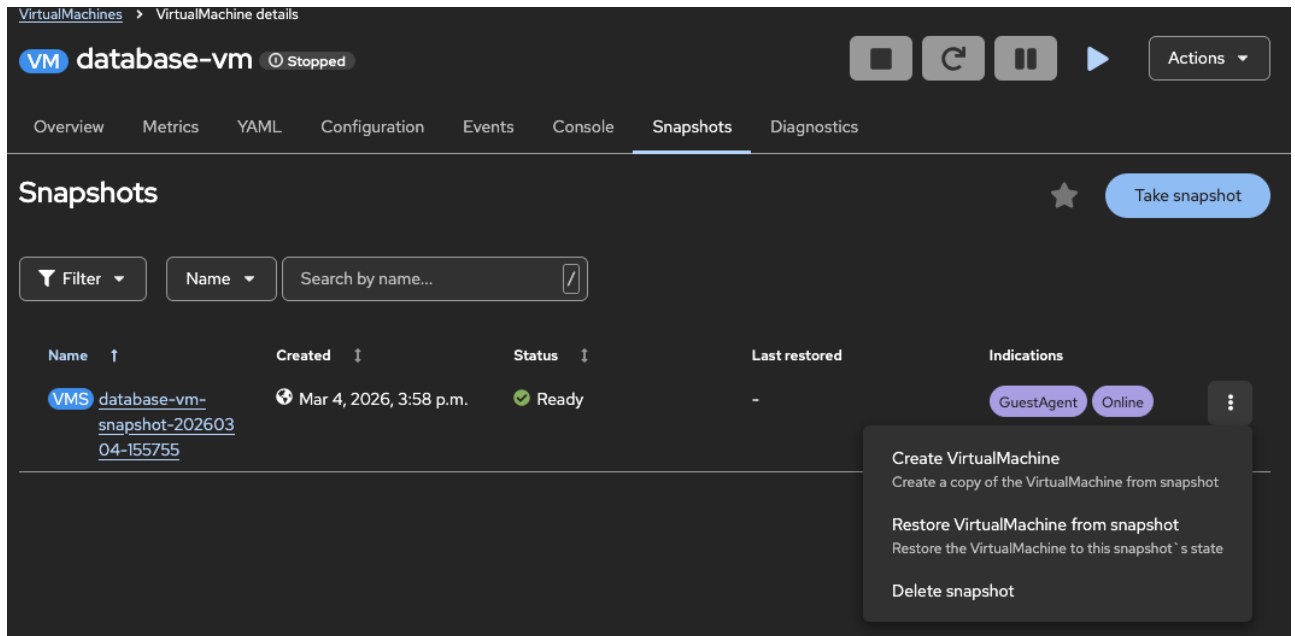
데이터베이스 VM 백업 VM 백업 데이터베이스 백업 데이터베이스



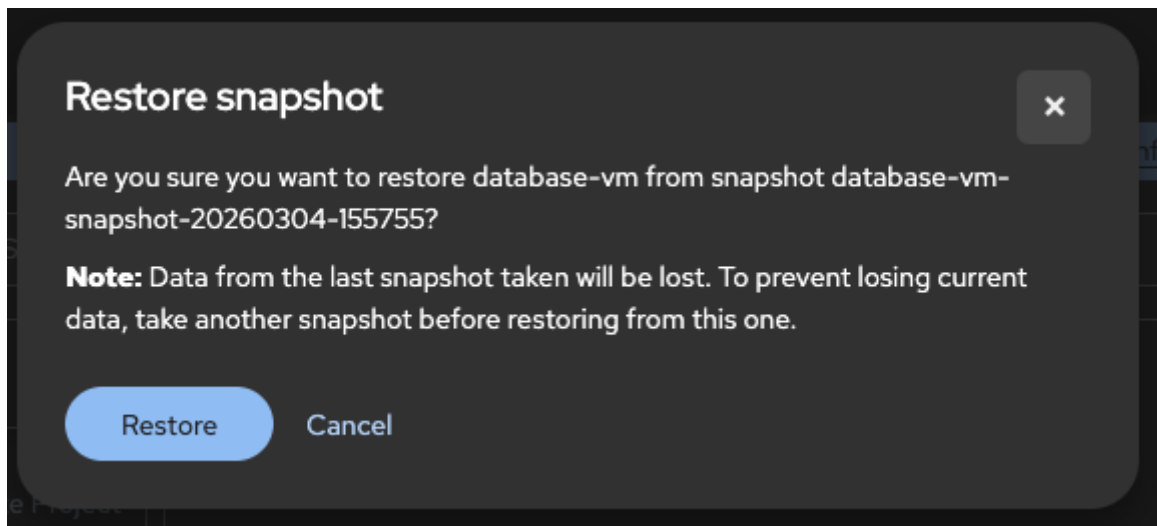
데이터베이스 백업 VM 백업 데이터베이스 VirtualMachineRestore spec  
 targetReadinessPolicy: StopTarget  
 FailImmediate (오류) WaitGracePeriod (5 초) WaitEventually (오류)  
 데이터베이스 백업 데이터베이스 백업 데이터베이스

**Web** 데이터베이스 백업

1. VM 백업 데이터베이스 백업
2. VM 백업 데이터베이스 백업 (데이터베이스 백업) 백업
3. **Snapshots** 데이터베이스 백업
4. 데이터베이스 백업 데이터베이스 백업 (3 초) 백업 데이터베이스
5. **Restore VirtualMachineSnapshot** 데이터베이스 백업



6.



CLI

VirtualMachineRestore

vm-restore.yaml

```
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineRestore
metadata:
  name: database-vm-restore-01
  namespace: vm-snapshots
spec:
  target:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: database-vm
  virtualMachineSnapshotName: database-vm-snapshot-01
```

確認コマンド

```
oc apply -f vm-restore.yaml
```

確認

仮想マシンリストを確認

```
oc get virtualmachinerestore -n vm-snapshots
```

結果:

| NAME                   | TARGETKIND     | TARGETNAME  | COMPLETE | RESTORETIME | ERROR |
|------------------------|----------------|-------------|----------|-------------|-------|
| database-vm-restore-01 | VirtualMachine | database-vm | true     | 2m          |       |

仮想マシンの詳細を確認:

```
oc describe virtualmachinerestore database-vm-restore-01 -n vm-snapshots
```

仮想マシンVMの起動

```
virtctl start database-vm -n vm-snapshots
```

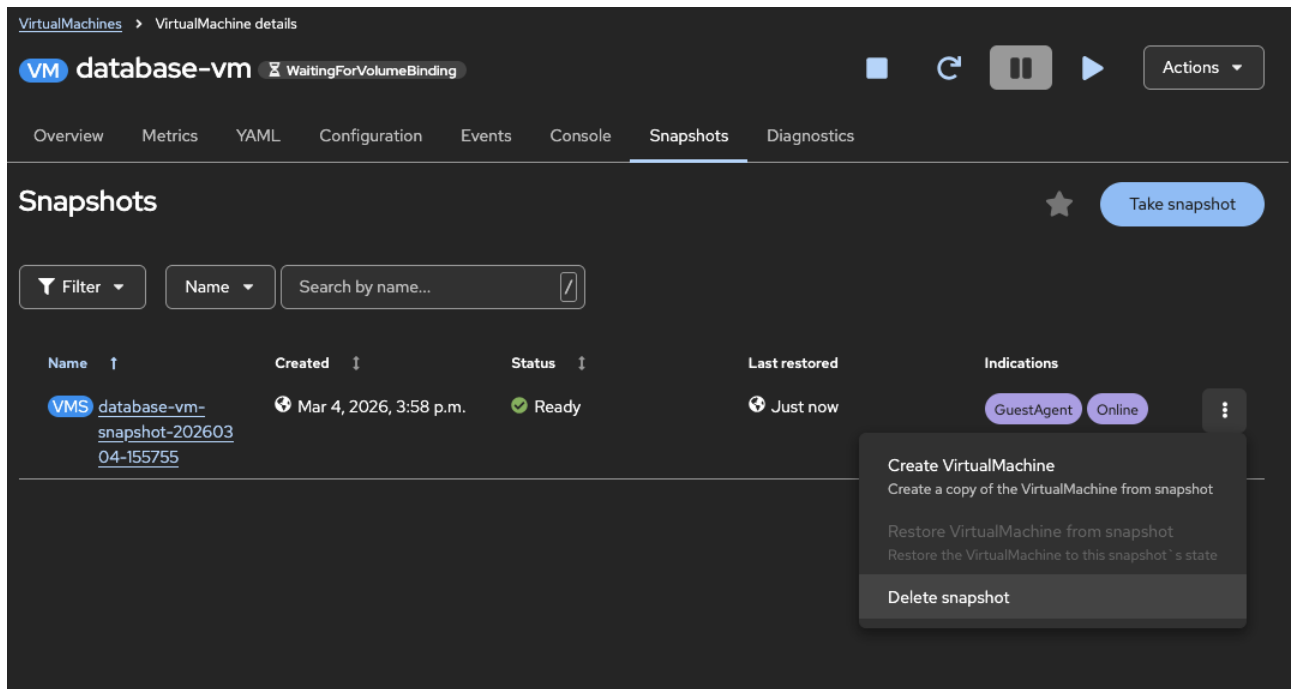
VMの起動を確認

```
oc get vm database-vm -n vm-snapshots
```

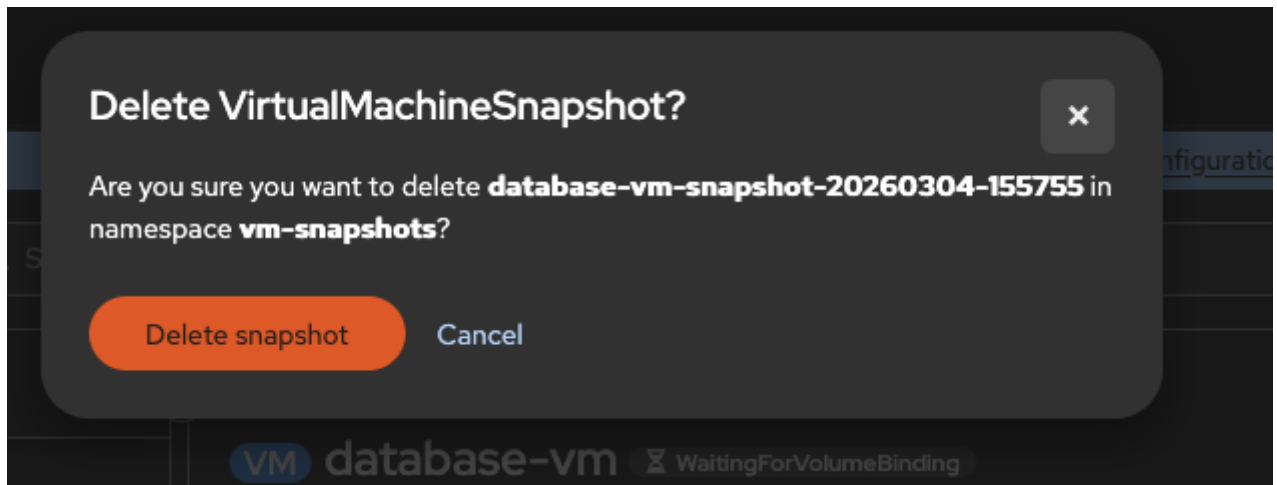
確認コマンド

**Web** 確認コマンド

1. VM の **Snapshots** 確認
2. 仮想マシンを再起動
3. **Delete snapshot** 確認



#### 4. 削除



#### CLI 実行

```
oc delete virtualmachinesnapshot database-vm-snapshot-01 -n vm-snapshots
```



VirtualMachineSnapshot は VirtualMachineSnapshotContent を参照して VolumeSnapshot を作成します。

#### 関連項目

#### CSI 概要

VM は CSI (Container Storage Interface) を使用してストレージを管理します。

- Red Hat OpenShift Data Foundation (ODF)

- AWS EBS CSI
- Microsoft Azure Disk CSI
- Google Cloud Persistent Disk CSI
- LVM Storage (LVMS) (aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa)
- VMware vSphere CSI
- NetApp Trident
- Pure Storage
- Dell CSI aaaaaa

## VolumeSnapshotClass aaaa

VolumeSnapshotClass aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaStorage Class aaaaaa snapshot class aaaaaaaaaaaaaaaaaa

```
# Storage Class aaaa
oc get storageclass

# aaaaa snapshot class aaaa
oc get volumesnapshotclass
```

aaaaaaaaaaaaaaaaaaaaa snapshot class a storage provisioner aaaaaaaaaaaaaaaaaaaaaa

aaaaaaaaaa

aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

- aaaaaaaa: aaaaa CSI aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
- aaaaaaa: VM aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
- aaaaaaaaaaaaaa: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
- aaaaaaaaaaaaaa: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

```
oc get volumesnapshot -n vm-snapshots -o wide
```

aaaa:

| NAME                                                            |             |               |              | READYTOUSE |
|-----------------------------------------------------------------|-------------|---------------|--------------|------------|
| SOURCEPVC                                                       | RESTORESIZE | SNAPSHOTCLASS | CREATIONTIME | AGE        |
| vmsnapshot-cd166db8-00be-4aa9-972e-dea669527bfd-volume-rootdisk |             |               |              | true       |
| database-vm-disk                                                | 30Gi        | lvms-vg1      | 9m           | 9m         |





## VolumeSnapshotClass

例: VolumeSnapshotClass

例: Storage Class

```
oc get storageclass -o jsonpath='{range
.items[*]}{.metadata.name}{"\t"}{.provisioner}{"\n"}{end}'
oc get volumesnapshotclass -o jsonpath='{range
.items[*]}{.metadata.name}{"\t"}{.driver}{"\n"}{end}'
```

VolumeSnapshotClass driver StorageClass provisioner

```
oc delete virtualmachinerestore --all -n vm-snapshots
oc delete virtualmachinesnapshot --all -n vm-snapshots
```

```
oc get virtualmachinesnapshot -n vm-snapshots
oc get volumesnapshot -n vm-snapshots
```

- OpenShift Virtualization VM
- Indication
- Web CLI
- VM
- VM
- VM

- Virtctl - VM CLI
- VM
- VM (VM)

- [VM 管理 - Red Hat](#)
- [Snapshot Restore API - KubeVirt](#)

# VM 管理

VM

OpenShift Virtualization は VM を管理するためのプラットフォームです。OS を管理するためのプラットフォームです。

VM は BIOS または UEFI を使用して OpenShift Virtualization を管理します。

管理

- OpenShift Virtualization Operator を OpenShift 4.18 にインストール
- CLI ツール: `oc`, `virtctl`, `kubectl`
- VM の管理
- Storage Class
- 管理: Quay.io/Docker Hub

## OpenShift Virtualization の管理

OpenShift Virtualization の管理

| 管理                           | 管理                                | 管理         |
|------------------------------|-----------------------------------|------------|
| PersistentVolume Claim (PVC) | 管理                                | VM の OS    |
| ContainerDisk                | 管理                                | VM の CI/CD |
| DataVolume                   | Containerized Data Importer (CDI) | HTTP/S3    |
| CloudInitNoCloud             | 管理 (Quay.io/Docker Hub)           | VM の管理     |

管理

OpenShift Virtualization 2 の管理

### BIOS (Legacy Mode)

- PC BIOS
- 管理
- (MBR 2TB)

- 環境構築
- 簡単な **Linux** 環境構築

## UEFI (Unified Extensible Firmware Interface)

- 日本語を英語に翻訳する
- 英語を日本語に翻訳する
- GPT を利用して日本語を英語に翻訳する (2TB の制限あり)
- Windows 11 を利用して Windows Server をインストールする
- 英語を日本語に翻訳する

□ □ □ □ □ □ □ □ □ □ □ □

VM KubeVirt bootOrder

□□□□□:

- **bootOrder** 00000000000000000000 (1 0 2 00000000)
- 00000000000000000000000000000000
- **bootOrder** 00000000000000000000000000000000
- 00000000000000000000 **bootOrder: 1** 00000000

## ❖❖❖❖ 1: PersistentVolumeClaim (PVC) ❖❖❖❖ (❖❖❖❖❖❖)

□□□□□□□□□□□□□□□□ VM □□□□□□□□

namespace

```
oc create namespace boot-demo
```



## cloud-init SSH 接続

## vm-boot-from-pvc.yaml

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-boot-pvc
  namespace: boot-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
```

```

    name: vm-boot-pvc-disk
spec:
  sourceRef:
    kind: DataSource
    name: fedora
    namespace: openshift-virtualization-os-images
  storage:
    resources:
      requests:
        storage: 30Gi
runStrategy: RerunOnFailure
template:
  metadata:
    labels:
      kubevirt.io/domain: vm-boot-pvc
spec:
  domain:
    cpu:
      cores: 2
    devices:
      disks:
        - disk:
            bus: virtio
            name: rootdisk
            bootOrder: 1
        - disk:
            bus: virtio
            name: cloudinitdisk
      interfaces:
        - masquerade: {}
          name: default
      rng: {}
    machine:
      type: pc-q35-rhel9.4.0
    memory:
      guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: vm-boot-pvc-disk
        name: rootdisk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd: { expire: False }
          ssh_pwauth: True

```

```
name: cloudinitdisk
```

□□□□□□:

- rootdisk □ **bootOrder: 1** □□□□□□□□□□□□□□□□
- DataVolume □□□□□ PVC □□□□□□□□□□□□□□□□
- cloud-init □□□□□□□□□□□□□□□□□□□□
- □□□□□□□□□□VM □□□□□□□□□□

□□□□□□:

```
oc apply -f vm-boot-from-pvc.yaml
oc get vm vm-boot-pvc -n boot-demo
oc get vmi vm-boot-pvc -n boot-demo
```

□□□□□□:

- □□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□ VM
- □□□□□□□□□□□□
- □□□□□□□□□□□□

## □□□□ 2: ContainerDisk □□□□□ (□□□)

ContainerDisk □□OCI □□□□□□□□□□□□□□□□□□□□Immutable (□□) □□□□□□□□□□□□□□□□

**vm-boot-from-containerdisk.yaml** □□□□□□□

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-boot-containerdisk
  namespace: boot-demo
spec:
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-boot-containerdisk
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
```

```

    bus: virtio
    name: containerdisk
    bootOrder: 1
  - disk:
      bus: virtio
      name: cloudinitdisk
  interfaces:
    - masquerade: {}
      name: default
  rng: {}
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - name: containerdisk
      containerDisk:
        image: quay.io/containerdisks/fedora:latest
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd: { expire: False }
          ssh_pwauth: True
        name: cloudinitdisk

```

□□□□□:

- ContainerDisk 類似於 VM 類似於磁碟映像檔
- 類似於磁碟映像檔
- 類似 CI/CD 類似於磁碟映像檔
- [quay.io/containerdisks](https://quay.io/containerdisks) 類似於磁碟映像檔

□□□□:

```
oc apply -f vm-boot-from-containerdisk.yaml
```

□□□□□□:

- 不可变集合
- CI/CD 不可变基础设施
- Immutable 不可变数据
- 不可变函数

- `vm` の `spec` の `dataVolumeTemplates`

### 例 3: HTTP から `DataVolume` を取得

`DataVolume` を HTTP/HTTPS から取得する `vm` の `spec` の `dataVolumeTemplates`

`vm-boot-from-http.yaml` の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-boot-http
  namespace: boot-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: vm-boot-http-disk
        annotations:
          cdi.kubevirt.io/storage.bind.immediate.requested: "true"
      spec:
        source:
          http:
            url:
              "https://download.fedoraproject.org/pub/fedora/linux/releases/43/Cloud/x86_64/images/Fedora-Cloud-Base-Generic-43-1.6.x86_64.qcow2"
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-boot-http
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
                bootOrder: 1
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
```





```

apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-multi-boot
  namespace: boot-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: vm-multi-boot-primary
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: vm-multi-boot-secondary
      spec:
        source:
          blank: {}
        storage:
          resources:
            requests:
              storage: 20Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-multi-boot
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: primarydisk
                bootOrder: 1
            - disk:
                bus: virtio
                name: secondarydisk
                bootOrder: 2

```

```

- cdrom:
  bus: sata
  name: installedcdrom
  bootOrder: 3
- disk:
  bus: virtio
  name: cloudinitdisk
interfaces:
- masquerade: {}
  name: default
rng: {}
machine:
  type: pc-q35-rhel9.4.0
memory:
  guest: 4Gi
networks:
- name: default
  pod: {}
volumes:
- dataVolume:
  name: vm-multi-boot-primary
  name: primarydisk
- dataVolume:
  name: vm-multi-boot-secondary
  name: secondarydisk
- name: installedcdrom
  containerDisk:
    image: quay.io/containerdisks/fedora:latest
- cloudInitNoCloud:
  userData: |-
    #cloud-config
    user: fedora
    password: fedora123
    chpasswd: { expire: False }
    ssh_pwauth: True
  name: cloudinitdisk

```

オプション:

1. **bootOrder: 1** - 第一起動デバイス (プライマリディスク)
2. **bootOrder: 2** - 第二起動デバイス (セカンダリディスク)
3. **bootOrder: 3** - CD-ROM (インストールメディア)

コマンド:

```
oc apply -f vm-multiple-boot-devices.yaml
```

確認:

- 直接 (DR) 启动
- 启动 OS 直接启动
- 启动 OS 直接启动
- 启动直接启动直接启动直接启动

## 直接 5: UEFI 直接启动

直接 BIOS 直接 UEFI 直接启动直接启动 VM 直接启动直接启动 Windows 直接启动直接启动

**vm-boot-uefi.yaml** 直接启动

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-boot-uefi
  namespace: boot-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: vm-boot-uefi-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-boot-uefi
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
                bootOrder: 1
            - disk:
                bus: virtio
                name: cloudinitdisk
```

```

    interfaces:
      - masquerade: {}
        name: default
    rng: {}
  firmware:
    bootloader:
      efi:
        secureBoot: false
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: vm-boot-uefi-disk
        name: rootdisk
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd: { expire: False }
          ssh_pwauth: True
        name: cloudinitdisk

```

□□□□:

- `firmware.bootloader.efi` □ UEFI □□□□□□□□□□
- `secureBoot: false` - □□□□□□□□□□ (□□□□□□□□ OS □□□□□□)
- □□□□□□ `pc-q35-rhel9.4.0` □ UEFI □□□□□□□□□□

□□□□:

```
oc apply -f vm-boot-uefi.yaml
```

□□□□□□:

- Windows 11 □□□□□□ Windows Server VM
- □□□□□□□□□□□□□□□□□□
- □□□□□□ (2TB □) □□□□□
- □□□□□□□□□□□□□□
- UEFI □□□□□□□□□□ Linux □□□□□□□□□□□□

## 例 6: UEFI (セキュアブート)

この例では、UEFI セキュアブートを有効にするための VM 設定を示しています。

**vm-boot-uefi-secureboot.yaml** の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-boot-secureboot
  namespace: boot-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: vm-boot-secureboot-disk
      spec:
        sourceRef:
          kind: DataSource
          name: fedora
          namespace: openshift-virtualization-os-images
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-boot-secureboot
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
                bootOrder: 1
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
          rng: {}
        firmware:
          bootloader:
            efi:
```

```
secureBoot: true
machine:
  type: pc-q35-rhel9.4.0
memory:
  guest: 4Gi
features:
  smm:
    enabled: true
networks:
  - name: default
    pod: {}
volumes:
  - dataVolume:
      name: vm-boot-secureboot-disk
      name: rootdisk
  - cloudInitNoCloud:
      userData: |-
        #cloud-config
        user: fedora
        password: fedora123
        chpasswd: { expire: False }
        ssh_pwauth: True
      name: cloudinitdisk
```

□□□□:

- `secureBoot: true` 安全ブート
- `features.smm.enabled: true` (System Management Mode) 有効
- 安全ブートは Linux と Windows の両方でサポートされています
- 安全ブートは BIOS/UEFI の設定で有効にする必要があります

□□□□:

```
oc apply -f vm-boot-uefi-secureboot.yaml
```

□□□□□□:

- 環境構築
- Windows 11 (環境構築)
- 環境構築
- 環境構築



VM UEFI

## 例 7: PXE (PXE)

PXE (Preboot Execution Environment) 環境で VM を起動する OS

**vm-boot-pxe.yaml** の内容

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: vm-boot-pxe
  namespace: boot-demo
spec:
  dataVolumeTemplates:
    - apiVersion: cdi.kubevirt.io/v1beta1
      kind: DataVolume
      metadata:
        name: vm-boot-pxe-disk
      spec:
        source:
          blank: {}
        storage:
          resources:
            requests:
              storage: 30Gi
  runStrategy: RerunOnFailure
  template:
    metadata:
      labels:
        kubevirt.io/domain: vm-boot-pxe
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
                bootOrder: 2
            - disk:
                bus: virtio
                name: cloudinitdisk
          interfaces:
            - masquerade: {}
              name: default
            - bridge: {}
              name: pxe-net
              bootOrder: 1
          rng: {}
```



```

machine:
  type: pc-q35-rhel9.4.0
memory:
  guest: 4Gi
networks:
  - name: default
    pod: {}
  - name: pxe-net
    multus:
      networkName: pxe-network
volumes:
  - dataVolume:
      name: vm-boot-pxe-disk
      name: rootdisk
  - cloudInitNoCloud:
      userData: |-
        #cloud-config
        user: fedora
        password: fedora123
        chpasswd: { expire: False }
        ssh_pwauth: True
      name: cloudinitdisk

```

Pod:

- Pod의 네트워크 인터페이스 (pxe-net)의 bootOrder: 1 설정 (기본값)
- Pod의 bootOrder: 2 설정 (기본값)
- Multus의 네트워크 인터페이스 (pxe-network) 설정
- Pod의 네트워크 인터페이스 (pxe-net)의 bootOrder: 1 설정 (기본값)
- Pod의 Pod 이름 (default) 설정

PXE 설정:

- Pod의 네트워크 인터페이스 (pxe-net)의 bootOrder: 1 설정 (기본값)
- Pod의 bootOrder: 2 설정 (기본값)
- DHCP 설정
- TFTP 설정
- VM의 PXE 설정

Pod:

- Pod의 OS 설정
- Pod의 네트워크 인터페이스
- Pod의 네트워크 인터페이스
- Pod의 네트워크 인터페이스



```

    rng: {}
  firmware:
    bootloader:
      bios:
        useSerial: true
  machine:
    type: pc-q35-rhel9.4.0
  memory:
    guest: 4Gi
  networks:
    - name: default
      pod: {}
  volumes:
    - dataVolume:
        name: vm-boot-menu-disk
        name: rootdisk
    - name: installedcdrom
      containerDisk:
        image: quay.io/containerdisks/fedora:latest
    - cloudInitNoCloud:
        userData: |-
          #cloud-config
          user: fedora
          password: fedora123
          chpasswd: { expire: False }
          ssh_pwauth: True
        name: cloudinitdisk

```

□□□□□□□□□□□□:

```

oc apply -f vm-boot-menu.yaml

# VNC□□□□□□□□□□□□□□□□
virtctl vnc vm-boot-menu -n boot-demo

# □□□□□□□□□□
virtctl console vm-boot-menu -n boot-demo

```

- □□□□ ESC □□□ F12 □□□□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□□□
- **useSerial: true** □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□□□□□

□□□□□□□□□□□□□□□□□□□□□□□□□□□□

## VM 操作

```
# VM 操作
oc get vm -n boot-demo

# VM 操作
oc get vmi -n boot-demo
```

```
# VM 操作 (vm-boot-pvc 操作 VM 操作)
oc describe vm vm-boot-pvc -n boot-demo
```

## 操作

```
# VM spec 操作 (vm-boot-pvc 操作 VM 操作)
oc get vm vm-boot-pvc -n boot-demo -o yaml | grep -A 5 bootOrder

# VM 操作 (vm-boot-uefi 操作 VM 操作)
oc get vm vm-boot-uefi -n boot-demo -o yaml | grep -A 10 firmware
```

## VM 操作

```
# VM 操作 (vm-boot-pvc 操作 VM 操作)
virtctl console vm-boot-pvc -n boot-demo

# VNC 操作 (vm-boot-menu 操作 VM 操作)
virtctl vnc vm-boot-menu -n boot-demo
```

## 操作

### VM 操作:

```
# 操作
dmesg | head -50

# 操作
systemd-analyze

# 操作
journalctl -b

# UEFI 操作 BIOS 操作
[ -d /sys/firmware/efi ] && echo "UEFI" || echo "BIOS"
```

## VM 启动顺序

步骤: 配置虚拟机 VM 启动顺序

步骤: VM 的 `bootOrder: 1` 配置项

步骤:

- 配置虚拟机 (虚拟机) 启动
- 配置 VM 启动顺序
- VM 启动顺序:

```
oc describe vmi vm-boot-pvc -n boot-demo
```

- 配置虚拟机启动顺序:

```
virtctl console vm-boot-pvc -n boot-demo
```

步骤: 配置虚拟机 VM 启动顺序

步骤: `secureBoot: true` 配置 VM 启动顺序

步骤:

- OS 启动顺序
- 配置 `smm.enabled: true` 配置项
- 配置虚拟机启动顺序
- 配置虚拟机启动顺序

```
# VM启动顺序配置
oc edit vm vm-boot-secureboot -n boot-demo
# secureBoot: true 或 secureBoot: false 配置
```

步骤: **PXE** 启动顺序

步骤: PXE 启动顺序 VM 启动顺序

步骤:

- PXE 启动顺序
- DHCP 或 PXE 启动顺序
- VM 启动顺序
- 配置 PXE 启动顺序 (DHCP/TFTP) 配置项
- 配置虚拟机启动顺序 `bootOrder` 配置项



- 10203 000000000000 (00000000)
- VM 00000000000000000000000000000000
- 00000000 BIOS/UEFI 000000000000000000000000

#### 0000000000

- 0000000000 UEFI 000000 (0000000000000000)
- BIOS 00000000000000000000000000000000
- 0000000000000000OS 00000000000000000000
- 0000000000000000 SMM 000000

#### 0000000000

- 000000000000000000 PVC 000000
- 0000 Immutable 0000000000 ContainerDisk 000000
- 000000000000000000 HTTP DataVolume 000000
- 00000000000000000000 ContainerDisk 000000000000

#### 00000000

- Windows 11 00000000000000000000000000000000
- 000000000000000000 BIOS 0000 UEFI 000000
- 00000000000000000000000000000000000000
- 00000000000000000000000000000000000000

#### 00000000

- 000000000000000000 virtio 00000000
- 00000000000000000000000000000000000000
- 000 Storage Class 000000 (0000000000 SSD)
- VM 000000000000 PVC 0000000000000000

#### 00000000

00000000000000000000000000000000000000:

```
# 0000VM0000
oc delete vm --all -n boot-demo

# namespace0000
oc delete namespace boot-demo
```



VM 0000000000000000 DataVolume 0 PVC 00000000000000000000

111

[illegible]

- ContainerDisk (PVC ContainerDisk HTTP DataVolume)
- ContainerDisk
- BIOS & UEFI ContainerDisk
- ContainerDisk
- PXE ContainerDisk
- ContainerDisk
- ContainerDisk
- ContainerDisk

VM

11

- [KubeVirt Booting From External Source](#)
- [KubeVirt Disks and Volumes](#)
- [OpenShift Virtualization Documentation](#)
- [KubeVirt Virtual Hardware](#)
- [UEFI Specifications](#)



目 次

cookbook 各章の目次は YAML の目次を参照してください

目 次

| 目次                               | 章  | 目次                                  |
|----------------------------------|----|-------------------------------------|
| <a href="#">Storage</a>          | 7  | LVM Operator のインストール                |
| <a href="#">Networking</a>       | 8  | LocalnetとUDNとBridge mapping のインストール |
| <a href="#">Virtual Machines</a> | 14 | 各章の目次 VM のインストール                    |
| <a href="#">Services</a>         | 3  | VM のインストール Kubernetes のインストール       |

Storage

LVM Storage Operator のインストール

| 目次                    | Kind              | 目次                                             | 目次                                      | 目次                   |
|-----------------------|-------------------|------------------------------------------------|-----------------------------------------|----------------------|
| lvm-namespace         | Namespa<br>ce     | openshift-lvm-storage<br>namespace のインストール     | <a href="#">LVM Operator</a><br>のインストール | <a href="#">YAML</a> |
| lvm-operatorgroup     | Operator<br>Group | LVM Operator の namespace<br>のインストール            | <a href="#">LVM Operator</a><br>のインストール | <a href="#">YAML</a> |
| lvm-subscription      | Subscript<br>ion  | OperatorHub から LVM Storage<br>Operator をインストール | <a href="#">LVM Operator</a><br>のインストール | <a href="#">YAML</a> |
| lvmcluster-with-wipe  | LVMClust<br>er    | 各章の目次 forceWipe<br>のインストール LVM のインストール         | <a href="#">LVM Operator</a><br>のインストール | <a href="#">YAML</a> |
| lvmcluster-clean-disk | LVMClust<br>er    | 各章の目次 (各) のインストール LVM<br>のインストール               | <a href="#">LVM Operator</a><br>のインストール | <a href="#">YAML</a> |
| test-pvc              | PVC               | 各章の目次 PVC のインストール                              | <a href="#">LVM Operator</a><br>のインストール | <a href="#">YAML</a> |
| test-pod              | Pod               | PVC のインストール Pod のインストール                        | <a href="#">LVM Operator</a><br>のインストール | <a href="#">YAML</a> |

Networking

localnetとUDNとBridge mapping のインストール

| 目次                           | Kind          | 目次                                 | 目次                                                               | 目次                   |
|------------------------------|---------------|------------------------------------|------------------------------------------------------------------|----------------------|
| localnet-secondary-namespace | Namespa<br>ce | localnet のインストール VM の<br>namespace | <a href="#">CUDN</a> のインストール <a href="#">Localnet</a><br>のインストール | <a href="#">YAML</a> |

| 名前                                | Kind          | 説明                                              | 関連リンク                                     | ファイル                 |
|-----------------------------------|---------------|-------------------------------------------------|-------------------------------------------|----------------------|
| localnet-bridge-mapping-nncp      | NNCP          | localnet を localnet1 に bridge 接続                | <a href="#">CUDN を Localnet で使用する</a>     | <a href="#">YAML</a> |
| cuda-localnet                     | CUDN          | localnet を ClusterUserDefinedNetwork            | <a href="#">CUDN を Localnet で使用する</a>     | <a href="#">YAML</a> |
| localnet-vlan-namespace           | Namespa<br>ce | VLAN localnet VM の namespace                    | <a href="#">NAD を VLAN localnet で使用する</a> | <a href="#">YAML</a> |
| localnet-vlan-bridge-mapping-nncp | NNCP          | VLAN を localnet-vlan に bridge 接続                | <a href="#">NAD を VLAN localnet で使用する</a> | <a href="#">YAML</a> |
| localnet-vlan-100-nad             | NAD           | VLAN 100 localnet の NetworkAttachmentDefinition | <a href="#">NAD を VLAN localnet で使用する</a> | <a href="#">YAML</a> |
| udn-primary-namespace             | Namespa<br>ce | UDN の namespace                                 | <a href="#">UDN を使用する</a>                 | <a href="#">YAML</a> |
| udn-primary                       | UDN           | VM の Layer 2 UserDefinedNetwork                 | <a href="#">UDN を使用する</a>                 | <a href="#">YAML</a> |

## Virtual Machines

以下の VM は、Kubernetes の VM として実行されます。

| 名前                       | Kind               | 説明                    | 関連リンク                                     | ファイル                 |
|--------------------------|--------------------|-----------------------|-------------------------------------------|----------------------|
| fedora-localnet-vm       | VirtualM<br>achine | localnet を Fedora VM  | <a href="#">CUDN を Localnet で使用する</a>     | <a href="#">YAML</a> |
| fedora-vlan-vm           | VirtualM<br>achine | VLAN 100 を Fedora VM  | <a href="#">NAD を VLAN localnet で使用する</a> | <a href="#">YAML</a> |
| fedora-vm-with-udn       | VirtualM<br>achine | UDN を Fedora VM       | <a href="#">UDN を使用する</a>                 | <a href="#">YAML</a> |
| fedora-dhcp-vm           | VirtualM<br>achine | DHCP を Fedora VM      | <a href="#">IP を</a>                      | <a href="#">YAML</a> |
| fedora-multi-network-vm  | VirtualM<br>achine | DHCP + IP を Fedora VM | <a href="#">IP を</a>                      | <a href="#">YAML</a> |
| fedora-vlan-static-ip-vm | VirtualM<br>achine | VLAN を IP を Fedora VM | <a href="#">IP を</a>                      | <a href="#">YAML</a> |
| golden-image-base-vm     | VirtualM<br>achine | VM                    | <a href="#">VM を</a>                      | <a href="#">YAML</a> |
| webserver-vm-from-golden | VirtualM<br>achine | VM                    | <a href="#">VM を</a>                      | <a href="#">YAML</a> |
| webserver-vm-2           | VirtualM<br>achine | 2 の VM                | <a href="#">VM を</a>                      | <a href="#">YAML</a> |

| 이름                      | Kind           | 설명                           | 참고 링크                                                | 파일 형식 |
|-------------------------|----------------|------------------------------|------------------------------------------------------|-------|
| fedora-access-vm        | VirtualMachine | SSH 접속을 위한 Fedora VM         | <a href="#">Linux</a><br><a href="#">인스턴스 생성 가이드</a> | YAML  |
| fedora-vlan-access-vm   | VirtualMachine | SSH 접속을 위한 VLAN 기반 Fedora VM | <a href="#">Linux</a><br><a href="#">인스턴스 생성 가이드</a> | YAML  |
| fedora-app-vm           | VirtualMachine | DNS 서비스를 위한 Fedora VM        | <a href="#">VM DNS</a>                               | YAML  |
| golden-image-datasource | DataSource     | 이미지 기반 PVC를 위한 DataSource    | <a href="#">인스턴스 생성 가이드</a>                          | YAML  |

## Services

VM 인스턴스를 Kubernetes 클러스터에 배포

| 이름                    | Kind    | 설명                           | 참고 링크                                                | 파일 형식 |
|-----------------------|---------|------------------------------|------------------------------------------------------|-------|
| fedora-vm-web-service | Service | VM에 Web 서비스를 위한 LoadBalancer | <a href="#">UDN</a>                                  | YAML  |
| ssh-nodeport-service  | Service | VM에 SSH 접속을 위한 NodePort      | <a href="#">Linux</a><br><a href="#">인스턴스 생성 가이드</a> | YAML  |
| headless-service      | Service | DNS 서비스를 위한 Headless         | <a href="#">VM DNS</a>                               | YAML  |

### 요약

#### 배포 방법

1. YAML 파일을 클러스터에 업로드
2. 클러스터에 배포
3. `oc apply -f <filename>.yaml` 실행

#### 예제

```
oc apply --dry-run=client -f <filename>.yaml
```

### 참고

- [인스턴스 생성 가이드](#)
- [인스턴스 생성 가이드](#)
- [인스턴스 생성 가이드](#)
- [인스턴스 생성 가이드](#)
- [VM 인스턴스 생성 가이드](#)

- VM 0000000000

# API

## 

OpenShift Virtualization API API

API OpenShift Virtualization OpenShift Container Platform

## 

- API
- API
- API
- API
- API
- API
- API

## 

- OpenShift Virtualization OpenShift v4.18
- kubeadmin
- REST API HTTP CRUD (GETPUTPOSTPATCHDELETE)JSON
- REST API (PostmancURL)

## 

### API

OpenShift Virtualization API

## 

OpenShift Virtualization



- KubeVirt API 00000000
- KubeVirt 00000000

## API □□□□□□□□



OpenShift 版本: v4.19, v4.20



OpenShift Virtualization API  
API

OpenShift Virtualization KubeVirt Kubernetes OpenShift (Kubernetes) json yaml API OpenShift Virtualization API

[illegible]

[yaml](#)
[\(300.00\)](#)
[OpenShift](#)
[Virtualization](#)
[KubeVirt](#)
[API](#)
[KubeVirt API reference](#)

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- **Virtualization APIs**
  - VirtualMachines
  - VirtualMachineInstances
  - Instance Types
  - VirtualMachineClusterInstanceTypes
  - VirtualMachineInstanceTypes
  - VirtualMachinePreferences ☐☐☐ VirtualMachineClusterPreferences
  - PreferredCPUTopology
  - vCPU SpreadOptions
- **Networking APIs**
  - Networks
  - Interfaces
  - Default Network
  - Multus CNI Plugin
  - NMState Operator

- [Storage APIs](#)
  - [PersistentVolumes](#)
  - [PersistentVolumeClaims](#)
  - [VolumeSnapshots](#)
  - [DataVolumes](#)
  - [DataSources](#)
  - [StorageClasses](#)
  - [VirtualMachineSnapshots](#)
  - [VirtualMachineRestores](#)
- [Clone API](#)
  - [VirtualMachineClones](#)
  - [Mutating a Cloned VM](#)
  - [Patching a Cloned VM](#)
  - [Cloning Strategies](#) & [StorageProfiles](#)
- [Export API](#)
  - [VirtualMachineExports](#)

## 📄 API

### VirtualMachines

VirtualMachine is OpenShift Virtualization's primary API resource (similar to Kubernetes' Pod)

VirtualMachine is VirtualMachineInstance (VM instance) resource's Status subresource

VirtualMachine has `spec` for VM configuration and metadata

- `instanceType` - VirtualMachine's `instanceType` (RAM & CPU configuration) is
- `volumes` - VirtualMachine's `volumes` is a list of `dataVolume` or `persistentVolumeClaims`
  - `dataVolume` - VirtualMachine's `DataVolume(s)` is a list of `DataVolume(s)` or `dataVolumeTemplates`
  - `persistentVolumeClaim` - `PersistentVolumeClaim` is a VM's
- `networks` - VM's `networks` is a list of `Pod/VM`
- `interfaces` - VM's `interfaces` is a list of `NIC` or `Pod/VM`
- `dataVolumeTemplates` - VirtualMachine's `DataVolume` is a list of `DataVolume` or `volumes`

VirtualMachine's `spec.runStrategy` is a VM's

- **Always** - 常に再起動 (失敗した場合) する VM の戦略
- **RerunOnFailure** - Always と同じ VM が失敗した場合 (エラーメッセージ) 再起動
- **Once** - VM が失敗したら一度だけ再起動
- **Manual** - VM が失敗したら POST によって手動で再起動
- **Halted** - `spec.runStrategy` が VM が `start` された VM が失敗した場合 (エラーメッセージ) しない



`spec.running` が VM が `spec.runStrategy` が VM が

`virtctl` は `VirtualMachine` を `start`、`stop`、`restart` する。KubeVirt は VM の `status` API を提供する。

VirtualMachine の `runStrategy` は VM が

- **start** - VM が `Halted` の `run strategy` が `Always`、`RerunOnFailure`、`Manual` の `run strategy` が
- **restart** - VM が `run strategy` が `Halted` の VM が
- **stop** - VM が `Always` の `run strategy` が `Halted`、`RerunOnFailure`、`Manual` の `run strategy` が

API は `Manual` の `run strategy` が VM が `stateChangeRequests[]` が `VirtualMachine` が

```
status:
  stateChangeRequests:
    - action: <Stop|Start> ①
      uid: <uid-of-the-virtual-machine-instance> ②
```

① `Stop`、`Start` が

② `VirtualMachineInstance` (`VirtualMachine` が) の `UID`、VM が `VMI` が `Start` が

VirtualMachine が API が VM `status` が

- **status.printableStatus** - VM が `Running` (成功/失敗) `Starting` (成功/失敗) `Stopped` が
- **status.runStrategy** - `run strategy` が `spec.runStrategy` が `Always`、`RerunOnFailure`、`Once`、`Manual`、`Halted` が
- **status.conditions["type": "Ready"]** - `conditions` が `type` - `Ready` (成功/失敗) `status` - `Reason` が `Message` - `reason code` が

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
```



```
metadata:
  name: vm-status-example
spec:
  ...
status:
  conditions:
    - lastProbeTime: "2025-11-10T15:00:46Z"
      lastTransitionTime: "2025-11-10T15:00:46Z"
      message: Guest VM is not reported as running
      reason: GuestNotRunning
      status: "False"
      type: Ready
```

## VirtualMachineInstances

## VirtualMachineInstance (VMI) ¶ OpenShift Virtualization ¶ ¶ ¶ ¶ ¶ VM ¶ ¶ ¶ ¶ ¶ ¶ ¶ ¶ ¶ ¶

000000VirtualMachineInstances    0    Kubernetes    Pod    0000000000Pod    0    QEMU    /    KVM  
 000000000000000000000000VM    0000000000    (000000000000000000)    0    PersistentVolumeClaim    0000000000000000    (000000)    000

VirtualMachine → VirtualMachineInstance → VM → VM template → spec.template → yml (json) →

VM 0000 VMI 0000000000000000 VM 000000 VMI 000000000000000000000000 VMI 000000000000000000000000  
(DataVolumesPVC00000000 PV) 000000000000000000

VM のインストール VM は yaml/json で定義される VMI をインストールする VMI インストール

## Instance Types

OpenShift Virtualization CPU (vCPU) (0: u1)

T (small-medium-large-xlarge-2xlarge)

(0: u1.xlarge)

(vCPU vs CPU) vCPU

## OpenShift Virtualization

- **Network (N)** - DPDK CPU/RAM 1:2
- **Overcommitted (O)** - RAM CPU/RAM 1:4
- **Compute Exclusive (CX)** - CPU CPU/RAM 1:2
- **General Purpose (U)** - CPU CPU/RAM 1:4
- **Memory Intensive (M)** - CPU/RAM 1:8

```

00000000000000000000000000000000 VirtualMachineClusterInstanceType 000000000000

```

## VirtualMachineClusterInstanceTypes

VirtualMachineClusterInstanceType      OpenShift  
Virtualization      VM      spec      ( )

VirtualMachineClusterInstanceTypes      spec      ( )

- **annotations** - annotations
- **cpu** ( ) - CPU
  - **dedicatedCPUPlacement** - VM vCPU CPU / **false**
  - **guest** ( ) - vCPU **0**
  - **isolateEmulatorThread** - VMI CPU / **false**
  - **maxSockets** -
  - **model** - VMI CPU **host-model**
  - **numa** - NUMA
    - **guestMappingPassthrough** - NUMA CPU NUMA { }
  - **realtime** - virt-launcher Pod VMI ( )
    - **mask** - vCPU "0-3,^1", "1,2,3", "1-2" libvirt
- **gpu** - GPU
  - **deviceName** - ""
  - **name** ( ) - GPU ""
  - **tag** -
  - **virtualGPUOptions** - vGPU
    - **display** - vGPU
      - **enabled** - vGPU **true**
      - **ramFB** - OS GPU
        - **enabled** - **true**
- **hostDevices** -
  - **deviceName** ( ) - ""
  - **name** ( ) - ""
  - **tag** -
- **memory** ( ) -
  - **guest** ( ) - OS ("4096Mi" "6Gi")
  - **hugepages** - VMI Hugepage
    - **pageSize** - Hugepage amd64 1Gi 2Mi
  - **maxGuest** - maxGuest guest

- **overcommitPercent** - 指定する値は、virt-launcher Pod のメモリ使用率の上限を示す。0
- nodeSelector** - 指定する値は、Kubernetes selector label
- **schedulerName** - 指定する値は、Kubernetes の scheduler の名前。K8s のデフォルトは

## VirtualMachineInstanceTypes

```
VirtualMachineInstanceType      VirtualMachineClusterInstanceType      namespace
namespace (project/namespace) project/namespace
```



```
specinstancetype      OpenShift      Virtualization      Web
spec.template.spec.domain.cpu      spec.template.spec.domain.memory
CPU      VirtualMachine
VM      VirtualMachineInstance
2
```

## VirtualMachinePreferences VirtualMachineClusterPreferences

[illegible]

```

0000000000000000 spec 000000 VirtualMachineClusterPreferences 00000000000000000000000000000000

```

OpenShift Virtualization `VirtualMachineClusterPreferences` OS `fedora` `rhel10` `win11` OS `UEFI` `BIOS` `VirtualMachineClusterPreference`

SecureBoot EFI VirtualMachineClusterPreference

```
apiVersion:instancetype.kubevirt.io/v1beta1
kind: VirtualMachineClusterPreference
metadata:
  name: rhel10-custom
spec:
  firmware:
    preferredUseEfi: true
    preferredUseSecureBoot: true
```



```
VirtualMachinePreference spec { VM { BIOS { preferredUseEfi {
spec.firmware.preferredUseBios { SecureBoot { UEFI
{
```

## PreferredCPUTopology

VirtualMachinePreferences 1 VirtualMachineInstanceType vCPU OS  
 (CPU ) CPU vCPU

VM の CPU 設定を指定する (CPU) 設定を指定する CPU 設定を指定する

preferredCPUTopology 設定を指定する

- **sockets** (ソケット) - vCPU 単位 OS 設定を指定する
- **cores** - vCPU 単位 OS 設定を指定する
- **threads** - vCPU 単位 OS 設定を指定する
- **spread** - vCPU 設定を指定する **spreadOptions** 設定を指定する (ソケット)
- **any** - vCPU 単位 OS 設定を指定する vCPU 単位 preference (ソケット) 設定を指定する

## vCPU SpreadOptions

preferredCPUTopology の **spread** 設定を指定する **spreadOptions** 設定を指定する vCPU

**spreadOptions** の 2 設定を指定する

- **across** - vCPU 設定を指定する
- **ratio** (ソケット 2) - 設定を指定する (N ソケット)

**across** 設定を指定する

- **SocketsCores** (ソケット) - vCPU 単位 N 設定を指定する
- **SocketsCoresThreads** - vCPU 単位 N ソケット 2 設定を指定する
- **CoresThreads** (2 ソケット vCPU 単位) - vCPU 設定を指定する 1 ソケット 2 設定を指定する

設定 **preferredCPUTopology** 設定 **VirtualMachinePreference** 設定を指定する

```
apiVersion: instancetype.kubevirt.io/v1beta1
kind: VirtualMachinePreference
metadata:
  name: cpu-topology-example
spec:
  cpu:
    preferredCPUTopology: spread
    spreadOptions:
      across: SocketsCoresThreads
      ratio: 6
```

## 設定 API

設定 **VirtualMachine** 設定 **VirtualMachineInstances** 設定 API 設定 **Multus CNI** 設定 **NMState Operator** 設定 **VM/VMi** 設定

**VirtualMachine** 設定 (ソケット) 設定 **networks** 設定 **interfaces** 設定

## Networks

VirtualMachine 对象包含 Pod 配置 (`spec.template.spec`) 的 `networks[]` 属性 ([]) 包含网络配置 (网络 `kind` 属性 Kubernetes 网络配置) 包含 VirtualMachine 网络配置 (网络) 属性

网络配置 `spec.template.spec.networks[]` 包含 2 个网络配置属性

- **pod** - 网络配置 Kubernetes 网络配置
- **multus** - Multus CNI 网络配置

## Interfaces

VirtualMachine 对象包含 VM 网络配置 (NIC) 属性 包含 Pod 网络配置 VM 网络配置 `virtio` 属性 (Windows 网络) 包含 OpenShift Virtualization 网络配置 (网络 NIC) 属性

网络配置 VirtualMachine 对象包含 `spec.domain.devices.interfaces[]` 网络配置 `network` 属性

网络配置包含 2 个网络配置属性

- **name** (网络) - VirtualMachine 的 `spec.template.spec.networks[]` 网络配置 `network` 属性
- (网络) - 网络配置 `binding` 属性:
  - **bridge** - Linux bridge 网络配置
  - **sriov** - 网络 SR-IOV VF (`vfi` 网络) 网络配置
  - **masquerade** - 网络 NAT 网络 IP 网络 (`nftables` 网络) 网络配置
  - **binding** - 网络 (网络) 网络配置:
    - **name** - 网络配置:
      - **l2bridge** - VM `interface` 网络 `network` 网络 Layer 2 网络配置
      - **passt** (网络配置) - Plug a Simple Socket Transport Pod 网络配置 OpenShift 网络配置
- **model** - 网络配置 (10/100 网络配置)
  - **e1000** - Intel 82545EM PCI 网络配置
  - **e1000e** - Intel 82574 PCI-E 网络配置
  - **virtio** - 网络配置 Linux 网络 (网络) 网络配置
- **macAddress** - 网络配置 MAC 网络
- **ports** - VM 网络配置
  - **name** - 网络配置
  - **port** - 网络 (0 网络 65535 网络配置)
- **pciAddress** - `0000:81:00.1` 网络配置 PCI 网络

VM 网络配置

□ □ □ □ □ □ □ □ □ □

OpenShift 使用 Kubernetes 的 Pod 来运行容器。Pod 的配置文件是 pod: {}。pod: {} 的配置文件是 masquerade: {}。

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: default-network-example
  namespace: my-namespace
spec:
  template:
    spec:
      domain:
        devices:
          interfaces:
            - name: default ①
              masquerade: {} ②
      networks:
        - name: default ①
          pod: {} ③
```

① domain.devices.interfaces (spec.template.spec 内) 仮想マシン 内の networks  
 仮想マシン 内の networks

② 配置 IP 地址 (NAT) 和 NIC 配置

### ③ 创建 Kubernetes Pod 资源

[illegible]

- OpenShift 部署在 Kubernetes 之上
- 部署在 namespace 下的 Pod 部署在 Kubernetes
- 部署在 namespace (命名) IP 部署在 Kubernetes 部署在 Kubernetes
- 部署在 namespace 下的 OpenShift Route 与 LoadBalancer 部署在 Kubernetes

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

- VM 透過 masquerade IP 與外部網路通訊
- 透過 project/namespace 與 Pod 通訊
- VLAN 網路
- 透過 LoadBalancer (AWS) 與外部網路通訊 (TCP 或 UDP 埠)

**Multus CNI** □□□□□

Multus 网络策略组 (CNI) 配置示例

Networks  Multus  CNi  OpenShift

□□□□□□□□□□□□□□□□

- **bridge** - 所有 Pod 都连接到同一个 Pod 网络 namespace 的 bridge 上
- **host-device** - 所有 Pod 都连接到同一个 Pod 网络 namespace 的 root network namespace 的 host-device 上
- **ipvlan** - 所有 Pod 的 MAC 地址都连接到同一个 VPC (VLAN) 上，所有 Pod 的 MAC 地址都连接到同一个 MAC 地址上
- **macvlan** - 所有 Pod 的 MAC 地址都连接到同一个 MAC 地址上
- **SR-IOV** - single root I/O virtualization 所有 Pod 都连接到同一个 SR-IOV 上

OpenShift (Kubernetes) Multus NetworkAttachmentDefinition  
project/namespace

```

OpenShift (Kubernetes) の cluster Network には spec.additionalNetworks[]
で、project/namespace 毎に NetworkAttachmentDefinition を
NetworkAttachmentDefinition を

```

## NMState Operator

NMState ("NM" = NetworkManager [RHEL/Fedora] Linux [Red Hat Enterprise Linux]) [Linux  
[Red Hat Enterprise Linux] API [ ]

NMState Operator 提供 API (NodeNetworkConfigurationPolicy) 与 Kubernetes Operator 集成  
 NMState 与 Kubernetes (即 OpenShift) 集成

00 Multus (linux-bridge) 00 Pod (00 VM) 00 Bridge Interface 00  
 NMState Operator 00

```

Multus                                (NetworkAttachmentDefinition)
Pod  VM                               Pod/VM

```

## □□□□□ API

```

#####OpenShift Virtualization #####Kubernetes
Kind #####

```

API

- **VolumeSnapshotContents** - [VolumeSnapshotContents](#) [VolumeSnapshotContent](#) [VolumeSnapshotContentList](#)
- **VirtualMachineSnapshotContents** - [VirtualMachineSnapshotContents](#) [VirtualMachineSnapshotContent](#) [VirtualMachineSnapshotContentList](#) [VirtualMachineSnapshot](#) [VirtualMachineSnapshotList](#) [PVCs](#) [VolumeSnapshots](#) [VolumeSnapshotClasses](#)
- **VolumeSnapshotClasses** - [VolumeSnapshotClasses](#) [StorageClasses](#) [VolumeSnapshotClasses](#) [VolumeSnapshotClass](#) [VolumeSnapshotClassList](#) [CSI](#) [VolumeSnapshot](#) [VolumeSnapshotList](#)

OpenShift

Virtualization の場合 Kubernetes の設定が複雑になる可能性がある

コンテナ環境では Kubernetes API を利用して (コンテナ環境では) コンテナの  
コンテナ環境では Kubernetes API を利用して (コンテナ環境では)

## PersistentVolumes

PersistentVolume (PV) は Kubernetes の [PersistentVolumeClaim](#) (PVC) を利用して Pod に PersistentVolume を (ReadWriteOnce/ReadWriteMany など) 接続する

## PersistentVolumeClaims

PersistentVolumeClaim (PVC) は Pod に PersistentVolume を Kubernetes の Kubernetes の PersistentVolume を PVC に 1:1 の割合で PV を (PV は ReadWriteMany など) 接続する PVC を

OpenShift は Kubernetes の (SAN / NAS) PersistentVolumes を [StorageClasses](#) PersistentVolumes を [StorageClasses](#) PersistentVolumes を

OpenShift Virtualization は [DataSources](#) は [DataVolumes](#) は kubernetes の PV は PVC を VirtualMachines を

StorageClasses は PersistentVolumeClaim を dataSource を VolumeSnapshot は PVC は PVC は dataSource を PVC は PVC を

## VolumeSnapshots

VolumeSnapshots は PersistentVolume を VM を PersistentVolume を

VolumeSnapshots は VirtualMachine を (VirtualMachineSnapshots) KubeVirt DataVolume は DataSource PersistentVolume を

## StorageClasses

StorageClass は PersistentVolumeClaim PersistentVolumes OpenShift Kubernetes を

StorageClasses は CSI OpenShift (Amazon Elastic Block Store など) API kubernetes API (PV PVC) を

StorageClasses OpenShift Virtualization OpenShift Virtualization



## DataVolumes

DataVolumes 与 PersistentVolumeClaims 一起使用以声明和请求持久化存储资源

DataVolumes 由 VM 使用以声明所需的 PVC。DataVolumes 由 PVC 通过 Containerized Data Importer (CDI) 创建。

DataVolumes (通过 CDI) 支持两种格式 (qcow2 和 raw) 的 PVC。DataVolumes 支持 3 种不同的格式。

- 本地存储
- 网络 (Web 存储) (网络存储)
- 本地存储 (本地存储 PVC)

CDI (通过 DataVolumes) 支持以下格式：

- **blank** - 空白存储
- **gcs** - GCS 存储
- **http** - HTTP/S URL 存储
- **imageio** - ovirt-imageio 存储
- **pvc** - 持久化 PersistentVolumeClaim 存储
- **registry** - 注册表存储
- **s3** - AWS 存储 S3 (MinIO) 存储
- **snapshot** - 快照 VolumeSnapshot 存储
- **upload** - 上传存储
- **vddk** - VMware 存储

## DataSources

DataSources 通过 DataVolume 声明持久化存储资源。PersistentVolumeClaim 通过 VolumeSnapshot 声明持久化存储资源。OpenShift Virtualization Web 控制台 DataSource 支持 Bootable Volumes。

DataSource **spec** 支持 1 种格式。

- **source** - **pvc** 和 **snapshot** (持久化存储)
  - **pvc** 和 **snapshot** - 持久化 PersistentVolumeClaim 和 VolumeSnapshot 持久化存储
  - **name** - pvc 和 snapshot 名称
  - **namespace** - pvc 和 snapshot 命名空间

## VirtualMachineSnapshots

VirtualMachineSnapshot 由 VM 使用以声明所需的/持久化存储资源。

VirtualMachineSnapshot 对象与 VolumeSnapshot 对象 namespace 相同  
VirtualMachineSnapshot 对象 Status 属性

VirtualMachineSnapshot 对象的 **source** VM 对象的 **1m3600s1h30m** 属性  
对象的 **failureDeadline** 属性 5m

## VirtualMachineRestores

对象 (对象) 对象 VM  
VirtualMachineRestore 对象

VirtualMachineRestore 对象 VM 的 **apiGroup****kind****name** 属性 **target** VM 对象的 **virtualMachineSnapshotName** 属性

## API

VM 对象 3 个

- VM 对象 (VolumeSnapshots 对象) PVC 对象 VM 对象 VirtualMachine 对象
- VirtualMachineSnapshot 对象 (对象 VirtualMachine 对象 VM/对象 MAC 对象)
- API 对象 VirtualMachine (对象 VirtualMachineSnapshot) 对象 VirtualMachineClone 对象

VirtualMachine 对象 VirtualMachineClones 对象

## VirtualMachineClones

API 对象 VirtualMachineClone 对象 VirtualMachine 对象 VM 对象 (对象 VM 对象)

对象 VirtualMachineClone 对象的 **source** 属性 (对象) **target** 属性 VirtualMachine 对象的 VM 对象的 **apiGroup****kind****name** 属性

对象 VM 对象

VM 对象 (labels 属性/属性 annotations) MAC 属性 BIOS / UEFI 属性 VM 的 CPU 属性 RAM 属性 VM 属性

对象 (对象) 对象

- **labelFilters** - 对象 VM 对象 label (metadata.labels 属性) 对象 VM 对象
- **annotationFilters** - 对象 VM 对象 annotation (metadata.annotations 属性) 对象 VM 对象
- **template** - VM / VMI **template** 对象 2 对象
  - **labelFilters** - 对象 VM 的 **template.metadata.labels** 属性 VM 对象 label 属性

- **annotationFilters** - 指定 VM の `template.metadata.annotations` によって VM の annotation をフィルタリング/スキップする

- **newMacAddresses** - 指定 VM の `interface` によって MAC アドレスを指定する<interface-name> によって指定する (00-01-ff-00-01-00) MAC アドレス 1 から 1 まで指定する

- **newSMBiosSerial** - 指定 SMBios の serial を指定する

- **patches** (v4.20+) - 指定 VM の `patches` によって json を指定する



- 指定する (label-or-annotation-name/\*) によって (!label-or-annotation-name) によって指定する
- 指定する VM の MAC アドレスを指定する

指定する `patches` によって `op` によって `path` `value` によって json を指定する (json を指定する) によって指定する

指定する `VirtualMachineClone spec` によって指定する

```
apiVersion: clone.kubevirt.io/v1beta1
kind: VirtualMachineClone
metadata:
  name: example-vm-clone
  namespace: my-namespace
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: my-virtual-app
  target:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: my-clone-app
  labelFilters:
    - "metadata-labels-to-include/*"
    - "!metadata-labels-to-omit/*"
  annotationFilters:
    - "metadata-annotations-to-include/*"
    - "!metadata-annotations-to-omit/*"
  template:
    labelFilters:
      - "template-labels-to-include/*"
      - "!template-labels-to-omit/*"
    annotationFilters:
      - "template-annotations-to-include/*"
      - "!template-annotations-to-omit/*"
  newMacAddresses:
    nameOfInterface: "ff-00-00-bb-aa-aa"
  newSMBiosSerial: "S3R14LNUM83R"
  patches:
```

```
- '{ "op": "add", "path": "/metadata/labels/my-app", "value": "cloned" }'
```

```
- '{ "op": "replace", "path": "/spec/instanceType/name", "value": "u1.xlarge" }'
```

VirtualMachine

VirtualMachine

VirtualMachine

VirtualMachine { } add VirtualMachine

```
apiVersion: clone.kubevirt.io/v1beta1
kind: VirtualMachineClone
metadata:
  name: missing-parent-patch-example
  namespace: my-namespace
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: example-source-vm
  patches:
    - '{ "op": "add", "path": "/spec/template/spec/domain/cpu", "value": { } }'
```

```
- '{ "op": "add", "path": "/spec/template/spec/domain/cpu/cores", "value": 6 }'
```



- VirtualMachine `cpu` `memory` `spec.instanceType`
- `spec.instanceType` VM `UI`
- CPU `InstanceTypes` `CPU` `VirtualMachine[Cluster]Preferences`
- `spec.instanceType` VM `cpu` `memory` VM

StorageProfiles

OpenShift Virtualization `cloneStrategy` (OpenShift/Kubernetes `StorageClass`) `OpenShift Virtualization` (`CDI`) `StorageClass` (`CSI`) `StorageProfile` `StorageProfile` `StorageClass`



`CDI` `StorageClass` `StorageProfile` `StorageClass` `DataVolumes` `PVC`

OpenShift Virtualization `VirtualMachine` `VM` (`VolumeSnapshot`) `PVC` `DataVolumes`

OpenShift Virtualization (`CSI`) `CSI`

OpenShift/Kubernetes CSI (Copy on Write) CoW (Copy on Write)

AWS EBS fast snapshot restoration (GitHub Issue)

StorageProfile cloneStrategy

- snapshot (VolumeSnapshotClass) CSI (CSI VolumeSnapshotClass) CDI
- copy Pod Pod
- csi-clone CSI API StorageClass StorageProfile

cloneStrategy csi-clone StorageProfile

```
apiVersion: cdi.kubevirt.io/v1beta1
kind: StorageProfile
metadata:
  name: example-provisioner
spec:
  claimPropertySets:
  - accessModes:
    - ReadWriteOnce
    volumeMode: Filesystem
    cloneStrategy: csi-clone
```

## API

OpenShift Virtualization VM API

VM OS OpenShift API IngressRouteNodePort



Ingress Route virtctl port-forward export proxy server

## Export Tokens

VM namespace export token K8s export proxy endpoint (x-kubevirt-export-token)



12

export token

```
apiVersion: v1
kind: Secret
metadata:
  name: export-token
stringData:
  token: abc098765421def
```

VirtualMachineExport

## VirtualMachineExports

VirtualMachineExport  
VirtualMachineVirtualMachineSnapshotPersistentVolumeClaim

export token tokenSecretRef source

VirtualMachine VM yml VM PVCDataVolumesMemoryDump  
volume VirtualMachineSnapshot PVC VM

VirtualMachineExport status URL  
VM

4

- **raw** - qcow2
- **gzip** - gzip
- **dir** - PVC URL /dir /path/to/file.txt
- **tar.gz** - gzip tar PVC

- **KubeVirt content-type** - PVC KubeVirt raw gzip
- **Archive content-type** - PVC KubeVirt dir tar.gz

VirtualMachineExport

```
apiVersion: export.kubevirt.io/v1beta1
kind: VirtualMachineExport
metadata:
  name: export-example ①
spec:
  tokenSecretRef: export-token ②
  source: ③
    apiGroup: "kubevirt.io"
    kind: VirtualMachine
```

name: virtual-machine-example

- ① 指定 VirtualMachineExport の名前
- ② export token を指定
- ③ 指定/指定 kind を指定

指定/指定

| apiGroup             | kind                   |
|----------------------|------------------------|
| kubevirt.io          | VirtualMachine         |
| snapshot.kubevirt.io | VirtualMachineSnapshot |
| ""                   | PersistentVolumeClaim  |

# AI 仮想マシン VM

## 

OpenShift Virtualization AI Claude Code Cursor IDE rh-virt Model Context Protocol (MCP) VM

rh-virt VM 10

## 

- Claude Code Cursor IDE rh-virt
- MCP OpenShift
- VM
- skill-first
- AI
- human-in-the-loop

## 

rh-virt VM 10

| vm-inventory         | namespace          |
|----------------------|--------------------|
| vm-create            |                    |
| vm-lifecycle-manager |                    |
| vm-clone             | cross namespace VM |
| vm-delete            | VM                 |
| vm-snapshot-create   | Storage Class VM   |
| vm-snapshot-list     | namespace VM       |
| vm-snapshot-restore  | VM                 |
| vm-snapshot-delete   | VM                 |
| vm-rebalance         | VM                 |



|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

[illegible]

- OpenShift Virtualization Operator [OpenShift 4.18+](#) [Podman 4.0+](#)
- Claude Code CLI [Cursor IDE](#)
- [MCP](#) [Podman \(4.0+\)](#)
- [KUBECONFIG](#)
- [namespace](#) [VirtualMachine](#) [RBAC](#) [ServiceAccount](#)

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|--|--|--|--|--|

## AI 環境と VM 環境

**rh-virt** **MCP** **vm-inventory**

[illegible]

vm-create

11

- [rh-virt](#) [仮想マシン](#)
- [OpenShift MCP](#) [管理](#)
- [OpenShift Virtualization](#) [仮想化](#)
- [Model Context Protocol](#) [API](#)

# rh-virt

11

rh-virt agentic MCP OpenShift vm-inventory

Claude Code    Cursor IDE    OpenShift    Virtualization    VM

□ □ □ □

- OpenShift Virtualization Operator [OpenShift 4.18+](#) [インストール](#)
- [OpenShift Container Platform](#) [Claude Code CLI](#) [Cursor IDE](#)
- Podman 4.0 [インストール](#)
- [OpenShift Container Platform](#) [kubeconfig](#) [KUBECONFIG](#)
- VirtualMachine [RBAC](#) [Cluster admin](#) [ServiceAccount](#)

OpenShift MCP Server

```
oc get virtualmachines -A
```

## AI Model Context Protocol (MCP) VM Agents

AI Model Context Protocol (MCP) is a standard for AI agents to interact with external data sources. In this example, we use the OpenShift MCP Server to manage VMs. The server is implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API. The server is implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API.

**rh-virt** is a plugin for OpenShift Virtualization VMs. It provides a set of 10 agents (vm-inventory, vm-create, vm-lifecycle-manager, vm-clone, vm-delete, vm-snapshot-create, vm-snapshot-list, vm-snapshot-restore, vm-snapshot-delete, vm-rebalance) that interact with the OpenShift Virtualization API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API.

OpenShift MCP Server is a plugin for OpenShift Virtualization VMs. It provides a set of 10 agents (vm-inventory, vm-create, vm-lifecycle-manager, vm-clone, vm-delete, vm-snapshot-create, vm-snapshot-list, vm-snapshot-restore, vm-snapshot-delete, vm-rebalance) that interact with the OpenShift Virtualization API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API.

OpenShift MCP Server

OpenShift MCP Server

1. **Claude Code** or **Cursor IDE** — AI agents that interact with the OpenShift MCP Server
2. **AGENTS.md** — A file that lists the agents and their capabilities
3. **SKILL.md** (optional) — A file that lists the skills and their capabilities
4. **OpenShift MCP Server** — Kubernetes KubeVirt API plugin that interacts with the OpenShift Virtualization API
5. **KUBECONFIG** — A file that contains the OpenShift MCP Server configuration

List all VMs using the `oc` CLI and the `Model Context Protocol (MCP)` API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API.

## rh-virt OpenShift MCP Server

Lola is a plugin for OpenShift Virtualization VMs.

**rh-virt** is a plugin for OpenShift Virtualization VMs. It provides a set of 10 agents (vm-inventory, vm-create, vm-lifecycle-manager, vm-clone, vm-delete, vm-snapshot-create, vm-snapshot-list, vm-snapshot-restore, vm-snapshot-delete, vm-rebalance) that interact with the OpenShift Virtualization API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API.

OpenShift MCP Server is a plugin for OpenShift Virtualization VMs. It provides a set of 10 agents (vm-inventory, vm-create, vm-lifecycle-manager, vm-clone, vm-delete, vm-snapshot-create, vm-snapshot-list, vm-snapshot-restore, vm-snapshot-delete, vm-rebalance) that interact with the OpenShift Virtualization API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API. The agents are implemented using the `oc` CLI and the `Model Context Protocol (MCP)` API.

```
lola --version
```

## agentic-plugins OpenShift MCP Server

```
git clone https://github.com/RHEcosystemAppEng/agentic-plugins.git
```

```
cd agentic-plugins
```

**rh-virt** をインストールし **lola** を実行

```
lola mod add rh-virt
```

インストールされた **rh-virt** を確認する

Claude Code をインストールする

```
lola install -f rh-virt -a claude-code
```

Cursor IDE をインストールする

```
lola install -f rh-virt -a cursor
```

Lola は MCP をサポートする IDE をインストールする

## MCP をインストール

**mcps.json** を作成

**rh-virt** は **openshift-virtualization** MCP をサポートする **mcps.json** をインストールする。Podman  
または Kubernetes を使用して KubeVirt API をインストールする

KUBECONFIG を設定する。KUBECONFIG を設定する

**mcps.json** を作成

```
{
  "mcpServers": {
    "openshift-virtualization": {
      "command": "podman",
      "args": [
        "run",
        "--rm",
        "-i",
        "--network=host",
        "--userns=keep-id:uid=65532,gid=65532",
        "-v", "${KUBECONFIG}:/kubeconfig:ro,Z",
        "--entrypoint", "/openshift-mcp-server",
        "quay.io/redhat-user-workloads/crt-nshift-lightspeed-tenant/openshift-mcp-
server@sha256:2f52c860f91ab3c8a5129b727bdef0d620e733013f073b10355866c45eafd053",
        "--kubeconfig", "/kubeconfig",
        "--toolsets", "core,kubevirt"
      ]
    }
  }
}
```

```

    ],
    "env": {
      "KUBECONFIG": "${KUBECONFIG}"
    }
  }
}
}
}

```

オプション:

- `--users=keep-id:uid=65532,gid=65532` `rootless` Podman ユーザーを `namespace` ユーザーとして実行
- `--SELinux,Z` `kubeconfig` SELinux レベルを `Z` に設定
- `--entrypoint /openshift-mcp-server` MCP サーバーを実行
- `--toolsets core,kubevirt` Kubernetes と KubeVirt の両方をサポート
- `--network=host` Kubernetes API に接続するためのネットワーク



SHA256 ハッシュを [quay.io](https://quay.io) から取得

## KUBECONFIG の設定

MCP ツール `KUBECONFIG` を `kubeconfig` のパスに設定

```
export KUBECONFIG="/path/to/your/kubeconfig"
```

確認コマンド

```
echo $KUBECONFIG
```

`mcp.json` に `${KUBECONFIG}` を設定し、MCP ツール `kubeconfig` を実行

## Podman の設定

Podman を MCP ツール `IDE` の実行環境として設定

```
podman pull quay.io/redhat-user-workloads/crt-nshift-lightspeed-tenant/openshift-mcp-server@sha256:2f52c860f91ab3c8a5129b727bdef0d620e733013f073b10355866c45eafd053
```

確認コマンド

agentic skill KUBECONFIG VirtualMachine:

namespace 名前空間:

vm-inventory vm-snapshot-list)

( / / vm-lifecycle-manager)

(vm-delete vm-snapshot-restore vm-snapshot-delete) VM  
 VM

protected: "true" VM vm-delete

vm-create VM ErrorUnschedulable  
 ( taint )

MCP

IDE KUBECONFIG

```
echo $KUBECONFIG
```

Podman

```
podman info --format '{{.Host.Os}}'
```

```
podman run --rm --entrypoint /openshift-mcp-server quay.io/redhat-user-workloads/crt-
nshift-lightspeed-tenant/openshift-mcp-
server@sha256:2f52c860f91ab3c8a5129b727bdef0d620e733013f073b10355866c45eafd053 --help
```

```
lola list
```

IDE AGENTS.md IDE

VM Permission Denied

ServiceAccount VirtualMachine RBAC

```
oc auth can-i create virtualmachines -A
```

```
oc auth can-i delete virtualmachines -A
```

oooooooooooooooooooooooooooo

oooooooo

oooooooooooooooooooooooooooooooooooo

ooo

- **rh-virt** oooooooooOpenShift Virtualization oooooo VM oooooooooooooo 10 oooooooooooooo
- OpenShift MCP oooooo rootless Podman oooooooooooooooooooooooooooooo KUBECONFIG ooooooooooooooooooooo
- Lola oooClaude Code ooo Cursor IDE o **rh-virt** ooooooooooooooooooooo ooooooooooooooooooooo
- AGENTS.md ooooooooooooooooooooo MCP ooooooooooooooooooooooooooooooooooooooooooooo
- ooooooooooooooooooooooooooooooooooooo Human-in-the-loop oooooooooooooo
- IDE o "List all VMs" oooooo**vm-inventory** ooooooooooooooooooooooooooooooooooooo

oo

- [AI ooooooooo VM ooooooooooooo](#)
- [rh-virt ooooooooooooo](#)
- [Lola Skill Pack Manager](#)
- [OpenShift MCP Server](#)
- [Model Context Protocol oo](#)
- [OpenShift Virtualization ooooooo](#)

## oooooooo VM oo

oooooooooooo**vm-create** oooooooooooooooooooooooooooooooooooooooooooooooooooooVM  
ooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooOS ooooooooooooooooooooooooooooo  
KubeVirt VirtualMachine ooooooooooooo

oooo

- [AI ooooooooo VM oooooo](#) ooooooooooooooooooooo (ooooooooooooooooooooMCP ooooooooooooooooooooooooooooo)
- OpenShift Virtualization ooooooooooooo OpenShift oooooo
- **KUBECONFIG** oooooooooooooo kubeconfig ooooooooooooooooooooo
- VirtualMachine ooooooooooooooooooooo namespace

oooooooooooooooooooo namespace ooooooooooooooooooooo

```
oc create namespace my-vms
oc create namespace development
```

111

**vm-create** [options] 6 [options]

1. **命名空间** — 在命名空间 `web-01` 中创建名为 `vm` 的虚拟机 (?: "Create a Fedora VM named `web-01` in namespace `dev`")  
命名空间 `web-01` 不存在，需要先创建命名空间
2. **命名空间** — 在命名空间 `web-01` 中创建名为 `vm` 的虚拟机 (?: "Create a Fedora VM in namespace `web-01`")  
命名空间 `web-01` 不存在，需要先创建命名空间
3. **命名空间** — 在命名空间 `web-01` 中创建名为 `vm` 的虚拟机 (?: "Create a Fedora VM in namespace `web-01`")  
命名空间 `web-01` 不存在，需要先创建命名空间
4. **VM 创建** — 在命名空间 `web-01` 中创建名为 `vm` 的虚拟机 (?: "Create a Fedora VM in namespace `web-01`")  
命名空间 `web-01` 不存在，需要先创建命名空间
5. **命名空间** — 在命名空间 `web-01` 中创建名为 `vm` 的虚拟机 (?: "Create a Fedora VM in namespace `web-01`")  
命名空间 `web-01` 不存在，需要先创建命名空间
6. **命名空间** — 在命名空间 `web-01` 中创建名为 `vm` 的虚拟机 (?: "Create a Fedora VM in namespace `web-01`")  
命名空间 `web-01` 不存在，需要先创建命名空间

## ステップ 1: Fedora VM

**Fedora**

1. IDE □□□□□□□□□□□□□□□□

Create a Fedora VM in namespace my-vms

- ## 2. `StorageClass` (namespace)

```
Here's the full configuration before I proceed:
```

```
Virtual Machine Configuration
```

| Parameter | Value     | Notes                 |
|-----------|-----------|-----------------------|
| VM Name   | fedora-vm | validated             |
| Namespace | my-vms    | from request          |
| OS        | fedora    | Fedora Linux (latest) |

```
Here's the full configuration before I proceed:
```

```
Virtual Machine Configuration
```

| Parameter | Value     | Notes                 |
|-----------|-----------|-----------------------|
| VM Name   | fedora-vm | validated             |
| Namespace | my-vms    | from request          |
| OS        | fedora    | Fedora Linux (latest) |

```
Here's the full configuration before I proceed:
```

```
Virtual Machine Configuration
```

| Parameter | Value     | Notes                 |
|-----------|-----------|-----------------------|
| VM Name   | fedora-vm | validated             |
| Namespace | my-vms    | from request          |
| OS        | fedora    | Fedora Linux (latest) |



|              |          |                               |
|--------------|----------|-------------------------------|
| Performance  | u1       | General Purpose               |
| Size         | medium   | 1 vCPU, 4Gi RAM               |
| Storage      | 30Gi     | root disk                     |
| StorageClass | lvms-vg1 | cluster default (topolvm.io)  |
| Autostart    | no       | start manually after creation |

This will create a VirtualMachine and consume cluster resources.  
Confirm: yes / no / modify?

### 3. 確認メッセージ

yes

### 4. 実行コマンド `vm_create` MCP を実行して VM を作成する

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: fedora-vm
  namespace: my-vms
spec:
 instancetype:
    kind: VirtualMachineClusterInstancetype
    name: u1.medium
  preference:
    kind: VirtualMachineClusterPreference
    name: fedora
  runStrategy: Always
  dataVolumeTemplates:
  - metadata:
      name: fedora-vm-rootdisk
    spec:
      sourceRef:
        kind: DataSource
        name: fedora
        namespace: openshift-virtualization-os-images
      storage:
```

```

resources:
  requests:
    storage: 30Gi
template:
spec:
  domain:
  devices: {}
  resources: {}
  terminationGracePeriodSeconds: 180
  volumes:
  - dataVolume:
    name: fedora-vm-rootdisk
    name: fedora-vm-rootdisk

```

##### 5. `DataVolume` `VirtualMachine` (205 `VirtualMachine`) `VM` `VirtualMachine`

✓ Virtual Machine Created Successfully

VM: fedora-vm (namespace: my-vms)

OS: Fedora | Size: medium (u1) | Storage: 30Gi | Status: Running

Accessing the VM

1. Serial console: `virtctl console fedora-vm -n my-vms`
2. VNC: OpenShift Console → Virtualization → VMs → fedora-vm → Console
3. SSH: Get IP from VMI, `ssh fedora@<ip>`

Default credentials: fedora

`VirtualMachine` `VirtualMachine`

```
oc get vm fedora-vm -n my-vms
```

Output:

```

NAME    AGE    STATUS    READY
fedora-vm 7m    Running    True

```

`DataVolume` `PVC` `VirtualMachine`

```
oc get datavolume -n my-vms
```

```

NAME                PHASE    PROGRESS    RESTARTS    AGE
fedora-vm-rootdisk Succeeded 100.0%      0           7m

```

```
oc get pvc -n my-vms
```

| NAME               | STATUS | VOLUME                                   | CAPACITY | ACCESS MODES | STORAGECLASS |
|--------------------|--------|------------------------------------------|----------|--------------|--------------|
| fedora-vm-rootdisk | Bound  | pvc-ebfcfd3d-f865-4186-b306-08dfc1d60272 | 30Gi     | RWO          | lvms-vg1     |

## 2: Fedora VM

[illegible]

## 1. IDE

Create a Fedora VM named `fedora-dev` with 2 vCPUs and 8GB RAM and a 50GB disk in the development namespace

2.

| Parameter    | Value       | Notes                         |
|--------------|-------------|-------------------------------|
| VM Name      | fedora-dev  | validated                     |
| Namespace    | development | from request                  |
| OS           | fedora      | Fedora Linux (latest)         |
| Performance  | u1          | General Purpose               |
| Size         | large       | 2 vCPU, 8Gi RAM               |
| Storage      | 50Gi        | root disk                     |
| StorageClass | lvms-vg1    | cluster default (topolvm.io)  |
| Autostart    | no          | start manually after creation |

| Parameter    | Value       | Notes                         |
|--------------|-------------|-------------------------------|
| VM Name      | fedora-dev  | validated                     |
| Namespace    | development | from request                  |
| OS           | fedora      | Fedora Linux (latest)         |
| Performance  | u1          | General Purpose               |
| Size         | large       | 2 vCPU, 8Gi RAM               |
| Storage      | 50Gi        | root disk                     |
| StorageClass | lvms-vg1    | cluster default (topolvm.io)  |
| Autostart    | no          | start manually after creation |

This will create a VirtualMachine and consume cluster resources.  
Confirm: yes / no / modify?

### 3. VM 启动失败 autostart 配置:

```
modify: change autostart to yes
```

#### 4. 虚拟机 autostart 虚拟机启动时自动启动 VM 虚拟机启动时自动启动

✓ Virtual Machine Created and Starting

```
VM: fedora-dev (namespace: development)
```

OS: Fedora | Size: large (u1) | Storage: 50Gi | Status: Provisioning

Provisioning: ~2-5 min (Provisioning → Running)

Default credentials: fedora

○○○○○○○○○○○○○○○○○○○○VM ○○○○○○○○

```
oc get vm fedora-dev -n development
```

```
NAME    AGE STATUS READY
fedora-dev 7m Running True
```

PVC □□□□□□□□□□□□□□□□

```
oc get pvc -n development
```

| NAME                | STATUS | VOLUME                                   | CAPACITY | ACCESS | MODES | STORAGECLASS |
|---------------------|--------|------------------------------------------|----------|--------|-------|--------------|
| fedora-dev-rootdisk | Bound  | pvc-35dbc583-8f54-47a8-8497-f941b72633e9 | 50Gi     | RWO    |       | lvms-vg1     |

□□□□□□□□ **3:** □□□□□□□□□□□□□□□□□□



```

Worker Node   taint   (?:
virtualization=true:NoSchedule)   taint   VM

```

VM

1. IDE

Create a Fedora VM named fedora-db with 4 vCPUs and 16GB RAM in the production namespace, memory-optimized, with 100GB storage, and autostart

2. m1 (memory-optimized) xlarge

Virtual Machine Configuration

| Parameter    | Value      | Notes                             |
|--------------|------------|-----------------------------------|
| VM Name      | fedora-db  | validated                         |
| Namespace    | production | from request                      |
| OS           | fedora     | Fedora Linux (latest)             |
| Performance  | m1         | Memory Optimized                  |
| Size         | xlarge     | 4 vCPU, 32Gi RAM                  |
| Storage      | 100Gi      | root disk                         |
| StorageClass | lvms-vg1   | cluster default (topolvm.io)      |
| Autostart    | yes        | starts immediately after creation |

This will create a VirtualMachine and consume cluster resources.  
Confirm: yes / no / modify?

3. VM

VM Scheduling Issue Detected

Status: ErrorUnschedulable  
Root Cause: Node taints – 0/3 nodes are available: 3 node(s) had taints

that the pod didn't tolerate

I consulted the `scheduling-errors` troubleshooting guide and found that the worker nodes have a `"virtualization=true:NoSchedule"` taint.

The `vm_create` tool does not support tolerations directly, but I can apply a post-creation workaround:

#### Recommended Solution

Action: Add toleration for `"virtualization=true:NoSchedule"` to the VM spec using `resources_create_or_update`, then restart the VM

Impact: The VM will be able to schedule on nodes with the `virtualization=true` taint

#### Options:

- 1) Apply workaround (add toleration and restart)
- 2) Manual fix (I'll show you the `oc patch` command)
- 3) Cancel (delete the VM)

#### 4. apply workaround

apply workaround

#### 5. VM spec toleration

##### ✓ Workaround Applied

Action: Added toleration for `virtualization=true:NoSchedule`, restarted VM

Status: Running

VM: `fedora-db` (namespace: `production`)

OS: Fedora | Size: `xlarge` (m1 - Memory) | Storage: `100Gi` | Status: Running

##### Accessing the VM

1. Serial console: `virtctl console fedora-db -n production`
2. VNC: OpenShift Console → Virtualization → VMs → `fedora-db` → Console
3. SSH: Get IP from VMI, `ssh fedora@ip`

Default credentials: `fedora`

`VirtualMachineClusterInstancetype` 2

| 名前 | 単位      | 説明                         |
|----|---------|----------------------------|
| u1 | 単位 (名前) | 仮想マシン CPU 単位 — 仮想マシンインスタンス |
| m1 | 名前      | 仮想マシンインスタンス                |
| o1 | 名前      | 仮想マシン — 仮想マシン              |



仮想マシンインスタンスの CPU 単位は、`oc get VirtualMachineClusterInstancetype` コマンドを実行して確認できます。

仮想マシン

OpenShift 4.22 仮想マシンインスタンス **u1** (単位) 仮想マシン CPU 単位

| 名前          | vCPU | RAM   |
|-------------|------|-------|
| nano        | 1    | 512Mi |
| micro       | 1    | 1Gi   |
| small       | 1    | 2Gi   |
| medium (名前) | 1    | 4Gi   |
| large       | 2    | 8Gi   |
| xlarge      | 4    | 16Gi  |
| 2xlarge     | 8    | 32Gi  |
| 4xlarge     | 16   | 64Gi  |
| 8xlarge     | 32   | 128Gi |

仮想マシン (m1) 仮想マシン vCPU 仮想マシン RAM 仮想マシン m1.large 2 vCPU 16Gi RAM 仮想マシン u1.large 2 vCPU 8Gi RAM 仮想マシン

仮想マシン 2 vCPU 8GB RAM 仮想マシン spec 仮想マシン

```
oc get VirtualMachineClusterInstancetype -o custom-columns="NAME:.metadata.name,CPU:.spec.cpu.guest,MEMORY:.spec.memory.guest" | grep u1
```

```
u1.2xlarge 8 32Gi
u1.2xmedium 2 4Gi
u1.4xlarge 16 64Gi
u1.8xlarge 32 128Gi
u1.large 2 8Gi
u1.medium 1 4Gi
u1.micro 1 1Gi
u1.nano 1 512Mi
u1.small 1 2Gi
u1.xlarge 4 16Gi
```

OS 詳細

OpenShift は Linux 仮想マシン (Fedora VM) を使用し、`openshift-virtualization-os-images` namespace にデフォルトでインストールされています。

インストールされた OS の一覧を確認します。

```
oc get datasource -n openshift-virtualization-os-images
```

| NAME            | AGE |
|-----------------|-----|
| centos-stream10 | 17d |
| centos-stream9  | 17d |
| fedora          | 17d |
| rhel10          | 17d |
| rhel7           | 17d |
| rhel8           | 17d |
| rhel9           | 17d |

| OS             | Image      |
|----------------|------------|
| Fedora (デフォルト) | fedora     |
| RHEL           | cloud-user |
| CentOS Stream  | centos     |



この表は、`DataSource` リソースの `image` フィールドに格納されている OS の名前と、`vm_create` URL に格納されているイメージの URL を示しています。

`vm_create`  
URL

インストール

`vm-create` コマンドを使用して、仮想マシンを作成します。

| OS           | Image              |
|--------------|--------------------|
| 30Gi (デフォルト) | 30Gi               |
| 50Gi         | 50Gi               |
| 100Gi        | 100Gi — 100Gi      |
| 200Gi        | 200Gi (例: "200Gi") |

インストールされた StorageClass の一覧を確認します。

- インストールされた StorageClass の一覧を確認します。
- デフォルトの StorageClass (`storageclass.kubernetes.io/is-default-class` フィールドが `true` のもの) を確認します。
- StorageClass の名前 (通常は CSI ドライバーの名前) を確認します。
- インストールされた DataVolume の一覧を確認します。



StorageClass

```
oc get sc
```

| NAME     | PROVISIONER | RECLAIMPOLICY | VOLUMEBINDINGMODE | ALLOWVOLUMEEXPANSION | AGE  |     |
|----------|-------------|---------------|-------------------|----------------------|------|-----|
| lvms-vg1 | (default)   | topolvm.io    | Delete            | WaitForFirstConsumer | true | 17d |

Pod VM

(`create a VM with a secondary network on vlan-100`) `vm_create`

## Human-in-the-Loop

VM

- 
- 
- yes** —
  - no** —
  - modify** — (`modify: change storage to 50Gi`)
- 

VM

(**ErrorUnschedulable**)

3

- taint** (`virtualization=true:NoSchedule`) — Worker Node VM (`tolerate`) **taint** (`virtualization=true:NoSchedule`) `toleration` VM spec
- VM CPU VM
- Node selector mismatch** — VM

エラーメッセージ (ErrorDataVolumeNotReady)

エラーメッセージを確認してください

- **DataVolume** **still** **provisioning** —  
DataVolume がまだプロビジョニング中です。
- **StorageClass not found** — **StorageClass** **StorageClass** **StorageClass** VM **StorageClass** VM **StorageClass**
- **namespace** — **namespace** **namespace**

エラーメッセージを確認してください

VM の操作

VM の操作

|          |                                                                                    |
|----------|------------------------------------------------------------------------------------|
| 操作       | コマンド                                                                               |
| コンソール    | <code>virtctl console &lt;vm-name&gt; -n &lt;namespace&gt;</code>                  |
| VNC 接続   | OpenShift Console → Virtualization → VirtualMachines → VM 名 → Console              |
| SSH      | VirtualMachineInstance 名 VM IP <code>ssh &lt;user&gt;@&lt;ip&gt;</code>            |
| ポートフォワード | <code>virtctl port-forward vmi/&lt;vm-name&gt; -n &lt;namespace&gt; 8080:80</code> |

VM の IP アドレスを確認:

```
oc get vmi fedora-vm -n my-vms -o jsonpath='{.status.interfaces[0].ipAddress}'
```

```
10.128.0.102
```

操作

OS のインストールと SSH の接続

```
# インストール
virtctl console fedora-vm -n my-vms
# ログイン:
sudo passwd fedora
```

|        |            |
|--------|------------|
| OS     | ユーザー名      |
| Fedora | fedora     |
| RHEL   | cloud-user |

|                 |                 |
|-----------------|-----------------|
| OperatingSystem | OperatingSystem |
| CentOS Stream   | centos          |

Example

VM lifecycle manager:

```
oc delete vm <vm-name> -n <namespace>
```

VirtualMachine DataVolume PersistentVolumeClaim  
Delete VM fedora-vms in namespace my-vms

```
oc delete vm fedora-vms -n my-vms
oc delete vm fedora-dev -n development
```

Delete VM fedora-vms in namespace my-vms

See

- [AI VM lifecycle manager](#)
- [rh-virt](#)
- [OpenShift MCP Server](#)
- [OpenShift Virtualization](#)
- [KubeVirt](#)

## AI VM lifecycle manager

vm-lifecycle-manager (Human-in-the-loop) VM lifecycle manager

See

- [AI VM lifecycle manager](#) (OpenShift MCP Server)
- [OpenShift Virtualization](#)
- [KUBECONFIG](#) kubeconfig
- VM lifecycle manager 1 (OpenShift MCP Server)

namespace

```
oc create namespace my-vms
```

この VM は IDE をインストールした **vm-create** をインストール

Create a Fedora VM named lifecycle-demo in namespace my-vms

この VM をインストール

```
oc get vm -n my-vms
```

例:

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| lifecycle-demo | 2m  | Running | True  |

例

**vm-lifecycle-manager** の例

**vm-lifecycle-manager** は start、stop、restart の 3 つの操作を実行

1. 操作を実行する - 特定の namespace の VM を (start、stop、restart) 操作を実行する
2. 例 - **vm\_lifecycle** MCP は VM の RunStrategy を restart 操作を実行する VM を 2 つの Kubernetes の resourceVersion を更新する
3. 例 - RunStrategy を VM の namespace に VM を更新する

**vm-delete** の例

**vm-delete** は 5 つの操作を実行

1. 例 - 特定の VM を **protected: "true"** に (特定の VM を) 削除する
2. 例 - 特定の VM を DataVolume を PVC に更新する
3. 例 - 特定の VM を (特定の VM を) 削除する (特定の VM を) 削除する 3 操作を実行する
4. 例 - 特定の VM を 削除する
5. 例 - 特定の VM を (特定の VM を) 削除する

例 **1: 特定の VM を**

特定の **vm-lifecycle-manager** は VM を削除

1. IDE をインストール

Start VM lifecycle-demo in namespace my-vms

2. 確認 VM lifecycle-demo の状態

## VM Lifecycle Operation

| Parameter | Value            | Impact             |
|-----------|------------------|--------------------|
| VM Name   | `lifecycle-demo` | from user          |
| Namespace | `my-vms`         | from user          |
| Action    | `start`          | consumes resources |

Confirm: yes/no

3. 確認 VM lifecycle-demo の状態

yes

4. 確認 VM lifecycle-demo の状態 RunStrategy は Halted 状態 Always 状態 VM は Running 状態

## VM Start Successful

\*\*VM\*\*: `lifecycle-demo` | \*\*Namespace\*\*: `my-vms` | \*\*RunStrategy\*\*: Always

\*\*Impact\*\*: Running, consuming resources (CPU/memory).

Access: virtctl console or SSH.

RunStrategy: Always (auto-restart on crash)

\*\*Next\*\*: "Show status of VM lifecycle-demo" or "List VMs in namespace my-vms"

確認 VM lifecycle-demo の状態

oc get vm lifecycle-demo -n my-vms

| NAME           | AGE | STATUS  | READY |
|----------------|-----|---------|-------|
| lifecycle-demo | 10m | Running | True  |

確認 VM lifecycle-demo の状態

確認 VM lifecycle-demo の状態

1. IDE 確認 VM lifecycle-demo の状態

Stop VM lifecycle-demo in namespace my-vms

2. 確認 VM lifecycle-demo の停止

## VM Lifecycle Operation

| Parameter | Value            | Impact            |
|-----------|------------------|-------------------|
| -----     | -----            | -----             |
| VM Name   | `lifecycle-demo` | from user         |
| Namespace | `my-vms`         | from user         |
| Action    | `stop`           | graceful shutdown |

Confirm: yes/no

3. 確認 VM lifecycle-demo の停止が成功している (RunStrategy が Halted になっている)

## VM Stop Successful

\*\*VM\*\*: `lifecycle-demo` | \*\*Namespace\*\*: `my-vms` | \*\*RunStrategy\*\*: Halted

\*\*Impact\*\*: Stopped, resources freed. State preserved.

Start: "Start VM lifecycle-demo in namespace my-vms"

RunStrategy: Halted (stays off)

\*\*Next\*\*: "Show status of VM lifecycle-demo" or "List VMs in namespace my-vms"

VM は `VirtualMachineInstance` (VMI) オブジェクトとして管理され、`VirtualMachine` オブジェクト (DataVolume/PVC) が VM の `Halting` ステータスを監視し、CPU を解放する。

### 3: VM の停止

Kubernetes API を使って VM の状態を確認する。

1. IDE で確認する

Restart VM lifecycle-demo in namespace my-vms

2. 確認 VM lifecycle-demo の再起動

## VM Lifecycle Operation

| Parameter | Value            | Impact    |
|-----------|------------------|-----------|
| -----     | -----            | -----     |
| VM Name   | `lifecycle-demo` | from user |
| Namespace | `my-vms`         | from user |

```
| Action | 'restart' | brief interruption ~1-2min |
```

Confirm: yes/no

### 3. 確認 VM の再起動が完了したかどうか

```
## VM Restart Successful
```

```
**VM**: 'lifecycle-demo' | **Namespace**: 'my-vms' | **RunStrategy**: Always
```

Composite restart steps completed:

1. Stopped VM -> verified status: Stopped
2. Started VM -> verified status: Running

**\*\*Impact\*\***: Running after stop+start. Brief interruption (~1-2min).  
Monitor app logs for any issues.

**\*\*Next\*\***: "Show status of VM lifecycle-demo" or "List VMs in namespace my-vms"

Pod IP MAC VirtualMachineInstance  
ID Kubernetes Service VMI

### 4: VM の削除

`vm-delete` VM を削除

#### 1. IDE で VM を削除

Delete VM lifecycle-demo in namespace my-vms

#### 2. VM の削除を確認

```
## VM Deletion - Review Scope
```

```
**VM**: 'lifecycle-demo' | **Namespace**: 'my-vms' | **Status**: Running
```

**\*\*Resources\*\***: VM 'lifecycle-demo' (Age: 10m, vCPU: 1, Memory: 4Gi)

**\*\*Storage\*\***: DataVolume 'lifecycle-demo-rootdisk' (30Gi),  
PVC 'lifecycle-demo-rootdisk' (30Gi) - Total: 30Gi

```
### Deletion Options
```

- \*\*1: VM Only\*\*** - Preserves storage for reuse
- \*\*2: VM + Storage\*\*** <- Recommended (test/dev) - Frees storage
- \*\*3: Cancel\*\***

Select (1, 2, or 3):

3. 2

2

4. VM

```
## PERMANENT DELETION - Typed Confirmation Required

**CANNOT BE UNDONE**

**Will delete**:
- VirtualMachine: `lifecycle-demo` (namespace: `my-vms`)
- DataVolume: `lifecycle-demo-rootdisk`
- PVC: `lifecycle-demo-rootdisk`
- All data lost

Type `lifecycle-demo` to confirm: _____
```

5.

lifecycle-demo

6. VM VM

```
## VM Deleted (Complete Cleanup)

**Deleted**: VM + DataVolume + PVC | **Freed**: 30Gi
**Impact**: Permanent removal. Cannot recover.
**Verify**: "List VMs in namespace my-vms" - VM should not appear
```

VM

```
oc get vm -n my-vms
```

No resources found in my-vms namespace.

```
oc get pvc -n my-vms
```

No resources found in my-vms namespace.



## 5: VM 環境

**protected: "true"** VM VM VM

[illegible]

```
oc label vm <vm-name> -n <namespace> protected=true
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc get vm <vm-name> -n <namespace> --show-labels
```

VM

## 1. IDE

Delete VM production-db in namespace production

2.

## Cannot Delete Protected VM

```
VM 'production-db' has label 'protected: "true"' - deletion blocked.
```

To remove the protection label (requires admin approval):

```
oc label vm production-db -n production protected-
```

Operation cancelled.

[illegible]

## □□□□ 6: VM □□□□□□□□□□□□□□

[illegible]

## 1. IDE 環境の構築

Start VM lifecycle-demo in namespace my-vms

## 2. VM 環境構築:

```
## VM Already Running
```

```
**VM**: 'lifecycle-demo' | **Namespace**: 'my-vms' | **Status**: Running
```

```
Result: No action taken - VM already in desired state.
To restart: "Restart VM lifecycle-demo in namespace my-vms"
```

## ステップ 7: VM を削除

VM を削除するには、VM を削除するコマンドを実行します。VM を削除するコマンドを実行します。

### 1. IDE を開く

```
Delete VM lifecycle-demo in namespace my-vms
```

### 2. 1 を入力する

1

### 3. VM を削除する VirtualMachine オブジェクト

```
## VM Deleted (VM Only)

**Deleted**: VM `lifecycle-demo`
**Preserved**: PVC `lifecycle-demo-rootdisk` (30Gi) - available for reattachment
**Impact**: VM definition removed. Storage retained.
```

PVC を namespace に表示する

```
oc get pvc -n my-vms
```

| NAME                    | STATUS | VOLUME                                   |
|-------------------------|--------|------------------------------------------|
| lifecycle-demo-rootdisk | Bound  | pvc-abc12345-6789-0def-ghij-klmnopqrstuv |
| 30Gi                    |        |                                          |
| RWO                     |        | lvms-vg1                                 |

## RunStrategy

running VM を削除する VirtualMachine オブジェクト runStrategy を削除する VM を削除する

| RunStrategy | RunStrategy | RunStrategy |
|-------------|-------------|-------------|
| start       | Always      | VM を削除する    |
| stop        | Halted      | VM を削除する    |
| restart     | Always      | VM を削除する    |

Always VM を削除する (VM を削除する OOM) VM を削除する VM

VM 状态 **Halted** 虚拟机 **VM** 生命周期策略 **runStrategy** 策略 **running**



KubeVirt 策略 **RerunOnFailure** (策略名称) 策略 **Manual** (策略名称)  
策略名称 **vm-lifecycle-manager** 策略名称 **2**  
策略名称 **Always** 策略名称 **Halted** 策略名称

VM 策略 **RunStrategy** 策略名称

```
oc get vm lifecycle-demo -n my-vms -o jsonpath='{.spec.runStrategy}'
```

## Human-in-the-Loop 策略

策略名称 **vm-lifecycle-manager**

策略名称 **(vm-lifecycle-manager)**:

- 策略名称 **VM** 策略名称 **namespace** 策略名称
- 策略名称 **"yes"** 策略名称
- 策略名称 **no** 策略名称 **VM** 策略名称
- 策略名称 (策略 **VM**) 策略名称 **VM** 策略名称

策略名称 **(vm-delete)**:

- 策略名称 **VM** 策略名称
- 策略名称 **VM** 策略名称
- 策略名称 **VM** 策略名称 (策略名称) 策略名称
- 策略名称
- 策略名称 **VM** (策略 **protected: "true"**) 策略名称

策略名称

策略名称

策略名称:

```
oc delete vm lifecycle-demo -n my-vms
```

策略名称 **IDE** 策略名称 **Delete VM lifecycle-demo in namespace my-vms** 策略名称 **vm-delete**

namespace 策略名称:

```
oc delete namespace my-vms
```

111

- [illegible]

11

- AI 環境構築 VM 環境
- 環境構築環境構築
- AI 環境構築環境構築環境構築
- rh-virt 環境構築環境構築
- OpenShift MCP 環境構築
- OpenShift Virtualization 環境構築
- KubeVirt RunStrategy 環境構築

## VM □□□□□□□□

[illegible]

□ □ □ □

- **AI** **Container VM** **Container** **Container** (Container Container Container MCP Container Container Container Container)
- OpenShift Virtualization Container OpenShift Container
- **KUBERNETES** Container **kubeconfig** Container Container
- Container Container Container Container 1 Container
- Container namespace Container VirtualMachine Container DataVolume/PVC Container Container Container RBAC Container

namespace

```
oc create namespace my-vm
oc create namespace qa-environment
```

この VM の ID を `vm-create` コマンドで取得:

```
Create a Fedora VM named web-server in namespace my-vms
```

この VM を取得:

```
oc get vm -n my-vms
```

結果:

| NAME       | AGE | STATUS  | READY |
|------------|-----|---------|-------|
| web-server | 5m  | Running | True  |

この

`vm-clone` コマンドで 5 を指定:

- このコマンドは、指定された namespace の VM の spec (DataVolume/PVC) を複製します。
- 指定された namespace の VM を複製します。
- このコマンドは、指定された VM を複製します (full copy、shared reference、new empty disk)。
- このコマンドは、指定された VM の IP、MAC、UUID、CPU、cloud-init、namespace の ResourceQuota を複製します (80% の制限)。
- このコマンドは、指定された VM を複製します (VirtualMachine)。

このコマンドは

VM を複製するときに、指定された VM の spec を複製します。

| 名前                 | 説明                                                  | オプション                  | デフォルト値                 |
|--------------------|-----------------------------------------------------|------------------------|------------------------|
| Clone Storage      | StorageProfile (CSI host-assisted copy) を使用して複製します。 | このオプションは、複製するときに使用します。 | 5x10                   |
| Reference Existing | 指定された VM の PVC を複製します。                              | このオプションは、複製するときに使用します。 | このオプションは、複製するときに使用します。 |

| 名前                | 説明                                         | 備考                         | 関連情報 |
|-------------------|--------------------------------------------|----------------------------|------|
| New Empty Storage | Storage Class を指定して新しい Storage Class を作成する | 名前 —<br>作成する Storage Class | 名前   |

Clone Existing Storage Reference

## 1. VM を作成する

VM を作成する

### 1. IDE を作成する

Clone web-server to web-server-test in namespace qa-environment

### 2. VM を作成する 30Gi を指定して qa-environment に web-server-test VM を作成する

#### ## VM Cloning - Storage Strategy

**\*\*Source VM\*\*:** `web-server` (namespace: `my-vms`)

**\*\*Source Storage\*\*:** web-server-rootdisk (30Gi)

**\*\*Select cloning strategy:\*\***

1. **\*\*Clone Storage\*\*** - Full copy, independent storage (~5-10 min, 30Gi new allocation)

2. **\*\*Reference Existing\*\*** - Shared disk (dangerous, both VMs access same storage)

3. **\*\*New Empty Storage\*\*** - Fresh disk, no data copied (30Gi new allocation)

4. **\*\*Cancel\*\*** - Abort operation

**\*\*Select option (1-4):\*\***

### 3. VM を作成する 1 VM を作成する

1

### 4. namespace に ResourceQuota を指定する

#### ## VM Clone Configuration - Review

**\*\*Source\*\*:** `web-server` (my-vms) - u1.medium, 1 vCPU, 4Gi

**\*\*Target\*\*:** `web-server-test` (qa-environment) - Same config, starts Stopped

**\*\*Storage\*\*:** Clone Storage - 30Gi independent copy (~5-10 min)

**\*\*Resource Impact\*\*:** 1 vCPU, 4Gi RAM, 30Gi disk

```
**What changes**: IP addresses, hostname, MAC addresses, firmware UUID  
**What's preserved**: Instance type, vCPU/memory, network config, cloud-init  
  
**Proceed with VM cloning? (yes/no)**
```

5.

yes

6.  **DataVolume**  **VirtualMachine**

```
## VM Cloned Successfully  
  
**Source**: `web-server` (my-vms)  
**Target**: `web-server-test` (qa-environment) - Status: Stopped (ready to start)  
**Config**: u1.medium, 1 vCPU, 4Gi, 30Gi  
  
**Storage**: Cloned in 7m23s - Independent storage, changes won't affect source  
  
**Next**: Start with "Start VM web-server-test in namespace qa-environment"
```

oc get vm -n qa-environment

| NAME            | AGE | STATUS  | READY |
|-----------------|-----|---------|-------|
| web-server-test | 8m  | Stopped | False |

oc get datavolume -n qa-environment

| NAME                     | PHASE     | PROGRESS | RESTARTS | AGE |
|--------------------------|-----------|----------|----------|-----|
| web-server-test-rootdisk | Succeeded | 100.0%   |          | 8m  |

**2:**

VM  (cloud-init)  VM   
OS

1. IDE

Clone web-server to clean-worker in namespace my-vms with empty storage

2. 3 VMs 생성

3

3. VM 생성 후 DataVolume 생성

## VM Cloned Successfully

\*\*Source\*\*: `web-server` (my-vms)

\*\*Target\*\*: `clean-worker` (my-vms) - Status: Stopped (ready to start)

\*\*Config\*\*: u1.medium, 1 vCPU, 4Gi, 30Gi

\*\*Storage\*\*: New empty storage created - OS installation may be required

\*\*Next\*\*: Start with "Start VM clean-worker in namespace my-vms"

3. VM 생성

vm-clone 1 VM 생성 후 DataVolume 생성

1. IDE 생성

Create 3 copies of web-server named app-01, app-02, app-03 in namespace qa-environment

2. VM 생성 후 DataVolume 생성

## Batch Cloning: 3 VMs from `web-server`

\*\*Source\*\*: `web-server` (my-vms) - u1.medium, 1 vCPU, 4Gi, 30Gi

\*\*Targets\*\*: app-01, app-02, app-03 (qa-environment)

\*\*Total Resource Impact\*\*: 3 vCPU, 12Gi RAM, 90Gi disk

\*\*Select storage strategy (applied to all clones, 1-4):\*\*

3. VM 생성 후 DataVolume 생성

## Clone 1 of 3: app-01

\*\*Source\*\*: web-server → \*\*Target\*\*: app-01 (qa-environment), 30Gi clone



**\*\*Proceed with VM cloning? (yes/no)\*\***

4. ☐ ☐ 1 ☐

## Batch Cloning Complete

**\*\*Results\*\*:**

app-01 (qa-environment) - Cloned in 6m12s

app-02 (qa-environment) - Cloned in 7m05s

app-03 (qa-environment) - Cloned in 5m48s

**\*\*Total Storage\*\*:** 90Gi cloned

**\*\*Next\*\*:** Start all with "Start VMs app-01, app-02, app-03 in namespace qa-environment"

☐

oc get vm -n qa-environment

| NAME            | AGE | STATUS  | READY |
|-----------------|-----|---------|-------|
| app-01          | 20m | Stopped | False |
| app-02          | 13m | Stopped | False |
| app-03          | 6m  | Stopped | False |
| web-server-test | 30m | Stopped | False |



☐

1

☐

5

☐

☐ 4: ☐

☐

VM

☐

PVC

☐

1. IDE ☐

Clone web-server to web-server-shared in namespace my-vms

2. ☐ 2 ☐

2

3. ☐

### WARNING: Shared Storage is Dangerous

Both VMs will share the same physical disk. Running both simultaneously risks data corruption.

This is only safe if the source VM remains stopped while the clone runs.

Recommended: Use Option 1 (Clone Storage) for an independent copy instead.

Proceed with shared storage anyway? (yes/cancel)

4.

cancel

5.

VM cloning cancelled by user. No resources were created.

## Namespace

**vm-clone**  Namespace  VM

Namespace

- Namespace  Namespace  PVC
- namespace
- **ResourceQuota**  namespace
- namespace (VM )  namespace ()  **RBAC**

Clone production-db from production namespace to staging namespace as staging-db

namespace



**vm-clone**  VM  (cloud-init )  
 VM

## Human-in-the-Loop

**vm-clone**

- —



```
oc delete vm web-server-test -n qa-environment
oc delete vm clean-worker -n my-vms
oc delete vm app-01 app-02 app-03 -n qa-environment
```

コマンドライン: IDE のコマンド "Delete VM web-server-test in namespace qa-environment" のコマンドは **vm-delete** のコマンドラインオプションを使用して実行します。

VM の削除:

```
oc delete vm web-server -n my-vms
```

namespace の削除:

```
oc delete namespace my-vms
oc delete namespace qa-environment
```

注意

- **vm-clone** コマンドは VM を複製して VM の固有の MAC アドレスと UUID を生成します。
- 3 つのコピータイプ: full copy (完全複製) shared reference (共有参照) new empty disk (新しい空ディスク)
- namespace の ResourceQuota を確認して、複製する VM の数を確認します。
- namespace に 5 つの VM を複製する場合は、namespace の ResourceQuota を確認します。
- namespace の RBAC を確認して、namespace に複製する VM の数を確認します。
- namespace の ResourceQuota を確認して、複製する VM の数を確認します。

関連

- [AI の VM の複製](#)
- [VM の複製](#)
- [AI の VM の複製](#)
- [rh-virt の複製](#)
- [OpenShift MCP Server](#)
- [OpenShift Virtualization の複製](#)
- [KubeVirt VirtualMachine API Reference](#)
- [CSI Volume Cloning](#)

## VM のスナップショット

スナップショットの作成 (vm-snapshot-create)、スナップショットのリスト (vm-snapshot-list)、スナップショットの復元 (vm-snapshot-restore)、スナップショットの削除 (vm-snapshot-delete)

このチュートリアルでは、OpenShift Virtualization を使用して、Kubernetes の namespace に VM を作成し、VM のスナップショットを作成する方法を説明します。

## 前提条件

- [AI 環境で VM を実行](#) の前提条件を満たす (Kubernetes のバージョンは 1.20 以上、MCP には 1.10 以上のバージョンが必要です)
- OpenShift Virtualization をインストールした OpenShift 環境
- `KUBECONFIG` 環境変数に `kubeconfig` のパスを設定する
- namespace 1 つに VM (ここでは `database-01` とする) を作成する
- CSI を使用して、VM のスナップショットを作成する (ここでは `VolumeSnapshotClass` を使用する)
- `VirtualMachineSnapshot` と `VirtualMachineRestore` の RBAC 権限を設定する

namespace `my-vm` を作成する

```
oc create namespace my-vm
```

namespace `my-vm` に VM `database-01` を作成する

```
Create a Fedora VM named database-01 in namespace my-vm
```

namespace `my-vm` に VM を取得する

```
oc get vm -n my-vm
```

出力結果:

| NAME        | AGE | STATUS  | READY |
|-------------|-----|---------|-------|
| database-01 | 5m  | Running | True  |

namespace `my-vm` に `VolumeSnapshotClass` を作成する

```
oc get volumesnapshotclass
```

namespace `my-vm` に `VolumeSnapshotClass` を取得する:

| NAME           | DRIVER     | DELETIONPOLICY | AGE |
|----------------|------------|----------------|-----|
| lvms-snapclass | topolvm.io | Delete         | 17d |

`VolumeSnapshotClass` を作成する

## 前提条件

4. 操作过程中需要人工干预，即需要 **Human-in-the-loop** (人工干预) 的情况。

### vm-snapshot-create 操作

**vm-snapshot-create** 操作需要满足以下前提条件：

1. **VM** 必须处于 **Running** 状态 — 创建快照时，**VM** 必须处于 **Running** 状态 (即虚拟机必须处于 **Running** 状态)。  
StorageClass 为 CSI 支持的 StorageClass，且 **VolumeSnapshotClass** 必须处于 **Active** 状态。
2. **VM** 必须支持快照功能 — 创建快照时，**VM** 必须支持快照功能。  
支持的 **VM** 类型包括：**QEMU**、**guest**、**agent** (freeze/thaw)。
3. 快照名称必须唯一 — 快照名称必须在命名空间中唯一，且符合 **YAML** 规范。
4. 操作成功 — 操作成功后，返回 **Succeeded**；操作失败，返回 **Failed**。

### vm-snapshot-list 操作

**vm-snapshot-list** 操作需要满足以下前提条件：  
**namespace** 必须存在 (即命名空间 **VM** 必须存在)。

### vm-snapshot-restore 操作

**vm-snapshot-restore** 操作需要满足以下前提条件：

1. **VM** 必须处于 **Running** 状态 — 恢复快照时，**VM** 必须处于 **Running** 状态。
2. 快照必须处于 **readyToUse: true** 状态 — 快照必须处于 **readyToUse: true** 状态。
3. 快照名称必须唯一 — 快照名称必须在命名空间中唯一。
4. 操作成功 — 操作成功后，返回 **yes/cancel**。
5. 操作失败 — 操作失败后，返回 **VirtualMachineRestore**。

### vm-snapshot-delete 操作

**vm-snapshot-delete** 操作需要满足以下前提条件：

1. 快照名称必须唯一 — 快照名称必须在命名空间中唯一。
2. 快照必须处于 **readyToUse: true** 状态 — 快照必须处于 **readyToUse: true** 状态。
3. 快照名称必须唯一 — 快照名称必须在命名空间中唯一。
4. 操作成功 — 操作成功后，返回 **yes/cancel**。

## 操作过程

操作过程中，需要人工干预的情况包括：**QEMU guest agent** 操作。

| VM ステータス | 接続状態          | 説明                                        |
|----------|---------------|-------------------------------------------|
| Stopped  | N/A           | VM が停止しているため、接続状態は N/A です。                |
| Running  | Connected     | VM が実行中であり、QEMU guest agent が正常に動作しています。  |
| Running  | Not connected | VM が実行中ですが、QEMU guest agent が正常に動作していません。 |

Linux VM では、QEMU guest agent をインストールし、Linux VM の VM 内で:

```
sudo dnf install qemu-guest-agent
sudo systemctl enable --now qemu-guest-agent
```

Windows VM では、Windows の VSS (Volume Shadow Copy Service) サービスが、VirtIO の VSS サービスと連携しています。

## ステップ 1: VM の準備

VM の準備

VM の準備は、VM の設定ファイルを確認することから始まります。

### 1. IDE の設定

Create a snapshot of database-01 before I upgrade it

### 2. namespace の確認

Which namespace is database-01 in?

### 3. namespace の確認

my-vms

### 4. VM の QEMU guest agent のインストール

## VM Snapshot - Review Configuration

```
**VM to Snapshot:**
- **Name**: `database-01`
- **Namespace**: `my-vms`
- **Status**: Running
- **Storage**: 30Gi
```







### ### Available Snapshots

| Snapshot Name                          | VM Name       | Status    | Created          | Age    | ReadyToUse |
|----------------------------------------|---------------|-----------|------------------|--------|------------|
| ----- ----- -----                      |               |           |                  |        |            |
| ----- ----- -----                      |               |           |                  |        |            |
| 'database-01-snapshot-20260714-100000' | 'database-01' | Succeeded | 2026-07-14 10:00 | 30 min | true       |
| 'database-01-weekly-backup'            | 'database-01' | Succeeded | 2026-07-07 02:00 | 7 days | true       |

**\*\*Total Snapshots\*\*:** 2

---

### ### Actions

**\*\*To restore from a snapshot:\*\***

"Restore VM database-01 from snapshot <snapshot-name>"

**\*\*To delete a snapshot:\*\***

"Delete snapshot <snapshot-name>"

**\*\*To create a new snapshot:\*\***

"Create snapshot of VM database-01"

VM namespace

Show me all snapshots in namespace my-vms

**3:**

vm-snapshot-restore

vm-

1. IDE

The database upgrade failed. Restore database-01 from snapshot database-01-snapshot-20260714-100000

2. namespace

Which namespace is database-01 in?

3. namespace

my-vms

4. 確認 VM の状態

```
## VM Must Be Stopped Before Restore

**VM**: `database-01` (namespace: `my-vms`)
**Status**: Running

**Safety Requirement**: VMs must be stopped before restore to prevent
data corruption.

**Options**:
1. "stop-and-restore" - Stop the VM first, then restore from snapshot
2. "cancel" - Cancel restore operation

How would you like to proceed?
```

5. 確認 VM の状態

stop-and-restore

6. 確認 VM の状態

```
## VM RESTORE - Data Loss Warning

THIS WILL REPLACE CURRENT VM STATE WITH SNAPSHOT DATA

### What Will Happen

**VM to Restore**: `database-01` (namespace: `my-vms`)
**Snapshot to Restore From**: `database-01-snapshot-20260714-100000`

**Current VM State** (WILL BE LOST):
- **Last Modified**: 2026-07-14 10:35:00 UTC
- **Changes Since Snapshot**: ALL changes made after
  2026-07-14 10:00:22 UTC WILL BE PERMANENTLY LOST

**Snapshot State** (WILL BE RESTORED):
- **Created**: 2026-07-14 10:00:22 UTC
- **Age**: 35 minutes

**Time Range of Data Loss**:
- ALL CHANGES in the last 35 minutes WILL BE LOST

### What Will Be Restored
- VM configuration (from snapshot time)
```

- Disk data (from snapshot time)

### What Will Be Lost

- ALL disk changes made after 2026-07-14 10:00:22 UTC
- ALL configuration changes made after 2026-07-14 10:00:22 UTC

---

CRITICAL: This restore is permanent. Current VM state cannot be recovered unless you create a snapshot now.

**\*\*To proceed with restore, type the snapshot name exactly as shown:\*\***

Type `database-01-snapshot-20260714-100000` to confirm: \_\_\_\_\_

7.

database-01-snapshot-20260714-100000

8.

## Typed Verification Passed

**\*\*Confirmation received for snapshot\*\*:** `database-01-snapshot-20260714-100000`

### Ready to Restore

**\*\*VM\*\*:** `database-01` (namespace: `my-vms`)

**\*\*From Snapshot\*\*:** `database-01-snapshot-20260714-100000`

**\*\*Impact\*\*:**

- Current VM state will be replaced with snapshot state
- All changes in the last 35 minutes will be permanently lost

---

**\*\*Proceed with VM restore? This action cannot be undone.\*\***

- Type "yes" to execute restore
- Type "cancel" to abort

Your choice: \_\_\_\_\_

9.

yes

10.

Restoring VM from snapshot...  
This may take several minutes...

## ## VM Restored Successfully

```
**VM**: 'database-01' (namespace: 'my-vms')
**Restored From**: Snapshot 'database-01-snapshot-20260714-100000'
```

### ### Restore Details

- ```
- **Snapshot Created**: 2026-07-14 10:00:22 UTC
- **Restore Completed**: 2026-07-14 10:37:15 UTC
- **VM Status**: Stopped (ready to start)
```

### ### Data Loss Confirmation

- All changes made after 2026-07-14 10:00:22 UTC have been lost

### ### Next Steps

```
**To start the restored VM:**
"Start VM database-01 in namespace my-vms"
```

00000000VM 00000000000000000000000000000000

```
oc get vm database-01 -n my-vms
```

```
NAME      AGE  STATUS  READY
database-01 45m  Stopped  False
```

IDE 開発環境 `vm-lifecycle-manager` VM 管理

```
Start VM database-01 in namespace my-vms
```

1111111111

```
virtctl start database-01 -n my-vms
```

□□□□□□□□ 4: □□□□□□□□□□□□□□

```

vm-snapshot-delete

```

## 1. IDE

```
Delete snapshot database-01-weekly-backup in namespace my-vms
```

## 2. 確認 VM のスナップショットの削除

```
## Snapshot Deletion - Review

**Snapshot to Delete**: `database-01-weekly-backup`

### Snapshot Details
- **Snapshot Name**: `database-01-weekly-backup`
- **VM**: `database-01`
- **Namespace**: `my-vms`
- **Created**: 2026-07-07 02:00:00 UTC
- **Age**: 7 days
- **Status**: Succeeded

### Impact of Deletion
- Snapshot will be permanently deleted
- This recovery point will be lost
- Cannot restore VM to this snapshot state after deletion
- Storage will be freed

### Recovery Impact

**Available snapshots for VM `database-01`:**
- `database-01-snapshot-20260714-100000` (created 2026-07-14 10:00)

---

**Type the snapshot name `database-01-weekly-backup` to confirm deletion
(cannot be undone):** _____
```

## 3. スナップショットの削除

```
database-01-weekly-backup
```

## 4. スナップショットの削除の完了

```
Deleting snapshot...
Snapshot `database-01-weekly-backup` deleted

## Snapshot Deleted Successfully

**Snapshot**: `database-01-weekly-backup`
(VM: `database-01`, namespace: `my-vms`)

### Deletion Summary
- Snapshot permanently deleted
- Storage freed
- Recovery point removed
```

This snapshot is still available for recovery.

```
oc get virtualmachinesnapshot -n my-vms
```

□□□□□ **5:** □□□□□□□□□□□□□□□□

## 1. IDE

2.

— — —

**\*\*Type the snapshot name `database-01-snapshot-20260714-100000` to confirm deletion (cannot be undone):\*\*** \_\_\_\_\_

Enter the snapshot name to confirm deletion (cannot be undone):

Enter the snapshot name to confirm deletion (cannot be undone):

Enter the snapshot name to confirm deletion (cannot be undone): VM snapshot

1. Enter the snapshot name to confirm deletion (cannot be undone)

Create a snapshot of database-01 in namespace my-vms

2. Enter the snapshot name to confirm deletion (cannot be undone)
3. Enter the snapshot name to confirm deletion (cannot be undone)
4. Enter the snapshot name to confirm deletion (cannot be undone)

Delete snapshot database-01-snapshot-20260714-100000 in namespace my-vms

5. Enter the snapshot name to confirm deletion (cannot be undone) VM snapshot

Restore database-01 from snapshot database-01-snapshot-20260714-100000

6. Enter VM name — Enter VM spec

Start VM database-01 in namespace my-vms

7. Enter VM name — Enter VM spec

List snapshots for VM database-01 in namespace my-vms

Enter the snapshot name to confirm deletion (cannot be undone)

Enter the snapshot name to confirm deletion (cannot be undone) VM snapshot

1. VM snapshot
2. Enter VM spec
3. VM snapshot
4. Enter VM spec



## Human-in-the-Loop

4

`vm-snapshot-list`:

- 

`vm-snapshot-create`:

- 
- 
- 

`vm-snapshot-restore`:

- 
- 
- 
- 

`vm-snapshot-delete`:

- 
- 
- 
- 

## VolumeSnapshotClass

VolumeSnapshotClass

```
oc get volumesnapshotclass
```

|      |       |       |  |    |
|------|-------|-------|--|----|
| QEMU | guest | agent |  | VM |
|------|-------|-------|--|----|

```
sudo dnf install qemu-guest-agent
sudo systemctl enable --now qemu-guest-agent
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
oc get vmi database-01 -n my-vms -o
jsonpath='{.status.conditions[?(@.type=="AgentConnected")].status}'
```

True

□□□ **In Progress** □□□□□□□□

10 InProgress VirtualMachineRestore

```
oc get virtualmachinerestore -n my-vms -o yaml
```

CSI

□ □ □ □ □ □ □

```
oc delete virtualmachinesnapshot --all -n my-vms
```

VM 00000000:

```
oc delete vm database-01 -n my-vms
```

namespace 命名空间:

```
oc delete namespace my-vms
```

111

- **vm-snapshot-create** 仮想マシンをバックアップする。バックアップは、指定した名前と時刻で命名される。バックアップは、指定した時刻に実行される。
- **vm-snapshot-list** 仮想マシンのバックアップの一覧を表示する。
- **vm-snapshot-restore** 仮想マシンをバックアップから復元する。復元するバックアップを選択する。復元するバックアップを選択したら、**yes/cancel** を入力して復元するかどうかを確認する。
- **vm-snapshot-delete** 仮想マシンのバックアップを削除する。削除するバックアップを選択する。削除するバックアップを選択したら、**yes/cancel** を入力して削除するかどうかを確認する。
- 仮想マシンをバックアップする前に、QEMU guest agent (Linux) または VSS (Windows) をインストールする必要がある。
- 仮想マシンをバックアップする前に、仮想マシンを停止する必要がある。 (Linux) 仮想マシン (Linux) 仮想マシン (Linux)

11

- AI 環境構築 VM 構築
- 環境構築環境構築
- AI 環境 VM 構築
- AI 環境構築環境構築環境構築
- rh-virt 環境構築
- OpenShift MCP Server
- OpenShift Virtualization 構築
- OpenShift Virtualization 構築
- KubeVirt Snapshot and Restore API

## Day-2

`rh-virt`      Day-2      6  
4  
`human-in-the-loop(human)`

□ □ □ □

- AI 애플리케이션 VM 호스팅 플랫폼 (클라우드 네이티브 MCP 애플리케이션 호스팅)
- 클라우드 네이티브 플랫폼
- AI 애플리케이션 VM 호스팅 및 AI 애플리케이션 네트워킹 플랫폼
- OpenShift Virtualization 플랫폼을 통해 2 개 Worker Node 및 OpenShift 클러스터
- KUBERNETES 플랫폼 kubeconfig 플랫폼
- VirtualMachine 플랫폼을 통해 RBAC 및

namespace

```
oc create namespace production-vms
```

VM 创建 IDE 虚拟机 `vm-create`

```
Create a Fedora VM named web-frontend in namespace production-vms
```

```
Create a Fedora VM named api-server in namespace production-vms
```

```
Create a Fedora VM named cache-node in namespace production-vms
```

VM 状態が **Stopped** の VM は 3 つあります

```
Start all three VMs in namespace production-vms
```

VM を起動するコマンド

```
oc get vm -n production-vms
```

結果:

| NAME         | AGE | STATUS  | READY |
|--------------|-----|---------|-------|
| api-server   | 3m  | Running | True  |
| cache-node   | 2m  | Running | True  |
| web-frontend | 5m  | Running | True  |

VM の管理

VM の管理には、`vm` というリソースが利用されます。このリソースは、`VM` という名前で呼ばれ、`namespace` という属性を持っています。

例えば、`1` という VM を作成するには、`vm-inventory` という VM を作成し、`vm-snapshot-create` という VM を作成します。この VM は `namespace` という属性を持っています。

VM の管理には、`vm` というリソースが利用されます。

| VM | VM (名前)              | VM (名前) |
|----|----------------------|---------|
| VM | VM (名前) VM namespace | VM (名前) |
| VM | VM (名前)              | VM (名前) |
| VM | VM (名前)              | VM (名前) |
| VM | VM (名前) VM namespace | VM (名前) |

VM の管理

**rh-virt** AGENTS.md 10

| VM                 | VM (名前)      |
|--------------------|--------------|
| "worker-02 VM" VM  | vm-inventory |
| "web-01 Fedora VM" | vm-create    |

|                                          |                      |
|------------------------------------------|----------------------|
| VM                                       | VM lifecycle manager |
| "VM api-server cache" "web-frontend"     | vm-lifecycle-manager |
| "web-server" "web-test" "golden-image" 5 | vm-clone             |
| "namespace dev" "VM old-test"            | vm-delete            |
| "api-server"                             | vm-snapshot-create   |
| "production-vms"                         | vm-snapshot-list     |
| ""                                       | vm-snapshot-delete   |
| "pre-upgrade" "api-server"               | vm-snapshot-restore  |
| "worker-02" "VM" "VM"                    | vm-rebalance         |

list show inventory vm-inventory  
migrate move rebalance drain vm-rebalance MCP  
MCP

vm-inventory 2 vm-snapshot-create 3  
vm-rebalance

1:

Worker Node (6)  
VM

1: vm-inventory

VM

1. IDE

List all VMs in namespace production-vms and show which node each VM is running on

2. vm-inventory MCP

```
## Virtual Machines in production-vms

| VM Name | Status | Node | vCPU | Memory | Age |
|-----|-----|-----|-----|-----|-----|
| web-frontend | Running | worker-01 | 1 | 4Gi | 25m |
| api-server | Running | worker-02 | 1 | 4Gi | 23m |
| cache-node | Running | worker-02 | 1 | 4Gi | 22m |

Summary:
- Total VMs: 3
- Running: 3
- Stopped: 0
```

- Nodes: worker-01 (1 VM), worker-02 (2 VMs)

namespace **worker-02** has 2 VMs (**api-server** and **cache-node**)

## 2: **vm-snapshot-create** VM

VM

### 1. IDE

Create snapshots of api-server and cache-node in namespace production-vms before maintenance

2. **vm-snapshot-create** Storage Class

## VM Snapshot - Pre-Maintenance Safety Net

### Snapshot 1 of 2: api-server

| Parameter     | Value                         |
|---------------|-------------------------------|
| VM Name       | api-server                    |
| Namespace     | production-vms                |
| Snapshot Name | api-server-pre-maintenance    |
| Storage Class | lvms-vg1 (CSI snapshots: yes) |
| VM Status     | Running (online snapshot)     |

Confirm: yes/no

### 3.

## Snapshots Created

api-server-pre-maintenance - Ready (32s)

cache-node-pre-maintenance - Ready (28s)

Both VMs have pre-maintenance snapshots.

If maintenance causes issues, restore with:

"Restore api-server from snapshot api-server-pre-maintenance"

oc get virtualmachinesnapshot -n production-vms

| NAME                       | SOURCEKIND     | SOURCENAME | PHASE     | READYTOUSE | AGE |  |
|----------------------------|----------------|------------|-----------|------------|-----|--|
| api-server-pre-maintenance | VirtualMachine | api-server | Succeeded | true       | 1m  |  |
| cache-node-pre-maintenance | VirtualMachine | cache-node | Succeeded | true       | 1m  |  |

3: **vm-rebalance** VM

**worker-02** VM

1. IDE

Migrate all VMs off worker-02 in namespace production-vms

2. **vm-rebalance** **worker-02** 2 VM

```
## VM Migration Plan - Node Evacuation

**Source Node**: worker-02
**Target**: Available nodes (worker-01, worker-03)

| VM      | Current Node | Target Node | Migration Type |
|-----|-----|-----|-----|
| api-server | worker-02 | worker-01 | Live |
| cache-node | worker-02 | worker-03 | Live |

**Migration Type**: Live (zero downtime)
**Impact**: VMs remain running during migration. Brief network blip possible.

Proceed with migration? (yes/no)
```

3.

```
## Migration Complete

api-server - Migrated to worker-01 (live, 45s)
cache-node - Migrated to worker-03 (live, 38s)

worker-02 now has 0 VMs. Safe for maintenance.
```



**vm-rebalance** ( ) VM  
 ( )  
 ( )

#### □□□□ 4: vm-inventory □□□□□□□□

XXXXXXXXXXXX VM XXXXXXXXXXXXXXXXXXXX

## 1. IDE

```
Verify no VMs are running on worker-02
```

## 2. 查看 vm-inventory 信息

```
## Virtual Machines in production-vms
```

| VM Name      | Status  | Node      | vCPU | Memory | Age |
|--------------|---------|-----------|------|--------|-----|
| web-frontend | Running | worker-01 | 1    | 4Gi    | 35m |
| api-server   | Running | worker-01 | 1    | 4Gi    | 33m |
| cache-node   | Running | worker-03 | 1    | 4Gi    | 32m |

```
Summary:
- worker-01: 2 VMs
- worker-02: 0 VMs (safe for maintenance)
- worker-03: 1 VM
```

All VMs evacuated from worker-02.

**□□□□ 5: □□□□□□□□□□□□**

VM **worker-02** cordon drain  
OpenShift

oc adm cordon worker-02

```
oc adm drain worker-02 --ignore-daemonsets --delete-emptydir-data
```

[illegible]

```
# 0: 0000000000000000
ssh core@worker-02 'sudo systemctl reboot'
```

uncordon

oc adm uncordon worker-02

Ready



```
oc get node worker-02
```

```
NAME      STATUS  ROLES  AGE  VERSION
worker-02 Ready   worker  30d  v1.28.6+openshift.1
```

## Step 6: vm-rebalance VM

Rebalance VMs in namespace production-vms across all nodes

### 1. IDE

Rebalance VMs in namespace production-vms across all nodes

### 2. vm-rebalance

```
## VM Rebalance Plan
```

```
**Current Distribution**:
```

```
worker-01: 2 VMs (web-frontend, api-server)
```

```
worker-02: 0 VMs
```

```
worker-03: 1 VM (cache-node)
```

```
**Proposed Distribution** (balanced):
```

```
worker-01: 1 VM (web-frontend)
```

```
worker-02: 1 VM (api-server)
```

```
worker-03: 1 VM (cache-node)
```

| VM         | Current Node | Target Node | Migration Type |
|------------|--------------|-------------|----------------|
| api-server | worker-01    | worker-02   | Live           |

```
**Migrations Required**:
```

```
**Impact**:
```

```
Proceed with rebalancing? (yes/no)
```

### 3. api-server

```
## Rebalance Complete
```

```
api-server - Migrated to worker-02 (live, 42s)
```

```
**Final Distribution**:
```

```
worker-01: 1 VM (web-frontend)
```

```
worker-02: 1 VM (api-server)
```

worker-03: 1 VM (cache-node)

Cluster is balanced. All VMs running.

VM namespace1 6

## 2:

2 4 VM

1: **vm-inventory** VM

1. IDE

List VMs in namespace production-vms

2. **vm-inventory**

## Virtual Machines in production-vms

| VM Name      | Status  | Node      | vCPU | Memory | Age |
|--------------|---------|-----------|------|--------|-----|
| web-frontend | Running | worker-01 | 1    | 4Gi    | 1h  |
| api-server   | Running | worker-02 | 1    | 4Gi    | 1h  |
| cache-node   | Running | worker-03 | 1    | 4Gi    | 1h  |

Summary: 3 VMs, all Running

**web-frontend**

2: **vm-clone**

1. IDE

Create 5 clones of web-frontend named loadtest-01 through loadtest-05 in namespace production-vms

2. **vm-clone** VM 5 storage strategy

## Batch Cloning: 5 VMs from `web-frontend`

**\*\*Source\*\***: `web-frontend` (production-vms) - u1.medium, 1 vCPU, 4Gi, 30Gi  
**\*\*Targets\*\***: loadtest-01, loadtest-02, loadtest-03, loadtest-04, loadtest-05

**\*\*Total Resource Impact\*\*:** 5 vCPU, 20Gi RAM, 150Gi disk

**\*\*Select storage strategy (applied to all clones, 1-4):\*\***

1. Clone Storage - Full copy (~5-10 min each, 150Gi total)
2. Reference Existing - Shared disk (dangerous)
3. New Empty Storage - Fresh disks
4. Cancel

### 3. 步骤 1 克隆虚拟机并克隆磁盘

**## Batch Cloning Complete**

**\*\*Results\*\*:**

loadtest-01 (production-vms) - Cloned in 6m12s  
loadtest-02 (production-vms) - Cloned in 7m05s  
loadtest-03 (production-vms) - Cloned in 5m48s  
loadtest-04 (production-vms) - Cloned in 6m33s  
loadtest-05 (production-vms) - Cloned in 5m55s

**\*\*Total Storage\*\*:** 150Gi cloned

**\*\*Status\*\*:** All 5 clones created (Stopped)

**\*\*Next\*\*:** Start all with "Start VMs loadtest-01 through loadtest-05"

### 步骤 3: **vm-lifecycle-manager** 克隆虚拟机并克隆磁盘

#### 1. IDE 克隆虚拟机并克隆磁盘

Start all loadtest VMs in namespace production-vms

#### 2. 使用 **vm-lifecycle-manager** 克隆虚拟机并克隆磁盘

**## VM Lifecycle Operation - Batch Start**

Starting 5 VMs in namespace production-vms:

| VM Name     | Action | Impact             |
|-------------|--------|--------------------|
| loadtest-01 | start  | consumes resources |
| loadtest-02 | start  | consumes resources |
| loadtest-03 | start  | consumes resources |
| loadtest-04 | start  | consumes resources |
| loadtest-05 | start  | consumes resources |

**\*\*Total Impact\*\*:** 5 vCPU, 20Gi RAM

Confirm starting all 5 VMs? (yes/no)

3. 確認 VM が Running 状態にある:

```
## Batch Start Complete

loadtest-01 - Running (RunStrategy: Always)
loadtest-02 - Running (RunStrategy: Always)
loadtest-03 - Running (RunStrategy: Always)
loadtest-04 - Running (RunStrategy: Always)
loadtest-05 - Running (RunStrategy: Always)

All 5 VMs started successfully.
```

ステップ 4: **vm-inventory** を実行する

1. IDE を開く:

```
Show all loadtest VMs with their IPs
```

2. **vm-inventory** を実行して **VirtualMachineInstance** の IP を取得する:

```
## Virtual Machines - Load Test Fleet

| VM Name | Status | Node | IP | vCPU | Memory |
|-----|-----|-----|-----|-----|-----|
| loadtest-01 | Running | worker-01 | 10.128.0.110 | 1 | 4Gi |
| loadtest-02 | Running | worker-02 | 10.128.1.85 | 1 | 4Gi |
| loadtest-03 | Running | worker-03 | 10.128.2.42 | 1 | 4Gi |
| loadtest-04 | Running | worker-01 | 10.128.0.111 | 1 | 4Gi |
| loadtest-05 | Running | worker-02 | 10.128.1.86 | 1 | 4Gi |

Summary: 5 VMs, all Running, distributed across 3 nodes
```

ステップ 1: **vm-inventory** を実行して VM の IP を取得する

ステップ 2: **vm-lifecycle-manager** を実行する

ステップ 3: **vm-rebalance** を実行して VM を再平衡する

- **VM** を **vm-inventory** を実行して VM の IP を取得する
- **Namespace** **production-vms** を実行して worker-02 の VM を再平衡する
- **vm-rebalance** を実行して VM を再平衡する
- **vm-snapshot-create** を実行して api-server-pre-maintenance

restore api-server from its pre-maintenance snapshot

- `vm-rebalance` try migrating it to worker-03 instead VM

VM namespace

- 
- (VM)
- 
- 

- VM 1 (VM)
- 
- namespace VM
- namespace VM

Day-2

|    | <code>vm-inventory</code> → <code>vm-snapshot-create</code> → <code>vm-rebalance</code> (manual) → <code>vm-inventory</code> → maintenance → <code>vm-rebalance</code> (automatic) |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <code>vm-snapshot-create</code> → upgrade → <code>vm-snapshot-restore</code> (if needed)                                                                                           |
|    | <code>vm-inventory</code> → <code>vm-clone</code> (batch) → <code>vm-lifecycle-manager</code> → <code>vm-inventory</code>                                                          |
|    | <code>vm-snapshot-list</code> → <code>vm-snapshot-delete</code> (old snapshots)                                                                                                    |
| VM | <code>vm-clone</code> (cross-namespace) → <code>vm-lifecycle-manager</code> → <code>vm-inventory</code>                                                                            |

```
oc delete virtualmachinesnapshot api-server-pre-maintenance cache-node-pre-maintenance -n production-vms
```

```
oc delete vm loadtest-01 loadtest-02 loadtest-03 loadtest-04 loadtest-05 -n production-vms
```

```
oc delete vm web-frontend api-server cache-node -n production-vms
```

namespace `production-vms`:

```
oc delete namespace production-vms
```

OpenShift Container Platform IDE `Delete VM loadtest-01 in namespace production-vms` `vm-delete`

□□□

- `Day-2` `inventory` `snapshot` `migrate` `verify` `maintenance` `rebalance` `6`
- `AGENTS.md` `inventory` `batch clone` `batch start` `verify` `4`
- `VM` `namespace`
- `rh-virt`

□□

- [AI](#) `VM`
- `rh-virt`
- [AI](#) `VM`
- [AI](#) `rh-virt`
- `rh-virt`
- `OpenShift MCP Server`
- `OpenShift Virtualization`
- `OpenShift`

**VM** `vm-rebalance`

`vm-rebalance` `OpenShift` `OpenShift`

(AI (KVM) VM  
VM  
VM

VM

- AI VM MCP
- OpenShift Virtualization OpenShift
- KUBECONFIG kubeconfig
- Worker Node 2
- VM (VM)
- (KVM) Worker Node HyperConverged CR
- ReadWriteMany (RWX) StorageClass (Ceph RBD, CephFS, ReadWriteOnce (RWO))
- Worker Node

namespace

```
oc create namespace my-vms
```

VM vm-create

```
Create a Fedora VM named web-01 in namespace my-vms
```

```
Create a Fedora VM named web-02 in namespace my-vms
```

```
Create a Fedora VM named db-01 in namespace my-vms
```

VM

```
oc get vm -n my-vms
```

:

| NAME   | AGE | STATUS  | READY |
|--------|-----|---------|-------|
| db-01  | 3m  | Running | True  |
| web-01 | 5m  | Running | True  |
| web-02 | 4m  | Running | True  |







### ## VM Migration Plan

| Parameter     | Value                                    |
|---------------|------------------------------------------|
| VM Name       | `web-01`                                 |
| Namespace     | `my-vms`                                 |
| Source Node   | `worker-01`                              |
| Target Node   | `worker-02`                              |
| Migration     | Live (zero downtime)                     |
| Storage       | web-01-rootdisk (30Gi, RWX) - compatible |
| Est. Duration | ~1-2 minutes                             |

**\*\*Impact\*\*:** No downtime. Active connections preserved.

Proceed with migration? (yes/no)

3.

yes

4. `VirtualMachineInstanceMigration`

### ## VM Migration Progress

Scheduling... Target pod created on worker-02  
Pre-copy... Memory transfer in progress (2.1 GiB)  
Iterative... Round 1: 45 MiB dirty pages  
Iterative... Round 2: 8 MiB dirty pages  
Switchover... VM paused on source, final pages transferred  
Running... VM resumed on worker-02  
Cleanup... Source pod terminated on worker-01

### ## VM Migration Successful

**\*\*VM\*\*:** `web-01` | **\*\*From\*\*:** worker-01 | **\*\*To\*\*:** worker-02

**\*\*Duration\*\*:** 1m 23s | **\*\*Type\*\*:** Live (zero downtime)

**\*\*Next\*\*:** "List VMs in namespace my-vms" or "Show status of VM web-01"

VM

```
oc get vmi web-01 -n my-vms -o jsonpath='{.status.nodeName}'
```

worker-02

## 2: ReadWriteOnce (RWO)

ReadWriteOnce VM

### 1. IDE

Move VM db-01 to worker-01 in namespace my-vms

### 2. VM PVC RWO

```
## VM Migration Plan

| Parameter | Value |
|-----|-----|
| VM Name   | `db-01` |
| Namespace | `my-vms` |
| Source Node | `worker-02` |
| Target Node | `worker-01` |
| Migration | Cold (brief downtime required) |
| Storage   | db-01-rootdisk (30Gi, RWO) - live incompatible |
| Est. Duration | ~3-5 minutes |

WARNING: Cold migration requires stopping the VM. The VM will be
unavailable for approximately 3-5 minutes. Active network connections
will be dropped.

**Cold migration steps**:
1. Stop VM on worker-02
2. Reschedule VM to worker-01
3. Start VM on worker-01

Proceed with cold migration? (yes/no)
```

### 3.

yes

### 4.

```
## Cold Migration Progress

Step 1/3: Stopping VM db-01 on worker-02... Stopped
Step 2/3: Rescheduling to worker-01... Scheduled
Step 3/3: Starting VM db-01 on worker-01... Running

## VM Migration Successful
```

```
**VM**: 'db-01' | **From**: worker-02 | **To**: worker-01
**Duration**: 3m 47s | **Type**: Cold (VM was stopped during migration)
**Downtime**: ~3m 47s

**Next**: Verify application connectivity. Clients may need to reconnect.
```

### Step 3: Drain worker-01

Drain worker-01 for maintenance in namespace my-vms

#### 1. IDE Drain Plan

```
Drain all VMs from worker-01 for maintenance in namespace my-vms
```

#### 2. Node Drain Plan: worker-01

VMs on worker-01: 2

| VM     | Storage Access | Migration Type | Target Node | Est. Duration |
|--------|----------------|----------------|-------------|---------------|
| web-01 | RWX            | Live           | worker-02   | ~1-2 min      |
| web-02 | RWX            | Live           | worker-02   | ~1-2 min      |

\*\*Target node capacity check\*\*:

- worker-02: 4/8 vCPU used, 12Gi/32Gi memory used - sufficient for 2 additional VMs

\*\*Total estimated time\*\*: ~3-5 minutes

\*\*Impact\*\*: Zero downtime for all VMs (all eligible for live migration)

Approve drain plan? (yes/no)

#### 3. Approve

yes

#### 4. Node Drain Progress

## Node Drain Progress

[1/2] Migrating web-01 (live)... Completed in 1m 15s

[2/2] Migrating web-02 (live)... Completed in 1m 32s

## Node Drain Complete

```
**Node**: worker-01 | **VMs migrated**: 2/2  
**Total duration**: 2m 47s | **Downtime**: None
```

worker-01 is now free of VMs and ready for maintenance.

**\*\*Next\*\*:** Cordon the node with "oc adm cordon worker-01" before starting maintenance to prevent new VMs from scheduling.

~~~~~ cordon ~~~~~

```
oc adm cordon worker-01
```

~~~~~ uncordon ~~~~~

```
oc adm uncordon worker-01
```

## ~~~~~ 4: ~~~~~

~~~~~ VM ~~~~~

### 1. IDE ~~~~~

```
Rebalance VMs across nodes in namespace my-vm
```

### 2. ~~~~~ VM ~~~~~

#### ## Cluster Analysis

| Node      | vCPU Used | vCPU Total | Mem Used | Mem Total | VMs | Load |
|-----------|-----------|------------|----------|-----------|-----|------|
| worker-01 | 1/8       | 12.5%      | 4Gi      | 32Gi      | 1   | Low  |
| worker-02 | 3/8       | 37.5%      | 12Gi     | 32Gi      | 3   | High |

**\*\*Imbalance detected\*\*:** worker-02 hosts 75% of VMs with 3x the resource utilization of worker-01.

#### ## Proposed Migration Plan

| VM     | From      | To        | Type | Reason                         |
|--------|-----------|-----------|------|--------------------------------|
| web-02 | worker-02 | worker-01 | Live | Balance CPU/memory utilization |

**\*\*After rebalancing\*\*:**

- worker-01: 2 VMs, ~25% CPU, ~25% memory
- worker-02: 2 VMs, ~25% CPU, ~25% memory

**\*\*Impact\*\*:** Zero downtime (all migrations are live)

Approve rebalancing plan? (yes/no)

3.

yes

4.

## Rebalancing Progress

[1/1] Migrating web-02 from worker-02 to worker-01 (live)... Completed in 1m 18s

## Rebalancing Complete

| Node      | Before           | After          |
|-----------|------------------|----------------|
| worker-01 | 1 VM, 12.5% CPU  | 2 VMs, 25% CPU |
| worker-02 | 3 VMs, 37.5% CPU | 2 VMs, 25% CPU |

**\*\*VMs migrated\*\*:** 1 | **\*\*Downtime\*\*:** None

Cluster load is now balanced across worker nodes.

```
oc get vmi -n my-vms -o custom-  
columns="NAME:.metadata.name,NODE:.status.nodeName,PHASE:.status.phase"
```

| NAME   | NODE      | PHASE   |
|--------|-----------|---------|
| db-01  | worker-01 | Running |
| web-01 | worker-02 | Running |
| web-02 | worker-01 | Running |

**vm-rebalance**  **VM**

**PVC**

| Access Mode         | Access Modes                | Access Modes  |
|---------------------|-----------------------------|---------------|
| ReadWriteMany (RWX) | ReadWriteMany               | ReadWriteMany |
| ReadWriteOnce (RWO) | ReadWriteOnce               | ReadWriteOnce |
| ReadOnlyMany (ROX)  | ReadOnlyMany (ReadOnlyMany) | ReadOnlyMany  |

VM or PVC Access Modes:

```
oc get pvc -n my-vms -o custom-columns="NAME:.metadata.name,ACCESS:.status.accessModes[0],CAPACITY:.status.capacity.storage"
```

| NAME            | ACCESS        | CAPACITY |
|-----------------|---------------|----------|
| db-01-rootdisk  | ReadWriteOnce | 30Gi     |
| web-01-rootdisk | ReadWriteMany | 30Gi     |
| web-02-rootdisk | ReadWriteMany | 30Gi     |

StorageClass

StorageClass Access Modes:

```
oc get storageclass -o custom-columns="NAME:.metadata.name,PROVISIONER:.provisioner,BINDING:.volumeBindingMode"
```

StorageClass Access Modes: CSI Access Modes: StorageClass (RWX Access Ceph RBD or CephFS or) StorageClass (local-volume Access LVM Access)

StorageClass Access Modes

vm-rebalance Access Modes

StorageClass Access Modes

StorageClass Access Modes cordon Access Modes

```
oc get nodes -o custom-columns="NAME:.metadata.name,STATUS:.status.conditions[-1].type,SCHEDULABLE:.spec.unschedulable"
```

| NAME      | STATUS | SCHEDULABLE |
|-----------|--------|-------------|
| worker-01 | Ready  | <none>      |

```
worker-02  Ready  <none>
```

SCHEDULABLE: true (unschedulable ☐) ☐ cordon ☐

**taint** ☐☐☐

VM Pod (tolerate) Taint Taint

```
oc get nodes -o custom-columns="NAME:.metadata.name,TAINTS:.spec.taints[*].key"
```

□ □ □ □ □ □

1. VM の CPU 使用率 (requests) (秒単位)

```
oc describe node worker-01 | grep -A 5 "Allocated resources"
```

[illegible]

## Human-in-the-Loop

vm-rebalance VM

□ □ □ □ :

- [illegible]

□ □ □ □ □ □ □ :

- [illegible]

□□□□□□□□:

- [illegible]

XXXXXXXXXXXXXXXXXXXXXXXX "no" XXX "cancel" XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX VM XXXXXXXXXX



## VM を削除

VM を削除するコマンド:

```
oc delete vm web-01 web-02 db-01 -n my-vms
```

IDE で `"Delete VM web-01 in namespace my-vms"` を検索すると `vm-delete` のコマンドが見つかります。

ノードを `cordoned` から `uncordoned` に戻す:

```
oc adm uncordon worker-01
```

namespace を削除:

```
oc delete namespace my-vms
```

## まとめ

- `vm-rebalance` コマンド (AI 対応) は VM を自動的に再配置します。
- `ReadWriteMany (RWX)` をサポートする VM を作成すると、他の VM と共有してアクセスできます。
- `ReadWriteOnce (RWO)` をサポートする VM を作成すると、他の VM と共有してアクセスできません。
- VM を削除すると、その VM が占有していたリソースは解放されます。
- CPU 使用率が一定以上になると、VM が自動的に再起動されます。
- 特定の VM に `taint` (汚染) を付与すると、他の VM はそのノードにスケジューリングされません。
- VM の状態を監視するためのダッシュボードがあります。

## 参考

- [AI 対応 VM の活用](#)
- [VM の作成と削除](#)
- [AI 対応 VM の活用](#)
- [AI 対応 VM の活用](#)
- [rh-virt のインストール](#)
- [OpenShift MCP Server](#)
- [OpenShift Virtualization のインストール](#)
- [OpenShift Live Migration のインストール](#)
- [KubeVirt Live Migration のインストール](#)

# Glossary

## OpenShift Virtualization

### CNV

Container Native Virtualization (CNV) Red Hat OpenShift Virtualization

### NAD

NetworkAttachmentDefinition (NAD) Pod VM

### NNCP

NodeNetworkConfigurationPolicy (NNCP) NMState (CR)

### UDN

User Defined Network (UDN) 2 3

### VLAN

Virtual LAN (VLAN) VLAN

### OVS

Open vSwitch (OVS)

### IPAM

IP Address Management (IPAM) IP

### PVC

PersistentVolumeClaim (PVC)

### DataVolume

VM CDI (CR)

### VMI

VirtualMachineInstance (VMI) VM

### SCC

SecurityContextConstraint (SCC) OpenShift Pod

### RBAC

Role-Based Access Control (RBAC) Kubernetes

### Localnet

### cloud-init

VM

## Virtctl

VM [管理](#) CLI [管理](#)

[管理](#)

- [管理](#) - OpenShift Virtualization [管理](#)
- [管理](#) - cookbook [管理](#)